

习题解答
《计算理论》

Jim Hefferon

<https://hefferon.net/computation>



关于习题解答

为教材习题提供答案时，需要作出许多选择：只答一部分还是全部？是否只向教师提供完整答案？给出完整解答还是概要？若只给概要，证明题又该如何处理？

我发现，写书时完成全部解答会让题目本身变得更好。解题的过程会检验本节是否包含所需的全部预备知识，也会检验题意是否清楚。我常常因此返回去修改题目或正文。此外，我也享受这个过程；既然如此，至少有人从中得到了乐趣。所以，我写出了这些答案。

不过，是否公开答案仍值得考虑。习题本来是要让学生亲手完成的；公开答案可能使一些学生只读题目和答案，便误以为自己已经理解。另一方面，如果只给不完整的解答、提示或部分答案，那些真诚尝试、偶尔受阻的读者又可能无从知道自己遗漏了什么。

有人建议只公开部分习题答案，把其余答案留给教师。这既会增加我为免费材料处理邮件的负担，也不切实际，因为我无法核实来信者的身份。更重要的是，这种建议假定读者都身处高校；事实上，许多读者在这一体系之外，只是希望学习。以我对互联网的经验，正如传统出版的微积分教材一样，任何想在网上寻找完整答案的学生都能在几分钟内找到。因此，以为其他教材不会让学生接触到全部习题答案，在我看来是不现实的。

总之，这里提供所有习题的完整答案，包括证明。

说明：本 PDF 与 2026-Sep-03 编译的教材版本相配。若将教材与本文件放在同一目录，点击教材中的习题编号会跳转到这里的答案，点击答案编号则会返回教材。

Jim Hefferon
University of Vermont

Cover: Flauros, the sixty-fourth demon and the Duke of Hell, who gives true answers of all things past, present, and future. Although, you should verify what he says for yourself. (Unless you can get him inside a triangle and then you're good. It is unclear whether that is a permanent triangle, or just one chalked out on the floor, or a triangle of roads, or rivers, or planets.) From *Dictionnaire Infernal*, Jacques Albin Simon Collin de Plancy, 1863.

Chapter I: 机械计算

I.1.11 它像程序的地方在于它是单一用途的，也就是说，就像我们可以有一个仅将输入翻倍而不做其他事情的程序一样，我们也可以有这样一个 Turing 机。它不像程序的地方在于它是硬件。

它像我们今天桌上电脑的地方在于它机械地将输入转换为输出。Turing 机不像这种电脑的地方在于它是为单一任务而特制的。

I.1.12 这个定义描述的是机器本身。关于机器动作的进一步描述，包括格局和转移的定义，紧随定义 1.4 之后。在某种程度上，这是对定义 1.4 的扩展。

换言之，正如 A Bauer 在 Stack Exchange 上对此问题的评论中所说，为什么道路不是汽车的一部分？

I.1.13 这并不是判断某个特定数学陈述是否为真的问题。显然，如果 $x = 2$ ，那么我们就可以判断 $x > 3$ 是否成立。类似地，我们可以判定 $\forall x \in \mathbb{N} [x^2 > 3]$ 的真假。而 *Entscheidungsproblem* 要求的是一个算法，它能输入任意数学陈述，并输出 ‘T’ 或 ‘F’。

I.1.14

(A) 以下是计算 2 的前驱过程中产生的纸带序列。

Step	Machine	Step	Machine	Step	Machine
0		3		6	
1		4		7	
2		5			

(B) 这是 2 与 2 求和的纸带图序列。

Step	Machine	Step	Machine	Step	Machine
0		5		10	
1		6		11	
2		7		12	
3		8			
4		9			

(c) 以下是前驱计算的格局序列。

$$\begin{aligned}
 &\langle q_0, 1, \varepsilon, 1 \rangle \vdash \langle q_0, 1, 1, \varepsilon \rangle \vdash \langle q_0, B, 11, \varepsilon \rangle \vdash \langle q_1, 1, 1, \varepsilon \rangle \vdash \langle q_1, B, 1, \varepsilon \rangle \vdash \langle q_2, 1, \varepsilon, \varepsilon \rangle \\
 &\vdash \langle q_2, B, \varepsilon, 1 \rangle \vdash \langle q_3, 1, \varepsilon, \varepsilon \rangle
 \end{aligned}$$

由于状态 q_3 没有对应的指令，因此 $\langle q_3, 1, \varepsilon, \varepsilon \rangle$ 是一个停机格局。

2 与 2 相加的计算序列稍长一些。

$$\begin{aligned} \langle q_0, 1, \varepsilon, 1B11 \rangle &\vdash \langle q_0, 1, 1, B11 \rangle \vdash \langle q_0, B, 11, 11 \rangle \vdash \langle q_1, B, 11, 11 \rangle \vdash \langle q_1, 1, 11, 11 \rangle \\ &\vdash \langle q_2, 1, 11, 11 \rangle \vdash \langle q_2, 1, 1, 111 \rangle \vdash \langle q_2, 1, \varepsilon, 1111 \rangle \vdash \langle q_2, B, \varepsilon, 11111 \rangle \\ &\vdash \langle q_3, B, \varepsilon, 11111 \rangle \vdash \langle q_3, 1, \varepsilon, 1111 \rangle \vdash \langle q_4, B, \varepsilon, 1111 \rangle \vdash \langle q_5, 1, \varepsilon, 111 \rangle \end{aligned}$$

I.1.15

- (A) 要证明对负数（或其他任何数学对象）的某些操作是可计算的，需要先为它们指定一种基于有限字母表上字符串的表示方法，然后展示如何使用 Turing 机对这些字符串进行机械计算。例如，要证明两个整数的乘法是机械的，我们可以使用字母表 $\Sigma = \{B, 0, \dots, 9, +, -\}$ 并将每个整数表示为一对 $\langle \text{sign}, \text{自然数} \rangle$ 。构造一台 Turing 机来处理四种可能的符号组合情况，虽然繁琐但完全可行。
- (B) 如果这里说的“问题”是指机器不停机，那么这并不是问题的根源。相反，这台机器只是一个简单的无限循环。
- (C) 我们可以给机器设定非顺序编号的状态。唯一的限制是我们遵循初始状态为 q_0 的约定。例如，机器 $\mathcal{P} = \{q_0 B B q_{50}, q_0 11 q_{50}, q_{50} 11 q_1, q_{50} 11 q_1\}$ 符合定义，并且在任何输入下都能一步到达状态 q_{50} 。

I.1.16 停机状态是 q_5 ，工作状态是 q_0 、 q_1 、 q_2 、 q_3 和 q_4 。

I.1.17 跟踪过程并不太有趣。

Step	Machine	Step	Machine	Step	Machine
0		4		8	
1		5		9	
2		6		10	
3		7			

I.1.18

(A) 在这台神秘机器上输入 11 的运行结果如下。

Step	Machine	Step	Machine
0		2	
1		3	

(B) 输入 1011 时，结果如下。

Step	Machine	Step	Machine
0		2	
1		3	

(C) 输入 110 时，结果如下。

Step	Machine	Step	Machine
0	1 1 0 q₀	3	1 1 0 q₀
1	1 1 0 q₁	4	1 1 0 q₄
2	1 1 0 q₀		

(D) 输入 1101 时，机器给出了如下的纸带序列。

Step	Machine	Step	Machine
0	1 1 0 1 q₀	4	1 1 0 1 q₁
1	1 1 0 1 q₁	5	1 1 0 1 q₄
2	1 1 0 1 q₀		
3	1 1 0 1 q₀		

(E) 当初始纸带为空时，运行情况如下。

Step	Machine
0	q₀
1	q₄

I.1.19 这是转移表。

Δ	B	0	1
q_0	B q_4	R q_0	R q_1
q_1	B q_4	R q_2	R q_0
q_2	B q_4	R q_0	R q_3
q_3	—	—	—

I.1.20 机器 $\mathcal{P}_{\text{blankones}} = \{q_0 \text{BB}q_2, q_0 \text{1B}q_1, q_1 \text{BR}q_0, q_1 \text{11}q_0\}$ 在包含两个 1 的纸带上的运行过程如下。

Step	Machine	Step	Machine
0	1 1 q₀	3	q₁
1	1 q₁	4	q₀
2	1 q₀	5	q₂

I.1.21

(A) 这个机器可以完成该任务；注意读写头始终不动。

$$\mathcal{P}_{\text{singlebitnot}} = \{q_0 \text{BB}q_1, q_0 \text{01}q_1, q_0 \text{10}q_1\}$$

这里展示输入 $\sigma = 0$ 时的运行情况。

Step	Machine
0	0 q_0
1	1 q_1

(B) 这个机器实现逻辑“与”。注意状态编号不是连续的；在编写机器时，留出编号间隔有时会很方便。例如，这里状态 q_{100} 被用作停机状态，因为显然不会用到一百个状态，所以这个编号是可用的。类似地， q_{10} 用于处理第一位是 0 的情况，而 q_{20} 用于处理第一位是 1 的情况。

$$\begin{aligned} \mathcal{P}_{\text{singlebitand}} = \{ & q_0 \text{BB}q_{100}, q_0 \text{0B}q_{10}, q_0 \text{1B}q_{20}, q_{10} \text{BR}q_{11}, q_{10} \text{00}q_{100}, q_{10} \text{11}q_{100}, \\ & q_{11} \text{BB}q_{100}, q_{11} \text{00}q_{100}, q_{11} \text{10}q_{100}, q_{20} \text{BR}q_{21}, q_{20} \text{00}q_{100}, q_{20} \text{10}q_{100}, \\ & q_{21} \text{BB}q_{100}, q_{21} \text{00}q_{100}, q_{21} \text{11}q_{100} \} \end{aligned}$$

这里展示机器在输入 $\sigma = 01$ 上的运行情况。

Step	Machine	Step	Machine
0	0 1 q_0	2	1 q_{11}
1	1 q_{10}	3	0 q_{100}

(C) 这是逻辑“或”机器。与上一小题类似，状态编号不连续。另请注意，这台机器基本上与“或”机器相同（原文如此，疑为“与”机器）。

$$\begin{aligned} \mathcal{P}_{\text{singlebitor}} = \{ & q_0 \text{BB}q_{100}, q_0 \text{0B}q_{10}, q_0 \text{1B}q_{20}, q_{10} \text{BR}q_{11}, q_{10} \text{00}q_{100}, q_{10} \text{11}q_{100}, \\ & q_{11} \text{BB}q_{100}, q_{11} \text{00}q_{100}, q_{11} \text{11}q_{100}, q_{20} \text{BR}q_{21}, q_{20} \text{00}q_{100}, q_{20} \text{10}q_{100}, \\ & q_{21} \text{BB}q_{100}, q_{21} \text{01}q_{100}, q_{21} \text{11}q_{100} \} \end{aligned}$$

这是它在输入 $\sigma = 01$ 上的运行情况。

Step	Machine	Step	Machine
0	0 1 q_0	2	1 q_{11}
1	1 q_{10}	3	1 q_{100}

I.1.22 该机利用 q_0 向右滑至末端，追加 01，再利用 q_2 向左滑回。

$$\mathcal{P}_{\text{append01}} = \{q_0 \text{B0}q_1, q_0 \text{0R}q_0, q_0 \text{1R}q_0, q_1 \text{B1}q_2, q_1 \text{0R}q_1, q_1 \text{1L}q_2, q_2 \text{BR}q_3, q_2 \text{0L}q_2, q_2 \text{1L}q_2\}$$

下面的例子展示了该机器在输入 $\sigma = 11$ 时的运行情况。

Step	Machine	Step	Machine	Step	Machine
0		4		8	
1		5		9	
2		6		10	
3		7			

I.1.23 这台机器可行。

$$\mathcal{P}_{\text{constantthree}} = \{q_0 \text{BB}q_2, q_0 \text{1B}q_1, q_1 \text{BR}q_0, q_1 \text{11}q_0, q_2 \text{B1}q_2, q_2 \text{1R}q_3, q_3 \text{B1}q_3, q_3 \text{1R}q_4, \\ q_4 \text{B1}q_4, q_4 \text{11}q_5, q_5 \text{BL}q_6, q_5 \text{1L}q_6, q_6 \text{BB}q_7, q_6 \text{1L}q_7\}$$

这是该机器处理输入 2 时的运行情况。

Step	Machine	Step	Machine	Step	Machine
0		5		10	
1		6		11	
2		7		12	
3		8		13	
4		9			

I.1.24 这台机器计算后继函数。

$$\mathcal{P}_{\text{successor}} = \{q_0 \text{B1}q_1, q_0 \text{1L}q_0\}$$

以下是输入为 3 时的纸带序列示例。

Step	Machine
0	
1	
2	

I.1.25

- (A) 我们从读写头位于一串 1 的开头开始。擦除这第一个 1，然后移动到序列的末尾，并越过空白符。写下两个 1，再移回初始序列的开头。重复此操作。

该机器有九条指令，字母表为 $\Sigma = \{1, B\}$ 。

$$\mathcal{P}_{\text{doubler}} = \{q_0 BBq_9, q_0 1Bq_1, q_1 BRq_2, q_1 1Rq_2, q_2 BRq_3, q_2 1Rq_2, q_3 B1q_4, q_3 1Rq_3, q_4 BBq_4, q_4 1Rq_5, q_5 B1q_6, q_5 11q_6, q_6 BLq_7, q_6 1Lq_6, q_7 BRq_8, q_7 1Lq_7, q_8 BRq_9, q_8 11q_0\}$$

下面是输入为 2 时的纸带序列。这里显示了空白符，因为可能出现两个空白符，容易造成混淆。分成了两部分只是为了允许在中间换页。

Step	Machine	Step	Machine	Step	Machine
0	1 1 q₀	5	B 1 B 1 q₄	10	B 1 B 1 1 q₇
1	B 1 q₁	6	B 1 B 1 B q₅	11	B 1 B 1 1 q₇
2	B 1 q₂	7	B 1 B 1 1 q₆	12	B 1 B 1 1 q₈
3	B 1 B q₂	8	B 1 B 1 1 q₆	13	B 1 B 1 1 q₀
4	B 1 B B q₃	9	B 1 B 1 1 q₆	14	B B B 1 1 q₁

这是第二部分。

Step	Machine	Step	Machine	Step	Machine
15	B B B 1 1 q₂	20	B B B 1 1 1 B q₅	25	B B B 1 1 1 1 q₆
16	B B B 1 1 q₃	21	B B B 1 1 1 1 q₆	26	B B B 1 1 1 1 q₇
17	B B B 1 1 q₃	22	B B B 1 1 1 1 q₆	27	B B B 1 1 1 1 q₈
18	B B B 1 1 B q₃	23	B B B 1 1 1 1 q₆	28	B B B 1 1 1 1 q₉
19	B B B 1 1 1 q₄	24	B B B 1 1 1 1 q₆		

(B) 要将一个用二进制表示的数加倍，只需在右侧追加一个 0。该机器有三条指令，且字母表为 $\Sigma = \{0, 1, B\}$ 。

$$\mathcal{P}_{\text{doublerbinary}} = \{q_0 BBq_3, q_0 00q_3, q_0 1Rq_1, q_1 B0q_2, q_1 0Rq_1, q_1 1Rq_1, q_2 BRq_3, q_2 0Lq_2, q_2 1Lq_2\}$$

对于输入 6，它产生如下纸带序列。

Step	Machine	Step	Machine	Step	Machine
0		4		8	
1		5		9	
2		6			
3		7			

I.1.26 这台 Turing 机可行。

$$\mathcal{P}_{\text{奇数}} = \{q_0 \text{BB}q_{100}, q_0 \text{1R}q_1, q_1 \text{BL}q_{100}, q_1 \text{1B}q_2, q_2 \text{BL}q_3, q_2 \text{1L}q_3, q_3 \text{BB}q_4, q_3 \text{1B}q_4, \\ q_4 \text{BR}q_5, q_4 \text{11}q_5, q_5 \text{BR}q_0, q_5 \text{11}q_0\}$$

例如，对于输入 3，其运行过程如下。

Step	Machine	Step	Machine	Step	Machine
0		3		6	
1		4		7	
2		5		8	

对于输入 4，其运行过程如下。

Step	Machine	Step	Machine	Step	Machine
0		5		10	
1		6		11	
2		7		12	
3		8		13	
4		9			

I.1.27 这台 Turing 机在向右移动时，如果遇到逗号，就将其替换为一个 1 然后向左折返收尾。具体来说，它会向左扫描到字符串的开头，擦除第一个 1，然后重新开始向右移动。

$$\mathcal{P}_{\text{addany}} = \{q_0 \text{BL}q_{10}, q_0 \text{1R}q_0, q_0 \text{,1}q_1, q_1 \text{BR}q_2, q_1 \text{1L}q_1, q_1 \text{,L}q_1, q_2 \text{BR}q_0, q_2 \text{1B}q_2, \\ q_{10} \text{BR}q_{100}, q_{10} \text{1L}q_{10}, q_{10} \text{,L}q_{10}\}$$

例如，对于输入 11,1,11，机器的运行过程如下。

Step	Machine	Step	Machine	Step	Machine
0	$\begin{array}{c} \boxed{11, 1, 11} \\ q_0 \end{array}$	6	$\begin{array}{c} \boxed{1111, 11} \\ q_1 \end{array}$	12	$\begin{array}{c} \boxed{111, 11} \\ q_0 \end{array}$
1	$\begin{array}{c} \boxed{11, 1, 11} \\ q_0 \end{array}$	7	$\begin{array}{c} \boxed{1111, 11} \\ q_2 \end{array}$	13	$\begin{array}{c} \boxed{111111} \\ q_1 \end{array}$
2	$\begin{array}{c} \boxed{11, 1, 11} \\ q_0 \end{array}$	8	$\begin{array}{c} \boxed{111, 11} \\ q_2 \end{array}$	14	$\begin{array}{c} \boxed{111111} \\ q_1 \end{array}$
3	$\begin{array}{c} \boxed{1111, 11} \\ q_1 \end{array}$	9	$\begin{array}{c} \boxed{111, 11} \\ q_0 \end{array}$	15	$\begin{array}{c} \boxed{111111} \\ q_1 \end{array}$
4	$\begin{array}{c} \boxed{1111, 11} \\ q_1 \end{array}$	10	$\begin{array}{c} \boxed{111, 11} \\ q_0 \end{array}$	16	$\begin{array}{c} \boxed{111111} \\ q_1 \end{array}$
5	$\begin{array}{c} \boxed{1111, 11} \\ q_1 \end{array}$	11	$\begin{array}{c} \boxed{111, 11} \\ q_0 \end{array}$	17	$\begin{array}{c} \boxed{111111} \\ q_1 \end{array}$

(表格被分成两组, 以便排版程序有地方插入分页符。)

Step	Machine	Step	Machine	Step	Machine
18	$\begin{array}{c} \boxed{111111} \\ q_2 \end{array}$	23	$\begin{array}{c} \boxed{111111} \\ q_0 \end{array}$	28	$\begin{array}{c} \boxed{111111} \\ q_{10} \end{array}$
19	$\begin{array}{c} \boxed{111111} \\ q_2 \end{array}$	24	$\begin{array}{c} \boxed{111111} \\ q_0 \end{array}$	29	$\begin{array}{c} \boxed{111111} \\ q_{10} \end{array}$
20	$\begin{array}{c} \boxed{111111} \\ q_0 \end{array}$	25	$\begin{array}{c} \boxed{111111} \\ q_0 \end{array}$	30	$\begin{array}{c} \boxed{111111} \\ q_{10} \end{array}$
21	$\begin{array}{c} \boxed{111111} \\ q_0 \end{array}$	26	$\begin{array}{c} \boxed{111111} \\ q_{10} \end{array}$	31	$\begin{array}{c} \boxed{111111} \\ q_{10} \end{array}$
22	$\begin{array}{c} \boxed{111111} \\ q_0 \end{array}$	27	$\begin{array}{c} \boxed{111111} \\ q_{10} \end{array}$	32	$\begin{array}{c} \boxed{111111} \\ q_{100} \end{array}$

I.1.28 不存在。给定一个格局 $\mathcal{C} = \langle q, s, \tau_L, \tau_R \rangle$, 它的一个前驱格局是 $\hat{\mathcal{C}} = \mathcal{C}$, 其中 $J = qssq$ 。

I.1.29

- (A) 机器可以翻转第一位, 然后右移并翻转第二位, 依此类推。由于我们事先知道比特的数目, 这意味着可以只设四组状态: 第一组执行翻转并右移, 第二组执行翻转并右移, 等等。(我们也可以处理任意数量的比特, 但那样会稍微困难一些。) 处理完毕后, 我们将读写头左移, 将其定位在非空白字符的最左端。

尽管题目说无需给出具体机器, 这里还是给出一台。在这台机器中, 状态 q_0 翻转第一位, 然后 q_1 右移。下一组翻转和右移的组合是 q_2 和 q_3 , 依此类推, 直到机器处理完第四位。然后通过状态 q_7 将读写头移回起始位置。

$$\begin{aligned} \mathcal{P}_{\text{bitwiseNOT}} = \{ & q_0 \text{BB}q_1, q_0 01q_1, q_0 10q_1, q_1 \text{BR}q_2, q_1 0\text{R}q_2, q_1 1\text{R}q_2, q_2 \text{BB}q_3, q_2 01q_3, q_2 10q_3, \\ & q_3 \text{BR}q_4, q_3 0\text{R}q_4, q_3 1\text{R}q_4, q_4 \text{BB}q_5, q_4 01q_5, q_4 10q_5, q_5 \text{BR}q_6, q_5 0\text{R}q_6, q_5 1\text{R}q_6, \\ & q_6 \text{BB}q_7, q_6 01q_7, q_6 10q_7, q_7 \text{BR}q_8, q_7 0\text{L}q_7, q_7 1\text{L}q_7 \} \end{aligned}$$

例如, 在输入 0110 上, 它执行如下操作。

Step	Machine	Step	Machine	Step	Machine
0	$\begin{array}{c} 0110 \\ q_0 \end{array}$	5	$\begin{array}{c} 1000 \\ q_5 \end{array}$	9	$\begin{array}{c} 1001 \\ q_7 \end{array}$
1	$\begin{array}{c} 1110 \\ q_1 \end{array}$	6	$\begin{array}{c} 1000 \\ q_6 \end{array}$	10	$\begin{array}{c} 1001 \\ q_7 \end{array}$
2	$\begin{array}{c} 1110 \\ q_2 \end{array}$	7	$\begin{array}{c} 1001 \\ q_7 \end{array}$	11	$\begin{array}{c} 1001 \\ q_7 \end{array}$
3	$\begin{array}{c} 1010 \\ q_3 \end{array}$	8	$\begin{array}{c} 1001 \\ q_7 \end{array}$	12	$\begin{array}{c} 1001 \\ q_8 \end{array}$
4	$\begin{array}{c} 1010 \\ q_4 \end{array}$				

- (B) 机器有两种情形，取决于 b_3 的值。如果 $b_3 = 0$ ，那么它右移，将下一位复制到当前位置，最后附加一个 0。类似地，如果 $b_3 = 1$ ，那么它右移复制，最后附加一个 1。与上一项类似，由于我们预先知道有四位，因此可以只设三组状态。
- (C) 一种实现方式是假设输入为两个四比特的串 $a_3a_2a_1a_0$ 和 $b_3b_2b_1b_0$ ，中间用空白符分隔。机器读取 a_3 ，根据 $a_3 = 0$ 或 $a_3 = 1$ 进入两个状态之一： r_0 或 r_1 。从状态 r_0 出发，它移过空白分隔符，然后读取 b_3 ，并将其改为 b_3 和 0 的逻辑“与”。类似地，从状态 r_1 出发，它移过空白分隔符，读取 b_3 ，并将其改为 b_3 和 1 的逻辑“与”。重复上述过程，总共进行四次迭代。

I.1.30

- (A) 按照机器的读写头初始位于第一个非空白字符（如果有的话）下方的约定， $\mathcal{P} = \{q_0\text{BB}q_1, q_011q_0\}$ 只在输入为空白时才停机。
- (B) 机器 $\mathcal{P} = \{q_0\text{BB}q_0, q_011q_1\}$ 只在输入为空白时才不停机。
- (C) 机器 $\mathcal{P} = \{q_0\text{BR}q_1, q_01Rq_1, q_1\text{BB}q_1, q_111q_2\}$ 停机当且仅当第二个字符是 1。（对第一个字符尝试同样的想法行不通，因为按照约定，第一个字符为空白仅当所有字符都是空白，因此第一个字符为空白的输入串只有一种。）

I.1.31 对于这台机器，如果第一个字符是 0，它就在该位置写下一个 1，标记该串属于此语言。否则，它就写下一个 0。然后，它擦除其余字符，并返回到第一个字符处。

$$\mathcal{P}_{\text{以零开头}} = \{q_001q_1, q_010q_1, q_0B0q_{10}, q_10Rq_2, q_11Rq_2, q_20Bq_3, q_21Bq_3, q_2\text{BB}q_4, q_3\text{BR}q_2, q_4\text{BL}q_4, q_409q_{10}, q_411q_{10}\}$$

这是该机器处理输入 01101 时的运行情况。

Step	Machine	Step	Machine	Step	Machine
0	$\begin{array}{c} 01101 \\ q_0 \end{array}$	6	$\begin{array}{c} 1BB01 \\ q_2 \end{array}$	12	$\begin{array}{c} 1BBBBB \\ q_4 \end{array}$
1	$\begin{array}{c} 11101 \\ q_1 \end{array}$	7	$\begin{array}{c} 1BBB1 \\ q_3 \end{array}$	13	$\begin{array}{c} 1BBBBB \\ q_4 \end{array}$
2	$\begin{array}{c} 11101 \\ q_2 \end{array}$	8	$\begin{array}{c} 1BBB1 \\ q_2 \end{array}$	14	$\begin{array}{c} 1BBBBB \\ q_4 \end{array}$
3	$\begin{array}{c} 1B101 \\ q_3 \end{array}$	9	$\begin{array}{c} 1BBBB \\ q_3 \end{array}$	15	$\begin{array}{c} 1BBBBB \\ q_4 \end{array}$
4	$\begin{array}{c} 1B101 \\ q_2 \end{array}$	10	$\begin{array}{c} 1BBBBB \\ q_2 \end{array}$	16	$\begin{array}{c} 1BBBBB \\ q_4 \end{array}$
5	$\begin{array}{c} 1BB01 \\ q_3 \end{array}$	11	$\begin{array}{c} 1BBBBB \\ q_4 \end{array}$	17	$\begin{array}{c} 1BBBBB \\ q_{10} \end{array}$

I.1.32 对于这台机器，如果第一个字符不是 0，它就在原位置写下一个 0，标记该串不属于此语言。如果第一个字符是 0，它就检查第二个字符是否为 1。若是，则保留该字符不动；否则就写下一个 0，标记该输入不属于此语言。然后，它擦除其余字符，并返回到标记该输入是否属于此语言的那个唯一字符处。

$$\mathcal{P}_{\text{以 } 01 \text{ 开头}} = \{q_0 0Bq_1, q_0 10q_3, q_0 B0q_{10}, q_1 BRq_2, q_2 0Rq_4, q_2 1Rq_4, q_2 B0q_{10}, q_3 0Rq_4, q_4 0Bq_5, q_4 1Bq_5, q_4 BBq_6, q_5 BRq_4, q_6 00q_{10}, q_6 11q_{10}, q_6 BLq_6\}$$

这是该机器处理输入 01101 时的运行情况。

Step	Machine	Step	Machine	Step	Machine
0	0 1 1 0 1 q ₀	6	B 1 B B 1 q ₅	11	B 1 B B B B q ₆
1	B 1 1 0 1 q ₁	7	B 1 B B 1 q ₄	12	B 1 B B B B q ₆
2	B 1 1 0 1 q ₂	8	B 1 B B B q ₅	13	B 1 B B B B q ₆
3	B 1 1 0 1 q ₄	9	B 1 B B B B q ₄	14	B 1 B B B B q ₆
4	B 1 B 0 1 q ₅	10	B 1 B B B B q ₆	15	B 1 B B B B q ₁₀
5	B 1 B 0 1 q ₄				

I.1.33 这台机器的关键在于，开头的 0 可能有零个。也就是说，机器应当接受以 1 开头的串。另一方面，如果输入串是 000，后面没有跟 1，那么它就不属于此语言。

因此，机器读入开头的 0（如果有的话）并将其擦除。如果它遇到一个 1，就将其保留，标记该输入属于此语言。然后，它擦除其余字符，并返回到标记该输入是否属于此语言的那个唯一字符处。否则，它写下一个 0，标记该输入不属于此语言。

$$\mathcal{P}_{\text{包含 } 1} = \{q_0 0Bq_1, q_0 1Rq_2, q_0 B0q_{10}, q_1 BRq_0, q_2 0Bq_3, q_2 1Bq_3, q_2 BLq_4, q_3 BRq_2, q_4 00q_{10}, q_4 11q_{10}, q_4 BLq_4\}$$

这是该机器处理输入 00110 时的运行情况。

Step	Machine	Step	Machine	Step	Machine
0	0 0 1 1 0 q ₀	5	B B 1 1 0 q ₂	10	B B 1 B B B q ₄
1	B 0 1 1 0 q ₁	6	B B 1 B 0 q ₃	11	B B 1 B B B q ₄
2	B 0 1 1 0 q ₀	7	B B 1 B 0 q ₂	12	B B 1 B B B q ₄
3	B B 1 1 0 q ₁	8	B B 1 B B q ₃	13	B B 1 B B B q ₁₀
4	B B 1 1 0 q ₀	9	B B 1 B B B q ₂		

I.1.34 这台机器的基本思想是，它以两组 1 开始。读写头先将第一组的第一个 1 擦除，然后移过去擦除第二组的最后一个 1。此时，原始两组 1 之间存在小于等于关系，当且仅当两组的剩余部分之间存在小于等于关系。反复执行上述操作，最终会以两种情况之一结束：第一种情况是第一个串为空（此时我们必须擦除第二个串可能剩余的部分，并打印一个 1）；第二种情况是第二个串为空但第一个串不为空（此时我们擦除第一个串剩余的部分）。第一种情况由以状态 q_{50} 开始的指令处理，第二种情况则由以状态 q_{75} 开始的指令处理。

$$\begin{aligned} \mathcal{P}_{\text{leq}} = \{ & q_0 \text{BR} q_{50}, q_0 \text{1B} q_1, q_1 \text{BR} q_2, q_1 \text{11} q_2, q_2 \text{BR} q_3, q_2 \text{1R} q_2, q_3 \text{BL} q_{75}, q_3 \text{1R} q_4, q_4 \text{BL} q_5, q_4 \text{1R} q_4, \\ & q_5 \text{BB} q_6, q_5 \text{1B} q_6, q_6 \text{BL} q_7, q_6 \text{11} q_7, q_7 \text{BL} q_8, q_7 \text{1L} q_7, q_8 \text{BR} q_0, q_8 \text{1L} q_8, \\ & q_{50} \text{BR} q_{51}, q_{50} \text{11} q_{51}, q_{51} \text{B1} q_{100}, q_{51} \text{1B} q_{52}, q_{52} \text{BR} q_{51}, q_{52} \text{11} q_{51}, \\ & q_{75} \text{BL} q_{76}, q_{75} \text{11} q_{76}, q_{76} \text{BL} q_{100}, q_{76} \text{1B} q_{77}, q_{77} \text{BL} q_{76}, q_{77} \text{1L} q_{76} \} \end{aligned}$$

例如，对于输入 2 和 3，它的运行过程如下。

Step	Machine	Step	Machine	Step	Machine
0		5		10	
1		6		11	
2		7		12	
3		8		13	
4		9		14	

（此纸带序列分为两部分是为了留出换行位置。）

Step	Machine	Step	Machine	Step	Machine
15		21		27	
16		22		28	
17		23		29	
18		24		30	
19		25		31	
20		26			

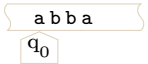
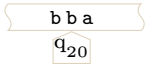
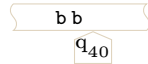
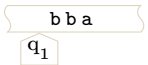
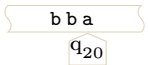
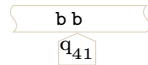
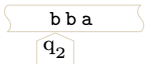
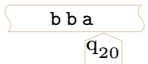
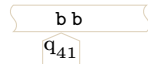
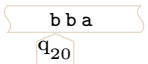
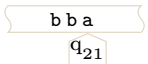
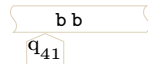
I.1.35 策略是：擦除第一个字符，滑到末尾，然后擦除最后一个字符（如果它与第一个字符匹配），接着重复这一过程。当字符串为空时，机器在纸带上写下一个 1，表示成功。

下面这台机器通过转移到状态 q_{20} 来记住第一个字符是 a。它通过转移到状态 q_{30} 来记住第一个字符是 b。从状态 q_{40} 开始，它移回剩余输入串的头。状态 q_{60} 用于处理机器检测到输入不是回文

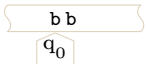
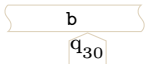
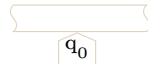
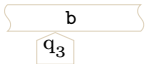
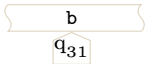
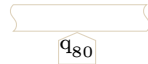
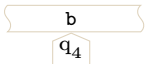
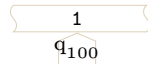
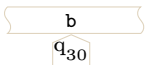
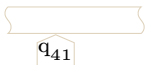
的情况，而状态 q_{80} 用于处理机器发现输入是回文的情况。最后，状态 $q_1, \dots, q_4$ 处理输入串（初始串，或经过若干次首尾修剪后初始串的剩余部分）只有一个字符的情况。

$$\begin{aligned} \mathcal{P}_{\text{pal}} = \{ & q_0 \text{BB}q_{80}, q_0 \text{aB}q_1, q_0 \text{bB}q_3, q_1 \text{BR}q_2, q_1 \text{aR}q_2, q_1 \text{bR}q_2, q_2 \text{BL}q_{80}, q_2 \text{aa}q_{20}, q_2 \text{bb}q_{20}, \\ & q_3 \text{BR}q_4, q_3 \text{aR}q_4, q_3 \text{bR}q_4, q_4 \text{BL}q_{80}, q_4 \text{aa}q_{30}, q_4 \text{bb}q_{30}, \\ & q_{20} \text{BL}q_{21}, q_{20} \text{aR}q_{20}, q_{20} \text{bR}q_{20}, q_{21} \text{BB}q_{60}, q_{21} \text{aB}q_{40}, q_{21} \text{bb}q_{60}, \\ & q_{30} \text{BL}q_{31}, q_{30} \text{aR}q_{30}, q_{30} \text{bR}q_{30}, q_{31} \text{BB}q_{60}, q_{31} \text{aa}q_{60}, q_{31} \text{bB}q_{40}, \\ & q_{40} \text{BL}q_{41}, q_{40} \text{aL}q_{41}, q_{40} \text{bL}q_{41}, q_{41} \text{BR}q_0, q_{41} \text{aL}q_{41}, q_{41} \text{bL}q_{41}, \\ & q_{60} \text{BB}q_{100}, q_{60} \text{aB}q_{61}, q_{60} \text{bB}q_{61}, q_{61} \text{BL}q_{60}, q_{61} \text{aa}q_{60}, q_{61} \text{bb}q_{60}, \\ & q_{80} \text{B1}q_{100}, q_{80} \text{a1}q_{100}, q_{80} \text{b1}q_{100} \} \end{aligned}$$

例如，这是机器在 **abba** 上的动作。

Step	Machine	Step	Machine	Step	Machine
0		4		8	
1		5		9	
2		6		10	
3		7		11	

(纸带序列分两部分显示，以便留出换页空间。)

Step	Machine	Step	Machine	Step	Machine
12		16		20	
13		17		21	
14		18		22	
15		19			

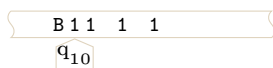
第二个例子是机器在 **aba** 上的动作。

Step	Machine	Step	Machine	Step	Machine
0		5		10	
1		6		11	
2		7		12	
3		8		13	
4		9		14	

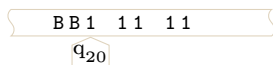
I.1.36 策略是先对起始的 1 方块制作两个副本，然后将读写头移到第一个副本的起始位置。这是一个初始格局的例子。



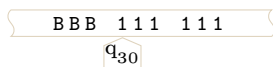
机器剥离最前面的 1，向右移动，放入一个 1 作为第一个复制块的起始，然后再放入一个 1 作为第二个复制块的起始。完成后，读写头移回起始位置。



接下来情况变得复杂。机器剥离最前面的 1，然后向右移动，在第一个复制块的末尾放入一个 1。但它放置这个 1 的位置正是分隔两个复制块的空白符。所以它必须接着向右移动，在移至第二个复制块末尾之前插入一个空白符。在第二个复制块末尾，它放入两个 1，第一个用于标记被剥离的最前面的 1，第二个用于标记插入的空白符。完成后，读写头移回起始位置。



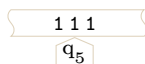
机器迭代上述步骤，直到它发现最前面的 1 序列现已变为空白。最后，它将读写头移到第一个复制块的起始位置。



I.1.37 设机器在两个不同的步骤具有相同的格局， $\mathcal{C}(t) = \mathcal{C}(\hat{t})$ 其中 $t < \hat{t}$ 。根据确定性， $\mathcal{C}(t+1) = \mathcal{C}(\hat{t}+1)$ 。类似地，对于 $i = 2, \dots$ 有 $\mathcal{C}(t+i) = \mathcal{C}(\hat{t}+i)$ 。这意味着机器每 $\hat{t}-t$ 步循环一次，因为当 $i = \hat{t}-t$ 时给出 $\mathcal{C}(t+(\hat{t}-t)) = \mathcal{C}(\hat{t}+(\hat{t}-t))$ 但 $\mathcal{C}(t+(\hat{t}-t)) = \mathcal{C}(\hat{t})$ 。

这个充分条件不是必要的，因为我们可以构造一个机器，它从不重复自己的格局，但也永不停机。一种方法是构造一个机器，它在纸带上写一个 1，然后向右移动，接着重复此操作。

I.1.38 考虑这个格局，机器处于状态 q_5 。



它可能是从这个格局和指令到达该格局的。



也可能是从这个



或者，以此类推，是这个。



I.2.2 Church 论题是一个断言，即“机械可计算的”（由离散和确定性机器计算）等价于“Turing 机可计算的”。它不是定理，因为我们无法从传统的数学公理中推导出它。

从某些方面看，它更像一个定义而非定理。但它不像通常的定义那样——在通常的定义中，我们为了便利而取一个词，以免反复说同一个短语，就像我们把“单身汉”定义为“未婚男子”或把“线性相关”定义为“ $x_0 = a_1x_1 + \dots + a_nx_n$ ”。相反，它更像微积分中对导数的定义，因为它表达了对一个基本概念的引入。

I.2.3

- (A) 和程序一样，Turing 机可以实现一个算法。但一个算法可以由不止一台 Turing 机实现。例如，我们可以用两台不同的 Turing 机对五个数字的列表进行希尔排序，哪怕只是通过重命名某些状态。用 M Davis (Davis 2016) 的话说，“[Church 论题] 没有说算法‘是什么’。它是关于算法‘能做什么和不能做什么’。”注：并没有一个被广泛接受的算法定义，这有点奇怪，因为算法是计算机科学研究的一个核心课题。
- (B) Turing 机是一种专用设备。例如，我们可以编写一台 Turing 机来将它的两个输入相乘。但今天我们所说的“计算机”指的是通用（即可编程）的设备。注：我们以后会看到存在通用的 Turing 机，所以这个区别是模糊的，更多是内涵上的而非精确定义上的。

I.2.4 原则上，我们口袋里的现代计算机能完成的任何事情，都可以用老式风格的机械装置来完成。更多细节，请参阅补充 B；那里介绍的底层构造细节完全可以用齿轮和杠杆来实现。因此，“机械可计算的”这个说法并非关于具体的实现方式，它涵盖的是任何离散、确定且物理上可实现的计算方式。

I.2.5 根据定义，纸带并非无限的，它是无界的。

可以打个比方：考虑字典里对“书”的定义。在那个定义中，并没有对页数设置上限。但没有哪一本实际装订成册的书有无限多页。同样地，任何停机的 Turing 机计算也不会使用无限长的纸带。

诚然，你可以编写一个不停机、只是一直不断使用更多纸带的 Turing 机，但在计算的任何步骤中，所用的纸带量都不是无限的。它永远不会耗尽——它是无界的——但它从来不是无限的。

即便我们确实将纸带理想化为无限，模型的作用本就是忽略其所模拟情境的某些方面。参见 (Wikipedia contributors 2016)。(另外，宇宙是否是有限的——无论是目前有限，还是在无限延展的未来中也始终有限——并不清楚。无论如何，这不是一个数学问题。)

I.2.6 Church 论题是第三项。

第一项是错误的，因为 Church 论题只针对离散且确定的机制。第二项是错误的，因为 Church 论题并不涉及人类计算员的能力或限制。有些人认为人类能做的比任何机械装置能做的更多，也有人推测可能存在原则上能比人类做得更多的机器，但 Church 论题所谈论的是任何离散且确定的机制所能完成的事情，与 Turing 机这一种机制所能完成的事情之间的等价性。最后一项没有切中要害，原因与第二项大致相同。

I.2.7 (源自 (Andreas Blass 2015).) 从这一论题得到的第一点益处是，它让我们能够将形式数学定理与现实世界中的可计算性问题联系起来。例如，关于“群的字问题是 Turing 不可判定的”这一定理，在现实世界中的解释是：不存在能够解决所有字问题实例的算法。

第二点益处体现在数学自身之中，特别是在可计算性理论里。在已发表的证明中，若要证明某事物是 Turing 可计算的，几乎从不通过展示一个 Turing 机程序（或任何实际编程语言中的程序）来进行。有时，如果事情足够简单，他们会给出某种伪代码。但最常见的情况是，他们仅提供一个算法的非形式化描述。读者需要自行看出这确实给出了一个算法（在直观意义上），因此，依照 Church-Turing 论题，它可以被 Turing 机模拟。通常的情况是，虽然 Turing 机编程专家（如果他们存在的话）能够轻车熟路地将直观算法转换为 Turing 机程序，但这样的程序会过于庞大和复杂，不值得写下来。

I.2.8 Lance Fortnow (兰斯·福特诺) 一语中的: “计算关乎过程, 关乎机器从一个状态到另一个状态的转移。计算不是关于输入和输出、点 A 和点 B, 而是关于这趟旅程。……你可以给 Turing 机输入一个实数的无穷多位数字 (Siegelmann), 让计算机彼此交互 (Wegner-Goldin), 或者让计算机执行一系列无穷无尽的任务 (Denning), 但在所有这些情况下, 过程保持不变, 每一步都遵循 Turing 的模型。” (Fortnow 2010)。

I.2.9

- (A) 是的, 这很清楚。为了计算 $f \circ g(x)$, 我们先计算 $y = g(x)$, 然后计算 $f(y)$ 。
 (B) 复合函数 $f \circ g$ 可能是可计算的, 也可能是不可计算的。一方面, 常值函数 $g(x) = 0$ 是可计算的, 而复合 $f \circ g(x)$ 等于 $f(0)$ 。由于 $f(0)$ 是固定的而非变量, 这个结果就是可计算的。
 另一方面, 如果 $g(x) = x$, 那么复合函数 $f \circ g(x) = f(x)$ 就是不可计算的。

I.2.10 我们可以用普通的编程语言 (也就是在二进制机器上) 来模拟一台三进制机器。而我们可以用 Turing 机来模拟二进制机器。因此, 我们可以用 Turing 机来模拟三进制机器。所以, 任何能在三进制机器上计算的东西, 也都能在 Turing 机上计算。

反过来更是轻而易举: 要在三进制机器上模拟二进制机器, 只需选定一个三值位并始终不用它。

I.2.11 Church 论题在这里的作用是, 它允许我们避免将 Turing 机作为四元组集合展示出来以执行这些计算。相反, 我们可以在通常的编程环境中描述如何执行它们。

- (A) 设 f_0 由 Turing 机 $\mathcal{P}_0$ 计算, f_1 由 $\mathcal{P}_1$ 计算。对于 $x \in \mathbb{N}$, 按以下方式计算 $h(x)$: 先在输入 x 上运行 $\mathcal{P}_0$ 一步, 然后在 x 上运行 $\mathcal{P}_1$ 一步, 再运行 $\mathcal{P}_0$ 一步, 如此交替进行, 直到两台机器都停机。一旦它们确实停机了, 就输出 1。
 (B) 与上一项类似, 对于 $x \in \mathbb{N}$, 按以下方式在输入 x 上计算该函数: 先在 x 上运行 $\mathcal{P}_0$ 一步, 然后在 x 上运行 $\mathcal{P}_1$ 一步, 再运行 $\mathcal{P}_0$ 一步, 如此交替进行。这里的不同之处在于, 只要有一台机器停机, 就输出 1。
 (C) 设 f 由 $\mathcal{P}$ 计算。固定 $x \in \mathbb{N}$ 。先在输入 0 上运行 $\mathcal{P}$ 一步。然后, 再在输入 0 上运行 $\mathcal{P}$ 一步, 并在输入 1 上运行 $\mathcal{P}$ 一步。之后, 再在输入 0 上运行 $\mathcal{P}$ 一步, 在输入 1 上运行 $\mathcal{P}$ 一步, 并开始在输入 2 上运行 $\mathcal{P}$ 。以此方式, 我们在机器对不同输入的运行之间轮转。

每当 $\mathcal{P}$ 在某个输入 i 上的计算停机时, 检查其纸带上剩余的内容是否等于 x 。若是, 则输出 1。

- (D) 设 f_0 由 Turing 机 $\mathcal{P}_0$ 计算, f_1 由 $\mathcal{P}_1$ 计算。对于 $x \in \mathbb{N}$, 在输入 x 上运行 $\mathcal{P}_0$ 直到它停机。假设输出为 y 。然后在输入 y 上运行 $\mathcal{P}_1$ 直到它停机, 所得输出即为 $h(x)$ 。由于 f_0 和 f_1 这两个函数都被给定为全函数, 输出最终一定会出现, 因此 h 也是全函数。
 (E) 这与上一项相同, 但函数 h 将是偏函数, 因为存在某些输入使得 $\mathcal{P}_0$ 不停机。

I.2.12 以下是一个直观上机械可计算的过程, 因此由 Church 论题, 它是可计算的。设输入为 n 。对于 $i = 0$, 计算 $f(i)$ 并查看它是否等于 n 。若是, 则输出 0。若不是, 我们就继续迭代: 取 $i = 1$ 并计算 $f(1)$ 。如果 $f(1)$ 等于 n , 则输出 0。否则, 重复此过程直到该过程输出某个值; 如果永远不输出, 则我们处于 h 定义中的第二种情况。

I.2.13 此过程是直观上机械可计算的, 因此由 Church 论题, 它是可计算的。设输入为 n 。在输入 n 上运行计算 f 的 Turing 机, 直到它停机 (如果它能停机的话)。如果停机了, 那么再运行计算 $g(n)$ 的 Turing 机, 直到它停机 (如果它能停机的话)。在那之后 (如果真有“在那之后”的话), 输出 1。否则, 我们就处于 h 定义中的第二种情况。

I.2.14 如果存在根, 那么这个过程最终会找到它。如果不存在根, 那么这个过程将永远运行下去, 进行无界搜索。

I.2.15 首先, 循环与不停机不是一回事——无限循环是不停机的一种方式, 但不是唯一方式。撇开这点不谈, 多带机不停机的一种方式忽略除第一条纸带外的所有纸带, 并以单带机不停机的方式不停机。(多带机还有其他不停机的方式, 但这表明让这样的机器停机并不更困难。)

I.2.16 策略是: 在纸带 0 上向右移动, 将补码位写到纸带 1 上, 直到纸带 0 的读写头遇到空白符。然后机器在纸带 1 上向左移动。(下面, 我们用 $\mathbf{xy}$ 来表示字符对, 而不是写成 $\langle \mathbf{x}, \mathbf{y} \rangle$ 的形式。)

Δ	00	01	0B	10	11	1B	B0	B1	BB
q_0	00, q_1	01, q_1	01, q_1	10, q_1	11, q_1	10, q_1	B0, q_1	B1, q_1	LL, q_2
q_1	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	LL, q_2
q_2	OL, q_2	OL, q_2	OR, q_3	1L, q_2	1L, q_2	1R, q_3	BL, q_2	BL, q_2	BR, q_3

(许多输入对不会出现。例如，如果机器处于状态 q_0 ，那么输入 00 是不会出现的。)

I.2.17 策略是在两条带上向右移动，将逻辑与的结果写到带 1 上，直到两个读写头都遇到空白符。然后机器在带 1 上向左移动。(在下文中，我们不将字符对写作 $\langle x, y \rangle$ ，而仅写作 xy 。)

Δ	00	01	0B	10	11	1B	B0	B1	BB
q_0	00, q_1	00, q_1	0B, q_1	10, q_1	11, q_1	1B, q_1	BB, q_1	BB, q_1	LL, q_2
q_1	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	RR, q_0	LL, q_2
q_2	LL, q_2	LL, q_2	OR, q_3	LL, q_2	LL, q_2	1R, q_3	BL, q_2	BL, q_2	BR, q_3

(其中许多输入对实际上不会出现。例如，如果机器处于状态 q_1 ，那么输入 B0 就不可能发生。)

I.2.18 也许可以预料到，我们最终会一无所有，因为没有任何一张钞票还留在我们手中。

I.3.10 “原始递归的”是指一个函数集合，而“原始递归”则是一种组合函数的方式。

I.3.11 该定义对所有 y (包括 $y = 0$) 都给出 1。

I.3.12

(A) $f(0) = 0$, $f(1) = f(2 \cdot 1 - 2) = f(0) = 0$

(B) 展开定义得到 $f(2) = f(2 \cdot 2 - 2) = f(2)$, 导致函数值无定义。

I.3.13

(A) 按照定义可得下表；例如， $F(1) = F(1 - 1) = F(0) = 42$ 。

n	0	1	2	3	4	5	6	7	8	9	10
$t(n)$	42	42	42	42	42	42	42	42	42	42	42

(B) 定义

$$F(y) = \begin{cases} g() & \text{— 若 } y = 0 \\ h(F(z), z) & \text{— 若 } y = \mathcal{S}(z) \end{cases}$$

是可行的，其中 g 是零元常函数 $g() = 42$ ，而 $h(a, b) = a$ 。

另外，由于 F 是常函数，我们也可以不借助原始递归模式，而用复合的方式构造它： $F(y) = \mathcal{S}(\mathcal{S}(\dots \mathcal{S}(y))) = \mathcal{S}^{42} \circ \mathcal{S}(y)$ 。

I.3.14 一种方法是注意到它是复合函数 $\text{plus_three} = \mathcal{S} \circ (\mathcal{S} \circ \mathcal{S})(x)$ 。

I.3.15 从原始递归定义的一般形式

$$\text{is_zero}(y) = \begin{cases} 1 & \text{— 若 } y = 0 \\ 0 & \text{— 若 } y = \mathcal{S}(z) \end{cases}$$

出发，我们取 g 为零元函数的复合 $g() = \mathcal{S}(Z())$ ，并取 h 为二元零函数 $h(a, b) = Z(a, b) = 0$ 。

I.3.16

(A) 这是前 11 个值；显然，这是平方数列。

n	0	1	2	3	4	5	6	7	8	9	10
$t(n)$	0	1	4	9	16	25	36	49	64	81	100

(B) 因为 s 是一元函数，根据元数记录，我们需要这样描述它。

$$s(y) = \begin{cases} g() & \text{— 若 } y = 0 \\ h(s(z), z) & \text{— 若 } y = \mathcal{S}(z) \end{cases}$$

所以 g 是零元函数, 因此是常数函数, $g() = Z() = 0$ 。对于 h , 我们需要 $h(s(z), z) = s(z) + (2 \cdot (z+1) - 1) = s(z) + (2z+1)$ 。所以我们取 $h(a, b) = \text{plus}(a, \mathcal{S}(\text{product}(\mathcal{S}(\mathcal{S}(Z()))), b))$ 。

I.3.17

(A) 以下是前 11 个三角数。

n	0	1	2	3	4	5	6	7	8	9	10
$t(n)$	0	1	3	6	10	15	21	28	36	45	55

(B) 因为 t 是一元函数, 根据元数记录, 我们需要像这样描述它。

$$t(y) = \begin{cases} 0 & - \text{若 } y = 0 \\ h(t(z), z) & - \text{若 } y = \mathcal{S}(z) \end{cases}$$

所以 g 是零元函数, 因此是常数函数, $g() = Z() = 0$ 。至于 h , 观察当 $n > 0$ 时, 我们有 $t(n) = t(n-1) + n$, 因为第 1 行有 1 个点, 第 2 行有 2 个点, 等等。所以我们想要 $h(t(z), z) = t(z) + (z+1)$, 将其放入所需格式得到 $h(a, b) = \text{plus}(a, \mathcal{S}(b))$ 。

I.3.18

(A) 这是前 6 个值, 表明 $d(n) = n^3$ 。

n	0	1	2	3	4	5
$d(n)$	0	1	8	27	64	125

(B) 给定的 d 是一元函数, 因此根据元数记录, g 是零元函数, h 是二元函数。

$$d(y) = \begin{cases} g() & - \text{若 } y = 0 \\ h(d(z), z) & - \text{若 } y = \mathcal{S}(z) \end{cases}$$

取 g 为常数 $g() = Z() = 0$ 。为了得到 $h(d(z), z) = d(z) + 3(z+1)^2 + 3(z+1) + 1$ 只需将其 (可能有点繁琐地) 翻译成低层函数。将 $d(z) + 3(z+1)^2 + 3(z+1) + 1$ 写成 $s + t + u + v$ 。那么 $h(a, b) = \text{plus}(s, \text{plus}(t, \text{plus}(u, v)))$ 其中 $s(a, b) = a$, 且 $u(a, b) = \text{product}(\mathcal{S}(\mathcal{S}(\mathcal{S}(Z()))), \text{plus}(b, \mathcal{S}(Z())))$, 且 $v(a, b) = \mathcal{S}(Z())$, 而 $t(a, b)$ 如下。

$$\text{product}(\mathcal{S}(\mathcal{S}(\mathcal{S}(Z()))), \text{product}(\text{plus}(b, \mathcal{S}(Z())), \text{plus}(b, \mathcal{S}(Z()))))$$

I.3.19

(A) 以下是 10 个值。

n	1	2	3	4	5	6	7	8	9	10
$h(n)$	1	3	7	15	31	63	127	255	511	1023

(B) 这是一个一元函数, 所以根据元数记录, 我们需要这个形式。

$$H(y) = \begin{cases} g() & - \text{若 } y = 0 \\ h(H(z), z) & - \text{若 } y = \mathcal{S}(z) \end{cases}$$

取 g 为常数函数 $g() = 1 = \mathcal{S}(Z())$ 。要得到 $h(H(z), z) = 2 \cdot H(z) + 1$, 取 $h(a, b) = \mathcal{S}(\text{product}(a, \mathcal{S}(\mathcal{S}(Z()))))$ 。

I.3.20 阶乘的推导如下。

$$\text{fact}(y) = \begin{cases} g() & \text{如果 } y = 0 \\ h(\text{fact}(z), \mathcal{S}(z)) & \text{如果 } y = \mathcal{S}(z) \end{cases}$$

这里 g 是零元函数 $g() = 1 = \mathcal{S}(Z())$ 而 h 是二元函数 $h(a, b) = \text{product}(a, \mathcal{S}(b))$ 。

I.3.21

- (A) 假设两者均大于零。若 $a > b > 0$ ，我们知道该怎么做。对于 $a < b$ 的情况，我们可以交换两者的顺序， $\gcd(a, b) = \gcd(b, a)$ ，因为等式两边都是找出所有公除数并取最大值。而对于 $a = b$ ，最大公约数就是 a 本身。

$$\gcd(a, b) = \begin{cases} 0 & \text{— 若 } a = b = 0 \\ a & \text{— 若 } b = 0 \text{ 且 } a \neq 0 \\ b & \text{— 若 } a = 0 \text{ 且 } b \neq 0 \\ a & \text{— 若 } a = b \\ \gcd(b, a) & \text{— 若 } a < b \\ \gcd(a - b, b) & \text{— 若 } a > b \end{cases}$$

为了有趣，我们可以写一个程序（它合并了上述的某些情况）。

```
(define (gcd-first a b)
  (cond
    [(zero? a) b]
    [(zero? b) a]
    [(> b a) (gcd-first b a)]
    [else (gcd-first (- a b) b)]))
```

- (B) 这些计算用手算很容易，但这里的结果来自程序：

```
> (gcd-first 28 12)
4
> (gcd-first 104 20)
4
> (gcd-first 300009 25)
1
```

- (C) 同样为了有趣，我们将其写成了 Racket 脚本。

```
(define (gcd-second a b)
  (if (zero? b)
      a
      (gcd-second b (remainder a b))))
```

结果如下。

```
> (gcd-second 28 12)
4
> (gcd-second 104 20)
4
> (gcd-second 300009 25)
1
```

I.3.22 这些解答使用了熟悉的记号。例如，我们写作 $x + y$ ，而不是 $\text{plus}(x, y)$ 。

- (A) 总是返回零的常数函数 $\mathcal{C}_0(\vec{x})$ 已包含在原始递归的定义（定义 3.8）中，即 $\mathcal{Z}(\vec{x}) = 0$ 。总是返回 1 的函数则是 $\mathcal{C}_1(\vec{x}) = \mathcal{S}(\mathcal{Z}(\vec{x}))$ ，返回 2 的函数是 $\mathcal{C}_2(\vec{x}) = \mathcal{S}(\mathcal{S}(\mathcal{Z}(\vec{x})))$ ，依此类推。由于后继、零和复合在构造原始递归函数时都是被允许的，因此每个 $\mathcal{C}_k$ 都是原始递归的。
- (B) 使用截断减法定义 $\max(\{x, y\}) = y + (x \dot{-} y)$ 。若 $x \geq y$ ，则它可化简为 $\max(\{x, y\}) = y + (x - y) = x$ ，而若 $x < y$ ，则它是 $\max(\{x, y\}) = y + 0 = y$ 。 $\max(\{x, y\}) = y + (x \dot{-} y)$ 的右边部分都是原始递归的，所以 $\max$ 函数也是原始递归的。类似地， $\min(\{x, y\}) = x + (y \dot{-} \max(\{x, y\}))$ 。
- (C) 令 $\text{absdiff}(x, y) = (x \dot{-} y) + (y \dot{-} x)$ 。若 $x \geq y$ ，则只有第二项为零，且 $\text{absdiff}(x, y) = x - y$ ，这符合预期。 $y > x$ 的情况是类似的。由于该表达式是由原始递归的部分通过原始递归的组合子放在一起构成的， $\text{absdiff}(x, y)$ 是原始递归的。

I.3.23 这些解答使用了熟悉的记号。例如，我们写 $x + y$ 而不写 $\text{plus}(x, y)$ 。我们也将省略显式说明：表达式由原始递归的部分通过原始递归的组合子构成，因此函数是原始递归的。

(A) 以下原始递归定义了 sgn 。

$$\text{sgn}(y) = \begin{cases} 0 & \text{若 } y = 0 \\ 1 & \text{若 } y = \mathcal{S}(z) \end{cases}$$

注意，在第一个子句中，我们写 0 作为 $Z(y)$ 的缩写，写 1 作为 $\mathcal{S}(Z(y))$ 的缩写。

(B) 使用原始递归即可定义。

$$\text{negsign}(y) = \begin{cases} 1 & \text{若 } y = 0 \\ 0 & \text{若 } y = \mathcal{S}(z) \end{cases}$$

但也请注意， $\text{negsign}(y) = 1 - \text{sgn}(y)$ ，即 $\text{negsign}(y) = 1 \dot{-} \text{sgn}(y)$ 。

(C) 第一个是 $\text{lessthan}(x, y) = \text{sgn}(y \dot{-} x)$ 而另一个是 $\text{greaterthan}(x, y) = \text{sgn}(x \dot{-} y)$ 。

I.3.24 这些解答使用了熟悉的记号。例如，我们写 $x + y$ 而不写 $\text{plus}(x, y)$ 。我们也将省略显式说明：表达式由原始递归的部分通过原始递归的组合子构成，因此函数是原始递归的。

(A) 我们有 $\text{not}(x) = 1 \dot{-} x$ 。对于两个输入的布尔函数， $\text{and}(x, y) = \text{sgn}(x) \cdot \text{sgn}(y) = \text{product}(\text{sgn}(x), \text{sgn}(y))$ 且 $\text{or}(x, y) = \text{sgn}(x + y)$ （另一种做法是 $\text{or}(x, y) = 1 \dot{-} ((1 \dot{-} x) \cdot (1 \dot{-} y))$ ）。

(B) $\text{equal}(x, y) = \text{negsign}(\text{lessthan}(x, y) + \text{greaterthan}(x, y))$

I.3.25 这些解答使用了熟悉的记号。例如，我们写 $x + y$ 而不写 $\text{plus}(x, y)$ 。我们也将省略显式说明：表达式由原始递归的部分通过原始递归的组合子构成，因此函数是原始递归的。

(A) $\text{notequal}(x, y) = \text{sign}(\text{lessthan}(x, y) + \text{greaterthan}(x, y))$

(B) 第一个是 $m(x) = 7 \cdot \text{equal}(x, 1) + 9 \cdot \text{equal}(x, 5) + 2 \cdot (\text{and}(\text{notequal}(x, 1)), \text{notequal}(x, 5))$ 。第二个类似， $n(x, y) = 7 \cdot \text{equal}(x, 1) \cdot \text{equal}(y, 2) + 9 \cdot \text{equal}(x, 5) \cdot \text{equal}(y, 5)$ 。

I.3.26 这些答案使用熟悉的记号。例如，我们写 $x + y$ 而不是 $\text{plus}(x, y)$ 。我们也将省略对以下事实的明确说明：这些表达式是由原始递归部分通过原始递归组合子组合而成，因此该函数是原始递归的。

(A) 由于 g 是给定的原始递归函数，函数 $h(a, b) = a + g(b)$ 也是原始递归的。利用这一点，以下原始递归式即可实现有界求和。

$$f(y) = \begin{cases} 0 & \text{若 } y = 0 \\ h(f(z), z) & \text{若 } y = \mathcal{S}(z) \end{cases}$$

举一个例子。我们将展开 $f(4)$ 。

$$\begin{aligned} f(4) &= h(f(3), 3) = f(3) + g(3) \\ &= h(f(2), 2) + g(3) = f(2) + g(2) + g(3) \\ &= h(f(1), 1) + g(2) + g(3) = f(1) + g(1) + g(2) + g(3) \\ &= h(f(0), 0) + g(1) + g(2) + g(3) = f(0) + g(0) + g(1) + g(2) + g(3) = g(0) + \cdots + g(4) \end{aligned}$$

(B) 这与有界求和类似。

(C) 考虑有界乘积函数。

$$P_p(\vec{x}, m) = p(\vec{x}, 0) \cdot p(\vec{x}, 1) \cdots p(\vec{x}, m-1) = \prod_{0 \leq i < m} p(\vec{x}, i)$$

这里需要注意两点。第一，如果 $P_p(\vec{x}, m) = 1$ ，那么 $P_p(\vec{x}, i) = 1$ 对所有 $i < m$ 成立。第二，如果存在某个 m 使得 $P_p(\vec{x}, m) = 0$ ，那么 $P_p(\vec{x}, n) = 0$ 对所有 $m \leq n$ 成立。因此序列 $P_p(\vec{x}, 0), P_p(\vec{x}, 1), P_p(\vec{x}, 2), \dots$ 由一连串的 1 后跟全 0 组成。开头的那些 1 一直持续到第一个使得谓词 $p(\vec{x}, n)$ 给出 0 的 n 为止。

接下来考虑有界求和函数。

$$S_{P_p}(\vec{x}, m) = \sum_{0 \leq i < m} P_p(\vec{x}, i)$$

上一段说明 $S_{P_p}(\vec{x}, m)$ 就是满足 $n < m$ 且 $p(\vec{x}, n) = 0$ 的最小数 n 。

现在分情况定义 $M(\vec{x}, m)$ 。

$$M(\vec{x}, m) = \begin{cases} m & - \text{若 } S_{P_p}(\vec{x}, m) = 0 \\ S_{P_p}(\vec{x}, m) & - \text{否则} \end{cases}$$

I.3.27 这些答案使用熟悉的记号。例如，我们写 $x + y$ 而不是 $\text{plus}(x, y)$ 。我们也将省略对以下事实的明确说明：这些表达式是由原始递归部分通过原始递归组合子组合而成，因此该函数是原始递归的。

(A) 我们可以用原始递归定义 U 。

$$U(\vec{x}, m) = \begin{cases} 1 & - \text{若 } m = 0 \\ p(\vec{x}, z) \cdot U(\vec{x}, z) & - \text{若 } m = \mathcal{S}(z) \end{cases}$$

例如，以下是 $U(\vec{x}, 3)$ 的计算。

$$\begin{aligned} U(\vec{x}, 3) &= p(\vec{x}, 2) \cdot U(\vec{x}, 2) \\ &= p(\vec{x}, 2) \cdot p(\vec{x}, 1) \cdot U(\vec{x}, 1) \\ &= p(\vec{x}, 2) \cdot p(\vec{x}, 1) \cdot p(\vec{x}, 0) \cdot U(\vec{x}, 0) \\ &= p(\vec{x}, 2) \cdot p(\vec{x}, 1) \cdot p(\vec{x}, 0) \cdot 1 \end{aligned}$$

(B) 先看 $m = 3$ 的例子。考虑这个。

$$\hat{E}(\vec{x}, 3) = 1 \div [(1 \div p(\vec{x}, 2)) \cdot (1 \div p(\vec{x}, 1)) \cdot (1 \div p(\vec{x}, 0))]$$

如果任意一个 $p(\vec{x}, i)$ 为 1，那么方括号内的整个表达式为 0，因此 $\hat{E}$ 为 1。如果所有 $p(\vec{x}, i)$ 都为 0，那么方括号内的表达式为 1，因此 $\hat{E}$ 为 0。

因此，令 $E(\vec{x}, m) = 1 \div \hat{E}(\vec{x}, m)$ ，其中后一个函数定义如下。

$$\hat{E}(\vec{x}, m) = \begin{cases} 0 & - \text{若 } m = 0 \\ (1 \div p(\vec{x}, z)) \cdot \hat{E}(\vec{x}, z) & - \text{若 } m = \mathcal{S}(z) \end{cases}$$

(C) 这直接从定义得出，只需注意到商 k 的一个合适上界是被除数 y ，即 $\text{divides}(x, y) = \exists d \leq y [y = x \cdot d]$ 。

(D) 与前一小题类似，我们只需要找到一个合适的上界。这里， $\text{prime}(y) = 1$ 当且仅当 $1 < y$ 且不存在 $x < y$ 满足 $x > 1$ 且 $\text{divides}(x, y)$ 。我们已验证所有这些组成部分都是原始递归的。

I.3.28 以下解答使用熟悉的记号。例如，我们写 $x + y$ 而不是 $\text{plus}(x, y)$ 。我们还将省略对以下事实的明确说明：这些表达式是由原始递归部分通过原始递归组合子组合而成，因此该函数是原始递归的。

(A) 这个表格一目了然。

a	0	1	2	3	4	5	6	7
$\text{rem}(a, 3)$	0	1	2	0	1	2	0	1

(B) 当 $\text{rem}(a, 3) = 2$ 时，这个关系不成立。

(C) 这由前一小题得出：第 (1) 项是 0，第 (2) 项也是 0，第 (3) 项是 $\text{rem}(z, 3) + 1$ 。

$$\text{rem}(a, 3) = \begin{cases} 0 & - \text{若 } a = 0 \\ 0 & - \text{若 } a = \mathcal{S}(z) \text{ 且 } \text{rem}(z, 3) + 1 = 3 \\ \text{rem}(z, 3) + 1 & - \text{若 } a = \mathcal{S}(z) \text{ 且 } \text{rem}(z, 3) + 1 \neq 3 \end{cases}$$

(D) 以下是按原始递归形式给出的显式定义。

$$\text{rem}(a, 3) = \begin{cases} 0 & - \text{若 } a = 0 \\ \text{product}(\mathcal{S}(\text{rem}(z, 3)), \text{notequal}(\mathcal{S}(\text{rem}(z, 3)), 3)) & - \text{若 } a = \mathcal{S}(z) \end{cases}$$

(E) 将定义中的 3 改为 b 。

$$\text{rem}(a, b) = \begin{cases} 0 & \text{— 若 } a = 0 \\ \text{product}(\mathcal{S}(\text{rem}(z, b)), \text{notequal}(\mathcal{S}(\text{rem}(z, b)), b)) & \text{— 若 } a = \mathcal{S}(z) \end{cases}$$

I.3.29 以下解答使用熟悉的记号。例如，我们写 $x + y$ 而不是 $\text{plus}(x, y)$ 。我们还将省略对以下事实的明确说明：这些表达式是由原始递归部分通过原始递归组合子组合而成，因此该函数是原始递归的。

(A) 这个除法很容易。

a	0	1	2	3	4	5	6	7	8	9	10
$\div(a, 3)$	0	0	0	1	1	1	2	2	2	3	3

(B) 一种表述方式是，除了当 $\text{rem}(a + 1, 3) = 0$ 之外，均有 $\div(a + 1, 3) = \div(a, 3)$ 。

(C) 基于前一小题，以下是按原始递归形式给出的显式定义（再次说明，两个输入的顺序并不重要）。

$$\div(a, 3) = \begin{cases} 0 & \text{— 若 } a = 0 \\ \text{plus}(\div(z, 3), \text{equal}(\text{rem}(\mathcal{S}(z), 3), 0)) & \text{— 若 } a = \mathcal{S}(z) \end{cases}$$

(D) 将定义中的 3 替换为 b 。

$$\div(a, b) = \begin{cases} 0 & \text{— 若 } a = 0 \\ \text{plus}(\div(z, b), \text{equal}(\text{rem}(\mathcal{S}(z), b), 0)) & \text{— 若 } a = \mathcal{S}(z) \end{cases}$$

I.3.30 这从有界极小化开始。

$$f(x, y) = \lfloor x/y \rfloor = \min_{t \leq x} [(t+1) \cdot y > x]$$

例如， $f(9, 2) = \lfloor 9/2 \rfloor = \min_{t \leq 9} [(t+1) \cdot 2 > 9]$ 得到 $t+1 = 5$ ，所以 $t = 4$ 。另一个例子是， $f(8, 2) = \lfloor 8/2 \rfloor = \min_{t \leq 8} [(t+1) \cdot 2 > 9]$ 也得到 $t+1 =$ ，所以 $t = 4$ 。将它用先前定义的函数表示，例如将乘号点改为调用 product 函数，是常规做法。

I.3.31

(A) $G(\langle 3, 1 \rangle) = 2^4 \cdot 3^2 = 144$ ， $G(\langle 2, 2, 2 \rangle) = 2^3 \cdot 3^3 \cdot 5^3 = 27\,000$ 。

(B) 前两个是 $A_0 = \langle 3, 2, 1 \rangle$ 和 $A_1 = \langle 1, 0 \rangle$ 。对于第三个，其质因数分解为 $343 = 7^3$ 。由于没有因子 2（或 3 或 5），因此不存在这样的序列 A_2 。

(C) 如果不使用后继函数，那么 $A_0 = \langle 1, 0 \rangle$ 和 $A_1 = \langle 1 \rangle$ 将具有相同的编码。

(D) 第一个很容易， $\bar{F}(0) = G(\langle \rangle) = 1$ 。因为 $F(0) = 1$ ，我们有 $\bar{F}(1) = G(\langle \rangle F(0)) = 2^2 = 4$ 。然后 $F(1) = 1$ ，所以 $\bar{F}(2) = G(\langle \rangle F(0), F(1)) = 2^2 \cdot 3^2 = 36$ 。最后， $F(2) = 2$ ，所以 $\bar{F}(3) = G(\langle \rangle F(0), F(1), F(2)) = 2^2 \cdot 3^2 \cdot 5^3 = 4500$ 。

I.3.32 这是一个实现。

```
(define (M x)
  (if (<= x 100)
      (M (M (+ x 11)))
      (- x 10)))
```

(A) 输出值为：如果 $x \in \{0, \dots, 101\}$ ，则 $M(x) = 91$ ，否则 $M(x) = x - 10$ 。

(B) 该函数是原始递归的，这可由习题 3.25 和习题 3.22 得到。

I.3.33 定义 3.8 中的每个初始函数都是全函数。每个原始递归函数都是通过有限次应用函数复合和原始递归，从初始函数派生而来。这两种组合操作都能从全函数输入得到全函数输出。（严格来说，最后两句话使用了归纳法。）

I.4.10 全递归函数是指所有输入上都收敛的可计算函数，即由一台在所有输入上都停机的机器所计算的函数。原始递归函数是指能够用定义 3.8 中的模式推导出来的函数。

每个原始递归函数都是全递归函数。反之不成立——存在不是原始递归的全递归函数，例如 Ackermann 函数。

I.4.11 $\mathcal{H}_4(2, 0) = 1$ 可以直接从定义得出。接着, $\mathcal{H}_4(2, 1) = \mathcal{H}_3(2, \mathcal{H}_4(2, 0))$, 由此得到 $\mathcal{H}_2(2, \mathcal{H}_3(2, 0)) = \mathcal{H}_1(2, \mathcal{H}_2(2, 0)) = 2$ 。

最后三个类似但很繁琐，因此最好的方法是编写脚本。以下是对本节正文给出的 $\mathcal{H}$ 定义的直接转录。

```
(define (H n x y)
  (cond
    [(= n 0) (+ y 1)]
    [(and (= n 1) (= y 0)) x]
    [(and (= n 2) (= y 0)) 0]
    [(and (> n 2) (= y 0)) 1]
    [else (H (- n 1) x (H n x (- y 1)))]))
```

这些调用给出了答案。

```
> (H 4 2 0)
1
> (H 4 2 1)
2
> (H 4 2 2)
4
> (H 4 2 3)
16
> (H 4 2 4)
65536
```

最后一个耗时较长；下面的第一个数字是 CPU 时间的毫秒数。

```
cpu time: 155726 real time: 157987 gc time: 53317
65536
```

所以它耗时 156 秒，大约三分钟（在一台标准商务笔记本电脑上）。以下是输出值的汇总。

y	0	1	2	3	4
$\mathcal{H}(4, 2, y)$	1	2	4	16	65 536

I.4.12 通过脚本计算得出 $\mathcal{H}_4(3, 3) = 7\,625\,597\,484\,987$ 。除以 $60 \cdot 60 \cdot 24 \cdot 365.25$ 得到年数约为二十五万，即 241 640 年。

I.4.13 我们有 $\mathcal{H}_3(3, 3) = 27$ 和 $\mathcal{H}_2(2, 2) = 4$ ，所以比值是 6.75。

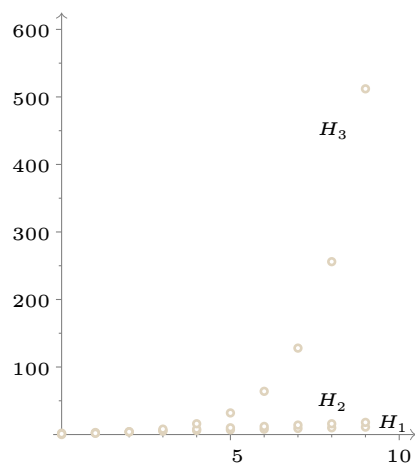
I.4.14 我们可以通过引理 4.2 或本节正文给出的 Racket 过程来获取数值。

```
> (for ([y (in-range 10)])
      (printf "(~a,~a),\n" y (H 1 2 y)))
(0,2),
(1,3),
(2,4),
(3,5),
(4,6),
(5,7),
(6,8),
(7,9),
(8,10),
(9,11),
```

下面汇总了输出结果。

y	0	1	2	3	4	5	6	7	8	9
$\mathcal{H}_1(2, y)$	2	3	4	5	6	7	8	9	10	11
$\mathcal{H}_2(2, y)$	0	2	4	6	8	10	12	14	16	18
$\mathcal{H}_3(2, y)$	1	2	4	8	16	32	64	128	256	512

图像如下。



I.4.15 编写一个小脚本能让计算工作更轻松。下面是数值表。

	$y = 0$	$y = 1$	$y = 2$	$y = 3$	$y = 4$	$y = 5$
$k = 0$	1	2	3	4	5	6
$k = 1$	2	3	4	5	6	7
$k = 2$	3	5	7	9	11	13
$k = 3$	5	13	29	61	125	253

I.4.16 下面的 Racket 程序阐释了这个定义。

```
(define (g x y)
  (if (= 100 (+ x y))
      0
      1))

(define (f x)
  (define (f-helper y) ; value of x inherited from enclosing defn of f
    (if (= 0 (g x y))
        y
        (f-helper (+ 1 y))))
  (f-helper 0))
```

Racket 会计算出这些值。

```
> (f 0)
100
> (f 1)
99
> (f 50)
50
> (f 100)
0
```

必须终止对 (f 101) 的计算，因为它会永远运行下去。

- (A) $f(0) = 100$
- (B) $f(1) = 99$
- (C) $f(50) = 50$
- (D) $f(100) = 0$
- (E) 无定义。

这是 f 的另一种描述。

$$f(x) = \begin{cases} 100 - x & \text{— 若 } x \leq 100 \\ \uparrow & \text{— 其他情形} \end{cases}$$

I.4.17

下面的 Racket 程序阐释了这个定义。

```
(define (g-mult x y)
  (if (= 100 (* x y))
      0
      1))

(define (f-mult x)
  (define (f-helper y) ; value of x inherited from enclosing defn of f-mult
    (if (= 0 (g-mult x y))
        y
        (f-helper (+ 1 y))))
  (f-helper 0))
```

Racket 会计算出其中一些值。

```
> (f 1)
100
> (f 50)
2
> (f 100)
1
```

- (A) $f(0)$ 的值无定义，因为不存在 y 使得 $0 \cdot y$ 等于 100。
- (B) $f(1) = 100$
- (C) $f(50) = 2$
- (D) $f(100) = 1$
- (E) 无定义，因为不存在 y 使得 $101 \cdot y$ 等于 100。

I.4.18 如前所述，已知最大的 Fermat 素数是 $F_4 = 65537$ 。

- (A) $f(0) = 5$
- (B) $f(1) = 17$
- (C) 我们不知道 $f(50)$ 是否有定义，因为我们不知道是否存在大于 F_{50} 的 Fermat 素数。
- (D) 我们不知道 $f(F_4)$ 是否有定义，因为我们不知道是否存在大于 F_{F_4} 的 Fermat 素数。

I.4.19 对于函数 f 的每个输入 x ，搜索满足 $y^3 \geq x^2$ 的第一个 y 。

- (A) $f(0) = 0$
- (B) $f(1) = 1$
- (C) $f(50) = 14$ ，因为 $50^2 = 2500$ ，而 $14^3 = 2744$ 。
- (D) $f(100) = 22$
- (E) $f(x) = \lceil x^{2/3} \rceil$ ，其中 ‘ceiling’ 记号表示取大于或等于 $x^{2/3}$ 的最小整数。

I.4.20 如果存在 $k \in \mathbb{N}$ 且 $k \geq 1$ 使得 $d \cdot k = n$ ，则称数 $d \in \mathbb{N}$ 整除 $n \in \mathbb{N}$ 。

- (A) 我们有 $\text{notrelprime}(0, 0) = 0$ ，因为 $d = 2$ 能同时整除它们。它之所以能整除 $n = 0$ ，是因为存在 $k \in \mathbb{N}$ 使得 $2 \cdot k = 0$ ，即 $k = 0$ 。

- (B) 能整除 1 的数只有 1 本身, 所以 $f(1) \uparrow$ 。
 (C) $f(2) = 2$
 (D) $f(3) = 3$
 (E) $f(4) = 2$
 (F) $f(42) = 2$
 (G) 如果 $x = 0$, 则 $f(x) = 0$ 。如果 $x = 1$, 则 $f(x)$ 无定义。否则 $f(x) = p$, 其中 p 是能整除 x 的最小素数。

I.4.21 以下 Racket 脚本有助于进行计算。

```
(define (g x y)
  (ceiling (- (/ (add1 x) (add1 y))
              1)))

(define (f x)
  (define (f-helper y)
    (if (= 0 (g x y))
        y
        (f-helper (add1 y))))

  (f-helper 0))
```

(A) 输出结果如下。

```
> (f 0)
0
> (f 1)
1
> (f 2)
2
> (f 3)
3
> (f 4)
4
> (f 5)
5
> (f 6)
6
```

这些值的解释源自下表相关的 $g(x, y)$ 值。

$g(x, y)$	$y = 0$	$y = 1$	$y = 2$	$y = 3$	$y = 4$	$y = 5$	$f(x)$
$x = 0$	0						0
$x = 1$	1	0					1
$x = 2$	2	1	0				2
$x = 3$	3	1	1	0			3
$x = 4$	4	2	1	1	0		4
$x = 5$	5	2	1	1	1	0	5

(B) 我们将论证 $f(x) = x$ 。固定 x 。要使 $g(x, y) = 0$, 我们需要 $(x+1)/(y+1)$ 小于或等于 1。满足此条件的第一个 y 是 x 本身。

I.4.22 不能。该集合中的所有函数都将是全函数, 即它们在所有输入上都会停机。但有些可计算函数不是全函数——有些有效过程在某些输入上不会停机。

I.4.23 引理 4.2 的证明表明了 $\mathcal{H}_1(x, y) = x + y$ 。根据 $\mathcal{H}$ 的定义可得此结论。

$$\mathcal{H}_2 = \begin{cases} 0 & \text{— 若 } y = 0 \\ \mathcal{H}_1(x, \mathcal{H}_2(x, y-1)) & \text{— 其他情形} \end{cases}$$

我们将用归纳法证明 $\mathcal{H}_2(x, y) = x \cdot y$ 。基本情况是 $y = 0$ ，此时 $x \cdot y = 0$ ，这与定义中的第一种情况相符。现在进行归纳步骤。假设对于 $y = 0, y = 1, \dots, y = k$ ，有 $\mathcal{H}_2(x, y) = x \cdot y$ 。对于 $y = k + 1$ ，由于 y 不可能为零，应用“否则”子句，于是 $\mathcal{H}_2(x, y) = \mathcal{H}_1(x, x \cdot k) = x + x \cdot k = x \cdot (1 + k) = x \cdot y$ 。

另一个等式的证明类似。根据 $\mathcal{H}$ 的定义可得此结论。

$$\mathcal{H}_3 = \begin{cases} 1 & - \text{若 } y = 0 \\ \mathcal{H}_2(x, \mathcal{H}_3(x, y - 1)) & - \text{其他情形} \end{cases}$$

我们将用归纳法证明 $\mathcal{H}_3(x, y) = x^y$ 。对于基本情况 $y = 0$ ，我们有 $x^0 = 1$ ，这与第一种情况相符。在归纳步骤中，假设归纳假设：对于 $y = 0, y = 1, \dots, y = k$ ，有 $\mathcal{H}_3(x, y) = x^y$ 。考虑 $y = k + 1$ 。由于 y 不为零，应用“否则”子句，得到 $\mathcal{H}_3(x, y) = \mathcal{H}_2(x, x^k) = x \cdot x^k = x^{1+k} = x^y$ 。

I.4.24 在每次应用递归时，要么第一自变量减小，要么第一自变量保持不变而第三自变量减小。此外，每当第三自变量减至零时，第一自变量就会减小，因此它最终也会减至零。如果第一自变量为零，则计算终止。

I.4.25 函数 `remtwo` 只是在 0 和 1 之间来回切换。

y	0	1	2	3	4	5	...
<code>remtwo(y)</code>	0	1	0	1	0	1	...

(A) 相应的原始递归也很简单：

$$\text{remtwo}(y) = \begin{cases} 0 & - \text{若 } y = 0 \\ 1 \div \text{remtwo}(z) & - \text{若 } y = S(z) \end{cases}$$

(像我们经常做的那样，为了清晰起见，这里使用了缩写。例如，我们没有写函数 $Z()$ ，而是直接写了数字 0；类似地，将 $S(Z())$ 缩写为 1。)

(B) 令 $f(n) = \mu y [y + \text{remtwo}(n) = 0]$ 。若 n 为偶数，则 $\text{remtwo}(n) = 0$ ，且 $y = 0$ 即可。若 n 为奇数，则 $\text{remtwo}(n) = 1$ ，并且不存在自然数 y 使得 $y + 1 = 0$ 。

I.4.26 我们可以通过运行机器直到它停机来计算 $f(x)$ 。这是 $x = 0$ 时的计算过程。

Step	Machine
0	q₀
1	1 q₁
2	1 q₁
3	1 q₂

这是 $x = 1$ 时的计算过程。

Step	Machine	Step	Machine
0	1 q₀	3	1 1 q₁
1	1 q₀	4	1 1 q₁
2	1 1 q₁	5	1 1 q₂

$x = 2$ 时情况类似。

<i>Step</i>	<i>Machine</i>	<i>Step</i>	<i>Machine</i>	<i>Step</i>	<i>Machine</i>
0		3		6	
1		4		7	
2		5			

这是 $x = 3$ 时的计算过程。

<i>Step</i>	<i>Machine</i>	<i>Step</i>	<i>Machine</i>	<i>Step</i>	<i>Machine</i>
0		4		7	
1		5		8	
2		6		9	
3					

这是 $x = 4$ 时的计算过程。

<i>Step</i>	<i>Machine</i>	<i>Step</i>	<i>Machine</i>	<i>Step</i>	<i>Machine</i>
0		4		8	
1		5		9	
2		6		10	
3		7		11	

最后，这是 $f(5)$ 的计算过程。

Step	Machine	Step	Machine	Step	Machine
0	1 1 1 1 1 q₀	5	1 1 1 1 1 q₀	10	1 1 1 1 1 1 q₁
1	1 1 1 1 1 q₀	6	1 1 1 1 1 1 q₁	11	1 1 1 1 1 1 q₁
2	1 1 1 1 1 q₀	7	1 1 1 1 1 1 q₁	12	1 1 1 1 1 1 q₁
3	1 1 1 1 1 q₀	8	1 1 1 1 1 1 q₁	13	1 1 1 1 1 1 q₂
4	1 1 1 1 1 q₀	9	1 1 1 1 1 1 q₁		

下表是对结果的总结。

x	0	1	2	3	4	5
$f(x)$	3	5	7	9	11	13

I.4.27 要计算 $f(x)$ ，我们可以运行机器直到它停机。这是 $x = 0$ 时的计算过程，

Step	Machine
0	q₀
1	1 q₂

这是 $x = 1$ 时的计算过程。

Step	Machine
0	1 q₀
1	1 q₁
2	1 1 q₂

$x = 2$ 时的计算与此相同。

Step	Machine
0	1 1 q₀
1	1 1 q₁
2	1 1 1 q₂

$x = 3$ 时的计算也是如此。

Step	Machine
0	1 1 1 q₀
1	1 1 1 q₁
2	1 1 1 1 q₂

这是 $x = 4$ 时的计算过程。

Step	Machine
0	1 1 1 1 q₀
1	1 1 1 1 q₁
2	1 1 1 1 1 q₂

最后，这是 $f(5)$ 的计算过程。

Step	Machine
0	1 1 1 1 1 q₀
1	1 1 1 1 1 q₁
2	1 1 1 1 1 1 q₂

下表是对结果的总结。

x	0	1	2	3	4	5
$f(x)$	2	3	3	3	3	3

对于这台机器，运行时间是常数。前一练习中的机器完成相同的工作，但其运行时间是输入大小的线性函数。

I.4.28 要计算不同输入下的 $f(x)$ ，一种方法是运行这台机器直到它停机。

(A) 这是 $x = 0$ 时的计算过程。

Step	Machine	Step	Machine
0	q₀	3	q₃
1	q₁	4	q₄
2	q₂		

(B) 这是 $x = 1$ 时的情形。

Step	Machine	Step	Machine
0	1 q₀	3	1 q₃
1	1 q₁	4	1 q₄
2	1 q₂		

(c) 这是 $x = 2$ 时的计算过程。

Step	Machine	Step	Machine
0	1 1 q₀	3	1 1 q₃
1	1 1 q₁	4	1 1 q₄
2	1 1 q₂		

(d) 下表是总结。

x	0	1	2
$f(x)$	4	4	4

这台机器会忽略其输入，先右移两格，再左移两格，然后停机。所以它的运行时间是常数，即 $f(x) = 4$ 。

I.4.29 这些值手算并不太难，但用一个小程序也很方便。

```
;; 3n+1 function
(define (H n)
  (if (even? n)
      (/ n 2)
      (+ (* 3 n) 1)))

(define (C n)
  (define (C-helper n k)
    (if (= 1 n)
        k
        (C-helper (H n) (add1 k))))
  (C-helper n 0))
```

(A) 脚本给出以下结果。

```
> (H 4)
2
> (H (H 4))
1
> (H (H (H 4)))
4
```

因此我们有：

k	0	1	2	3
$H^k(4)$	4	2	1	4

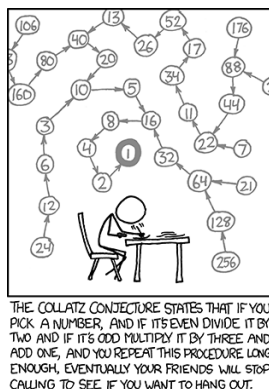


图 1, FOR QUESTION I.4.29: 由 xkcd.com 提供

(B) 脚本显示:

```
> (C 4)
2
```

这与前一项的结果相符, 即 H 作用两次后结果为 1。

(C) 以下是连续计算得到的值。

```
> (H 5)
16
> (H (H 5))
8
> (H (H (H 5)))
4
> (H (H (H (H 5))))
2
> (H (H (H (H (H 5)))))
1
```

因此 $C(5) = 5$, 这与程序结果一致。

```
> (C 5)
5
```

(D) 连续计算得到的值为 11、34、17、52、26、13、40、20、10、5、16、8、4、2 和 1, 所以 $C(11) = 14$ 。

(E) 各值如下。

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
$C(n)$	0	1	7	2	5	8	16	3	19	6	14	9	9	17	15	4	12	20	20

图 1 表明, XKCD 对 Collatz 猜想也有话要说。

I.4.30 如果 D 是原始递归的, 那么它就是某个 f_i 。但这样一来 $D(i) = f_i(i) + 1 = D(i) + 1$, 这是不可能的。

I.A.1 以下是以五个 1 作为初始输入的命令行运行记录。

```
$ ./turing-machine.rkt -f machines/pred.tm -c "1" -r "1111"
step 0: q0: *1*1111
step 1: q0: 1*1*111
step 2: q0: 11*1*11
step 3: q0: 111*1*1
step 4: q0: 1111*1*
step 5: q0: 11111*B*
```

```
step 6: q1: 1111*1*B
step 7: q1: 1111*B*B
step 8: q2: 111*1*BB
step 9: q2: 11*1*1BB
step 10: q2: 1*1*11BB
step 11: q2: *1*111BB
step 12: q2: *B*1111BB
step 13: q3: B*1*111BB
step 14: HALT
```

以下是其纸带图格式。

Step	Machine	Step	Machine	Step	Machine
0	11111 q₀	5	11111 q₀	10	1111 q₂
1	11111 q₀	6	11111 q₁	11	1111 q₂
2	11111 q₀	7	1111 q₁	12	1111 q₂
3	11111 q₀	8	1111 q₂	13	1111 q₃
4	11111 q₀	9	1111 q₂		

这是以空纸带作为初始输入的命令调用，

```
$ ./turing-machine.rkt -f machines/pred.tm -c " "
step 0: q0: *B*
step 1: q1: *B*B
step 2: q2: *B*BB
step 3: q3: B*B*B
step 4: HALT
```

以及对应的纸带图序列。

Step	Machine
0	q₀
1	q₁
2	q₂
3	q₃

I.A.2 这模拟了 1 + 2。

```
$ ./turing-machine.rkt -f machines/pred.tm -c "1" -r " 11"
step 0: q0: *1* 11
step 1: q0: 1*B*11
step 2: q1: 1*B*11
step 3: q1: 1*1*11
```

```

step 4: q2: 1*1*11
step 5: q2: *1*111
step 6: q2: *B*1111
step 7: q3: *B*1111
step 8: q3: B*1*111
step 9: q4: B*B*111
step 10: q5: BB*1*11
step 11: HALT

```

这是对应的纸带图。

Step	Machine	Step	Machine	Step	Machine
0		4		8	
1		5		9	
2		6		10	
3		7			

0 + 2 的命令

```
$ ./turing-machine.rkt -f machines/pred.tm -c " " -r "11"
```

给出这些纸带图。

Step	Machine	Step	Machine	Step	Machine
0		3		6	
1		4		7	
2		5		8	

0 + 0 的命令

```
$ ./turing-machine.rkt -f machines/pred.tm -c " "
```

给出了以下纸带图序列。

Step	Machine	Step	Machine	Step	Machine
0		3		6	
1		4		7	
2		5		8	

I.A.3 这个十条指令的机器

$$\mathcal{P}_{\text{addthree}} = \{q_0 1Rq_0, q_0 BBq_1, q_1 B1q_1, q_1 1Rq_2, q_2 B1q_2, q_2 1Rq_3, \\ q_3 B1q_3, q_3 11q_4, q_4 1Lq_4, q_4 BRq_5\}$$

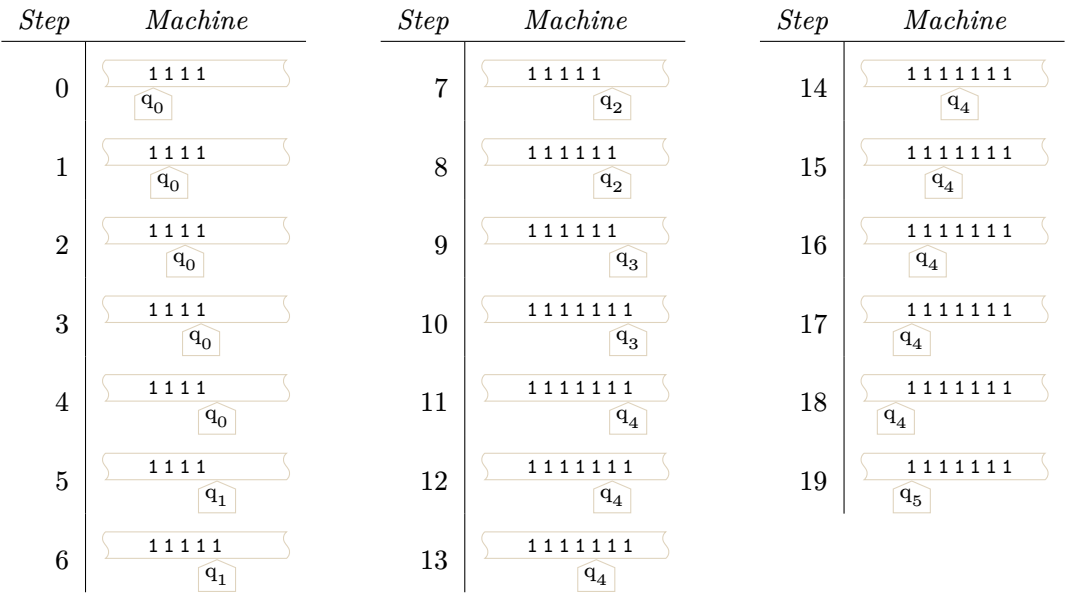
与模拟器的源文件一致。

```
0 1 R 0
0 B B 1
1 B 1 1
1 1 R 2
2 B 1 2
2 1 R 3
3 B 1 3
3 1 1 4
4 B R 5
4 1 L 4
```

输入为 4 的命令

```
$ ./turing-machine.rkt -f machines/addthree.tm -c "1" -r "111"
```

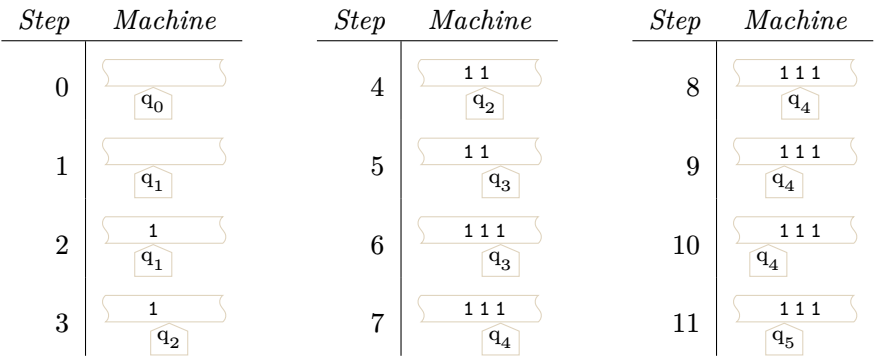
给出了以下纸带图。

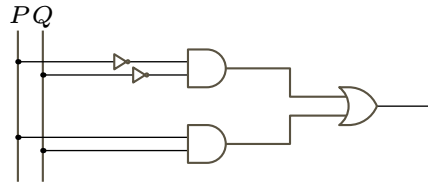


输入为 0 的命令

```
$ ./turing-machine.rkt -f machines/addthree.tm -c " " "
```

给出了以下纸带图序列。



图 2, FOR QUESTION I.B.1: $PXOR Q$ 的回路。

(A) 像下面这样就可以了。

$$\mathcal{P}_0 = \{q_{10}ORq_{10}, q_{10}1Rq_{10}, q_{10}BBq_{100}\}$$

(B) 这会向后移动, 将格子擦除为空白。

$$\mathcal{P}_1 = \{q_{20}0Bq_{21}, q_{20}1Bq_{21}, q_{20}BBq_{100}, q_{21}0Lq_{20}, q_{21}1Lq_{20}, q_{21}BLq_{20}\}$$

(C) 这里, 粗略地说, q_0 表示尚未匹配到子串的任何部分, q_1 表示已看到 0, 机器正在寻找 1, 而 q_2 表示已看到 01, 机器正在寻找 0。接着, 四个状态 q_3 、 q_4 、 q_5 和 q_6 表示失败 (它们以 q_6 在空白纸带上写入 0 而结束)。四个状态 q_7 、 q_8 、 q_9 和 q_{10} 表示成功。

$$\begin{aligned} \mathcal{P}_{\text{decide010}} = \{ & q_0ORq_1, q_01Rq_0, q_0BBq_3, q_1ORq_1, q_11Rq_2, q_1BBq_3, q_2ORq_7, q_211q_0, q_2BBq_3, \\ & q_3ORq_3, q_31Rq_3, q_3BLq_4, q_40Bq_5, q_41Bq_5, q_4BBq_6, \\ & q_50Lq_4, q_51Lq_4, q_5BLq_4, q_600q_{100}, q_610q_{100}, q_6B0q_{100}, \\ & q_7ORq_7, q_71Rq_7, q_7BLq_8, q_80Bq_9, q_81Bq_9, q_8BBq_{10}, \\ & q_90Lq_8, q_91Lq_8, q_9BLq_8, q_{10}01q_{100}, q_{10}11q_{100}, q_{10}B1q_{100}\} \end{aligned}$$

I.B.1

(A) 真值表如下。

P	Q	$PXOR Q$
0	0	0
0	1	1
1	0	1
1	1	0

表达式为 $(\neg P \wedge \neg Q) \vee (P \wedge Q)$ 。

(B) $PXOR Q$ 的回路如图 Figure 2 on page 37 所示。

I.B.2

(A) 根据定义, 输入/输出行为如下:

P	Q	$P \rightarrow Q$
0	0	1
0	1	1
1	0	0
1	1	1

因此, 布尔表达式为 $(\neg P \wedge \neg Q) \vee (\neg P \wedge Q) \vee (P \wedge Q)$ 。

(B) 蕴含运算的回路如图 Figure 3 on page 38 所示。

I.B.3 表达式为 $(\neg P \wedge Q) \vee (P \wedge Q)$ 。回路如图 Figure 4 on page 38 所示。

I.B.4 两个回路如图 Figure 5 on page 38 所示。

(A) 在表达式中, 每个子句对应输出为 1 的一行。

$$(\neg P \wedge \neg Q \wedge R) \vee (\neg P \wedge Q \wedge \neg R) \vee (P \wedge \neg Q \wedge R)$$

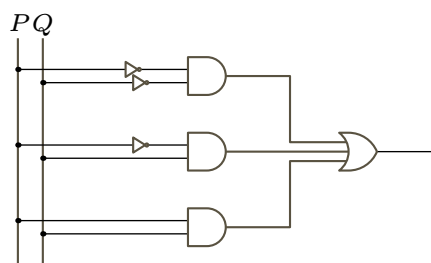


图 3, FOR QUESTION I.B.2: 蕴含运算的回路。

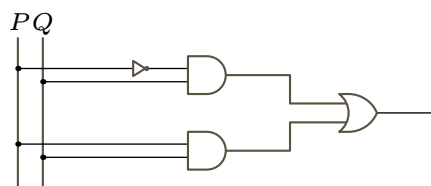


图 4, FOR QUESTION I.B.3: 未命名运算的回路 (不妨称之为 Q)。

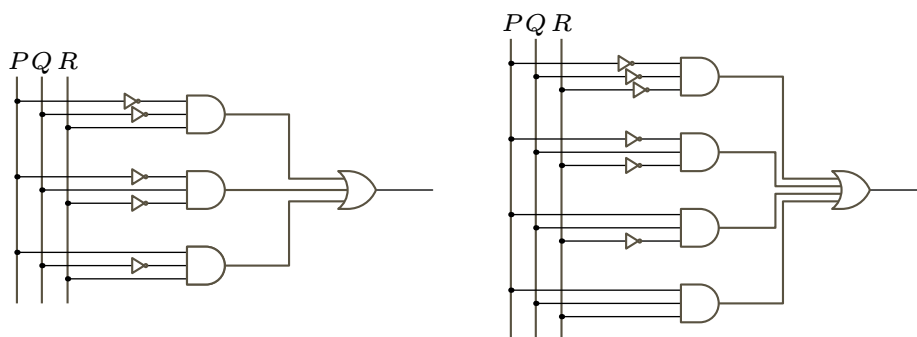
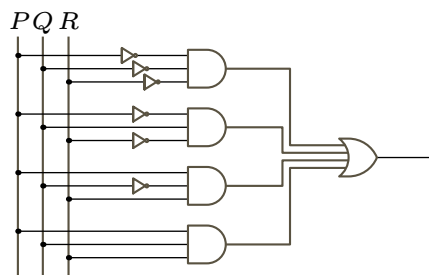


图 5, FOR QUESTION I.B.4: 两个未命名运算表的回路。

图 6, FOR QUESTION I.B.5: $P = R$ 的回路。

(B) 子句对应输出为 1 的行。

$$(\neg P \wedge \neg Q \wedge \neg R) \vee (\neg P \wedge Q \wedge \neg R) \vee (P \wedge Q \wedge \neg R) \vee (P \wedge Q \wedge R).$$

I.B.5 输入/输出行为如下:

P	Q	R	
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

该表直接给出了析取范式 (DNF) 表达式:

$$(\neg P \wedge \neg Q \wedge \neg R) \vee (\neg P \wedge Q \wedge \neg R) \vee (P \wedge \neg Q \wedge R) \vee (P \wedge Q \wedge R)$$

回路如图 Figure 6 on page 39 所示。

I.B.6 当且仅当有两个或三个输入为 1 时, 该行的输出列才为 1。

P	Q	R	
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

这是一个析取范式 (DNF) 表达式:

$$(\neg P \wedge Q \wedge R) \vee (P \wedge \neg Q \wedge R) \vee (P \wedge Q \wedge \neg R) \vee (P \wedge Q \wedge R)$$

回路如图 Figure 7 on page 40 所示。

I.B.7

(A) 当且仅当有三个或四个输入为 1 时, 该行的输出列才为 1。

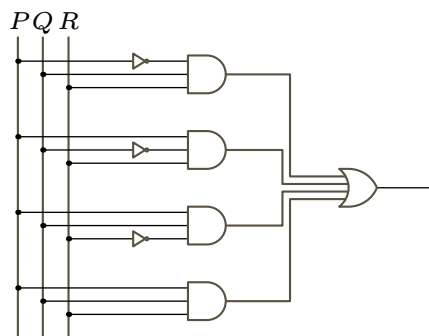


图 7, FOR QUESTION I.B.6: 三输入多数运算符的回路。

P	Q	R	S	
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

此 DNF 表达式包含所有至少有三个变量未被否定的子句。

$$(\neg P \wedge Q \wedge R \wedge S) \vee (P \wedge \neg Q \wedge R \wedge S) \vee (P \wedge Q \wedge \neg R \wedge S) \vee (P \wedge Q \wedge R \wedge \neg S) \vee (P \wedge Q \wedge R \wedge S)$$

- (B) DNF 表达式必须由至少有三个变量未被否定的子句构成。其中，恰好有三个未被否定的变量的子句有 $\binom{5}{3} = 10$ 个，恰好有一个被否定的变量（即四个未被否定的变量）的子句有 $\binom{5}{4} = 5$ 个，没有变量被否定的子句有 $\binom{5}{5} = 1$ 个。

$$\begin{aligned} & (\neg P \wedge \neg Q \wedge R \wedge S \wedge T) \vee (\neg P \wedge Q \wedge \neg R \wedge S \wedge T) \vee (\neg P \wedge Q \wedge R \wedge \neg S \wedge T) \vee (\neg P \wedge Q \wedge R \wedge S \wedge \neg T) \\ & \vee (P \wedge \neg Q \wedge \neg R \wedge S \wedge T) \vee (P \wedge \neg Q \wedge R \wedge \neg S \wedge T) \vee (P \wedge \neg Q \wedge R \wedge S \wedge \neg T) \\ & \vee (P \wedge Q \wedge \neg R \wedge \neg S \wedge T) \vee (P \wedge Q \wedge \neg R \wedge S \wedge \neg T) \vee (P \wedge Q \wedge R \wedge \neg S \wedge \neg T) \\ & \vee (\neg P \wedge Q \wedge R \wedge S \wedge T) \vee (P \wedge \neg Q \wedge R \wedge S \wedge T) \vee (P \wedge Q \wedge \neg R \wedge S \wedge T) \\ & \vee (P \wedge Q \wedge R \wedge \neg S \wedge T) \vee (P \wedge Q \wedge R \wedge S \wedge \neg T) \\ & \vee (P \wedge Q \wedge R \wedge S \wedge T) \end{aligned}$$

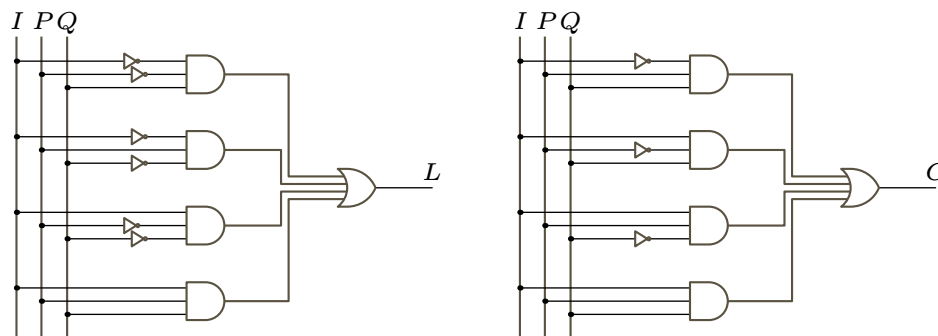


图 8, FOR QUESTION I.B.8: 单列加法的部分和与进位电路。

(A) 计算过程如下表所示。进位出现在从右数的第二、第三和第五列。

	1	1	10	11	1
+	1	0	0	1	
	1	0	1	0	0

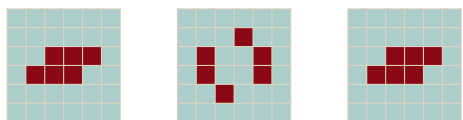
(B) 定义给出了以下两个输入/输出行为。这里所谓的“最低有效位”通常被称为“全加器”。(也常用的是“半加器”，它不考虑进位输入，因此只有两个输入。)

进位输入 I	P	Q	最低有效位 L	进位输出 C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

对于 L ，一个布尔表达式是 $(\neg I \wedge \neg P \wedge Q) \vee (\neg I \wedge P \wedge \neg Q) \vee (I \wedge \neg P \wedge \neg Q) \vee (I \wedge P \wedge Q)$ 。对于 C ，一个析取范式 (DNF) 表达式是 $(\neg I \wedge P \wedge Q) \vee (I \wedge \neg P \wedge Q) \vee (I \wedge P \wedge \neg Q) \vee (I \wedge P \wedge Q)$ 。

(C) 图 Figure 8 on page 41 给出了对应的电路。

I.C.4 澡盆是静止生命。蟾蜍以周期 2 振荡。



I.C.5 不能。例如，在前文关于最简单非平凡模式的讨论中，有两个不同的 3×3 游戏板，它们的下一代都会完全死亡。因此，从一个全死的游戏板，你无法判断它的前身是什么。

此外，由于存在演化到同一后继的不同游戏板对，根据计数原理，必然也存在一些没有前驱的游戏板——即在它之前没有任何游戏板。这样的游戏板被称为“伊甸园”。对于这些游戏板，如果我们试图倒推时间，问题并不在于从导致当前状态的多个可能的前身中选择哪一个——而是根本不存在这样的前身。

I.C.6

(A) 对于 3×3 的网格，每个细胞要么活要么死，所以有 2^9 种网格。对于 $n \times n$ ，是 2^{n^2} 种。

(B) 由于有 $2^9 = 512$ 个不同的 3×3 游戏板，我们借助程序来计算。

```
; Input a natural number less than 2^9, output a grid with associated 0's and 1's
; For instance, n=28 is 11100 in binary so this returns the vector of vectors [[0 0 0] [0 1 1] [
```

```

(define (number->three-by-three-grid n)
  (if (>= n (expt 2 9))
    (error "input too large; must be less than 2^9")
    (let* ([bitstr (number->string n 2)]
           [pad-bitstr (make-string (- 9 (string-length bitstr)) #\0)]
           [length-9-bitstr (string-append pad-bitstr bitstr)]
           [input-style-bitstr (string-replace (string-replace length-9-bitstr "0" ".") "1" "*")]
           [list-of-lines (list (substring input-style-bitstr 0 3)
                                (substring input-style-bitstr 3 6)
                                (substring input-style-bitstr 6 9))])
      (parse-lines-to-grid list-of-lines))))

; Test if a grid is empty
(define (grid-empty? g)
  (let ([size (grid-size g)])
    (grid-equal? g (grid-create (first size) (second size)))))

; Count the number of 3x3 grids that are alive after the specified number of generations
(define (number-alive generations)
  (let ([counter 0])
    (for ([n (in-range (expt 2 9))])
      (displayln (~a "n=" n "\ngrid=" (grid->string (number->three-by-three-grid n)))))
      (let* ([initial-u (universe (number->three-by-three-grid n) (list 0 0))]
             [list-of-universes (run initial-u generations)]
             [ending-grid (universe-grid (last list-of-universes))])
        (when (not (grid-empty? ending-grid))
          (set! counter (add1 counter))
          (displayln " not blank"))))
      (displayln (~a "Number of 3x3 boards surviving is " counter))))

```

运行程序得到如下结果。

```

> (number-alive 1)
... lots of lines ...
n=510
grid=***
***
**
not blank
n=511
grid=***
***
***
not blank
Number of 3x3 boards surviving is 461

```

(c) 同样使用程序得到如下结果。

```

> (number-alive 10)
... lots of lines ...
n=510
grid=***
***
**
not blank
n=511
grid=***
***
***

```

not blank
Number of 3x3 boards surviving is 249

I.D.8 将 2^{65536} 重写为 $(10^{\log_{10} 2})^{65536}$ 。这大约是 $(10^{0.3010299957})^{65536} \approx 10^{19728.30}$ 。所以有 19729 位数字。

I.D.9 这涉及到统一性的问题。想象我们有一个原始递归函数的枚举 f_0, f_1 , 等等。假设第零级是 $\mathcal{A}_0 = f_9$, 第一级是 $\mathcal{A}_1 = f_{121}$, 而 $\mathcal{A}_2 = f_{800}$, 等等。我们得到的是, 将 $\mathcal{A}$ 的下标与 f 的下标关联起来的函数不是原始递归的。也就是说, 在这个设想的情境中, $\mathcal{A}_i = f_{g(i)}$ 但 g 不是原始递归的。

这有点像说蛋糕的所有层都很美味, 但制作蛋糕时, 第 1 层的口味选择完全不顾及第 0 层的口味, 可能是咖啡蛋糕和蜂鸟蛋糕。

I.D.10

(A) 我们可以将此函数表示为复合 $f(y) = \mathcal{S}(\mathcal{S}(y))$ 。后继函数是第 1 级。复合在此基础增加两级, 因此 f 整体是第 3 级。

(B) 我们有

$$\text{pred}(y) = \begin{cases} \mathcal{Z}(x) = 0 & \text{若 } x = 0 \\ \mathcal{J}_0(z) = z & \text{若 } y = \mathcal{S}(z) \end{cases}$$

零函数是第零级, 投影函数也是第零级。原始递归增加三级, 因此前驱函数是第 3 级。

I.D.11 在 $\mathcal{A}$ 的定义中, $k = 0$ 子句的求值显然终止。在其他两个子句中, 要么 k 减小, 要么 k 不减小但 y 减小。当 y 递减到零时, k 就会减小, 因此最终它也会达到零, 而我们已经注意到这将使得求值终止。

I.D.12

(A) 我们将通过对 k 进行归纳来验证该命题 (其“一般情形”可由此直接得出)。

$$\mathcal{A}(k, y+1) > \mathcal{A}(k, y) \quad (*)$$

在 $k = 0$ 的基础步骤中, $\mathcal{A}(0, y+1) = y+2$, 而 $\mathcal{A}(0, y) = y+1$, 所以 $(*)$ 显然成立。

对于归纳步骤, 假设 $(*)$ 对 $k = 0, 1, \dots, n$ 成立, 并考虑 $k = n+1$ 的情形。那么根据 $\mathcal{A}$ 的定义, 我们得到等式 $\mathcal{A}(n+1, y+1) = \mathcal{A}(n, \mathcal{A}(n+1, y)) > \mathcal{A}(n+1, y)$, 而不等式可由引理 D.2 的 A 得出。

(B) 我们将通过对 y 进行归纳来证明该命题。

$$\mathcal{A}(k+1, y) \geq \mathcal{A}(k, y+1) \quad (*)$$

$\mathcal{A}$ 定义的第二条是 $\mathcal{A}(k+1, 0) = \mathcal{A}(k, 1)$, 所以 $y = 0$ 的基础步骤显然成立。

对于归纳步骤, 假设 $(*)$ 对 $y = 0, 1, \dots, n$ 成立。考虑 $y = n+1$ 的情形, 涉及 $\mathcal{A}(k+1, n+1)$ 。 $\mathcal{A}$ 定义的第三条是 $\mathcal{A}(k+1, n+1) = \mathcal{A}(k, \mathcal{A}(k+1, n))$ 。于是 $\mathcal{A}(k+1, n) \geq \mathcal{A}(k, n+1) \geq (n+1)+1$ 成立, 这首先是因为归纳假设, 其次是因为引理 D.2 的 A。由此, 根据前一项的“一般情形”及引理 D.2 的 B, 我们有 $\mathcal{A}(k, \mathcal{A}(k+1, n)) \geq \mathcal{A}(k, (n+1)+1)$, 因此得到 $\mathcal{A}(k+1, n+1) \geq \mathcal{A}(k, (n+1)+1)$, 得证。

(C) 这里要证的命题是 $\mathcal{A}(k, y) > k$ 。反复应用前一项, 即引理 D.2 的 C, 即可证明。

$$\mathcal{A}(k, y) = \mathcal{A}(k-1, y+1) = \mathcal{A}(k-2, y+2) = \dots = \mathcal{A}(0, y+(k+1))$$

那么定义的第一条给出 $\mathcal{A}(0, y+(k+1)) = y+(k+1)$, 它大于 k 。

(D) 对于 $\mathcal{A}(k+1, y) > \mathcal{A}(k, y)$, 由引理 D.2 的 C 出发, 然后应用引理 D.2 的 B。

$$\mathcal{A}(k+1, y) \geq \mathcal{A}(k, y+1) > \mathcal{A}(k, y)$$

“一般情形”通过对上述过程进行迭代可得。

(E) 这是要证的命题。

$$\mathcal{A}(k+2, y) > \mathcal{A}(k, 2y) \quad (*)$$

我们将对 y 进行归纳。对于 $y = 0$ 的基础步骤，我们使用前一项的“一般情形”部分，即引理 D.2 的 E，来得到 $\mathcal{A}(k+2, y) = \mathcal{A}(k+2, 0) > \mathcal{A}(k, 0) = \mathcal{A}(k, 2y)$ 。

对于归纳步骤，假设 $(*)$ 对 $y = 0, \dots, n$ 成立，并考虑 $y = n+1$ 的情形。 $\mathcal{A}$ 定义中的第三条是 $\mathcal{A}(k+2, n+1) = \mathcal{A}(k+1, \mathcal{A}(k+2, n))$ 。归纳假设是 $\mathcal{A}(k+2, n) > \mathcal{A}(k, 2n)$ ，于是由引理 D.2 的 B 的“一般情形”可得此式。

$$\mathcal{A}(k+2, n+1) = \mathcal{A}(k+1, \mathcal{A}(k+2, n)) > \mathcal{A}(k+1, \mathcal{A}(k, 2n))$$

将引理 D.2 的 C 应用于该式。

$$\mathcal{A}(k+2, n+1) > \mathcal{A}(k+1, \mathcal{A}(k, 2n)) \geq \mathcal{A}(k, \mathcal{A}(k, 2n) + 1)$$

现在，根据引理 D.2 的 A， $\mathcal{A}(k, 2n) \geq 2n+1$ ，因此 $\mathcal{A}(k, 2n) + 1 \geq 2n+2 = 2(n+1)$ 。于是 $\mathcal{A}(k+2, n+1) > \mathcal{A}(k, 2(n+1))$ ，符合要求。

I.E.2 代码如下。

```
# swap-two.loop  Input x,y and output y,x
r2 = r0
r0 = r1
r1 = r2
```

这是一个示例调用。

```
jim@millstone:src/scheme/prologue$ ./loop.rkt -f machines/swap-two.loop -p "1 2 3" -o 2
2 1
```

I.E.3 我们可以用连续两次递增来实现第一种的效果，而第二种的效果可以通过先清零再递增来实现。

I.E.4 左旋转代码是对右旋转代码的改编。

```
# rotate-shift-left.loop  input x,y,z, output y,z,x
r3 = r0
r0 = r1
r1 = r2
r2 = r3
```

这是一个示例调用。

```
jim@millstone:src/scheme/prologue$ ./loop.rkt -f machines/rotate-shift-left.loop -p "1 2 3" -o 3
2 3 1
```

将两者组合起来得到如下程序。

```
# rotate-shift-right
r3 = r2
r2 = r1
r1 = r0
r0 = r3
# now rotate-shift-left
r3 = r0
r0 = r1
r1 = r2
r2 = r3
```

这是一个示例调用。

```
jim@millstone:src/scheme/prologue$ ./loop.rkt -f machines/concatenate-rotations.loop -p "1 2 3" -o 3
1 2 3
```

I.E.5 下面是一个程序。

```
# power.loop Return  $r0 \wedge r1$ 
r2 = r1 # save the inputs in higher registers
r1 = r0 #
r0 = 0
r0 = r0 + 1 # cover the r1=0 case
r3 = 0 # Initialize
r3 = r3 + 1 # to 1
loop r2
  r0 = 0
  loop r3
    loop r1
      r0 = r0 + 1
    end
  end
  r3 = r0
end
```

这里是两个示例调用。

```
jim@millstone:src/scheme/prologue$ ./loop.rkt -f machines/power.loop -p "3 2"
9
jim@millstone:src/scheme/prologue$ ./loop.rkt -f machines/power.loop -p "3 0"
1
```

I.E.6 第一个程序是这样的；请注意它运行了五次。

```
> (for ([i (in-range 5)])
      (set! i 1)
      (displayln (~a "i=" i)))
i=1
i=1
i=1
i=1
i=1
```

第二个程序是这样的。

```
> (for ([i (in-range 5)])
      (displayln (~a "i=" i))
      (set! i 1))
i=0
i=1
i=2
i=3
i=4
```


Chapter II: 背景

II.1.20 要证明这个映射是单射的,我们证明 $g(n) = g(\hat{n})$ 仅当 $n = \hat{n}$ 时才可能发生。假设 $g(n) = g(\hat{n})$, 则有 $3n = 3\hat{n}$ 。两边除以 3 得到 $n = \hat{n}$ 。

要证明这个映射是满射的,我们证明陪域中的每个元素 $y \in \{3k \mid k \in \mathbb{N}\}$ 都是定义域中某个元素的像。因为 y 是 3 的倍数, $y/3$ 是一个自然数。由于 $y = g(y/3)$, 因此 y 确实是定义域中一个元素的像。

II.1.21 这是个类型错误。不是集合具有单射性或满射性;而是集合间的对应关系 f 具有这些性质。对于 $D = \{n^2 \mid n \in \mathbb{N}\}$ 与 $C = \{n^3 \mid n \in \mathbb{N}\}$, 这种自然的对应关系 $f: D \rightarrow C$ 是 $f(n) = n^{3/2}$ 。

II.1.22

- (A) 由 $f(x) = 2x$ 给出的函数 $f: \mathbb{Z} \rightarrow \mathbb{Z}$ 是单射的但不是满射的。验证它是单射的很简单: 假设对 $x, \hat{x} \in \mathbb{Z}$ 有 $f(x) = f(\hat{x})$, 即 $2x = 2\hat{x}$, 然后两边约去 2 得到 $x = \hat{x}$ 。要证明它不是满射的, 只需指出一个陪域元素 $y \in \mathbb{Z}$, 它不是任何定义域元素的像。例如 $y = 1$, 因为所有定义域元素的像都是偶数。
- (B) 由 $g(x) = 2x - 1$ 给出的函数 $g: \mathbb{Z} \rightarrow \mathbb{Z}$ 也是单射的但不是满射的。单射性的论证与前一项类似: 假设对 $x, \hat{x} \in \mathbb{Z}$ 有 $g(x) = g(\hat{x})$, 即 $2x - 1 = 2\hat{x} - 1$ 。两边加上 1, 再约去 2, 得到 $x = \hat{x}$ 。这个函数不是满射的, 因为陪域元素 $y = 2$ 不是任何定义域元素的像—— $y = 2$ 是偶数, 但对任意输入整数 x , $2x - 1$ 都是奇数。
- (C) 它们不互为逆函数。例如, $f \circ g(1) = f(g(1)) = f(1) = 2$ 。

II.1.23

- (A) 函数 $f: \mathbb{N} \rightarrow \mathbb{N}$ (定义为 $f(n) = n + 1$) 是单射的但不是满射的。验证单射性: 假定 $n_0, n_1 \in \mathbb{N}$ 满足 $f(n_0) = f(n_1)$, 从而 $n_0 + 1 = n_1 + 1$, 然后两边减去 1 得到 $n_0 = n_1$ 。它不是满射的, 因为陪域元素 $0 \in \mathbb{N}$ 不是任何定义域元素的像。
- (B) 映射 $f: \mathbb{Z} \rightarrow \mathbb{Z}$ (定义为 $f(n) = n + 1$) 是一个双射——它既单射又满射。单射性验证: 假定 $f(n_0) = f(n_1)$, 从而 $n_0 + 1 = n_1 + 1$, 减去 1 得到 $n_0 = n_1$ 。满射性验证: 固定某个陪域元素 $y \in \mathbb{Z}$, 注意定义域元素 $x = y - 1$ 满足 $f(x) = y$ 。
- (C) 函数 $f: \mathbb{N} \rightarrow \mathbb{N}$ (定义为 $f(n) = 2n$) 是单射的但不是满射的。对于单射性, 假定 $n_0, n_1 \in \mathbb{N}$ 满足 $f(n_0) = f(n_1)$ 。于是 $2n_0 = 2n_1$, 消去 2 得到 $n_0 = n_1$ 。它不是满射的, 因为陪域元素 $y = 1$ 不是任何定义域元素的像——所有的像都是偶数。
- (D) 函数 $f: \mathbb{Z} \rightarrow \mathbb{Z}$ (定义为 $f(n) = 2n$) 是单射的但不是满射的。单射性验证与上一项类似: 假定对于某些 $n_0, n_1 \in \mathbb{Z}$ 有 $f(n_0) = f(n_1)$, 则 $2n_0 = 2n_1$, 消去 2 得到 $n_0 = n_1$ 。同样如上一项, 此映射不是满射的, 因为陪域元素 $y = 1$ 不在函数的值域内——所有值域元素都是偶数。
- (E) 函数 $f: \mathbb{Z} \rightarrow \mathbb{N}$ (定义为 $f(n) = |n|$) 不是单射的, 但是满射的。它不是单射的, 因为两个定义域元素 $n_0 = -1$ 和 $n_1 = 1$ 映射到相同的输出: $f(n_0) = 1 = f(n_1)$ 。为证明 f 是满射映射, 考虑陪域元素 $y \in \mathbb{N}$, 并注意到若取定义域元素 $x \in \mathbb{Z}$ 使得 $x = y$, 那么有 $f(x) = y$ 。

II.1.24

- (A) 这个函数是一个双射。要验证它是单射的, 假定 $f(q_0) = f(q_1)$, 其中 $q_0, q_1 \in \mathbb{Q}$, 从而 $q_0 + 3 = q_1 + 3$, 两边减去 3 得到 $q_0 = q_1$ 。对于满射性, 考虑陪域元素 $y \in \mathbb{Q}$, 注意定义域元素 $q \in \mathbb{Q}$ (令 $q = y - 3$) 满足 $f(q) = y$ 。
- (B) 这个函数不是一个双射。它不是满射的, 因为陪域元素 $1/2 \in \mathbb{Q}$ 不是任何一个定义域元素 $z \in \mathbb{Z}$ 的像——每个这样的像 $f(z) = z + 3$ 都是整数。(这个函数是单射的: 假定 $f(z_0) = f(z_1)$, 从而 $z_0 + 3 = z_1 + 3$, 两边减去 3 得到 $z_0 = z_1$ 。但由于它不是满射的, 所以它不是双射。)
- (C) 这不是一个双射。它甚至不是一个函数, 因为它不是良定义的——两个有理数 $q_0 = 1/2$ 和 $q_1 = 2/4$ 相等, 即 $q_0 = q_1$, 但它们对应的 $|a \cdot b|$ 值不同, 因为 $|1 \cdot 2| = 2$ 而 $|2 \cdot 4| = 8$ 。

II.1.25

- (A) 这个集合是有限的。它与 $I = \{0, 1, 2\}$ 有相同的势，由函数 $f: I \rightarrow \{1, 2, 3\}$ (定义为 $f(x) = x + 1$) 见证，通过观察可知该映射既单射又满射。
- (B) 由完全平方数组成的集合 $S = \{0, 1, 4, 9, 16, \dots\}$ 是无限的。不存在 $I = \{0, 1, \dots, n-1\}$ 和 S 之间的对应，因为不存在满射函数——对于任何函数 $f: I \rightarrow S$ ，集合 $\{f(0), f(1), \dots, f(n-1)\}$ 不可能等于 S ，因为它有一个最大元素，而 S 没有最大元素。
- (C) 素数集合是无限的。可采用与上一项相同的论证。
- (D) 根据代数基本定理，一个五次多项式至多有五个实数根。因此这个集合的势至多为五，所以是有限的。

II.1.26

- (A) 由 $f(0) = 3, f(1) = 4, f(2) = 5$ 给出的映射 $f: \{0, 1, 2\} \rightarrow \{3, 4, 5\}$ ，即 $f(x) = x + 3$ ，是一个双射。经检查，陪域中的每个元素都与定义域中唯一一个元素相关联。
- (B) 函数 $f(x) = x^3$ 是这两个集合之间的一个双射。对于每个完全立方数，都有且仅有一个与之关联的整数立方根。

II.1.27

- (A) 由 $0 \mapsto \pi, 1 \mapsto \pi + 1, 3 \mapsto \pi + 2, 7 \mapsto \pi + 3$ 给出的函数 f ，经检查既是单射的又是满射的。
- (B) 令 $E = \{2n \mid n \in \mathbb{N}\}$ 且 $S = \{n^2 \mid n \in \mathbb{N}\}$ 。考虑由 $f(x) = (x/2)^2$ 给出的映射 $f: E \rightarrow S$ 。为证明 f 是单射的，假设 $f(x_0) = f(x_1)$ 。则 $(x_0/2)^2 = (x_1/2)^2$ ，且由于这些是自然数，它们的平方根相等，即 $x_0/2 = x_1/2$ 。因此 $x_0 = x_1$ ，该映射是单射的。
为证明满射性，任取 $y \in S$ 。因为 y 是完全平方数，所以存在自然数 $n \in \mathbb{N}$ 使得 $y = n^2$ 。令 $x = 2n$ 。显然 x 是偶数，且 $f(x) = y$ 。
- (C) 第二个区间的宽度是第一个的 $2/3$ ，因此考虑函数 $f(x) = (2/3) \cdot x - (5/3)$ 。它是单射的，因为 $f(x_0) = f(x_1)$ 意味着 $(2/3) \cdot x_0 - (5/3) = (2/3) \cdot x_1 - (5/3)$ ，经简单代数运算可得 $x_0 = x_1$ 。为证明 f 是满射的，固定 $y \in (-1..1)$ 。令 $x = (y + (5/3)) / (2/3)$ (这来自于求解方程 $y = (2/3)x - (5/3)$ 中的 x)。当 y 属于陪域 $(-1..1)$ 时，此 x 便属于定义域 $(1..4)$ ，且进一步的代数运算表明 $f(x) = y$ 。

II.1.28 题目没有明确说明哪个集合是定义域、哪个是陪域。我们将证明由 $f(x) = 1/x$ 给出的 $f: (0..1) \rightarrow (1..\infty)$ 既是单射的又是满射的。

为证单射性，假设 $f(x_0) = f(x_1)$ ，其中 $x_0, x_1 \in (0..1)$ 。则 $1/x_0 = 1/x_1$ ，交叉相乘得 $x_1 = x_0$ 。
为证满射性，设 $y \in (1..\infty)$ ，并注意到若取 $x = 1/y$ ，则 $f(x) = y$ 且 $x \in (0..1)$ 。

II.1.29 记这两个集合为 $D = \{1, 2, 3, 4, \dots\}$ 和 $C = \{7, 10, 13, 16, \dots\}$ 。我们将证明由公式 $f(x) = 3(x-1) + 7$ 所描述的映射 $f: D \rightarrow C$ 给出了这两个集合之间的一个双射。该函数是单射的，因为由 $f(x_0) = f(x_1)$ 可推出 $3(x_0-1) + 7 = 3(x_1-1) + 7$ ，然后减去 7、除以 3、再加 1 便得 $x_0 = x_1$ 。它是满射的，因为若 y 是陪域 C 中的元素，则它具有形式 $y = 3n + 4$ (其中 $n \in \mathbb{N}^+$)。该等式可以写成 $y = 3(n-1) + 7$ 。经代数运算可得，对于 $n \in \mathbb{N}^+$ 有 $(y-7)/3 = n-1$ ，且 $n = ((y-7)/3) + 1$ 是 $\mathbb{N}^+ = D$ 中的元素，满足 $f(n) = y$ 。

II.1.30

- (A) 显而易见的映射如下。

x	0	1	2	3	4	5	6	7	8	9
$f(x)$	48	49	50	51	52	53	54	55	56	57

通过观察可知它是单射的，即检查上述对应关系可以发现，绝不会有二个不同的定义域元素映射到同一个陪域元素。类似地，通过观察可知它也是满射的。

- (B) 逆映射关联的元素对与前一项表格中相同，但定义域和陪域互换，因此这里 A 是定义域， C 是陪域。

y	48	49	50	51	52	53	54	55	56	57
x	0	1	2	3	4	5	6	7	8	9

如前一项，通过观察可知该映射既是单射的又是满射的。

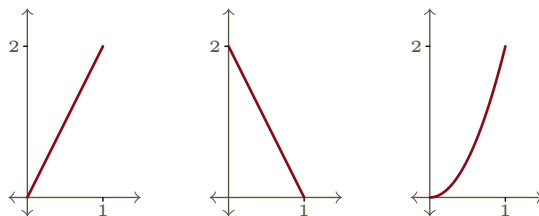


图 9, FOR QUESTION II.1.32: 对应 $f_0, f_1, f_2: (0..1) \rightarrow (0..2)$ 由 $f_0(x) = 2x$ 、 $f_1(x) = 2 - 2x$ 和 $f_2(x) = 2x^2$ 给出。

II.1.31 对于每对集合，我们可以定义一个以第一个集合为定义域的函数，或者定义一个以第二个集合为定义域的函数。我们选择较方便的方式。

- (A) 将偶数集写成 $E = \{2n \mid n \in \mathbb{N}\}$ 。考虑函数 $f: \mathbb{N} \rightarrow E$ ，定义为 $f(m) = 2m$ 。为验证 f 是单射的，假设两个输入产生相同的输出 $f(m) = f(\hat{m})$ ，即 $2m = 2\hat{m}$ ，两端除以 2 便得两个输入相同， $m = \hat{m}$ 。

为验证 f 是满射的，取陪域中的元素 $y \in E$ ，则 $y = 2n$ （其中 $n \in \mathbb{N}$ ）。此时 $n = y/2$ 满足 $f(n) = y$ 。注意 n 属于定义域 $\mathbb{N}$ ，因为 y 作为陪域中的元素是偶数，除以二后得到的仍是自然数。

- (B) 将奇数集写成 $M = \{2n + 1 \mid n \in \mathbb{N}\}$ 。考虑 $g: \mathbb{N} \rightarrow M$ ，定义为 $n \mapsto 2n + 1$ 。为验证 g 是单射的，假设 $g(m_0) = g(m_1)$ ，则 $2m_0 + 1 = 2m_1 + 1$ ，两端减 1 再除以 2，得 $m_0 = m_1$ 。

为验证 g 是满射的，取陪域中的元素 $y \in M$ 。则它具有形式 $y = 2k + 1$ （其中 k 为某个自然数）。这意味着存在自然数 k 在 g 下映射到 y ，即 $k = (y - 1)/2$ 。

- (C) 一种方法是引用前面的两个小题，并利用一个双射的逆也是一个双射，从而得出复合 $g \circ f^{-1}$ 是从偶数集到奇数集的一个双射。

另一种直接的方法是考虑映射 $p: E \rightarrow M$ ，定义为 $p(n) = n + 1$ 。按照前两项的方法验证 p 是单射且满射的。

II.1.32 自然的对应 $f_0: (0..1) \rightarrow (0..2)$ 是 $f_0(x) = 2x$ 。另外两个对应有多种选择，其中两个对应 $f_0, f_1, f_2: (0..1) \rightarrow (0..2)$ 为 $f_1(x) = 2 - 2x$ 和 $f_2(x) = 2x^2$ 。Figure 9 on page 49 展示了这三个函数的图像。它们是不同的函数，因为它们在输入 $x = 1/3$ 处返回不同的值。

每个函数都是单射且满射的，因为图像显示，每条过 $y \in (0..2)$ 的水平线与其图像恰交于一点。

为了用代数方法验证它们是双射，可以 f_1 为例。它是单射的，因为若 $f(x_0) = f(x_1)$ 则 $2 - 2x_0 = 2 - 2x_1$ ，然后减去 2 并除以 -2 得 $x_0 = x_1$ 。它是满射的，因为若 $y \in (0..2)$ ，则数 $x = (y - 2)/(-2)$ 是 $(0..1)$ 中的元素，且满足 $f(x) = 2 - 2 \cdot ((y - 2)/(-2)) = y$ 。

II.1.33 函数

$$\hat{f}(n) = \begin{cases} 1 & \text{若 } n = 0 \\ 0 & \text{若 } n = 1 \\ n^2 & \text{若 } n > 1 \end{cases}$$

与例 1.8 中的函数 f 不同，因为它们在输入 0 时给出不同的输出。 $\hat{f}$ 是满射的，这一点很清楚，因为每个完全平方数都是某个输入对应的输出。 $\hat{f}$ 是单射的，这一点也很清楚，尽管写出证明稍显繁琐。假设 $\hat{f}(x_0) = \hat{f}(x_1)$ ，则有四种情况： x_0 和 x_1 都属于集合 $\{0, 1\}$ ，只有第一个属于，只有第二个属于，或者两者都不属于。四种情况都很容易处理。

II.1.34 为验证 f 是单射的，假设 $f(x_0) = f(x_1)$ ，于是有 $c^{x_0} = c^{x_1}$ 。两边取以 c 为底的对数，得 $x_0 = x_1$ 。

为验证满射性，考虑陪域中的元素 $y \in (0.. \infty)$ 。数 $x = \log_c(y)$ 有定义且是 f 的定义域 $\mathbb{R}$ 中的元素。显然， $f(x) = f(\log_c(y)) = c^{\log_c(y)} = y$ 。

II.1.35 两个集合 $P_2 = \{2^k \mid k \in \mathbb{N}\} = \{1, 2, 4, 8, \dots\}$ 和 $P_3 = \{3^k \mid k \in \mathbb{N}\} = \{1, 3, 9, \dots\}$ 都是 $\mathbb{N}$ 的子集。自然的对应 $g: P_2 \rightarrow P_3$ 是将具有相同指数的元素关联起来，即 $g(2^k) = 3^k$ 。这个函数是单射

的, 因为如果 $g(2^{k_0}) = g(2^{k_1})$ 那么 $3^{k_0} = 3^{k_1}$, 两边取以 3 为底的对数表明输入相同, 即 $k_0 = k_1$ 。这个函数是满射的, 因为如果 $y \in P_3$, 则其形式为 $y = 3^k$, 并且它是 P_2 中 2^k 的像。

显而易见的推广是: 如果两个自然数 $a, b \in \mathbb{N}$ 都大于 1, 那么 $P_a = \{a^k \mid k \in \mathbb{N}\}$ 和 $P_b = \{b^k \mid k \in \mathbb{N}\}$ 是等势的自然数集合。论证与上一段相同。

II.1.36 当然, 对于每一项都有许多可能的答案。

(A) 一个是 $f_0: \mathbb{N} \rightarrow \mathbb{N}$, 定义为 $f_0(n) = n+1$ 。它是单射的, 因为如果 $f_0(n) = f_0(\hat{n})$, 那么 $n+1 = \hat{n}+1$, 从而 $n = \hat{n}$ 。它不是满射的, 因为没有 $n \in \mathbb{N}$ 映射到 0。

第二个是 $f_1: \mathbb{N} \rightarrow \mathbb{N}$, 定义为 $f_1(n) = n^2$ 。它是单射的, 因为如果 $f_1(n) = f_1(\hat{n})$, 那么 $n^2 = \hat{n}^2$, 并且由于这些是自然数, 因而是非负的, 所以 $n = \hat{n}$ 。这个函数不是满射的, 因为没有自然数映射到 2。

(B) 函数

$$f_0(x) = \begin{cases} x/2 & \text{若 } x \text{ 为偶数} \\ (x-1)/2 & \text{若 } x \text{ 为奇数} \end{cases} \quad f_0(x) \begin{array}{c|cccccc} x & 0 & 1 & 2 & 3 & 4 & 5 & \dots \\ \hline f_0(x) & 0 & 0 & 1 & 1 & 2 & 2 & \dots \end{array}$$

是满射的, 因为每个 $y \in \mathbb{N}$ 都是 $x = 2y$ 的像。它不是单射的, 因为 $f_0(1) = f_0(0)$ 。

第二个是 $f_1(x) = \lfloor \sqrt{x} \rfloor$ (回顾向下取整运算 $\lfloor n \rfloor$ 给出小于或等于 n 的最大整数)。

$$f_1(x) \begin{array}{c|cccccccccccc} x & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & \dots \\ \hline f_1(x) & 0 & 1 & 1 & 1 & 2 & 2 & 2 & 2 & 2 & 3 & 3 & \dots \end{array}$$

它是满射的, 因为每个 $y \in \mathbb{N}$ 都是 $x = y^2$ 的像。它不是单射的, 因为 $f_1(1) = f_1(2)$ 。

(C) 一个这样的映射是 $f_0(x) = 0$ 。它不是单射的, 因为 $f_0(0) = f_0(1)$ 。它不是满射的, 因为没有自然数输入映射到 1。

一个非常数的此类函数是前一项第二个函数的平方, $f_1(x) = (\lfloor \sqrt{x} \rfloor)^2$ 。

$$f_1(x) \begin{array}{c|cccccccccccc} x & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & \dots \\ \hline f_1(x) & 0 & 1 & 1 & 1 & 4 & 4 & 4 & 4 & 4 & 9 & 9 & \dots \end{array}$$

它不是单射的, 因为 $f_1(1) = f_1(2)$ 。它不是满射的, 因为 2 不是平方数, 所以没有数映射到 2。

(D) 既单射又满射的自然函数是恒等函数 $f_0(x) = x$ 。它是单射的, 因为如果 $f_0(n) = f_0(\hat{n})$, 那么 $n = \hat{n}$, 直接由 f_0 的定义可得。它是满射的, 因为任何陪域元素 $y \in \mathbb{N}$ 都是其自身在 f_0 下的像, 即 $f_0(y) = y$ 。

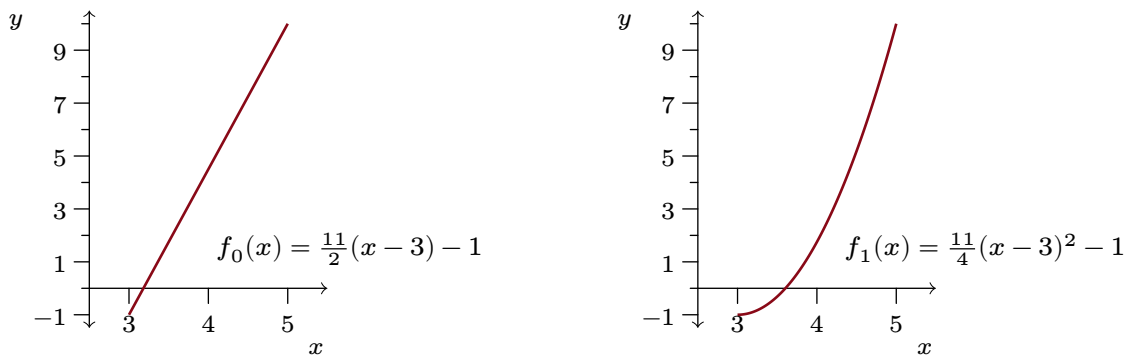
另一个双射交换偶数和奇数。

$$f_1(x) = \begin{cases} x+1 & \text{若 } x \text{ 为偶数} \\ x-1 & \text{若 } x \text{ 为奇数} \end{cases} \quad f_1(x) \begin{array}{c|cccccc} x & 0 & 1 & 2 & 3 & 4 & 5 & \dots \\ \hline f_1(x) & 1 & 0 & 3 & 2 & 5 & 4 & \dots \end{array}$$

首先注意 $f_1: \mathbb{N} \rightarrow \mathbb{N}$; 即, 如果 $x \in \mathbb{N}$, 那么 $f_1(x) \in \mathbb{N}$ 。为验证 f_1 是单射的, 假设 $f_1(n) = f_1(\hat{n})$ 。有两种情况。如果 n 是奇数, 则 $f_1(n) = n-1$ 是偶数, 因此 $f_1(\hat{n})$ 也是偶数, 这意味着 $\hat{n}$ 是奇数, 从而 $n-1 = \hat{n}-1$, 得 $n = \hat{n}$ 。偶数情况类似。为验证 f_1 是满射的, 考虑陪域中的点 $y \in \mathbb{N}$ 。这里也有两种情况, 我们只做奇数情况, 假设 y 是奇数。那么 $y = f_1(x)$, 其中 $x = y-1$, 且 x 是定义域 $\mathbb{N}$ 中的元素, 因为如果 y 是奇数, 则 $y-1 \geq 0$ 。

II.1.37 令 $D = (3..5)$, $C = (-1..10)$ 。

一种对应是 $f_0(x) = (11/2)(x-3) - 1$ 。 $f_0: D \rightarrow C$ 这一点不是显而易见的, 因此我们先验证它: 若 $3 < x < 5$, 则各项减去 3, 乘以 11/2, 再减去 1, 得 $-1 < f_0(x) < 10$ 。接着, 我们证明这个映射是满射的。如果 $y \in C$, 那么 $y = f_0(x)$, 其中 $x = (2/11)(y+1) + 3$ 。要验证这个输入 x 属于定义域 D , 从 $-1 < y < 10$ 出发, 各项加 1, 乘以 2/11, 再加 3, 最终得 $3 < x < 5$ 。最后, 我们证明映射 f_0 是单射的。如果 $f_0(x) = f_0(\hat{x})$, 那么 $(11/2)(x-3) - 1 = (11/2)(\hat{x}-3) - 1$, 两边加 1, 乘以 2/11, 再加 3, 便得 $x = \hat{x}$ 。

图 10, FOR QUESTION II.1.37: $(3..5)$ 与 $(-1..11)$ 之间的两个对应。

另一种对应是 $f_1(x) = (11/4)(x-3)^2 - 1$ 。与 f_0 类似, 我们首先验证 $f_1: D \rightarrow C$: 若 $3 < x < 5$, 则减去 3, 取平方, 乘以 $11/4$, 再从各项减去 1, 得 $-1 < f_1(x) < 10$ 。为证明 f_1 是满射的, 假设 $y \in C$ 。那么 $y = f_1(x)$, 其中 $x = \sqrt{(4/11)(y+1)} + 3$ 。为验证 $x \in D$, 从 $-1 < y < 10$ 出发, 各项加 1, 乘以 $4/11$, 开平方 (这不会改变不等号的方向, 因为所有数都是非负的), 再加 3, 最终得 $3 < x < 5$ 。为证明此映射是单射的, 假设 $f_1(x) = f_1(\hat{x})$, 从而有 $(11/4)(x-3)^2 - 1 = (11/4)(\hat{x}-3)^2 - 1$ 。两边加 1, 乘以 $4/11$, 开平方 (这一操作是确定的, 因为所有的 x 都大于零), 再加 3, 便得 $x = \hat{x}$ 。

Figure 10 on page 51 表明这两个函数不相同, 同时也提供了另一种论证方式来证明每个函数都是双射: 对于每个函数, 对于每个 $y \in (-1..10)$, 水平线 y 都与图像恰好交于一点。

II.1.38 在每种情况下, 我们都构造一个从一个集合到另一个集合的函数, 并验证它既是单射的又是满射的。对于第三小题, 请注意: 一个集合在问题中先出现, 并不意味着它必须作为对应中的定义域。

(A) 令 $S_4 = \{4k \mid k \in \mathbb{N}\}$ 且 $S_5 = \{5k \mid k \in \mathbb{N}\}$, 令 $f: S_4 \rightarrow S_5$ 为 $f(x) = (5/4)x$ 。它是单射的, 因为 $f(x_0) = f(x_1)$ 意味着 $(5/4)x_0 = (5/4)x_1$, 所以 $x_0 = x_1$ 。它是满射的, 因为对于陪域中的任意元素 $y = 5k$ (其中 $k \in \mathbb{N}$), 显然有 $y = f(4k)$ 。

(B) 这两个集合是同一个集合, 即它们包含相同的数。它们通过恒等函数构成对应。

(C) 记 $T = \{0, 1, 3, 6, \dots\} = \{n(n+1)/2 \mid n \in \mathbb{N}\}$ (这些数被称为三角数)。考虑函数 $f: \mathbb{N} \rightarrow T$, 其定义为 $f(n) = n(n+1)/2$ 。该映射是单射的, 因为连续函数 $\hat{f}: \mathbb{R} \rightarrow \mathbb{R}$ (由 $\hat{f}(x) = x(x+1)/2$ 给出) 是根为 0 和 -1 的抛物线, 因此在 $x \geq 0$ 的正数部分, 水平线 y 最多与图像交于一点。 $\hat{f}$ 是单射的这一事实意味着它的限制 f 也是单射的。由定义可知, 函数 f 是满射的, 因为 T 被定义为该函数的值域。

II.1.39

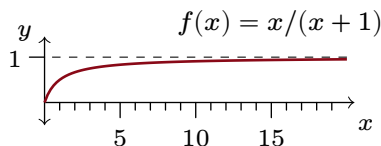
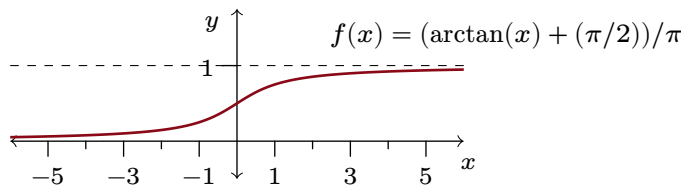
(A) 一个例子是 $f(x) = 2x$ 。这个映射是单射的, 因为如果 $f(x) = f(\hat{x})$, 那么 $2x = 2\hat{x}$, 从而 $x = \hat{x}$ 。它是满射的, 因为对于任意 $0 < y < 2$, 取 $x = y/2$, 则有 $f(x) = y$ 且 $0 < x < 1$ 。

(B) 一个例子是 $f(x) = (b-a) \cdot x + a$ 。注意, $f: (0..1) \rightarrow (a..b)$, 因为如果 $0 < x < 1$, 那么乘以 $b-a$ 得到 $0 < x(b-a) < (b-a)$ (由于 $a < b$, 不等号方向不变), 再加上 a 即得 $a < x(b-a) + a < b$ 。进一步, 这个函数是单射的: 因为 $b-a \neq 0$, 该直线斜率非零, 故通过水平线检验 (如需代数论证, 从 $f(x) = f(\hat{x})$ 出发, 减去 a 再除以 $b-a$, 可得 $x = \hat{x}$)。最后, 证明它是满射的。固定 $y \in (a..b)$ 。注意, 取 $x = (y-a)/(b-a)$ 则 $f(x) = y$ 。剩下的就是验证 $x \in (0..1)$: 从 $a < y < b$ 出发, 减去 a 再除以 $b-a$, 得到 $0 < (y-a)/(b-a) < 1$ (如前所述, 由于 $b-a > 0$, 不等号方向不变)。

(C) 一个例子是 $f(x) = x + a$ 。该函数是单射的, 因为 $f(x) = f(\hat{x})$ 意味着 $x + a = \hat{x} + a$, 从而 $x = \hat{x}$ 。它是满射的, 因为如果 $y \in (a.. \infty)$, 那么令 $x = y - a$ (且 $x \in (0.. \infty)$), 则有 $y = f(x)$ 。

(D) 由相似三角形得到 $x/(x+1) = f(x)/1$ 。Figure 11 on page 52 表明这是一个双射, 因为对于每个 $y \in (0..1)$, 过该高度的水平线与函数图像恰好交于一点。

II.1.40 考虑函数 $f: \mathbb{N} \rightarrow S$, 由 $f(m) = m+2\sqrt{2} = 2^{1/(m+2)}$ 给出。它是单射的, 因为如果 $f(m) = f(\hat{m})$, 则 $2^{1/(m+2)} = 2^{1/(\hat{m}+2)}$ 。对两边取对数 $\lg(2^{1/(m+2)}) = \lg(2^{1/(\hat{m}+2)})$, 得到 $(1/(m+2)) \cdot \lg(2) = (1/(\hat{m}+2)) \cdot \lg(2)$, 于是 $1/(m+2) = 1/(\hat{m}+2)$ 。因此 $m = \hat{m}$ 。

图 11, FOR QUESTION II.1.39: $(0.. \infty)$ 与 $(0..1)$ 之间的一个双射。图 12, FOR QUESTION II.1.42: $\mathbb{R}$ 与 $(0..1)$ 之间的一个双射。

该函数是满射的, 因为如果 $y \in S$, 则存在 $n \in \mathbb{N}$ 且 $n \geq 2$ 使得 $y = 2^{1/n}$ 。因此, 对某个 $x \in \mathbb{N}$, 有 $y = 2^{1/(x+2)} = f(x)$ 。

II.1.41 回想一下, 正如每个自然数都有一个有限位的十进制表示 (且表示是唯一的), 每个自然数也有一个唯一的二进制表示。

(A) 将每个以 1 开头的比特串 σ 视为某个自然数 n 的二进制表示。那么, 由 $\sigma \mapsto n-1$ 给出的映射 $f: B \rightarrow \mathbb{N}$ 显然是一个双射。

(B) 在每个串 $\tau \in B^*$ 前添加 1, 得到 $\sigma = 1 \frown \tau$ 。现在就可以应用 (a) 中的论证了。

II.1.42 正切函数是从 $(-\pi/2.. \pi/2)$ 到 $\mathbb{R}$ 的对应, 所以其逆函数反正切是反方向的对应。因此, $g(x) = (\arctan(x) + (\pi/2))/\pi$ 给出了一个从 $\mathbb{R}$ 到 $(0..1)$ 的对应。参见 Figure 12 on page 52, 该图表明 g 是单射的且满射的, 因为对于陪域 $(0..1)$ 中的每个元素 y , 过 y 的水平线与函数图像恰好交于一点。

II.1.43

(A) 由 $g(x) = x \cdot (r_1/r_0)$ 给出的映射 $g: I_0 \rightarrow I_1$ 是满射的, 因为如果 $y \in I_1 = [0..2\pi r_1]$, 那么 $y = g(x)$, 其中 $x = y \cdot (r_0/r_1)$, 并且 $0 \leq y < 2\pi r_1$ 蕴涵着 $0 \leq y \cdot (r_0/r_1) < 2\pi r_0$ 。欲证此映射是单射的, 假设 $g(x) = g(\hat{x})$, 则 $x \cdot (r_1/r_0) = \hat{x} \cdot (r_1/r_0)$, 因此 $x = \hat{x}$ 。

(B) 映射 $g(x) = a + x \cdot (b-a)$ 是一个双射。欲证其是单射的, 假设 $g(x) = g(\hat{x})$, 从而有 $a + x \cdot (b-a) = a + \hat{x} \cdot (b-a)$ 。两边减去 a 并除以 $b-a$, 可得 $x = \hat{x}$ 。此映射也是满射的, 因为 $y \in [a..b]$ 是 $x = (y-a)/(b-a)$ 的像, 且 $a \leq y < b$ 蕴涵着 $0 \leq y-a < b-a$, 因此 $0 \leq (y-a)/(b-a) < 1$ 。

(C) 由前一小项知, 存在双射 $f: [0..1) \rightarrow [a..b)$ 和 $g: [0..1) \rightarrow [c..d)$ 。那么复合映射 $g \circ f^{-1}$ 的定义域为 $[a..b)$, 陪域为 $[c..d)$, 且是一个双射。

(这也可以通过考虑由 $f(x) = c + (x-a) \cdot (d-c)/(b-a)$ 给出的映射 $f: [a..b) \rightarrow [c..d)$ 来轻松验证。)

II.1.44 假设 f 不是单射的, 欲推出矛盾。那么存在 $x, \hat{x} \in D$, 满足 $f(x) = f(\hat{x})$ 但 $x \neq \hat{x}$ 。然而两者中必有一个较小; 不失一般性, 假设 $x < \hat{x}$ 。那么 $x < \hat{x}$ 但 $f(x) = f(\hat{x})$, 这与函数严格递增矛盾。是的, 同样的论证对 $D \subseteq \mathbb{N}$ 也成立。

II.1.45

(A) 令 $\mathcal{E} = \{2n \mid n \in \mathbb{N}\}$ 。自然的对应是 $f: \mathbb{N} \rightarrow \mathcal{E}$, 由 $n \mapsto 2n$ 给出。此映射是单射的, 因为如果 $f(n) = f(\hat{n})$ 则 $2n = 2\hat{n}$, 从而 $n = \hat{n}$ 。它是满射的, 因为如果 $y \in \mathcal{E}$, 则对于某个 $k \in \mathbb{N}$ 有 $y = 2k$, 令 $n = k$ 即有 $y = f(n)$ 。

(B) 令 $T = \{n \in \mathbb{N} \mid n > 4\}$ 。考虑 $f: \mathbb{N} \rightarrow T$, 由 $n \mapsto n+5$ 给出。这是单射的, 因为如果 $f(n) = f(\hat{n})$ 则 $n+5 = \hat{n}+5$, 且 $n = \hat{n}$ 。它是满射的, 因为如果 $y \in T$, 则 $y = f(x)$ 其中 $x = y-5$, 并注意 $y \in T$ 蕴涵着 $y-5 \in \mathbb{N}$ 。所以 f 是一个双射。

II.1.46 我们将证明, 无论有限集合 $S \subset \mathbb{N}$ 的势是多少, 都存在一个双射 $f: \mathbb{N} \rightarrow \mathbb{N} - S$ 。我们将使用归纳法, 每一步中有限集的元素个数比前一步多一。

在证明之前, 我们先给一个例子。假设我们有一个五元素集合 $S_5 = \{2, 5, 8, 10, 11\}$ 以及这个双射 $f_5: \mathbb{N} \rightarrow \mathbb{N} - S_5$ 。

$$\begin{array}{c|cccccccc} n & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & \dots \\ f_5(n) & 0 & 1 & 3 & 4 & 6 & 7 & 9 & 12 & \dots \end{array}$$

现在我们添加第六个元素, 令 $S = S_5 \cup \{9\}$ 。我们想要一个双射 $f: \mathbb{N} \rightarrow \mathbb{N} - S$ 。自然的做法是注意到 $9 = f_5(6)$, 然后按如下方式定义 f 。

$$f(n) = \begin{cases} f_5(n) & - \text{若 } n < 6 \\ f_5(n+1) & - \text{若 } n > 6 \end{cases} \quad \begin{array}{c|cccccccc} n & 0 & 1 & 2 & 3 & 4 & 5 & 6 & \dots \\ f(n) & 0 & 1 & 3 & 4 & 6 & 7 & 12 & \dots \end{array}$$

归纳基础是 $S = \emptyset$, 此时恒等函数 $f(n) = n$ 显然是从 $\mathbb{N}$ 到 $\mathbb{N} - S$ 的单射且满射的映射。

对于归纳步, 取 $k \in \mathbb{N}$, 假设该命题对势小于等于 k 的任意 $\mathbb{N}$ 的子集成立, 并考虑满足 $|S| = k+1$ 的 $S \subset \mathbb{N}$ 。固定任意 $s \in S$, 并令 $S_k = S - \{s\}$ 。由归纳假设, 存在一个双射 $f_k: \mathbb{N} \rightarrow \mathbb{N} - S_k$ 。设 $\hat{n}$ 为满足 $f_k(\hat{n}) = s$ 的数, 并按如下方式定义 $f: \mathbb{N} \rightarrow \mathbb{N} - S$ 。

$$f(n) = \begin{cases} f_k(n) & - \text{若 } n < \hat{n} \\ f_k(n+1) & - \text{若 } n > \hat{n} \end{cases}$$

函数 $f: \mathbb{N} \rightarrow \mathbb{N} - S$ 是单射的, 因为它是 f_k 的限制, 而 f_k 是从 $\mathbb{N}$ 到 $\mathbb{N} - \hat{S}$ 的单射的映射。类似地, f 是满射的, 因为 f_k 是从 $\mathbb{N}$ 到 $\mathbb{N} - S_k$ 的满射。因此 f 是 $\mathbb{N}$ 与 $\mathbb{N} - S$ 之间的一个双射, 所以二者等势。

II.1.47

- (A) 逆函数 $f^{-1}: C \rightarrow D$ 由 $f^{-1}(\text{黑桃}) = 0$, $f^{-1}(\text{红桃}) = 1$, $f^{-1}(\text{梅花}) = 2$, 和 $f^{-1}(\text{方块}) = 3$ 给出。通过观察可知它既是单射的又是满射的。
- (B) 假设 $f: D \rightarrow C$ 是一个双射。考虑这个二元关系, 即有序对的集合, $f^{-1} = \{\langle y, x \rangle \mid f(x) = y\}$ 。我们将证明 f^{-1} 是一个函数, 这意味着我们要证明它是良定义的, 即每个输入 $y \in C$ 都恰好与一个输出 $x \in D$ 相关联。

每个 $y \in C$ 至少与一个 x 相关联, 因为函数 f 是满射的。每个 $y \in C$ 至多与一个 x 相关联, 因为 f 是单射的。

- (C) 为了证明 $f^{-1}: C \rightarrow D$ 是单射的, 假设 $f^{-1}(y) = f^{-1}(\hat{y})$ 。那么存在一个 $x \in D$, 使得 $f(x) = y$ 且 $f(x) = \hat{y}$ 。这与 f 是一个函数相矛盾, 因为函数必须是良定义的。

为了证明 f^{-1} 是满射的, 考虑其陪域中的一个元素 $\hat{x} \in D$ 。根据函数的定义, 存在一个 $\hat{y} \in C$ 满足 $f(\hat{x}) = \hat{y}$, 于是有 $\hat{x} = f^{-1}(\hat{y})$ 。

II.1.48 一个方向是直接的: 根据引理 1.5, 没有有限集会与其自身的真子集等势。

对于另一个方向, 固定一个无限集 S , 来证明它与其自身的某个真子集等势。(注: 和别处一样, 我们这里自由地使用了选择公理。) 选择任意元素 $s_0 \in S$ 。那么 $S - \{s_0\}$ 非空 (否则 S 就不是无限集), 所以选择 $s_1 \in S - \{s_0\}$ 。继续这种方式, 得到 $\{s_0, s_1, s_2, \dots\}$ 。那么这就是一个从 S 到 $S - \{s_0\}$ 的双射。

$$f(x) = \begin{cases} s_{i+1} & - \text{若 } x = s_i \text{ for some } i \in \mathbb{N} \\ x & - \text{其他情形} \end{cases}$$

II.1.49 记函数为 $f: D \rightarrow C$, 并假设 D 有限。

- (A) 我们将通过对定义域元素个数的归纳, 来证明以下断言对定义域有限的所有函数都成立: $|\text{dom}(f)| \geq |\text{ran}(f)|$

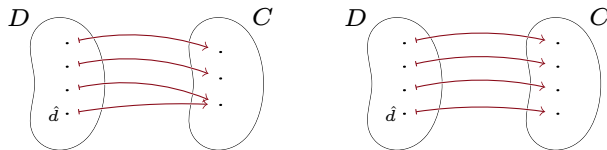


图 13, FOR QUESTION II.1.49: 有限函数的定义域元素个数不小于其值域元素个数。

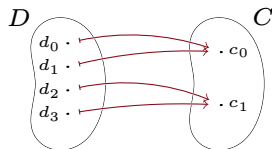


图 14, FOR QUESTION II.1.49: 不是单射的有限函数, 其定义域元素个数多于值域。

归纳基础: 定义域没有任何元素, 即 $\text{dom}(f) = \emptyset$ 。定义域为空的唯一函数是空函数。空函数的值域也是空集, $\text{ran}(f) = \emptyset$, 因此 $|\text{dom}(f)| \geq |\text{ran}(f)|$ 。

归纳步骤: 固定 $k \in \mathbb{N}$, 假设对任何定义域有 0 个、1 个、……、或 k 个元素的函数, 该断言都成立。现在考虑 $|D| = k + 1$ 的情况。

由于 $k + 1 > 0$, 存在 $\hat{d} \in D$ 。令 $\hat{D} = D - \{\hat{d}\}$, 并考虑限制函数 $f|_{\hat{D}}: \hat{D} \rightarrow C$ 。该限制函数的定义域有 k 个元素, 因此归纳假设适用, 得到 $|\text{dom}(f|_{\hat{D}})| \geq |\text{ran}(f|_{\hat{D}})|$ 。

为了把元素 $\hat{d}$ 加回去, 有两种情况; 见 Figure 13 on page 54。第一种情况, 即图中左侧所示, 是 f 的值域与限制函数 $f|_{\hat{D}}$ 的值域相同。另一种情况是 f 的值域比限制函数的值域多一个元素。无论哪种情况, 在两边同时加 1 即可完成证明。

$$|\text{dom}(f)| = |\text{dom}(f|_{\hat{D}})| + 1 \geq |\text{ran}(f|_{\hat{D}})| + 1 \geq |\text{ran}(f)|$$

- (B) 假设函数是单射的。我们将通过对定义域元素个数的归纳来证明定义域与值域大小相同。这个证明类似于上一项, 只是它仅有 Figure 13 on page 54 右侧所示的那一种情况。

归纳基础: 函数的定义域没有任何元素, 即 $\text{dom}(f) = \emptyset$ 。该函数的值域也是空集, 因此 $|\text{dom}(f)| = |\text{ran}(f)|$ 。

归纳步骤: 固定 $k \in \mathbb{N}$, 假设对任何定义域有 0 个、1 个、……、或 k 个元素的单射函数, 定义域与值域的大小都相同。现在设 $|D| = k + 1$ 。

由于 $k + 1 > 0$, 存在 $\hat{d} \in D$ 。令 $\hat{D} = D - \{\hat{d}\}$ 。考虑限制函数 $f|_{\hat{D}}: \hat{D} \rightarrow C$ 。因为 f 是单射的, 该限制函数也是单射的。归纳假设适用, 给出 $|\text{dom}(f|_{\hat{D}})| = |\text{ran}(f|_{\hat{D}})|$ 。两边同时加 1 得 $|\text{dom}(f)| = |\text{dom}(f|_{\hat{D}})| + 1 = |\text{ran}(f|_{\hat{D}})| + 1 = |\text{ran}(f)|$ 。

- (C) Figure 14 on page 54 图示了证明思路。

证明如下: 设 f 是一个定义域有限且不是单射的函数。固定其定义域的一个编号, $D = \{d_0, d_1, \dots, d_n\}$ 。考虑

$$\hat{D} = \{d_i \in D \mid i \text{ 是在满足 } f(d_j) = f(d_i) \text{ 的指标 } j \text{ 中最小的}\}$$

(在图示中, $f(d_0) = f(d_1)$ 且 $f(d_2) = f(d_3)$), 因此该集合为 $\{d_0, d_2\}$ 。

由于 f 不是单射的, $\hat{D}$ 是 D 的真子集。限制函数 $f|_{\hat{D}}: \hat{D} \rightarrow C$ 是单射的。由上一项可知, 其值域与定义域的元素个数相同。 f 的值域与 $f|_{\hat{D}}$ 的值域相同。但 f 的定义域比 $f|_{\hat{D}}$ 的定义域元素更多。因此 f 的定义域元素个数多于其值域。

- (D) 设 D 和 C 为有限集。若存在双射 $f: D \rightarrow C$, 则由前两项可知这两个集合的元素个数相同。

反之, 假设两者元素个数相同。固定每个集合的一个顺序: $D = \{d_0, \dots, d_k\}$ 和 $C = \{c_0, \dots, c_k\}$ 。那么由 $f(d_i) = c_i$ 定义的映射 $f: D \rightarrow C$ 显然是一个双射。

II.2.19 表格在第 0 列与第 1 列之间来回交替。

$n \in \mathbb{N}$	6	7	8	9	10	11	12
$f(n) \in \{0, 1\} \times \mathbb{N}$	$\langle 0, 3 \rangle$	$\langle 1, 3 \rangle$	$\langle 0, 4 \rangle$	$\langle 1, 4 \rangle$	$\langle 0, 5 \rangle$	$\langle 1, 5 \rangle$	$\langle 0, 6 \rangle$

公式为 $x(n) = (1 + (-1)^{n+1})/2$ 与 $y(n) = \lfloor n/2 \rfloor$ 。

II.2.20

- (A) $\langle 50, 50 \rangle$ 的前一个有序偶是 $\langle 49, 51 \rangle$, 后一个有序偶是 $\langle 51, 49 \rangle$ 。可验证它们分别对应 $\text{cantor}(50, 50) = 5\,100$ 、 $\text{cantor}(49, 51) = 5\,099$ 与 $\text{cantor}(51, 49) = 5\,101$ 。
- (B) $\langle 100, 4 \rangle$ 的前一个有序偶是 $\langle 99, 5 \rangle$, 后一个有序偶是 $\langle 101, 3 \rangle$ 。可验证它们分别对应 $\text{cantor}(100, 4) = 5\,560$ 、 $\text{cantor}(99, 5) = 5\,559$ 与 $\text{cantor}(101, 3) = 5\,561$ 。
- (C) $\langle 4, 100 \rangle$ 的前一个有序偶是 $\langle 3, 101 \rangle$, 后一个有序偶是 $\langle 5, 99 \rangle$ 。可验证它们分别对应 $\text{cantor}(4, 100) = 5\,464$ 、 $\text{cantor}(3, 101) = 5\,463$ 与 $\text{cantor}(5, 99) = 5\,465$ 。
- (D) $\langle 0, 200 \rangle$ 的前一个有序偶是 $\langle 199, 0 \rangle$, 后一个有序偶是 $\langle 1, 199 \rangle$ 。可验证它们分别对应 $\text{cantor}(0, 200) = 20\,100$ 、 $\text{cantor}(199, 0) = 20\,099$ 与 $\text{cantor}(1, 199) = 20\,101$ 。
- (E) $\langle 200, 0 \rangle$ 的前一个有序偶是 $\langle 199, 1 \rangle$, 后一个有序偶是 $\langle 0, 201 \rangle$ 。可验证它们分别对应 $\text{cantor}(200, 0) = 20\,300$ 、 $\text{cantor}(199, 1) = 20\,299$ 与 $\text{cantor}(0, 201) = 20\,301$ 。

II.2.21

- (A) 自然的对应 $f_T: \mathbb{N} \rightarrow T$ 为 $f_T(n) = 2n$ 。自然的对应 $f_F: \mathbb{N} \rightarrow T$ 由 $f_F(m) = 5m$ 给出。

为验证 f_T 是单射的, 假设 $f_T(n_0) = f_T(n_1)$, 即 $2n_0 = 2n_1$, 两边约去 2 得 $n_0 = n_1$ 。为验证 f_T 是满射的, 取 $y \in T$ 。由定义有 $y = 2k$, 于是 $y = f_T(k)$ 。验证 f_N 是单射的和满射的方法与此相同。

$n \in \mathbb{N}$	0	1	2	3	4	5	6	7	8	9
$f_T(n) \in T$	0	2	4	6	8	10	12	14	16	18
$f_F(n) \in F$	0	5	10	15	20	25	30	35	40	45

- (B) 一种自然的对应会按升序给出输出值。

$n \in \mathbb{N}$	0	1	2	3	4	5	6	7	8	9
$f(n) \in T \cup F$	0	2	4	5	6	8	10	12	14	15

II.2.22 以下给出函数 $f: \mathbb{N} \rightarrow \mathbb{N} \times \{0, 1\}$ 的最初几个对应关系。

n	0	1	2	3	4	5	...
$f(n)$	$\langle 0, 0 \rangle$	$\langle 0, 1 \rangle$	$\langle 1, 0 \rangle$	$\langle 1, 1 \rangle$	$\langle 2, 0 \rangle$	$\langle 2, 1 \rangle$	...

根据此表, 不难给出一个足以计算 f 的描述。

$$f(n) = \begin{cases} \langle \lfloor n/2 \rfloor, 0 \rangle & \text{若 } n \text{ 为偶数} \\ \langle \lfloor n/2 \rfloor, 1 \rangle & \text{若 } n \text{ 为奇数} \end{cases}$$

由此可得 $f(0) = \langle 0, 0 \rangle$ 、 $f(10) = \langle 5, 0 \rangle$ 、 $f(100) = \langle 50, 0 \rangle$ 以及 $f(101) = \langle 50, 1 \rangle$ 。

根据上表很容易找到其逆函数 $f^{-1}(i, j) = 2i + j$ 的公式。于是有 $f^{-1}(2, 1) = 5$ 、 $f^{-1}(20, 1) = 41$ 及 $f^{-1}(200, 1) = 401$ 。

II.2.23 $\{0, 1, 2\} \times \mathbb{N}$ 的成员是形如 $\langle x, y \rangle$ 的数对, 其中第一项是 0、1 或 2 之一, 第二项是一个自然数。根据明显的对应关系, 下表给出了最初的一些关联。

编号	0	1	2	3	4	5	6	7	8	...
数对	$\langle 0, 0 \rangle$	$\langle 1, 0 \rangle$	$\langle 2, 0 \rangle$	$\langle 0, 1 \rangle$	$\langle 1, 1 \rangle$	$\langle 2, 1 \rangle$	$\langle 0, 2 \rangle$	$\langle 1, 2 \rangle$	$\langle 2, 2 \rangle$	...

从编号到数对的对应公式如下。

$$f(n) = \begin{cases} \langle 0, \lfloor n/3 \rfloor \rangle & \text{若 } 3 \text{ divides } n \\ \langle 1, \lfloor n/3 \rfloor \rangle & \text{若 } n \bmod 3 \text{ is } 1 \\ \langle 2, \lfloor n/3 \rfloor \rangle & \text{若 } n \bmod 3 \text{ is } 2 \end{cases}$$


```
;; Return the pair following p in Cantor's correspondence
(define (next-pair p)
  (let ((x (first p))
        (y (second p)))
    (if (= y 0)
        (list 0 (+ x 1))
        (list (+ x 1) (- y 1)))))

;; Find pair n by brute force
(define (brute-force-pairing n)
  (define (bfp-helper j pr)
    (if (zero? j)
        pr
        (bfp-helper (sub1 j) (next-pair pr))))
  (bfp-helper n (list 0 0)))
```

图 15, FOR QUESTION II.2.26: 输入一个数并通过暴力枚举找到对应数对的 Racket 代码。

简而言之: $f(n) = \langle n \bmod 3, \lfloor n/3 \rfloor \rangle$ 。因此, $f(0) = \langle 0, 0 \rangle$ 、 $f(10) = \langle 1, 3 \rangle$ 、 $f(100) = \langle 1, 33 \rangle$ 和 $f(1000) = \langle 1, 333 \rangle$ 。从数对到编号的函数 $f^{-1}: \{0, 1, 2\} \times \mathbb{N} \rightarrow \mathbb{N}$ 由上表也容易得到: $f^{-1}(x, y) = x + 3y$ 。由此, $f^{-1}(1, 2) = 7$ 、 $f^{-1}(1, 20) = 61$ 和 $f^{-1}(1, 200) = 601$ 。

II.2.24 下表是 $\{0, 1, 2, 3\} \times \mathbb{N}$ 的一个枚举 f 的初始部分。

编号	0	1	2	3	4	5	6	7	8	...
数对	$\langle 0, 0 \rangle$	$\langle 1, 0 \rangle$	$\langle 2, 0 \rangle$	$\langle 3, 0 \rangle$	$\langle 0, 1 \rangle$	$\langle 1, 1 \rangle$	$\langle 2, 1 \rangle$	$\langle 3, 1 \rangle$	$\langle 0, 2 \rangle$	...

从编号到数对的对应公式为 $f(n) = \langle n \bmod 4, \lfloor n/4 \rfloor \rangle$, 所以 $x(n) = n \bmod 4$ 且 $y(n) = \lfloor n/4 \rfloor$ 。

一般地, 对于 $k \in \mathbb{N}$, $\{0, 1, 2, \dots, k\} \times \mathbb{N}$ 的一个枚举 f 是 $f(n) = \langle n \bmod (k+1), \lfloor n/(k+1) \rfloor \rangle$ 。

II.2.25 例 2.4 中的表格展示到了 $n = 6$ 。

编号	7	8	9	10	11	12	13	14	15	16
数对	$\langle 1, 2 \rangle$	$\langle 2, 1 \rangle$	$\langle 3, 0 \rangle$	$\langle 0, 4 \rangle$	$\langle 1, 3 \rangle$	$\langle 2, 2 \rangle$	$\langle 3, 1 \rangle$	$\langle 4, 0 \rangle$	$\langle 0, 5 \rangle$	$\langle 1, 4 \rangle$

II.2.26 计算这个函数的一种方法是暴力枚举。给定 n , 程序可以按升序生成数对 $\langle 0, 0 \rangle$ 、 $\langle 0, 1 \rangle$ 、 $\langle 1, 0 \rangle$ 、 $\langle 0, 2 \rangle$ 、..., 同时计数, 直到生成匹配的数对。Figure 15 on page 56 提供了实现此功能的 Racket 代码。

II.2.27 根据推论 2.13, 因为集合 $A - B$ 是 A 的子集, 所以它是可数的。类似地, $B - A \subseteq B$ 也是可数的。然后, 根据同一结果, 它们的并集也是可数的, 即对称差是可数的。

II.2.28 令 $S = \{a + bi \mid a, b \in \mathbb{Z}\}$ 。显然, 由 $g(a, b) = a + bi$ 定义的映射 $g: \mathbb{Z} \times \mathbb{Z} \rightarrow S$ 是一个双射。引理 2.8 指出 $\mathbb{Z} \times \mathbb{Z}$ 是可数的, 因此存在一个双射 $f: \mathbb{N} \rightarrow \mathbb{Z} \times \mathbb{Z}$ 。那么复合映射 $g \circ f: \mathbb{N} \rightarrow S$ 就是一个双射。

II.2.29 下表展示了该例中的算法过程。第二列按升序给出了数对。第三列列出了搜索循环得到的有理数结果。例如, 循环编号 $n = 1$ 时, 首先生成 $\langle 1, 0 \rangle$ 并因其分母为零而拒绝它, 然后生成 $\langle 0, 2 \rangle$ 并因有理数 $0 = 0/2$ 已被枚举而拒绝它, 接着生成 $\langle 1, 1 \rangle$, 由于 $1 = 1/1$ 是迄今为止还未被枚举的有理数, 我们得到 $f(1) = 1$ 。

n	数对	有理数 $f(n)$
0	$\langle 0, 0 \rangle, \langle 0, 1 \rangle$	0
1	$\langle 1, 0 \rangle, \langle 0, 2 \rangle, \langle 1, 1 \rangle$	1
2	$\langle 2, 0 \rangle, \langle 0, 3 \rangle, \langle 1, 2 \rangle$	$1/2$
3	$\langle 2, 1 \rangle$	2
4	$\langle 3, 0 \rangle, \langle 0, 4 \rangle, \langle 1, 3 \rangle$	$1/3$
5	$\langle 2, 2 \rangle, \langle 3, 1 \rangle$	3
6	$\langle 4, 0 \rangle, \langle 0, 5 \rangle, \langle 1, 4 \rangle$	$1/4$
7	$\langle 2, 3 \rangle$	$2/3$
8	$\langle 3, 2 \rangle$	$3/2$
9	$\langle 4, 1 \rangle$	4
10	$\langle 5, 0 \rangle, \langle 0, 6 \rangle, \langle 1, 5 \rangle$	$1/5$
11	$\langle 2, 4 \rangle, \langle 3, 3 \rangle, \langle 4, 2 \rangle, \langle 5, 1 \rangle$	5

II.2.30

- (A) 集合 $S - T$ 是一个可数集合的子集，因此它是可数的。
 (B) 集合 $S - T$ 不可能是有限的，否则 $T \cup (S - T) = S$ 将成为两个有限集合的并集，从而也是有限的。
 (C) 是的，集合 $\mathbb{N}$ 是可数无限的，偶数集合 $2\mathbb{N} \subseteq \mathbb{N}$ 也是无限的，并且 $\mathbb{N} - 2\mathbb{N}$ ，即奇数集合，是无限的。

II.2.31 一个无限集合包含某个元素 a_0 。因为它无限的，所以它不止有一个元素，因此它还包含某个 $a_1 \neq a_0$ 。照此方式继续进行，我们得到可数无限集合 $\{a_0, a_1, \dots\}$ 。（如果它是 $\mathbb{N}$ 的子集，则可以取大于 a_0 的最小元素作为 a_1 ，从而明确这一选择，并按此方式继续。）

II.2.32

- (A) 由 $f(a_0, \dots, a_n) = a_n x^n + \dots + a_1 x + a_0$ 给出的映射 $f: \mathbb{Z}^{n+1} \rightarrow \mathbb{Z}_n[x]$ 显然是一个双射。（我们并非声称不同的多项式作为函数显然是不同的，我们只是声称它们作为多项式是不同的。）因为推论 2.9 表明存在一个双射 $g: \mathbb{N} \rightarrow \mathbb{Z}^{n+1}$ ，所以我们有一个从 $\mathbb{N}$ 到 $\mathbb{Z}_n[x]$ 的双射 $f \circ g$ 。
 (B) 集合 $\mathbb{Z}[x]$ 是 $i \in \mathbb{N}$ 时所有 $\mathbb{Z}_i[x]$ 的并集，而可数多个可数集合的并集是可数的。

II.2.33 令 $f: \mathbb{N} \rightarrow S$ 为一个双射。将 $f(n)$ 记为 s_n ，则由 $s_n \mapsto s_{n+1}$ 给出的映射 $g: S \rightarrow S$ 是单射的但不满射的。

II.2.34 我们将利用函数 $\text{cantor}: \mathbb{N}^2 \rightarrow \mathbb{N}$ 是一个双射的事实。

先验证 cantor_3 是满射。固定陪域中的元素 $n \in \mathbb{N}$ 。因为 cantor 是满射，存在对子 $\langle w, z \rangle \in \mathbb{N}^2$ 使得 $\text{cantor}(w, z) = n$ 。同样因为 cantor 是满射，存在对子 $\langle x, y \rangle \in \mathbb{N}^2$ 使得 $\text{cantor}(x, y) = w$ 。于是 $\text{cantor}_3(x, y, z) = \text{cantor}(\text{cantor}(x, y), z) = n$ 。

接下来验证 cantor_3 是单射。假设 $\text{cantor}_3(a_0, b_0, c_0) = \text{cantor}_3(a_1, b_1, c_1)$ 。那么 $\text{cantor}(\text{cantor}(a_0, b_0), c_0) = \text{cantor}(\text{cantor}(a_1, b_1), c_1)$ 。函数 cantor 是单射，因此由上式可得 $\text{cantor}(a_0, b_0) = \text{cantor}(a_1, b_1)$ 且 $c_0 = c_1$ 。再次应用 cantor 的单射性，得到 $a_0 = a_1$ 、 $b_0 = b_1$ ，以及已得的 $c_0 = c_1$ 。

II.2.35 要确信这些结果，可以写一小段脚本。

```
sage: def cantor(x,y):
....:     return x+ ( (x+y)*(x+y+1)/2 )
....:
sage: def cantor3(x,y,z):
....:     return cantor(cantor(x,y),z)
....:
```

(A) $c_3(0, 0, 0) = 0$ 、 $c_3(1, 2, 3) = 62$ 、 $c_3(3, 3, 3) = 402$

(B) $c_3(0, 0, 0) = 0$ 、 $c_3(0, 0, 1) = 1$ 、 $c_3(0, 1, 0) = 2$ 、 $c_3(0, 0, 2) = 3$ 、 $c_3(0, 1, 1) = 4$ 、 $c_3(1, 0, 0) = 5$

(c) 代入即得。

$$\begin{aligned} c_3(x, y, z) &= \left(x + \frac{(x+y)(x+y+1)}{2} \right) + \left(\frac{\left(x + \frac{(x+y)(x+y+1)}{2} + z \right) \left(x + \frac{(x+y)(x+y+1)}{2} \right) + z + 1}{2} \right) \\ &= \frac{1}{8} \cdot ((x+y+1)(x+y) + 2x + 2z + 2) \cdot ((x+y+1)(x+y) + 2x + 2z) \\ &\quad + \frac{1}{2} \cdot (x+y+1)(x+y) + x \end{aligned}$$

最后这个是由机器计算的，而非手算。

```
sage: x,y,z = var('x,y,z')
sage: cantor3(x,y,z)
1/8*((x + y + 1)*(x + y) + 2*x + 2*z + 2)*((x + y + 1)*(x + y) + 2*x + 2*z) + 1/2*(x + y + 1)*(x
```

II.2.36 将 $x = y = i$ 代入 $\text{cantor}(x, y) = x + (x+y)(x+y+1)/2$ 得到 $\text{cantor}(i, i) = i + (2i)(2i+1)/2 = 2i^2 + 2i$ 。即 $2i(i+1)$ 。它显然是 2 的倍数，但由于 i 与 $i+1$ 是连续整数，其中一个会再贡献一个因子 2。因此整个表达式是 4 的倍数。

II.2.37 固定 b 。对 $n \in \mathbb{N}$ ，令 S_N 是那些因为从位置 N 开始与 b 相同而与 b 等价的序列的集合。这个集合是有限的，有 2^N 个元素，因为 $\hat{b} \in S_N$ 与 b 的不同仅可能在前 N 位。（例如，如果 $b = \langle 0, 1, 0, 1, 0, 1, \dots \rangle$ 且 $N = 1$ ，那么 S_N 中除 b 外唯一的元素是 $\hat{b} = \langle 1, 1, 0, 1, 0, 1, \dots \rangle$ 。）因此，与 b 等价的序列的总数就是这些有限集 S_N 在所有可数个 N 上的并集。

II.2.38 设 S_0, S_1 和 S_2 是可数无限的，并假设 $f_0: \mathbb{N} \rightarrow S_0, f_1: \mathbb{N} \rightarrow S_1$ 和 $f_2: \mathbb{N} \rightarrow S_2$ 是满射。那么 $\hat{f}: \mathbb{N} \rightarrow S_0 \cup S_1 \cup S_2$ 由

$$\hat{f}(n) = \begin{cases} f_0(n/3) & \text{若 } n \text{ is a multiple of 3, that is, } n \bmod 3 = 0 \\ f_1((n-1)/3) & \text{若 } n \bmod 3 = 1 \\ f_2((n-2)/3) & \text{若 } n \bmod 3 = 2 \end{cases}$$

给出（简言之， $\hat{f}(n) = f_{n \bmod 3}(\lfloor n/3 \rfloor)$ ）。下面的表格示意

n	0	1	2	3	4	5	6	...
$\hat{f}(n)$	$f_0(0)$	$f_1(0)$	$f_2(0)$	$f_0(1)$	$f_1(1)$	$f_2(1)$	$f_3(0)$	...

说明了为什么这个函数是满射。应用引理 2.12，可知该并集是可数的。

II.2.39 一个函数 $f: \{0, 1\} \rightarrow \mathbb{N}$ 由两条信息决定： $f(0)$ 和 $f(1)$ 。因此我们可以将其视为一个对子 $\langle f(0), f(1) \rangle$ 。换言之，在函数集合 $\{f \mid f: \{0, 1\} \rightarrow \mathbb{N}\}$ 与对子集合 $\mathbb{N} \times \mathbb{N}$ 之间存在一个自然对应，而后可数的可数性是已知的。

II.2.40 因为 S 是可数的，引理 2.12 说明要么 S 是空集，要么存在一个满射函数 $g: \mathbb{N} \rightarrow S$ 。若 S 是空集，则值域集也是空集，因此是可数的。否则，函数 $f \circ g: \mathbb{N} \rightarrow \text{ran}(f)$ 是满射。

II.2.41 两个有限集的 Cartesian 积是有限的，因为 Cartesian 积中的元素个数等于各集合元素个数的乘积。

接下来考虑一个有限集 $S = \{s_0, \dots, s_{m-1}\}$ 和一个可数无限集 $T = \{t_0, t_1, \dots\}$ 。我们将证明 $S \times T$ 是可数的（ $T \times S$ 的情形类似）。按行枚举它，即利用通常的商-余数关系 $n = d \cdot m + r$ ，则 $f(n) = \langle s_r, t_d \rangle$ 。（例如， $m = 3$ 情形的初始部分

n	0	1	2	3	4	5	6	...
$f(n)$	$\langle s_0, t_0 \rangle$	$\langle s_1, t_0 \rangle$	$\langle s_2, t_0 \rangle$	$\langle s_0, t_1 \rangle$	$\langle s_1, t_1 \rangle$	$\langle s_2, t_1 \rangle$	$\langle s_0, t_2 \rangle$	...

，其中第一项循环 $s_0, s_1, s_2, s_0, \dots$ ，而第二项则在无限集中逐步增加。）显然这个函数是一个双射。

II.2.42

- (A) 长度为 5 的字符串集合 $\Sigma^5 = \{\langle s_0, \dots, s_4 \rangle \mid s_i \in \Sigma\}$ 是五个有限集的 Cartesian 积, 每个集合有 40 个字符。所以 Σ^5 有 40^5 个元素。(顺便提一下, $40^5 = 102\,400\,000$ 。) 因此它是可数的。
- (B) 集合 $\Sigma^0 \cup \Sigma^1 \cup \dots \cup \Sigma^5$ 是有限多个有限集的并。所以它是有限的, 从而是可数的。
- (C) 集合 $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \dots$ 是可数多个有限集的并。由推论 2.13 可知它是可数的。
- (D) 程序是有限字符串。所以所有程序的集合是 Σ^* 的子集。推论 2.13 说明这个集合是可数的。

II.2.43

- (A) 这些值容易手算或通过脚本计算。

	$m = 0$	$m = 1$	$m = 2$	$m = 3$
$n = 0$	0	2	4	6
$n = 1$	1	5	9	13
$n = 2$	3	11	19	27
$n = 3$	7	23	39	55

- (B) 每个大于零的自然数都是素数的唯一乘积, $2^{e_2} 3^{e_3} \dots p^{e_p}$ 。所以每个这样的数都是一个 2 的最大幂 2^n 与一个奇数 $2m+1$ 的乘积。因此对于 $\langle n, m \rangle$ 对应于 $2^n \cdot (2m+1)$, 所以定义在 $\mathbb{N} \times \mathbb{N}$ 上、由 $g(n, m) = 2^n \cdot (2m+1) - 1$ 给出的映射 g 是一个双射, 其值域为 $\mathbb{N}$ 。
- (C) 下表显示在盒子枚举下, $\langle 3, 4 \rangle$ 对应于 19。

$y = 4$	16	17	18	19	20
$y = 3$	9	10	11	12	21
$y = 2$	4	5	6	13	22
$y = 1$	1	2	7	14	23
$y = 0$	0	3	8	15	24
	$x = 0$	$x = 1$	$x = 2$	$x = 3$	$x = 4$

II.2.44 以下是一个从 $\mathbb{N}$ 到 $\mathbb{Q}$ 的满射。给定自然数输入 n , 分解出前三个质数, 得到 $n = 2^{e_2} 3^{e_3} 5^{e_5} \cdot k$, 其中 k 不能被 2、3 或 5 整除。如果 e_2 是奇数, 则输出为 $-e_3/(e_5+1)$; 如果 e_2 是偶数, 则输出为 $e_3/(e_5+1)$ 。

II.2.45

- (A) $\text{cantor}(2, 1) = 8$
- (B) 解方程 $8 = 1 + [(1+y)(1+y+1)]/2$ 会导出二次方程 $0 = y^2 - 3y - 12$ 。利用求根公式得到 $(-3 \pm \sqrt{57})/2$, 即两个不同的解: $y \approx 2.27$ 和 $y \approx -5.27$ 。图 Figure 16 on page 60 显示了其图像。

II.3.10 无穷不同于穷尽。列表 $0, 2, 4, \dots$ 是无穷的, 但并未穷尽所有的自然数。

II.3.11 该并集并非全体实数集。它不包含无理数。例如, $\pi = 3.14\dots$ 不在这些集合中的任何一个里, 因此也不在并集中。你的同学只数了那些以零结尾的数。

II.3.12 Cantor 定理指出幂集的元素多于原集合。

- (A) 集合 $\{0, 1, 2\}$ 有三个元素。其幂集有八个。

$$\mathcal{P}(\{0, 1, 2\}) = \{\{0, 1, 2\}, \{0, 1\}, \{0, 2\}, \{1, 2\}, \{0\}, \{1\}, \{2\}, \emptyset\}$$

- (B) 集合 $\{0, 1\}$ 有两个元素, 而其幂集 $\mathcal{P}(\{0, 1\}) = \{\{0, 1\}, \{0\}, \{1\}, \emptyset\}$ 有四个。
- (C) 集合 $\{0\}$ 只有一个元素, 但其幂集 $\mathcal{P}(\{0\}) = \{\{0\}, \emptyset\}$ 有两个。
- (D) 空集有零个元素, 而其幂集 $\mathcal{P}\emptyset = \{\emptyset\}$ 有一个。

II.3.13 关于势的“ $\leq$ ”的定义是: 如果存在一个从 A 到 B 的单射函数, 则 $|A| \leq |B|$ 。

- (A) 函数 $f: S \rightarrow \hat{S}$ 由 $f(1) = 11$, $f(2) = 12$, 和 $f(3) = 13$ 给出, 通过观察可知它是单射的。
- (B) 函数 $f: T \rightarrow \hat{T}$ 由 $f(1) = 11$, $f(2) = 12$, 和 $f(3) = 13$ 给出, 通过观察可知它是单射的。
- (C) 同样通过观察, 函数 $f: U \rightarrow \mathbb{O}$ 由 $f(1) = 1$, $f(2) = 3$, 和 $f(3) = 5$ 给出, 是单射的。
- (D) 函数 $f: \mathbb{E} \rightarrow \mathbb{O}$ 由 $f(x) = x+1$ 给出, 是一个单射函数。验证很简单: $f(x_0) = f(x_1)$ 蕴含 $x_0+1 = x_1+1$, 因此 $x_0 = x_1$ 。

II.3.14 两者的区别在于元素所属的集合不同。

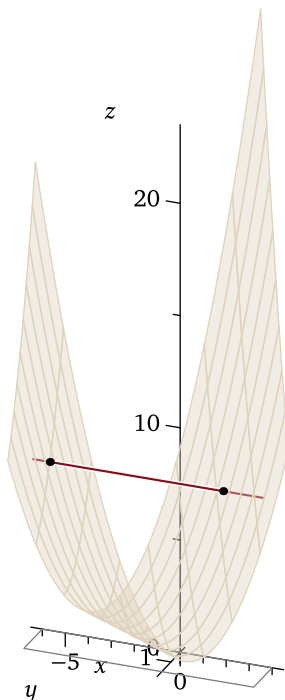


图 16, FOR QUESTION II.2.45: Cantor 解配对函数, 其中直线对应 $x = 1$ 且 $z = 8$ 。

- (A) 集合 $S = \{n \in \mathbb{Z} \mid n + 3 < 5\} = \{\dots, -2, -1, 0, 1\}$ 是可数的。由 $n \mapsto -n + 1$ 定义的函数 $f: \mathbb{N} \rightarrow S$ 是一个双射, 这一点容易验证。
- (B) 不可数; 它是实数集 $(-\infty, 2)$ 。(在映射 $x \mapsto \ln(2 - x)$ 下, 该集合对应于 $\mathbb{R}$ 。)

II.3.15

- (A) 不可数。(函数 $x \mapsto \ln(x - 5)$ 是到 $\mathbb{R}$ 的一个双射。)
- (B) 可数。(因为它是作为自然数的子集给出的, 而且该集合是 $\{1, 2, 3\}$, 是有限的。)
- (C) 此集合不可数。(集合 $S = [1, 4) \subset \mathbb{R}$ 的势至少与 $T = (1, 4) \subset \mathbb{R}$ 相同, 因为由 $x \mapsto x$ 给出的包含映射 $\iota: T \rightarrow S$ 是单射的。区间 T 与 $\mathbb{R}$ 等势。一种理解方式是, 从区间 $(-\pi/2, \pi/2)$ 与 $\mathbb{R}$ 之间由 $x \mapsto \tan(x)$ 给出的对应出发。将该区间缩放 $\pi/3$ 倍再平移以适配定义域 T , 即可得到 $x \mapsto \tan((\pi/3) \cdot (x - 1) - (\pi/2))$ 是一个双射。)
- (D) 可数。($\mathbb{N}$ 的任何子集都是可数的。)

II.3.16 对于二元集 $A_2 = \{0, 1\}$, 其幂集 $\mathcal{P}(A_2) = \{\{0, 1\}, \{0\}, \{1\}, \{\}\}$ 有四个元素。构造一个从 A_2 到 $\mathcal{P}(A_2)$ 的函数, 需要为 0 和 1 分别指定像。因此有 $4^2 = 16$ 个不同的函数。

function f_i	$f_i(0)$	$f_i(1)$	function f_i	$f_i(0)$	$f_i(1)$
f_0	$\{\}$	$\{\}$	f_8	$\{1\}$	$\{\}$
f_1	$\{\}$	$\{0\}$	f_9	$\{1\}$	$\{0\}$
f_2	$\{\}$	$\{1\}$	f_{10}	$\{1\}$	$\{1\}$
f_3	$\{\}$	$\{0, 1\}$	f_{11}	$\{1\}$	$\{0, 1\}$
f_4	$\{0\}$	$\{\}$	f_{12}	$\{0, 1\}$	$\{\}$
f_5	$\{0\}$	$\{0\}$	f_{13}	$\{0, 1\}$	$\{0\}$
f_6	$\{0\}$	$\{1\}$	f_{14}	$\{0, 1\}$	$\{1\}$
f_7	$\{0\}$	$\{0, 1\}$	f_{15}	$\{0, 1\}$	$\{0, 1\}$

三元集 A_3 有 $|\mathcal{P}(A_3)| = 8$ 。从 A_3 到 $\mathcal{P}(A_3)$ 的函数数量是 $8^3 = 512$ 。一般而言, 当 $|A| = n$ 时, 幂集有 $|\mathcal{P}(A_3)| = 2^n$ 个元素。函数数量是 $(2^n)^n = 2^{(n^2)}$ 。(作为验证, $n = 2$ 得到 $2^4 = 16$ 且 $n = 3$ 得到 $2^9 = 512$ 。)

II.3.17

(A) 这些是从 S 到 T 的函数。

function f_i	$f_i(0)$	$f_i(1)$
f_0	10	10
f_1	10	11
f_2	11	10
f_3	11	11

单射的函数是 f_1 和 f_2 。

(B) 这些是从 S 到 T 的函数。

function f_i	$f_i(0)$	$f_i(1)$
f_0	10	10
f_1	10	11
f_2	10	12
f_3	11	10
f_4	11	11

function f_i	$f_i(0)$	$f_i(1)$
f_5	11	12
f_6	12	10
f_7	12	11
f_8	12	12

单射的函数是 f_1 、 f_2 、 f_3 、 f_5 、 f_6 和 f_7 。

II.3.18

(A) 答案是 (iii) 有限，因为两个有限集的并集是有限的。(以下所有选项都成立：(ii) 可数或不可数，(iii) 有限，(iv) 可数，以及 (v) 有限、可数无限或不可数。但最强、最具体的是 (iii)。)

(B) 最强的是 (iv)。

(C) 最强的是 (i)。

(D) 最强的是 (iv)。(交集可能是有限的，但也存在可数的情况，例如 $A = \mathbb{N}$ 和 $B = \mathbb{R}$ 。因此我们能做出的最强的一般性陈述是 (iv)。)

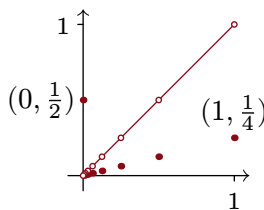
II.3.19 问题不要求证明，但这里的括号注释给出了理由。

(A) 除最后一项外，其他都是可能的。((i) 和 (ii) 的一个例子是 $\text{id}: \{0, 1\} \rightarrow \{0, 1\}$ 。(iii) 和 (iv) 的一个例子是 $\text{id}: \mathbb{N} \rightarrow \mathbb{N}$ 。引理 2.12 排除了 (v)。)

(B) 所有都是可能的。((i) 和 (ii) 的一个例子是 $\text{id}: \{0, 1\} \rightarrow \{0, 1\}$ 。(iii) 和 (iv) 的一个例子是 $\text{id}: \mathbb{N} \rightarrow \mathbb{N}$ 。(v) 的一个例子是 $\iota: \mathbb{N} \rightarrow \mathbb{R}$ ，其中 $\iota(x) = x$ 。)

II.3.20 根据提示，考虑以下函数。

$$f(x) = \begin{cases} 1/2 & \text{— 若 } x = 0 \\ 1/2^{n+2} & \text{— 若 } x = 1/2^n \text{ for } n \in \mathbb{N} \\ x & \text{— 其他情形} \end{cases}$$



论证函数 f 是单射且满射的一种方法是看它满足水平线测试，即对于每个 $t \in (0..1)$ ，水平线 $y = t$ 都恰好有一个对应的输入 x 。

我们可以展开说明。首先考虑输出 $y \in (0..1)$ 满足 $y \neq 1/2^n$ ($n \in \mathbb{N}^+$) 的情况。那么 y 恰好有一个对应的输入，即 $x = y$ 。另一种情况是 $y = 1/2^n$ (某个 $n \in \mathbb{N}^+$)。这里也有且仅有一个对应的输入 x 。具体来说，如果 $n = 1$ ，那么输入是 $x = 0$ ，它在定义域 $[0..1]$ 中。如果 $n > 1$ ，那么输入是 $x = 1/2^{n-2}$ ，它也是 $[0..1]$ 中的一个元素。

II.3.21 定理 3.7 说对于任意集合，其势严格小于其幂集的势。所以一个势大于 $\mathbb{R}$ 的集合是 $\mathcal{P}(\mathbb{R})$ 。(另一个是 $\mathcal{P}(\mathcal{P}(\mathbb{R}))$ 。)

II.3.22

(A) 有限比特串的集合 $\mathbb{B}^*$ 是集合 $\mathbb{B}^k = \{\langle b_0 b_1 \dots b_{k-1} \rangle \mid k \in \mathbb{N}\}$ 的并集。集合 $\mathbb{B}^k$ 是有限的，因为它有 2^k 个元素 (当 $k = 0$ 时，它只包含空串)，所以这是可数多个可数集合的并集。

(B) 假设我们可以计数无限比特串的集合, 即它有一个穷举枚举 $f_0, f_1, \dots$. 令 $g: \mathbb{B} \rightarrow \mathbb{B}$ 由 $g(0) = 1$ 和 $g(1) = 0$ 给出. 那么由 $F(i) = g(f_i(i))$ 定义的无限比特串 $F: \mathbb{N} \rightarrow \mathbb{B}$ 不等于任何一个 f_i , 这与枚举是穷举的矛盾.

II.3.23 设 S 和 T 满足 $S \subseteq T$. 那么包含映射, 即由 $\iota(s) = s$ 给出的映射 $\iota: S \rightarrow T$, 显然是单射的. 所以由定义 3.4, $|S| \leq |T|$.

II.3.24 我们按照提示进行. 将所有这样的函数组成的集合记作 $S = \{f: \mathbb{N} \rightarrow \mathbb{N} \mid \text{ran}(f) \text{ 是有限集}\}$. 假设 S 是可数的, 从而存在一个双射 $h: \mathbb{N} \rightarrow S$. 对于所有自然数 i , $h(i)$ 的值是一个函数, 我们将其记作 f_i .

定义映射 $g: \mathbb{N} \rightarrow \mathbb{N}$: 若 $x \neq 0$ 则 $g(x) = 0$, 否则 $g(x) = 1$. 考虑由 $k(i) = g(f_i(i))$ 给出的函数 $k: \mathbb{N} \rightarrow \mathbb{N}$. 这个映射的值域是有限的, 即 $\text{ran}(k) = \{0, 1\}$, 但 k 不等于 S 中的任何一个 f_i , 因为它们在输入 i 处的取值不同.

II.3.25 例 2.10 表明有理数集是可数的. 假设无理数集也是可数的. 那么因为两个可数集的并是可数的, 实数集将是可数的. 但实数集是不可数的, 因此无理数集也不可数.

II.3.26

(A) 将 z 的小数展开式写成 $z = 0.z_0z_1z_2\dots$. 由于 g 的输出永远不会是 9, 所以 z 不以全 9 结尾. 因此 z 有唯一的小数展开式. 对于每个 q_i , z 的小数展开式与 q_i 的不同之处在于 $z_i \cdot 10^{-(i+1)} \neq q_{i,i} \cdot 10^{-(i+1)}$, 这由 g 的定义可知. 因为它们的小数展开式不同, 并且都不以全 9 结尾, 所以它们是不同的数.

由于 z 不等于任何有理数, 所以它是无理数.

(B) 如果 $d = \sum_{n \in \mathbb{N}} d_{n,n} \cdot 10^{-(n+1)}$ 是有理数, 那么它的小数展开式会循环. 也就是说, 存在一个小数位, 使得从该位之后, 展开式由一串数字 $d_j d_{j+1} \dots d_{j+k}$ 不断重复组成. 但这样一来, 前一小题中的 z 也会循环, 即从同一个小数位之后, 它的展开式将由一串数字 $g(d_j)g(d_{j+1}) \dots g(d_{j+k})$ 不断重复组成. 这将使得 z 成为有理数, 但 z 并不是.

(C) 在定理 3.1 中, 我们从一个声称包含所有实数的列表开始, 然后构造出一个不在该列表中的实数. 这里, 我们从一个包含所有有理数的列表开始, 构造出了一个不是有理数的数. 因此它仍然是一个实数, 只不过不是有理数罢了.

II.3.27 在证明之前, 作为归纳步骤的动机, 考虑 $S = \{0, 1, 2, 3\}$. 这些是 $\mathcal{P}(S)$ 的十六个元素. 第一行是不包含 3 的集合, 而第二行是包含 3 的集合.

$$\begin{array}{cccccccc} \{\}, & \{0\}, & \{1\}, & \{2\}, & \{0, 1\}, & \{0, 2\}, & \{1, 2\}, & \{0, 1, 2\}, \\ \{3\}, & \{0, 3\}, & \{1, 3\}, & \{2, 3\}, & \{0, 1, 3\}, & \{0, 2, 3\}, & \{1, 2, 3\}, & \{0, 1, 2, 3\} \end{array}$$

底行的每个集合与顶行的对应集合相同, 只是多加了一个 3.

我们通过对 $|S|$ 进行归纳来证明. 基本情况是 $|S| = 0$, 即 S 是空集. 那么 $\mathcal{P}(S) = \{\emptyset\}$ 且 $|\mathcal{P}(S)| = 1 = 2^0$, 符合要求.

对于归纳步骤, 固定 $k \in \mathbb{N}^+$, 假设命题对 $n = 0, n = 1, \dots, n = k$ 成立, 并考虑一个集合 S 满足 $|S| = k + 1$. 因为 $|S| = k + 1$, 集合 S 至少有一个元素 $\hat{s} \in S$. 归纳假设适用于 $S - \{\hat{s}\}$, 因此 $\mathcal{P}(S - \{\hat{s}\})$ 有 2^k 个元素. 如上所示, $\mathcal{P}(S)$ 中不包含 $\hat{s}$ 的元素与 $\mathcal{P}(S)$ 中包含 $\hat{s}$ 的元素之间存在双射. 但是 $\mathcal{P}(S)$ 中不包含 $\hat{s}$ 的集合正是 $\mathcal{P}(S - \{\hat{s}\})$ 中的集合. 因此, $\mathcal{P}(S)$ 的元素总数是 $2 \cdot 2^k$, 等于 2^{k+1} .

II.3.28 假设 $f(s_i) = H$. 我们将论证 $s_i \in H$ 和 $s_i \notin H$ 都不成立. 如果 $s_i \in H$, 那么学生 s_i 是自己名单上的一员, 因此根据谦逊的定义, s_i 不谦逊, 这与此情形的假设矛盾. 另一方面, 如果 $s_i \notin H$, 那么 s_i 不在自己名单上, 因此 s_i 是谦逊的, 这也与此情形的假设矛盾.

II.3.29 假设不然, 即 $\mathcal{C}$ 是所有集合的集合. 那么 $\mathcal{P}(\mathcal{C})$ 将是一个集合的集合, 因此我们将有 $\mathcal{P}(\mathcal{C}) \subseteq \mathcal{C}$. 这将导致 $|\mathcal{P}(\mathcal{C})| \leq |\mathcal{C}|$, 从而与 Cantor 定理矛盾.

II.3.30 一种方法是每次取两个对角线位; 例如将 01 改为 10, 并将任何其他两位序列改为 01.

II.3.31 在原型 $1.000\dots = 0.999\dots$ 中, 我们将两种表示写作 $x_0 = 1, x_{-1} = 0, x_{-2} = 0, \dots$, 和 $y_0 = 0, y_{-1} = 9, y_{-2} = 9, \dots$. 注意, 这些下标 (即小数位) 是像 0 和 -1 这样的整数, 并非自然数.

(A) 在 $a + ar + ar^2 + ar^3 + \dots = a/(1-r)$ 中, 令 $a = 9$ 且 $r = 0.1$. 左边是 $9 + 0.9 + 0.09 + 0.009 + \dots =$

9.999 99... 右边是 $9/(1 - (0.1)) = 9/0.9 = 10$ 。两边减去 9 可得 $0.999 99 \dots = 1$ 。

- (B) 考虑一个有两种表示的实数。在这两种表示中存在一个最左边的非零小数位 $N \in \mathbb{Z}$ 。因此，将这两种表示写为 $\vec{x} = \langle x_N, x_{N-1}, \dots \rangle$ 和 $\vec{y} = \langle y_N, y_{N-1}, \dots \rangle$ 。这些表示需要满足条件 $x_i, y_i \in \{0, 1, \dots, 9\}$ ，并且需要满足它们表示同一个数。

$$\sum_{i \leq N} x_i \cdot 10^i = \sum_{i \leq N} y_i \cdot 10^i \quad (*)$$

将 (*) 改写成差值形式。

$$0 = \left(\sum_{i \leq N} x_i \cdot 10^i \right) - \left(\sum_{i \leq N} y_i \cdot 10^i \right) = \sum_{i \leq N} (x_i - y_i) \cdot 10^i$$

设 $\vec{x}$ 和 $\vec{y}$ 首次不同的小数位是第 M 位，即当 $i \in \{N, N-1, \dots, M+1\}$ 时 $x_i = y_i$ ，且 $x_M \neq y_M$ 。 x_M 和 y_M 中必有一个较大；不妨设 $x_M > y_M$ 。去掉开头的零，现在差值从第 M 位开始。

$$0 = \sum_{i \leq M} (x_i - y_i) \cdot 10^i = (x_M - y_M) \cdot 10^M + \sum_{i \leq (M-1)} (x_i - y_i) \cdot 10^i \quad (**)$$

我们在右边将第一项单独提出，因为我们需要证明 $x_M = y_M + 1$ （正如原型中 $\vec{x} = \langle 1, 0, 0, \dots \rangle$ 和 $\vec{y} = \langle 0, 9, 9, \dots \rangle$ 的情况那样）。考虑尾部， $\sum_{i \leq (M-1)} (x_i - y_i) \cdot 10^i$ 。为了使总和为零，这个尾部必须抵消 $(x_M - y_M) \cdot 10^M$ ，由于 $x_M > y_M$ ，这是一个正值。尾部所能提供的最大抵消量出现在每个 $x_i - y_i$ 都等于 -9 时。应用几何级数公式可得该最大值。

$$-9 \cdot 10^{M-1} - 9 \cdot 10^{M-2} + \dots = -9 \cdot 10^{M-1} \cdot \left[1 + \frac{1}{10} + \frac{1}{100} + \dots \right] = \frac{-9 \cdot 10^{(M-1)}}{1 - (1/10)} = \frac{-9 \cdot 10^{(M-1)}}{9/10} = -10^M$$

因此 $x_M - y_M = 1$ ，因为尾部无法抵消更大的差值。

- (C) 再次考虑上一项的 (**)，既然我们已经证明了 $x_M - y_M = 1$ 。

$$0 = 1 \cdot 10^M + \sum_{i \leq (M-1)} (x_i - y_i) \cdot 10^i$$

我们要证明每个 $x_i - y_i$ 都等于 -9 。我们只处理 $i = M-1$ 的情况，因为其他情况与此非常类似。

如果 $x_{M-1} - y_{M-1} \geq -8$ ，那么 (**) 变为：

$$\begin{aligned} 0 &= 1 \cdot 10^M + (x_{M-1} - y_{M-1}) \cdot 10^{(M-1)} + \sum_{i \leq (M-2)} (x_i - y_i) \cdot 10^i \\ &\geq 10 \cdot 10^{(M-1)} - 8 \cdot 10^{(M-1)} + \sum_{i \leq (M-2)} (x_i - y_i) \cdot 10^i \\ &= 2 \cdot 10^{(M-1)} + \sum_{i \leq (M-2)} (x_i - y_i) \cdot 10^i \end{aligned}$$

与上一项类似，为了使整个和为负，尾部必须抵消第一项。通过令每个 $x_i - y_i$ 为 -9 ，可以得到最大的尾部。再次应用无限几何级数的求和公式。

$$-9 \cdot 10^{M-2} - 9 \cdot 10^{M-3} + \dots = -9 \cdot 10^{M-2} \cdot \left[1 + \frac{1}{10} + \frac{1}{100} + \dots \right] = \frac{-9 \cdot 10^{(M-2)}}{1 - (1/10)} = \frac{-9 \cdot 10^{(M-2)}}{9/10} = -10^{M-1}$$

尾部不足以抵消 $2 \cdot 10^{(M-1)}$ ，所以 $x_{M-1} - y_{M-1} \geq -8$ 是不可能的。

(A) 这些是 S 中元素的链。

$s \in S$	Chain
0	... 0, a, 0, a ...
1	... 1, b, 1, b ...
2	... 2, d, 3, c, 2, d ...
3	—与 2 的链相同—

这些是 T 中元素的链。

$t \in T$	Chain
a	... a, 0, a, 0 ...
b	... b, 1, b, 1 ...
c	... c, 2, d, 3, c, 2 ...
d	—与 c 的链相同—

(B) 函数 f 是单射的, 因为如果 $f(s) = f(\hat{s})$, 那么 $s+1 = \hat{s}+1$, 因此 $s = \hat{s}$ 。函数 g 同理也是单射的。

与 $0 \in S$ 关联的链是 $0, 1, 2, 3 \dots$ 它包含两个集合的所有元素, 因此该链是唯一的。它还有一个首元素, 即 T 中不存在 0 的 g -原像。

(C) 如果它们并非已经不相交, 即 $S \cap T \neq \emptyset$, 那么我们可以简单地将 S 的所有元素涂成绿色, 将 T 的所有元素涂成金色, 从而使它们不相交。更精确地说, 考虑集合 $\{0\} \times S$ (即形如 $\langle 0, s \rangle$ 的对) 和集合 $\{1\} \times T$ (即形如 $\langle 1, t \rangle$ 的对)。由于任意两个元素在第一项上不同, 这些集合是不相交的。

显然, 它们对应于原始集合 S 和 T 。因此, 我们可以将结果重新表述为不相交的版本: 如果同时有 $|\{0\} \times S| \leq |\{1\} \times T|$ 和 $|\{1\} \times T| \leq |\{0\} \times S|$, 那么 $|\{0\} \times S| = |\{1\} \times T|$ 。

(D) 首先证明每个元素都在一个唯一的链中。固定某个 $x \in S \cup T$ 。因为我们假设两个集合不相交, 所以作用于 x 的函数运算是确定的, 即如果 $x \in S$, 则对于链我们接下来考虑 $f(x)$; 如果 $x \in T$, 则我们接下来考虑 $g(x)$ 。由此, x 右侧的所有链元素都确定了。而且, 因为两个函数都是单射的, 左侧的所有元素也同样确定了。

关于四种可能性, 一条链要么是有限的 (即会重复), 要么是无限的。如果它重复, 则属于情况 (i)。如果它是无限的, 那么它要么没有起始元素 (因此属于情况 (ii)), 要么有。如果它有起始元素, 那么该元素要么来自集合 S (如情况 (iii)), 要么来自集合 T (如情况 (iv))。

(E) 下面我们将分别考虑每种链类型的对应。但首先注意, 因为每种情况都是使用函数 f 和 g 的限制来定义 h , 所以在每种情况下 h 都是良定义的。也就是说, 这些映射都不会将同一个输入关联到两个不同的输出。

类型 (i) 的链在经过 n 个元素后重复, $s_0, t_0, s_1, t_1, \dots, s_{n-1}, t_{n-1}, s_n = s_0$ 其中 $n > 0$ 。(它不能经过单个元素后就重复——不可能有 $s_0 = t_0$ ——因为集合 S 和 T 不相交。) 因此, 我们可以取这个循环中的第一个元素为 S 的成员。函数 f 建立如下关联。

$$s_0 \mapsto t_0 \quad s_1 \mapsto t_1, \quad \dots \quad s_{n-1} \mapsto t_{n-1}$$

这显然构成了链中元素之间的一个双射。它显然是满射到集合 $\{t_0, t_1, \dots, t_{n-1}\}$ 的, 并且它是单射的, 因为 t_j 是互异的, 否则链会更短。

类型 (iii) 的链

$$s, f(s), g(f(s)), f(g(f(s))), \dots$$

将偶数索引的项与奇数索引的项相关联。

$$s \mapsto f(s) \quad g(f(s)) \mapsto f(g(f(s))), \dots$$

也就是说, 我们定义 h 的方式是将 S 的成员映射到 T 的成员。显然这是一个双射。

类型 (ii) 的链是类似的。

$$\dots f^{-1}(g^{-1}(s)), g^{-1}(s), s, f(s), g(f(s)), f(g(f(s))), \dots$$

函数 h 遵循 f 的作用建立如下关联。

$$\dots \quad f^{-1}(g^{-1}(s)) \mapsto g^{-1}(s) \quad s \mapsto f(s) \quad g(f(s)) \mapsto f(g(f(s))), \quad \dots$$

这个关联显然也是一个双射。

需要稍加思考的是类型 (iv) 的链。

$$t, g(t), f(g(t)), g(f(g(t))), \dots$$

我们希望对应函数 h 从 S 映射到 T 。考虑如下关联。

$$g(t) \mapsto t \quad g(f(t)) \mapsto f(t), \quad \dots$$

因为函数 g 是单射的，其逆在其值域上有定义。所以，用这些关联来定义 h 得到一个良定义的函数。与上面的链类型类似，这显然也是一个双射。

II.4.6 通用 Turing 机也是一台普通机器，因为它只是列表 $\mathcal{P}_0, \mathcal{P}_1, \dots$ 中的另一台机器。或许你的朋友想问的是：“通用 Turing 机做了哪些非通用机器不做事？”每台机器都做一项工作。有些机器接受单个输入然后将其加倍，有些则将输入解释为两个数然后相加。通用 Turing 机期望两个输入 e 和 x ，并且它的输出与机器 $\mathcal{P}_e$ 在输入 x 上的输出相同，包括当那台机器不停机时，它也不停机。它的通用性在于，任何可以机械产生的输入-输出行为都可以由这一台设备产生。

II.4.7 每一台通用计算机，包括标准的笔记本电脑或手机，本质上都是一台通用 Turing 机。

注：有些观察者反对“通用计算机本质上是 Turing 机”这一主张，理由是笔记本电脑没有无限的内存。这话没错。但同样正确的是，这样的机器可以连接到外部存储器，也许是云存储，也许是允许它在纸带上做标记的设备，从而不受限于内置的 RAM。

有人还会反驳说，物理宇宙中只有那么多基本粒子可以用来制造比特，因此内存终究不是无限的。这或许是真的——它似乎与当前物理学相符，尽管我们也可以反过来担心，在大爆炸变成大挤压（或其他什么情况）之前，我们并没有足够的时间耗尽所有的比特。

然而，在关于 Church 论题的章节中我们曾指出，Turing 机的定义使直观上的可计算概念变得精确。也就是说，“Turing 机”的意义在于，它是思考算法的正确模型；它是一种数学理想化。笔记本电脑是同一概念的物理实例化。

从某种意义上说，它们不能是同一个东西，因为那会是类型错误——一个是“数学对象”，另一个是“物理对象”。同理，木工用的丁字尺是直角这一数学理想的实例化。断言丁字尺不合法，理由是没有谁能造出直角精确到比宇宙中基本粒子数还多的小数位的完美丁字尺，这似乎忘记了我们觉得思考模型有用的原因。实例化的本质就是理想化。

II.4.8 能。如果 $\mathcal{P}_{e_0}$ 和 $\mathcal{P}_{e_1}$ 是通用的，那么给第一台输入 e_1 和 x ，就会产生与第二台完全相同的输入输出行为。同样地，给 $\mathcal{P}_{e_0}$ 输入 e_0 和 x ，该机器的行为将与其自身完全相同。

II.4.9 通用机并不是用自己的状态去模拟另一台机器的状态。相反，它用代码来表示那些状态，并操作这些代码。简而言之，它通过软件进行模拟，因此通用机自身拥有的状态数量与此无关。

II.4.10 是的，根据引理 2.17，每个可计算函数都有无穷多个索引。因此，如果 $\mathcal{P}_e$ 是通用的，那么其关联函数 ϕ_e 就有无穷多个互不相同的索引： $\phi_e = \phi_{e_0} = \phi_{e_1} = \dots$ 。这些机器 $\mathcal{P}_{e_0}, \mathcal{P}_{e_1}, \dots$ 各不相同，但它们具有相同的输入/输出行为，所以都是通用的。

II.4.11 我们用 $f_{i,a}$ 表示将 x_i 固定为值 a 后得到的函数。

(A) 得到的单变量函数是 $f_{0,4}(x_1) = 12 + 4x_1$ 。

(B) 得到的单变量函数是 $f_{0,5}(x_1) = 15 + 5x_1$ 。

(C) 得到的单变量函数是 $f_{1,0}(x_0) = 3x_0$ 。

II.4.12 我们使用 $\hat{f}$ 作为每个函数的名称。

(A) 得到的二元函数是 $\hat{f}(x_1, x_2) = 1 + 2x_1 + 3x_2$ 。

(B) 结果是二元函数 $\hat{f}(x_1, x_2) = 2 + 2x_1 + 3x_2$ 。

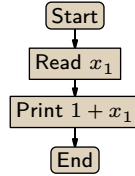
(C) 这给出 $\hat{f}(x_1, x_2) = a + 2x_1 + 3x_2$ 。

(D) 这得到一元函数 $\hat{f}(x_2) = 11 + 3x_2$ 。

(E) 结果是 $\hat{f}(x_2) = a + 2b + 3x_2$ 。

II.4.13 从机器的流程图我们得到关联函数 $\phi_{e_0}(x_0, x_1) = x_0 + x_1$ 。

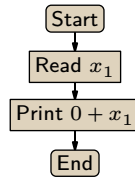
(A) $s_{1,1}$ 的两个下标意味着我们将固定第一个输入，将第二个留作变量。所以 $\phi_{s_{1,1}(e_0,1)}$ 将是一个单输入函数。参数 e_0 和 1 表明我们正在处理 Turing 机 e_0 (即问题陈述中给出流程图的机器)，且输入被固定为 $x_0 = 1$ 。该流程图给出了结果的示意。



该函数为 $\phi_{s_{1,1}(e_0,1)}(x_1) = 1 + x_1$ 。

(B) 其值为 $\phi_{s_{1,1}(e_0,1)}(0) = 1$ 、 $\phi_{s_{1,1}(e_0,1)}(1) = 2$ 和 $\phi_{s_{1,1}(e_0,1)}(2) = 3$ 。

(C) 记号 $s_{1,1}(e_0, 0)$ 意味着我们处理 Turing 机 e_0 ，将第一个输入固定为 $x_0 = 0$ ，并将第二个输入留作变量。该流程图给出了其思想。

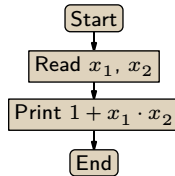


该函数为 $\phi_{s_{1,1}(e_0,0)}(x_1) = 0 + x_1 = x_1$ 。

(D) 我们有 $\phi_{s_{1,1}(e_0,0)}(0) = 0$ 、 $\phi_{s_{1,1}(e_0,0)}(1) = 1$ 和 $\phi_{s_{1,1}(e_0,0)}(2) = 2$ 。

II.4.14 机器的流程图给出关联函数为 $\phi_{e_0}(x_0, x_1, x_2) = x_0 + x_1 \cdot x_2$ 。

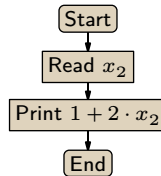
(A) 记号 $s_{1,2}(e_0, 1)$ 表明我们处理 $\mathcal{P}_{e_0}$ ，将第一个输入固定为 $x_0 = 1$ ，并将第二个和第三个输入留作变量。该流程图勾勒了其行为。



该函数为 $\phi_{s_{1,2}(e_0,1)}(x_1, x_2) = 1 + x_1 \cdot x_2$ 。

(B) 我们有 $\phi_{s_{1,2}(e_0,1)}(0, 1) = 1$ 、 $\phi_{s_{1,2}(e_0,1)}(1, 0) = 1$ 和 $\phi_{s_{1,2}(e_0,1)}(2, 3) = 7$ 。

(C) $s_{2,1}(e_0, 1, 2)$ 表示我们从 $\mathcal{P}_{e_0}$ 开始，将第一个输入固定为 $x_0 = 1$ ，第二个输入固定为 $x_1 = 2$ ，并将第三个输入留作变量。该流程图勾勒了结果。



该函数为 $\phi_{s_{2,1}(e_0,1,2)}(x_2) = 1 + 2 \cdot x_2$ 。

(D) 其值为 $\phi_{s_{2,1}(e_0,1,2)}(0) = 1$ 、 $\phi_{s_{2,1}(e_0,1,2)}(1) = 3$ 和 $\phi_{s_{2,1}(e_0,1,2)}(2) = 5$ 。

II.4.15 机器的流程图表明关联函数如下。

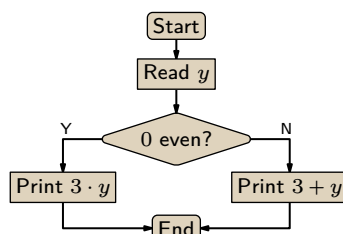
$$\phi_{e_0}(x_0, x_1) = \begin{cases} x_1 & \text{若 } x_0 \leq 1 \\ \uparrow & \text{其他情形} \end{cases}$$

- (A) $\phi_{s_{1,1}(e_0,0)}(x_1) = x_1$
 (B) $\phi_{s_{1,1}(e_0,0)}(5) = 5$
 (C) $\phi_{s_{1,1}(e_0,1)}(x_1) = x_1$
 (D) $\phi_{s_{1,1}(e_0,1)}(5) = 5$
 (E) $\phi_{s_{1,1}(e_0,2)}(x_1) \uparrow$
 (F) $\phi_{s_{1,1}(e_0,2)}(5) \uparrow$

II.4.16 从机器的流程图我们得到，这是相关联的函数：

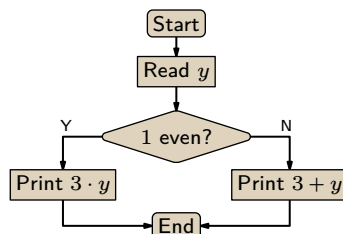
$$\phi_{e_0}(x_0, x_1, y) = \begin{cases} x_1 \cdot y & - \text{若 } x_0 \text{ 为偶数} \\ x_1 + y & - \text{其他情形} \end{cases}$$

(A) 此函数 $\mathcal{P}_{s(e_0,0,3)}$ 有 $x_0 = 0$ ，因此 x_0 是偶数。



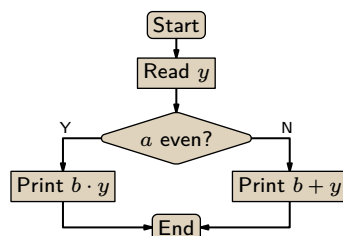
因此该函数是 $\phi_{s(e_0,0,3)}(y) = 3y$ 。于是， $\phi_{s(e_0,0,3)}(5) = 15$ 。

(B) 此函数有 $x_0 = 1$ ，为奇数。下面的流程图概略描述了 $\mathcal{P}_{s(e_0,1,3)}$ 。



计算出的函数是 $\phi_{s(e_0,1,3)}(y) = 3 + y$ ，因而 $\phi_{s(e_0,1,3)}(5) = 8$ 。

(C) 这是概略描述 $\mathcal{P}_{s(e_0,a,b)}$ 的流程图。



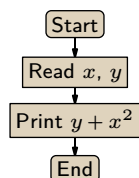
此流程图与问题陈述中的流程图之间的区别在于，这里的机器并不读入 x_0 或 x_1 。 a 和 b 这两个参数是硬编码在程序体中的。这是一个由 a 和 b 参数化的函数族，这些函数的索引可以利用 s - m - n 函数 s ，从 e_0 、 a 和 b 统一地计算出来。

$$\phi_{s(e_0,a,b)}(y) = \begin{cases} b \cdot y & - \text{若 } a \text{ 为偶数} \\ b + y & - \text{其他情形} \end{cases}$$

II.4.17 从由 $\psi(x, y) = y + x^2$ 定义的函数 $\psi: \mathbb{N}^2 \rightarrow \mathbb{N}$ 开始。它直观上是可计算的，因为它可由下面流程图概略描述的机器计算。

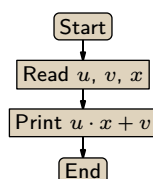
```
;; Return the first character of the file (raise exception if file empty)
(define (first-char fn)
  (let ([contents (port->string (open-input-file fn) #:close? #t)])
    (string-ref contents 0)))
```

图 17, FOR QUESTION II.4.21: 返回文件首字符的 Racket 代码。



依据 Church 论题, 存在一台 Turing 机计算它。设该机器的索引为 e_0 , 即假设 $\psi(x, y) = \phi_{e_0}(x, y)$ 。根据 s - m - n 定理, 存在一个由 n 参数化的可计算函数族 $\phi_{s_{1,1}(e_0, n)}$, 其具有期望的行为 $\phi_{s_{1,1}(e_0, n)}(y) = y + n^2$ 。取 $g(n)$ 为 $s_{1,1}(e_0, n)$ (e_0 不是变量, 而是上述机器的固定索引)。

II.4.18 从函数 $\psi(u, v, x) = ux + v$ 开始。它直观上是可计算的, 如下面流程图概略所示。



依据 Church 论题, 存在一台 Turing 机计算它。设该机器的索引为 e_0 , 即假设 $\psi(u, v, x) = \phi_{e_0}(u, v, x)$ 。 s - m - n 定理给出了一个由 m 和 b 参数化的可计算函数族 $\phi_{s_{2,1}(e_0, m, b)}$, 其具有期望的行为 $\phi_{s_{2,1}(e_0, m, b)}(x) = m \cdot x + b$ 。取 $g(m, b)$ 为 $s_{2,1}(e_0, m, b)$ (注意: e_0 不是变量, 它是来自流程图的 Turing 机的固定索引)。

II.4.19

- (A) 这与 $\phi_{e_1}(5)$ 的结果相同, 即它收敛并给出 20。
- (B) 这将返回 $\text{cantor}(e_0, 5)$ 的值的 4 倍。我们不知道 e_0 的值, 所以最多只能说到这一步。
- (C) 由第一项可知, 这将返回 $\phi_{e_0}(20)$ 。因为 $\text{pair}(20) = \langle 5, 0 \rangle$, 所以这是 $\phi_5(0)$ 的值, 无论其具体是多少。

II.4.20

- (A) 因为 $\text{pair}(4) = \langle 1, 1 \rangle$, 它返回的值与 $\phi_1(1)$ 相同, 无论该值是什么。
- (B) 它与 $\phi_{e_1}(4)$ 的结果相同, 即结果是 6。
- (C) 它返回 $\phi_4(e_1)$ 的值, 无论该值是什么。
- (D) 它返回 $2 + \text{cantor}(e_0, 4)$ 。 $\text{cantor}(e_0, 4)$ 的值依赖于 e_0 , 而我们不知道 e_0 。我们最多只能说 $\text{cantor}(e_0, 4) = e_0 + (e_0 + 4)(e_0 + 5)/2$ 。

II.4.21 图 Figure 17 on page 68 中提供了 Racket 代码。以下是将其运行于自身源代码时的结果。

```
> (first-char "first-char.rkt")
#\#
```

II.4.22 图 Figure 18 on page 69 中提供了 Racket 代码。以下是将其运行于自身源代码时的结果。

```
jim@millstone:~/computing/src/scheme/background$ ./self-modifying.rkt
Counter=11
jim@millstone:~/computing/src/scheme/background$ ls -l self-modifying.rkt
-rw-rw-r-- 1 jim jim 1463 Aug 28 10:17 self-modifying.rkt
jim@millstone:~/computing/src/scheme/background$ chmod u+x self-modifying.rkt
```

```

#!/usr/bin/env racket
#lang racket
(define COUNTER 11)

;; Get list of file lines, where the counter defined in the third line is incremented
(define (intake fn)
  (let* ([contents (port->string (open-input-file fn) #:close? #t)]
        [lines-in (string-split contents "\n")]
        [third-line-in (string-split (third lines-in))]
        [later-lines-in (cdr (cdr (cdr lines-in)))]
        [counter (add1 (string->number (string-trim (third third-line-in) ")))])
    [third-line-out (string-join (list "(define"
                                         "COUNTER"
                                         (number->string counter)
                                         ")")
                                " "
                                #:before-last "")]
    [lines-out (append (list (first lines-in)
                              (second lines-in)
                              third-line-out)
                        later-lines-in)])
  lines-out))

;; Output the strings in a list to the named file
(define (update fn str-list)
  (let ([out-port (open-output-file fn #:exists 'replace #:permissions #o664)])
    ; permit rw-rw-r--
    (write-string (string-join str-list "\n") out-port)
    (close-output-port out-port)))

;; When this file is called from the command line it runs these commands
(update "self-modifying.rkt" (intake "self-modifying.rkt"))
(displayln (~a "Counter=" COUNTER))

```

图 18, FOR QUESTION II.4.22: 自修改的 Racket 代码。

```
jim@millstone:~/computing/src/scheme/background$ ./self-modifying.rkt
Counter=12
```

运行后，源代码的第三行现在显示为 (define COUNTER 12)。

II.5.7 关键在于，我们已经证明了存在一些计算机无法完成的任务，而停机问题就是这类任务的一个具体例子。我们还会看到更多，但这个问题实际上是基准例子。

至于它为何有意义，Turing 机定义的意义在于提供一个运行算法的模型平台。我们正在建模的算法是否会停机，即使给予无界的机会，也是重要且有趣的。

II.5.8 错。这样的函数是存在的，只是这个函数不可计算。也就是说，不存在行为是该函数的 Turing 机。

II.5.9

- (A) 第一个正确。如果我们能找到商店，就能买到面包。
- (B) 第一个正确。如果我们能展示一个因子，就能证明这个数是合数。
- (C) 第二个正确。

II.5.10 不可解性结果并不是说你不能证明某台特定的机器做了某件特定的事。给定的这台机器确实所有输入上都不停机，这确实很明显。相反，这些结果是关于**统一性**的：它们说的是，没有一台机器能够接收任意索引 e 并正确判断 $\mathcal{P}_e$ 是否具有那种行为。

II.5.11 首先，我们不仅要看一个重复的状态-字符对，而是要看一个重复的**格局**，即一个由状态、字符、读写头位置和纸带内容组成的重复元组。

但除此之外，**停机问题** 不仅仅是检测一台机器是否有重复格局的问题。还有其他方式可以导致不停机。这里有一台机器可以永不重复格局地永远运行下去。

$$\mathcal{P} = \{q_0 B1q_0, q_0 1Rq_0\}$$

它不断地写一个 1 然后右移。

II.5.12 当然，我们可以让一个运行时环境做到这一点。（尽管监测设备可能必须比笔记本电脑更大。）

但**停机问题** 并不是说我们不能判断程序 e 在输入 x 下是否在像这样的计算机上停机。它适用于具有**潜在无界资源**的机器类。在这样的机器上，计算并不局限于仅有限多个格局。下面是一个例子，这段代码只是不断地加 1。

```
;; Run forever without repeating a configuration
(define (go-up)
  (do ([i 0 (add1 i)])
      (#f (displayln "routine halt"))
      (displayln (~a "i=" i))))
```

在一个无界设备上，上述运行时环境将不会标记由这段代码产生的计算

```
> i=0
> i=1
> i=2
> :
```

（在这位学生的计算机上，这个 Racket 程序最终会溢出内存，从而以这种方式重复一个格局）。

II.5.13 不是，这不是一个不可解的问题。要么 f 在所有输入上都停机，要么并非如此。只有两种可能。因此，就像书中讨论 **停机问题** 时那样，总体而言，这个问题可由某个计算机程序解决——要么是那个对任何输入都打印 ‘yes’ 的程序，要么是那个打印 ‘no’ 的程序。（这可能令人有些不安：虽然我们不知道两个判定程序中哪一个适用，但两者都是可计算的。）

II.5.14

- (A) 错。这样的函数是存在的。准确地说，问题不可解是因为不存在这样的机械可计算函数。
- (B) 错。Church 论题断言 Turing 机是能力最强的计算模型。不可解问题的存在是有效计算固有的。

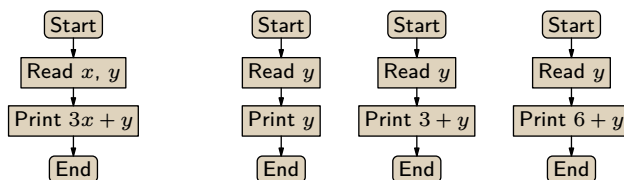


图 19, FOR QUESTION II.5.16: 与函数 $f = \phi_{e_0}$ 关联的机器, 以及 $x = 0$ 、 $x = 1$ 、 $x = 2$ 时与 $\phi_{s(e_0, x)}$ 对应的机器。

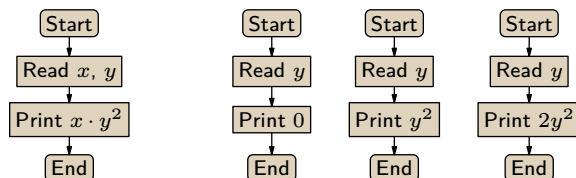


图 20, FOR QUESTION II.5.16: 与函数 $f = \phi_{e_0}$ 关联的机器, 以及 $x = 0$ 、 $x = 1$ 、 $x = 2$ 时与 $\phi_{s(e_0, x)}$ 对应的机器。

II.5.15 任何有限集都是可计算的, 所以答案是肯定的, 它是可计算的。将“可计算”与“可知”混为一谈, 这其中存在微妙之处。

II.5.16

- (A) 函数 $f(x, y) = 3x + y$ 是直观上可计算的, 如 Figure 19 on page 71 左侧所示。由 Church 论题可知, 存在一台具有该行为的 Turing 机; 令该机器的索引为 e_0 , 使得 $f(x, y) = \phi_{e_0}(x, y)$ 对所有 $x, y \in \mathbb{N}$ 都成立。应用 s - m - n 定理对 x 进行参数化, 得到这个可计算函数族: $\phi_{s(e_0, x)}(y) = 3x + y$, 对任意 x , 它都是一个单输入函数。一些相关机器的流程图草图见 Figure 19 on page 71 的右侧。
- (B) 函数 $f(x, y) = xy^2$ 是直观上可计算的, 如 Figure 20 on page 71 所示。由 Church 论题可知, 存在一台具有该行为的 Turing 机 $\mathcal{P}_{e_0}$, 使得 $f(x, y) = \phi_{e_0}(x, y)$ 对所有 $x, y \in \mathbb{N}$ 都成立。应用 s - m - n 定理对 x 进行参数化, 得到 $\phi_{s(e_0, x)}(y) = xy^2$, 这是一个单输入函数族。一些相关机器的流程图草图见 Figure 20 on page 71 的右侧。
- (C) 函数 f 是直观上可计算的, 如 Figure 21 on page 72 左侧所示。由 Church 论题可知, 存在一台具有该行为的 Turing 机 $\mathcal{P}_{e_0}$, 使得 $f(x, y) = \phi_{e_0}(x, y)$ 对所有 $x, y \in \mathbb{N}$ 都成立。应用 s - m - n 定理对 x 进行参数化。得到的单输入函数族如下。

$$\phi_{s(e_0, x)}(y) = \begin{cases} x & \text{若 } x \text{ 为奇数} \\ 0 & \text{其他情形} \end{cases}$$

相关机器的流程图草图见 Figure 21 on page 72 的右侧。

II.5.17 三个问题的答案是相同的。让一台 Turing 机运行固定的步数是可判定的 (在第三项中, 对于任何给定的输入 e , 步数也是固定的)。

II.5.18 我们将证明这个函数不是机械可计算的。

$$\text{total_decider}(e) = \begin{cases} 1 & \text{若 } \phi_e(y) \downarrow \text{ for all } y \\ 0 & \text{其他情形} \end{cases}$$

如 Figure 22 on page 72 左侧的流程图所示, 函数

$$\psi(x, y) = \begin{cases} 42 & \text{若 } \phi_x(x) \downarrow \\ \uparrow & \text{其他情形} \end{cases}$$

在直觉上是可计算的。因此, Church 论题表明存在一台计算它的 Turing 机; 令该机器的索引为 e_0 , 使得 $\psi(x, y) = \phi_{e_0}(x, y)$ 。 s - m - n 定理给出了一个由 x 参数化的单输入函数族。此类机器的草图如右

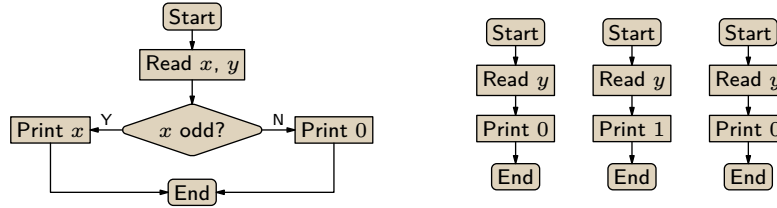


图 21, FOR QUESTION II.5.16: 函数 $f = \phi_{e_0}$ 的机器草图, 以及 $x = 0$ 、 $x = 1$ 、 $x = 2$ 时与 $\phi_{s(e, x_0)}$ 关联的机器草图。

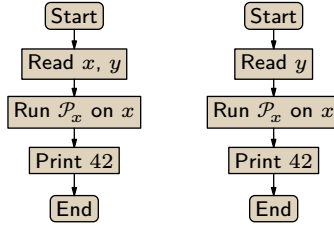


图 22, FOR QUESTION II.5.18: 与函数 $\psi = \phi_e$ 关联的机器, 以及 $\phi_{s(e, x)}$ 对应的机器。

图所示。注意, $\phi_x(x) \downarrow$ 当且仅当 $\text{total_decider}(s(e_0, x)) = 1$ 。

由于 s 是可计算的, 如果我们能机械地计算 total_decider , 那么我们就机械地解决 **停机** 问题。我们无法机械地解决 **停机** 问题, 因此我们无法机械地计算 total_decider 。

II.5.19 我们将证明这个函数不是机械可计算的。

$$\text{square_decider}(e) = \begin{cases} 1 & \text{若 } \phi_e(y) = y^2 \text{ for all } y \\ 0 & \text{其他情形} \end{cases}$$

首先考虑这个函数。

$$\psi(x, y) = \begin{cases} y^2 & \text{若 } \phi_x(x) \downarrow \\ \uparrow & \text{其他情形} \end{cases}$$

Figure 23 on page 73 左侧的流程图表明 ψ 在直觉上是可计算的。因此, 根据 Church 论题, 存在一台 Turing 机计算它。令该机器的索引为 e_0 。应用 s - m - n 定理得到一个由 x 参数化的函数族。对应的机器如右图所示。那么, $\phi_x(x) \downarrow$ 当且仅当 $\text{square_decider}(s(e_0, x)) = 1$ 。

根据这个等价性, 由于函数 s 是可计算的, 如果我们能机械地计算 square_decider , 那么我们就机械地解决 **停机** 问题。但我们无法机械地解决 **停机** 问题, 因此也就无法机械地计算 square_decider 。

II.5.20 我们将证明这是不可计算的。

$$\text{same_value_decider}(e) = \begin{cases} 1 & \text{若 } \phi_e(y) = \phi_e(y+1) \text{ for some } y \\ 0 & \text{其他情形} \end{cases}$$

如 Figure 24 on page 73 左侧的流程图所示, 函数

$$\psi(x, y) = \begin{cases} 42 & \text{若 } \phi_x(x) \downarrow \\ \uparrow & \text{其他情形} \end{cases}$$

在直觉上是可计算的。因此, 根据 Church 论题, 存在一台 Turing 机计算 ψ ; 令该机器的索引为 e_0 , 使得 $\psi(x, y) = \phi_{e_0}(x, y)$ 。应用 s - m - n 定理得到一个由 x 参数化的函数族, 即 $\phi_{s(e_0, x)}$, 其关联机器的

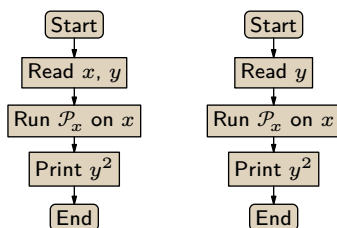


图 23, FOR QUESTION II.5.19: 与函数 $\psi = \phi_{e_0}$ 关联的机器, 以及与 $\phi_{s(e_0, x)}$ 关联的机器。

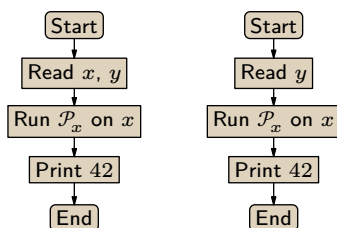


图 24, FOR QUESTION II.5.20: 与函数 $\psi = \phi_{e_0}$ 关联的机器, 以及 $\phi_{s(e_0, x)}$ 对应的机器。

草图如右图所示。当且仅当机器能通过中间方框时, 这些机器输出的函数才具有常数值 42。也就是说, $\phi_x(x) \downarrow$ 当且仅当 $\text{same_value_decider}(s(e_0, x)) = 1$ 。

由于函数 s 是可计算的, 根据上一句话, 如果我们能机械地计算 $\text{same_value_decider}$, 那么我们就机械地解决 **停机问题**。但我们无法做到这一点。

II.5.21 我们将说明此函数不可计算; 没有 Turing 机具有此输入-输出行为。

$$\text{diverges_on_five_decider}(e) = \begin{cases} 1 & \text{若 } \phi_e(5) \uparrow \\ 0 & \text{其他情形} \end{cases}$$

函数

$$\psi(x, y) = \begin{cases} 42 & \text{若 } \phi_x(x) \downarrow \\ \uparrow & \text{其他情形} \end{cases}$$

可以通过 Figure 25 on page 74 左侧的流程图直观地计算。因此 Church 论题断言, 存在一台计算它的 Turing 机。设该机器的索引为 e_0 , 即 $\psi(x, y) = \phi_{e_0}(x, y)$ 。s-m-n 定理给出一族以 x 为参数的函数, 其机器在同一图的右侧以草图形式给出。那么, $\phi_x(x) \downarrow$ 当且仅当 $\text{diverges_on_five_decider}(s(e_0, x)) = 0$ 。(我们也可以这样表述: $\phi_x(x) \downarrow$ 当且仅当 $1 - \text{diverges_on_five_decider}(s(e_0, x)) = 1$ 。)由于函数 s 是机械可计算的, 这意味着, 如果我们能机械地计算 $\text{diverges_on_five_decider}$, 我们就机械地解决停机问题。但我们不能机械地解决停机问题, 因此我们不能机械地计算 $\text{diverges_on_five_decider}$ 。

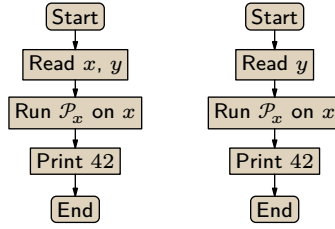
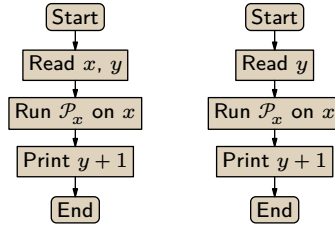
II.5.22 我们要说明此函数不可计算。

$$\text{diverge_on_odds_decider}(e) = \begin{cases} 1 & \text{若 } \phi_e(y) \uparrow \text{ for all odd } y \in \mathbb{N} \\ 0 & \text{其他情形} \end{cases}$$

前一项中给出的论证在这里同样适用。

II.5.23 我们将说明没有 Turing 机具有此输入-输出行为。

$$\text{successor_decider}(e) = \begin{cases} 1 & \text{若 } \phi_e(y) = y + 1 \text{ for all } y \in \mathbb{N} \\ 0 & \text{其他情形} \end{cases}$$

图 25, FOR QUESTION II.5.21: 与 $\psi = \phi_{e_0}$ 关联的机器, 以及计算 $\phi_{s(e_0, x)}$ 的机器。图 26, FOR QUESTION II.5.23: 与 $\psi = \phi_{e_0}$ 关联的机器, 以及与 $\phi_{s(e_0, x)}$ 关联的机器。

函数

$$\psi(x, y) = \begin{cases} y + 1 & \text{— 若 } \phi_x(x) \downarrow \\ \uparrow & \text{— 其他情形} \end{cases}$$

可以通过 Figure 26 on page 74 左侧的流程图直观地计算。因此 Church 论题断言, 存在一台计算它的 Turing 机。设该机器的索引为 e_0 , 即 $\psi(x, y) = \phi_{e_0}(x, y)$ 。s-m-n 定理给出一族以 x 为参数的函数, 其机器在该图的右侧以草图形式给出。那么, $\phi_x(x) \downarrow$ 当且仅当 $\text{successor_decider}(s(e_0, x)) = 1$ 。函数 s 是机械可计算的, 因此, 如果我们能机械地计算 successor_decider , 我们就能机械地解决停机问题。但我们无法做到这一点, 因此我们不能机械地计算 successor_decider 。

II.5.24 我们将说明没有 Turing 机具有此输入-输出行为。

$$\text{diverges_on_twice_input_decider}(e) = \begin{cases} 1 & \text{— when for some } y \in \mathbb{N}, \text{ both } \phi_e(y) \uparrow \text{ and } \phi_e(2y) \uparrow \\ 0 & \text{— 其他情形} \end{cases}$$

函数

$$\psi(x, y) = \begin{cases} 42 & \text{— 若 } \phi_x(x) \downarrow \\ \uparrow & \text{— 其他情形} \end{cases}$$

可以通过 Figure 27 on page 75 左侧的流程图直观地计算。因此 Church 论题断言, 存在一台计算它的 Turing 机。设该机器的索引为 e_0 。应用 s-m-n 定理, 得到一族以 x 为参数的函数, 其机器在图的右侧以草图形式给出。那么, $\phi_x(x) \downarrow$ 当且仅当 $\text{diverges_on_twice_input_decider}(s(e_0, x)) = 0$ 。(我们也可以改写为: $\phi_x(x) \downarrow$ 当且仅当 $1 - \text{diverges_on_twice_input_decider}(s(e_0, x)) = 1$ 。)因此, 如果我们能有效地计算 $\text{converges_on_twice_input_decider}$, 我们就能有效地解决停机问题。但我们不能有效地解决停机问题, 所以我们不能有效地计算 $\text{converges_on_twice_input_decider}$ 。

II.5.25 第二个可解: 只需运行机器一千步。第一个不可解。证明可以用前面练习的填空形式给出: (1) `halts_on_153`, (2) $\phi_e(153) \downarrow$, (3) 42, (4) `Print 42`。

II.5.26 这个问题可解。因为 x 和 y 不可能大于 c , 我们只需写一个程序, 输入 $\text{cantor}(x, y)$ (其中 $0 \leq x, y \leq c$), 应用解配对函数得到 x 和 y , 然后代入 $ax + by$, 看结果是否等于 c 。这是一个有界搜索, 它会停机。

II.5.27

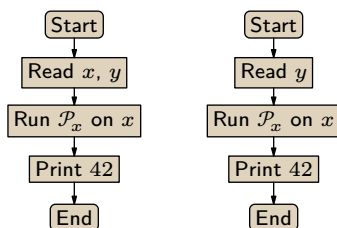


图 27, FOR QUESTION II.5.24: 计算 $\psi = \phi_{e_0}$ 的机器以及计算 $\phi_{s(e_0, x)}$ 的机器的概略图。

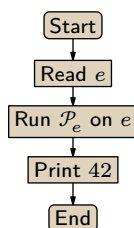


图 28, FOR QUESTION II.5.30: 一个定义域不可判定的单台机器。

- (A) 不可解。(验证类似于本节其他练习。)
- (B) 不可解。(验证与本组许多其他练习类似。)
- (C) 无法判断；这取决于 Turing 机编号的具体方式，即取决于 $\mathcal{P}_4$ 的细节。
- (D) 可解。从 e 我们可以确定五元组集合 $\mathcal{P}_e$ 。它是有限的，所以我们可以直接搜索所需的指令。

II.5.28

- (A) (1) `same_output_as_input`, (2) 存在 $y \in \mathbb{N}$ 使得 $\phi_e(y) = y$, (3) y , (4) `Print y`。
- (B) (1) `outputs_42`, (2) 存在 $y \in \mathbb{N}$ 使得 $\phi_e(y) = 42$, (3) 42, (4) `Print 42`。
- (C) (1) `ever_adds_2`, (2) 存在 $y \in \mathbb{N}$ 使得 $\phi_e(y) = y + 2$, (3) $y + 2$, (4) `Print y + 2`。

II.5.29 假设我们能计算 K_0 。即假设存在 Turing 机 $\mathcal{P}_{e_0}$ ，使得当 $\phi_e(x) \downarrow$ 时 $\phi_{e_0}(\langle e, x \rangle) = 1$ ，当 $\phi_e(x) \uparrow$ 时 $\phi_{e_0}(\langle e, x \rangle) = 0$ 。那么我们就解决停机问题，就能判定 K 的成员资格，只需将 $\mathcal{P}_{e_0}$ 应用于输入 $\langle e, e \rangle$ 。

II.5.30

- (A) 这样一台机器以 e 为输入，使用一台通用 Turing 机在输入 e 上运行 $\mathcal{P}_e$ ，然后退出。Figure 28 on page 75 给出了它的流程图。要说明其对应可计算函数的定义域不是机械可计算的，只需注意：如果我们能计算其定义域，就能计算出停机问题 停机问题的解。
- (B) 对于任意 Turing 机 $\mathcal{P}_e$ 和任意固定的输入 y ，答案要么是“是的，它在 y 上停机”，要么是“不，它不停机”。我们可以写出两个程序，各对应一个答案，其中一个程序是正确的。

II.5.31

- (A) 这是可解的。索引 e 在计算上等价于其源代码，因为我们使用的机器编号是第 68 页所定义的意义下可接受的。所以，给定 e ，首先将其转换为一组 Turing 机指令，即 $\mathcal{P}_e$ 。最后，统计出现在该 Turing 机源代码中的状态数量即可。
- (B) 这是不可解的；函数

$$\text{halts_on_empty_decider}(e) = \begin{cases} 1 & \text{若 } \mathcal{P}_e \text{ halts when started on an empty tape} \\ 0 & \text{其他情形} \end{cases}$$

不是机械可计算的。为此，考虑这个函数。

$$\psi(x) = \begin{cases} 42 & \text{若 } \phi_x(x) \downarrow \\ \uparrow & \text{其他情形} \end{cases}$$

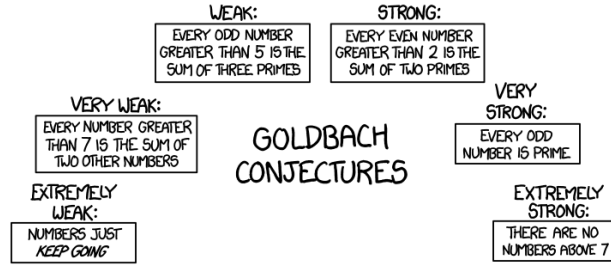


图 29, FOR QUESTION II.5.35: Goldbach 的种种猜想。(致谢 xkcd.com)

它直观上是机械可计算的，因此根据 Church 论题，存在一台计算它的 Turing 机。设该机的索引为 e_0 。应用 s - m - n 定理对 x 进行参数化，得到一个一致可计算的函数族 $\phi_{s(e_0, x)}$ 。（这些 $\phi_{s(e_0, x)}$ 是没有输入的函数；这没问题，因为对应的 Turing 机根本不从纸带上读取任何输入。）

那么， $\phi_x(x) \downarrow$ 当且仅当 $\text{halts_on_empty_decider}(s(e_0, x)) = 1$ 。如果 $\text{halts_on_empty_decider}$ 是机械可计算的，那么我们就机械地解决停机问题 停机问题。因此 $\text{halts_on_empty_decider}$ 不是机械可计算的。

(c) 这是可解的。根据索引 e ，使用一台通用 Turing 机在输入 e 上运行 $\mathcal{P}_e$ 一百步。到那时，它要么已经停机，要么没有。

II.5.32 是。因为如果 K 是有限的，那么它就是可计算的，这是由于任何有限集都是可计算的。例如，对于一个有限集，我们可以用一串 if-then-else 语句来判定一个元素是否属于它。

II.5.33 假；它的确是无限的，但它是不可数无限的。存在不可数无限多个问题，也就是从 $\mathbb{N}$ 到 $\mathbb{N}$ 的函数。但是 Turing 机只有可数多台，因此可解问题也只有可数个。所以，不可解问题有不可数多个。

II.5.34 要么 $\mathcal{P}_e$ 确实在所有输入上都停机，要么不是。因此只有两种可能，而每种情况下的答案都是可计算的（答案要么是“是”，要么是“否”）。

注意，这与下述不可解问题形成对比：给定 $e \in \mathbb{N}$ ，判定 $\mathcal{P}_e$ 是否在所有输入上都停机。按本题的提法，并不寻求一个对索引 e 一致的算法。

II.5.35 设想一台 Turing 机，它对任何输入都忽略之，只是循环运行，搜索一个大于二但不是两个素数之和的偶数。如果这台 Turing 机停机了（即找到了反例），那么该猜想就是假的。否则，该猜想就是真的。因此，如果我们能解决停机问题，就可以将它应用于这台机器，从而判定这个问题。

XKCD 对 Goldbach 的幽默解读参见 Figure 29 on page 76。

II.5.36 我们可以写一个程序，它接收输入，忽略它，然后执行一个循环，搜索大于 7 的解，一旦找到就停机。只要有了停机问题的解法，就能检查那个程序是否会停机，从而判定这个问题。

II.5.37 所有函数 $f: \mathbb{N} \rightarrow \mathbb{N}$ 的集合是两个子集的不交并：可计算的函数和不可计算的函数。我们知道可计算函数的集合是可数的。因为两个可数集的并集是可数的，所以如果不可计算函数的集合也是可数的，那么总体上就只有可数多个函数，但事实并非如此。因此，不可计算函数的集合是不可数的。

II.5.38 固定一个元素 $k \in K$ 。以下函数 $f: \mathbb{N}^2 \rightarrow \mathbb{N}$ 的值域是 K 。

$$f(x, n) = \begin{cases} x & \text{若 } \mathcal{P}_x \text{ with input } x \text{ halts in } n \text{ steps} \\ k & \text{其他情形} \end{cases}$$

显然该函数是可计算的，并且对所有输入对 $x, n \in \mathbb{N}^2$ 都收敛。对于 K 的所有元素，都存在一个 n 使得 x 出现在值域中。并且，基于同样的原因，只有 K 中的元素会出现在值域中。

II.5.39 设有两个可判定语言 $\mathcal{L}_0$ 和 $\mathcal{L}_1$ 。因为它们可判定，所以存在 Turing 机 $\mathcal{P}_{e_0}$ 和 $\mathcal{P}_{e_1}$ 来判定成员资格。

$$\phi_{e_0}(x) = \begin{cases} 1 & \text{若 } x \in \mathcal{L}_0 \\ 0 & \text{若 } x \notin \mathcal{L}_0 \end{cases} \quad \phi_{e_1}(x) = \begin{cases} 1 & \text{若 } x \in \mathcal{L}_1 \\ 0 & \text{若 } x \notin \mathcal{L}_1 \end{cases}$$

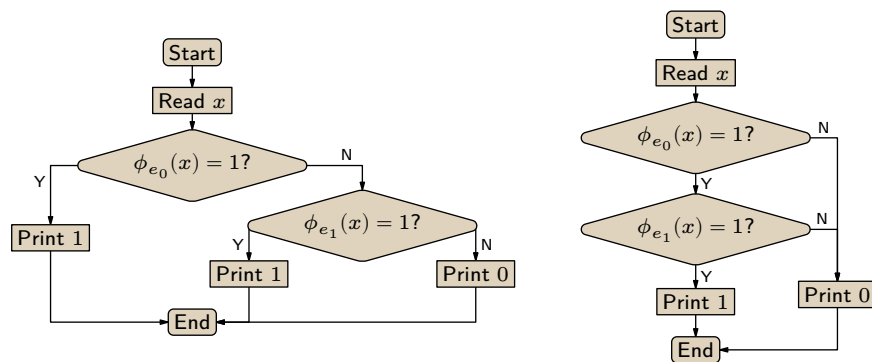


图 30, FOR QUESTION II.5.39: 判定可判定语言并与交的机器示意图。

注意，这些机器在所有输入上都停机。

- (A) 判定 $\mathcal{L}_0 \cup \mathcal{L}_1$ 成员资格的机器示意图见 Figure 30 on page 77 左侧。一个数如果属于其中一个集合，就属于并集。因此，给定 x ，该机器首先在输入 x 上运行 $\mathcal{P}_{e_0}$ 。如果返回 1，那么 x 属于并集，于是新机器输出 1。否则，新机器在输入 x 上运行 $\mathcal{P}_{e_1}$ 并返回结果。
- (B) 判定 $\mathcal{L}_0 \cap \mathcal{L}_1$ 成员资格的机器示意图在同一图的右侧。一个数如果同时属于两个集合，就属于交集。因此，给定 x ，该机器首先在输入 x 上运行 $\mathcal{P}_{e_0}$ 。如果返回 0，那么 x 不属于交集，于是新机器输出 0。否则，新机器在输入 x 上运行 $\mathcal{P}_{e_1}$ 并返回结果。
- (C) 补运算是直接的。对于 $\mathcal{L}_0$ 的补集，新机器只需运行 $\mathcal{P}_{e_0}$ ，并用 1 减去结果。

II.6.10 Rice 定理并没有说所有事情都是不可能的。例如，它并没有说编写一个实现 $f(x) = x^2$ 的例程是不可能的。

它所断言不可能的是：编写一个例程，在给定源代码后，能够正确判定这段源代码（无论写得多么巧妙，或故意写得多么晦涩）是否实现了 f 。（这种判定器之所以有意义，是因为我们可以把它应用到第一段提到的那个例程上，从而确切地知道它在所有情况下都能正确工作。）

II.6.11 不是，它不是索引集。我们可以写出两台行为相同的 Turing 机，例如，对任何输入 x ，它们都输出 x 。其中一台只需几步就能做到这一点（平凡的 Turing 机 $\mathcal{P} = \{\}$ 在零步内即可完成），而另一台需要 101 步。第二台机器的索引属于 $\mathcal{I}$ ，但第一台机器的索引不属于。

II.6.12 使得 $\emptyset \subseteq \{x \mid \phi_e(x) \downarrow\}$ 成立的索引 $e \in \mathbb{N}$ 的集合是整个 $\mathbb{N}$ 。这个问题是可解的，尽管解是平凡的：对任何 e ，答案都是“是”。

II.6.13

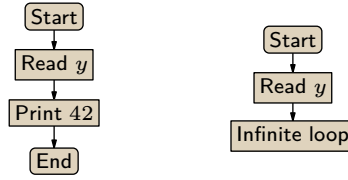
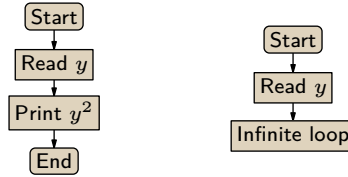
- (A) 真，这是关于机器的输入/输出行为的。如果两台机器具有相同的输入/输出行为，并且其中一台在输入 5 上停机，那么另一台也必然如此。
- (B) 假，我们可以造出两台行为相同，但其中一台有四条指令而另一台没有的机器。例如，我们可以取一台有四条指令的机器，然后通过添加不可达代码来构造一台新机器，这些无实际作用的指令起始于原四条指令未曾引用过的状态。
- (C) 真。如果两台机器计算相同的函数，并且第一台机器的函数是 $x \mapsto 2x$ ，那么第二台机器的函数也是如此。

II.6.14 在每种情况下，我们都将论证：存在两台机器 $\mathcal{P}$ 和 $\hat{\mathcal{P}}$ ，它们具有相同的输入/输出行为， $\phi = \hat{\phi}$ ，但其中一台机器具有该性质而另一台没有。

- (A) 平凡的 Turing 机 $\mathcal{P} = \{\}$ 在零步内停机，其行为是 $\phi(x) = x$ 。我们可以轻易地造出另一台具有相同行为，但耗费超过一百步的机器。

$$\hat{\mathcal{P}} = \{q_0 \text{BB} q_1, q_0 \text{11} q_1, q_1 \text{BB} q_2, q_1 \text{11} q_2, \dots, q_{100} \text{BB} q_{101}, q_{100} \text{11} q_{101}\}$$

- (B) 与上一条类似，平凡的机器在输入 0 上使用零个带方格。下面这台机器在输入 0 上访问一百零一

图 31, FOR QUESTION II.6.18: 用于证明集合 $\mathcal{I}$ 非平凡的那些机器。图 32, FOR QUESTION II.6.19: 用于证明平方函数索引集合 $\mathcal{I}$ 非平凡的那些机器。

个带方格。

$$\hat{\mathcal{P}} = \{q_0 \text{BL} q_1, q_0 \text{11} q_{102}, q_1 \text{BL} q_2, q_1 \text{1L} q_2, \dots, q_{100} \text{BL} q_{101}, q_{100} \text{1L} q_{101}\}$$

两台机器的行为相同，都是 $\phi(x) = x$ 。

(c) 平凡的机器不包含任何涉及状态 q_{10} 的指令。下面这台机器

$$\hat{\mathcal{P}} = \{q_0 \text{BB} q_1, q_0 \text{11} q_1, q_1 \text{BB} q_2, q_1 \text{11} q_2, \dots, q_{10} \text{BB} q_{11}, q_{10} \text{11} q_{11}\}$$

具有相同的行为，并且包含一条涉及该状态的指令。

II.6.15 描述空集的方式有很多。一种是 $\{e \mid \mathcal{P}_e \text{ 求解 停机问题}\}$ 。

II.6.16 描述所有索引的集合 $\mathbb{N}$ 的一种方式 $\{e \mid \text{机器 } \mathcal{P}_e \text{ 在输入 3 上停机或不停机}\}$ 。

II.6.17 记法 $\phi_e(x) = y$ 表示 $\phi_e(x)$ 收敛且输出值为 y 。

- (A) $\{e \in \mathbb{N} \mid \phi_e(7) = 7\}$
- (B) $\{e \in \mathbb{N} \mid \phi_e(e) = e\}$
- (C) $\{e \in \mathbb{N} \mid \text{存在 } y \text{ 使 } \phi_{2e}(y) = 7\}$
- (D) $\{e \in \mathbb{N} \mid \phi_e(7) \downarrow \text{ 且其值为素数}\}$

II.6.18 令 $\mathcal{I} = \{e \in \mathbb{N} \mid \phi_e \text{ 是全函数}\}$ 。我们将证明它是一个非平凡的索引集。

首先证明 $\mathcal{I}$ 不是空集。考虑 Figure 31 on page 78 左侧所勾勒的程序。根据 Church 论题，存在一台符合此草图的 Turing 机。它有一个索引，且该索引是 $\mathcal{I}$ 的元素。所以 $\mathcal{I}$ 非空。

接下来证明 $\mathcal{I}$ 不是整个 $\mathbb{N}$ 。考虑同一图右侧所勾勒的程序。根据 Church 论题，存在一台符合此草图的 Turing 机。它的索引不是 $\mathcal{I}$ 的元素，因此 $\mathcal{I}$ 不是整个 $\mathbb{N}$ 。

最后，证明 $\mathcal{I}$ 是一个索引集。假设 $e \in \mathcal{I}$ 且 $\hat{e}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in \mathcal{I}$ ，我们知道对所有的 $a \in \mathbb{N}$ 有 $\phi_e(a) \downarrow$ 。由于 $\phi_e \simeq \phi_{\hat{e}}$ ，我们因此知道对所有的 $a \in \mathbb{N}$ 有 $\phi_{\hat{e}}(a) \downarrow$ 。于是， $\hat{e} \in \mathcal{I}$ ，从而 $\mathcal{I}$ 是一个索引集。

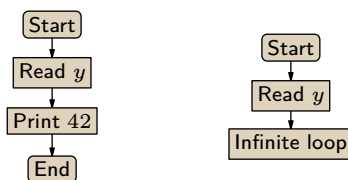
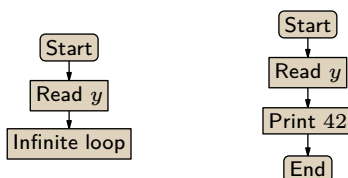
因此，Rice 定理表明，判定“全函数性”的问题，即给定 e 时 Turing 机 $\mathcal{P}_e$ 是否在所有输入上都停机的问題，是不可解的。

II.6.19 令 $\mathcal{I} = \{e \in \mathbb{N} \mid \text{对所有 } y \text{ 均有 } \phi_e(y) = y^2\}$ 。我们将证明它是一个非平凡的索引集。

首先证明 $\mathcal{I}$ 不是空集。考虑 Figure 32 on page 78 左侧所勾勒的程序。根据 Church 论题，存在一台符合此草图的 Turing 机。它有一个索引，且该索引是 $\mathcal{I}$ 的元素。所以 $\mathcal{I}$ 非空。

接下来证明 $\mathcal{I}$ 不是整个 $\mathbb{N}$ 。考虑同一图右侧所勾勒的程序。根据 Church 论题，存在一台符合此草图的 Turing 机。它的索引不是 $\mathcal{I}$ 的元素，因此 $\mathcal{I}$ 不是整个 $\mathbb{N}$ 。

最后，证明 $\mathcal{I}$ 是一个索引集。假设 $e \in \mathcal{I}$ 且 $\hat{e}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in \mathcal{I}$ ，我们知道对所有 $y \in \mathbb{N}$

图 33, FOR QUESTION II.6.20: 用于证明集合 $\mathcal{I}$ 非平凡的机器。图 34, FOR QUESTION II.6.21: 用于证明 $\mathcal{I}$ 非平凡的 Turing 机概述。

有 $\phi_e(y) = y^2$ 。由于 $\phi_e \simeq \phi_{\hat{e}}$ ，我们因此知道对所有 $y \in \mathbb{N}$ 有 $\phi_{\hat{e}}(y) = y^2$ 。于是， $\hat{e} \in \mathcal{I}$ ，从而 $\mathcal{I}$ 是一个索引集。

由此，Rice 定理表明，给定 e 时判定 Turing 机 $\mathcal{P}_e$ 是否计算其输入的平方的问题，是不可解的。

II.6.20 我们将证明 $\mathcal{I} = \{e \in \mathbb{N} \mid \text{存在 } y \in \mathbb{N} \text{ 使 } \phi_e(y) = \phi_e(y+1)\}$ 是一个非平凡的索引集。

首先证明 $\mathcal{I}$ 非空。考虑 Figure 33 on page 79 左侧所勾勒的程序。它在直观上是可计算的，因此根据 Church 论题，存在一台符合此草图的 Turing 机。该 Turing 机的索引是 $\mathcal{I}$ 的一个元素。所以 $\mathcal{I}$ 非空。

接下来论证 $\mathcal{I} \neq \mathbb{N}$ 。考虑同一图右侧所概述的程序。根据 Church 论题，存在一台符合此描述的 Turing 机。它的索引不是 $\mathcal{I}$ 的元素，因此 $\mathcal{I}$ 不是整个 $\mathbb{N}$ 。

最后，证明 $\mathcal{I}$ 是一个索引集。假设 $e \in \mathcal{I}$ ，并且 $\hat{e}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in \mathcal{I}$ ，存在 $y \in \mathbb{N}$ 使得 $\phi_e(y) = \phi_e(y+1)$ 。由于 $\phi_e \simeq \phi_{\hat{e}}$ ，我们知道对于同一个 y ，有 $\phi_{\hat{e}}(y) = \phi_{\hat{e}}(y+1)$ 。因此也有 $\hat{e} \in \mathcal{I}$ 。所以 $\mathcal{I}$ 是一个索引集。

于是，根据 Rice 定理，判定 $\mathcal{I}$ 成员资格的问题是不可解的。

II.6.21 我们将证明 $\mathcal{I} = \{e \in \mathbb{N} \mid \phi_e(5) \uparrow\}$ 是一个非平凡的索引集。

首先证明 $\mathcal{I} \neq \emptyset$ 。考虑 Figure 34 on page 79 左侧所勾勒的程序。它在直观上是可计算的，因此根据 Church 论题，存在一台符合此草图的 Turing 机。该机器的索引是 $\mathcal{I}$ 的一个元素。

接下来论证 $\mathcal{I} \neq \mathbb{N}$ 。考虑同一图右侧所概述的程序。根据 Church 论题，存在一台符合此描述的 Turing 机。它的索引不是 $\mathcal{I}$ 的元素。

最后，证明 $\mathcal{I}$ 是一个索引集。假设 $e \in \mathcal{I}$ ，并且 $\hat{e}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in \mathcal{I}$ ，我们知道 $\phi_e(5) \uparrow$ 。但 $\phi_e \simeq \phi_{\hat{e}}$ ，所以我们也有 $\phi_{\hat{e}}(5) \uparrow$ 。因此， $\hat{e} \in \mathcal{I}$ 。所以 $\mathcal{I}$ 是一个索引集。

由此，Rice 定理表明判定 $\mathcal{I}$ 成员资格的问题是不可解的。

II.6.22 我们将证明 $\mathcal{I} = \{e \in \mathbb{N} \mid \text{对所有奇数 } y \text{ 均有 } \phi_e(y) \uparrow\}$ 是一个非平凡的索引集。

首先证明 $\mathcal{I} \neq \emptyset$ 。考虑 Figure 35 on page 80 左侧所勾勒的程序。它在直观上是可计算的，因此根据 Church 论题，存在一台符合此草图的 Turing 机。该机器的索引是 $\mathcal{I}$ 的一个元素。

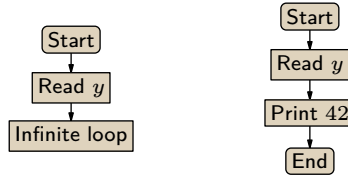
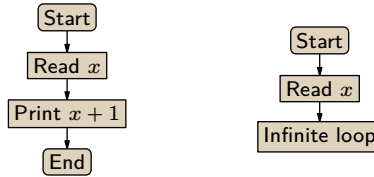
接下来验证 $\mathcal{I} \neq \mathbb{N}$ 。考虑同一图右侧所概述的程序。根据 Church 论题，存在一台符合此描述的 Turing 机。它的索引不是 $\mathcal{I}$ 的元素。

最后，证明 $\mathcal{I}$ 是一个索引集。假设 $e \in \mathcal{I}$ ，并且 $\hat{e}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in \mathcal{I}$ ，我们知道对于所有奇数输入 y ， $\phi_e(y) \uparrow$ 。由于 $\phi_e \simeq \phi_{\hat{e}}$ ，我们也知道对于所有奇数 y ， $\phi_{\hat{e}}(y) \uparrow$ 。因此， $\hat{e} \in \mathcal{I}$ ，所以 $\mathcal{I}$ 是一个索引集。

由此，Rice 定理得出，判定 $\mathcal{I}$ 成员资格的问题是不可解的。

II.6.23 我们将证明 $\mathcal{I} = \{e \in \mathbb{N} \mid \phi_e(x) = x + 1\}$ 是一个非平凡的索引集。

首先证明 $\mathcal{I} \neq \emptyset$ 。考虑 Figure 36 on page 80 左侧的程序草图。它在直观上是可计算的，因此由

图 35, FOR QUESTION II.6.22: 用于证明 $\mathcal{I}$ 非平凡的机器。图 36, FOR QUESTION II.6.23: 用于证明 $\mathcal{I}$ 非平凡的机器。

Church 论题，存在一台符合此草图的 Turing 机。该机器的索引属于 $\mathcal{I}$ 。

接着证明 $\mathcal{I} \neq \mathbb{N}$ 。考虑该图右侧的程序草图。由 Church 论题，存在一台与之相符的 Turing 机。它的索引不属于 $\mathcal{I}$ 。

最后证明 $\mathcal{I}$ 是一个索引集。假设 $e \in \mathcal{I}$ 且 $\hat{e}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in \mathcal{I}$ ，我们知道对所有输入 x 都有 $\phi_e(x) = x + 1$ 。由于 $\phi_e \simeq \phi_{\hat{e}}$ ，我们也知道 $\phi_{\hat{e}}(x) = x + 1$ 对所有 x 成立。因此， $\hat{e} \in \mathcal{I}$ 。所以 $\mathcal{I}$ 是一个索引集。

于是，由 Rice 定理，判定一个索引是否属于 $\mathcal{I}$ 的问题是不可解的。

II.6.24 我们将证明 $\mathcal{I} = \{e \in \mathbb{N} \mid \text{存在 } x \text{ 使 } \phi_e(x) \uparrow \text{ 且 } \phi_e(2x) \uparrow\}$ 是一个非平凡的索引集。

首先证明 $\mathcal{I} \neq \emptyset$ 。考虑 Figure 37 on page 81 左侧的程序草图。它在直观上是可计算的，因此由 Church 论题，存在一台符合此草图的 Turing 机。该机器的索引属于 $\mathcal{I}$ 。

接着论证 $\mathcal{I} \neq \mathbb{N}$ 。考虑 Figure 37 on page 81 右侧的程序草图。由 Church 论题，存在一台与之相符的 Turing 机。它的索引不属于 $\mathcal{I}$ 。

最后证明 $\mathcal{I}$ 是一个索引集。假设 $e \in \mathcal{I}$ 且 $\hat{e}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in \mathcal{I}$ ，我们知道存在某个输入 x ，使得 $\phi_e(x) \uparrow$ 且 $\phi_e(2x) \uparrow$ 。由于 $\phi_e \simeq \phi_{\hat{e}}$ ，我们知道对同一个 x ， $\phi_{\hat{e}}(x) \uparrow$ 且 $\phi_{\hat{e}}(2x) \uparrow$ 。因此， $\hat{e} \in \mathcal{I}$ 。所以 $\mathcal{I}$ 是一个索引集。

于是，由 Rice 定理，判定一个索引是否属于 $\mathcal{I}$ 的问题是不可解的。

II.6.25 第一小题中的索引集是 6.18 中那个索引集的补集。后续练习 6.35 证明，索引集的补集也是索引集。但我们下面不依赖这些练习，直接验证所给问题是不可解的。

(A) 问题是：给定索引 e ，判定是否存在 $y \in \mathbb{N}$ 使得 $\phi_e(y) \uparrow$ 。为了证明该问题不可解，我们将验证 $\mathcal{I} = \{e \in \mathbb{N} \mid \text{存在 } y \in \mathbb{N} \text{ 使 } \phi_e(y) \uparrow\}$ 是一个非平凡的索引集。

集合 $\mathcal{I}$ 非空，因为我们可以写一个在所有输入上都不停机的程序，由 Church 论题知，存在具有该行为的 Turing 机。该机器的索引属于 $\mathcal{I}$ 。

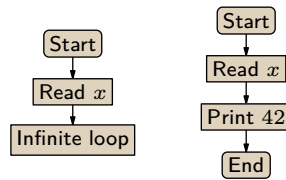
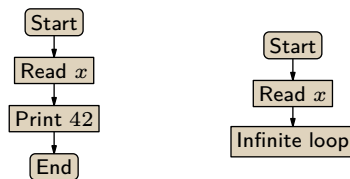
该集合不等于 $\mathbb{N}$ ，因为我们可以写一个返回其输入的程序，由 Church 论题知，存在具有该行为的 Turing 机。该机器的索引不属于 $\mathcal{I}$ 。

最后验证 $\mathcal{I}$ 是一个索引集。假设 $e \in \mathcal{I}$ 且 $\hat{e}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in \mathcal{I}$ ，我们知道存在至少一个 $y \in \mathbb{N}$ 使得 $\phi_e(y) \uparrow$ 。由于两者行为相同， $\phi_{\hat{e}}(y) \uparrow$ 对至少一个 y 也成立。因此 $\hat{e} \in \mathcal{I}$ ，于是 $\mathcal{I}$ 是一个索引集。

(B) 令 $\mathcal{I} = \{e \in \mathbb{N} \mid \text{存在 } y \in \mathbb{N} \text{ 使 } \phi_e(y) \downarrow\}$ 。我们将验证它是一个非平凡的索引集。

集合 $\mathcal{I}$ 非空，因为我们可以写一个在所有输入上都输出 0 的程序。由 Church 论题可知，存在具有该行为的 Turing 机。该机器的索引属于 $\mathcal{I}$ 。

集合 $\mathcal{I}$ 不等于 $\mathbb{N}$ ，因为我们可以写一个在所有输入上都不停机的程序，由 Church 论题，存在具有该行为的 Turing 机，且该机器的索引不属于 $\mathcal{I}$ 。

图 37, FOR QUESTION II.6.24: 用于证明 $\mathcal{I}$ 非平凡的机器。图 38, FOR QUESTION II.6.26: 用于论证 $\mathcal{I}$ 非平凡的机器。

既然 $\mathcal{I}$ 已是非平凡的，最后来验证它是一个索引集。假设 $e \in \mathcal{I}$ 且 $\hat{e}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in \mathcal{I}$ ，我们知道存在至少一个 $y \in \mathbb{N}$ 使得 $\phi_e(y) \downarrow$ 。两者行为相同，因此 $\phi_{\hat{e}}(y) \downarrow$ 对至少一个 y 也成立。因此 $\mathcal{I}$ 是一个索引集。

II.6.26 每个答案仅给出编号对应的填空内容。

- (A) (1) $\phi_e(x) \downarrow$ 对所有是 5 的倍数的输入 x 都成立。(2) 及 (3) 见 Figure 38 on page 81。(4) $\phi_e(5k) \downarrow$ 对所有 $k \in \mathbb{N}$ 成立。(5) $\phi_{\hat{e}}(5k) \downarrow$ 对所有 $k \in \mathbb{N}$ 也成立。
- (B) (1) 存在某个输入 x 使得 $\phi_e(x) = 7$ 。(2) 及 (3) 见 Figure 39 on page 82。(4) 存在某个 $x \in \mathbb{N}$ 使得 $\phi_{\hat{e}}(x) = 7$ 。(5) $\phi_{\hat{e}}(x) = 7$ 对同一个 x 成立。

II.6.27 所有有限集都是可计算的。对于有限集，我们可以很容易地写出一个程序来充当其特征函数；只需考虑有限种情况。（依据 Church 论题，若存在这样的程序，就存在这样的 Turing 机。）

II.6.28

- (A) 我们将证明这是一个非平凡索引集： $\mathcal{I} = \{e \in \mathbb{N} \mid \mathcal{P}_e \text{ 判定一个无限语言}\}$ 。

首先验证 $\mathcal{I}$ 非空。我们可以写一台对所有输入都输出 1 的 Turing 机，这台机器接受 $\mathcal{L} = \mathbb{B}$ 中的每一个串，而 $\mathbb{B}$ 是无限集。因此这台 Turing 机的索引属于 $\mathcal{I}$ 。

其次验证 $\mathcal{I}$ 不等于 $\mathbb{N}$ 。我们可以写一台对所有输入都输出 0 的 Turing 机。这台机器接受空集，而空集不是无限的。因此这台 Turing 机的索引不是 $\mathcal{I}$ 的元素。

最后，为验证 $\mathcal{I}$ 是索引集，假设 $e \in \mathcal{I}$ 且 $\hat{e} \in \mathbb{N}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。由于 $e \in \mathcal{I}$ ， $\mathcal{P}_e$ 判定的语言是无限的。因为 $\phi_e \simeq \phi_{\hat{e}}$ ， $\mathcal{P}_{\hat{e}}$ 输出 1 的比特串集合等于 $\mathcal{P}_e$ 输出 1 的集合，所以 $\mathcal{P}_{\hat{e}}$ 也判定一个无限语言。因此 $\hat{e} \in \mathcal{I}$ ， $\mathcal{I}$ 是索引集。

- (B) 我们将证明 $\mathcal{I} = \{e \in \mathbb{N} \mid \mathcal{P}_e \text{ 接受 } 101\}$ 是一个非平凡索引集。首先， $\mathcal{I}$ 非空，因为我们可以写一台对所有输入都输出 1 的 Turing 机，这台机器接受串 101。该机的索引是 $\mathcal{I}$ 的一个元素。

其次， $\mathcal{I}$ 不等于 $\mathbb{N}$ ，因为我们可以写一台对所有输入都输出 0 的 Turing 机，这台机器不接受串 101。该机的索引不是 $\mathcal{I}$ 的元素。

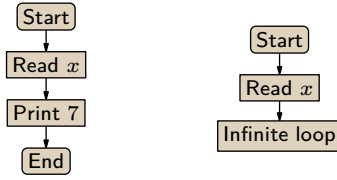
最后，验证 $\mathcal{I}$ 是索引集。假设 $e \in \mathcal{I}$ 且 $\hat{e} \in \mathbb{N}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in \mathcal{I}$ ，机器 $\mathcal{P}_e$ 接受 101。因为 $\phi_e \simeq \phi_{\hat{e}}$ ，机器 $\mathcal{P}_{\hat{e}}$ 也接受 101。因此 $\hat{e} \in \mathcal{I}$ ， $\mathcal{I}$ 是索引集。

II.6.29 我们用 Rice 定理，考虑 $\mathcal{I} = \{e \mid \mathcal{P}_e \text{ 接受每个 } \sigma \in \mathbb{B}^*\}$ 。

首先，我们可以写一台在所有输入上都停机并输出 1 的 Turing 机，这台机器的索引是 $\mathcal{I}$ 的成员，所以 $\mathcal{I}$ 不是空集。

其次，我们可以写一台在所有输入上都停机并输出 0 的 Turing 机，这台机器的索引不是 $\mathcal{I}$ 的成员，所以 $\mathcal{I} \neq \mathbb{N}$ 。

为证明 $\mathcal{I}$ 是索引集，假设 $e \in \mathcal{I}$ 且数 $\hat{e}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in \mathcal{I}$ ，机器 $\mathcal{P}_e$ 接受所有比特串。因为 $\phi_e \simeq \phi_{\hat{e}}$ ，机器 $\mathcal{P}_{\hat{e}}$ 也接受所有比特串。因此 $\hat{e} \in \mathcal{I}$ ， $\mathcal{I}$ 是非平凡索引集。

图 39, FOR QUESTION II.6.26: 用于论证 J 非平凡的机器。

II.6.30 我们可以构造两台 Turing 机 $\mathcal{P}_e$ 和 $\mathcal{P}_{\hat{e}}$, 它们具有相同行为, $\phi_e \simeq \phi_{\hat{e}}$, 但 $e \in J$ 而 $\hat{e} \notin J$ 。下面是两台这样的机器。

$$\mathcal{P}_e = \{q_0 \text{BB}q_0, q_0 11q_0, q_1 \text{BB}q_1, q_1 11q_1\} \quad \mathcal{P}_{\hat{e}} = \{q_0 \text{BB}q_0, q_0 11q_0\}$$

第一台具有不可达状态 q_1 。

II.6.31 这个问题是不可解的。我们无法通过机器是否“继续运行”(即没有立即停止)来判断任何事情。它可能继续做相当复杂的事情然后最终停止, 或者做复杂的事情且永不停止。

更详细地说, 我们可以将 **停机问题** 归约到这个问题。考虑这个 $\psi: \mathbb{N}^2 \rightarrow \mathbb{N}$ 。

$$\psi(x, y) = \begin{cases} 42 & \text{若机器 } \mathcal{P}_x \text{ 在 } x \text{ 上停机且 } y = \varepsilon \\ \uparrow & \text{其他情形} \end{cases}$$

(我们假设这台机器的纸带字母表是 $\Sigma = \{\mathbf{B}, 1\}$ 。) 这直观上是可计算的, 所以 Church 论题表明存在具有这种行为的一台 Turing 机, 并且我们可以假设它的索引是 e_0 。

应用 s - m - n 定理对 x 进行参数化, 给出一族机器和相关的可计算函数, 如下所示。

$$\phi_{s(e_0, x)}(y) = \begin{cases} 42 & \text{若机器 } \mathcal{P}_x \text{ 在 } x \text{ 上停机且 } y = \varepsilon \\ \uparrow & \text{其他情形} \end{cases}$$

那么 $\phi_x(x) \downarrow$ 当且仅当函数 $\phi_{s(e_0, x)}$ 只在 $y = \varepsilon$ 上停机。所以如果我们能机械地检查后者, 那么我们就机械地解决 **停机问题**, 这当然是不可能的。

II.6.32 填充引理 (I.2.17) 说每个可计算函数都有无限多个索引。所以如果集合包含一个 $e \in \mathbb{N}$, 那么它必须包含无限多个索引 $\hat{e}$, 使得 $\phi_e \simeq \phi_{\hat{e}}$ 。

II.6.33 我们称每个集合为 J 。

- (A) 假设 $e \in J$ 且 $\hat{e}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in J$, 机器 $\mathcal{P}_e$ 至少在五个输入上停机, 即函数 ϕ_e 至少在五个输入上收敛。因为 $\phi_e \simeq \phi_{\hat{e}}$, 函数 $\phi_{\hat{e}}$ 也至少在五个输入上停机, 即相同的五个输入。因此 $\hat{e} \in J$ 。
- (B) 假设 $e \in J$ 且 $\hat{e} \in \mathbb{N}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in J$, 函数 ϕ_e 是单射的。因为 $\phi_e \simeq \phi_{\hat{e}}$, 函数 $\phi_{\hat{e}}$ 也是单射的。因此 $\hat{e} \in J$ 。
- (C) 假设 $e \in J$ 且 $\hat{e} \in \mathbb{N}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in J$, 函数 ϕ_e 要么是全函数, 要么 $\phi_e(3) \uparrow$ 。如果 ϕ_e 是全函数, 因为 $\phi_e \simeq \phi_{\hat{e}}$, 函数 $\phi_{\hat{e}}$ 也是全函数。如果 $\phi_e(3) \uparrow$, 因为它们行为相同, $\phi_{\hat{e}}(3) \uparrow$ 也发散。无论哪种情况, $\hat{e} \in J$ 。

II.6.34

- (A) 关系 $\simeq$ 是自反的, 因为 Turing 机 $\mathcal{P}_e$ 与自身行为相同: $\phi_e \simeq \phi_e$ 。它是对称的, 因为如果 $\mathcal{P}_e$ 与 $\mathcal{P}_{\hat{e}}$ 行为相同, 那么反之也成立。传递性同样显然。
- (B) 每个等价类是一个自然数集合, 其中的元素 $e \in \mathbb{N}$ 。两个整数在同一个类中, $e, \hat{e} \in \mathcal{C}$ 当对应的 Turing 机 $\mathcal{P}_e$ 和 $\mathcal{P}_{\hat{e}}$ 的输入输出行为相同, 即 $\phi_e \simeq \phi_{\hat{e}}$ 。(例如, 一个类包含所有将输入加倍的 Turing 机的索引, 即 $\mathcal{C} = \{e \in \mathbb{N} \mid \text{对所有 } x \text{ 均有 } \phi_e(x) = 2x\}$ 。)
- (C) 我们按照提示来证明。假设 J 是一个索引集, e 是 J 的一个元素, 并且它属于关系 $\simeq$ 的等价类 $\mathcal{C}$ 。令 $\hat{e}$ 是类 $\mathcal{C}$ 的另一个元素, 因此 $\phi_{\hat{e}} \simeq \phi_e$ 。因为 J 是一个索引集, 所以 $\hat{e} \in J$ 也成立。因此, 如果 J 包含 $\mathcal{C}$ 的一个元素, 那么它包含 $\mathcal{C}$ 的每一个元素。

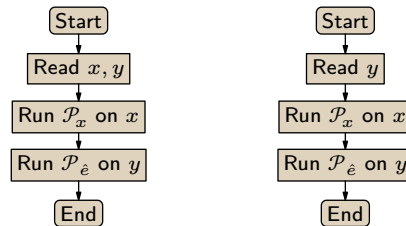


图 40, FOR QUESTION II.6.36: Rice 定理证明一半所用机器示意图。

II.6.35

(A) 设 $\mathcal{J}$ 是一个索引集, 并考虑其补集 $\mathcal{J}^c$ 。假设 $e \in \mathcal{J}^c$, 并且假设 $\hat{e} \in \mathbb{N}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。我们将证明 $\hat{e} \in \mathcal{J}^c$ 。

要么 $\hat{e} \in \mathcal{J}$, 要么 $\hat{e} \in \mathcal{J}^c$ 。如果 $\hat{e} \in \mathcal{J}$, 那么因为 $\phi_e \simeq \phi_{\hat{e}}$ 且 $\mathcal{J}$ 是一个索引集, 我们会有 $e \in \mathcal{J}$ 也成立。这与假设 $e \in \mathcal{J}^c$ 矛盾。因此 $\hat{e} \in \mathcal{J}^c$ 。

(B) 取一族索引集 $\mathcal{J}_j$, 考虑并集 $\mathcal{J} = \cup_j \mathcal{J}_j$ 。假设 $e \in \mathcal{J}$, 并且假设 $\hat{e} \in \mathbb{N}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。我们将证明 $\hat{e} \in \mathcal{J}$ 。

数 e 是并集 $\cup_j \mathcal{J}_j = \mathcal{J}$ 的一个元素, 因此它是某个 $\mathcal{J}_{j_0}$ 的元素。因为 $\phi_e \simeq \phi_{\hat{e}}$ 且 $\mathcal{J}_{j_0}$ 是一个索引集, 数 $\hat{e}$ 是 $\mathcal{J}_{j_0}$ 的一个元素, 因此也是并集 $\mathcal{J}$ 的一个元素。所以 $\mathcal{J}$ 是一个索引集。

(C) 它在交运算下封闭。设 $\mathcal{J}_j$ 是一族索引集, 并令 $\mathcal{J} = \cap_j \mathcal{J}_j$ 。假设 $e \in \mathcal{J}$, 并且假设 $\hat{e} \in \mathbb{N}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。我们将证明 $\hat{e} \in \mathcal{J}$ 。

由于 e 是交集的一个元素, 即 $\cap_j \mathcal{J}_j = \mathcal{J}$, 所以它是每个集合 $\mathcal{J}_j$ 的元素。因为 $\phi_e \simeq \phi_{\hat{e}}$ 且每个 $\mathcal{J}_j$ 是一个索引集, 数 $\hat{e}$ 也是每个 $\mathcal{J}_j$ 的元素。既然 $\hat{e}$ 是每个 $\mathcal{J}_j$ 的元素, 它也是交集 $\mathcal{J}$ 的元素。因此 $\mathcal{J}$ 是一个索引集。

II.6.36 本节中已完成的那半部分证明固定了一个索引 $e_0 \in \mathbb{N}$, 使得 $\phi_{e_0}(y) \uparrow$ 对所有输入 $y \in \mathbb{N}$ 成立。它随后处理了 $e_0 \notin \mathcal{J}$ 的情况, 所以这里剩下要做的就是 $e_0 \in \mathcal{J}$ 的情况。

因为 $\mathcal{J}$ 是非平凡的, 它不等于 $\mathbb{N}$ 。所以存在某个 $\hat{e} \notin \mathcal{J}$ 。由于 $\mathcal{J}$ 是索引集, $\phi_{e_0} \not\simeq \phi_{\hat{e}}$ 。因此存在某个 y 使得 $\phi_{\hat{e}}(y) \downarrow$ 。

考虑 Figure 40 on page 83 左侧的流程图。根据 Church 论题, 存在一台 Turing 机具有该行为; 称之为 $\mathcal{P}_e$ 。应用 s - m - n 定理对 x 进行参数化, 得到一致可计算的函数族 $\phi_{s(e,x)}$, 其计算过程如 Figure 40 on page 83 右侧所示。我们这样构造机器: 如果 $\phi_x(x) \uparrow$, 那么 $\phi_{s(e,x)} \simeq \phi_{e_0}$, 因此 $s(e,x) \in \mathcal{J}$ 。进一步, 如果 $\phi_x(x) \downarrow$, 那么 $\phi_{s(e,x)} \simeq \phi_{\hat{e}}$, 因此 $s(e,x) \notin \mathcal{J}$ 。于是, 如果 $\mathcal{J}$ 是机械可计算的, 从而我们能够有效判定是否 $s(e,x) \in \mathcal{J}$, 那么我们就机械地解决停机问题。当然, 这不可能。

II.7.7 只对了一半。说一个集合若能被 Turing 机枚举就是可计算枚举的, 这是对的。但“但不是可计算的”是错的。例如, 偶数集是可计算枚举的, 因为它显然是可有效列举的, 并且它也是可计算的, 因为其特征函数是可计算的。

II.7.8 “可有效列举”的意思是, 某个机器过程将其作为输出集合给出。有些 Turing 机无论输入什么都不会停机, 这些机器所列举的就是空集。

II.7.9 对每个部分, 我们将给出一个全可计算函数 $f: \mathbb{N} \rightarrow \mathbb{N}$, 使其值域为给定集合。

- (A) 恒等函数 $f(x) = x$ 即可。
- (B) 函数 $f(x) = 2x$ 即可。
- (C) $f(x) = x^2$
- (D) 这个函数可行。

$$f(x) = \begin{cases} 5 & \text{— 若 } x = 0 \\ 7 & \text{— 若 } x = 1 \\ 11 & \text{— 其他情形} \end{cases}$$

II.7.10 对每一项，我们构造一个有效函数 $f: \mathbb{N} \rightarrow \mathbb{N}$ 。

- (A) 对应关系 $n \mapsto$ 第 n 个素数 是一个函数。显然我们可以编写一个程序来计算它，因此由 Church 论题可知它是有效的。
- (B) 为了计算这个函数，我们可以用暴力搜索：给定 n ，遍历自然数 $0, 1, \dots$ ，检查每个数的各位数字是否按非递增顺序出现，输出第 n 个这样的数。

II.7.11 以上四种情况的回答都是肯定的。对于第一种和第四种情况，我们可以编写一台 Turing 机，其输入/输出行为是充当恒等函数 $x \mapsto x$ ，这台机器列出了所有的自然数。因此，所有自然数的集合是可计算枚举的。

对于第二种和第三种情况，存在在任何输入上都不停机的 Turing 机，所以空集是可计算枚举的。

II.7.12 第一个是可计算的，因为要判定一个数 e 是否属于这个集合，我们只需模拟二十步。第二个是可计算枚举但不可计算的。它是可计算枚举的，因为我们可以对索引 $e \in \mathbb{N}$ 进行交错并行计算。它是不可计算的，因为显然如果我们能解决它，就能解决判定 $\{e \mid \phi_e(4) \downarrow\}$ 的成员资格的问题，而由 Rice 定理可知我们无法解决那个问题。

II.7.13 对于集合而言，“可判定的”与“可计算的”含义相同，即其特征函数是全可计算函数。同时，“半可判定的”与“可计算枚举的”含义相同。

- (A) 这个集合是可计算的，因为我们只需运行机器一百步。由于它是可计算的，所以它也是可计算枚举的。
- (B) 这个集合也是可计算的，同样因为我们只需运行机器一百步。由于它是可计算的，所以它也是可计算枚举的。

II.7.14 第二个陈述是真的，这由引理 7.5 可得。第一个陈述是假的；一个例子是：集合 $\mathbb{N}$ 是可计算枚举的，但其真子集 K 是不可计算的。

II.7.15 这个函数确实是一个枚举，但除非它是可计算的，否则它就不是一个可计算枚举。一个可数但不可计算枚举的集合的例子是 K^c 。它是可数的，因为它是可数集合 $\mathbb{N}$ 的一个子集，并且由推论 7.6 可知它是不可计算枚举的。

II.7.16 简言之，采用交错并行。为了得到枚举，首先让 $\mathcal{P}_0$ 在输入 5 上运行一步。然后让 $\mathcal{P}_0$ 在输入 5 上运行第二步，同时让 $\mathcal{P}_1$ 在输入 5 上运行一步。接着让 $\mathcal{P}_0$ 运行第三步，同时让 $\mathcal{P}_1$ 运行其第二步，并让 $\mathcal{P}_2$ 在输入 5 上运行一步。依此方式继续。枚举函数 $f: \mathbb{N} \rightarrow \mathbb{N}$ 定义为： $f(0)$ 是第一个停机的这类计算的索引， $f(1)$ 是第二个，依此类推。

II.7.17 因为只有可数多台 Turing 机来执行枚举。

II.7.18 不是。空集是可计算枚举的，但是有限的。

II.7.19

- (A) 该集合是可计算的。给定一个索引 e ，我们需要判定 $\mathcal{P}_e$ 是否包含这样的指令。索引是源码等价的，这意味着存在一个程序，它能读入索引 e 并输出 $\mathcal{P}_e$ 的指令集。然后我们可以通过扫描该指令集，检查是否存在以 q_4 开头的指令，来决定 e 是否属于该集合。
- (B) 该集合是可计算枚举但不可计算的。要说明它是可计算枚举的，可以对索引 $e \in \mathbb{N}$ 使用交错并行，在输入 4 上运行各 $\mathcal{P}_e$ 。也就是说，先在输入 4 上运行 $\mathcal{P}_0$ 一步。接着，让 $\mathcal{P}_0$ 在输入 4 上再运行一步，同时在输入 4 上运行 $\mathcal{P}_1$ 一步。继续这个过程，当一个计算停机时，就将其索引 e 枚举入集合。至于该集合不可计算，由 Rice 定理容易证明它不是可计算集。
- (C) 可计算的。给定 e ，通过在输入 4 上运行 $\mathcal{P}_e$ 一百步来判断它是否属于该集合。

II.7.20 要枚举它，使用交错并行。首先在输入 2 上运行 $\mathcal{P}_0$ 一步。然后在输入 2 上运行 $\mathcal{P}_0$ 第二步，同时在输入 2 上运行 $\mathcal{P}_1$ 一步。接着，运行 $\mathcal{P}_0$ 第三步、 $\mathcal{P}_1$ 第二步，并在输入 2 上运行 $\mathcal{P}_2$ 一步。以此类推，继续这个过程。令 $f(0)$ 为第一个停机并输出 4 的计算的索引，令 $f(1)$ 为第二个这样的索引，等等。

II.7.21 当然有很多正确答案。一个答案是取 K 作为第一个集合。第二个集合取去掉最小元素的 K ，第三个集合取去掉最小的两个元素的 K 。

一个或许不那么取巧的答案是取 K 再加上前两节中作为例子使用过的一些集合，例如 $\{e \in \mathbb{N} \mid \phi_e(3) \downarrow\}$ 和 $\{e \in \mathbb{N} \mid \text{存在 } x \in \mathbb{N} \text{ 使 } \phi_e(x) = 7\}$ 。后两者通过交错并行过程是可计算枚举的。

II.7.22

- (A) 使用交错并行来枚举它。与之前一些交错并行例子的不同之处在于，这里有两个数， e 和 x 。因此回想一下 Cantor 配对函数。

Number	0	1	2	3	4	5	6	...
Pair	$\langle 0, 0 \rangle$	$\langle 0, 1 \rangle$	$\langle 1, 0 \rangle$	$\langle 0, 2 \rangle$	$\langle 1, 1 \rangle$	$\langle 2, 0 \rangle$	$\langle 0, 3 \rangle$	...

在交错并行循环的第一次迭代中，找到表格中的第一对 $\langle 0, 0 \rangle$ （我们取第一个数为 e ，第二个为 x ），并在输入 0 上运行 $\mathcal{P}_0$ ，执行一步。在循环的第二次迭代中，在输入 0 上运行 $\mathcal{P}_0$ 执行第二步，同时（由于 $\langle 0, 1 \rangle$ 是第二对）在输入 1 上运行 $\mathcal{P}_0$ 执行一步。在第三次迭代中，在输入 0 上运行 $\mathcal{P}_0$ 执行第三步，在输入 1 上运行 $\mathcal{P}_0$ 执行第二步，并在输入 0 上运行 $\mathcal{P}_1$ 执行一步。当 $\mathcal{P}_e$ 在 x 上停机时，枚举函数输出 $\langle e, x \rangle$ 。

- (B) 这可以直接从引理 7.3 得出。

II.7.23 不，原因有二。

首先，可计算集类与可计算枚举集类并非不相交。实际上，由引理 7.5，可计算集类是可计算枚举集类的一个子集。

此外，集合 $\{W_e \mid e \in \mathbb{N}\}$ 是可数的，因为该集合中的元素 W_e 是由自然数索引的。由于 $\mathbb{N}$ 的子集的数量是不可数的（因为 $\mathbb{N}$ 的幂集的势大于 $\mathbb{N}$ ），所以 $\mathbb{N}$ 的子集比可计算枚举集要多。因此，存在 $\mathbb{N}$ 的子集不是可计算枚举的。

II.7.24 可计算集也是可计算枚举的，所以只要我们证明了 Tot 不是可计算枚举的，也就证明了它既不可计算也非可计算枚举。

为导出矛盾，假设 $\text{Tot} = \{e \mid \text{对所有 } x \text{ 均有 } \phi_e(x) \downarrow\}$ 是可计算枚举的，并由函数 $f: \mathbb{N} \rightarrow \mathbb{N}$ 枚举。根据提示，这给出了一个类似于 section 5 中关于停机问题的表格，因为第 i, j 项 $\phi_{f(i)}(j)$ 必定停机。对角化：考虑由 $h(i) = 1 + \phi_{f(i)}(i)$ 给出的函数 $h: \mathbb{N} \rightarrow \mathbb{N}$ 。该函数是有效的、全的，且不等于 Tot 中的任何成员，矛盾。

II.7.25 由 Rice 定理容易证明集合 $S = \{e \mid \phi_e(3) \uparrow\}$ 不可计算。显然，其补集 $S^c = \{e \mid \phi_e(3) \downarrow\}$ 是可计算枚举的——只需交错并行，并在元素收敛时枚举它们。如果 S 也是可计算枚举的，那么由引理 7.5 它将是可计算的，这就矛盾了。

II.7.26 设该无限可计算枚举集为 W 并固定一个可计算枚举 $\phi(0), \phi(1), \dots$ 我们将定义一个无限的可计算子集 $S \subseteq W$ 。

令 S 的第一个元素为 $s_0 = \phi(0)$ 。取第二个元素 s_1 为枚举中第一个大于 s_0 的数；由于 W 是无限的，这样的数终将出现。一般地，令 s_{i+1} 为枚举中第一个大于 s_i 的元素（与 s_1 的情况一样，由于 W 无限，这样的元素终将出现）。

集合 S 是可计算枚举的，因为我们刚才描述了一个枚举它的机械过程。它是可计算的，因为给定一个数 $n \in \mathbb{N}$ ，要判定它是否是 S 的元素，只需等待，直到它被枚举进 S ，或者有一个比它大的数被枚举进 S （这表明 $n \notin S$ ）。

II.7.27

- (A) 我们可以根据 Church 论题来论证：通过在输入 e 上运行 $\mathcal{P}_e$ （使用一台通用 Turing 机）来计算 $\text{steps}(e)$ ，若它停机，则输出步数。
- (B) 我们断言，如果 steps 不仅是可计算的而且还是全函数，那么我们就解决停机问题——停机问题。原因是，要判定 $e \in K$ ，首先计算 $\text{steps}(e)$ 。然后在输入 e 上运行 $\mathcal{P}_e$ 共 $\text{steps}(e)$ 步。若该计算在这么多步之后仍未停机，则它将永不停机。
- (C) 这与前面的论证相同，只是用 f 代替了 steps 。

II.7.28 假设 S 是可计算的，那么其特征函数

$$\mathbb{1}_S(x) = \begin{cases} 1 & \text{若 } x \in S \\ 0 & \text{若 not} \end{cases}$$

是一个可计算函数。为了按递增次序枚举 S ，运行 $\mathbb{1}_S(0), \mathbb{1}_S(1), \dots$ ，由于函数是可计算的，每个都必须收敛，然后 S 的第一个元素是最小的，第二个元素是第二小的，等等。

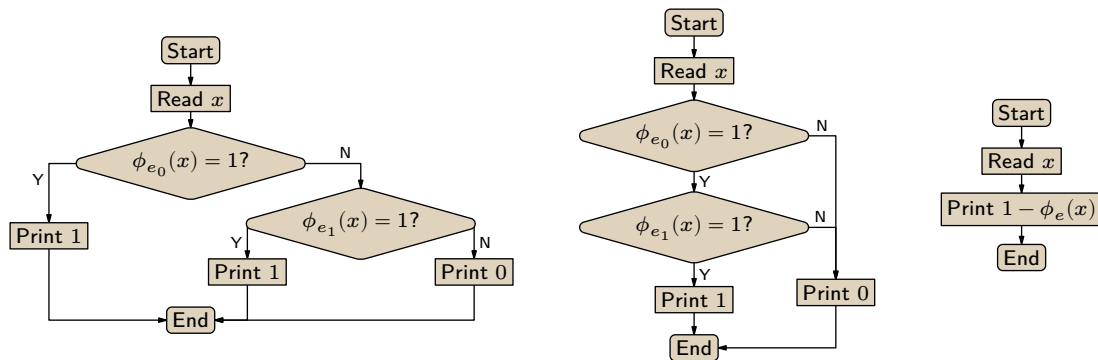


图 41, FOR QUESTION II.7.30: 可计算集合的并、交、补的判定机草图。

更形式化地说, 定义枚举函数如下: 设 s_0 是 S 的最小元素, 则 $f(0) = s_0$; 若我们已得到 $f(0), \dots, f(k)$ 的值, 则 $f(k+1)$ 是满足 $x > f(k)$ 且 $\mathbb{1}_S(x) = 1$ 的最小 x 。因为 S 是无限的, 总存在这样的 x 。

对于另一个方向, 假设 S 是一个可由可计算函数 h 按严格递增次序枚举的无限集。给定 x , 要判定 $x \in S$, 计算 $h(0), h(1), \dots$ 直到输出为 x (于是 $x \in S$) 或者输出大于 x (于是 $x \notin S$)。

II.7.29 考虑序对集合 $S = \{ \langle a, b \rangle \mid a \in K \text{ 且 } b \in K^c \}$ 。它不是可计算枚举的, 否则枚举它将枚举其第二分量, 但 K^c 不是可计算枚举的。类似地, 它的补集也不是可计算枚举的, 否则枚举它将枚举 S 的第一分量集合的补集, 这同样不可能, 因为 K^c 不是可计算枚举的。

II.7.30

- (A) 否。集合 $\mathbb{N}$ 是可计算的, 但其子集 K 不可计算。
- (B) 是。设 $S_0 \subseteq \mathbb{N}$ 和 $S_1 \subseteq \mathbb{N}$ 分别通过函数 ϕ_{e_0} 和 ϕ_{e_1} 可计算。计算并集的机器草图见 Figure 41 on page 86。
- (C) 是。计算交集的机器草图也在同一图中。
- (D) 是。计算补集的机器草图也在同一图中。

II.7.31

- (A) 否。集合 $\mathbb{N}$ 是可计算枚举的。停机问题集 K 的补集 K^c 是 $\mathbb{N}$ 的子集, 并且不是可计算枚举的。
- (B) 是。设 $W_{e_0}, W_{e_1} \subseteq \mathbb{N}$ 是可计算枚举的, 分别通过 $\phi_{e_0}, \phi_{e_1}: \mathbb{N} \rightarrow \mathbb{N}$ 枚举。交错并行枚举这两个集合。即, 先让 $\mathcal{P}_{e_0}$ 和 $\mathcal{P}_{e_1}$ 在输入 0 上各运行一步。然后, 让 $\mathcal{P}_{e_0}$ 和 $\mathcal{P}_{e_1}$ 在输入 0 上再运行一步, 同时让它们在输入 1 上各运行一步。如此继续, 当某个数被枚举进 W_{e_0} 或 W_{e_1} 时, 就将其枚举进它们的并集。
- (C) 是。如上一项所述, 交错并行枚举这两个集合。当某个数已被枚举进 W_{e_0} 和 W_{e_1} 两者中时, 再将其枚举进它们的交集。
- (D) 否。停机问题集 K 是可计算枚举的, 但其补集不是。

II.7.32 我们必须证明: 如果集合 S 是全可计算函数的值域或是空集, 那么 S 是某个部分可计算函数的定义域。

我们先证从左到右的蕴含。若 S 为空集, 则它是处处发散的部分可计算函数的定义域。否则 S 是全可计算函数 f 的值域; 我们将描述一个定义域为 S 的部分可计算函数 g 。计算 g 的方法如下: 给定输入 $x \in \mathbb{N}$, 枚举 $f(0), f(1), \dots$ 如果 x 出现在这个序列中, 则停止对 g 的计算 (并输出某个任意值)。如果 x 从未出现, 则 $g(x)$ 的计算永不停机。这样, g 的定义域就是 S 。

对于另一个方向的蕴含, 假设 S 是某个部分可计算函数 $\hat{g}$ 的定义域。我们必须证明 S 要么是空集, 要么是某个全可计算函数的值域。部分可计算函数的定义域可以是空集; 如果 S 是空集, 则结论成立。否则固定某个 $s_0 \in S$, 我们将构造一个值域为 S 的全可计算函数 $\hat{f}$ 。给定 $n \in \mathbb{N}$, 并行运行 $\hat{g}(0), \hat{g}(1), \dots, \hat{g}(n)$ 的计算, 每个各运行 n 步。其中某些计算可能会停机。定义 $\hat{f}(n)$ 为满足以下条件的、最小的 k : $\hat{g}(k)$ 的计算在 n 步内停机, 且 $k \notin \{\hat{f}(0), \hat{f}(1), \dots, \hat{f}(n-1)\}$ 。如果不存在这样的 k ,

则定义 $\hat{f}(n) = s_0$ (这保证了 $\hat{f}$ 是全函数)。

最后验证 $\hat{f}$ 确实是所需的枚举。若 $t \notin S$, 则 $\hat{g}(t)$ 的计算永不停机, 因此 t 永远不会被 $\hat{f}$ 枚举。若 $t \in S$, 则 $\hat{g}(t)$ 的计算最终必在若干步 (设为 n_t) 后停机。于是数 t 将被 $\hat{f}$ 加入输出队列, 即它至多在 $\hat{f}(n_t + t)$ 时被枚举。

II.8.17 我们并不假设存在一个原则上物理可实现的设备, 能在有限时间内正确回答 “ $\mathcal{P}_e$ 在输入 e 上停机吗?” 这类问题。但在论证中使用这样的措辞是可以的: “假设我们有一个 **停机问题** 神谕机, 那么我们可以……”

评论。值得一提的是, 神谕机这个概念, 无论是本节描述的, 还是扩展到诸如可以解决密码学问题、或能在线性时间内对数字列表排序的神谕机, 在实际问题中都非常常用。

II.8.18 判定器是一台 Turing 机, 其计算出的函数是某个集合的特征函数。神谕机本身是一个集合, 作为 Turing 机的附加部分, 供机器查询。

II.8.19 首先, 它们非常有趣且令人着迷。除此之外, 它们有助于我们理解问题之间的关系。这包括试图理解哪些问题容易、哪些问题困难。

在复杂性章节中, 我们将扩展这一思想, 以帮助理解许多日常的、实际的问题是如何相互关联的。

II.8.20 他们的回答抓住了我们考虑神谕机计算的原因的精髓, 但在两个方面有所欠缺。第一, 我们可以使用一个可计算集合 (例如空集) 作为神谕机。(尽管, 确实, 使用一个可计算神谕机并不能解决任何没有该神谕机就无法解决的问题。)

第二点不足是, 他们的回答完全没有提及机制。神谕机集合本身并不解决问题。我们可能会非正式地这么说, 但作为定义, 我们应该说: 我们让一台 Turing 机能够查询关于神谕机成员资格的答案, 然后该机器的输入-输出行为才是问题的解。

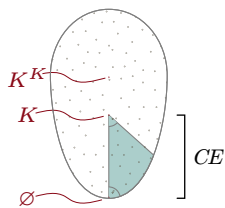
II.8.21 我们将第一个问题理解为: 是否存在一个最强大的神谕机, 能够解决所有问题? 答案是否定的。给定集合 $X \subseteq \mathbb{N}$, 判定 K^X (即相对于该神谕机的停机问题) 的成员资格的问题无法仅凭 X 来解决。

第二个问题是: 给定集合 $S \subseteq \mathbb{N}$, 是否存在一个足以判定 S 成员资格的神谕机? 答案是肯定的: 神谕机 S 本身就可以。

II.8.22 每个神谕机只能计算某些问题的答案。(存在可数多个 $\mathcal{P}_e^X$, 但 $\mathbb{N}$ 的子集有不可数多个。)因此, 对于每个神谕机集合, 都存在它无法解决的问题。具体来说, 它无法解决相对于该集合的停机问题。

II.8.23 在一个停机的神谕机计算中, 确实只有有限次对神谕机的调用。但当计算的输入是 0 时进行的调用, 很可能与输入是 1 时进行的调用不同。

II.8.24 它在 K 之上。



II.8.25 回想一下, “ A Turing 可归约到 B ” 的记号是 $A \leq_T B$ 。

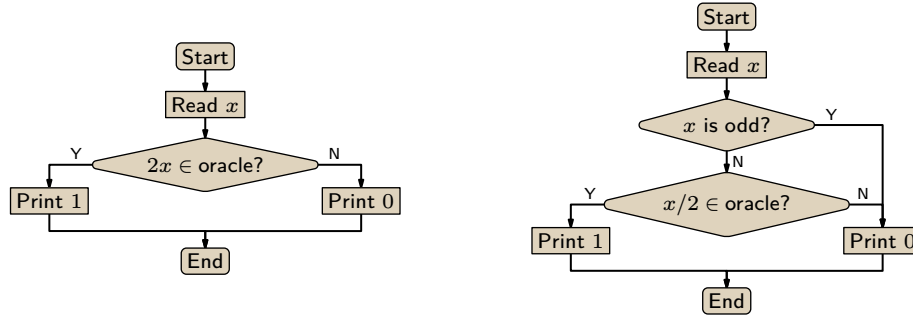
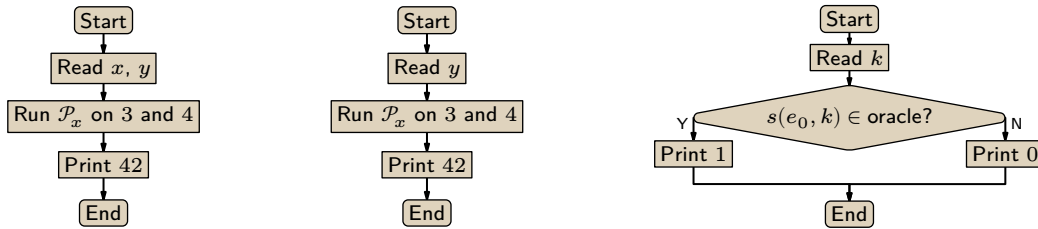
(A) 此为假。例如, 空集可归约到 **停机问题** 集, 即 $\emptyset \leq_T K$; 然而, 虽然空集的判定器是平凡的, K 却没有可计算的判定器。

正确的陈述恰恰相反: 如果 $A \leq_T B$, 那么我们可以使用 B 的判定器来判定关于集合 A 的成员问题。

(B) 假。与上一项类似, 如果 A 是空集, 那么它是可计算的。此时 $\emptyset \leq_T K$, 但 K 是不可判定的。正确的方向是反过来: 如果 B 是可判定的, 那么 A 也是可判定的。

(C) 真。如果 $A \leq_T B$, 则存在某个索引 $e \in \mathbb{N}$, 使得函数 ϕ_e^B 是 A 的特征函数, 即 $\phi_e^B = \mathbb{1}_A$ 。有了这个函数, 如果我们能计算 B , 那我们就能用它来计算 A 。因此, 如果 A 是不可计算的, 那么 B 也必定是不可计算的。

II.8.26 参见 Figure 42 on page 88。

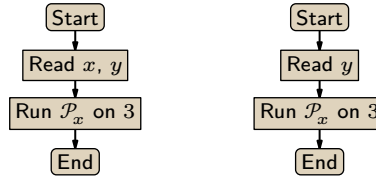
图 42, FOR QUESTION II.8.26: 证明 $B \leq_T 2B$ 和 $2B \leq_T B$ 。图 43, FOR QUESTION II.8.28: 集合 $\{x \mid \phi_x(3) \downarrow \text{ 且 } \phi_x(4) \downarrow\}$ Turing 可归约到 K 。

(A) 左边的流程图一目了然。

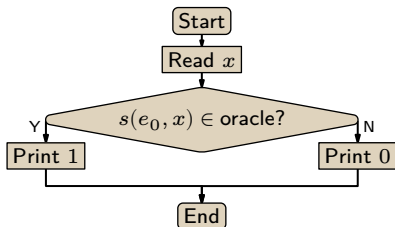
(B) 对于右边的流程图, 注意如果输入 x 是奇数, 那么这台机器必须回答“否”, 因为任何奇数都不可能是 $2B$ 的元素。

II.8.27 从下面的函数 $\psi: \mathbb{N} \rightarrow \mathbb{N}$ 开始。

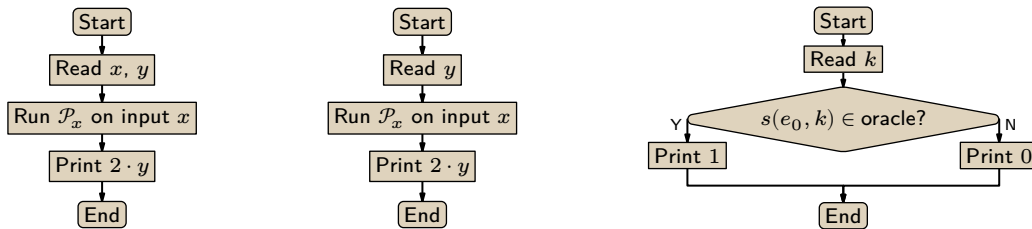
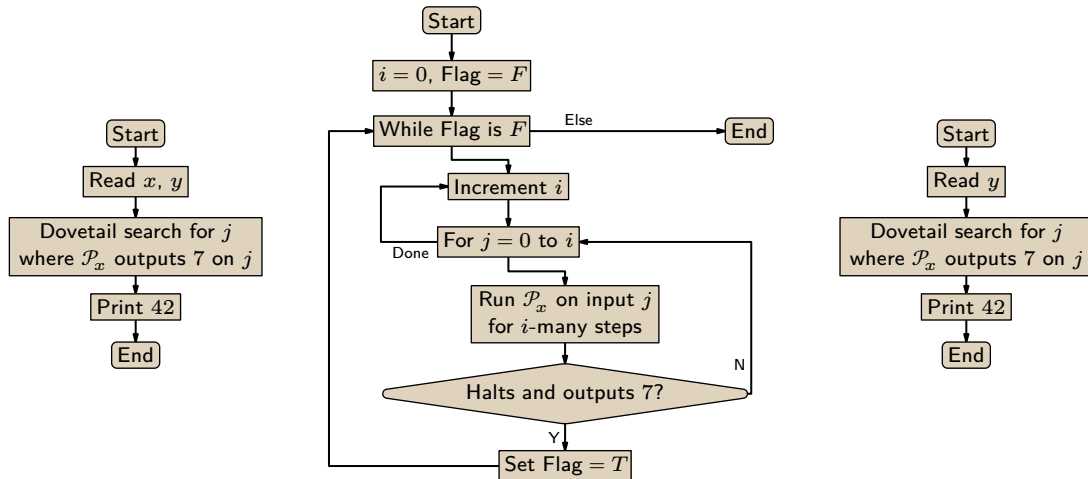
$$\psi(x, y) = \begin{cases} \mathcal{P}_x(3) & \text{— 若 } \mathcal{P}_x(3) \downarrow \\ \uparrow & \text{— 其他情形} \end{cases}$$



由 Church 论题, 第一个流程图表明 ψ 是机械可计算的。设其索引为 e_0 。应用 S-m-n 定理对 x 进行参数化, 得到右边草图的机器, 其索引为 $s(e_0, x)$ 。这台机器的行为不依赖于其输入 y , 所以 $\phi_{s(e_0, x)}(s(e_0, x)) \downarrow$ 当且仅当 $\phi_x(3) \downarrow$ 。因此, 给定 x , 我们可以借助 K 神谕机通过这台机器来测试 $x \in A$ 是否成立。



II.8.28 Figure 43 on page 88 左边是一个程序的流程图草图。由 Church 论题, 存在一台 Turing 机计算它。设该机器的索引为 e_0 。应用 S-m-n 定理得到一族由 x 参数化的机器。该族的第 x 个成员显示在中间。显然, 如果它对任何输入停机, 则它对所有输入都停机。因此, $\mathcal{P}_x$ 在输入 3 和 4 上停机当且仅当 $s(e_0, x) \in K$ 。换言之, 存在一个可计算函数 $x \mapsto s(e_0, x)$, 使得 $x \in \{e \mid \phi_e(3) \downarrow \text{ 且 } \phi_e(4) \downarrow\}$ 当且仅当 $s(e_0, x) \in K$ 。所以 $\{e \mid \phi_e(3) \downarrow \text{ 且 } \phi_e(4) \downarrow\} \leq_T K$ 。

图 44, FOR QUESTION II.8.30: 证明 $K \leq_T \{e \mid \text{对所有输入 } y \text{ 均有 } \phi_e(y) = 2y\}$ 。图 45, FOR QUESTION II.8.31: 用 K 神谕机计算 $\{x \mid \text{存在 } j \text{ 使 } \phi_x(j) = 7\}$ 。

现在我们利用可计算函数 $x \mapsto s(e_0, x)$ 来构造那台神谕机机器。参见图中右边的图表。

II.8.29 根据定义, $K_0 = \{\langle e, x \rangle \mid \phi_e(x) \downarrow\}$ 。所以 $e \in S$ 当且仅当 $\langle e, 3 \rangle \in K_0$ 。

II.8.30 令 $D = \{e \mid \text{对所有输入 } y \text{ 均有 } \phi_e(y) = 2y\}$ 。考虑下面这个函数。

$$\psi(x, y) = \begin{cases} 2y & \text{若 } \phi_x(x) \downarrow \\ \uparrow & \text{其他情形} \end{cases}$$

根据 Figure 44 on page 89 左侧的流程图, 由 Church 论题可知, 存在一台 Turing 机, 其输入/输出行为是 ψ 。令那台 Turing 机的索引为 e_0 , 于是有 $\phi_{e_0} = \psi$ 。

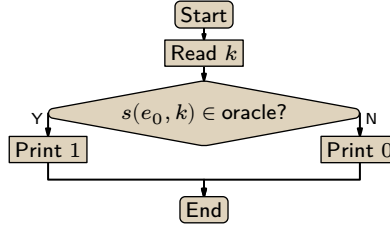
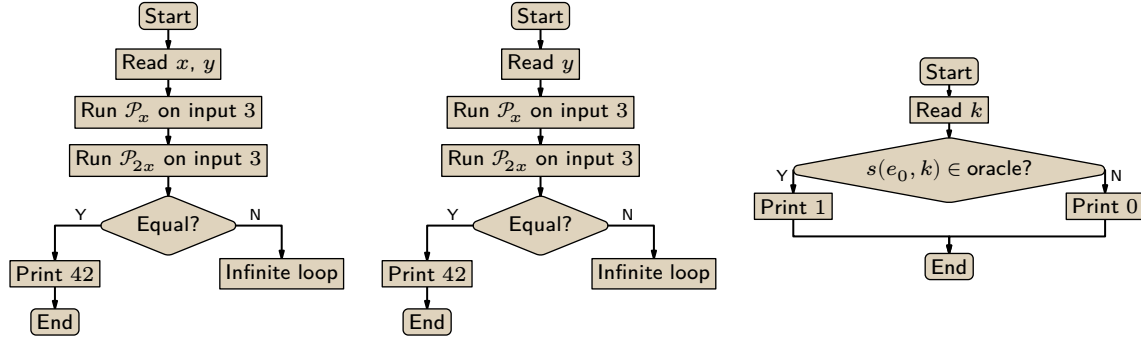
应用 S-m-n 定理, 得到一族函数 $\phi_{s(e_0, x)}$, 如图中中间部分所示。那么 $x \in K$ 当且仅当 $s(e_0, x) \in D$ 。图的右侧是一台神谕机机器, 它表明 $K \leq_T D$, 如所要求。

II.8.31

(A) 令 $\mathcal{J} = \{e \mid \text{存在 } j \text{ 使 } \phi_e(j) = 7\}$ 。该集合非空, 因为其中一个元素是一台对所有输入都输出 7 的 Turing 机的索引。该集合不是整个 $\mathbb{N}$, 因为其中一个不属于它的元素是一台在所有输入上都不停机的 Turing 机的索引。

为了看出 $\mathcal{J}$ 是一个索引集, 假设 $e \in \mathcal{J}$, 并且 $\hat{e}$ 满足 $\phi_e \simeq \phi_{\hat{e}}$ 。因为 $e \in \mathcal{J}$, 所以存在一个输入 j 使得 $\phi_e(j) = 7$ 。由于 $\phi_e \simeq \phi_{\hat{e}}$, 对于同一个 j , 我们有 $\phi_{\hat{e}}(j) = 7$, 因此 $\hat{e} \in \mathcal{J}$ 也成立。

(B) Figure 45 on page 89 左侧是一个程序流程图草图。第二张图更详细地说明了“交错并行”涉及的内容。由 Church 论题, 存在一台 Turing 机来计算这个流程图。令该机器的索引为 e_0 。应用 S-m-n 定理, 得到一族由 x 参数化的机器, 如第三个流程图所示。显然, 它对所有输入 y 都停机, 当且仅当 $\mathcal{P}_x$ 对某个输入输出了一个 7。因此, $\mathcal{P}_x$ 曾输出一个 7 当且仅当 $s(e_0, x) \in K$ 。这意味着存在一个可计算函数 $x \mapsto s(e_0, x)$, 其中 $x \in \{e \mid \text{存在 } j \text{ 使 } \phi_e(j) = 7\}$ 当且仅当 $s(e_0, x) \in K$ 。

图 46, FOR QUESTION II.8.31: 从 K 计算 $\{e \mid \text{存在 } j \text{ 使 } \phi_e(j) = 7\}$ 的神谕机器。图 47, FOR QUESTION II.8.32: 集合 $\{x \mid \phi_x(3) \downarrow = \phi_{2x}(3) \downarrow\}$ 与 K 是 T 等价的。

为了看出 $\{e \mid \text{存在 } j \text{ 使 } \phi_e(j) = 7\} \leq_T K$, 考虑 Figure 46 on page 90 中的神谕机器。如果将 K 神谕机接入这台机器, 那么根据上一段所述, 它的行为就是 $\{e \mid \text{存在 } j \text{ 使 } \phi_e(j) = 7\}$ 的特征函数。

II.8.32 Figure 47 on page 90 左侧是一个计算的草图。由 Church 论题, 存在一台具有相同行为的 Turing 机。令那台机器的索引为 e_0 。应用 S-m-n 定理, 得到一族由 x 参数化的机器, 其第 x 个成员 $\mathcal{P}_{s(e_0, x)}$ 如中间图所示。显然, 它对所有输入 y 都停机当且仅当 $x \in S$ 。特别地, 它对输入 $y = s(e_0, x)$ 停机当且仅当 $x \in S$ 。因此 $s(e_0, x) \in K$ 当且仅当 $x \in S$ 。由此容易看出, 将 K 神谕机接入右侧的神谕机机器, 就使其表现为 S 的特征函数。故而 $S \leq_T K$ 。

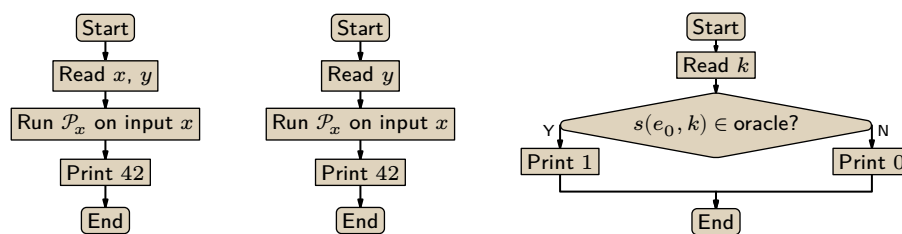
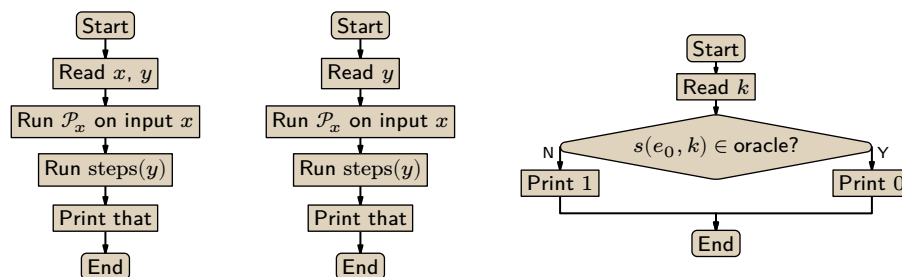
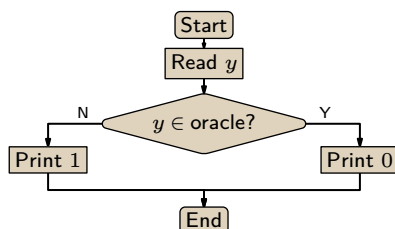
II.8.33 考虑 Figure 48 on page 91 左侧所勾勒的例程。由 Church 论题, 存在一台具有该行为的 Turing 机。令其索引为 e_0 。应用 S-m-n 定理, 得到一族机器 $\mathcal{P}_{s(e_0, x)}$, 如中间所示。观察到 $x \in K$ 当且仅当 $s(e_0, x) \in \text{Tot}$ 。右侧是一台神谕机机器, 当我们接入一个 Tot 神谕机时, 它的行为将是 K 的特征函数。因此 $K \leq_T \text{Tot}$ 。

II.8.34

- (A) 反设函数 steps 有一个可计算的扩展 g , 根据可扩展的定义, g 必须是全函数。那么我们可以通过运行 $\mathcal{P}_x$ 在输入 x 上最多 $g(x)$ 步来解决 停机问题。如果 $\phi_x(x)$ 的计算到那时还未停机, 它就永远不会停机。
- (B) 考虑 Figure 49 on page 91 左边概述的程序。根据 Church 论题, 存在具有该行为的 Turing 机。设其索引为 e_0 。应用 s-m-n 定理对 x 进行参数化, 得到一族如中间所示的程序, $\mathcal{P}_{s(e_0, x)}$ 。如果 $x \notin K$, 那么 $\phi_{s(e_0, x)}(y)$ 对所有输入 y 都无定义, 并且这个函数是 Ext 的一个成员, 例如, 总是返回 0 的函数 (显然是可计算的) 就是它的一个扩展。如果 $x \in K$, 那么 $\phi_{s(e_0, x)}(y) = \text{steps}(y)$ 对所有输入 y 都成立, 并且根据第一项, 这不是 Ext 的成员。因此, $k \in K$ 当且仅当 $s(e_0, k) \notin \text{Ext}$ 。右边的带神谕机的机器, 当连接到一个 Ext 神谕机时, 将作为 K 的特征函数。所以 $K \leq_T \text{Ext}$ 。

II.8.35 计算将以完全相同的方式进行, 因此会得到相同的结果。

II.8.36 带神谕机的机器见 Figure 50 on page 91。该机器在连接到一个 A^c 神谕机时, 将作为 A 的特征函数。

图 48, FOR QUESTION II.8.33: 用 Tot 神谕机计算 K 。图 49, FOR QUESTION II.8.34: 使用 Ext 神谕机计算 K 。图 50, FOR QUESTION II.8.36: 从 A^c 神谕机计算 A 。

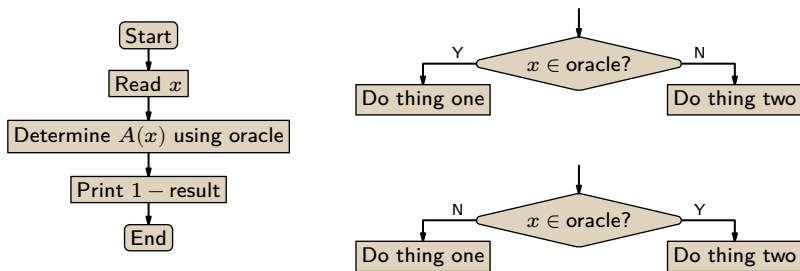


图 51, FOR QUESTION II.8.42: 与补集相关的带神谕机的机器。

II.8.37 根据引理 8.5, 可归约到 $\emptyset$ 的集合都是可计算的。但 K 不可计算。

II.8.38 任何 $\mathbb{N}$ 的子集都可以作为神谕机, 所以有不可数多个神谕机。

II.8.39 只有可数多个程序可能包含 `oracle` 调用。也就是说, 只有可数多个带神谕机的 Turing 机, 由 $\mathcal{P}_e^X$ 中的索引 e 枚举。

II.8.40 一个例子是 $C = \{\text{cantor}(0, a) \mid a \in A\} \cup \{\text{cantor}(1, b) \mid b \in B\}$ 。显然从这个集合我们可以回答关于 A 和 B 中成员资格的问题。

II.8.41

- (A) 否。停机问题集 K 是自然数集 $\mathbb{N}$ 的子集, 但 $K \not\leq_T \mathbb{N}$, 因为停机问题是不可解的。
- (B) 否。停机问题集 K 不 Turing 可归约到 $\mathbb{N} = K \cup K^c$, 但 $K \subseteq K \cup K^c$ 。
- (C) 否。停机问题集 K 不 Turing 可归约到 $\emptyset$, 但 $\emptyset = K \cap K^c$ 。
- (D) 是。我们可以写出这个使用神谕机的程序: 读取输入 x , 如果它是神谕机的一个元素则输出 0, 否则输出 1。

II.8.42 参考 Figure 51 on page 92。

- (A) 真。这在左侧的流程图中展示。基本上, 交换 ‘yes’ 和 ‘no’ 的输出。
- (B) 真。这在中间的流程图中展示。将所有形如上方所示的神谕机调用替换为下方所示的调用 (唯一区别是交换了 ‘Y’ 和 ‘N’)。
- (C) 真。结合前两项。

II.8.43

- (A) 自然的方法是相对化定义 7.1, 并定义集合 B 是相对于集合 A 可计算枚举的 (computably enumerable in the set A), 如果 B 是某个 A -可计算函数的值域。即, B 相对于 A 是 c.e. 的, 如果 $B = \{\phi_e^A(x) \mid x \in \mathbb{N}\}$ 对某个索引 e 成立 (该函数可能是偏函数)。
- (B) 如引理 8.5 所示, 我们可以不引用神谕机而输出 $\mathbb{N}$ 的元素。函数 $\phi^A(x) = x$ 是可计算的, 且永远不需要引用神谕机。
- (C) 交错并行。首先, 运行 $\mathcal{P}_0^A$ 在输入 0 上的一步计算。然后, 运行 $\mathcal{P}_0^A$ 在输入 0 上的第二步计算, 以及 $\mathcal{P}_1^A$ 在输入 1 上的一步计算。接下来, 运行 $\mathcal{P}_0^A$ 在输入 0 上的第三步计算, $\mathcal{P}_1^A$ 在输入 1 上的第二步计算, 以及 $\mathcal{P}_2^A$ 在输入 2 上的一步计算。如此继续下去, 随着时间推移, 其中一些计算会停机。当计算 $\mathcal{P}_e^A(e)$ 停机时, 将索引 e 枚举入 K^A 。(因此在枚举函数中, $\phi^A(i)$ 是以这种方式列出的第 i 个索引。)

II.9.6 在日常编程中, 我们很容易写出两段不同的源代码, 它们却具有相同的输入/输出行为, 即计算的是同一个函数。类似地, 说这两段代码是同一个函数, 即 $\phi_{g(v)} = \phi_{\phi_v(v)}$, 这不同于说它们由同一台 Turing 机产生, 即索引相同。

II.9.7

- (A) 将不动点定理应用于全可计算函数 $f(x) = x + 7$ 。
- (B) 将不动点定理应用于全可计算函数 $f(x) = 2x$ 。

II.9.8 无论我们使用什么样的编号方案 (只要它是可接受的), 总存在一台 Turing 机, 它计算的函数与索引比它大 5 的那台机器相同, 也就是说, 存在某个 $k \in \mathbb{N}$ 使得 $\phi_k = \phi_{k+5}$ 。

自然的推广是: 对于任何可接受的编号方案以及任何常数 $c \in \mathbb{N}$, 都存在一台 Turing 机, 它计

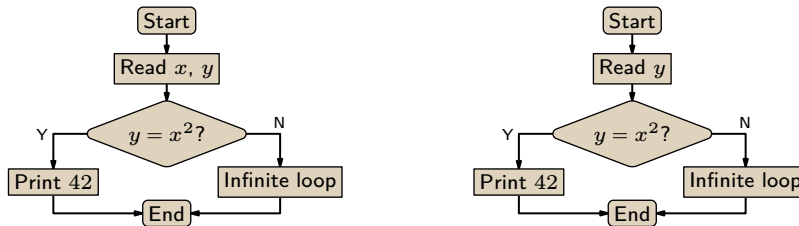


图 52, FOR QUESTION II.9.10: 勾勒出计算 $\psi = \phi_{e_0}$ 的机器和计算 $\phi_{s(e_0, x)}$ 的机器的流程图。

算的函数与索引比它大 c 的那台机器相同。即存在一个 $k \in \mathbb{N}$, 使得 $\phi_k = \phi_{k+c}$ 。

II.9.9

- (A) 对于任何 (可接受的) Turing 机编号方案, 都存在一台机器, 它计算的函数与索引为其三倍的那台机器所计算的函数相同。即, 存在 $k \in \mathbb{N}$ 使得 $\phi_k = \phi_{3k}$ 。
- (B) 对于任何可接受编号方案, 都存在一台 Turing 机, 它计算的函数与索引为其平方的那台机器所计算的函数相同。即, 存在 $k \in \mathbb{N}$ 使得 $\phi_k = \phi_{k^2}$ 。
- (C) 令 $f: \mathbb{N} \rightarrow \mathbb{N}$ 为此函数。

$$f(x) = \begin{cases} 1 & \text{若 } x = 5 \\ 0 & \text{其他情形} \end{cases}$$

对于 Turing 机的任何可接受编号, 都将存在 $k \in \mathbb{N}$ 使得 $\phi_k = \phi_{f(k)}$ 。(例如, 根据填充引理, 存在许多满足 $\phi_j = \phi_0$ 的索引 j 。)

- (D) 若该函数为 $f(x) = 42$, 则不动点定理表明, 对于任何可接受编号, 都存在 $k \in \mathbb{N}$ 使得 $\phi_k = \phi_{f(k)} = \phi_{42}$ 。这并不有趣, 因为我们根据填充引理早已知道存在无穷多个这样的索引 k 。

II.9.10

- (A) 考虑下方左侧的函数。它由 Figure 52 on page 93 左侧草图所示的程序计算。根据 Church 论题, 存在一台具有此行为的 Turing 机。令该机器的索引为 e_0 。

$$\psi(x, y) = \begin{cases} 42 & \text{若 } y = x^2 \\ \uparrow & \text{其他情形} \end{cases} \quad \phi_{s(e_0, x)}(y) = \begin{cases} 42 & \text{若 } y = x^2 \\ \uparrow & \text{其他情形} \end{cases}$$

应用 s - m - n 定理, 得到右侧的函数族。

- (B) 数 e_0 是固定的, 因为它是计算 ψ 的 Turing 机的索引。因此定义 $g: \mathbb{N} \rightarrow \mathbb{N}$ 为 $g(x) = s(e_0, x)$ 。
- (C) 函数 g 是全可计算的, 因为 s 是全可计算的。对 g 应用不动点定理, 得到 $m \in \mathbb{N}$ 满足 $\phi_m = \phi_{g(m)} = \phi_{s(e_0, m)}$ 。根据构造, 最后一个函数仅在输入 $y = m^2$ 时停机。因此 $W_m = \{m^2\}$ 。

II.9.11

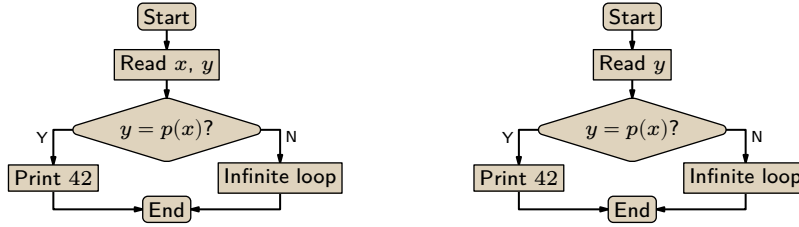
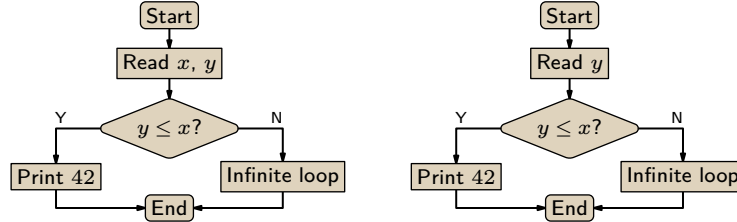
- (A) 我们可以用标准编程语言 (如 Racket) 编写一个程序, 输入 n 并输出第 n 个素数。

```
(require math/number-theory)
> (nth-prime 5)
13
> (nth-prime 666)
4987
```

- (B) 考虑下方左侧的函数, 它由 Figure 53 on page 94 左侧草图所示的机器计算。根据 Church 论题, 该函数有一个索引; 令此索引为 e_0 。

$$\psi(x, y) = \begin{cases} 42 & \text{若 } y = p(x) \\ \uparrow & \text{其他情形} \end{cases} \quad \phi_{s(e_0, x)}(y) = \begin{cases} 42 & \text{若 } y = p(x) \\ \uparrow & \text{其他情形} \end{cases}$$

应用 s - m - n 定理, 得到右侧以 x 为参数的一致可计算函数族。注意到 $\mathcal{P}_{s(e_0, x)}$ 仅在第 x 个素数输

图 53, FOR QUESTION II.9.11: 勾勒出仅在输入为第 m 个素数时才停机的机器 $\mathcal{P}_m$ 的流程图。图 54, FOR QUESTION II.9.14: $\psi = \phi_{e_0}$ 的机器草图, 以及 $\phi_{s(e_0, x)}$ 的机器草图。

入时停机。

- (c) 数 e_0 是固定的, 所以 $s(e_0, x)$ 仅是变量 x 的函数。为强调这一点, 定义 $g: \mathbb{N} \rightarrow \mathbb{N}$ 为 $g(x) = s(e_0, x)$ 。不动点定理给出了一个 $m \in \mathbb{N}$ 满足 $\phi_m = \phi_{g(m)} = \phi_{s(e_0, m)}$, 而该函数仅在输入是第 m 个素数时停机。

II.9.12 考虑下面左侧的函数。根据 Church 论题, 它是可计算的, 因此有一个索引; 设该索引为 e_0 。

$$\psi(x, y) = \begin{cases} 42 & \text{若 } y = 10^x \\ \uparrow & \text{其他情形} \end{cases} \quad \phi_{s(e_0, x)}(y) = \begin{cases} 42 & \text{若 } y = 10^x \\ \uparrow & \text{其他情形} \end{cases}$$

应用 s - m - n 定理, 得到右侧的一致可计算函数族。观察到 $\phi_{s(e_0, x)}$ 仅在 $y = 10^x$ 时收敛。数 e_0 是固定的, 因此定义 $g: \mathbb{N} \rightarrow \mathbb{N}$ 为 $g(x) = s(e_0, x)$ 。不动点定理给出 $m \in \mathbb{N}$ 使得 $\phi_m = \phi_{g(m)} = \phi_{s(e_0, m)}$, 于是 $W_m = \{10^m\}$ 。

II.9.13 考虑函数 $f: \mathbb{N} \rightarrow \mathbb{N}$, 定义为 $f(x) = y$, 其中 y 的每一位数字都比 x 的对应位数字大一, 除了 x 中的 9 变为 y 中的 0。因此 $f(35) = 46$, $f(29) = 30$, $f(99) = 0$ 。显然这个函数是可计算的全函数。应用不动点定理。

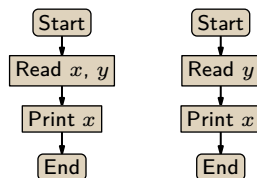
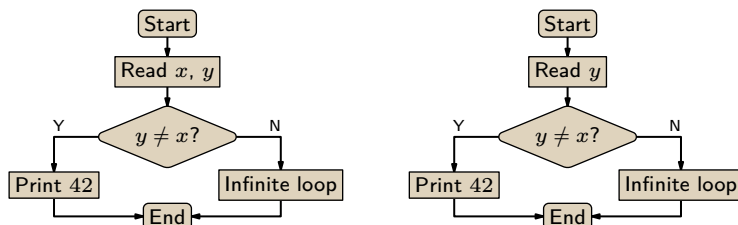
II.9.14 考虑下面左侧的函数。它由 Figure 54 on page 94 左侧草图所示的机器计算, 因此根据 Church 论题, 该机器有一个 Turing 机索引。设该索引为 e_0 。

$$\psi(x, y) = \begin{cases} 42 & \text{若 } y \leq x \\ \uparrow & \text{其他情形} \end{cases} \quad \phi_{s(e_0, x)}(y) = \begin{cases} 42 & \text{若 } y \leq x \\ \uparrow & \text{其他情形} \end{cases} \quad (*)$$

应用 s - m - n 定理得到右侧的机器族, 它们计算 (*) 式右边的函数族。观察到 $\phi_{s(e_0, x)}$ 仅对小于等于 x 的输入 y 收敛。

因为 e_0 是左侧机器的索引, 所以它是固定的。因此 $s(e_0, x)$ 不是两变量函数, 而是单变量 x 的函数。定义 $g: \mathbb{N} \rightarrow \mathbb{N}$ 为 $g(x) = s(e_0, x)$ 。那么 g 是一个一元可计算全函数。由不动点定理 9.1, 函数 g 有一个不动点, 即存在 $n \in \mathbb{N}$ 使得 $\phi_n = \phi_{g(n)} = \phi_{s(e_0, n)}$ 。于是 $W_n = \{0, 1, \dots, n\}$ 。

II.9.15 参见 Figure 55 on page 95 中的流程图。左边是一个输出其第一个输入的计算, 即 $\phi_{e_0}(x, y) = x$ 。根据 Church 论题, 存在一台具有该行为的 Turing 机; 设其索引为 e_0 。右边我们应用了 s - m - n 定理, 对第一个输入进行参数化, 得到 $\phi_{s(e_0, x)}(y) = x$ 。现在令 $f(x) = s(e_0, x)$ 。它是全且可计算的, 因

图 55, FOR QUESTION II.9.15: ϕ_{e_0} 和 $\phi_{s(e_0, x)}$ 的流程图, 对应输出自身索引的机器。图 56, FOR QUESTION II.9.17: $\psi = \phi_{e_0}$ 及 $\phi_{s(e_0, x)}$ 的示意图。

此不动点定理适用。于是存在自然数 k 使得 $\phi_k = \phi_{f(k)} = \phi_{s(e_0, k)}$ 。由于对所有 y 有 $\phi_{s(e_0, k)}(y) = k$, 证毕。

II.9.16 是的, 我们可以取 $f(x) = x$ 。

II.9.17

(A) 下面左侧的函数由 Figure 56 on page 95 左侧草图所示的机器计算。根据 Church 论题, 它有一个索引; 记该索引为 e_0 。

$$\psi(x, y) = \begin{cases} 42 & \text{若 } y \neq x \\ \uparrow & \text{其他情形} \end{cases} \quad \phi_{s(e_0, x)}(y) = \begin{cases} 42 & \text{若 } y \neq x \\ \uparrow & \text{其他情形} \end{cases}$$

s - m - n 定理对 x 进行参数化, 得到右侧的函数族。数 e_0 是固定的, 所以 $s(e_0, x)$ 是一个单变量函数。因此, 定义 $g: \mathbb{N} \rightarrow \mathbb{N}$ 为 $g(x) = s(e_0, x)$ 。由不动点定理, 存在 $m \in \mathbb{N}$ 满足 $\phi_m = \phi_{g(m)} = \phi_{s(e_0, m)}$, 于是 $W_m = \mathbb{N} - \{m\}$ 。

(B) 这样的可计算枚举集 W_m 不存在。根据定义, W_m 是集合 $\{x \mid \phi_m(x) \downarrow\}$ 。它不可能同时也是集合 $\{x \mid \phi_m(x) \uparrow\}$ 。

II.9.18

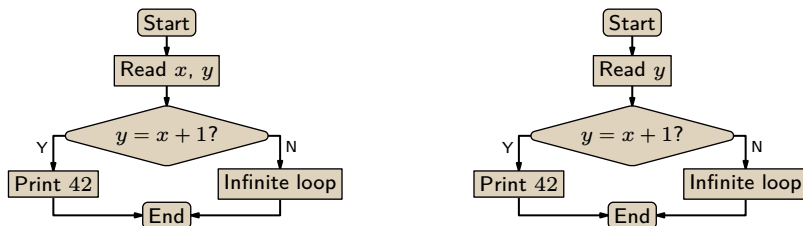
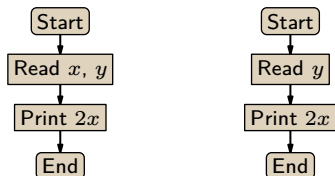
(A) 考虑函数 $\psi: \mathbb{N}^2 \rightarrow \mathbb{N}$, 定义为: 若 $y = x + 1$ 则 $\psi(x, y) = 42$, 否则 $\psi(x, y) \uparrow$ 。该函数由 Figure 57 on page 96 左侧的机器计算。因此, 根据 Church 论题, 存在具有该行为的 Turing 机。记该机器为 $\mathcal{P}_{e_0}$ 。应用 s - m - n 定理得到右侧的机器, 它计算函数 $\phi_{s(e_0, x)}(y)$, 其性质是仅在 $x + 1$ 上收敛。令 $g: \mathbb{N} \rightarrow \mathbb{N}$ 由 $g(x) = s(e_0, x)$ 给出。由不动点定理, 存在 $m \in \mathbb{N}$ 满足 $\phi_m = \phi_{g(m)} = \phi_{s(e_0, m)}$, 该函数的定义域为 $\{m + 1\}$ 。

(B) 考虑函数 $\psi(x, y) = 2x$ 。它由 Figure 58 on page 96 左侧草图所示的机器计算。因此根据 Church 论题, 存在具有此行为的 Turing 机。记其索引为 e_0 。应用 s - m - n 定理得到该图右侧的机器族。定义 $g: \mathbb{N} \rightarrow \mathbb{N}$ 为 $g(x) = s(e_0, x)$ (数 e_0 是一个常数, 所以 $s(e_0, x)$ 是一个单变量函数)。由不动点定理, 得到一个数 m 满足 $\phi_m = \phi_{g(m)}$, 由于 $\phi_{g(m)}$ 的值域是单元素集 $\{2m\}$, 即证。

II.9.19 由推论 9.3, 存在满足此条件的索引 e 。

$$\phi_e(y) = \begin{cases} \downarrow & \text{若 } y = e \\ \uparrow & \text{其他情形} \end{cases} \quad (*)$$

这意味着 $e \in K$ 。

图 57, FOR QUESTION II.g.18: 定义域为 $\{m+1\}$ 的函数 ϕ_m 的流程图。图 58, FOR QUESTION II.g.18: 值域为 $\{2m\}$ 的函数 ϕ_m 的流程图。

现在, 根据 Padding 引理, 存在 $\hat{e} \neq e$ 使得 $\phi_{\hat{e}} \simeq \phi_e$ 。于是 (*) 给出 $\phi_{\hat{e}}(\hat{e}) \uparrow$, 因此 $\hat{e} \neq K$ 。所以 K 不是索引集。

II.9.20

- (A) 集合 F 是有限的, 所以由 F 中索引计算的函数集合是有限的。但存在无穷多个偏可计算函数, 因此必然存在不被 F 中任何索引计算的函数。
- (B) 对于不动点 x , 有两种可能。如果 $x \in F$, 则 $h(x) = e_0$ 。由于 e_0 是 g 的索引, 而 g 在 F 中没有索引, 不可能 $\phi_{h(x)} = \phi_{e_0} = \phi_x$ 。

另一种情况是 $x \notin F$ 。那么 $\phi_{h(x)} = \phi_{f(x)}$, 但因为 $x \notin F$, 所以 $\phi_{f(x)} \neq \phi_x$, 因此 $\phi_{h(x)} \neq \phi_x$ 。

II.A.1 一种方案是将巴士 B_0 的乘客安排到房间 0、100、200 等等。然后将巴士 B_1 的乘客安排到房间 1、101、201 等等。一般地, 将 $b_{i,j}$ 安排到房间 $i + 100j$ 。

II.A.2 首先移动现有的客人, 将现在住在房间 n 的人安排到房间 $2n$ 。这样就空出了奇数编号的房间。然后, 巴士 B_i 的第 j 个人住进 $f(p_{i,j}) = 2 \cdot \text{cantor}(i, j) + 1$ 。

II.A.3 每个人是一个三元组, $\langle i, j, k \rangle = \langle \text{楼层号}, \text{巴士车位号}, \text{该巴士上的人员号} \rangle$ 。分配房间的自然方式是腾出奇数号房间, 然后将 $p_{i,j,k}$ 放入 $2 \cdot \text{cantor}(\text{cantor}(i, j), k) + 1$ 。

II.A.4 不, 幂集 $\mathcal{P}(\mathbb{R})$ 的成员比 $\mathbb{R}$ 多。

II.D.3 函数 Δ 有 $n \cdot 2$ 个输入和 $(n+1) \cdot 2 \cdot 2$ 个输出。所以一共有 $(2n)^{4(n+1)}$ 台机器。

II.D.4 Figure 59 on page 97 中的 Racket 代码可以计算它。这些值是 6, 16, 34, 64, 114, 196, 334, 564, 946, 1584, 2646, 4416, 7366, 和 12284。

II.D.5 本题并不是要求我们构造出一个可计算函数; 事实上, 不存在这样的可计算函数。

如果 $\phi_0(0)$ 收敛, 定义 $f(0) = 1 + \phi_0(0)$, 否则定义 $f(0) = 1$ 。(注意 f 不是一个可计算函数。)类似地定义 $f(1)$ 为 $1 + f(0)$, $1 + \phi_0(1)$ 和 $1 + \phi_1(1)$ 的最大值, 其中如果后两项中任一项发散, 则不将其纳入最大值。对所有 n 同样处理。

更形式化地, 定义 $F(e, x)$ 为 $\phi_e(x)$ (如果它收敛), 否则为 0。那么 $f(i)$ 是 $\{F(0, i), \dots, F(i, i)\}$ 的最大值再加 1。

II.D.6 根据提示, 注意到输出在集合 $\{0, 1\}$ 中的函数有不可数多个, 即每个输出都小于或等于 1。Turing 机只有可数多个, 所以可计算函数只有可数多个, 因此输出以 1 为界的可计算函数也只有可数多个。故存在输出以 1 为界的不可计算函数。

II.D.7

- (A) 最好用一个例子来回答: 这里是 $\mathcal{M}_4$ 。

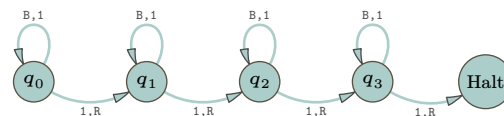

```

(define (g n)
  (let ([i (modulo n 3)])
    (cond
      [(= i 0) (/ (+ (* n 5) 18) 3)]
      [(= i 1) (/ (+ (* n 5) 22) 3)]
      [else -1])))

; Show values computed while finding Busy Beaver 5.
; r values; usually initially use r=0
; N cap on number of values to compute; use N=10 for instance
(define (BB5 r N)
  (let ([val (g r)])
    (if (> val 0)
      (begin
        (writeln (number->string val))
        (BB5 val (- N 1)))
      (writeln "done"))))

```

图 59, FOR QUESTION II.D.4: 在计算 BB(5) 过程中发现各值的 Racket 代码。



- (B) 第一个性质成立是因为 $f(m)$ 是 $F(m)$ 表达式中的一项；类似地，第二个性质成立是因为 m^2 也是一项。第三个性质成立是因为当 $m > 0$ 时所有项都是正数。
- (C) 如果从空白纸带开始, 这台机器首先产生 j 个 1, 然后产生 $F(j)$ 个 1, 最终在纸带上留下 $F(F(j))$ 个 1。因此其生产力为 $F(F(j))$ 。
- (D) 由于 F 严格递增, $F(j + 2n_F) < F(F(j))$ 。结合得到 $f(j + 2n_F) < F(j + 2n_F) < F(F(j)) \leq \Sigma(j + 2n_F)$, 如所需。

II.E.1

```

> (cantor-pairing 42)
'(6 2)

```

II.E.2

```

> (cantor-pairing 666)
'(0 36)

```

II.E.3

```

> (cantor-unpairing 3 0)
9
> (for ([i '(0 1 2 3 4 5 6 7 8 9)])
  (display (cantor-pairing-omega i)) (newline))
()
(0)
(0 0)
(1)
(0 1)
(0 0 0)
(2)
(1 0)
(0 0 1)
(0 0 0 0)

```


Chapter III: 语言、文法与图

III.1.18

- (A) 五个串是：000、001、010、011 和 100。
(B) 这五个满足条件：0、1、00、11 和 000。（空串属于这个语言吗？这可能存在争议，但可以肯定的是，空串的首尾字符并不不同。）

III.1.19 不是。根据定义，语言是串的集合，而根据定义，串的长度是有限的。对于一些实数，其十进制表示不能构成有限长的串；例如 $\pi = 3.14\dots$ ，再比如 $1/3 = 0.33\dots$ 。

III.1.20 两个都是。

III.1.21 如果 $\beta = \langle b_0, \dots, b_{i-1} \rangle$ 是一个串，那么 $\beta \cap \beta^R = \langle b_0, \dots, b_{i-1} \rangle \cap \langle b_{i-1}, \dots, b_0 \rangle$ 显然是一个回文。（这对空串也成立。）存在不是这种形式的回文。例如 $\gamma = 010$ ，它不能分解为 $\gamma = \beta \cap \beta^R$ ，因为它的长度是奇数。

III.1.22

- (A) 这些是 $\mathcal{L}_0 \cap \mathcal{L}_1$ 的成员，其中第一行的串以空串开头，第二行的串以单个 **a** 开头，依此类推。

$\varepsilon, \mathbf{b}, \mathbf{bb}, \mathbf{bbb},$
 $\mathbf{a}, \mathbf{ab}, \mathbf{abb}, \mathbf{abbb}$
 $\mathbf{aa}, \mathbf{aab}, \mathbf{aabb}, \mathbf{aabbb}$
 $\mathbf{aaa}, \mathbf{aaab}, \mathbf{aaabb}, \mathbf{aaabbb}$

- (B) 这些是 $\mathcal{L}_1 \cap \mathcal{L}_0$ 的成员。

$\varepsilon, \mathbf{a}, \mathbf{aa}, \mathbf{aaa},$
 $\mathbf{b}, \mathbf{ba}, \mathbf{baa}, \mathbf{baaa}$
 $\mathbf{bb}, \mathbf{bba}, \mathbf{bbaa}, \mathbf{bbaaa}$
 $\mathbf{bbb}, \mathbf{bbba}, \mathbf{bbbba}, \mathbf{bbbbaa}$

- (C) 我们有如下结果。

$$\mathcal{L}_0^2 = \mathcal{L}_0 \cap \mathcal{L}_0 = \{\emptyset, \mathbf{a}, \mathbf{aa}, \mathbf{aaa}, \mathbf{aaaa}, \mathbf{aaaaa}, \mathbf{aaaaaa}\}$$

- (D) 正确答案有很多，但最自然的一组十个成员是： ε 、**a**、**aa**、 $\dots\dots$ 、 $\mathbf{a}^9$ 。

III.1.23

- (A) 该语言由任意多个 **a** 后接单个 **b** 构成。因此五个成员是 **b**、**ab**、**aab**、**aaab** 和 **aaaab**。
(B) 该语言由若干个 **a** 后接相同数量的 **b** 构成。因此五个成员是 ε 、**ab**、**aabb**、**aaabbb** 和 **aaaabbbb**。
(C) 该语言是由一串 **1** 后接一串 **0** 构成的串的集合，其中 **0** 的数量比 **1** 的数量多一。五个成员是 **0**、**100**、**11000**、**1110000** 和 **111100000**。
(D) 该语言由 $\mathbb{B}^*$ 中的串构成，这些串先是一串 **1**，接着是一串 **0**，最后是一个 **1**。其中，**0** 的数量是前导 **1** 数量的两倍。五个成员是 **1**、**1001**、**1100001**、**1³0⁶1**，和 **1⁴0⁸1**。

III.1.24

- (A) $\mathcal{L}^2 = \{\mathbf{aa}, \mathbf{aab}, \mathbf{aba}, \mathbf{abab}\}$
(B) 我们先将集合中加入以 **aa** 开头的串，然后是以 **aab** 开头的串，以此类推。结果是 $\mathcal{L}^3 = \{\mathbf{aaa}, \mathbf{aaab}, \mathbf{aaba}, \mathbf{aabab}, \mathbf{abaa}, \mathbf{ababab}, \mathbf{abababab}, \mathbf{ababababab}\}$
(C) $\mathcal{L}^1 = \mathcal{L} = \{\mathbf{a}, \mathbf{ab}\}$

(D) $\mathcal{L}^0 = \{\varepsilon\}$

III.1.25

(A) $\mathcal{L}_0 \cap \mathcal{L}_1 = \{\text{ab}, \text{abb}, \text{abb}, \text{abbb}\}$

(B) $\mathcal{L}_1 \cap \mathcal{L}_0 = \{\text{ba}, \text{bab}, \text{bba}, \text{bbab}\}$

(C) $\mathcal{L}_0^2 = \{\text{aa}, \text{aab}, \text{aba}, \text{abab}\}$

(D) $\mathcal{L}_1^2 = \{\text{bb}, \text{bbb}, \text{bbbb}\}$

(E) $\mathcal{L}_0^2 \cap \mathcal{L}_1^2 = \{\text{aabb}, \text{aabbb}, \text{aabbbb}, \text{aabbbbb}, \text{ababb}, \text{ababbb}, \text{ababbbb}, \text{ababbbbb}\}$

III.1.26

(A) 最小可能值是三，当 $\mathcal{L}_1$ 是 $\mathcal{L}_0$ 的子集时取到。最大可能值是五，当两个语言互不相交时取到。

(B) 最小可能值是零，当两个语言互不相交时取到。最大可能值是二，当 $\mathcal{L}_1$ 是 $\mathcal{L}_0$ 的子集时取到。

(C) 最大可能值是六，例如当 $\mathcal{L}_0 = \{\text{a}, \text{b}, \text{c}\}$ 且 $\mathcal{L}_1 = \{\text{x}, \text{y}\}$ 时，得到 $\mathcal{L}_0 \cap \mathcal{L}_1 = \{\text{ax}, \text{ay}, \text{bx}, \text{by}, \text{cx}, \text{cy}\}$ 。

最小可能值是四，例如当 $\mathcal{L}_0 = \{\varepsilon, \text{a}, \text{aa}\}$ 且 $\mathcal{L}_1 = \{\varepsilon, \text{a}\}$ 时，得到 $\mathcal{L}_0 \cap \mathcal{L}_1 = \{\varepsilon, \text{a}, \text{aa}, \text{aaa}\}$ 。

(D) 最小可能值是三，例如当 $\mathcal{L}_1 = \{\varepsilon, \text{a}\}$ 时，得到 $\mathcal{L}_1^2 = \{\varepsilon, \text{a}, \text{aa}\}$ 。最大可能值是四，例如当 $\mathcal{L}_1 = \{\text{a}, \text{b}\}$ 时，得到 $\mathcal{L}_1^2 = \{\text{aa}, \text{ab}, \text{ba}, \text{bb}\}$ 。

(E) 唯一可能的数量是 2。 $\mathcal{L}_1$ 中的两个元素反转后不可能成为同一个元素，否则它们在反转前就是同一个元素了。（即使其中一个元素是空串，此结论也成立。）

(F) 交集可以是无限的，例如当 $\mathcal{L}_0 = \{\varepsilon, \text{a}, \text{b}\}$ 且 $\mathcal{L}_1 = \{\varepsilon, \text{a}\}$ 时，其中 $\text{a}^n \in \mathcal{L}_0^* \cap \mathcal{L}_1^*$ 对所有的 $n \in \mathbb{N}$ 都成立。交集的大小最小可以是 1，例如当 $\mathcal{L}_0 = \{\text{a}, \text{b}, \text{c}\}$ 且 $\mathcal{L}_1 = \{\text{x}, \text{y}\}$ 时，此时 $\mathcal{L}_0^* \cap \mathcal{L}_1^*$ 的唯一元素是空串。

III.1.27 它是空集，即 $\emptyset^* = \emptyset$ 。根据定义， $\mathcal{L}^* = \{\sigma_0 \cap \dots \cap \sigma_{k-1} \mid k \in \mathbb{N} \text{ 且 } \sigma_0, \dots, \sigma_{k-1} \in \mathcal{L}\}$ 。如果该语言为空，那么“ $\sigma_0, \dots, \sigma_{k-1} \in \mathcal{L}$ ”这一条件永远无法满足。

III.1.28 不相等。设 $\mathcal{L} = \{\text{a}, \text{b}\}$ 并设 $k = 2$ 。那么该语言的 2 次幂是 $\mathcal{L}^k = \{\text{aa}, \text{ab}, \text{ba}, \text{bb}\}$ ，而由 2 次幂组成的语言是 $\{\sigma^2 \mid \sigma \in \mathcal{L}\} = \{\text{aa}, \text{bb}\}$ 。

III.1.29 是，但仅在一个边界情形下不同。如果 $\mathcal{L} = \emptyset$ ，那么 $\mathcal{L}^* = \emptyset$ ，而 $(\mathcal{L} \cup \{\varepsilon\})^* = \{\varepsilon\}$ 。

我们将证明，如果 $\mathcal{L} \neq \emptyset$ ，那么这两个集合相等，即 $\mathcal{L}^* = (\mathcal{L} \cup \{\varepsilon\})^*$ 。包含关系 $\mathcal{L}^* \subseteq (\mathcal{L} \cup \{\varepsilon\})^*$ 是显然的。

对于另一个方向，即要证 $(\mathcal{L} \cup \{\varepsilon\})^* \subseteq \mathcal{L}^*$ ，假设 $\sigma \in (\mathcal{L} \cup \{\varepsilon\})^*$ ，其中 $\sigma = \sigma_0 \cap \dots \cap \sigma_{n-1}$ 对某个 $n \in \mathbb{N}$ 成立，且所有的 σ_i 都是 $\mathcal{L} \cup \{\varepsilon\}$ 中的元素。如果对所有 $i \in \{0, \dots, n-1\}$ 都有串 σ_i 等于 ε ，那么 $\sigma = \varepsilon$ ，并且它属于 $\mathcal{L}^*$ ，因为 $\mathcal{L} \neq \emptyset$ 且对任意 $\rho \in \mathcal{L}$ 都有 $\varepsilon = \rho^0$ 。否则，至少存在一个 $i \in \{0, \dots, n-1\}$ 使得 $\sigma_i \neq \varepsilon$ 。在这种情况下，从串 σ 中去掉那些等于空串的 σ_j ，可得对某个 $k \leq n$ 有 $\sigma \in \mathcal{L}^k$ ，因此 $\sigma \in \mathcal{L}^*$ 。

III.1.30 对任意语言，我们记其字母表为 Σ 。

(A) 考虑两个不相等的串 $\sigma_0 \neq \sigma_1$ 。它们不可能都是空串；如果其中一个是空串而另一个不是，那么显然 $\sigma_0^2 \neq \sigma_1^2$ ，因为这两个平方中有一个是空串而另一个不是。现在假设 σ_0 和 σ_1 都不是空串。如果它们长度不同，那么它们的平方长度也不同。剩下的情况是它们为非空且等长的串，即 $\sigma_0 = \langle s_{0,0}, s_{0,1}, \dots, s_{0,n} \rangle$ 且 $\sigma_1 = \langle s_{1,0}, s_{1,1}, \dots, s_{1,n} \rangle$ 对某个 $n \in \mathbb{N}$ 成立。取定最小的索引 i 使得 $s_{0,i} \neq s_{1,i}$ 。对于这个相同的 i ， σ_0^2 和 σ_1^2 在第 i 个位置不同，因此它们不相等。

上一段表明，如果 $\mathcal{L}$ 有 k 个不同的元素 $\mathcal{L} = \{\sigma_0, \dots, \sigma_{k-1}\}$ ，那么串 $\sigma_0^2, \dots, \sigma_{k-1}^2$ 互不相同，因此 $\mathcal{L}^2$ 至少有 k 个不同的元素。

(B) 语言 $\mathcal{L} = \{\varepsilon\}$ 满足 $\mathcal{L}^2 = \{\varepsilon\}$ 。

(C) 设 $\mathcal{L} = \{\sigma_0, \sigma_1, \dots, \sigma_{k-1}\}$ ，其中 $k \in \mathbb{N}^+$ （我们假定这些串彼此互不相同）。那么列表 $\sigma_0 \cap \sigma_0, \sigma_0 \cap \sigma_1, \dots, \sigma_{k-1} \cap \sigma_{k-1}$ 的长度为 k^2 。因此， $\mathcal{L}^2$ 可能具有的最大元素个数是 k^2 。

(D) 考虑语言 $\mathcal{L} = \{\sigma_0, \sigma_1, \dots, \sigma_{k-1}\}$ ，其中这 k 个串长度均为 1 且互不相等，因此每个串由单个符号组成，且其使用的符号与任何其他串所使用的符号都不相同。显然，列表 $\sigma_0 \cap \sigma_0, \sigma_0 \cap \sigma_1, \dots, \sigma_{k-1} \cap \sigma_{k-1}$ 中的所有成员都互不相同，所以 $\mathcal{L}^2$ 有 k^2 个元素。

III.1.31 回顾 $\mathcal{L}^k = \{\sigma_0 \cap \dots \cap \sigma_{k-1} \mid \sigma_i \in \mathcal{L}\}$ 以及 $\mathcal{L}^* = \{\sigma_0 \cap \dots \cap \sigma_{k-1} \mid k \in \mathbb{N} \text{ 且 } \sigma_0, \dots, \sigma_{k-1} \in \mathcal{L}\}$ 。如果 $\mathcal{L} = \emptyset$ ，那么根据定义有 $\mathcal{L}^* = \{\varepsilon\}$ 。如果 $\mathcal{L} = \emptyset$ ，那么根据定义有 $\mathcal{L}^0 = \{\varepsilon\}$ ，而对任意 $k > 0$ ，都有 $\mathcal{L}^k = \emptyset$ 。

因此剩下的就是论证 $\mathcal{L} \neq \emptyset$ 的情况。我们先证明 $\mathcal{L}^k \subseteq \mathcal{L}^*$ 。如果 $k = 0$ ，因为我们假设 $\mathcal{L} \neq \emptyset$ ，可以取 $\sigma \in \mathcal{L}$ ，则有 $\sigma^0 = \varepsilon$ 。因此 $\mathcal{L}^0 = \{\varepsilon\}$ 且 $\varepsilon \in \mathcal{L}^*$ 。现在假设 $k > 0$ ，并取定 $\sigma \in \mathcal{L}^k$ ，满足 $\sigma = \sigma_0 \hat{\ } \dots \hat{\ } \sigma_{k-1}$ 。那么根据 $\mathcal{L}^*$ 的定义，有 $\sigma \in \mathcal{L}^*$ 。

最后我们证明 $\mathcal{L}^* \subseteq \mathcal{L}^0 \cup \mathcal{L}^1 \cup \dots$ 。取定 $\sigma \in \mathcal{L}^*$ （注意 $\mathcal{L} \neq \emptyset$ ），于是存在某个 $k \in \mathbb{N}$ 使得 $\sigma = \sigma_0 \hat{\ } \dots \hat{\ } \sigma_{k-1}$ 。注意 $\sigma \in \mathcal{L}^k$ （即使 $k = 0$ 也成立）。因此 $\mathcal{L}^* \subseteq \mathcal{L}^0 \cup \mathcal{L}^1 \cup \dots$ 。

III.1.32 所有形如 $\sigma_1 \hat{\ } \sigma_0$ （其中 $\sigma_1 \in \mathcal{L}_1$ 且 $\sigma_0 \in \mathcal{L}_0$ ）的拼接所构成的集合是空集，因为 $\mathcal{L}_0$ 中不存在成员 σ_0 。

III.1.33

- (A) 它们是 ε 、 a 、 aa 、 b 以及 bb 。
- (B) 并集的元素个数等于第一个语言的元素个数加上第二个语言的元素个数。因为字母表不相交，所以两个语言之间没有重叠。
- (C) 并集中的串可能包含来自任一字母表的字符。因此，一个在 Σ_0 上的语言与一个在 Σ_1 上的语言的并集，是一个在 $\Sigma_0 \cup \Sigma_1$ 上的语言。
- (D) 要属于这两个语言的交集，一个串必须只包含同时属于这两个字母表的字符。因此，一个在 Σ_0 上的语言与一个在 Σ_1 上的语言的交集，是一个在 $\Sigma_0 \cap \Sigma_1$ 上的语言。

III.1.34

- (A) 是的，但这与语言有限无关。根据我们的定义，字母表是有限的。（附录 A 有相关回顾。）
- (B) 因为语言有限，要么语言为空，此时可取 $B = 0$ ；要么语言中存在最长串，可取 B 为该串的长度。
- (C) 设 $\mathcal{L}_0, \dots, \mathcal{L}_{k-1}$ 是 Σ^* 上的有限语言（ $k \in \mathbb{N}$ ）。则它们的并也是有限的，因为它至多包含 $|\mathcal{L}_0| + \dots + |\mathcal{L}_{k-1}|$ 个元素。
- (D) 设 $\mathcal{L}_0, \dots, \mathcal{L}_{k-1}$ 是 Σ^* 上的有限语言。则它们的交至多包含 $\min(|\mathcal{L}_0|, \dots, |\mathcal{L}_{k-1}|)$ 个元素，因此也是有限的。拼接 $\mathcal{L}_0 \hat{\ } \mathcal{L}_1 \hat{\ } \dots \hat{\ } \mathcal{L}_{k-1}$ 中的串数至多是乘积 $|\mathcal{L}_0| \cdot |\mathcal{L}_1| \cdot \dots \cdot |\mathcal{L}_{k-1}|$ 。
- (E) 如果 $\Sigma = \{a, b\}$ ，则有限语言 $\mathcal{L} = \{\}$ 的补集是 $\mathcal{L}^c = \Sigma^*$ ，它是无限的，因为它对所有 n 都包含 a^n 。对于同一字母表， $\mathcal{L} = \{a, b\}$ 是有限的，但 $\mathcal{L}^*$ 是无限的。

III.1.35 在 $\hat{\mathcal{L}}$ 中，所有串的长度均为偶数，因此它包含所有偶数长度的回文。语言 $\mathcal{L}$ 则包含任意长度的所有回文。

III.1.36 前缀为 ε , a , b , ab , bb , aba , bba , $abaa$, $abaab$, 以及 $abaaba$ 。

III.1.37

- (A) 将 $\sigma_0 \hat{\ } \dots \hat{\ } \sigma_{m-1} \in \mathcal{L}^m$ 与 $\tau_0 \hat{\ } \dots \hat{\ } \tau_{n-1} \in \mathcal{L}^n$ 拼接得到 $\sigma_0 \hat{\ } \dots \hat{\ } \sigma_{m-1} \hat{\ } \tau_0 \hat{\ } \dots \hat{\ } \tau_{n-1} \in \mathcal{L}^{m+n}$ 。
- (B) 根据定义， $\mathcal{L}_0 \hat{\ } \mathcal{L}_1 = \{\sigma_0 \hat{\ } \sigma_1 \mid \sigma_0 \in \mathcal{L}_0 \text{ 且 } \sigma_1 \in \mathcal{L}_1\}$ 。如果两者之一是空集，则条件永远不满足，因此结果是空集。
- (C) 对于任意串 $\sigma \in \mathcal{L}^*$ ，我们有 $\sigma^0 = \varepsilon$ 。因此，如果语言中有任何串，那么将它们取零次幂都会得到空串，所以结果是 $\mathcal{L}^0 = \{\varepsilon\}$ 。
- (D) 如果 $\mathcal{L} = \emptyset$ 会使得 $\mathcal{L}^0 = \emptyset$ ，那么对于任意语言 $\hat{\mathcal{L}}$ ，扩展前一项的公式会得到 $\hat{\mathcal{L}} = \hat{\mathcal{L}}^1 = \hat{\mathcal{L}}^{1+0} = \hat{\mathcal{L}} \hat{\ } \emptyset = \emptyset$ （最后的等式由第二项得出），这是不合理的。

III.1.38

- (A) 考虑集合 $S = \Sigma \times \dots \times \Sigma$ （我们避免将其称为 Σ^n ，因为二者不同，Cartesian 积是 n 元组的集合，而 Σ^n 是 Σ^* 的一个子集）。它是可数多个可数集的 Cartesian 积，因此根据第二章的推论 2.9，它是可数的。所以存在单射且满射的函数 $g: \mathbb{N} \rightarrow S$ 。将其自变量拼接起来 $f(\sigma_0, \dots, \sigma_{n-1}) = \sigma_0 \hat{\ } \dots \hat{\ } \sigma_{n-1}$ 的函数 $\text{concat}: S \rightarrow \Sigma^n$ 显然也是满射的。因此复合 $f \circ g: \mathbb{N} \rightarrow \Sigma^n$ 是满射的。那么根据第二章的引理 2.12， Σ^n 是可数的。
- (B) 集合 $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \dots$ 是可数多个可数集的并集，因此根据第二章的推论 2.13，它是一个可数集。

III.1.39 我们需要验证的是：对拼接运算任意两种加括号的方式都产生同一个串。为避免两种形式混淆，仅在此论证中将 $\{\sigma_0 \hat{\ } \dots \hat{\ } \sigma_{k-1} \mid \sigma_i \in \mathcal{L}\}$ 记作 ${}^k\mathcal{L}$ 。

我们首先证明 ${}^k\mathcal{L}$ 的任何元素都是 $\mathcal{L}^k$ 的元素。我们进行一个简单的归纳论证。基础情形是：按定义， ${}^0\mathcal{L}$ 和 $\mathcal{L}^0$ 都等于 $\{\varepsilon\}$ ；并且对于任何 $\sigma_0 \in \mathcal{L}$ ，有 $\sigma_0 \in \mathcal{L}^1$ 和 $\sigma_0 \in {}^1\mathcal{L}$ 。归纳假设是： $\sigma_0 \hat{\ } \dots \hat{\ } \sigma_{i-1} \in {}^i\mathcal{L}$ 蕴含 $\sigma_0 \hat{\ } \dots \hat{\ } \sigma_{i-1} \in \mathcal{L}^i$ ，其中 $i \in \{0, \dots, j\}$ 。对于 $i = j+1$ 的归纳步骤，假设我们有一个由 $j+1$ 个串构成的拼接： $\sigma_0 \hat{\ } \dots \hat{\ } \sigma_{j-1} \hat{\ } \sigma_j \in {}^{j+1}\mathcal{L}$ 。应用串拼接运算的结合律将其重写为 $(\sigma_0 \hat{\ } \dots \hat{\ } \sigma_{j-1}) \hat{\ } \sigma_j$ ，

根据本练习中 $\mathcal{L}^{j+1}$ 的定义, $(\sigma_0 \frown \dots \frown \sigma_{j-1}) \frown \sigma_j \in \mathcal{L}^{j+1}$ 。

另一方面是: 对于任意 k , $\mathcal{L}^k$ 的任何元素都是 ${}^k\mathcal{L}$ 的元素。这里我们也用归纳法。同样地, $k=0$ 和 $k=1$ 的基础情形是: 按定义, ${}^0\mathcal{L}$ 和 $\mathcal{L}^0$ 等于 $\{\varepsilon\}$; 并且对于任何 $\sigma_0 \in \mathcal{L}$, 有 $\sigma_0 \in \mathcal{L}^1$ 和 $\sigma_0 \in {}^1\mathcal{L}$ 。归纳假设是: 如果 $\sigma \in \mathcal{L}^i$, 那么 $\sigma \in {}^i\mathcal{L}$, 其中 $i \in \{0, \dots, k\}$ 。 $\mathcal{L}^j$ 的任何元素都具有形式 $(\sigma_0 \frown \dots \frown \sigma_{j-1}) \frown \sigma_j$ 。利用串拼接的结合律, 我们可以去掉括号将其写为 $\sigma_0 \frown \dots \frown \sigma_{j-1} \frown \sigma_j$ (更准确地说, 任何加括号方式都给出相同的最终串), 这是 ${}^{j+1}\mathcal{L}$ 的一个元素。

III.1.40 命题为真。作为论证的启发, 构造反例时自然会想到的语言是 $\mathcal{L} = \{\mathbf{a}, \mathbf{aa}, \dots\}$ 。然而, $\mathcal{L} \frown \mathcal{L}$ 是 $\mathcal{L}$ 的真子集, 因为它不包含 $\mathbf{a}$ 。

于是, 考虑 $\mathcal{L} \neq \emptyset$ 的情况, 目标是证明 $\varepsilon \in \mathcal{L}$ 。该语言非空, 因此至少包含一个元素。在所有这些串中, 设最短的长度为 $\ell \in \mathbb{N}$ 。如果 $\ell > 0$, 那么 $\mathcal{L} \frown \mathcal{L}$ 的任何元素的长度至少为 2ℓ , 当 $\ell > 0$ 时该长度严格大于 ℓ 。但根据练习假设 $\mathcal{L} \frown \mathcal{L} = \mathcal{L}$, 由此产生矛盾。另一种可能性是 $\ell = 0$, 这意味着 $\varepsilon \in \mathcal{L}$ 。

III.1.41 每个语言都是可数的, 因为它是 Σ^* 的子集, 而根据字母表的定义 Σ 是有限的, 所以 Σ^* 是可数的。但实数有不可数多个。

III.1.42

- (A) 对于任意集合, 无论是否为语言, 并和交都是交换的。
- (B) 我们有 $\mathcal{L} \frown \{\varepsilon\} = \{\sigma \frown \varepsilon \mid \sigma \in \mathcal{L}\} = \{\sigma \mid \sigma \in \mathcal{L}\} = \mathcal{L}$ 。
- (C) 设 $\Sigma = \{\mathbf{a}, \mathbf{b}\}$, 取 $\mathcal{L}_0$ 为单字符串集合 $\{\mathbf{a}\}$, 类似地令 $\mathcal{L}_1 = \{\mathbf{b}\}$ 。那么 $\mathcal{L}_0 \frown \mathcal{L}_1$ 也仅有单个串, 即 $\{\mathbf{ab}\}$, 而 $\mathcal{L}_1 \frown \mathcal{L}_0 = \{\mathbf{ba}\}$ 。
- (D) 这可直接由串的拼接满足结合律得出: $(\sigma_0 \frown \sigma_1) \frown \sigma_2$ 的第 i 个元素与 $\sigma_0 \frown (\sigma_1 \frown \sigma_2)$ 的第 i 个元素相同。
- (E) 这可直接由对于串, $(\sigma_0 \frown \sigma_1)^R$ 等于 $\sigma_1^R \frown \sigma_0^R$ 得出。
- (F) 一个串是 $(\mathcal{L}_0 \cup \mathcal{L}_1) \frown \mathcal{L}_2$ 的元素, 当且仅当它具有 $\sigma \frown \tau$ 形式, 其中 $\tau \in \mathcal{L}_2$, 并且 $\sigma \in \mathcal{L}_0$ 或 $\sigma \in \mathcal{L}_1$ 。这成立, 当且仅当 $\tau \in \mathcal{L}_2$ 且 $\sigma \in \mathcal{L}_0$, 或者 $\tau \in \mathcal{L}_2$ 且 $\sigma \in \mathcal{L}_1$ 。而这又等价于 $\sigma \in (\mathcal{L}_0 \frown \mathcal{L}_2) \cup (\mathcal{L}_1 \frown \mathcal{L}_2)$ 。右分配律的论证同理。
- (G) 对任意语言 $\mathcal{L}$, 其定义为 $\emptyset \frown \mathcal{L} = \{\sigma_0 \frown \sigma_1 \mid \sigma_0 \in \emptyset \text{ 且 } \sigma_1 \in \mathcal{L}\}$ 。条件 “ $\sigma_0 \in \emptyset$ ” 永远无法满足, 因此该集合为空。 $\mathcal{L} \frown \emptyset$ 同理。
- (H) 对于任意字符串, 无论拼接如何加括号, 都可以去掉括号。例如, $((\sigma_0 \frown \sigma_1) \frown \sigma_2) \frown \sigma_3$ 等于 $((\sigma_0 \frown (\sigma_1 \frown \sigma_2)) \frown \sigma_3$, 因为两者的第 i 个元素相等。(我们可以通过归纳法轻易证明该陈述。) 由此可直接得出结论。

III.2.11

- (A) 我们约定开始符号是第一条列出的规则的头部, 因此它是 $\langle \text{expr} \rangle$ 。
- (B) 终结符是 $\mathbf{a}$ 、 $\mathbf{b}$ 、 $\dots$ 、 $\mathbf{z}$ 。
- (C) 非终结符是 $\langle \text{expr} \rangle$ 、 $\langle \text{term} \rangle$ 和 $\langle \text{factor} \rangle$ 。
- (D) 最后一行有二十七条重写规则。另外两行共有四条, 所以总共是三十一条。
- (E) 两个是 $\mathbf{a+b}$ 和 $\mathbf{j*h+k*m}$ 。
- (F) 三个是 $\mathbf{a+}$ 、 $\mathbf{+}$ 和 $\mathbf{**}$ 。

III.2.12

- (A) 根据我们的约定 (即开始符号是第一条列出规则的头部), 开始符号是 $\langle \text{sentence} \rangle$ 。
- (B) 终结符是 $\mathbf{the}$ 、 $\mathbf{young}$ 、 $\mathbf{caught}$ 、 $\mathbf{man}$ 和 $\mathbf{ball}$ 。
- (C) 非终结符是 $\langle \text{sentence} \rangle$ 、 $\langle \text{noun phrase} \rangle$ 、 $\langle \text{verb phrase} \rangle$ 、 $\langle \text{article} \rangle$ 、 $\langle \text{noun} \rangle$ 、 $\langle \text{adjective} \rangle$ 和 $\langle \text{verb} \rangle$ 。
- (D) 三个是 $\mathbf{the\ ball\ caught\ the\ young\ man}$ 、 $\mathbf{the\ ball\ caught\ the\ man}$ 和 $\mathbf{the\ man\ caught\ the\ man}$ 。
- (E) 长度为 1 的串 $\mathbf{ball}$ 无法推导, 因为它是 $\langle \text{noun} \rangle$, 而 $\langle \text{noun} \rangle$ 前面必须有 $\langle \text{adjective} \rangle$ 。类似地, $\mathbf{man\ man\ man}$ 也无法推导。第三个是 ε 无法推导, 因为任何完成的推导最终都必须以非空的终结符结束。

III.2.13

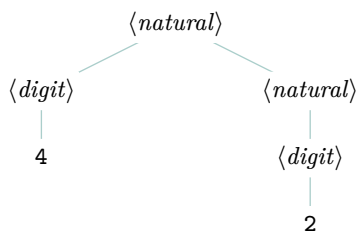


图 60, FOR QUESTION III.2.13: 42 的语法分析树。

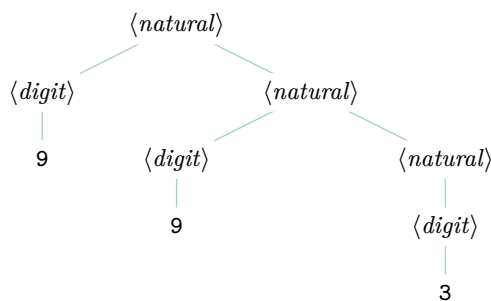


图 61, FOR QUESTION III.2.13: 与推导 993 相关联的语法分析树。

(A) 字母表是 $\Sigma = \{0, \dots, 9\}$ 。终结符是 $0, \dots, 9$ 。非终结符是 $\langle natural \rangle$ 和 $\langle digit \rangle$ 。开始符号是 $\langle natural \rangle$ 。

(B) 对于产生式

$$\langle natural \rangle \rightarrow \langle digit \rangle$$

头部是 $\langle natural \rangle$ ，体部是 $\langle digit \rangle$ 。对于产生式

$$\langle natural \rangle \rightarrow \langle digit \rangle \langle natural \rangle$$

头部是 $\langle natural \rangle$ ，体部是 $\langle digit \rangle \langle natural \rangle$ 。对于产生式

$$\langle digit \rangle \rightarrow 0$$

头部是 $\langle digit \rangle$ ，体部是 0 。其余九条产生式与此类似。

(C) 两个元字符是 $\rightarrow$ 和 $|$ 。

(D) 下面是一个推导。

$$\begin{aligned} \langle natural \rangle &\Rightarrow \langle digit \rangle \langle natural \rangle \\ &\Rightarrow 4 \langle natural \rangle \\ &\Rightarrow 4 \langle digit \rangle \\ &\Rightarrow 42 \end{aligned}$$

相应的语法分析树见 Figure 60 on page 103。

(E) 下面是一个推导。

$$\begin{aligned} \langle natural \rangle &\Rightarrow \langle digit \rangle \langle natural \rangle \\ &\Rightarrow \langle digit \rangle \langle digit \rangle \langle natural \rangle \\ &\Rightarrow \langle digit \rangle \langle digit \rangle \langle digit \rangle \\ &\Rightarrow \langle digit \rangle 9 \langle digit \rangle \\ &\Rightarrow \langle digit \rangle 93 \\ &\Rightarrow 993 \end{aligned}$$

语法分析树见 Figure 61 on page 103。

(F) 我们拿到一个串，然后试着从开始符号出发，找到一种方式最终到达该串。无休止地扩展并不是我们的目的。

(G) 这里给出一种答案。

$$\begin{aligned}\langle integer \rangle &\rightarrow +\langle natural \rangle \mid -\langle natural \rangle \mid \langle natural \rangle \\ \langle natural \rangle &\rightarrow \langle digit \rangle \mid \langle digit \rangle \langle natural \rangle \\ \langle digit \rangle &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9\end{aligned}$$

(H) 可以，前一项的文法允许我们推导出 +0 和 -0。这是 +0 的一个推导。

$$\begin{aligned}\langle integer \rangle &\Rightarrow +\langle natural \rangle \\ &\Rightarrow +\langle digit \rangle \\ &\Rightarrow +0\end{aligned}$$

顺便说一下，这里有一个文法，它不允许在其语言中出现像 +0 这样的串。

$$\begin{aligned}\langle integer \rangle &\rightarrow 0 \mid \langle nonzero\ natural \rangle \mid \langle sign \rangle \langle nonzero\ natural \rangle \\ \langle nonzero\ natural \rangle &\rightarrow \langle nonzero\ digit \rangle \langle digits \rangle \\ \langle digits \rangle &\rightarrow \langle digits \rangle \langle digit \rangle \mid \langle digit \rangle \\ \langle sign \rangle &\rightarrow + \mid - \\ \langle digit \rangle &\rightarrow 0 \mid \langle nonzero\ digit \rangle \\ \langle nonzero\ digit \rangle &\rightarrow 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9\end{aligned}$$

III.2.14

(A) 下面是对 ‘the car hit a wall’ 的一个推导。

$$\begin{aligned}\langle 句子 \rangle &\Rightarrow \langle 主语 \rangle \langle 谓语 \rangle \Rightarrow \langle 冠词 \rangle \langle 名词 \rangle \langle 谓语 \rangle \\ &\Rightarrow \langle 冠词 \rangle \langle 名词 \rangle \langle 动词 \rangle \langle 直接宾语 \rangle \Rightarrow \langle 冠词 \rangle \langle 名词 \rangle \langle 动词 \rangle \langle 冠词 \rangle \langle 名词 \rangle \\ &\Rightarrow the \langle 名词 \rangle \langle 动词 \rangle \langle 冠词 \rangle \langle 名词 \rangle \Rightarrow the\ car \langle 动词 \rangle \langle 冠词 \rangle \langle 名词 \rangle \\ &\Rightarrow the\ car\ hit \langle 冠词 \rangle \langle 名词 \rangle \Rightarrow the\ car\ hit\ a \langle 名词 \rangle \\ &\Rightarrow the\ car\ hit\ a\ wall\end{aligned}$$

(B) 下面是对 ‘the car hit the wall’ 的一个推导。

$$\begin{aligned}\langle 句子 \rangle &\Rightarrow \langle 主语 \rangle \langle 谓语 \rangle \Rightarrow \langle 冠词 \rangle \langle 名词 \rangle \langle 谓语 \rangle \\ &\Rightarrow the \langle 名词 \rangle \langle 谓语 \rangle \Rightarrow the\ car \langle 谓语 \rangle \\ &\Rightarrow the\ car \langle 动词 \rangle \langle 直接宾语 \rangle \Rightarrow the\ car\ hit \langle 直接宾语 \rangle \\ &\Rightarrow the\ car\ hit \langle 冠词 \rangle \langle 名词 \rangle \Rightarrow the\ car\ hit\ the \langle 名词 \rangle \\ &\Rightarrow the\ car\ hit\ the\ wall\end{aligned}$$

(C) 下面是对 ‘the wall hit a car’ 的一个推导。

$$\begin{aligned}\langle 句子 \rangle &\Rightarrow \langle 主语 \rangle \langle 谓语 \rangle \Rightarrow \langle 冠词 \rangle \langle 名词 \rangle \langle 谓语 \rangle \\ &\Rightarrow \langle 冠词 \rangle \langle 名词 \rangle \langle 动词 \rangle \langle 直接宾语 \rangle \Rightarrow \langle 冠词 \rangle \langle 名词 \rangle \langle 动词 \rangle \langle 冠词 \rangle \langle 名词 \rangle \\ &\Rightarrow \langle 冠词 \rangle \langle 名词 \rangle hit \langle 冠词 \rangle \langle 名词 \rangle \Rightarrow \langle 冠词 \rangle wall\ hit \langle 冠词 \rangle \langle 名词 \rangle \\ &\Rightarrow \langle 冠词 \rangle wall\ hit \langle 冠词 \rangle car \Rightarrow \langle 冠词 \rangle wall\ hit\ a\ car \\ &\Rightarrow the\ wall\ hit\ a\ car\end{aligned}$$

III.2.15

(A) 这是 ‘dog bites man’ 的一个推导。

$$\begin{aligned}\langle sentence \rangle &\Rightarrow \langle subject \rangle \langle predicate \rangle \Rightarrow \langle article \rangle \langle noun1 \rangle \langle predicate \rangle \\ &\Rightarrow \langle article \rangle \langle noun1 \rangle \langle verb \rangle \langle direct\ object \rangle \Rightarrow \langle article \rangle \langle noun1 \rangle \langle verb \rangle \langle article \rangle \langle noun2 \rangle \\ &\Rightarrow \varepsilon \langle noun1 \rangle \langle verb \rangle \langle article \rangle \langle noun2 \rangle \Rightarrow \varepsilon\ dog \langle verb \rangle \langle article \rangle \langle noun2 \rangle \\ &\Rightarrow \varepsilon\ dog\ bites \langle article \rangle \langle noun2 \rangle \Rightarrow \varepsilon\ dog\ bites\ \varepsilon \langle noun2 \rangle \\ &\Rightarrow \varepsilon\ dog\ bites\ \varepsilon\ man = dog\ bites\ man\end{aligned}$$

(B) 开始符号 $\langle sentence \rangle$ 展开为 $\langle subject \rangle \langle predicate \rangle$ 。对于前者， $\langle subject \rangle$ 展开为 $\langle article \rangle \langle noun1 \rangle$ ，而 $\langle noun1 \rangle$ 必须展开为终结符 **dog** 或 **flea** 之一。然而，**man** 只能从 $\langle noun2 \rangle$ 推导出来， $\langle noun2 \rangle$ 只能从 $\langle direct-object \rangle$ 推导出来，而 $\langle direct-object \rangle$ 又只能从 $\langle predicate \rangle$ 推导出来。因此，**man** 不可能是句子的第一个词，因为它必须位于 **dog** 或 **flea** 之后。

III.2.16 不存在非终结符 $\langle \text{dog/flea} \rangle$ 或 $\langle \text{man/dog} \rangle$ 。你的朋友把非终结符和终结符混淆了。

III.2.17 最外层的运算是 $*$ ，所以我们先处理它。接下来得到括号和 $+$ 。推导过程如下。

$$\begin{aligned}
 \langle \text{expr} \rangle &\Rightarrow \langle \text{term} \rangle \\
 &\Rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \\
 &\Rightarrow \langle \text{factor} \rangle * \langle \text{factor} \rangle \\
 &\Rightarrow (\langle \text{expr} \rangle) * \langle \text{factor} \rangle \\
 &\Rightarrow (\langle \text{term} \rangle + \langle \text{expr} \rangle) * \langle \text{factor} \rangle \\
 &\Rightarrow (\langle \text{term} \rangle + \langle \text{term} \rangle) * \langle \text{factor} \rangle \\
 &\Rightarrow (\langle \text{factor} \rangle + \langle \text{term} \rangle) * \langle \text{factor} \rangle \\
 &\Rightarrow (a + \langle \text{term} \rangle) * \langle \text{factor} \rangle \\
 &\Rightarrow (a + \langle \text{factor} \rangle) * \langle \text{factor} \rangle \\
 &\Rightarrow (a + b) * \langle \text{factor} \rangle \\
 &\Rightarrow (a + b) * c
 \end{aligned}$$

III.2.18

(A) 首先是最左推导。

$$\begin{aligned}
 S &\Rightarrow T b U \Rightarrow a T b U \Rightarrow a a T b U \Rightarrow a a \varepsilon b U = a a b U \\
 &\Rightarrow a a b a U \Rightarrow a a b a b U \Rightarrow a a b a b \varepsilon = a a b a b
 \end{aligned}$$

接下来是最右推导。

$$\begin{aligned}
 S &\Rightarrow T b U \Rightarrow T b a U \Rightarrow T b a b U \Rightarrow T b a b \varepsilon = T b a b \\
 &\Rightarrow a T b a b \Rightarrow a a T b a b \Rightarrow a a \varepsilon b a b = a a b a b
 \end{aligned}$$

(B) 这是最左推导。

$$\begin{aligned}
 S &\Rightarrow T b U \Rightarrow \varepsilon b U = b U \Rightarrow b a U \\
 &\Rightarrow b a a U \Rightarrow b a a b U \Rightarrow b a a b \varepsilon = b a a b
 \end{aligned}$$

这是最右推导。

$$\begin{aligned}
 S &\Rightarrow T b U \Rightarrow T b a U \Rightarrow T b a a U \Rightarrow T b a a b U \\
 &\Rightarrow T b a a b \varepsilon = T b a a b \Rightarrow \varepsilon b a a b = b a a b
 \end{aligned}$$

(C) 开始符号展开后得到的符号串包含终结符 b 。

III.2.19

(A) 这是 $aabb$ 的一个推导，

$$S \Rightarrow aABb \Rightarrow aaBb \Rightarrow aabb$$

这是 $aaabb$ 的一个推导，

$$S \Rightarrow aABb \Rightarrow aaABb \Rightarrow aaaBb \Rightarrow aaabb$$

这是 $aabbb$ 的一个推导。

$$S \Rightarrow aABb \Rightarrow aABb \Rightarrow aaBb \Rightarrow aaBbb \Rightarrow aabbb$$

(B) 第一行产生式表明，每个推导出的串都必须以 a 开头并以 b 结尾。因此三个无法推导出的串是 a 、 ba 和空串 ε 。

(C) $\mathcal{L} = \{a^n b^m \mid n, m \geq 2\}$

III.2.20

下面给出一个。

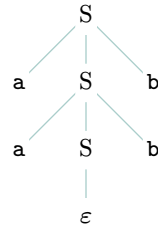
$$S \rightarrow TU$$

$$T \rightarrow aTb \mid \varepsilon$$

$$U \rightarrow bUa \mid \varepsilon$$

下面是 $a^2 b^{(2+1)} a^1 = aabbbba$ 的一个推导。

$$S \Rightarrow TU \Rightarrow aTbU \Rightarrow aaTbbU \Rightarrow aa\varepsilon bbU = aabbU \Rightarrow aabbbUa \Rightarrow aabbbb\varepsilon a = aabbbba$$

图 62, FOR QUESTION III.2.21: a^2b^2 的语法分析树。

III.2.21 Figure 62 on page 106 展示了这棵树。

III.2.22 我们将用归纳法证明, 在整个推导过程中, 产生的所有结果要么是 $a^k S b^k$ 这种形式, 要么是 $a^k b^k$ 这种形式。根据文法生成语言的定义, 只有不含任何非终结符的串才属于该语言, 由此即可完成验证。

归纳法基于推导步骤的数量, 也就是规则应用的次数。归纳基础是规则应用次数为零的情况。此时得到的串是 'S', 这正是所需形式在 $k = 0$ 时的情况。

对于归纳步骤, 假设结论在规则应用次数为 $n = 0, n = 1, \dots, n = k$ 时成立, 现在考虑 $n = k + 1$ 的情况。在第 k 步之后, 对得到的串应用一条规则, 即可得到第 $k + 1$ 步的串。归纳假设是, 这样一个串具有两种形式之一: $a^k S b^k$ 或 $a^k b^k$ 。规则无法应用于后一种形式的串, 因此只考虑前一种。

对 $a^k S b^k$ 应用第一条规则, 可得 $a^{k+1} S b^{k+1}$, 它具有正确的形式。第二条规则则得到 $a^{k+1} b^{k+1}$, 同样具有正确的形式。

III.2.23 该文法不存在能够终止的推导, 因此它生成空语言, $\mathcal{L} = \emptyset$ 。

III.2.24 下面给出一个。

$$\begin{aligned}
 \langle \text{identifier} \rangle &\rightarrow \langle \text{letter} \rangle \mid \langle \text{letter-or-digit-string} \rangle \\
 \langle \text{letter-or-digit-string} \rangle &\rightarrow \langle \text{letter} \rangle \langle \text{letter-or-digit-string} \rangle \\
 &\quad \mid \langle \text{digit} \rangle \langle \text{letter-or-digit-string} \rangle \\
 &\quad \mid \varepsilon \\
 \langle \text{letter} \rangle &\rightarrow a \mid \dots \mid z \mid A \mid \dots \mid Z \\
 \langle \text{digit} \rangle &\rightarrow 0 \mid \dots \mid 9
 \end{aligned}$$

III.2.25

$$\begin{aligned}
 \langle \text{identifier} \rangle &\rightarrow \langle \text{letter} \rangle \langle \text{second} \rangle \\
 \langle \text{second} \rangle &\rightarrow \langle \text{letter-or-digit} \rangle \langle \text{third} \rangle \mid \varepsilon \\
 \langle \text{third} \rangle &\rightarrow \langle \text{letter-or-digit} \rangle \langle \text{fourth} \rangle \mid \varepsilon \\
 \langle \text{fourth} \rangle &\rightarrow \langle \text{letter-or-digit} \rangle \\
 \langle \text{letter-or-digit} \rangle &\rightarrow \langle \text{letter} \rangle \mid \langle \text{digit} \rangle \\
 \langle \text{letter} \rangle &\rightarrow A \mid \dots \mid Z \\
 \langle \text{digit} \rangle &\rightarrow 0 \mid \dots \mid 9
 \end{aligned}$$

III.2.26 它是空语言, $\mathcal{L} = \emptyset$ 。

III.2.27

(A) Figure 63 on page 107 展示了一种推导。

(B) 根据该文法的第一条规则, 得到包含多个子句的表达式的唯一方法是用 $\wedge$ 连接这些子句。

III.2.28

$$\begin{aligned}
 \text{(A)} \quad \langle \text{string} \rangle &\rightarrow \langle \text{letter} \rangle \langle \text{string} \rangle \mid \varepsilon \\
 \langle \text{letter} \rangle &\rightarrow a \mid \dots \mid z \\
 \text{(B)} \quad \langle \text{string1} \rangle &\rightarrow \langle \text{digit} \rangle \langle \text{string2} \rangle \\
 \langle \text{string2} \rangle &\rightarrow \langle \text{digit} \rangle \langle \text{string2} \rangle \mid \varepsilon \\
 \langle \text{digit} \rangle &\rightarrow 0 \mid \dots \mid 9
 \end{aligned}$$

III.2.29

$$\begin{aligned}
\langle \text{CNF} \rangle &\Rightarrow (\langle \text{Disjunction} \rangle) \wedge \langle \text{CNF} \rangle \\
&\Rightarrow (\langle \text{Disjunction} \rangle) \wedge (\langle \text{Disjunction} \rangle) \\
&\Rightarrow (\langle \text{Literal} \rangle \vee \langle \text{Disjunction} \rangle) \wedge (\langle \text{Disjunction} \rangle) \\
&\Rightarrow (\langle \text{Literal} \rangle \vee \langle \text{Literal} \rangle) \wedge (\langle \text{Disjunction} \rangle) \\
&\Rightarrow (\langle \text{Variable} \rangle \vee \langle \text{Literal} \rangle) \wedge (\langle \text{Disjunction} \rangle) \\
&\Rightarrow (x_0 \vee \langle \text{Literal} \rangle) \wedge (\langle \text{Disjunction} \rangle) \\
&\Rightarrow (x_0 \vee \neg \langle \text{Variable} \rangle) \wedge (\langle \text{Disjunction} \rangle) \\
&\Rightarrow (x_0 \vee \neg x_1) \wedge (\langle \text{Disjunction} \rangle) \\
&\Rightarrow (x_0 \vee \neg x_1) \wedge (\langle \text{Literal} \rangle \vee \langle \text{Disjunction} \rangle) \\
&\Rightarrow (x_0 \vee \neg x_1) \wedge (\langle \text{Literal} \rangle \vee \langle \text{Literal} \rangle) \\
&\Rightarrow (x_0 \vee \neg x_1) \wedge (\langle \text{Variable} \rangle \vee \langle \text{Literal} \rangle) \\
&\Rightarrow (x_0 \vee \neg x_1) \wedge (x_1 \vee \langle \text{Literal} \rangle) \\
&\Rightarrow (x_0 \vee \neg x_1) \wedge (x_1 \vee \langle \text{Variable} \rangle) \\
&\Rightarrow (x_0 \vee \neg x_1) \wedge (x_1 \vee x_2)
\end{aligned}$$

图 63, FOR QUESTION III.2.27: 推导 $(x_0 \vee \neg x_1) \wedge (x_1 \vee x_2)$ 。

(A) 根据这个推导的开头，我们很容易就能推导出该地址。

$$\begin{aligned}
\langle \text{邮政地址} \rangle &\Rightarrow \langle \text{姓名} \rangle \text{换行} \langle \text{街道地址} \rangle \text{换行} \langle \text{城镇} \rangle \\
&\Rightarrow \langle \text{名字部分} \rangle \langle \text{姓氏} \rangle \langle \text{可选后缀} \rangle \text{换行} \langle \text{街道地址} \rangle \text{换行} \langle \text{城镇} \rangle \\
&\Rightarrow \langle \text{名字} \rangle \langle \text{姓氏} \rangle \langle \text{可选后缀} \rangle \text{换行} \langle \text{街道地址} \rangle \text{换行} \langle \text{城镇} \rangle \\
&\Rightarrow \langle \text{字符串} \rangle \langle \text{姓氏} \rangle \langle \text{可选后缀} \rangle \text{换行} \langle \text{街道地址} \rangle \text{换行} \langle \text{城镇} \rangle \\
&\Rightarrow \langle \text{字符串} \rangle \langle \text{字符串} \rangle \langle \text{可选后缀} \rangle \text{换行} \langle \text{街道地址} \rangle \text{换行} \langle \text{城镇} \rangle \\
&\Rightarrow \langle \text{字符串} \rangle \langle \text{字符串} \rangle \varepsilon \text{换行} \langle \text{街道地址} \rangle \text{换行} \langle \text{城镇} \rangle \\
&= \langle \text{字符串} \rangle \langle \text{字符串} \rangle \text{换行} \langle \text{门牌号} \rangle \langle \text{街道名} \rangle \langle \text{公寓号} \rangle \text{换行} \langle \text{城镇} \rangle \\
&\Rightarrow \langle \text{字符串} \rangle \langle \text{字符串} \rangle \text{换行} \langle \text{数字串} \rangle \langle \text{街道名} \rangle \langle \text{公寓号} \rangle \text{换行} \langle \text{城镇} \rangle \\
&\Rightarrow \varepsilon \langle \text{字符串} \rangle \text{换行} \langle \text{数字串} \rangle \langle \text{街道名} \rangle \langle \text{公寓号} \rangle \text{换行} \langle \text{城镇} \rangle \\
&= \langle \text{字符串} \rangle \text{换行} \langle \text{数字串} \rangle \langle \text{街道名} \rangle \langle \text{公寓号} \rangle \text{换行} \langle \text{城镇} \rangle \\
&\Rightarrow \langle \text{字符串} \rangle \text{换行} \langle \text{数字串} \rangle \langle \text{字符串} \rangle \langle \text{公寓号} \rangle \text{换行} \langle \text{城镇} \rangle \\
&\Rightarrow \langle \text{字符串} \rangle \text{换行} \langle \text{数字串} \rangle \langle \text{字符串} \rangle \varepsilon \text{换行} \langle \text{城镇} \rangle \\
&= \langle \text{字符串} \rangle \text{换行} \langle \text{数字串} \rangle \langle \text{字符串} \rangle \text{换行} \langle \text{城镇} \rangle \\
&\Rightarrow \langle \text{字符串} \rangle \text{换行} \langle \text{数字串} \rangle \langle \text{字符串} \rangle \text{换行} \langle \text{城镇名} \rangle, \langle \text{州或地区} \rangle \\
&\Rightarrow \langle \text{字符串} \rangle \text{换行} \langle \text{数字串} \rangle \langle \text{字符串} \rangle \text{换行} \langle \text{字符串} \rangle, \langle \text{州或地区} \rangle \\
&\Rightarrow \langle \text{字符串} \rangle \text{换行} \langle \text{数字串} \rangle \langle \text{字符串} \rangle \text{换行} \langle \text{字符串} \rangle, \langle \text{字符串} \rangle
\end{aligned}$$

(B) 问题在于 ‘221B’ 中的 ‘B’；它不属于数字串。

(C) 一些可能的修改是：允许地址超过一行、允许没有门牌号的地址，以及允许姓名或地址中包含非 ASCII 字符。

注：在实践中，最好的方法可能就是直接把地址当作一整块字符来处理。例如，参见 <https://github.com/kdeldycke/awesome-falsehood?tab=readme-ov-file#postal-addresses>。

III.2.30

(A) 这两条规则就够了： $S \rightarrow 11S \mid \varepsilon$ 。

(B) 与上一题类似，这是直截了当的： $S \rightarrow 111S \mid \varepsilon$ 。

III.2.31

(A) 下面是一个长度为一的句子的推导。

$$\langle \text{sentence} \rangle \Rightarrow \text{buffalo} \langle \text{sentence} \rangle \Rightarrow \text{buffalo} \varepsilon = \text{buffalo}$$

下面是一个长度为二的句子的推导。

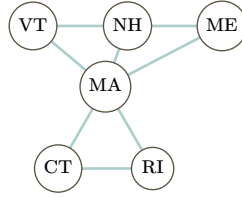


图 64, FOR QUESTION III.3.16: 新英格兰各州之间的相邻关系。

$$\begin{aligned}\langle \text{sentence} \rangle &\Rightarrow \text{buffalo} \langle \text{sentence} \rangle \Rightarrow \text{buffalo buffalo} \langle \text{sentence} \rangle \\ &\Rightarrow \text{buffalo buffalo } \varepsilon = \text{buffalo buffalo}\end{aligned}$$

接着，是一个长度为三的句子推导。

$$\begin{aligned}\langle \text{sentence} \rangle &\Rightarrow \text{buffalo} \langle \text{sentence} \rangle \Rightarrow \text{buffalo buffalo} \langle \text{sentence} \rangle \\ &\Rightarrow \text{buffalo buffalo buffalo} \langle \text{sentence} \rangle \Rightarrow \text{buffalo buffalo buffalo } \varepsilon \\ &= \text{buffalo buffalo buffalo}\end{aligned}$$

- (B) 在英语中，作为名词，“buffalo”指的是美洲野牛，或是纽约州的一座城市。作为动词，它的意思是愚弄或迷惑某人。因此，长度为一的句子可能是一个命令，指示听者去迷惑某人。长度为二的句子可能是命令同一个听者去迷惑纽约州北部那座城市的全体居民。长度为三的句子可能是在特意敦促美洲野牛去迷惑整座城市。（加上标点可能会更清楚：“Buffalo, buffalo Buffalo!”）

III.2.32

- (A) 这是 (a . b) 的一个推导。

$$\begin{aligned}\langle S\text{-表达式} \rangle &\Rightarrow (\langle S\text{-表达式} \rangle . \langle S\text{-表达式} \rangle) \Rightarrow (\langle \text{原子符号} \rangle . \langle S\text{-表达式} \rangle) \\ &\Rightarrow (\langle \text{原子符号} \rangle . \langle \text{原子符号} \rangle) \Rightarrow (\langle \text{字母} \rangle \langle \text{原子部分} \rangle . \langle \text{原子符号} \rangle) \\ &\Rightarrow (\langle \text{字母} \rangle \varepsilon . \langle \text{原子符号} \rangle) = (\langle \text{字母} \rangle . \langle \text{原子符号} \rangle) \\ &\Rightarrow (\text{a} . \langle \text{原子符号} \rangle) \Rightarrow (\text{a} . \langle \text{字母} \rangle \langle \text{原子部分} \rangle) \\ &\Rightarrow (\text{a} . \langle \text{字母} \rangle \varepsilon) = (\text{a} . \langle \text{字母} \rangle) \Rightarrow (\text{a} . \text{b})\end{aligned}$$

- (B) 这是 (a . (b . c)) 的一个推导。

$$\begin{aligned}\langle S\text{-表达式} \rangle &\Rightarrow (\langle S\text{-表达式} \rangle . \langle S\text{-表达式} \rangle) \\ &\Rightarrow (\langle S\text{-表达式} \rangle . (\langle S\text{-表达式} \rangle . \langle S\text{-表达式} \rangle)) \\ &\Rightarrow (\langle S\text{-表达式} \rangle . (\langle S\text{-表达式} \rangle . \langle S\text{-表达式} \rangle)) \\ &\Rightarrow (\langle \text{原子符号} \rangle . (\langle S\text{-表达式} \rangle . \langle S\text{-表达式} \rangle)) \\ &\Rightarrow (\langle \text{字母} \rangle \langle \text{原子部分} \rangle . (\langle S\text{-表达式} \rangle . \langle S\text{-表达式} \rangle)) \\ &\Rightarrow (\text{a} \langle \text{原子部分} \rangle . (\langle S\text{-表达式} \rangle . \langle S\text{-表达式} \rangle)) \\ &\Rightarrow (\text{a} \varepsilon . (\langle S\text{-表达式} \rangle . \langle S\text{-表达式} \rangle)) = (\text{a} . (\langle S\text{-表达式} \rangle . \langle S\text{-表达式} \rangle)) \\ &\Rightarrow (\text{a} . (\langle \text{原子符号} \rangle . \langle S\text{-表达式} \rangle)) \Rightarrow (\text{a} . (\langle \text{原子符号} \rangle . \langle \text{原子符号} \rangle)) \\ &\Rightarrow (\text{a} . (\langle \text{字母} \rangle \langle \text{原子部分} \rangle . \langle \text{原子符号} \rangle)) \\ &\Rightarrow (\text{a} . (\langle \text{字母} \rangle \langle \text{原子部分} \rangle . \langle \text{字母} \rangle \langle \text{原子部分} \rangle)) \\ &\Rightarrow (\text{a} . (\langle \text{字母} \rangle \varepsilon . \langle \text{字母} \rangle \langle \text{原子部分} \rangle)) = (\text{a} . (\langle \text{字母} \rangle . \langle \text{字母} \rangle \langle \text{原子部分} \rangle)) \\ &\Rightarrow (\text{a} . (\langle \text{字母} \rangle . \langle \text{字母} \rangle \varepsilon)) = (\text{a} . (\langle \text{字母} \rangle . \langle \text{字母} \rangle)) \\ &\Rightarrow (\text{a} . (\text{b} . \langle \text{字母} \rangle)) \Rightarrow (\text{a} . (\text{b} . \text{c}))\end{aligned}$$

III.3.16

- (A) Figure 64 on page 108 展示了该图。其中缩写为：VT 是佛蒙特州，NH 是新罕布什尔州，ME 是缅因州，MA 是麻萨诸塞州，CT 是康涅狄格州，RI 是罗德岛。
- (B) 该图是一个环，见 Figure 65 on page 109。
- (C) 该图为 Figure 66 on page 109。
- (D) 该图为 Figure 67 on page 110。

III.3.17

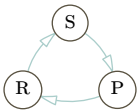


图 65, FOR QUESTION III.3.16: 石头、布、剪刀之间的‘击败’关系。

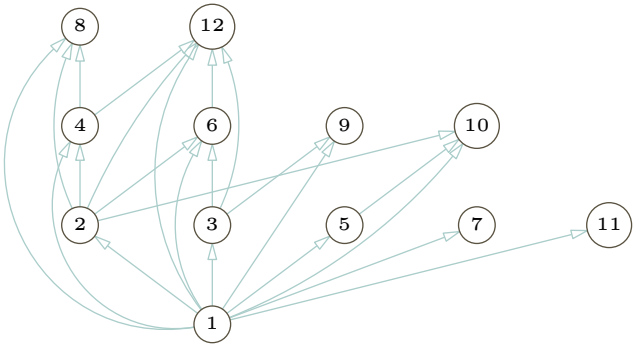


图 66, FOR QUESTION III.3.16: 较小数字之间的整除关系。

	顶点可 重复？	边可 重复？	能否为 闭的？	能否为 开的？
游走	Y	Y	Y	Y
迹	Y	N	Y	Y
回路	Y	N	Y	N
路	N	N	Y	Y
圈	N	N	Y	N

注意，一条路的第一个和最后一个顶点可以相同，但其他顶点不能相同。一个圈则必须满足其首尾顶点相同，且其他顶点不重复。

III.3.18 例如，根据给定信息可知，修 Calculus I 先于修 Calculus III。但由于它被其他边所隐含，出于视觉设计的考虑，我们将其从先修结构图中省略，如 Figure 68 on page 110 所示。

III.3.19

- (A) 图见 Figure 69 on page 111。
- (B) 矩阵如下。

$$\mathcal{M}(\mathcal{G}) = \begin{matrix} & v_0 & v_1 & v_2 & v_3 & v_4 & v_5 \\ \begin{matrix} v_0 \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{matrix} & \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 \end{pmatrix} \end{matrix}$$

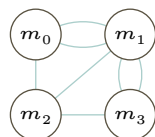


图 67, FOR QUESTION III.3.16: 各地块之间的桥梁连接关系。

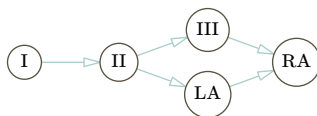


图 68, FOR QUESTION III.3.18: 课程间的先修结构, 使用最少数量的边表示。

(c) 共有六个顶点, 从中选取四个有十五种选法。下表列出了所有可用边。

线	顶点	最大边集	线	顶点	最大边集
0	v_0, v_1, v_2, v_3	v_0v_1, v_0v_3	9	v_0, v_1, v_4, v_5	$v_0v_1, v_0v_5, v_1v_4, v_4v_5$
1	v_0, v_1, v_2, v_4	v_0v_1, v_0v_3, v_1v_4	10	v_0, v_2, v_4, v_5	v_0v_5, v_4v_5
2	v_0, v_1, v_3, v_4	$v_0v_1, v_0v_3, v_1v_4, v_3v_4$	11	v_1, v_2, v_4, v_5	v_1v_4, v_4v_5
3	v_0, v_2, v_3, v_4	v_0v_3, v_3v_4	12	v_0, v_3, v_4, v_5	$v_0v_3, v_0v_5, v_3v_4, v_4v_5$
4	v_1, v_2, v_3, v_4	v_1v_4, v_3v_4	13	v_1, v_3, v_4, v_5	v_1v_4, v_3v_4, v_4v_5
5	v_0, v_1, v_2, v_5	v_0v_1, v_0v_5	14	v_2, v_3, v_4, v_5	v_3v_4, v_4v_5
6	v_0, v_1, v_3, v_5	v_0v_1, v_0v_3, v_0v_5			
7	v_0, v_2, v_3, v_5	v_0v_3, v_0v_5			
8	v_1, v_2, v_3, v_5				

(利用 Figure 69 on page 111 来核对最后一列的各项会很方便。) 满足条件的图共有三个: $G_2 = \{v_0, v_1, v_3, v_4\}$ 、 $G_9 = \{v_0, v_1, v_4, v_5\}$ 和 $G_{12} = \{v_0, v_3, v_4, v_5\}$ 。

(D) 与上一小题相同, 也是这三个。

III.3.20

(A) 见图 Figure 70 on page 111。

(B) 见图 Figure 71 on page 111。

(C) 边数为 $\binom{n}{2} = n(n+1)/2$ 。

III.3.21 见图 Figure 72 on page 111。

III.3.22

(A) 共有十个顶点: $\mathcal{N} = \{v_0, \dots, v_9\}$ 。有十五条边:

$$\mathcal{E} = \{v_0v_1, v_0v_4, v_0v_5, v_1v_2, v_1v_6, v_2v_3, v_2v_7, v_3v_4, v_3v_8, v_4v_9, v_5v_7, v_5v_8, v_6v_8, v_6v_9, v_7v_9\}$$

(B) 游走 $\langle v_0v_5, v_5v_7 \rangle$ 的长度为二。游走 $\langle v_0v_1, v_1v_2, v_2v_7 \rangle$ 的长度为三。

(C) 一条闭游走是 $\langle v_4v_0, v_0v_1, v_1v_2, v_2v_3, v_3v_4 \rangle$ 。一条开游走是 $\langle v_4v_9, v_9v_6, v_6v_8, v_8v_3, v_3v_2 \rangle$ 。

(D) 一个圈是 $\langle v_5v_0, v_0v_1, v_1v_6, v_6v_8, v_8v_5 \rangle$ 。

(E) 是的, 这个图是连通的, 因为我们可以找到任意两个顶点之间的一条路。(我们通过观察即可看出这一点, 尽管如果有疑问, 我们也可以列出所有顶点对并给出它们之间的路。)

III.3.23 如果只有两种颜色, 圈中的顶点必须交替着色。但是, 如果我们绕外围的五个节点走一圈, 那么起始节点和第五个节点将着上相同的颜色。

III.3.24

(A) 见图 Figure 73 on page 111。

(B) 色数为三; 见同一幅图中的右侧图。色数不可能小于三, 例如因为 ABC 构成了一个三角形。

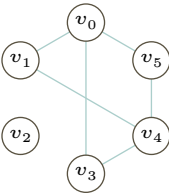


图 69, FOR QUESTION III.3.19: 一个包含若干条边的图。

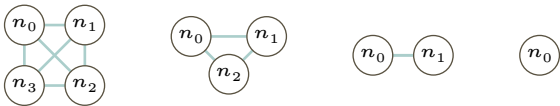


图 70, FOR QUESTION III.3.20: 完全图 K_4 、 K_3 、 K_2 和 K_1 。

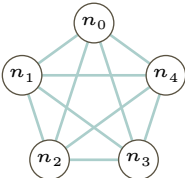


图 71, FOR QUESTION III.3.20: 完全图 K_5 。

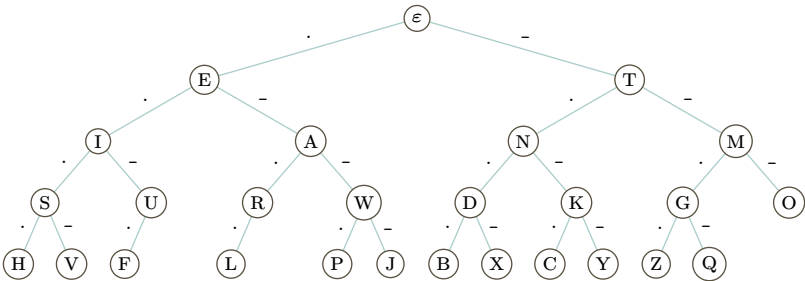


图 72, FOR QUESTION III.3.21: ASCII 字母 Morse 码表示的前缀关系图。

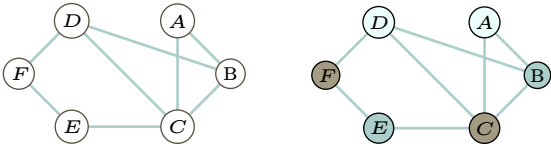
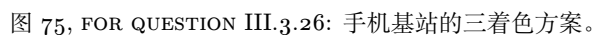
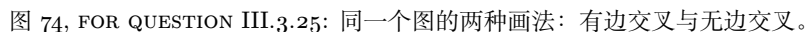


图 73, FOR QUESTION III.3.24: 鱼类物种间的不可共处关系。


$$\begin{array}{c} \text{O-} \\ \text{O+} \\ \text{A-} \\ \text{A+} \\ \text{B-} \\ \text{B+} \\ \text{AB-} \\ \text{AB+} \end{array} \begin{pmatrix} \text{O-} & \text{O+} & \text{A-} & \text{A+} & \text{B-} & \text{B+} & \text{AB-} & \text{AB+} \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

III.3.30 这两个图都是完全图，这意味着除了没有自环外，所有可能的节点间连接都存在。它们的邻

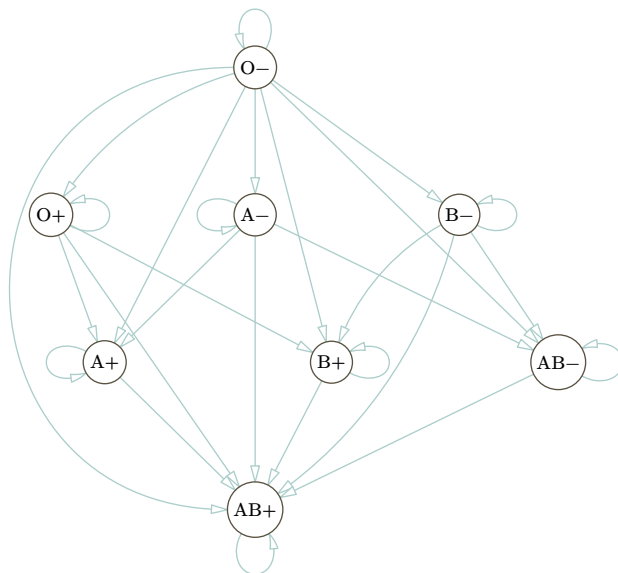


图 76, FOR QUESTION III.3.28: 血型相容性有向图。

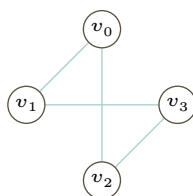


图 77, FOR QUESTION III.3.31: 与给定矩阵对应的图。

接矩阵如下。

$$\begin{array}{c}
 \begin{array}{c} u_0 \\ u_1 \\ u_2 \\ u_3 \end{array}
 \begin{pmatrix}
 u_0 & u_1 & u_2 & u_3 \\
 0 & 1 & 1 & 1 \\
 1 & 0 & 1 & 1 \\
 1 & 1 & 0 & 1 \\
 1 & 1 & 1 & 0
 \end{pmatrix}
 \end{array}
 \quad
 \begin{array}{c}
 \begin{array}{c} v_0 \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \\ v_7 \end{array}
 \begin{pmatrix}
 v_0 & v_1 & v_2 & v_3 & v_4 & v_5 & v_6 & v_7 \\
 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \\
 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 \\
 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 \\
 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 \\
 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \\
 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 \\
 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0
 \end{pmatrix}
 \end{array}$$

III.3.31 对应的简单图如 Figure 77 on page 113 所示。

III.3.32 如果树为空，则着色是平凡的。否则，固定一个顶点。给它着第一种颜色，给它的所有邻点着第二种颜色。给所有与初始顶点距离为 2 的顶点着第一种颜色。一般地，对于与初始顶点距离为 k 的顶点，若 k 为偶数则着第一种颜色，若 k 为奇数则着第二种颜色。树中没有圈，因此这种分配是良定义的。

III.3.33 正确答案不止一个，这里我们只给出一个。

(A) 该函数可由顶点名称暗示： $a \mapsto A$, $b \mapsto B$, 等等。通过观察可知，该函数是单射且满射的，因此它是一个双射。

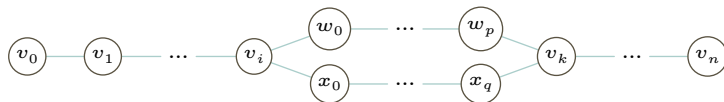


图 78, FOR QUESTION III.3.35: 两条不同的路会形成一个圈。

- (B) 左边图的边为: $ax, ay, az, bx, by, bz, cx, cy$ 与 cz 。右边图的边为: $AX, AY, AZ, BX, BY, BZ, CX, CY$ 与 CZ 。在上述映射下, 边 ax 对应 AX , 边 ay 对应 AY , 等等。再次通过观察可知, 这些边之间存在对应关系。

III.3.34 两个图的度序列都是 $\langle 3, 3, 3, 3, 3, 3 \rangle$ 。

III.3.35

- (A) 路是顶点互不相同的迹 (除非起点可能等于终点)。这条迹满足这一条件。
 (B) 顶点 B 出现了两次, 但并非同时出现在起点和终点。
 (C) Figure 78 on page 114 可以说明。根据定义, 每棵树都是连通的, 因此存在路。反设从 v_0 到 v_n 有两条不同的路。那么必然存在某个顶点 v_i 使两条路在此分离, 也存在某个顶点 v_k 使它们再次汇合。但这会形成一个圈: 从 v_i 出发, 沿着其中一条路走到 v_k , 再沿着另一条路返回 v_i 。这与树没有圈相矛盾。

III.3.36 有三个度为 2 的节点和四个度为 1 的节点。度序列是 $\langle 2, 2, 2, 1, 1, 1, 1 \rangle$ 。

III.3.37

- (A) 广度优先遍历是序列 $\langle A, B, C, D, E, F, G \rangle$ 。
 (B) 深度优先遍历是 $\langle A, B, D, E, C, F, G \rangle$ 。

III.3.38

- (A) 每条边我们选择两个顶点。因此可能的边的数量是 $\binom{n}{2} = n \cdot (n-1)/2$ 。
 (B) 对于每条边, 我们要么包含它, 要么不包含它。所以对于 n 个顶点, 共有 $2^{n \cdot (n-1)/2}$ 个图。(其中一些图可能同构, 但它们是不同的图, 因为它们连接的顶点不同。)
 (C)

n	0	1	2	3	4	5	6
$2^{n \cdot (n-1)/2}$	1	1	2	8	64	1024	32768

III.3.39

- (A) 根据图同构的定义 (定义 3.15), 若两个图 $\mathcal{G}$ 和 $\hat{\mathcal{G}}$ 同构, 则它们的顶点之间存在对应关系。因此它们的顶点数量相同。
 (B) 图同构的定义要求边也对应。所以同构的图边数量相同。
 (C) 假设 $\mathcal{G} = \langle \mathcal{N}, \mathcal{E} \rangle$ 和 $\hat{\mathcal{G}} = \langle \hat{\mathcal{N}}, \hat{\mathcal{E}} \rangle$ 通过函数 f 同构。设顶点 $v \in \mathcal{N}$ 的度为 k 。那么在集合 $\mathcal{E}$ 中, 顶点 v 出现 k 次 (对于该顶点只出现一次的边计一次, 对于 v 上的环计两次)。
 现在考虑顶点 $f(v)$ 和集合 $\hat{\mathcal{E}}$ 。由于 f 给出了边之间的对应关系, 该顶点在集合中也必然出现 k 次 (这里同样, 对于该顶点只出现一次的边计一次, 对于其上的环计两次)。因此 $f(v)$ 的度为 k 。
 (D) 这一项与上一项相同, 只是将单个顶点 v 替换为包含所有的顶点 $v_0, \dots, v_n$ 。
 (E) 我们可以应用前面的多项结论。最简单的一点是它们的顶点数量不同。另一点是度序列不同: 左边图的度序列是 $\langle 3, 3, 3, 3 \rangle$, 而右边图的度序列是 $\langle 4, 4, 4, 4, 4, 4, 4, 4 \rangle$ 。
 (F) 显然它们具有相同的度序列, $\langle 3, 1, 1, 1, 1, 1 \rangle$ 。假设两个图通过对应关系 f 同构。那么 f 必须将两个度为 3 的顶点关联起来。左边图中有两条不同的长度为 2 的路, 从度为 3 的顶点出发。右边图没有两条这样的路, 它只有一条。

III.3.40

- (A) $\mathcal{G}_0$ 的顶点是 $\mathcal{N}_0 = \{v_0, \dots, v_7\}$ 。边如下。

$$\mathcal{E}_0 = \{v_0v_1, v_0v_2, v_0v_4, v_1v_3, v_2v_3, v_2v_6, v_4v_5, v_4v_6, v_5v_7, v_6v_7\}$$

$\mathcal{G}_1$ 的顶点是 $\mathcal{N}_1 = \{n_0, \dots, n_7\}$ 。边如下。

$$\mathcal{E}_1 = \{n_0n_4, n_0n_5, n_1n_4, n_1n_5, n_2n_3, n_2n_6, n_3n_4, n_3n_7, n_5n_6, n_6n_7\}$$

(B) 度序列相等。每个都是 $\langle 3, 3, 3, 3, 2, 2, 2, 2 \rangle$ 。

(C) 下表是左图的边的像。

$\mathcal{G}_0$ 的边	v_0v_1	v_0v_2	v_0v_4	v_1v_3	v_2v_3	v_2v_6	v_4v_5	v_4v_6	v_5v_7	v_6v_7
像边	n_6n_2	n_6n_7	n_6n_5	n_2n_3	n_7n_3	n_7n_0	n_5n_1	n_5n_0	n_1n_4	n_0n_4

该列表中没有 n_3n_4 ，但 $\mathcal{G}_1$ 有这样一条边。此外，该列表列出了 n_7n_0 ，但 $\mathcal{G}_1$ 没有这样一条边。所以，给定的顶点对应关系不能导出边之间的对应关系。这个映射不是一个图同构。

(D) 图 $\mathcal{G}_0$ 包含一个由四个度为 3 的顶点构成的圈。但 $\mathcal{G}_1$ 没有这样的圈。

III.3.41

(A) 存在一条从 v_i 到 v_j 的长度为 2 的游走，当且仅当存在某个 v_k ，使得 v_iv_k 和 v_kv_j 都是边。因此，这样的游走的数量是 $m_{i,1}m_{1,j} + m_{i,2}m_{2,j} + \cdots m_{i,n}m_{n,j}$ ，其中每个 $m_{a,b}$ 是 $\mathcal{M}(\mathcal{G})$ 的一项。这个和就是 $(\mathcal{M}(\mathcal{G}))^2$ 的第 i, j 项。

(B) 基础步骤由 $\mathcal{M}(\mathcal{G})$ 的定义直接成立。

对于归纳步骤，假设命题对 $n = 2, \dots, n = p$ 成立，并考虑 $n = p + 1$ 的情况。存在一条从 v_i 到 v_j 的长度为 $p + 1$ 的游走，当且仅当存在某个 v_k ，使得存在一条从 v_i 到 v_k 的长度为 p 的游走，并且 v_kv_j 是一条边。根据归纳假设，这样的游走的数量是 $q_{i,1}m_{1,j} + q_{i,2}m_{2,j} + \cdots + q_{i,n}m_{n,j}$ ，其中每个 $q_{i,b}$ 是 $(\mathcal{M}(\mathcal{G}))^p$ 的一项，而 $m_{b,j}$ 是 $\mathcal{M}(\mathcal{G})$ 的一项。这个求和就是 $(\mathcal{M}(\mathcal{G}))^{p+1}$ 的第 i, j 项。

III.3.42

(A) 我们将维护一个可达顶点的集合 R 。首先，将 q_0 放入集合 R_0 。第一步，找出所有与 q_0 有边相连的顶点，并将它们放入集合 $R_1 \supseteq R_0$ 。在第 $k + 1$ 步，将所有与 R_k 中任一顶点有边相连的顶点放入一个集合 $R_{k+1} \supseteq R_k$ 。当没有新顶点出现时算法结束；因为图被假定有有限个顶点，所以这必定在有限步内发生。结果就是集合 R 。不可达的顶点就是那些不属于 R 的顶点。

(B) 对于左侧的图，步骤如下。

步骤 k	R_k
0	$\{w_0\}$
1	$\{w_0, w_1\}$
2	$\{w_0, w_1, w_2\} = R$

右侧图的步骤如下。

步骤 k	R_k
0	$\{w_0\}$
1	$\{w_0, w_1\}$
2	$\{w_0, w_1, w_3\} = R$

III.3.43 固定一个顶点 v_0 。图是连通的，因此对于其他每个顶点，都存在一条从 v_0 出发到达该顶点的路。

对于 v_0 的每个邻点，都存在一个顶点集合，其中的顶点可以通过一条经过该邻点的路从 v_0 到达。顶点有无穷多个，因此必存在一个（不等于 v_0 的）邻点，使得以这种方式可达的顶点集合是无限的。选取这样一个邻点，称其为 v_1 。

现在进行迭代：根据 v_1 的选择，存在无穷多个顶点可以通过一条以边 v_0v_1 开始的路到达。因为 v_1 只有有限个邻点，所以存在一个与 v_1 相邻的 v_2 （且 v_2 不同于 v_0 或 v_1 ），通过 v_2 有路可以到达图中无穷多个顶点。用这种方式，我们得到一条包含无穷多个顶点的路。

III.A.4

$\langle zip \rangle ::= \langle digit \rangle \langle digit \rangle \langle digit \rangle \langle digit \rangle \langle digit \rangle [- \langle digit \rangle \langle digit \rangle \langle digit \rangle \langle digit \rangle]$
 $\langle digit \rangle ::= 0 \mid \dots \mid 9$

III.A.5 当我们在前面添加一个字符时，也同时在后面添加它。

$\langle palindrome \rangle ::= a \langle palindrome \rangle a \mid b \langle palindrome \rangle b \mid \dots z \langle palindrome \rangle z \mid \langle letter \rangle \mid \varepsilon$
 $\langle letter \rangle ::= a \mid \dots z$

注意，这里将 ε 也视作回文。

III.A.6 这是一种写法。

```

<course code> ::= <dept> <space> <course-number>
<dept> ::= <two-letter> | <three-letter>
<two-letter> ::= <letter><letter>
<three-letter> ::= <two-letter><letter>
<course-number> ::= <digit><digit><digit>
<digit> ::= 0 | ... 9
<letter> ::= A | ... Z

```

非终结符 $\langle space \rangle$ 生成一个难以直观显示的空格。

我们也可以像下面这样，逐个列举院系名称。

```

<dept> ::= MA | PSY | ...

```

这种做法更符合实际，不会允许无意义的院系名称。但其维护上的劣势在于，如果院系代码发生变化，文法描述也必须随之修改。（在文法中强制要求符合实际往往要付出代价。）

III.A.7

(A) 可以这样改写。

```

<pointfloat> ::= <intpart> <fraction> | <fraction> | <intpart> .

```

(B) 用递归来替换 + 算子。

```

<intpart> ::= <intpart> <digit> | <digit>

```

对于 $\langle exponent \rangle$ ，列举各种情况。

```

<exponent> ::= e <intpart> | e+ <intpart> | e- <intpart> | E <intpart> | E+ <intpart> | E-
<intpart>

```

III.A.8

(A) 这是标准表示法的文法。

```

<roman-numeral> ::= <thousands> <hundreds> <tens> <ones>
<thousands> ::= M*
<hundreds> ::= C | CC | CCC | CCCC | D | DC | DCC | DCCC | DCCCC | ε
<tens> ::= X | XX | XXX | XXXX | L | LX | LXX | LXXX | LXXXX | ε
<ones> ::= I | II | III | IIII | V | VI | VII | VIII | VIIII | ε

```

(B) 这是减法记法的罗马数字文法。

```

<roman-numeral> ::= <thousands> <hundreds> <tens> <ones>
<thousands> ::= M*
<hundreds> ::= CM | CD | (D | ε) (ε | C | CC | CCC)
<tens> ::= XC | XL | (L | ε) (ε | X | XX | XXX)
<ones> ::= IX | IV | (V | ε) (ε | I | II | III)

```

III.A.9

(A) 下面是一个推导。为了简洁起见，我们做了缩写，例如在第三行一次性放入了多个 $\langle statement \rangle$ ，而不是逐个放入。

```

<program> ⇒ { <statement-list> }
          ⇒ { <statement> ; <statement> ; <statement> ; }
          ⇒ { <data-type> <identifier> ; <statement> ; <statement> ; }
          ⇒ { int <identifier> ; <statement> ; <statement> ; }
          ⇒ { int <letter> ; <statement> ; <statement> ; }
          ⇒ { int A ; <statement> ; <statement> ; }
          ⇒ { int A ; <identifier> = <expression> ; <statement> ; }
          ⇒ { int A ; <letter> = <expression> ; <statement> ; }
          ⇒ { int A ; A = <expression> ; <statement> ; }

```

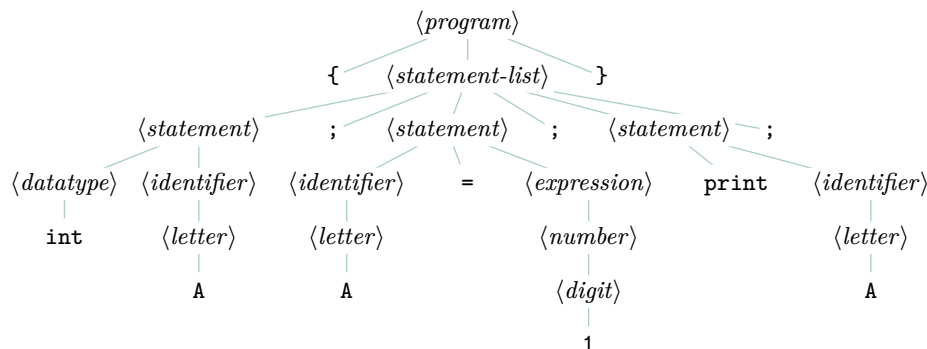


图 79, FOR QUESTION III.A.9: 类 C 语言的语法分析树

$\Rightarrow \{ \text{int } A ; A = \langle \text{number} \rangle ; \langle \text{statement} \rangle ; \}$
 $\Rightarrow \{ \text{int } A ; A = \langle \text{digit} \rangle ; \langle \text{statement} \rangle ; \}$
 $\Rightarrow \{ \text{int } A ; A = 1 ; \langle \text{statement} \rangle ; \}$
 $\Rightarrow \{ \text{int } A ; A = 1 ; \text{print } \langle \text{identifier} \rangle ; \}$
 $\Rightarrow \{ \text{int } A ; A = 1 ; \text{print } \langle \text{letter} \rangle ; \}$
 $\Rightarrow \{ \text{int } A ; A = 1 ; \text{print } A ; \}$

语法分析树见 Figure 79 on page 117。

(B) 是的，因为所有推导都必须从 $\langle \text{program} \rangle$ 出发并经过 $\langle \text{statement-list} \rangle$ ，这迫使它们带有花括号。

III.A.10 在下面的部分推导中，我们做了一些缩写，例如一次性放入多个 $\langle s\text{-expression} \rangle$ 来形成 $\langle \text{list} \rangle$ ，而不是每次只放一个。

$\langle s\text{-expression} \rangle \Rightarrow \langle \text{list} \rangle$
 $\Rightarrow (\langle s\text{-expression} \rangle \langle s\text{-expression} \rangle \langle s\text{-expression} \rangle)$
 $\Rightarrow (\langle s\text{-expression} \rangle \langle \text{list} \rangle \langle s\text{-expression} \rangle)$
 $\Rightarrow (\langle s\text{-expression} \rangle (\langle s\text{-expression} \rangle \langle s\text{-expression} \rangle) \langle s\text{-expression} \rangle)$
 $\Rightarrow (\langle \text{atomic-symbol} \rangle (\langle s\text{-expression} \rangle \langle s\text{-expression} \rangle) \langle s\text{-expression} \rangle)$
 $\Rightarrow (\langle \text{letter} \rangle \langle \text{atom-part} \rangle (\langle s\text{-expression} \rangle \langle s\text{-expression} \rangle) \langle s\text{-expression} \rangle)$
 $\Rightarrow (\langle \text{letter} \rangle \langle \text{letter} \rangle \langle \text{atom-part} \rangle (\langle s\text{-expression} \rangle \langle s\text{-expression} \rangle) \langle s\text{-expression} \rangle)$
 $\Rightarrow (\langle \text{letter} \rangle \langle \text{letter} \rangle \langle \text{letter} \rangle \langle \text{atom-part} \rangle (\langle s\text{-expression} \rangle \langle s\text{-expression} \rangle) \langle s\text{-expression} \rangle)$
 $\Rightarrow (\langle \text{letter} \rangle \langle \text{letter} \rangle \langle \text{letter} \rangle \langle \text{letter} \rangle \langle \text{atom-part} \rangle (\langle s\text{-expression} \rangle \langle s\text{-expression} \rangle) \langle s\text{-expression} \rangle)$
 $\Rightarrow (\langle \text{letter} \rangle \langle \text{letter} \rangle \langle \text{letter} \rangle \langle \text{letter} \rangle \langle \text{atom-part} \rangle (\langle s\text{-expression} \rangle \langle s\text{-expression} \rangle) \langle s\text{-expression} \rangle)$
 $\Rightarrow (\text{c } \langle \text{letter} \rangle \langle \text{letter} \rangle \langle \text{letter} \rangle \langle \text{atom-part} \rangle (\langle s\text{-expression} \rangle \langle s\text{-expression} \rangle) \langle s\text{-expression} \rangle)$
 $\Rightarrow (\text{c o } \langle \text{letter} \rangle \langle \text{letter} \rangle \langle \text{atom-part} \rangle (\langle s\text{-expression} \rangle \langle s\text{-expression} \rangle) \langle s\text{-expression} \rangle)$
 $\Rightarrow (\text{c o n } \langle \text{letter} \rangle \langle \text{atom-part} \rangle (\langle s\text{-expression} \rangle \langle s\text{-expression} \rangle) \langle s\text{-expression} \rangle)$
 $\Rightarrow (\text{c o n s } \langle \text{atom-part} \rangle (\langle s\text{-expression} \rangle \langle s\text{-expression} \rangle) \langle s\text{-expression} \rangle)$
 $\Rightarrow (\text{c o n s } \langle \text{empty} \rangle (\langle s\text{-expression} \rangle \langle s\text{-expression} \rangle) \langle s\text{-expression} \rangle)$
 $\Rightarrow (\text{c o n s } " " (\langle s\text{-expression} \rangle \langle s\text{-expression} \rangle) \langle s\text{-expression} \rangle)$

将剩余的非终结符展开为字母很繁琐，因此我们到此为止。

III.A.11

(A) 例如，这里我们将利用 BNF 记法一次替换多个非终结符，而不是每次只替换一个。

$\langle \text{format-spec} \rangle \Rightarrow 0 \langle \text{width} \rangle \langle \text{type} \rangle$
 $\Rightarrow 0 \langle \text{integer} \rangle \langle \text{type} \rangle$
 $\Rightarrow 0 \langle \text{digit} \rangle \langle \text{type} \rangle$
 $\Rightarrow 03 \langle \text{type} \rangle$

$\Rightarrow 0\ 3\ f$

- (B) $\langle format-spec \rangle \Rightarrow \langle sign \rangle \# 0 \langle width \rangle \langle type \rangle$
 $\Rightarrow + \# 0 \langle width \rangle \langle type \rangle$
 $\Rightarrow + \# 0 \langle integer \rangle \langle type \rangle$
 $\Rightarrow + \# 0 \langle digit \rangle \langle type \rangle$
 $\Rightarrow + \# 0\ 2 \langle type \rangle$
 $\Rightarrow + \# 0\ 2\ X$

III.B.1

- (A) 以下是代码。

```
[ne (node-add-child! nb "e")]
[nf (node-add-child! nc "f")]
[ng (node-add-child! nc "g")]
[nh (node-add-child! ng "h")]
[ni (node-add-child! ng "i")]
)
t))

;; void --> DAG of nodes
;; Make the Cantor graph.
(define (cantor-DAG-make)
  (let* ([t (graph-create "0,0")]
```

- (B) 运行定义后，从解释器得到以下输出。

```
> (define t (binary-tree-make))
> (traverse-dfs t 0 show-node-name)
a
  c
    g
      i
        h
      f
    b
      e
      d
```

- (C) 我们得到这个。

```
> (define t (binary-tree-make))
> (traverse-bfs t show-node-name)
a
  c
  b
    e
    d
    g
    f
      i
      h
```

Chapter IV: 自动机

IV.1.18

(A)	Step	Machine	Step	Machine
	0	abba q₀	3	a q₂
	1	bba q₁	4	q₁
	2	ba q₂		

因为 q_1 是一个接受状态，所以机器接受此串。

(B)	Step	Machine	Step	Machine
	0	bab q₀	2	b q₁
	1	ab q₀	3	q₂

机器不接受此串，因为 q_2 不是接受状态。

(C)	Step	Machine	Step	Machine	Step	Machine
	0	bbaabbaa q₀	3	abbaa q₁	6	aa q₂
	1	baabbaa q₀	4	bbaa q₁	7	a q₁
	2	aabbaa q₀	5	baa q₂	8	q₁

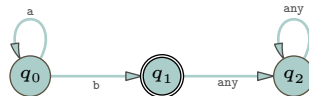
机器接受此串。

IV.1.19 错。例如，例 1.8 中的机器识别的语言由有效整数的十进制表示组成，因此包含以下每一个串：0、1、2、……

IV.1.20 语言不识别串，是机器识别串。也许他们想说的是他们有一个包含空串的语言？或者，他们想说的是他们拥有的是一台机器的语言，并且这台机器接受空串？

IV.1.21 首先，“ n 是无限的”是错误的。该集合是一个语言 $\{b, ab, a^2b, \dots\}$ ，并且在该语言的每个串中， a 的数量是有限的。

其次，肯定存在一台机器能识别那个语言。下面这台有限状态机就可以。



IV.1.22 每消费一个输入字符就会引起一次转移。所以，一个长度为 n 的输入串会使机器经历 n 次转移。（这也涵盖了空串 ϵ 不引起任何转移的情况。）经过 n 次转移，机器将访问过 $n+1$ 个不一定不同的状态，包括初始状态。

IV.1.23

- (A) 一个串属于这个语言当且仅当它由两个等长的 a 串组成，中间由一个 b 分隔。属于该语言的五个串是 aba 、 $aabaa$ 、 b 、 $aaabaaa$ 和 $aaaabaaaa$ 。五个不属于该语言的串是 ba 、 $aaba$ 、 a 、 $abba$ 和 $bbabb$ 。
- (B) 这个语言中的串由一些 a 后跟一个 b ，再跟一些 a 组成。两边的 a 的数量不必相同，且都可以为零。属于该语言的五个串是 $abaa$ 、 $aabaa$ 、 ab 、 $baaaa$ 和 b 。五个不属于该语言的串是 bab 、 $aabb$ 、 bb 、 $aabaab$ 和 a 。
- (C) 一个串属于这个语言的条件是它以 b 开头，后跟任意多个 a （包括零个）。属于该语言的五个串是 baa 、 ba 、 b 、 ba^3 和 ba^4 。五个不属于该语言的串是 a 、 ε 、 $abaaa$ 、 $baaab$ 和 bb 。
- (D) 一个串属于这个语言当且仅当它由一些 a 后跟一个 b ，再跟一些 a 组成，且第二段 a 的数量比第一段多两个。属于该语言的五个串是 $abaaa$ 、 $aabaaaa$ 、 a^3ba^5 、 a^4ba^6 和 baa 。五个不属于该语言的串是 aba 、 $abaaaa$ 、 ba 、 b 和 bab 。
- (E) 这个语言的成员是“双倍”的串，即由某个串与自身拼接得到。属于该语言的五个串是 ε 、 $babbab$ 、 aa 、 $bbbbabbbba$ 和 $abbaabba$ 。五个不属于该语言的串是 aba 、 b 、 $aaabaab$ 、 $babaa$ 和 bbb 。

IV.1.24

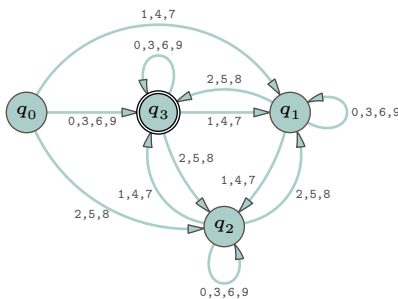
- (A) 对于例 1.6，唯一的接受状态是 q_2 。对于例 1.8，唯一的接受状态也是 q_2 。对于例 1.9，接受状态是 q_3 、 q_6 和 q_8 。对于例 1.10，接受状态是 q_0 。
- (B) 只有例 1.10 接受空串 ε ，因为只有在这台机器中 q_0 是接受状态。
- (C) 对于例 1.6，接受的最短串是长度为二的串 00 。对于例 1.8，接受的最短串是任意一个长度为一位的数字串，例如 6 。对于例 1.9，接受的最短串是任意一个长度为三的串： jpg 、 png 或 pdf 。对于例 1.10，接受的最短串是空串 ε 。
- (D) 例 1.6 的语言是包含至少两个 0 且有偶数个 1 的串的集合。该语言是无限的，因为它包含诸如 ε 、 00 、 0000 、 0^6 等串。例 1.8 的语言是无限的，因为它包含 0 、 1 、 2 、...等串。例 1.9 的语言恰好包含三个元素， $\mathcal{L} = \{jpg, png, and pdf\}$ 。对于例 1.10，其语言是无限的，因为它包含 0 、 3 、 6 、...等串。

IV.1.25 以下是计算过程。

$$\langle q_0, 2332 \rangle \vdash \langle q_2, 332 \rangle \vdash \langle q_2, 32 \rangle \vdash \langle q_2, 2 \rangle \vdash \langle q_1, \varepsilon \rangle$$

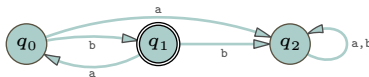
由于 q_1 不是接受状态，机器拒绝 2332 。

IV.1.26 我们必须修改机器，使其初始状态不是接受状态。



本节给出了设计建议：每个状态应有一个直观的功能，该功能服务于机器的目的，且是唯一的。在该机器中，有两个状态 q_0 和 q_3 ，它们的直观含义是：到目前为止看到的数字之和模 3 余 0。然而，它们不同，因为初始状态 q_0 不是接受状态。

IV.1.27 这是转移图。



语言是 $\{\sigma \in \{a, b\}^* \mid \sigma = b(ab)^n, \text{ 其中 } n \in \mathbb{N}\}$ 。

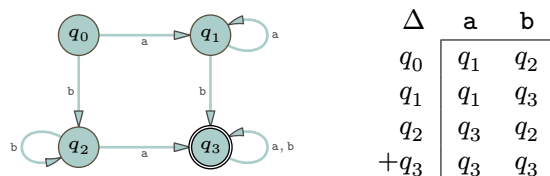


图 80, FOR QUESTION IV.1.29: 接受至少包含一个 a 和一个 b 的串的有限状态机。

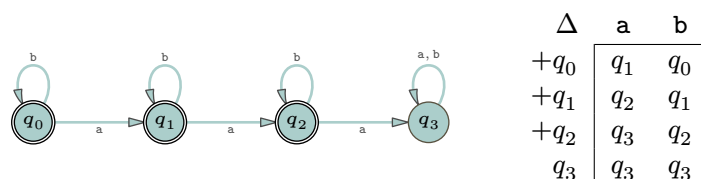


图 81, FOR QUESTION IV.1.29: 接受少于三个 a 的串的有限状态机。

IV.1.28 语言是 $\Sigma = \{a, b\}$ 上恰含一个 b 的串的集合。换句话说, $\mathcal{L}(\mathcal{M}) = \{a^n b a^m \mid n, m \in \mathbb{N}\}$ 。

IV.1.29 正文中建议通过描述每个状态的直观含义来设计机器。虽然问题没有要求,但这里每个子项我们都给出这些含义。

(A) 五个成员是 ab 、 $baab$ 、 ba 、 $b^7 a^3$ 和 $ab^6 a$ 。五个非成员是 $aaaa$ 、 $bbbb$ 、 a 、 ε 和 b 。我们赋予状态如下含义。

状态	描述
q_0	初始状态
q_1	至少看到一个 a, 没有 b
q_2	至少看到一个 b, 没有 a
q_3	至少看到一个 a 和一个 b

圆圈图和转移表如 Figure 80 on page 121 所示。

(B) 五个成员是: bb 、 ε 、 aba 、 a 和 $bbbaab$ 。五个非成员是 aaa 、 $baaa$ 、 a^5 、 $bababa$ 和 a^{10} 。我们赋予状态如下含义。

状态	描述
q_0	初始状态, 尚未看到任何 a
q_1	看到一个 a, 可能还有一些 b
q_2	看到两个 a
q_3	看到三个或更多 a

圆圈图和转移表见 Figure 81 on page 121。

(C) 五个成员是 ab 、 bab 、 aab 、 $baab$ 和 $abab$ 。五个非成员: a 、 bbb 、 ε 、 b 和 $abababb$ 。我们赋予状态如下含义。

状态	描述
q_0	初始状态, 刚才未看到 a
q_1	看到一个 a, 等待 b
q_2	刚看到 ab

转移图和转移表如 Figure 82 on page 122 所示。

(D) 五个成员是 $aabb$ 、 $aaabb$ 、 $aabbb$ 、 $a^7 b^9$ 和 $a^2 b^{200}$ 。五个非成员是 ε 、 a 、 ab 、 ba 和 b 。我们赋予状态如下含义。

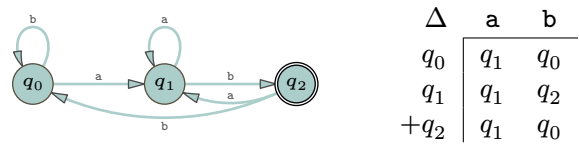


图 82, FOR QUESTION IV.1.29: 接受以 **ab** 结尾的串的有限状态机。

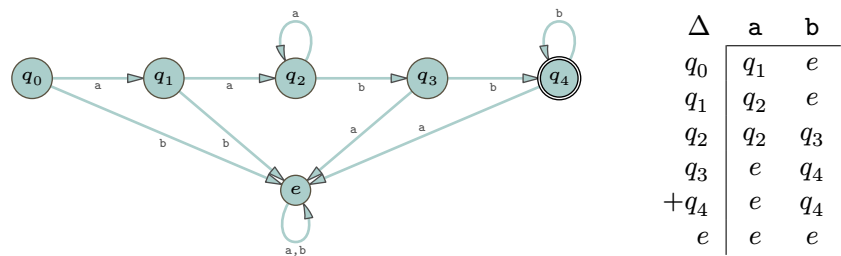


图 83, FOR QUESTION IV.1.29: 用于至少两个 **a** 后接至少两个 **b** 的串的有限状态机。

状态	描述
q_0	初始状态，尚未看到任何输入
q_1	看到一个字符，是 a
q_2	至少看到两个 a ，没有 b
q_3	至少看到两个 a 和一个 b
q_4	至少看到两个 a 和至少两个 b
$q_5 = e$	错误状态，无法恢复

圆圈图和转移表见 Figure 83 on page 122。

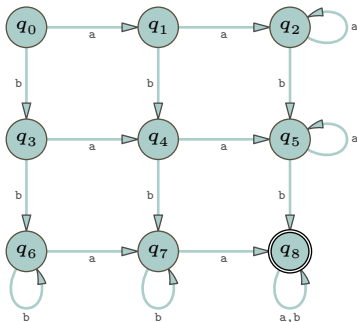
(E) 五个成员是 **aabba**、**aabb**、**abb**、**bb** 和 **a⁵bb⁷**。五个非成员是 ϵ 、**bab**、**aabaa**、**aabbbbaa** 和 **aabbaab**。我们赋予状态如下含义。

状态	描述
q_0	截至目前看到若干 a （包括零个）
q_1	看到若干 a 后接一个 b
q_2	看到若干 a 、两个 b ，以及若干 a （包括零个）
e	错误状态

转移函数的两种表示见 Figure 84 on page 123。

IV.1.30 这些练习的导言建议，通过列举五个语言成员和五个非成员来理解语言描述。虽然本题没有要求，但作为额外的补充，这里给出一些例子：语言的五个元素是 **aabb**、**bbaa**、**abab**、**aababb** 和 **a³b⁴**。五个非元素是 ϵ 、**a**、**b**、**aab** 和 **bbbbabbbb**。

以下是机器。



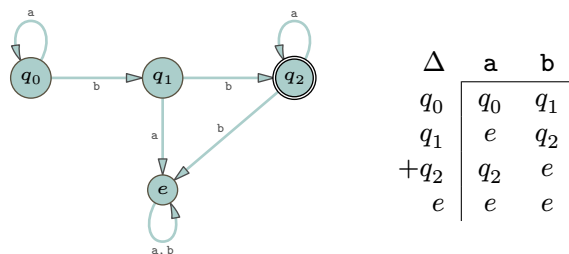
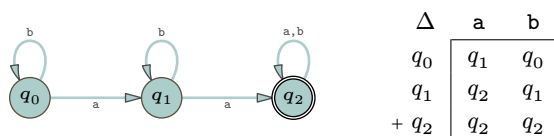


图 84, FOR QUESTION IV.1.29: 用于具有若干 a、接着两个 b、然后若干 a 的串的有限状态机。

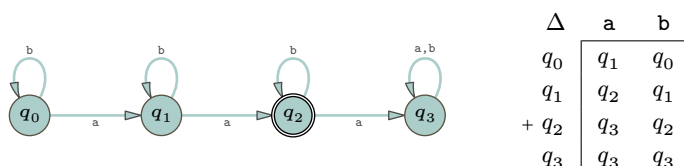
第一行状态 q_0 、 q_1 和 q_2 的直观含义是：当机器处于这些状态时，它目前看到零个 b。第二行的状态表示机器目前看到了一个 b。第三行则是机器已经看到两个（或更多）b 的状态。类似地，第一列的状态是机器目前还未看到任何 a；第二列是机器目前看到了一个 a；最后一列则是机器已经看到两个或更多 a 的状态。

IV.1.31 在这些练习的开头，建议通过列举该语言的五个成员和五个非成员来理清语言描述。虽然问题未作要求，但我们在此对每个语言都提供这些列表。同样，在正文中也建议通过描述每个状态的直观含义来设计机器。虽然这不是必须的，题目也未作要求，我们在此对每一项也给出这些说明。

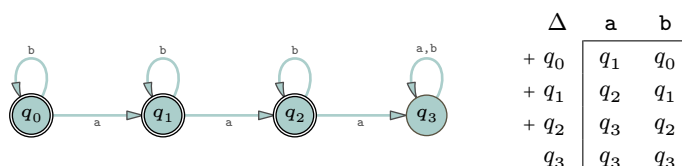
- (A) 五个语言元素是 aa、aaa、aab、aba 和 baa。五个非元素是 ϵ 、a、b、abbbbb 和 ba。此机器仅接受该语言中的串。



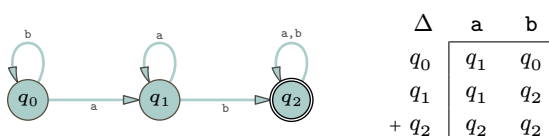
- (B) 五个语言元素是 aa、aab、aba、baa 和 aabb。五个非元素是 ϵ 、a、ba、aaa 和 baaa。



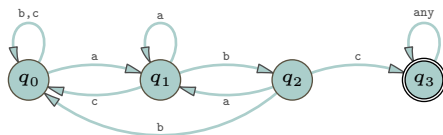
- (C) 五个语言元素是 ϵ 、a、b、aa 和 ab。五个非元素是 aaa、aaab、aaba、abaa 和 baaa。



- (D) 五个语言元素是 ab、aab、aba、abb 和 bab。五个非元素是 ϵ 、a、b、ba 和 bbbbaaaa。



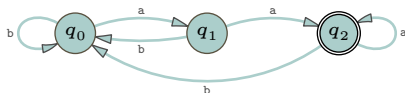
IV.1.32 为了接受包含子串 abc 的串，机器必须有一个状态，其直观含义是“刚刚看到 a，接下来寻找 bc”，这个状态就是 q_1 。状态 q_2 表示机器刚处理完子串 ab，正在寻找 c。而 q_3 表示机器已经看到了子串 abc。最后， q_0 是当机器到目前为止，还未看到潜在子串 abc 的第一个字符时所处的状态。下面是这个机器。



注意，如果机器处理 ab 后接着处理 b ，它会回到 q_0 ；但如果处理 ab 后接着处理 a ，它不会回到 q_0 ，而是进入 q_1 。这可由 q_1 的直观含义推出。

IV.1.33

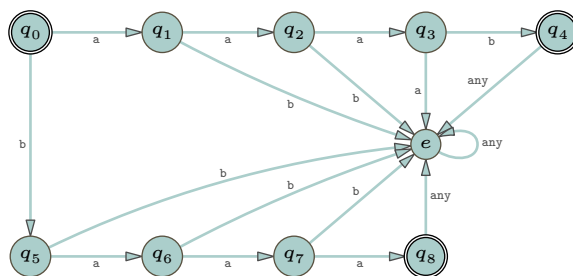
(A) 属于该语言的五个串是： aa 、 aaa 、 baa 、 $aaaa$ 和 $abaa$ 。不属于该语言的五个串是： ε 、 a 、 b 、 ab 、 ba 和 aba 。机器如下。



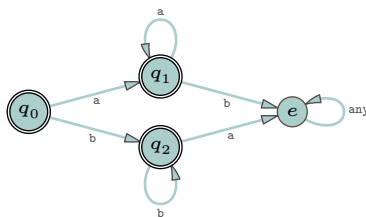
(B) 该语言中唯一的串是 ε 。即 $\mathcal{L} = \{\varepsilon\}$ 。不属于该语言的五个串是： a 、 b 、 aa 、 ab 和 ba 。机器如下。



(C) 该语言中只有两个串： $\mathcal{L} = \{aaab, baaa\}$ 。不属于该语言的五个串是： ε 、 a 、 b 、 aa 和 ab 。机器如下。

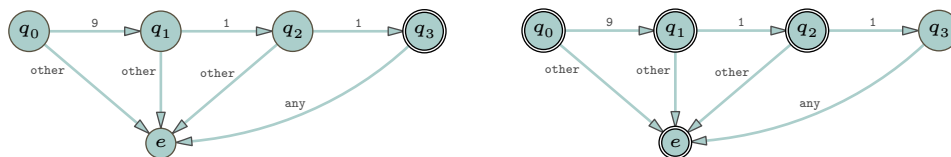


(D) 属于该语言的五个串是： ε 、 a 、 b 、 aa 和 bb 。不属于该语言的五个串是： ab 、 ba 、 aab 、 aba 和 baa 。机器如下。



注意 q_0 是接受状态，因为 ε 是该语言的成员。

IV.1.34 第一个机器只接受 911 ，而第二个接受除 911 外的任何串。注意，在这两个机器中，接受状态的集合是互补的。



IV.1.35 以下是长度为三的串。

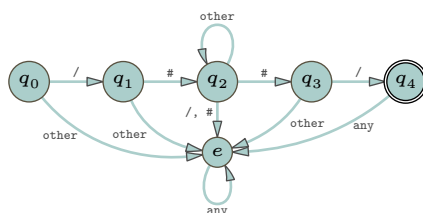
输入串 σ	aaa	aab	aba	abb	baa	bab	bba	bbb
$\hat{\Delta}(\sigma)$	q_1	q_2	q_1	q_2	q_1	q_2	q_1	q_0

以下是长度为四的串。

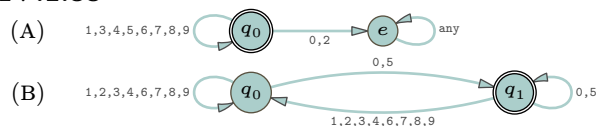
输入串 σ	aaaa	aaab	aaba	aabb	abaa	abab	abba	abbb
$\hat{\Delta}(\sigma)$	q_1	q_2	q_1	q_2	q_1	q_2	q_1	q_2
	baaa	baab	baba	babb	bbaa	bbab	bbba	bbbb
	q_1	q_2	q_1	q_2	q_1	q_2	q_1	q_0

IV.1.36 它输出初始状态, $\hat{\Delta}(\varepsilon) = q_0$ 。

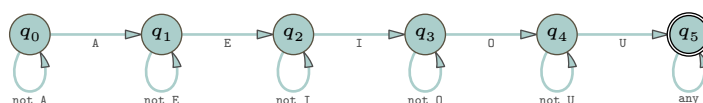
IV.1.37



IV.1.38

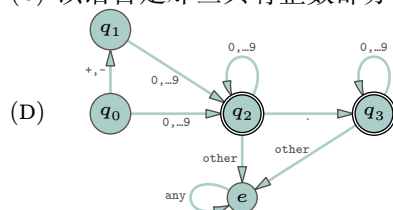


IV.1.39 注意, 虽然串 $\sigma = \text{AEAIQAUUA}$ 中有一个 A 出现在 E 之后, 但机器仍应接受它, 因为该串中存在一个由有序元音构成的子序列。



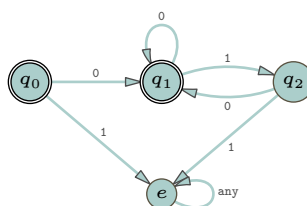
IV.1.40

- (A) 五个属于该语言的串: +1、123、-123.45、-0.0、0。五个不属于该语言的串: +、+-、...、12.3.4、2+3。
- (B) 不包含, 根据 $\langle \text{posreal} \rangle$ 规则, 小数点前必须至少有一位数字。
- (C) 该语言是那些具有整数部分、小数点和小数部分的实数表示的集合。

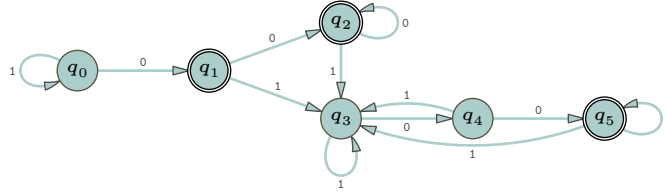


IV.1.41 虽然练习未要求, 我们为每种语言列出了五个属于该语言的串和五个不属于该语言的串。

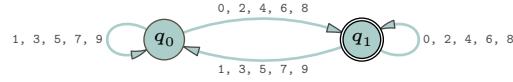
- (A) 五个属于该语言的串: 010、0010、0、0101000、 ε 。五个不属于该语言的串: 10、0110、0101、01001、1。



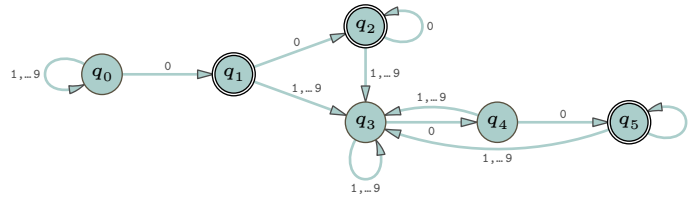
- (B) 一个串是 4 的倍数的二进制表示当且仅当它以两个零 00 结尾, 或者它就是 0。五个属于该语言的串: 100、0100、1000、10100、0。五个不属于该语言的串: 10、1、1001、001、 ε 。



(c) 五个属于该语言的串：0、20、12、88、08。五个不属于该语言的串：1、01、2221、17、 ε 。



(d) 这与第二项基本相同。一个串是 100 的倍数的十进制表示当且仅当它以两个零 00 结尾，或者它就是 0。五个属于该语言的串：100、0100、2000、60300、0。五个不属于该语言的串：10、8、1009、001、 ε 。



IV.1.42 一个数能被四整除，当且仅当其十进制表示以 00、04、 $\dots\dots$ 96 结尾，或者是单个数字 0、4 或 8。一共是 28 种情况。用图表示会显得很杂乱，但这是一种有限语言，并且对于任何有限语言，都存在一个识别该语言的有限状态机。（我们将在关于非确定性的章节中看到另一种能画出更清晰状态图的构造方法。）

IV.1.43 这是该示例的转移函数。

Δ	0	1
q_0	q_1	q_3
q_1	q_2	q_4
$+ q_2$	q_2	q_5
q_3	q_4	q_0
q_4	q_5	q_1
q_5	q_5	q_2

(A) 如果输入串是 $\sigma = 0$ ，那么去掉最后一个字符得到 $\sigma = \tau \frown 0 = \varepsilon \frown 0$ 根据定义得到如下结果。

$$\hat{\Delta}(0) = \Delta(\hat{\Delta}(\varepsilon), 0) = \Delta(q_0, 0) = q_1$$

（在最左边的表达式中，0 是一个长度为 1 的串，而在等式其余部分，0 是一个字符。我们忽略这种无伤大雅的差异。）类似地， $\hat{\Delta}(1) = q_3$ 。

(B) 对于 $\sigma = 00$ ，去掉最后一个字符得到 $\sigma = \tau \frown 0 = 0 \frown 0$ （这里第一个 0 是一个长度为 1 的串，而第二个是一个字符）。然后利用上一小题中计算出的 $\hat{\Delta}(0)$ 。

$$\hat{\Delta}(00) = \Delta(\hat{\Delta}(0), 0) = \Delta(q_1, 0) = q_2$$

其他三个类似。

$$\hat{\Delta}(01) = q_4 \quad \hat{\Delta}(10) = q_4 \quad \hat{\Delta}(11) = q_0$$

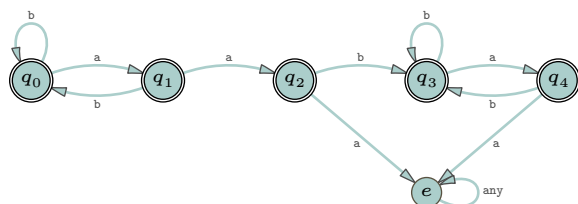
(c) 对于每一个长度为 3 的串，利用上一小题的计算结果。以下是第一个。

$$\hat{\Delta}(000) = \Delta(\hat{\Delta}(00), 0) = \Delta(q_2, 0) = q_2$$

其他的以同样方式计算。

$\hat{\Delta}(000)$	$\hat{\Delta}(001)$	$\hat{\Delta}(010)$	$\hat{\Delta}(011)$	$\hat{\Delta}(100)$	$\hat{\Delta}(101)$	$\hat{\Delta}(110)$	$\hat{\Delta}(111)$
q_2	q_5	q_5	q_1	q_5	q_1	q_1	q_3

IV.1.44



IV.1.45

(A) 只有一台这样的机器。

Δ	0	1
q_0	q_0	q_0

(B) 共有 16 台机器，列表如下。

Δ_0	0	1	Δ_1	0	1	Δ_2	0	1	Δ_{15}	0	1
q_0	q_0	q_0	q_0	q_0	q_0	q_0	q_0	q_0	q_0	q_1	q_1
q_1	q_0	q_0	q_1	q_0	q_1	q_1	q_1	q_0	q_1	q_1	q_1

(C) 考虑有 n 个状态的情况， $Q = \{q_0, \dots, q_{n-1}\}$ 。转移表的每一行有两列，因此不同的行有 $n \cdot n = n^2$ 种可能。整个转移表需要选取 n 个这样的行，共有 $n^2 \cdot n^2 \dots n^2 = n^{2n}$ 种选取方式。(D) 对于 n 个状态中的每一个，它要么是接受状态，要么不是。额外的因子 2^n 使得不同的转移表总数等于 $2^n n^{2n}$ 。

IV.1.46 问题情境可能处于的状态取决于人、狼、羊和卷心菜分别位于近岸还是远岸。允许的转移是：人带狼过河、人带羊过河、人带卷心菜过河，以及人独自过河。如果人离开后，狼与羊独处（无论在近岸还是远岸），那就是一个错误状态，羊与卷心菜单处亦是如此。（人、狼、羊在一起不算错误，因为人和它们在一起。）

位置		备注
近岸	远岸	
人、狼、羊、卷心菜	—	初始状态
人、狼、羊	卷心菜	
人、狼、卷心菜	羊	
人、羊、卷心菜	狼	
人、羊	狼、卷心菜	
人、狼	羊、卷心菜	错误
人、卷心菜	狼、羊	错误
人	狼、羊、卷心菜	错误
狼、羊、卷心菜	人	错误
狼、羊	人、卷心菜	错误
狼、卷心菜	人、羊	
狼	人、羊、卷心菜	
羊、卷心菜	人、狼	错误
羊	人、狼、卷心菜	
卷心菜	人、狼、羊	
—	人、狼、羊、卷心菜	接受状态

图 Figure 85 on page 128 中所示的有限状态机涵盖了所有这些可能性。例如，标记为 ‘n: mwg,f: c’ 的行表示人在近岸与狼和羊在一起，而卷心菜单独在远岸。所有错误被合并为一个单一状态。因此，图中表格就没有标记为 ‘n: wg,f: mc’ 的行。初始状态是 q_0 ，唯一的接受状态是 q_9 。

除了第一段提到的两个错误之外，如果人试图带狼过河，但狼与人并不在同侧，这也是一种错误，对羊和卷心菜亦然。

圆圈与箭头图没有画出指向错误状态的箭头，因为那样会使图面过于杂乱。有了这张图，寻找解就容易了。图中一条路径是 $q_0-q_5-q_2-q_8-q_3-q_7-q_4-q_9$ 。用文字描述，这条路径是：人先把羊带到对岸，

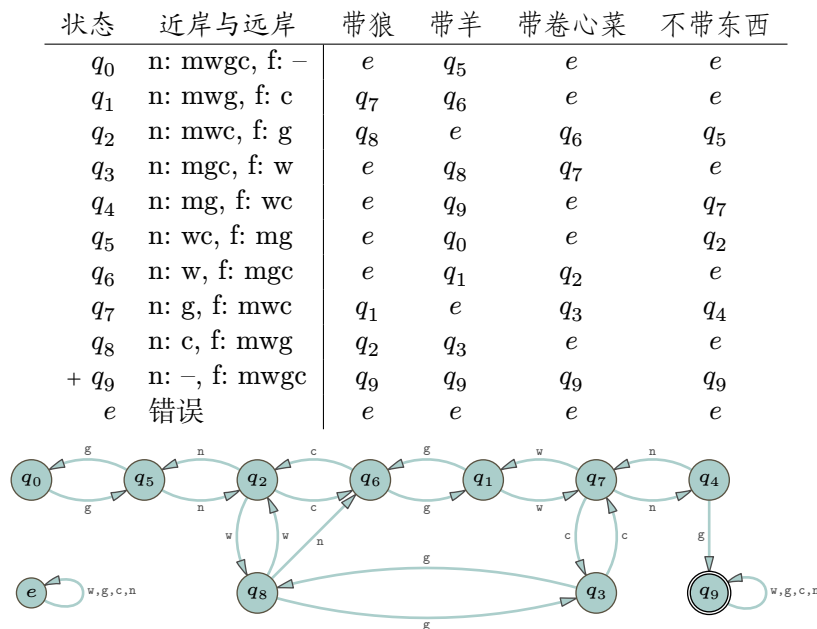


图 85, FOR QUESTION IV.1.46: 狼-羊-卷心菜问题的有限状态机。

并将其留在那里。他独自返回近岸。然后他把狼从近岸带到对岸。接着他把羊带回近岸。他把羊留在近岸，带上卷心菜将其带到对岸。最后，他独自返回近岸，并最终把羊带到对岸。

IV.1.47 对于这个至少包含两个字符的字母表，我们记作 Σ 。我们固定该字母表中的一个字符 s 。

(A) 我们将证明在该字母表上存在无限多个不同的有限状态机。下面这些机器之所以不同，既因为它们的状态数不同，也因为它们识别的语言不同。

Δ	s	其他	Δ	s	其他	Δ	s	其他
$+ q_0$	q_0	q_0	$+ q_0$	q_1	q_0	$+ q_0$	q_1	q_0
			q_1	q_0	q_0	q_1	q_2	q_0
						q_2	q_0	q_0

(B) 根据定义，可能的状态集是 $Q = \{q_0, q_1, \dots\}$ 。然后我们可以对有限状态机进行计数，因为对于任意有限多个状态的集合 $\{q_0, \dots, q_k\}$ ，只有有限多个转移表。因此，使用这些状态的有限状态机的数量是可数的。可数多个可数集合的并仍是可数的，所以，所有有限状态集合上的机器数量也是可数的。

(C) 因为 Σ 至少有两个成员，通常的对角化构造表明 Σ 的幂集是不可数的。

IV.2.23 关键点在于，表中的每一项都是状态的集合。左边是例 2.7 的机器，右边是例 2.8 的机器。

Δ	a	b	Δ	a	b	c
q_0	$\{q_0, q_1\}$	$\{q_0, q_2\}$	$+ q_0$	$\{q_1\}$	$\{\}$	$\{\}$
q_1	$\{q_3\}$	$\{\}$	q_1	$\{\}$	$\{\}$	$\{q_0\}$
q_2	$\{\}$	$\{q_3\}$				
$+ q_3$	$\{q_3\}$	$\{q_3\}$				

IV.2.24

(A) 接受。从 q_0 出发，可以经过一次 ε -转移到达接受状态 q_2 。也就是说，一条接受的极大路径是 $\langle q_0, \varepsilon \rangle \vdash \langle q_2, \varepsilon \rangle$ ，其中的转移是一次 ε -转移。

(B) 不接受。从 q_0 出发，机器可以读取 0 并转移到 q_1 ，也可以先走一次 ε -转移到 q_2 再读取 0，这会使机器进入无状态。 q_2 和无状态都不是接受状态。

Input:		Input:		Input:	
			a		b
q_3		q_3	$\vdash$	q_3	q_3
$\perp$		$\perp$		$\perp$	$\perp$
q_0		q_0	$\vdash$	q_2	q_1
Step: 0		Step: 0		1	Step: 0
					1

图 86, FOR QUESTION IV.2.27: 串 ε 、a 和 b 的计算树。

更形式化地说, q_0 的 ε -闭包是 $\{q_0, q_2\}$ 。这两个状态在读取字符 0 后, 所转移到的状态的 ε -闭包都不包含任何接受状态。

- (c) 接受。对于这个输入, 机器可以从 q_0 转移到 q_1 , 再到 q_2 , 然后绕循环再次回到 q_2 。更形式化地说, 这是一列以接受状态结尾的允许转移序列。

$$\langle q_0, 011 \rangle \vdash \langle q_1, 11 \rangle \vdash \langle q_2, 1 \rangle \vdash \langle q_2, \varepsilon \rangle$$

- (D) 不接受。从 q_0 开始, 纸带上第一个字符是 0, 机器可以做两件事。如果它选择 ε -转移, 那么当它读取 0 时, 就会进入无状态, 而这不是一个接受状态。

否则, 0 会使机器转移到 q_1 。接着, 对于下一个纸带字符 1, 机器转移到 q_2 。最后, 对于第三个字符 0, 机器转移到无状态。

- (E) 有两个: 01111 和 11111。

IV.2.25 在这个班上, 有人说某个主题“只是数学抽象”, 这就像在空气动力学课上有人反对说, 这都“不过是空气而已”。没错, 我们研究的正是这个。

换个说法, 如果有人把我们这里所做的关于计算的思考, 与数据结构课程中关于计算的思考相比较, 并推崇后者作为“真实”的典范, 那未免有些牵强。“真实”就是给孩子换尿布、嫁接果树、给饥饿者提供食物。我们所做的一切, 都是这样或那样的抽象。

如果那人回应道: “好吧, 但抽象可能太过头, 以至于偏离了我们当初选择这个专业的初衷”, 那么这个看法就更理性了。我们在这里介绍非确定性, 众多原因之一是: 要理解第五章的内容, 就必须掌握它。该章节描述了当今该领域最重要的问题—— P 对 NP 问题——而这个问题将直接影响所有从业者。

况且, 这也挺有意思的。光是这点, 难道不也值得考虑一下吗?

IV.2.26 如果始终走 ε -转移, 在 q_0 和 q_1 之间不停地循环下去, 确实不会得到成功的结果。但定义并不要求处理输入串时不存在任何错误的走法, 只要求必须存在一种正确的走法。

如果存在一系列转移, 使机器到达一个处于接受状态的停机格局, 那么机器就接受这个输入串。对于这里这台机器, 可以从 q_0 开始, 读入第一个 a 后转到 q_1 , 再沿 ε -转移回到 q_0 , 读入第二个 a 后转到 q_1 , 最后读入 b 转到接受状态 q_2 , 从而接受 aab。

$$\langle q_0, aab \rangle \vdash \langle q_1, ab \rangle \vdash \langle q_0, ab \rangle \vdash \langle q_1, b \rangle \vdash \langle q_2, \varepsilon \rangle$$

IV.2.27 前三个串的计算树见 Figure 86 on page 129。输入串 aa 和 ab 的计算树见 Figure 87 on page 130。串 ba 和 bb 的计算树见 Figure 88 on page 130。

IV.2.28 这是转移序列。

$$\begin{aligned} \langle q_0, 010101110 \rangle \vdash \langle q_0, 101011110 \rangle \vdash \langle q_0, 01011110 \rangle \vdash \langle q_1, 101110 \rangle \\ \vdash \langle q_2, 01110 \rangle \vdash \langle q_3, 1110 \rangle \vdash \langle q_4, 110 \rangle \vdash \langle q_5, 10 \rangle \vdash \langle q_6, 0 \rangle \vdash \langle q_7, \varepsilon \rangle \end{aligned}$$

IV.2.29

- (A) ε -闭包如下。

<i>Input:</i>	a		a	
	q_3	$\vdash$	q_3	$\vdash$ q_3
	$\perp$			
	q_0	$\vdash$	q_2	
<i>Step:</i>	0		1	2

<i>Input:</i>	a		b	
	q_3	$\vdash$	q_3	$\vdash$ q_3
	$\perp$			$\perp$
	q_0	$\vdash$	q_2	$\vdash$ q_0
<i>Step:</i>	0		1	2

图 87, FOR QUESTION IV.2.27: 串 aa 和 aa 的计算树。

<i>Input:</i>	b		a	
			q_3	$\vdash$ q_3
			$\perp$	
	q_3		q_0	$\vdash$ q_2
	$\perp$		$\perp$	
	q_0	$\vdash$	q_1	
<i>Step:</i>	0		1	2

<i>Input:</i>	b		b	
				q_3
				$\perp$
			q_3	$\vdash$ q_0
			$\perp$	$\perp$
	q_3		q_0	$\vdash$ q_1
	$\perp$		$\perp$	
	q_0	$\vdash$	q_1	
<i>Step:</i>	0		1	2

图 88, FOR QUESTION IV.2.27: 串 ba 和 bb 的计算树。

状态 q	q_0	q_1	q_2
ε -闭包 $\hat{E}(q)$	$\{q_0\}$	$\{q_1, q_2\}$	$\{q_2\}$

- (B) 它不接受空串, 因为 q_0 的 ε -闭包中不包含任何接受状态。
- (C) 它接受两个单字符的串 **a** 和 **b**。例如, 对于串 **a**, 机器可以从 q_0 转移到 q_1 , 然后进行一次 ε -转移到 q_2 , 而 q_2 是一个接受状态。
- (D) 给定机器可以通过以下方式接受 **aab**: 从 q_0 开始, 读入第一个 **a** 后转移到 q_0 , 读入第二个 **a** 后转移到 q_1 , 进行一次 ε -转移到 q_2 , 然后读入 **b** 转移到 q_2 , 从而结束于一个接受状态。

$$\langle q_0, \mathbf{aab} \rangle \vdash \langle q_0, \mathbf{ab} \rangle \vdash \langle q_1, \mathbf{b} \rangle \vdash \langle q_2, \mathbf{b} \rangle \vdash \langle q_2, \varepsilon \rangle$$

- (E) 五个最短的串是: **a**、**b**、**aa**、**ab** 和 **aab**。
- (F) 五个不被接受的串是: 空串 ε 、**ba**、**baa**、**bab** 和 **baaa**。

IV.2.30

Δ	a	b	ε
q_0	$\{q_0, q_1\}$	$\{q_1\}$	$\{\}$
q_1	$\{\}$	$\{\}$	$\{q_2\}$
$+ q_2$	$\{\}$	$\{q_0, q_2\}$	$\{\}$

IV.2.31

- (A) $\langle q_0, 011 \rangle \vdash \langle q_2, 11 \rangle \vdash \langle q_2, 1 \rangle \vdash \langle q_2, \varepsilon \rangle$
- (B) $\langle q_0, 00011 \rangle \vdash \langle q_1, 0011 \rangle \vdash \langle q_1, 011 \rangle \vdash \langle q_1, 11 \rangle \vdash \langle q_2, 1 \rangle \vdash \langle q_2, \varepsilon \rangle$
- (C) 接受, 因为状态 q_0 是接受状态。
- (D) 它接受 0, 因为可以通过以下步骤结束于一个接受状态。

$$\langle q_0, 0 \rangle \vdash \langle q_2, \varepsilon \rangle$$

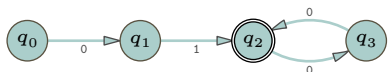
它不接受 1, 因为没有这样的步骤序列。

- (E) 五个是: ε 、0、01、011 和 001。
- (F) 这里是五个: 1、10、11、00 和 111。
- (G) 一种描述是 $\mathcal{L} = \{\sigma \in \mathbb{B}^* \mid \sigma = 0^i 1^j, \text{ 其中 } i \geq 1 \text{ 且 } j \geq 0\}$ 。(注意当 $i = j = 0$ 时 $\sigma = \varepsilon$ 。)

IV.2.32 状态 q_0 在第 $m = 1$ 列达到极限。状态 q_1 在第 0 列达到极限。状态 q_2 在第 2 列达到极限， q_3 在第 1 列达到极限。

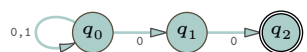
	$m = 0$	1	2	3	$\hat{E}(q)$
q_0	$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_1\}$
q_1	$\{q_1\}$	$\{q_1\}$	$\{q_1\}$	$\{q_1\}$	$\{q_1\}$
q_2	$\{q_2\}$	$\{q_2, q_3\}$	$\{q_1, q_2, q_3\}$	$\{q_1, q_2, q_3\}$	$\{q_1, q_2, q_3\}$
q_3	$\{q_3\}$	$\{q_1, q_3\}$	$\{q_1, q_3\}$	$\{q_1, q_3\}$	$\{q_1, q_3\}$

IV.2.33 非确定性机器不需要错误状态。

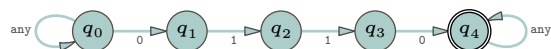


IV.2.34

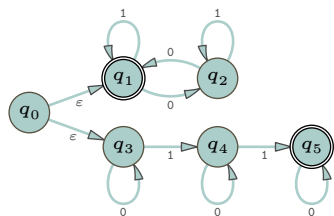
(A)



(B)



(C)



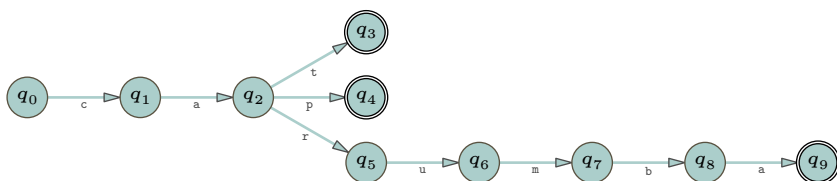
(D)



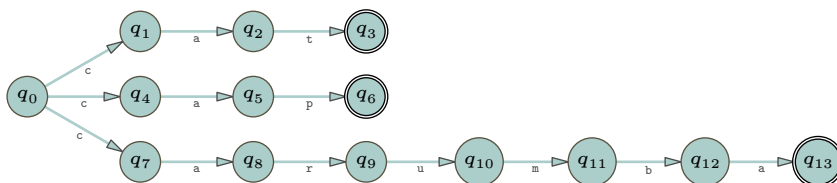
IV.2.35



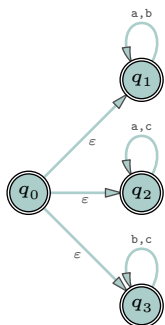
IV.2.36 解法不止一种。这是一种。



这是另一种。

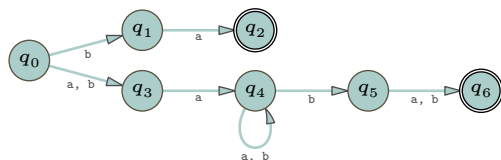


IV.2.37 这个语言包含空串。

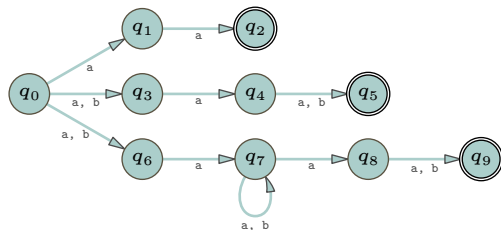


IV.2.38

(A)

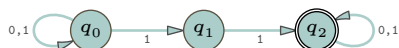


(B)

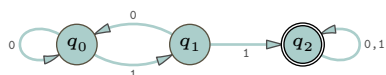


IV.2.39 任何 **b** 后面都必须跟着一个 **a**，除非它出现在串的末尾；此时，末尾部分必须是一串不含 **a** 的 **b**。简而言之，该语言是 $\mathcal{L} = \{\sigma \in \{a, b\}^* \mid \sigma \text{ 不含有子串 } bba\}$ 。

IV.2.40 下面是一台非确定性机器



而这台是确定性的。

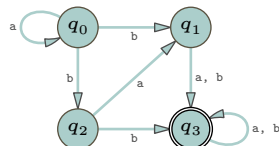


IV.2.41 这是针对左边机器和右边机器的结果。

Δ_{D_0}	0	1	Δ_{D_1}	0	1
$r_0 = \{\}$	r_0	r_0	$r_0 = \{\}$	r_0	r_0
$r_1 = \{q_0\}$	r_4	r_0	$r_1 = \{q_0\}$	r_7	r_0
$r_2 = \{q_1\}$	r_3	r_3	$r_2 = \{q_1\}$	r_3	r_3
$+ r_3 = \{q_2\}$	r_0	r_1	$+ r_3 = \{q_2\}$	r_0	r_5
$r_4 = \{q_0, q_1\}$	r_7	r_3	$r_4 = \{q_0, q_1\}$	r_7	r_3
$+ r_5 = \{q_0, q_2\}$	r_4	r_1	$+ r_5 = \{q_0, q_2\}$	r_7	r_5
$+ r_6 = \{q_1, q_2\}$	r_3	r_5	$+ r_6 = \{q_1, q_2\}$	r_3	r_5
$+ r_7 = \{q_0, q_1, q_2\}$	r_7	r_5	$+ r_7 = \{q_0, q_1, q_2\}$	r_7	r_5

IV.2.42 字母表是 $\Sigma = \{a, b\}$ 。

(A)



- (B) 一个串在该语言中，当且仅当它具有以下形式： $\sigma_0 \hat{\ } \sigma_1 \hat{\ } \sigma_2$ ，其中 $\sigma_0 = \mathbf{a}^k \mathbf{b}$ 对某个 $k \in \mathbb{N}$ 成立，且 σ_1 是 $\mathbf{aa}$ 、 $\mathbf{ab}$ 或 $\mathbf{b}$ 之一，而 σ_2 是 Σ^* 中的任意成员。
- (C) 这是对应的确定性机器的转移函数。

Δ_D	a	b
$r_0 = \{ \}$	r_0	r_0
$r_1 = \{q_0\}$	r_1	r_8
$r_2 = \{q_1\}$	r_4	r_4
$+ r_3 = \{q_2\}$	r_2	r_4
$r_4 = \{q_3\}$	r_4	r_4
$r_5 = \{q_0, q_1\}$	r_7	r_{14}
$+ r_6 = \{q_0, q_2\}$	r_5	r_{14}
$r_7 = \{q_0, q_3\}$	r_7	r_{14}
$+ r_8 = \{q_1, q_2\}$	r_9	r_4
$r_9 = \{q_1, q_3\}$	r_4	r_4
$+ r_{10} = \{q_2, q_3\}$	r_9	r_4
$+ r_{11} = \{q_0, q_1, q_2\}$	r_{12}	r_{14}
$r_{12} = \{q_0, q_1, q_3\}$	r_7	r_{14}
$+ r_{13} = \{q_0, q_2, q_3\}$	r_{12}	r_{14}
$+ r_{14} = \{q_1, q_2, q_3\}$	r_9	r_4
$+ r_{15} = \{q_0, q_1, q_2, q_3\}$	r_{12}	r_{14}

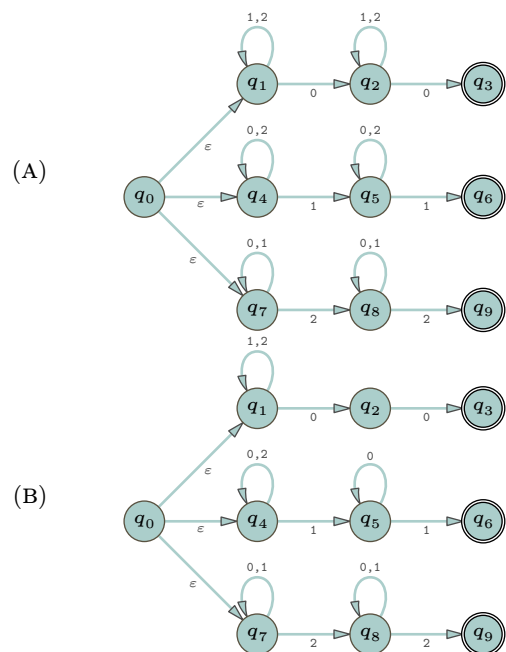
IV.2.43



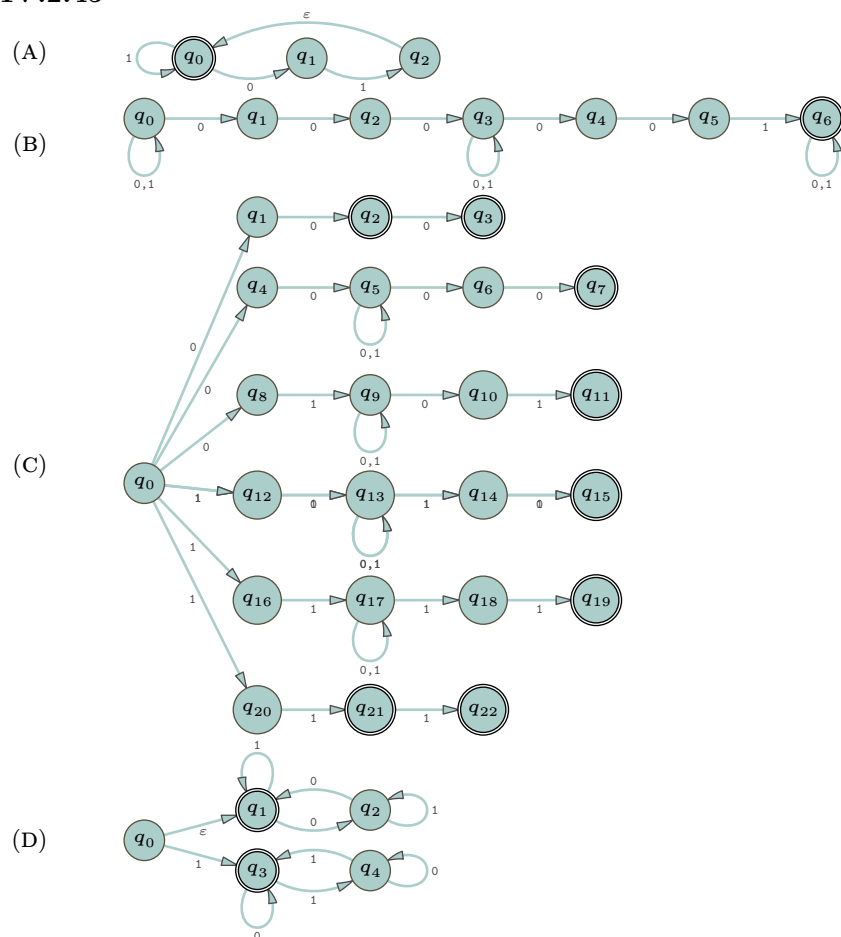
相应的确定性机器有八个状态。其接受状态是那些包含 q_2 的状态。

Δ_D	0	1
$r_0 = \{ \}$	r_0	r_0
$r_1 = \{q_0\}$	r_0	r_2
$r_2 = \{q_1\}$	r_3	r_0
$+ r_3 = \{q_2\}$	r_3	r_3
$r_4 = \{q_0, q_1\}$	r_3	r_2
$+ r_5 = \{q_0, q_2\}$	r_3	r_6
$+ r_6 = \{q_1, q_2\}$	r_3	r_3
$+ r_7 = \{q_0, q_1, q_2\}$	r_3	r_6

IV.2.44



IV.2.45



IV.2.46 转移图如下。



这两个机器的转移表如下。

Δ	a	b	c
$+ q_0$	$\{\}$	$\{\}$	$\{\}$

Δ	a	b	c
q_0	$\{q_1\}$	$\{q_1\}$	$\{q_1\}$
$+ q_1$	$\{q_1\}$	$\{q_1\}$	$\{q_1\}$

IV.2.47

(A) 这是 **abb** 的一个推导。

$$S \Rightarrow aA \Rightarrow abB \Rightarrow abb$$

这是 **aabb** 的一个推导。

$$S \Rightarrow aA \Rightarrow aaA \Rightarrow aabB \Rightarrow aabb$$

此外，这是 **abbb** 的一个推导。

$$S \Rightarrow aA \Rightarrow abB \Rightarrow abbB \Rightarrow abbb$$

这就验证了第一个串被机器接受。

$$\langle S, abb \rangle \vdash \langle A, bb \rangle \vdash \langle B, b \rangle \vdash \langle F, \varepsilon \rangle$$

第二个串如下。

$$\langle S, aabb \rangle \vdash \langle A, abb \rangle \vdash \langle A, bb \rangle \vdash \langle B, b \rangle \vdash \langle F, \varepsilon \rangle$$

第三个串类似。

$$\langle S, abbb \rangle \vdash \langle A, bbb \rangle \vdash \langle B, bb \rangle \vdash \langle B, b \rangle \vdash \langle F, \varepsilon \rangle$$

(B) 该机器和文法的语言是 $\mathcal{L} = \{a^i b^j \mid i > 0, j > 1\}$ 。

IV.2.48 这两个问题都是可解的。对于每一道题，我们都将勾勒一个算法。

(A) 显然，我们可以为确定性有限状态机编写一个模拟器。这个模拟器是通用的，因为它接收 Δ 表和输入串作为输入。根据 Church 论题，既然我们能在现代计算机上编写它，就能为 Turing 机编写一个。当给定输入串 σ 时，机器将在不超过 $|\sigma|$ 次转移内处理完它，因此模拟过程必定会停机。然后我们只需检查结束状态是否为接受状态即可。

(B) 章节正文讨论了如何通过交错并行（即时间切片）来模拟非确定性计算。或者，我们可以使用幂集算法将其转换为确定性机器，然后模拟该确定性机器。之后，我们就可以像前一项那样，回答这台机器是否接受给定输入串 σ 的问题。

IV.2.49

(A) 结果如下。

$E(q, m)$	$m = 0$	1	2	3
q_0	$\{q_0\}$	$\{q_0, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_3\}$
q_1	$\{q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_1, q_3\}$
q_2	$\{q_2\}$	$\{q_2\}$	$\{q_2\}$	$\{q_2\}$
q_3	$\{q_3\}$	$\{q_3\}$	$\{q_3\}$	$\{q_3\}$

因此， $\hat{E}(q_0) = \{q_0, q_3\}$ 、 $\hat{E}(q_1) = \{q_0, q_1, q_3\}$ 、 $\hat{E}(q_2) = \{q_2\}$ 和 $\hat{E}(q_3) = \{q_3\}$ 。

(B) 情况类似。

$E(q, m)$	$m = 0$	1	2	3
q_0	$\{q_0\}$	$\{q_0\}$	$\{q_0\}$	$\{q_0\}$
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_1, q_2\}$
q_2	$\{q_2\}$	$\{q_2\}$	$\{q_2\}$	$\{q_2\}$

因此， $\hat{E}(q_0) = \{q_0\}$ 、 $\hat{E}(q_1) = \{q_1, q_2\}$ 和 $\hat{E}(q_2) = \{q_2\}$ 。

	$a*b$	a^*	$\emptyset$	ε	$b(a b)a$	$(a b)(\varepsilon a)a$
ε	F	T	F	T	F	F
a	F	T	F	F	F	F
b	T	F	F	F	F	F
aa	F	T	F	F	F	T
ab	T	F	F	F	F	F
ba	F	F	F	F	F	T
bb	F	F	F	F	F	F
aaa	F	T	F	F	F	T
aab	T	F	F	F	F	F
aba	F	F	F	F	F	F
abb	F	F	F	F	F	F
baa	F	F	F	F	T	T
bab	F	F	F	F	F	F
bba	F	F	F	F	T	F
bbb	F	F	F	F	F	F

图 89, FOR QUESTION IV.3.19: 串匹配测试。

IV.3.16 每个答案都用下划线标出了匹配部分。

(A) 是。一个匹配是 $\underline{00} \underline{1} \underline{0}$ 。

(B) 是。这个可行: $\underline{1} \underline{0} \underline{1}$ 。

(C) 否。要匹配一个含有 0 的串, 那个 0 必须是最后一个字符。

(D) 是。一个匹配是 $\underline{1} \underline{01}$ 。

(E) 是; 这是一个星号表示“零个”(即“零个或多个”)的例子。空串 ε 与串 01 的拼接等于给定的串 01。所以我们可以将 01 写成 $\varepsilon \wedge 01$, 并识别这个匹配: $\underline{\varepsilon} \underline{0} \underline{1}$ 。

IV.3.17

(A) 五个匹配的串是 0、01、011、0111、以及 $01^4 = 01111$ 。五个不匹配的串是 ε 、1、10、11、以及 100。

(B) 五个匹配的串是 ε 、01、0101、010101、以及 $(01)^4$ 。五个不匹配的串是 0、1、00、10 以及 11。

(C) 只有两个匹配的串: 101 和 111。五个不匹配的串是 ε 、0、1、00、01、以及 10。

(D) 五个匹配的串是 0、01、00、1 以及 10。五个不匹配的串是 ε 、001、0001、101、以及 1101。

(E) 匹配串: 无。不匹配串: ε 、0、1、00、01、以及 10。

IV.3.18

(A) 该语言由任意数量的 a (其数量可以为零), 后跟一个 c , 再后跟任意数量的 b 所组成的串构成。

(B) 该语言由至少包含一个 a 的串构成。

(C) 该语言中的每个串以一个 a 开头, 后跟任意数量的 a 和 b , 并以两个 b 结尾。

IV.3.19 Figure 89 on page 136 展示了结果表格。观察发现正则表达式 $\emptyset$ 从不匹配任何串。

IV.3.20 答案如 Figure 90 on page 137 所示。

IV.3.21 首先, $(0^*1)^*$ 匹配 01001, 因为我们可以把它分解成 $01001 = 01 \wedge 001$, 并且 01 和 001 都匹配 (0^*1) 。其次, $(0^*1)^*$ 匹配 00000101, 因为 $00000101 = 000001 \wedge 01$, 而且这两部分都匹配 (0^*1) 。因此, Kleene 星的含义更接近于“重复匹配括号内的模式”。

根据定义, 星号运算对应的语言是其内部语言做星号运算的结果: $\mathcal{L}(R^*) = (\mathcal{L}(R))^*$ 。后者的定义涉及分解: $\mathcal{L}(R)^* = \{\sigma_0 \wedge \dots \wedge \sigma_{k-1} \mid k \in \mathbb{N} \text{ 且 } \sigma_0, \dots, \sigma_{k-1} \in \mathcal{L}(R)\}$ 。

IV.3.22 正则表达式不是唯一的。例如, 以下是字母表 $\Sigma = \{a, b\}$ 上至少包含三个连续 a 的串的语言的两个正则表达式: $(a|b)^*aaa(a|b)^*$ 和 $(a|b)^*aaaaa^*(a|b)^*$ 。

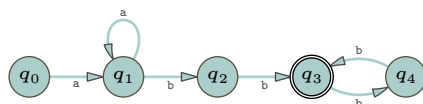
	0*1	1*0	ε	0(0 1)*	(100)(ε 1)0*
ε	F	F	F	T	F
0	F	T	F	F	T
1	T	F	F	F	F
00	F	F	F	F	T
01	T	F	F	F	T
10	F	T	F	F	F
11	F	F	F	F	F
000	F	F	F	F	T
001	T	F	F	F	T
010	F	F	F	F	T
011	F	F	F	F	T
100	F	F	F	F	F
101	F	F	F	F	F
110	F	T	F	F	F
111	F	F	F	F	F

图 90, FOR QUESTION IV.3.20: 串匹配测试。

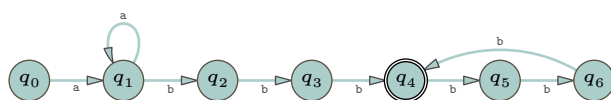
IV.3.23 一个例子是 $(0|1)^*111(0|1)^*$ 。

IV.3.24 对于每种语言, 有很多组可能的五个串作为答案。我们在此列出多组可能中的一组。

(A) 以下是 $\mathcal{L}_0$ 的五个成员: abb 、 $aabb$ 、 ab^4 、 aab^4 、 ab^6 。以下是五个不属于的串: ε 、 a 、 b 、 ab 、 $abba$ 、 ba 。一个正则表达式是 $aa*bb(bb)^*$ 。这是一个 (非确定性) 有限状态机, 它接受这个语言。



(B) 以下是 $\mathcal{L}_1$ 的五个成员: $abbb$ 、 $aabbb$ 、 ab^6 、 aab^6 、 ab^9 。五个不属于的串是: ε 、 a 、 b 、 $abbbb$ 、 $abbbba$ 、 ba 。一个正则表达式是 $aa*bbb(bbb)^*$ 。这是一个接受 $\mathcal{L}_1$ 的非确定性有限状态机。



IV.3.25 $((a|b)^*)|((a|c)^*)|((b|c)^*)$

IV.3.26

(A) $b(a|b)^*$

(B) $(a|b)^*a(a|b)$

(C) 两个字符出现的顺序可能是 $\sigma = \dots a \dots b \dots$ 或 $\sigma = \dots a \dots b \dots$ 。所以以下表达式可行: $(a|b)^*((a(a|b)^*b)|(b(a|b)^*a))$

(D) a 必须三个一组地出现: $(b*ab*ab*a)*b*$ 。

IV.3.27

(A) $aba(a|b)^*$

(B) $(a|b)^*aba$

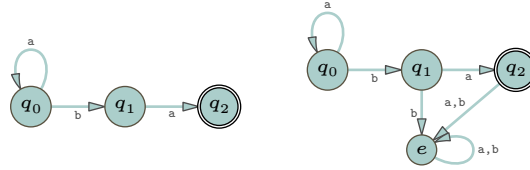
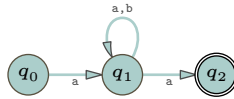
(C) $(a|b)^*aba(a|b)^*$

IV.3.28

(A) 必须至少有一个 1, 并且额外的 1 必须成对出现: $(0*10*1)^*0*10^*$ 。

(B) 正则表达式 $(\varepsilon|1)(01)^*(\varepsilon|0)$ 是可行的。注意它匹配空串, 同时也匹配串 0 和串 1。

(C) 二进制下, 八的倍数以三个 0 结尾: $(0|1)^*000$ 。该表达式匹配带有前导 0 的二进制数, 但不匹配 0。如果你不喜欢这样, 可以改用 $(1(0|1)^*000)|0$ 。

图 91, FOR QUESTION IV.3.33: 接受语言 a^*ba 的有限状态机。图 92, FOR QUESTION IV.3.33: 接受语言 $ab^*(a|b)^*a$ 的有限状态机, 该语言与 $a(a|b)^*a$ 描述的相同。**IV.3.29**

- (A) 一个答案是: $((ba)^*bb^*)^*$ 。注意它匹配空串, 并且匹配 **babab**、**bbbbab** 以及 **b**。
 (B) $(a|b)^*((bba^*bb)|(bbb))(a|b)^*$

IV.3.30

- (A) 这里的 0 成对出现: $(1^*01^*01^*)^*1^*$ 。注意它匹配 ϵ 以及 1。
 (B) 以下是三个 1, 且不使用 Kleene 星号: $0^*10^*10^*1(0|1)^*$ 。
 (C) 我们可以从上一项的表达式出发, 对于其中的 0, 用第一项的表达式替换。

$$(((1^*01^*01^*)^*1^*)^*1^*((1^*01^*01^*)^*1^*)^*1^*((1^*01^*01^*)^*1^*)^*1^*((1^*01^*01^*)^*1^*)^*|1)^*$$

IV.3.31

- (A) $(a(a|b)^*a)|(b(a|b)^*b)$
 (B) $((aaa)^*b(aaa)^*)|(a(aaa)^*ba(aaa)^*)|(aa(aaa)^*baa(aaa)^*)$

IV.3.32

- (A) 这要求该数以 1 开头: $1(0|1)^*0$ 。为了也允许数字 0, 可以使用表达式 $(1(0|1)^*0)|0$ 。
 (B) $(1(0|1)^*0)|(0(1|0)^*1)$
 (C) $\epsilon|0|1|(0(10)^*(1|\epsilon))|(1(01)^*(0|\epsilon))$

IV.3.33

- (A) Figure 91 on page 138 左侧是一个非确定性有限状态机。如果你需要确定性的机器, 请使用右侧的那个。
 (B) 一个非确定性机器见 Figure 92 on page 138。注意 $b^*(a|b)^*$ 化简为 $(a|b)^*$, 所以机器处理的就是这个。我们已经给出了将非确定性有限状态机转换为确定性有限状态机的方法, 所以如果你需要确定性的机器, 可以将上述非确定性机器视为一种简写。

IV.3.34 我们将构造可达的状态集合, 则不可达的状态就是其余的状态。

- (A) 可达的状态集合是 $S_2 = S_3 = \dots = \{q_0, q_1, q_2\}$ 。因此, 不可达的状态集合是 $\{q_3\}$ 。

i	S_i
0	$\{q_0\}$
1	$\{q_0, q_1\}$
2	$\{q_0, q_1, q_2\}$
3	$\{q_0, q_1, q_2\}$

- (B) 可达的状态集合是 $S_2 = S_3 = \dots = \{q_0, q_1, q_3\}$ 。因此, 不可达的状态集合是 $\{q_2, q_4\}$ 。

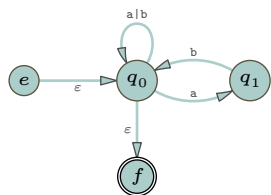
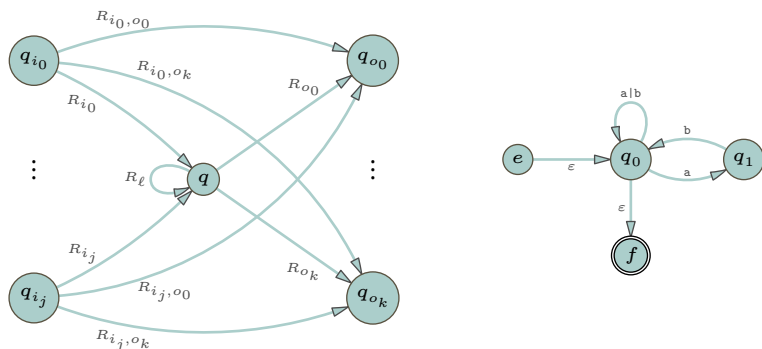
图 93, FOR QUESTION IV.3.35: $a(b|c)$ 的推导与语法分析树。

i	S_i
0	$\{q_0\}$
1	$\{q_0, q_1\}$
2	$\{q_0, q_1, q_3\}$
3	$\{q_0, q_1, q_3\}$

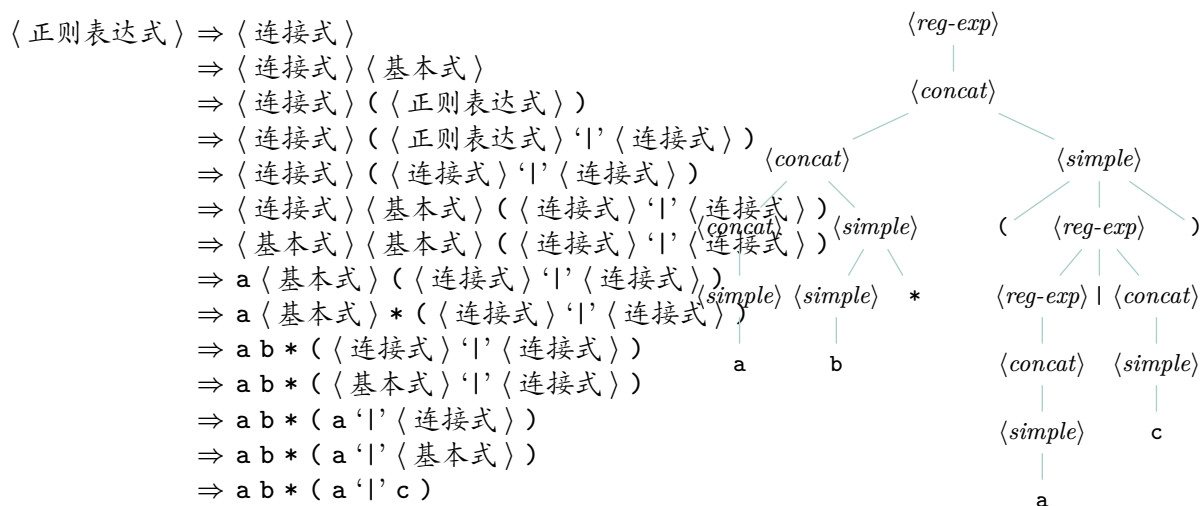
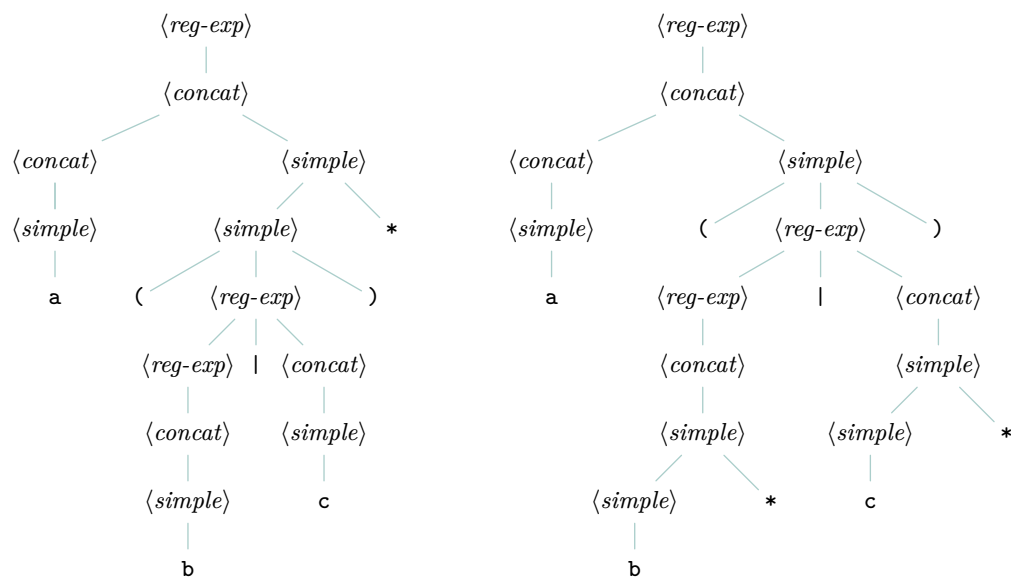
IV.3.35

(A) Figure 93 on page 139 包含了推导过程与语法分析树。

(B) Figure 94 on page 140 展示了推导过程与语法分析树。

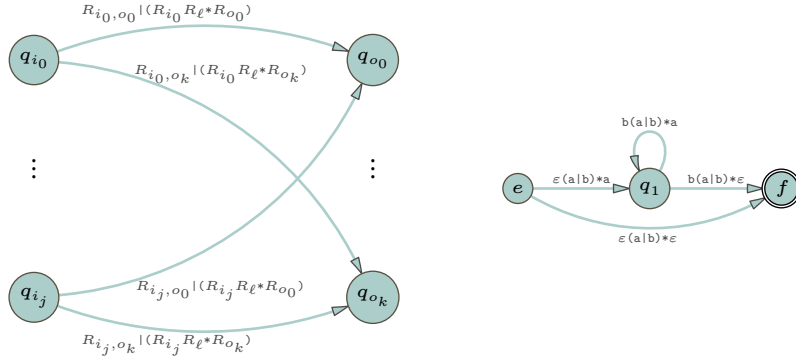
IV.3.36 Figure 95 on page 140 展示了这两棵语法分析树。**IV.3.37**(A) 这就是 $\hat{\mathcal{M}}$ 。(B) 为了方便, 这里先给出引理 3.14 证明中的前半部分图表, 以及 $\hat{\mathcal{M}}$ 。

状态 q_0 有两个状态有箭头指向它, 有两个状态从它接收箭头, 此外还有一个自环。因此我们得到图中符号的如下对应关系。

图 94, FOR QUESTION IV.3.35: $ab*(a|c)$ 的推导与语法分析树。图 95, FOR QUESTION IV.3.36: $a(b|c)^*$ 与 $a(b^*|c^*)$ 的语法分析树。

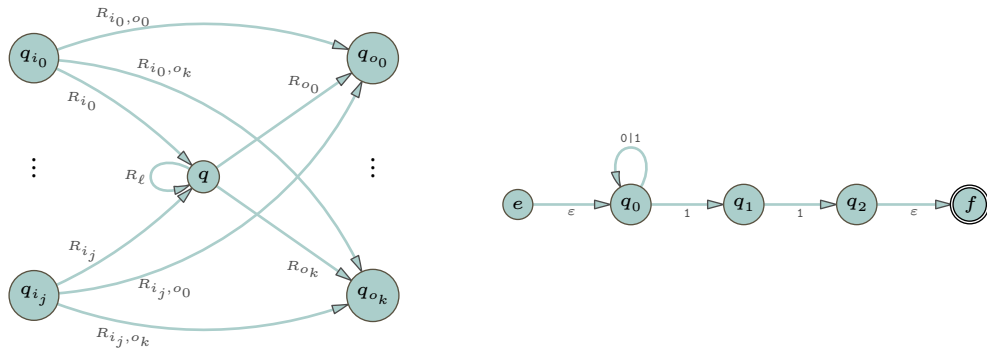
q	q_{i_0}	q_{i_1}	q_{o_0}	q_{o_1}	R_{i_0}	R_{i_1}	R_{i_0,o_0}	R_{i_0,o_1}	R_{i_1,o_0}	R_{i_1,o_1}	R_ℓ	R_{o_0}	R_{o_1}
q_0	e	q_1	q_1	f	ε	b	$-$	$-$	$-$	$-$	$a b$	a	ε

(c) 这是引理 3.14 证明中的后半部分图表，以及消除 q_0 后的机器 $\hat{\mathcal{M}}$ 。



IV.3.38

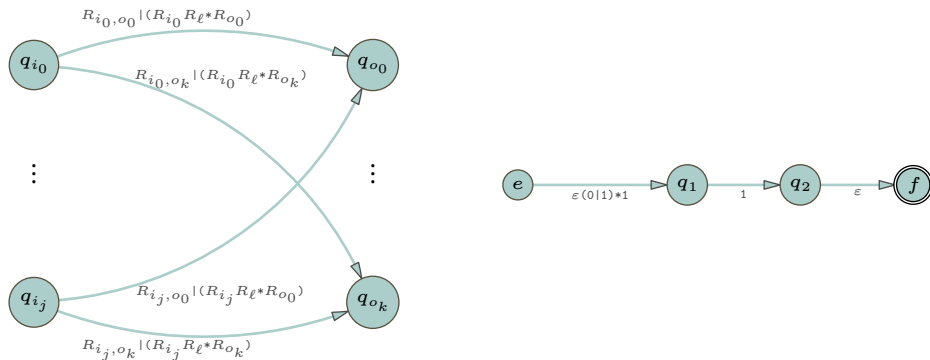
(A) 这是 $\hat{\mathcal{M}}$ 以及引理 3.14 证明中图示的前半部分。



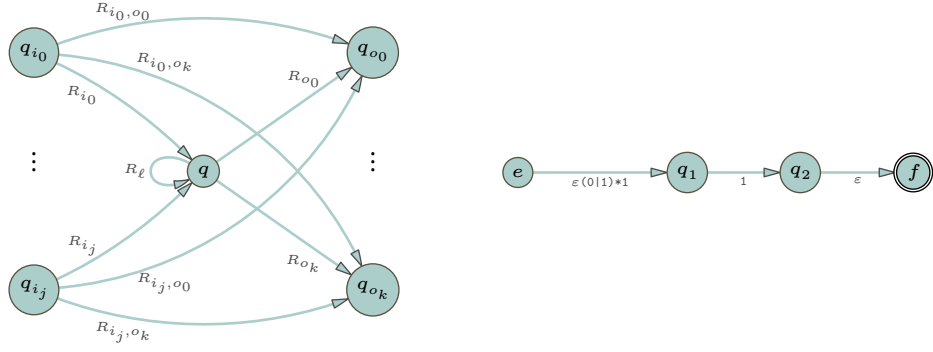
状态 q_0 有一个状态向它发出箭头，还有一个状态接收从它发出的箭头。因此我们得到图示记号的如下转译。

q	q_{i_0}	q_{o_0}	R_{i_0}	R_{i_0,o_0}	R_ℓ	R_{o_0}
q_0	e	q_1	ε	$-$	$0 1$	1

这是引理 3.14 证明中图示的后半部分，以及消除 q_0 后的机器 $\hat{\mathcal{M}}$ 。



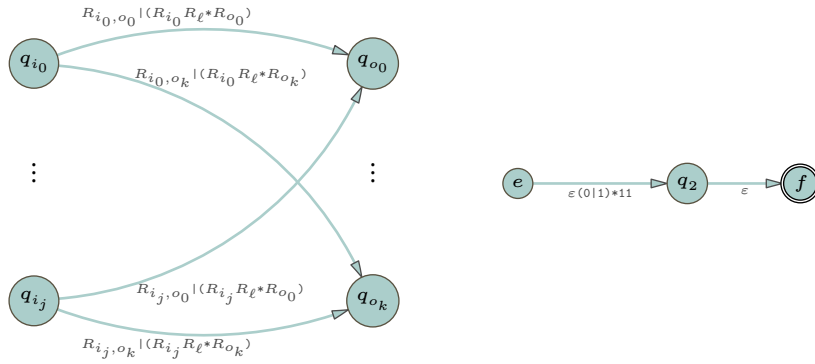
(B) 这是上一答案中的机器，以及引理 3.14 证明中图示的前半部分。



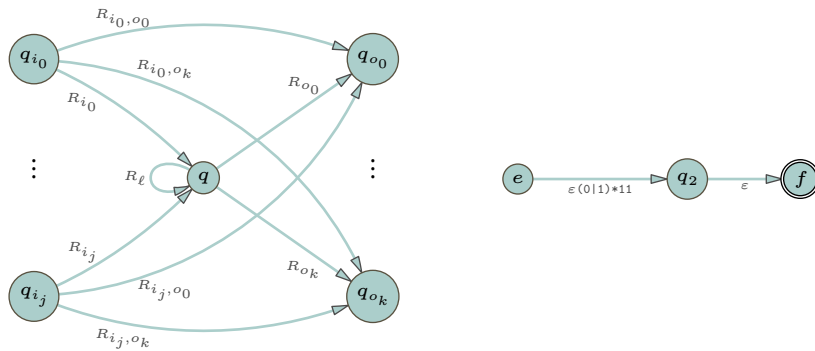
状态 q_1 有一个状态向它发出箭头，还有一个状态接收从它发出的箭头。因此我们得到图示记号的如下转译。

$$\begin{array}{c|c} \begin{array}{ccc} q & q_{i_0} & q_{o_0} \\ \hline q_1 & e & q_2 \end{array} & \begin{array}{cccc} R_{i_0} & R_{i_0, o_0} & R_{\ell} & R_{o_0} \\ \hline \varepsilon(0|1)*1 & - & - & 1 \end{array} \end{array}$$

这是引理 3.14 证明中图示的后半部分，以及消除 q_1 后的机器 $\hat{\mathcal{M}}$ 。



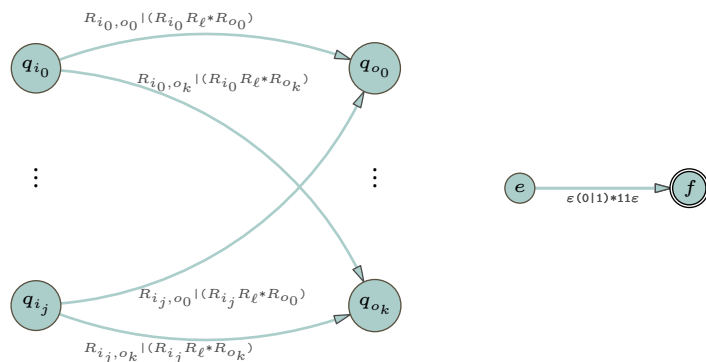
(c) 这是上一答案中的机器，以及图示的前半部分。



状态 q_2 有一个状态向它发出箭头，还有一个状态接收从它发出的箭头。因此我们得到图示记号的如下转译。

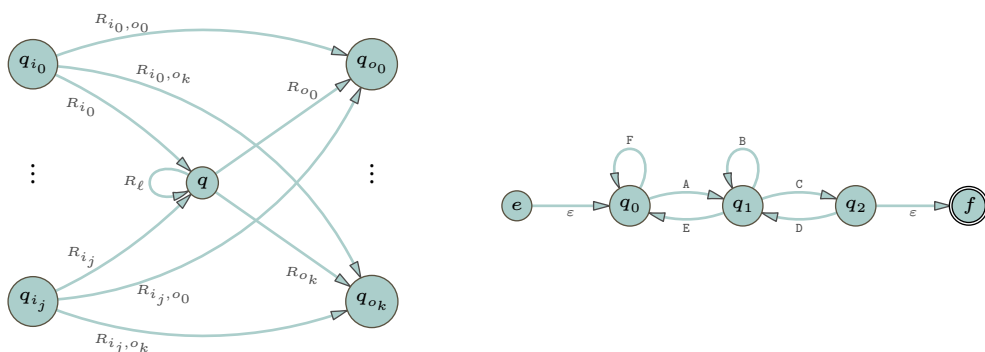
$$\begin{array}{c|c} \begin{array}{ccc} q & q_{i_0} & q_{o_0} \\ \hline q_2 & e & f \end{array} & \begin{array}{cccc} R_{i_0} & R_{i_0, o_0} & R_{\ell} & R_{o_0} \\ \hline \varepsilon(0|1)*11 & - & - & \varepsilon \end{array} \end{array}$$

这是图示的后半部分，以及消除 q_2 后的机器。



(D) 正则表达式是 $\varepsilon(0|1)^*11\varepsilon$ 。

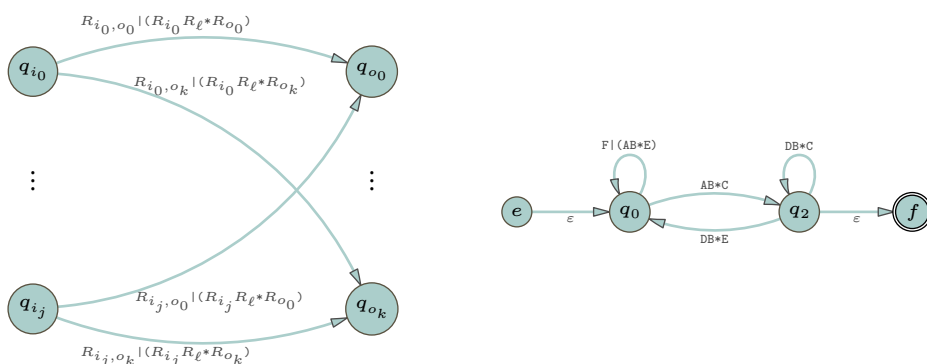
IV.3.39 这是 $\hat{\mathcal{M}}$ 以及引理 3.14 证明中图示的前半部分，以供参考。



状态 q_1 有两个状态向它发出箭头，并且有两个状态接收从它发出的箭头。因此，我们得到图示记号的如下转译。

q	q_{i_0}	q_{i_1}	q_{o_0}	q_{o_1}	R_{i_0}	R_{i_1}	R_{i_0, o_0}	R_{i_0, o_1}	R_{i_1, o_0}	R_{i_1, o_1}	R_ℓ	R_{o_0}	R_{o_1}
q_1	q_0	q_2	q_0	q_2	A	D	F	—	—	—	B	E	C

以下是图示的后半部分，以及消除 q_1 后的机器 $\hat{\mathcal{M}}$ 。



IV.3.40 第一个可以被同一台 k 状态机识别，只需将接受状态变为非接受状态，非接受状态变为接受状态。第二个在 $\mathcal{L} = \emptyset$ 且 $k = 1$ 时不能被 k 状态机识别。

IV.3.41 我们可以利用 Kleene 定理来处理这两项。

- (A) 对机器 $\mathcal{M}$ 进行变换，得到新机器 $\hat{\mathcal{M}}$ ，使其接受状态集为 S 。
- (B) 将机器 $\mathcal{M}$ 变换为 $\hat{\mathcal{M}}$ ，使其初始状态为给定的第一个单一状态，同时使上一项中的集合 S 仅包含结束状态。然后应用上一项给出的论证。

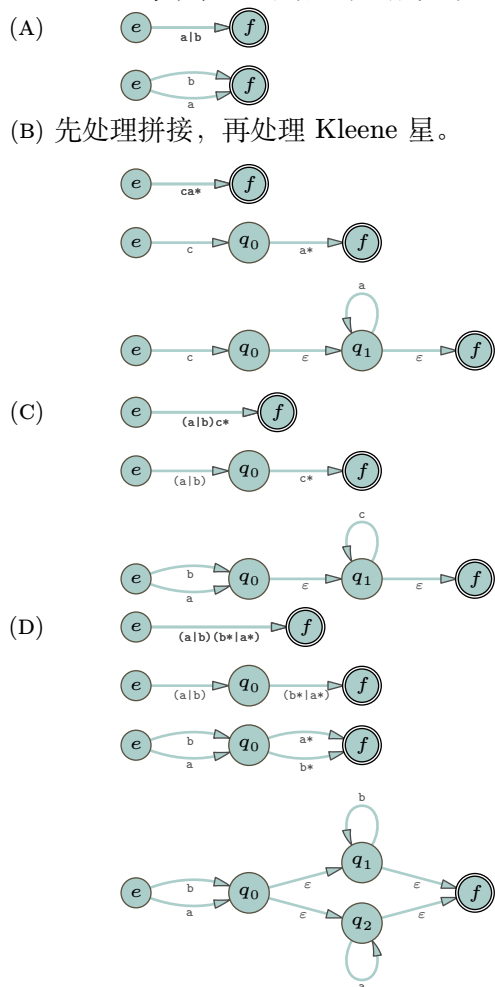
IV.3.42 我们将分两部分来证明：第一部分证明这个语言集合是无限的，第二部分证明它至多可数。请记住，如附录 A 所定义，字母表是非空且有限的。

首先证明这样的语言有无限多个。令 x 是 Σ 中的一个元素。单元素语言 $\mathcal{L}_1 = \{x\}$ 由只有一个字符的正则表达式 $R_1 = x$ 描述。单元素语言 $\mathcal{L}_2 = \{xx\}$ 由有两个字符的正则表达式 $R_2 = xx$ 描述。一般地, $\mathcal{L}_k$ 是一个单元素语言, 它与所有 $j \neq k$ 的 $\mathcal{L}_j$ 都不同, 并且由一个包含 k 个字符的正则表达式 R_k 描述。这就产生了无限多个由 Σ 上的正则表达式描述的不同语言。

接下来证明这个语言集合是可数的。每个正则表达式只对应一个语言, 因此, 如果我们能证明正则表达式的数量是可数的, 那么我们就完成了证明。正则表达式是 $\Sigma \cup \{ \}, (, |, * \}$ 上的一个串, 而这是一个有限集合。因此, 对于任意 $n \in \mathbb{N}$, 长度为 n 的正则表达式只有有限多个。第二章的推论 2.13 表明: 可数个可数集的并集是可数的。因为有限集是可数的, 所以对所有 n 取长度为 n 的正则表达式集合的并集, 结果是可数的。

我们通过证明 Σ 上所有语言的集合, 即所有 $\mathcal{L} \subseteq \Sigma^*$ 构成的集合, 是不可数的, 来完成整个证明。首先, Σ^* 是可数无限的, 因为长度为 0 的 $\sigma \in \Sigma^*$ 有可数个, 长度为 1 的也有可数个, 等等。所有语言 $\mathcal{L} \subseteq \Sigma^*$ 的集合是 Σ^* 的幂集, 因此其基数大于 Σ^* 的基数, 这使它成为不可数集。所以, 存在不能被任何正则表达式描述的语言。

IV.3.43 每个表达式的步骤进展如下。



IV.3.44

- (A) 因为它接受输入 ε , 初始状态 q_0 必须是一个接受状态。如果 q_0 是唯一的接受状态, 那么该机器会由于存在从 q_0 到自身的循环而接受 a , 从而也会接受 aa 。但 aa 不在语言 $\mathcal{L}_1$ 中, 因此必须至少还有一个其他接受状态。
- (B) 因为它接受输入 ε , 初始状态 q_0 必须是一个接受状态。如果 q_0 是唯一的接受状态, 那么该机器会由于存在从 q_0 到自身的循环而接受 a , 从而也会接受 aaa 。该串不是语言 $\mathcal{L}_2$ 的成员, 所以必须至少有一个不同于 q_0 的接受状态。

如果机器通过处于状态 q_0 来接受 aa ，那就意味着存在循环，从而该机器也会接受串 $aaaa$ 。但该串不在语言 $\mathcal{L}_2$ 中，因此在此输入上的扩展转移函数 $\hat{E}(aa)$ 不等于状态 q_0 。类似地，如果 $\hat{E}(aa)$ 等于 $\hat{E}(a)$ ，那么在该状态上输入 a 会形成一个自环，从而机器也会接受 aaa 。但该机器不接受 aaa ，因为该串不在语言 $\mathcal{L}_2$ 中，所以必须有第三个接受状态。

- (c) 这是前两小题的自然推广。考虑 $\mathcal{L}_n = \{\varepsilon, a, \dots, a^n\}$ 。对于任意 $i < n$ ，如果 $\hat{E}(a^n)$ 等于 $\hat{E}(a^i)$ ，那么就会存在循环，从而机器会接受额外的串。

IV.4.7 正则语言在交集下封闭，并不意味着非正则语言在交集下也封闭。这里有一个反例。引理 4.2 说明，在字母表 $\Sigma_0 = \{a, b\}$ 上存在一个非正则的语言 $\mathcal{L}_0$ 。类似地，在字母表 $\Sigma_1 = \{c, d\}$ 上也存在一个非正则的语言 $\mathcal{L}_2$ 。它们的交集是空集，因为两个字母表不共享任何字符，而空语言是正则的（我们可以将其视为字母表 $\Sigma_0 \cup \Sigma_1$ 上的一个语言）。

IV.4.8 这不是一个良定义的问题。

原则上，我们可以取 Σ 为某个固定词典中的所有单词，并且可以断言没有英语句子的长度超过一百万个单词（参见 (Wikipedia contributors 2020)）。那么，我们可能会声称英语是正则语言，因为我们可以列出所有我们认为是合法的句子，而这样的句子是有限的。然而，这似乎并不令人满意。

否则，我们必须选择一个文法。但是自然语言的边界并不明确：例如，“My god, it is her.” 是有效的吗？尽管有些人可能不接受，说它应该是 “My god, it is she.”？（另见 <https://cs.stackexchange.com/q/116127/50343>。）

IV.4.9 在某个固定字母表上的有限语言。

另一个答案是：若存在 Turing 机 $\mathcal{P}$ ，使得当 $\sigma \in \mathcal{L}$ 时，以 σ 作为纸带内容启动 $\mathcal{P}$ 后它会停机，而当 $\sigma \notin \mathcal{L}$ 时它不停机，则称语言 $\mathcal{L}$ 是可计算枚举的。不难看出，可计算枚举的语言类在交与并运算下封闭，但在补运算下不封闭。

IV.4.10

- (A) 假，空语言可以被任何没有接受状态的有限状态机识别。它也可以用正则表达式 $\emptyset$ 描述。
 (B) 假。正确的表述是：两个正则语言的交集是正则的。例如，对于非正则语言 $\mathcal{L}$ ，交集 $\mathcal{L} \cap \mathbb{B}^* = \mathcal{L}$ 不是正则的。
 (C) 假。语言 $\mathbb{B}^*$ 是任何一个所有状态均为接受状态的有限状态机所识别的串的集合。它也可以用正则表达式 $(0|1)^*$ 描述。
 (D) 假。无限语言 $\{ab, abb, \dots\}$ 可以用正则表达式 ab^* 描述，且任意一对串的第二个位置上的字符都是 a 。

IV.4.11 第一句是真的，第二句是假的。

- (A) 如果该语言由串 $\sigma_0, \dots, \sigma_{n-1}$ 组成，那么一个正则表达式是 $\sigma_0 | \dots | \sigma_{n-1}$ 。如果该语言为空，则其正则表达式为 $\emptyset$ 。
 (B) 字母表 $\Sigma = \{a, b\}$ 上由正则表达式 a^* 描述的语言是 $\{\varepsilon, a, aa, \dots\}$ ，并且它是无限的。

IV.4.12 是的。用二进制表示时，2 的幂是由以 1 开头、后面跟着若干个 0 的比特串表示的。因此，正则表达式 10^* 就描述了该语言。（如果允许开头的 0，那么正则表达式是 0^*10^* 。）

IV.4.13 证明的一种方法是给出每个语言的正则表达式。

- (A) $a(a|b)^*a$
 (B) $b^*(ab^*a)^*b^*$

IV.4.14 要么该集合是 $S_0 = \{9^n \mid n < N\}$ （其中 $N \in \mathbb{N}$ ），要么是 $S_1 = \{9^n \mid n \in \mathbb{N}\}$ 。两者都是正则的。

IV.4.15 因为该语言是正则的，所以存在一个描述它的正则表达式 R 。那么正则表达式 $1R$ 就描述了 $\hat{\mathcal{L}}$ 。

IV.4.16 集合 $Q_0 \times Q_1$ 的大小是 $n_0 \cdot n_1$ 。

IV.4.17 这两台机器的转移表如下。

Δ_0	a	b	Δ_1	a	b
+ q_0	q_0	q_1	+ s_0	s_1	s_0
+ q_1	q_2	q_1	+ s_1	s_0	s_0
q_2	q_2	q_0			

下面是没有指定接受状态的积机器（的转移表）。

Δ	a	b
(q_0, s_0)	(q_0, s_1)	(q_1, s_0)
(q_0, s_1)	(q_0, s_0)	(q_1, s_0)
(q_1, s_0)	(q_2, s_1)	(q_1, s_0)
(q_1, s_1)	(q_2, s_0)	(q_1, s_0)
(q_2, s_0)	(q_2, s_1)	(q_0, s_0)
(q_2, s_1)	(q_2, s_0)	(q_0, s_0)

(A) 要使这台机器接受两个语言的交集，需选取 $F = \{(q_0, s_1), (q_1, s_1)\}$ ，也就是说，当且仅当 q_i 和 s_j 都是接受状态时， (q_i, s_j) 才是接受状态。

(B) 取 (q_i, s_j) 为接受状态当且仅当 q_i 或 s_j 中至少有一个是接受状态，因此需选取 $F = \{(q_0, s_0), (q_0, s_1), (q_1, s_0), (q_1, s_1)\}$ 。

IV.4.18 这是 $\mathcal{M}$ 的转移表。

Δ_0	a	b
s_0	s_0	s_1
+ s_1	s_1	s_0

此机器与自身的积有四个状态。

Δ_1	a	b
(s_0, s_0)	(s_0, s_0)	(s_1, s_1)
(s_0, s_1)	(s_0, s_1)	(s_1, s_0)
(s_1, s_0)	(s_1, s_0)	(s_0, s_1)
+ (s_1, s_1)	(s_1, s_1)	(s_0, s_0)

如上所示，接受状态集合按交集情形定义，以 $F_\cap = \{(s_1, s_1)\}$ 。我们也可以改用并集对应的集合 $F_\cup = \{(s_1, s_1), (s_0, s_1), (s_1, s_0)\}$ （尽管另外两个状态是不可达的）。

IV.4.19 如下左侧的机器接受包含至少两个 0 的串 $\sigma \in \mathbb{B}^*$ 。右侧机器接受包含偶数个 1 的串。

Δ_0	0	1
q_0	q_1	q_0
q_1	q_2	q_1
+ q_2	q_2	q_2

Δ_1	0	1
+ s_0	s_0	s_1
s_1	s_1	s_0

积构造给出如下结果。接受状态集合的设置对应两个语言的交集。

Δ	0	1
(q_0, s_0)	(q_1, s_0)	(q_0, s_1)
(q_0, s_1)	(q_1, s_1)	(q_0, s_0)
(q_1, s_0)	(q_2, s_0)	(q_1, s_1)
(q_1, s_1)	(q_2, s_1)	(q_1, s_0)
+ (q_2, s_0)	(q_2, s_0)	(q_2, s_1)
(q_2, s_1)	(q_2, s_1)	(q_2, s_0)

IV.4.20

(A) 真。每个语言都是语言 Σ^* 的子集。

(B) 假。对于任何字母表，只有一个元素的语言是正则的，并且它唯一的子集是空语言，空语言也是正则的。

(C) 假。取 $\mathcal{L}_0$ 为非正则，取 $\mathcal{L}_1$ 为空语言 $\mathcal{L}_1 = \{\}$ ，它是正则的。它们的并集是 $\mathcal{L}_0$ 。

(D) 真。这是引理 4.3 的一部分。

IV.4.21 答案是 (B): 它可能是正则的, 也可能不是正则的。固定一个字母表 Σ 和 Σ 上的一个非正则语言 $\mathcal{L}$ 。正则语言与非正则语言拼接得到正则结果的一个例子是 $\Sigma^* \cap \mathcal{L} = \Sigma^*$ 。得到非正则结果的一个拼接例子是 $\{s\} \cap \mathcal{L}$, 其中 $s \in \Sigma$ (一个简单的论证表明, 如果这个语言正则, 则 $\mathcal{L}$ 正则)。

IV.4.22 两者皆假。同一个例子对两者都适用。对于 $\mathcal{L}_0$, 使用所有比特串的集合 $\mathbb{B}^*$ 。对于 $\mathcal{L}_1$, 使用它的任意一个非正则子集 (其存在性由引理 4.2 保证)。

IV.4.23 根据引理 4.3, 正则语言在 Kleene 星和拼接运算下是封闭的。

IV.4.24 令 Σ 为 $\mathcal{L}$ 的字母表。 Σ 上所有偶数长度串的集合是正则的, 因为显然存在一个只接受这些串的有限状态机 (或者, 一个正则表达式是 $((s_0 s_0) | (s_0 s_1) | \dots | (s_n s_n))^*$, 其中 $\Sigma = \{s_0, \dots, s_n\}$)。 $\mathcal{L}$ 与此集合的交集是两个正则语言的交集, 因此它是正则的。

IV.4.25 令已知为非正则的集合 $\{a^n b^n \in \{a, b\}^* \mid n \in \mathbb{N}\}$ 为 $\mathcal{L}_0$ 。令由正则表达式 $a^* b^*$ 描述的语言为 $\mathcal{L}_1$ 。令语言 $\{\sigma \in \{a, b\}^* \mid \sigma \text{ 包含相同数量的 } a \text{ 和 } b\}$ 为 $\mathcal{L}_2$ 。如果 $\mathcal{L}_2$ were regular then because the intersection of regular languages is regular, $\mathcal{L}_2 \cap \mathcal{L}_1$ 将是正则的。但该集合正是 $\mathcal{L}_0$, 而它是非正则的。

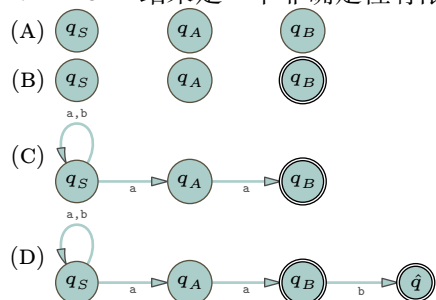
IV.4.26 如果存在一个有限状态机 $\mathcal{M}$ 能够接受这个语言, 那么我们就用 Turing 机来解决停机问题。这是因为我们显然可以将该有限状态机编写为子程序。然后, 给定 $e \in \mathbb{N}$, 将 1^e 输入该子程序。如果它接受, 则 $e \in K$; 否则 $e \notin K$ 。

IV.4.27

- (A) 对于每一个字 $\sigma \in \Sigma^*$, 单字语言 $\mathcal{L} = \{\sigma\}$ 是正则的。但是, 任意一个非正则语言都可以表示为若干这种语言的并集。
- (B) 类似地, 对于每一个字 $\sigma \in \Sigma^*$, 语言 $\mathcal{L} = \Sigma^* - \{\sigma\}$ 是正则的。任意一个非正则语言 $\mathcal{L}$ 都可以表示为若干这种语言的交集, 具体来说, 就是对所有 $\sigma \notin \mathcal{L}$ 所对应的语言取交集。

IV.4.28 所有串构成的语言 $\mathcal{L} = \mathbb{B}^*$ 是正则的, 并且是不可数的。

IV.4.29 结果是一个非确定性有限状态机。



IV.4.30 为了证明封闭性, 我们必须证明, 例如, 如果 $\mathcal{L}$ 是正则的, 那么 $\text{pref}(\mathcal{L})$ 也是正则的。

- (A) 固定一个识别 $\mathcal{L}$ 的有限状态机 $\mathcal{M}$ 。将其转换为新机器 $\hat{\mathcal{M}}$, 它具有相同的状态和转移, 但其中接受状态更多。具体来说, 找出 $\mathcal{M}$ 中所有这样的状态 $q \in \mathcal{M}$: 从 q 出发可以到达 $\mathcal{M}$ 的某个接受状态。(也就是说, 存在一系列字符转移, 能将 $\mathcal{M}$ 从 q 带到 $\mathcal{M}$ 的某个接受状态。)在 $\hat{\mathcal{M}}$ 中, 将每一个这样的 q 设为接受状态。

这个 $\hat{\mathcal{M}}$ 识别 $\text{pref}(\mathcal{L})$ 。这是因为, σ 是某个 $\alpha \in \mathcal{L}$ 的前缀, 当且仅当它将它机器 $\mathcal{M}$ 从其初始状态 q_0 带到某个状态 q , 而从 q 出发, 继续处理 τ 将使 $\sigma \cap \tau = \alpha$ 被接受。

- (B) 同样, 固定一个识别 $\mathcal{L}$ 的有限状态机 $\mathcal{M}$ 。我们同样从相同的状态和转移出发, 将其转换为新机器 $\hat{\mathcal{M}}$ 。

设 S 是 $\mathcal{M}$ 中从 q_0 可达的状态集合。引入一个新状态 r , 并定义从 r 到 S 中每个状态的 ϵ 转移, 由此得到 $\hat{\mathcal{M}}$ 。这个新状态 r 就是 $\hat{\mathcal{M}}$ 的初始状态。那么 $\hat{\mathcal{M}}$ 就识别 $\text{suff}(\mathcal{L})$ 。

- (C) 和前面一样, 固定一个识别 $\mathcal{L}$ 的有限状态机 $\mathcal{M}$ 。我们从相同的状态和转移出发, 将其转换为新机器 $\hat{\mathcal{M}}$ 。

向新机器中引入一个陷阱状态 e , 它不是接受状态, 并且任何输入都导致它循环回到 e 。取 $\mathcal{M}$ 中的每一个非接受状态 q , 并用这个陷阱状态替换它。也就是说, 所有进入 q 的转移现在都指向 e , 并且我们略去 q 及其所有向外的转移。

这个机器接受一个 $\mathcal{L}$ 中的串, 当且仅当它也接受该串的所有前缀。

IV.4.31

- (A) 引理 4.3 表明正则语言的集合在并运算下封闭。
 (B) 这个恒等式表明, 证明了在补运算下封闭也就证明了在差运算下封闭。
 (C) 固定某个字母表 Σ 上的正则语言 $\mathcal{L}$ 。它被某个输入字母表为 Σ 的确定性的有限状态机 $\mathcal{M}$ 所识别。

定义一个新机器 $\hat{\mathcal{M}}$, 它的状态集和转移函数与 $\mathcal{M}$ 相同, 但其接受状态是 $\mathcal{M}$ 接受状态的补集, 即 $F_{\hat{\mathcal{M}}} = Q_{\mathcal{M}} - F_{\mathcal{M}}$ 。我们将证明这台机器的语言是 $\Sigma^* - \mathcal{L}$ 。

因为 $\mathcal{M}$ 是确定性的, 每个输入串 τ 都唯一对应一个最终所处的状态, 即机器消耗 τ 的最后一个字符后所进入的状态, 记为 $\hat{\Delta}(\tau)$ 。注意到, $\hat{\Delta}(\tau) \in F_{\hat{\mathcal{M}}}$ 当且仅当 $\hat{\Delta}(\tau) \notin F_{\mathcal{M}}$ 。因此, $\hat{\mathcal{M}}$ 的语言就是 $\mathcal{M}$ 的语言的补集; 既然这个语言能被一台有限状态机识别, 它也是正则的。

IV.4.32 先证一个方向, 假设机器接受至少一个长度为 k 的串 σ , 其中 $n \leq k < 2n$ 。在处理 σ 的过程中, 机器必须经过至少 n 次转移。但机器只有 n 个状态, 从 q_0 出发并恰好访问其他所有状态一次也只需要 $n-1$ 次转移。因此, 由鸽巢原理, 在处理串 σ 时, 机器会两次访问某个状态 q 。也就是说, 机器在处理过程中形成了一个循环。

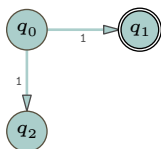
换句话说, 被接受的串 σ 可以分解为 $\sigma = \alpha \wedge \beta \wedge \gamma$, 其中处理 α 部分将机器从初始状态带到循环入口 (从 q_0 到 q), β 部分使机器沿一个非平凡的循环绕行一周 (即 $\beta \neq \varepsilon$ 将机器从 q 带回到 q), 而 γ 部分将机器带到某个接受状态。这样, 显然机器也接受 $\alpha \wedge \beta \wedge \beta \wedge \gamma$ 等无穷多个串, 所以被识别的语言是无限的。(注意, 我们可以取 β 的长度小于 n , 否则机器的大小将意味着循环内部存在子循环。)

反过来, 假设被接受的串的语言是无限的。那么其中至少有一个串的长度大于 n ; 固定一个满足此条件的最短串。如上段所述, 因为机器只有 n 个状态, 由鸽巢原理, 被接受的串 σ 可分解为 $\sigma = \alpha \wedge \beta \wedge \gamma$, 其中 β 部分是一个非平凡的循环, 即处理 α 将机器从 q_0 带到某个 q , 然后处理 $\beta \neq \varepsilon$ 将机器从 q 带回到 q , 最后处理 γ 部分将机器带到某个接受状态。由于 σ 是满足上述条件的最短串, 有 $|\beta| < n$ 。于是总长度 $|\sigma| < 2n$ 。

IV.4.33

- (A) 对于第一个, $\hat{h}(01) = \hat{h}(0 \wedge 1) = \hat{h}(0) \wedge \hat{h}(1) = \mathbf{aba}$ 。类似地, $\hat{h}(10) = \mathbf{baa}$ 与 $\hat{h}(101) = \mathbf{baaba}$ 。
 (B) 描述 $\hat{\mathcal{L}} = \{\sigma \wedge 1 \mid \sigma \in \mathbb{B}^*\}$ 的正则表达式是 $(0|1)^*1$ 。描述 $\hat{h}(\hat{\mathcal{L}})$ 的正则表达式是 $(\mathbf{a|ba})^*\mathbf{ba}$ 。
 (C) 设 $\mathcal{L}$ 由正则表达式 R 描述。将函数 h 应用到 R 中的每个非元符号上。结果是一个描述 $h(\mathcal{L})$ 的正则表达式。

IV.4.34 对于这台机器, 其语言包含 1。



如果我们对接受状态集取补集, 那么所得机器的语言也包含 1。

IV.4.35

- (A) 固定 σ_0 。我们对 σ_1 的长度进行归纳来证明。对于基始, 考虑 $\sigma_1 = \varepsilon$ 和 $\sigma_1 = a$ (其中 $a \in \Sigma$) 两种情况。两个命题 $(\sigma_0 \wedge \varepsilon)^R = (\sigma_0)^R = (\varepsilon)^R \wedge (\sigma_0)^R$ 和 $(\sigma_0 \wedge a)^R = (a)^R \wedge (\sigma_0)^R$ 显然成立。

对于归纳步骤, 假设当 $|\sigma_1| = 0, 1, \dots, k$ 时命题对任意 σ_1 都成立, 考虑 $|\sigma_1| = k+1$ 的情况。此时 $\sigma_1 = \tau \wedge a$, 其中字符 $a \in \Sigma$, 串 $\tau \in \Sigma^*$ 的长度为 k 。那么 $(\sigma_0 \wedge \sigma_1)^R = (\sigma_0 \wedge \tau \wedge a)^R$ 。加上括号得到 $((\sigma_0 \wedge \tau) \wedge a)^R$, 并应用基始情况得到 $a^R \wedge (\sigma_0 \wedge \tau)^R$ 。归纳假设给出 $= a \wedge \tau^R \wedge \sigma_0^R$, 最后注意到这就是 $\sigma_1^R \wedge \sigma_0^R$ 。

- (B) 我们对正则表达式的长度进行归纳证明。没有长度为 0 的正则表达式, 因此基始情况是 $|R| = 1$ 。若 $R = \emptyset$, 则根据条件 (1), R^R 是正则表达式 $\emptyset$, 并且 $\mathcal{L}(R) = \mathcal{L}(R^R)$, 因为两者都等于空集, 符合要求。若 $R = \varepsilon$, 则根据条件 (2), $R^R = \varepsilon$, 且 $\mathcal{L}(R) = \{\varepsilon\} = \mathcal{L}(R^R)$, 同样符合要求。若 $R = x$, 其中 $x \in \Sigma$, 则根据条件 (3), $R^R = x$, 且 $\mathcal{L}(R) = \{x\} = \mathcal{L}(R^R)$ 。

现在进行归纳步骤。假设命题对长度为 $n = 1, 2, \dots, k$ 的正则表达式 R 成立, 考虑长度为 $n = k+1$ 的表达式。我们已经在本练习的第一小题中处理了它是拼接 $R = R_0 \wedge R_1$ 的情况。

若 $R = R_0 | R_1$, 则根据条件 (5), $\mathcal{L}(R^R) = \mathcal{L}((R_0 | R_1)^R) = \mathcal{L}(R_0^R | R_1^R)$ 。根据定义 3.3 后关于

正则表达式交替操作所对应语言的定义, 这就是 $\mathcal{L}(R_0^R) \cup \mathcal{L}(R_1^R)$ 。归纳假设给出它等于 $\mathcal{L}(R_0)^R \cup \mathcal{L}(R_1)^R$, 这等于 $\{\sigma_0^R \mid \sigma_0 \in \mathcal{L}(R_0)\} \cup \{\sigma_1^R \mid \sigma_1 \in \mathcal{L}(R_1)\}$, 再次根据定义 3.3 后的内容, 它等于 $\mathcal{L}(R)^R$ 。

最后一种情况类似。若 $R = R_0^*$, 则 $\mathcal{L}(R^R) = \mathcal{L}((R_0^R)^*)$ 。根据定义 3.3 后的定义, 它等于 $\{\sigma_0 \frown \dots \frown \sigma_m \mid \sigma_i \in \mathcal{L}(R_0^R)\}$ 。重写得到 $\{\tau_0^R \frown \dots \frown \tau_m^R \mid \tau_j \in \mathcal{L}(R_0)\}$ 。由本练习的第一小题, 它等于 $\{(\tau_m \frown \dots \frown \tau_0)^R \mid \tau_j \in \mathcal{L}(R_0)\}$, 即集合 $(\mathcal{L}(R_0)^*)^R$ 。

IV.5.8 没有这样的正则表达式。定义 3.3 中的语法没有能力表达这种构造。

如果我们固定, 比方说, $n = 2$, 那么我们可以构造一个正则表达式 $0^2 1^3$ 。我们可以使用交替运算来连接有限个这类表达式, 如 $0|011|0^2 1^3| \dots 0^9 1^{10}$ 所示。但没有正则表达式能描述语言 $\mathcal{L}$ 中所有的串。

IV.5.9 与 $\sigma = \alpha\beta\gamma$ 相比, 串 $\alpha\gamma$ 省略了 β 。根据定义, σ 的括号是平衡的。因为 β 由至少一个开括号组成, 并且没有闭括号, 所以省略 β 意味着在 $\alpha\gamma$ 中, 开括号的数量至少比闭括号的数量少一个。

IV.5.10 如果一个语言是有限的, $\mathcal{L} = \{\sigma_0, \dots, \sigma_n\}$, 那么通过令泵长度为 $p > \max(|\sigma_0|, \dots, |\sigma_n|)$, 泵引理的结论仍然成立。

IV.5.11 不对。第一个问题是, 这个说法不合理。可能有多个有限状态机识别同一个语言, 所以一个语言的“状态数”并不是良定义的。

即便我们退一步说, 他们是想固定某一台特定的机器, 比如这样说: “如果 $\mathcal{M}$ 接受一个语言, 并且该语言有一个串的长度大于 $\mathcal{M}$ 的状态数, 那么它不可能是正则语言”, 这仍然不合理, 因为如果存在一台有限状态机, 那么该语言当然是正则的。

就算我们绞尽脑汁去理解他们的意思, 认为他们是想说: “如果在论证中我们假设该语言被一台有限状态机识别, 然后我们接着证明该语言有一个串的长度大于那台机器的状态数, 那么我们就得到了想要的矛盾”, 那么, 这个说法虽然在技术上合理了, 但仍然是错的。例如, 存在一台有限状态机接受符合 a^* 描述的串, 而该语言当然有长度大于机器状态数的串。

IV.5.12 泵引理只允许我们得出 $\alpha\beta$ 由 $($ 组成, 而不是说这个子串包含了所有的 $($ 。子串 γ 确实包含了所有的闭括号, 即 $)$, 但我们知道的仅此而已——它也可能包含了一些开括号。

IV.5.13

- (A) (1) $a^p b c^p$, (2) 串 $\alpha \frown \beta$ 完全由 a 组成, (3) β 至少包含一个 a , (4) $\alpha\gamma$, (5) 至少少一个 a , 但 c 的数量相同。(对于 (4) 和 (5), 其他选择也是可能的。)
- (B) (1) $a^p b^{2p}$, (2) 串 $\alpha \frown \beta$ 完全由 a 组成, (3) β 至少包含一个 a , (4) $\alpha\beta^2\gamma$, (5) 至少多一个 a , 但 b 的数量相同。(对于 (4) 和 (5), 其他选择也是可能的。)

IV.5.14

- (A) 语言中的串由一串 a 后跟一串 b 组成, 且 b 的数量比 a 多两个。 $\mathcal{L}_0$ 的五个元素是 $bb, abbb, aabbbb, a^3 b^5$ 和 $a^4 b^6$ 。五个不属于该语言的串是 $\epsilon, a, ab, aabbbb$ 和 $baabbbb$ 。

为导出矛盾, 假设 $\mathcal{L}_0$ 是正则的。根据泵引理, 它有一个泵长度 p 。考虑 $\sigma = a^p b^{p+2}$ 。这个串是语言中的一个成员, 其长度大于等于 p , 因此泵引理给出一个分解 $\sigma = \alpha \frown \beta \frown \gamma$, 满足三个条件。条件 (1) 是 $|\alpha\beta| \leq p$, 而 σ 的前 p 个字符都是 a , 因此子串 α 和 β 只包含 a 。条件 (2) 是 β 非空, 所以它至少包含一个 a 。

考虑列表 $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 其中第二个串 $\alpha\beta^2\gamma$ 比 $\sigma = \alpha\beta\gamma$ 有更多的 a , 但没有更多的 b 。这意味着 $\alpha\beta^2\gamma$ 不是语言 $\mathcal{L}_0$ 的元素, 与泵引理的条件 (3) 矛盾。

- (B) 用文字描述, 这个语言由 a 后跟 b 再后跟 c 组成, 唯一的限制是 a 的数量等于 c 的数量。因此 $\mathcal{L}_1$ 的五个元素是: $\epsilon, ac, b, aabcc$ 和 bb 。五个不属于该语言的串是: aac, acc, ab, bbc 和 ca 。

为导出矛盾, 假设 $\mathcal{L}_1$ 是正则的。那么它有一个泵长度 p 。考虑 $\sigma = a^p c^p$ 。因为 $\sigma \in \mathcal{L}_1$ 且 $|\sigma| \geq p$, 泵引理说明它可以分解为 $\sigma = \alpha \frown \beta \frown \gamma$, 满足三个条件。由条件 (1), 前两个子串合起来很短, $|\alpha\beta| \leq p$ 。因此这两个子串完全由 a 组成。由条件 (2), 第二个子串 β 非空, 所以它至少包含一个 a 。最后, 条件 (3) 表明串 $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 都是语言 $\mathcal{L}_1$ 的成员。但是串 $\alpha\gamma$ 与 $\sigma = \alpha\beta\gamma$ 相比, 至少少一个 a , 但 c 的数量相同。这意味着 σ 中 a 的数量与 c 的数量不同, 因此 $\alpha\gamma$ 不是该语言的成员, 矛盾。

- (C) 在 $\mathcal{L}_2$ 中, 串由 a 后跟 b 组成, 且 a 的数量小于 b 的数量。所以属于 $\mathcal{L}_2$ 的五个串是 $abb, abbb,$

abbbb, ab^5 和 a^4b^5 。五个不属于该语言的串是 ε , a , ab , aab 和 $baabbbb$ 。

为导出矛盾, 假设 $\mathcal{L}_2$ 是正则的并设其泵长度为 p 。串 $\sigma = a^p b^{p+1}$ 是语言的一个元素, 其长度大于等于泵长度。因此泵引理给出一个分解 $\sigma = \alpha\beta\gamma$, 满足三个条件。条件 (1), $|\alpha\beta| \leq p$, 说明这两个子串只包含 a 。条件 (2), β 非空, 说明它至少包含一个 a 。

考虑以下串 $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 与 $\sigma = \alpha\beta\gamma$ 相比, 串 $\alpha\beta^2\gamma$ 有更多的 a , 但没有更多的 b 。因此 a 的数量至少为 $p+1$, 而 b 的数量固定为 $p+1$ 。这意味着 $\alpha\beta^2\gamma$ 不是语言 $\mathcal{L}_2$ 的成员, 这与泵引理的条件 (3) 矛盾。

IV.5.15

(A) 属于该语言的五个串是 aaa , $aaab$, $aaabb$, $aaabbb$ 和 $aaabbbb$ 。这个语言是正则的, 描述它的正则表达式是 $aaab^*$ 。

(B) 这个语言中的五个串是 bbb , $abbbb$, $aabbbbb$, a^3b^6 和 a^4b^7 。这个语言不是正则的。

为了导出矛盾, 假设该语言是正则的。泵引理表明该语言有一个泵长度 p 。串 $\sigma = a^p b^{p+3}$ 是该语言的一个成员, 且其长度大于或等于泵长度。因此它可分解为 $\sigma = \alpha\beta\gamma$, 且满足三个条件。第一个条件是 $|\alpha\beta| \leq p$, 所以两个子串 α 和 β 只包含 a 。第二个条件是 $|\beta| > 0$, 所以子串 β 至少包含一个 a 。

最后, 考虑列表 $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 条件 (3) 表明该列表中的任何元素都是该语言的成员。然而, 列表的第一个元素比 σ 含有更少的 a 但含有相同数量的 b , 因此 a 的数量不比 b 的数量少三个。这就产生了矛盾。

(C) 这个语言中的五个串是 ε , aab , b , $aaaabbbb$ 和 $abbbb$ 。这个语言是正则的。描述它的正则表达式是 a^*b^* 。

(D) 这个语言中的五个串是 b^{13} , ab^{14} , b^{14} , a^2b^{15} 和 ab^{15} 。这个语言不是正则的。

为了导出矛盾, 假设它是正则的。那么该语言有一个泵长度 p 。令 $\sigma = a^p b^{p+13}$ 。根据泵引理, 它分解为 $\sigma = \alpha\beta\gamma$, 满足三个条件。第一个条件的一个推论是: 前两个子串只包含 a 。第二个条件的一个推论是: β 至少包含一个 a 。

考虑 $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 。串 $\alpha\beta^2\gamma$ 比 σ 至少多一个 a 。因此, b 的数量减去 a 的数量不大于 12, 这意味着它不是该语言的成员。这与泵引理的第三个条件相矛盾。

IV.5.16

(A) 子串 $\alpha\beta$ 匹配 a^* , 而 γ 匹配 $a^*bb\dots b$, 其中有 $2p$ 个 b 。

(B) 子串 $\alpha\beta$ 匹配 a^* , 而 γ 匹配 $a^*bb\dots b$, 其中有 $p+5$ 个 b 。

(C) 子串 $\alpha\beta$ 匹配 a^* , 而 γ 匹配 $a^*ba\dots a$, 其中有 $p+1$ 个 a 。

(D) 子串 $\alpha\beta$ 匹配 a^* , 而 γ 匹配 $a^*b\dots b$, 其中有 p 个 b 。

(E) 子串 $\alpha\beta$ 匹配 a^* , 而 γ 匹配 $a^*bba\dots a$, 其中有 p 个 a 。

IV.5.17

(A) 该语言的五个成员是 ε , $abab$, $babbab$, $aaaa$ 和 $aaabaaab$ 。

(B) 假设相反, 即该语言是正则的。由泵引理, 该语言有泵长度 p 。考虑 $\sigma = a^p b \wedge a^p b$ 。它是该语言的成员, 且其长度大于或等于 p , 因此存在满足三个条件的分解 $\sigma = \alpha\beta\gamma$ 。第一个条件, $|\alpha\beta| \leq p$, 表明这两个子串只包含 a 。第二个条件, $|\beta| > 0$, 表明第二个子串非空, 因此至少含有一个 a 。

现在考虑第三个条件中的列表: $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 比较 $\alpha\gamma$ 与 $\sigma = \alpha\beta\gamma$ 。由于 $\alpha\gamma$ 中省略了 β , 在第一个 b 之前, $\alpha\gamma$ 比 σ 含有更少的 a 。但在第二个 b 之前, 两个串的 a 数量相同。因此 $\alpha\gamma$ 不是该语言的成员, 这与第三个条件矛盾。

(C) 前一部分的论证中有一个标记 b 最终落在子串 γ 中。本部分尝试的论证没有标记, 因此我们无法将串 σ 与列表中的串 $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 进行比较。你的朋友需要选择一个带有某种标记的串。

IV.5.18 该语言中的五个串是 cb , $acbb$, $aacbbb$, $aaacbbbb$ 和 $aaaacbbbbb$ 。

为了证明这个语言不是正则的, 假设它是正则的。那么该语言有泵长度 p 。考虑 $\sigma = a^p cb^{p+1}$ 。因为此串是该语言的成员且长度至少为 p , 泵引理表明它可分解为 $\sigma = \alpha\beta\gamma$, 且满足三个条件。

第一个条件, $|\alpha\beta| \leq p$, 意味着这两个子串 α 和 β 只包含 a 。第二个条件, $|\beta| > 0$, 意味着 β 至少包含一个 a 。

现在, 考虑列表 $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 中的第一个, 即 $\alpha\gamma$ 。由前一段可知, 与 $\sigma = \alpha\beta\gamma$ 相比, 此串至少少一个 a , 但 b 的数量相同。因此 a 的数量不比 b 的数量少一, 此串不在该语言中, 这与第三个

条件矛盾。

IV.5.19 令此语言为 $\mathcal{L} = \{\sigma \in \{a, b\}^* \mid a \text{ 的数量大于 } b \text{ 的数量}\}$ 。为导出矛盾, 假设它是正则的。那么泵引理适用, 该语言有泵长度 p 。串 $\sigma = a^{p+1}b^p$ 是该语言的一个元素, 且其长度大于或等于 p 。因此存在满足三个条件的分解 $\sigma = \alpha\beta\gamma$ 。条件 (1) 给出 $|\alpha\beta| \leq p$, 所以这两个子串只包含 a 。条件 (2) 给出 β 非空, 因此它至少包含一个 a 。

矛盾来自条件 (3) 的列表 $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 。该列表中的第一个串 $\alpha\gamma$ 与 $\sigma = \alpha\beta\gamma$ 相比, 至少少一个 a , 而 b 的数量相同。因此它不是语言 $\mathcal{L}$ 的成员, 这与条件 (3) 矛盾。此矛盾表明 $\mathcal{L}$ 不是正则的。

IV.5.20 第一个不是正则的, 而第二个是正则的。

(A) 用反证法, 假设这个语言是正则的。那么它有一个泵长度 p 。取字符串 $\sigma = a^{p^2}b^p$ 。它是该语言的成员, 其长度大于或等于 p , 因此它可以分解为 $\sigma = \alpha\beta\gamma$, 且满足三个条件。条件 (1) 推出 α 和 β 都只包含 a 。条件 (2) 推出 β 至少包含一个 a 。

现在考虑条件 (3) 给出的列表: $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 与 $\sigma = \alpha\beta\gamma$ 相比, 列表项 $\alpha\gamma$ 中至少少了一个 a , 但 b 的数量相同。因此, a 的数量不再是 b 数量的平方, 所以 $\alpha\gamma$ 不在该语言中。这与条件 (3) 矛盾。

(B) 描述这个语言的一个正则表达式是 $aaaaa*bbbbbb*$ 。

IV.5.21 该语言是正则的。这是一个技巧题: 取 $\alpha = \varepsilon$ 可得该集合等于 $\mathbb{B}^*$ 。

IV.5.22 假设它是正则的, 并令其泵长度为 p (为方便下面括号内的说明, 可取 $p > 1$)。取字符串 $\sigma = 1^{p!}$ 。该串是 $\mathcal{L}$ 的成员且长度至少为 p , 所以泵引理说它能分解为 $\sigma = \alpha\beta\gamma$, 并满足三个条件。条件 (2) 推出 $|\beta| > 0$ 。考虑列表 $\alpha\gamma, \alpha\beta^2\gamma, \alpha\beta^3\gamma, \dots$ 从第一项之后, 列表中相邻元素的长度差总是 $|\beta|$ 。但是, 如提示所指, 相继阶乘之间的差不固定, 而是无界增长。(换种说法, $\sigma = 1^{p!}$ 与 $\hat{\sigma} = 1^{(p+1)!}$ 的长度差是 $(p+1)! - p! = ((p+1) - 1) \cdot p! = p \cdot p!$ 。但 $\sigma = \alpha\beta\gamma$ 与 $\alpha\beta^2\gamma$ 的长度差是 $|\beta|$, 它小于 $p \cdot p!$, 因为 β 是长度为 $p!$ 的串的子串, 而且我们取了 $p > 1$ 。) 因此, 列表中存在一个不属于该语言的元素, 这与条件 (3) 矛盾。故该语言不是正则的。

IV.5.23 第一个是正则的, 由正则表达式 0^*10^* 描述。第二个不是正则的。为证明这一点, 用反证法假设它是正则的。那么它有一个泵长度 p 。令 $\sigma = 0^p10^p$ 。于是 σ 是该语言的成员且长度至少为 p , 所以泵引理说它能分解为 $\sigma = \alpha\beta\gamma$, 并满足三个条件。条件 (1) 推出子串 α 和 β 只包含 0 。条件 (2) 说明 β 至少包含一个 0 。

考虑条件 (3) 列表中的串 $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 根据前一段, 第一个串 $\alpha\gamma$ 中在 1 之前的 0 比 σ 少, 但在 1 之后的 0 数量相同。因此它不是该语言的成员, 这与泵引理的第三个条件矛盾。

IV.5.24 语言 $\mathcal{L}_4 = \{a^i b^j c^k \mid i, j, k \in \mathbb{N}, i + j = k \text{ 且 } k < 4\}$ 是有限的, 而任何有限语言都是正则的。

至于所有总和的语言, 即 $\mathcal{L} = \{a^i b^j c^k \mid i, j, k \in \mathbb{N} \text{ 且 } i + j = k\}$, 假设它是正则的并固定其泵长度 p 。固定 $\sigma = a^p b^0 c^p$ 。因为 $|\sigma| \geq p$, 所以它可以分解为 $\sigma = \alpha\beta\gamma$, 且满足泵引理的三个条件。根据第一个条件, 长度 $|\alpha\beta|$ 小于或等于 p , 因此 $\alpha\beta$ 只包含 a 。根据第二个条件, 串 β 非空, 所以它至少包含一个 a 。第三个条件则说 $\alpha\gamma$ 也属于该语言, 但这导致矛盾, 因为这个串中 a 的数量比 c 的数量少, 并且它没有 b (因为 σ 本来就没有 b), 所以 $\alpha\gamma$ 不属于 $\mathcal{L}$ 。

IV.5.25

(A) 它是正则的, 因为它是有限的, 即 $\{00000, 000001, 00010, \dots, 11111\}$ 。

(B) 语言 $\mathcal{L} = \{\sigma \in \mathbb{B}^* \mid 0 \text{ 的数量减去 } 1 \text{ 的数量等于五}\}$ 不是正则的。假设它是正则的。设该语言的泵长度为 p , 并令 $\sigma = 0^{p+5}1^p \in \mathcal{L}$ 。它的长度至少为 p , 所以将其分解为满足三个条件的 $\sigma = \alpha\beta\gamma$ 。条件 (1), 即 $|\alpha\beta| \leq p$, 意味着这两个子串只包含 0 。条件 (2) 表明 β 非空, 因此它至少包含一个 0 。

条件 (3) 表明列表 $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 上的每个串都是 $\mathcal{L}$ 的成员, 但此处并非如此。由上一段可知, 串 $\alpha\gamma$ 比串 $\sigma = \alpha\beta\gamma$ 至少少一个 0 , 但两者的 1 数量相同。因此它不是 $\mathcal{L}$ 的元素。由此矛盾, 我们证明了 $\mathcal{L}$ 不是正则的。

IV.5.26 记这个语言为 $\mathcal{L} = \{0^m 1^n \in \mathbb{B}^* \mid m \neq n\}$ 。其中 $\hat{\mathcal{L}} = \{0^m 1^m \in \mathbb{B}^* \mid m \in \mathbb{N}\}$, 我们有 $\hat{\mathcal{L}} = \mathbb{B}^* - \mathcal{L}$ 。根据定理 4.5, 正则语言在集合差运算下是封闭的。显然 $\mathbb{B}^*$ 是正则的——它可以用正则表达式 $(0|1)^*$ 描述——所以如果 $\mathcal{L}$ 是正则的, 那么 $\hat{\mathcal{L}}$ 也必然是正则的。但 $\hat{\mathcal{L}}$ 不是正则的, 因为它是平衡括号的语言 (见例 5.2), 只不过是使用 0 代替开括号, 用 1 代替闭括号。

IV.5.27 遵循提示, 一个串要成为语言 $\mathcal{L}$ 的成员, 其转入的次数必须等于转出的次数。例如, 串

$[0.10ex][1pt]aaa[0.10ex][1pt]b[0.10ex][1pt]aaa[0.10ex][1pt]bbb$ 不是 $\mathcal{L}$ 的成员, 因为存在两次从 **a** 块转入 **b** 块的转移, 但只有一次从 **b** 块转出回到 **a** 块的转移。

简而言之, $\mathcal{L}$ 就是那些最后一个字符与第一个字符相同的串的集合。因此, 对应的正则表达式是 $(\varepsilon|a|b|a(a|b)^*a)|(b(a|b)^*b)$ 。

IV.5.28 泵引理的证明使用鸽笼原理来表明至少存在一个循环, 然后基于该循环进行推理。即使后面还有另一个循环, 这个证明也不会受影响。

IV.5.29 反设这个语言是正则的, 并固定一个泵长度 p 。考虑条件 (3) 给出的列表。

$$\alpha\gamma, \alpha\beta^2\gamma, \alpha\beta^3\gamma, \dots, \alpha\beta^n\gamma, \dots \quad n \geq 2$$

这个列表中字符串之间的间隙 (除了第一个间隙) 大小都相同, 即 $|\alpha\beta^{n+1}\gamma| - |\alpha\beta^n\gamma| = |\beta|$ 。因此对于 $n \geq 2$, $\alpha\beta^n\gamma$ 的长度是 $|\alpha\beta^2\gamma| + (n-2) \cdot |\beta|$ 。取 n 为 $|\alpha\beta^2\gamma| + 2$, 并注意到 $\alpha\beta^n\gamma$ 的长度不是素数。这样一来, 并非条件 (3) 列表中的所有字符串都属于该语言, 这就得到了想要的矛盾。

IV.5.30

- (A) **c** 的数量是 **a** 的数量与 **b** 的数量的乘积。五个这样的串是 **abc**、**abbcc**、**abbbccc**、**aabcc** 和 **aabbccccc**。
 (B) 反设这个语言 $\mathcal{L}$ 是正则的。泵引理说明 $\mathcal{L}$ 有一个泵长度 p 。考虑 $\sigma = a^p b^p c^{(p^2)}$ 。该串的长度大于 p , 因此它可分解为 $\sigma = \alpha\beta\gamma$, 且满足三个条件。条件 (1) 是 $|\alpha\beta| \leq p$, 所以子串 α 和 β 都完全由 **a** 组成。条件 (2) 是 β 不是空串, 因此 β 至少包含一个 **a**。

条件 (3) 是列表 $\alpha\gamma, \alpha\beta^2\gamma, \alpha\beta^3\gamma, \dots$ are also members of the language. We will get the contradiction from the first one, $\alpha\gamma$ 。与 $\sigma = \alpha\beta\gamma$ 相比, 在第一个串 $\alpha\gamma$ 中, β is gone. So the number of **a**'s in $\alpha\gamma$ 的数量小于 $\sigma = \alpha\beta\gamma$ 中的数量, 但 **b** 的数量和 **c** 的数量却保持不变。因此, 在 $\alpha\gamma$ 中, **a** 的数量乘以 **b** 的数量并不等于 **c** 的数量。

IV.5.31 回忆附录 A 中定义的字符串记号: 在字符串 τ 中, 第 j 个字符是 $\tau[j]$ 。索引从零开始计数, 意味着第一个字符是 $\tau[0]$ 。并且, $\tau[j:]$ 是从字符 $\tau[j]$ 开始直到末尾的子串。因此, 对于 $\tau = 012345$, 我们有 $\tau[0] = 0$, 且 $\tau[1:] = 12345$ 。

- (A) 正则表达式 $(b|c)^*|((aaa)^*(b|c)^*)$ 描述了此语言。
 (B) 假设 $\mathcal{L}_0$ 是正则的, 并固定某个泵长度 $p > 1$ (泵引理脚注允许我们将泵长度设置为至少 2)。考虑 $\sigma = ab^p c^p$ 。它被分解为 $\sigma = \alpha\beta\gamma$, 且满足条件。第一个条件意味着 α 和 β 只包含 **a** 和 **b**。第二个条件意味着 β 包含至少一个字符, 且必然不是 **c**。第三个条件则在考虑 $\alpha\gamma$ 时导致矛盾, 因为 $\alpha\gamma$ 包含与 σ 相同数量的 **c**, 但要么没有 **a**, 要么有更少的 **b**。
 (C) 如果 $\mathcal{L}$ 是正则的, 那么集合差 $\mathcal{L} - \mathcal{L}_1 = \mathcal{L}_0$ 将是正则的, 但前一项已证明它不是。
 (D) 泵引理只对长度至少为泵长度 $p = 3$ 的字符串施加限制。对于一个形如 $\sigma = ab^n c^n$ 的字符串, 条件 $|\sigma| \geq p$ 意味着 $n \geq 1$ 。在此基础上, 我们将证明一个满足三个条件的分解是: $\alpha = \varepsilon$ 、 $\beta = a$, 以及 $\gamma = b^n c^n$ 。前两个条件 ($|\alpha\beta| \leq 3 = p$ 且 β 不是空串) 是显然的。要完成证明, 我们必须验证列表 $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 中的每个字符串都是 $\mathcal{L}$ 的元素。第一个是 $\alpha\gamma = b^n c^n$, 它是 $\mathcal{L}_1$ 的一个元素 (其中定义中的 $k = 0$)。接下来是 $\alpha\beta^2\gamma = aab^n c^n$, 它是 $\mathcal{L}_1$ 的一个元素 (其中 $k = 2$)。其余字符串, 即 $i > 2$ 时的 $\alpha\beta^i\gamma = a^i b^n c^n$, 类似地也是 $\mathcal{L}_1$ 的元素 (其中 $k = i$)。
 (E) 对于一个形如 $\sigma = \tau \in \{b, c\}^*$ 的字符串, 要满足 $|\sigma| \geq p = 3$, 我们需要 $|\tau| \geq 3$ 。这里一个可行的分解是: $\alpha = \varepsilon$ 、 $\beta = \tau[0] = x$ (其中 $x = b$ 或 $x = c$), 以及 $\gamma = \tau[1:]$ 。那么前两个条件得到满足: $|\alpha\beta| \leq p$ 且 $\beta \neq \varepsilon$ 。要完成证明, 我们检查列表 $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 中的每个字符串都是 $\mathcal{L}$ 的元素。第一个是 $\alpha\gamma = \tau[1:]$, 它是 $\mathcal{L}_1$ 的一个元素 (其中 $k = 0$)。列表中其余部分的字符串, 即 $i \geq 2$ 时的 $\alpha\beta^i\gamma = x^i \tau[1:]$, 类似地也是 $\mathcal{L}_1$ 的元素 (其中 $k = 0$)。
 (F) 考虑形如 $\sigma = a^k \tau$ 的字符串, 其中 $k \geq 2$ 且 $\tau \in \{b, c\}^*$ 。泵引理只要求我们考虑满足 $|\sigma| \geq p = 3$ 的字符串 σ 。一个可行的分解是: $\alpha = aa$ 、 $\beta = x$ (其中 x 是字符串中的第三个字符, 即 $x = \sigma[2]$), 以及 $\gamma = \sigma[3:]$ 。注意, 如果 $x = a$, 则 γ 匹配正则表达式 $a^*(b|c)^*$; 而如果 $x \neq a$, 则 γ 匹配 $(b|c)^*$ 。

此分解满足泵引理的前两个条件, 因为显然 $|\alpha\beta| \leq p$ 且 $\beta \neq \varepsilon$ 。剩下的任务是检查第三个条件, 即列表 $\alpha\gamma, \alpha\beta^2\gamma, \dots$ 中的每个字符串都是 $\mathcal{L}$ 的元素。

列表的第一个元素是 $\alpha\gamma$ 。假设 β 的单个字符 x 是 **a**。我们已经注意到 γ 匹配正则表达式 $a^*(b|c)^*$, 因此拼接 $\alpha\gamma$ 匹配 $aaa^*(b|c)^*$, 从而是 $\mathcal{L}_1$ 的一个元素 (其中 $k \geq 2$)。假设 x 不是

a. 我们已经注意到此时 γ 匹配 $(b|c)^*$, 因此拼接 $\alpha\gamma$ 匹配 $aa(b|c)^*$, 使其成为 $\mathcal{L}_1$ 的一个元素 (其中 $k = 2$)。

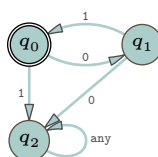
列表其余部分的字符串具有形式 $\alpha\beta^i\gamma$ (其中 $i \geq 2$)。如果 β 的字符 x 是 a , 那么 $\alpha\beta^i\gamma$ 匹配左边的正则表达式; 如果 β 的字符 x 不是 a , 那么 $\alpha\beta^i\gamma$ 匹配右边的正则表达式。

$$aa(\underbrace{aa \dots a}_{i\text{-many}})a^*(b|c)^* \quad aa(\underbrace{xx \dots x}_{i\text{-many}})(b|c)^*$$

无论哪种情况, 该字符串都是 $\mathcal{L}_1$ 的元素。

IV.5.32

(A) 下面这台机器满足要求。



它是最小的, 因为从 q_0 出发, 输入 1 必须转移到一个错误状态, 而输入 0 必须转移到另一个不同的 (同样也是非接受的) 状态。由于 q_0 是接受状态, 所以总共需要三个状态。

(B) 前一项表明, 识别 $\mathcal{L}$ 的最小有限状态机具有三个状态。因此, 根据泵引理的证明, 语言中任何长度至少为三个字符的字都可以被泵, 也就是说, 可以分解成满足引理条件的形式。唯一的长度为 2 的字 $\sigma = 01$, 可以用 $\alpha = \varepsilon$, $\beta = 01$ 和 $\gamma = \varepsilon$ 来泵。所以最小泵长度至多是 2。至于 $p = 1$, 该语言不包含长度为 1 的字。因此, 该语言的最小泵长度是 1, 因为如果 $\sigma \in \mathcal{L}$ 且 $|\sigma| \geq 1$, 则自动有 $|\sigma| \geq 2$, 并且 σ 可以被泵。

IV.6.7 只用下面两条指令的机器, 会一直不停地从栈中弹出 $\perp$ 然后再把它压回去。

指令	输入	输出
0	$q_0, \varepsilon, \perp$	$q_1, \perp$
1	$q_1, \varepsilon, \perp$	$q_0, \perp$

下面这台机器会让栈无限增长。

指令	输入	输出
0	$q_0, \varepsilon, \perp$	$q_1, \text{'g0 } \perp$
1	$q_1, \varepsilon, \text{'g0}$	$q_0, \text{'g0 g0'}$
2	$q_0, \varepsilon, \text{'g0}$	$q_1, \text{'g0 g0'}$

IV.6.8 每个格局都是一个三元组: 当前状态、剩余的纸带记号和栈。

(A) 我们得到如下计算。

$$\langle q_0, [] [] B, \perp \rangle \vdash \langle q_0,] [] B, \text{'g0 } \perp \rangle \vdash \langle q_0, [] B, \perp \rangle \vdash \langle q_0,] B, \text{'g0 } \perp \rangle \vdash \langle q_0, B, \perp \rangle \vdash \langle q_1, \text{'', } \perp \rangle$$

最后一个格局的纸带为空且状态为 q_1 , 因此机器接受初始输入串。

(B) 这个计算过程与上一个大不相同。

$$\langle q_0,] [] [B, \perp \rangle \vdash \langle q_0, [] [B, \text{' } \rangle$$

此时机器的栈已空。它不能开始下一步, 因为每一步都要先弹出栈顶字符, 而现在没有这样的字符。于是计算停止, 输入被拒绝。

IV.6.9 这是接受输入 bacab 的计算树中的一条极大路径。

Step	Machine	Instruction
0	b a c a b B q₀	⊥ 1
1	a c a b B q₀	g¹ ⊥ 5
2	c a b B q₀	g⁰ g¹ ⊥ 8
3	a b B q₁	g⁰ g¹ ⊥ 10
4	b B q₁	g¹ ⊥ 11
5	B q₁	⊥ 12
6	q₃	⊥

在状态 q_0 中，当机器在纸带上读到 a 时，它会将 g_0 压入栈；读到 b 时，则压入 g_1 。读取 c 会使机器切换到状态 q_1 。在这个阶段，如果机器正在读 a 并且栈顶字符是 g_0 ，那么它将消耗该 a ，弹出栈顶的 g_0 ，然后继续执行。对于 b 和 g_1 的情况也类似。（但是，如果纸带上是 a 而栈顶却是 g_1 ，或者纸带上是 b 而栈顶却是 g_0 ，那么计算就会无法继续。）最后，机器读到输入串的结尾（包括标记 B ），进入接受状态 q_2 。

IV.6.10

(A) 当此机器消耗一个 a 时，它将两个 g_0 压入栈。当它读到中间的标记 c 时，便切换到弹出模式。唯一的接受状态是 q_2 。

指令	输入	输出
0	$q_0, a, \perp$	$q_0, 'g_0 g_0 \perp'$
1	$q_0, c, \perp$	$q_1, '\perp'$
2	q_0, a, g_0	$q_0, 'g_0 g_0 g_0'$
3	q_0, c, g_0	$q_1, 'g_0'$
4	q_1, b, g_0	$q_1, ''$
5	$q_1, B, \perp$	$q_2, \perp$

(B) 这是对上一小题机器的改编，只是丢弃第一个 a 。唯一的接受状态是 q_3 。

指令	输入	输出
0	$q_0, a, \perp$	$q_1, '\perp'$
1	$q_1, a, \perp$	$q_1, 'g_0 g_0 \perp'$
2	$q_1, c, \perp$	$q_2, '\perp'$
3	q_1, a, g_0	$q_1, 'g_0 g_0 g_0'$
4	q_1, c, g_0	$q_2, 'g_0'$
5	q_2, b, g_0	$q_2, ''$
6	$q_2, B, \perp$	$q_3, \perp$

IV.6.11 这是一个正则语言，因此我们不需要栈。这台下推机实现了所示的非确定性有限状态机；其接受状态是 q_2 。

```

graph LR
    start(( )) --> q0((q0))
    q0 -- 0 --> q1((q1))
    q1 -- "0, 1" --> q1
    q1 -- 1 --> q2(((q2)))
  
```

IV.6.15 $S \rightarrow 0S0 \mid 1S1 \mid \varepsilon$

IV.6.16 该语言是 $\mathcal{L}_{\text{ELP}} = \{\sigma\sigma^R \mid \sigma \in \mathbb{B}^*\}$ 。

用反证法, 假设 $\mathcal{L}_{\text{ELP}}$ 是正则的。那么泵引理说明它有一个泵长度, 我们记为 p 。考虑串 $\sigma = 0^p 110^p$ 。

它是 $\mathcal{L}$ 中长度大于 p 的元素。因此它分解为三个子串 $\sigma = \alpha\beta\gamma$ 满足条件。条件 (1) 是前缀 $\alpha\beta$ 的长度小于或等于 p 。因为 σ 的前 p 个字符都是 0, 这意味着 α 和 β 都只包含 0。条件 (2) 是 β 非空, 所以它至少包含一个 0。

考虑 $\alpha\gamma$ 。与 $\sigma = \alpha\beta\gamma$ 相比, 这个串缺少了 β , 这意味着在 $\alpha\gamma$ 的子串 11 之前, 0 的数量比 σ 中的少。但 11 子串之后 0 的数量与 σ 相同, 因此它不是 $\mathcal{L}_{\text{ELP}}$ 的成员。这与泵引理的条件 (3) 矛盾。

IV.6.17

- (A) 下推机是一个五元组的有限集合。这五个分量中的每一个都来自一个可数集合。因此, 下推机的总数是可数的。
- (B) 每台机器接受一个语言。因此, 语言的数目至多和机器的数目一样多。
- (C) 存在不可数多个语言。

IV.6.18 我们可以用 Turing 机来模拟一台下推机。

IV.A.21

- (A) `.*abc*.*`
- (B) `.*abc+.*`
- (C) `.*abc?.*`
- (D) `.*abc2.*`
- (E) `.*abc2,5.*`
- (F) `.*abc2,.*`
- (G) `.*a(b|c).*` 或 `.*a[bc].*`

IV.A.22

- (A) `.*abe.*` 注意, 实际上你通常可以只写 `abe`; 请查阅你所使用语言的文档。
- (B) `[a-zA-Z0-9]*`
- (C) `\d{1,3}`

IV.A.23

- (A) `(a|e|i|o|u).*bc.*`
- (B) `(a|e|i|o|u)(bc|.*bc).*`
- (C) `.*a.*e.*i.*o.*u.*`
- (D) `(.*a.*e.*i.*o.*u.*)|(.*a.*e.*i.*u.*o.*)|...|(.*u.*o.*i.*e.*a.*)` 共有 $5! = 120$ 个子句。

你可能会用程序来写这个正则表达式。

IV.A.24 `.*([.*|{.*}]).*`

IV.A.25 `\d{10}`

IV.A.26

- (A) `(\d{3}|\(\d{3}\)) \d{3}-\d{4}`
- (B) 我们必须同时将空格和区号作为可选项提取出来: `(\d{3} |\(\d{3}\)) ?\d{3}-\d{4}`。

IV.A.28 `\d{2}:\d{2}:\d{2}(. \d+)?|\d{2}(:\d{2})?`

IV.A.29 一个例子是 `[-\+]?(\d)+\.?(\d)*\.\d+`

IV.A.30

- (A) `4(\d15|\d12)`
- (B) `(5[1-5]\d2|222[1-9]|22[3-9]\d|2[3-6]\d2|27[01]\d|2720)\d12`
- (C) `3[47][0-9]13`

IV.A.31

- (A) 我们可以将六种格式作为备选项。

`[A-Z]\d2 \d[A-Z]2|[A-Z]\d \d[A-Z]2|[A-Z]\d[A-Z] \d[A-Z]2|[A-Z]2\d2
\d[A-Z]2|[A-Z]2\d \d[A-Z]2|[A-Z]2\d[A-Z] \d[A-Z]2`

(B) 我们可以提取结尾 $[A-Z]^2$ 的公因子。

$[A-Z]^2 \backslash d^2 | [A-Z] \backslash d | [A-Z] \backslash d [A-Z] | [A-Z]^2 \backslash d^2 | [A-Z]^2 \backslash d | [A-Z]^2 \backslash d [A-Z]) \backslash d [A-Z]^2$

IV.A.32 一个合适的正则表达式是 `textasciicircum g.n.{3}i.$`，匹配到的结果是 “gynaecia”，意为花中心皮或雌蕊的集合。

IV.A.34 $\sim M0,4(CM|CD|D?C0,3)(XC|XL|L?X0,3)(IX|IV|V?I0,3)\$$

IV.A.35 假设它是一个正则语言，并令其泵长度为 p 。考虑 $\sigma = a^p b a^p b$ 它是 $\mathcal{L}$ 中一个长度大于 p 的元素，因此根据泵引理，它可以分解为 $\sigma = \alpha \beta \gamma$ ，且满足三个条件。根据第一个条件， $\alpha \beta$ 的长度小于 p ，因此这两个子串完全由 a 组成。第二个条件是 β 不是空串。第三个条件说明，字符串 $\alpha \gamma$ 、 $\alpha \beta^2 \gamma$ 、 $\dots$ 全都是 $\mathcal{L}$ 的成员。注意，第一个字符串不是平方串，因为省略 β 会从前缀中去掉至少一个 a ，但不会从后缀中去掉。这就产生了矛盾。

IV.A.36

(A) 我们取字母表为 $\Sigma = \mathbb{B}$ 。假设它是正则的，并令其泵长度为 p 。考虑 $\sigma = 0^p 1 0^p$ ，它是该语言的一个成员。根据泵引理，它可以分解为 $\sigma = \alpha \beta \gamma$ ，并满足三个条件。由条件一和条件二， $\alpha \beta$ 完全由 0 组成，因此是位于 1 之前的前缀的一部分，且 β 非空。由条件三，字符串 $\alpha \gamma$ 也是 $\mathcal{L}$ 的成员。但这导致了矛盾，因为去掉 β 意味着 $\alpha \gamma$ 中 1 之前的 0 比之后的要少。

(B) $(a^*)b \setminus 1$

IV.A.37

(A) `. *r`

(B) `. *i . *t . *`

(C) `. *foo . *`

IV.A

(A) `HELP`

(B)

	$(A B C) \setminus 1$	$(AB OE SK)$
<code>. *M?O . *</code>	B	O
<code>(AN FE BE)</code>	B	E

IV.B.18 语言中的每一个串 $\mathcal{L} = \{ab^n \mid n \in \mathbb{N}\} = \{a, ab, abb, \dots\}$ 都是 $\mathcal{E}_{\mathcal{L},1}$ 的成员。在其他串中， $\mathcal{E}_{\mathcal{L},0}$ 仅包含空串 ε ，而所有其他串都在 $\mathcal{E}_{\mathcal{L},2}$ 中。

(A) $bba \in \mathcal{E}_{\mathcal{L},2}$

(B) $abb \in \mathcal{E}_{\mathcal{L},1}$

(C) $babab \in \mathcal{E}_{\mathcal{L},2}$

(D) $abbbb \in \mathcal{E}_{\mathcal{L},1}$

IV.B.19

(A) 起始字符串的等价类是 $bba \in \mathcal{E}_{\mathcal{L},2}$ 。追加后得到 $bba \hat{\ } a = bbaa$ ，其等价类为 $bbaa \in \mathcal{E}_{\mathcal{L},2}$ 。这与例 B.14 中给出的 Δ 表的左下角条目（即标有 ‘a’ 的那一列最下方的条目）相符。

(B) 输入的等价类是 $\varepsilon \in \mathcal{E}_{\mathcal{L},0}$ 。追加后得到 $\varepsilon \hat{\ } a = a \in \mathcal{E}_{\mathcal{L},1}$ 。这与 Δ 表的左上角条目相符。

(C) 起始等价类是 $abbb \in \mathcal{E}_{\mathcal{L},1}$ 。追加后得到 $abbb \hat{\ } a = abbbba \in \mathcal{E}_{\mathcal{L},2}$ ，这与转移表的中间左侧条目相匹配。

(D) 起始等价类是 $aa \in \mathcal{E}_{\mathcal{L},2}$ 。追加后得到 $aa \hat{\ } a = aaa \in \mathcal{E}_{\mathcal{L},2}$ ，这与表的左下角条目相符。

IV.B.20 我们将按照例子中给出的直观思路进行证明。固定某个 $i \in \{0, 1, 2, 3\}$ 。我们首先证明，如果 $\sigma_0, \sigma_1 \in \mathcal{E}_{\mathcal{L},i}$ ，那么它们是不可区分的。考虑一个比特串 τ ，并令 $k \in \mathbb{N}$ 为 τ 中 1 的个数。那么 $\sigma_0 \hat{\ } \tau$ 中 1 的个数是 $i + k$ 除以 4 的余数，而这也正是 $\sigma_1 \hat{\ } \tau$ 中 1 的个数。因此， $\sigma_0 \hat{\ } \tau \in \mathcal{L}$ 当且仅当 $\sigma_1 \hat{\ } \tau \in \mathcal{L}$ 。

最后证明，如果 $\sigma_0 \in \mathcal{E}_{\mathcal{L},i}$ 且 $\sigma_1 \in \mathcal{E}_{\mathcal{L},j}$ ，其中 $j \neq i$ ，那么它们是可区分的。如果 $\tau = 1^{4-i}$ ，那么由于 $i \neq j$ ，我们有 $\sigma_0 \hat{\ } \tau \in \mathcal{L}$ 但 $\sigma_1 \hat{\ } \tau \notin \mathcal{L}$ ，所以它们可区分。

IV.B.21

(A) 对三个串追加 0 得到 $\sigma_0 \hat{\ } 0 = 000$ 、 $\sigma_1 \hat{\ } 0 = 110110$ ，以及 $\sigma_2 \hat{\ } 0 = 001^8 0$ 。三者都属于 $\mathcal{E}_{\mathcal{L},0}$ 。查看机器的转移表可知，从类 $\mathcal{E}_{\mathcal{L},0}$ 出发，在输入 0 下转移到的正是这个类。

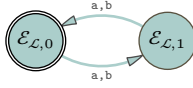
(B) 追加 1 得到 $\sigma_0 \hat{\ } 1 = 001$ 、 $\sigma_1 \hat{\ } 1 = 110111$ ，以及 $\sigma_2 \hat{\ } 1 = 001^8 1$ 。三者都属于 $\mathcal{E}_{\mathcal{L},1}$ 。机器的转移表显示，从类 $\mathcal{E}_{\mathcal{L},0}$ 出发，在输入 1 下转移到的正是这个类。

(C) 当然，可以选取很多组三个串，但作为示例，我们可以取 $\sigma_0 = 100$ 、 $\sigma_1 = 010$ ，以及 $\sigma_2 = 111101$ 。追加 0 得到 $\sigma_0 \hat{\ } 0 = 1000$ 、 $\sigma_1 \hat{\ } 0 = 0100$ ，以及 $\sigma_2 \hat{\ } 0 = 1111010$ 。三者都属于 $\mathcal{E}_{\mathcal{L},1}$ 。机器的转移表显示， $\mathcal{E}_{\mathcal{L},2}$ 确实是从类 $\mathcal{E}_{\mathcal{L},1}$ 出发在输入 0 下转移的结果。

至于追加 1，我们得到 $\sigma_0 \hat{\ } 1 = 1001 \in \mathcal{E}_{\mathcal{L},2}$ 、 $\sigma_1 \hat{\ } 1 = 0101 \in \mathcal{E}_{\mathcal{L},2}$ ，以及 $\sigma_2 \hat{\ } 1 = 1111011 \in \mathcal{E}_{\mathcal{L},2}$ 。机器的转移表与此一致，显示从类 $\mathcal{E}_{\mathcal{L},1}$ 出发在输入 1 下转移的结果是 $\mathcal{E}_{\mathcal{L},2}$ 。

IV.B.22 该例给出的两个类为 $\mathcal{E}_{\mathcal{L},0} = \{\sigma \mid |\sigma| \text{ 为偶数}\}$ 和 $\mathcal{E}_{\mathcal{L},1} = \{\sigma \mid |\sigma| \text{ 为奇数}\}$ 。我们可以从每个类中任选一个串作为代表。选择空串 ε 作为 $\mathcal{E}_{\mathcal{L},0}$ 的代表。选择长度为 1 的串 a 作为 $\mathcal{E}_{\mathcal{L},1}$ 的代表。由于 $\mathcal{E}_{\mathcal{L},0}$ 包含了 $\mathcal{L}$ 中的所有串，所以唯一的接受状态是 $\mathcal{E}_{\mathcal{L},0}$ 。同时，因为它包含空串，所以它也是初始状态。

Δ	a	b
$+\varepsilon \in \mathcal{E}_{\mathcal{L},0}$	$a \in \mathcal{E}_{\mathcal{L},1}$	$b \in \mathcal{E}_{\mathcal{L},1}$
$a \in \mathcal{E}_{\mathcal{L},1}$	$aa \in \mathcal{E}_{\mathcal{L},0}$	$ab \in \mathcal{E}_{\mathcal{L},0}$



IV.B.23 这些类分别为 $\mathcal{E}_{\mathcal{L},0} = \{\varepsilon\}$ 、 $\mathcal{E}_{\mathcal{L},1} = \{\sigma \in \Sigma^* \mid |\sigma| = 1\} = \{a, b\}$ 、 $\mathcal{E}_{\mathcal{L},2} = \{\sigma \in \Sigma^* \mid |\sigma| = 2\} = \{aa, ab, ba, bb\}$ 、 $\mathcal{E}_{\mathcal{L},3} = \{\sigma \in \Sigma^* \mid |\sigma| = 3\}$ ，以及 $\mathcal{E}_{\mathcal{L},4} = \{\sigma \mid |\sigma| \geq 4\}$ 。为了确定转移，我们可以从每个类中任选一个串作为代表。 $\mathcal{E}_{\mathcal{L},0}$ 的代表必须取空串 ε 。 $\mathcal{E}_{\mathcal{L},1}$ 的代表可以取长度为 1 的串 a 。 $\mathcal{E}_{\mathcal{L},2}$ 的代表可以选 aa ； $\mathcal{E}_{\mathcal{L},3}$ 的代表可以用 aaa ； $\mathcal{E}_{\mathcal{L},4}$ 的代表可以取 $aaaa$ 。

Δ	a	b
$\varepsilon \in \mathcal{E}_{\mathcal{L},0}$	$a \in \mathcal{E}_{\mathcal{L},1}$	$b \in \mathcal{E}_{\mathcal{L},1}$
$a \in \mathcal{E}_{\mathcal{L},1}$	$aa \in \mathcal{E}_{\mathcal{L},2}$	$ab \in \mathcal{E}_{\mathcal{L},2}$
$aa \in \mathcal{E}_{\mathcal{L},2}$	$aaa \in \mathcal{E}_{\mathcal{L},3}$	$aab \in \mathcal{E}_{\mathcal{L},3}$
$+aaa \in \mathcal{E}_{\mathcal{L},3}$	$aaaa \in \mathcal{E}_{\mathcal{L},4}$	$aaab \in \mathcal{E}_{\mathcal{L},4}$
$aaaa \in \mathcal{E}_{\mathcal{L},4}$	$aaaaa \in \mathcal{E}_{\mathcal{L},4}$	$aaaab \in \mathcal{E}_{\mathcal{L},4}$



由于 $\mathcal{E}_{\mathcal{L},3}$ 包含了 $\mathcal{L}$ 中的所有串，所以唯一的接受状态是 $\mathcal{E}_{\mathcal{L},3}$ 。初始状态是包含空串的类，即 $\mathcal{E}_{\mathcal{L},0}$ 。

IV.B.24 我们首先验证 $\mathcal{E}_{\mathcal{L},0} = \{\sigma \in \{a, b\}^* \mid \sigma \text{ 以 } b \text{ 结尾}\}$ 中的所有串都是 $\sim_{\mathcal{L}}$ 等价的。固定 $\sigma_0, \sigma_1 \in \mathcal{E}_{\mathcal{L},0}$ 。取 $\tau \in \{a, b\}^*$ ，考虑 $\sigma_0 \hat{\ } \tau$ 和 $\sigma_1 \hat{\ } \tau$ 。分两种情况。若 $\tau \neq \varepsilon$ ，则 $\sigma_0 \hat{\ } \tau$ 和 $\sigma_1 \hat{\ } \tau$ 是否都属于 $\mathcal{L}$ 仅由 τ 的最后一个字符决定，因此 σ_0 和 σ_1 具体是什么并不起作用。若 $\tau = \varepsilon$ ，则 $\sigma_0 \hat{\ } \tau = \sigma_0$ 和 $\sigma_1 \hat{\ } \tau = \sigma_1$ 都属于 $\mathcal{L}$ 。无论哪种情况， $\sigma_0 \hat{\ } \tau \in \mathcal{L}$ 当且仅当 $\sigma_1 \hat{\ } \tau \in \mathcal{L}$ 。

同样的论证适用于 $\mathcal{E}_{\mathcal{L},1} = \{\sigma \in \{a, b\}^* \mid \sigma \text{ 以 } a \text{ 结尾}\}$ 中的串。

IV.B.25 注意，由 $\sigma \in \{a, b\}^*$ ，如果 τ 非空，则 $\sigma \hat{\ } \tau$ 属于该语言当且仅当 τ 以 a 结尾。

(A) 上述观察意味着，任意一对 $\sigma_0, \sigma_1 \in \mathcal{L}$ 都不存在区分扩展，因为 $\sigma_0 \hat{\ } \tau$ 和 $\sigma_1 \hat{\ } \tau$ 属于 $\mathcal{L}$ 当且仅当 τ 为空或以 a 结尾。此外， $\mathcal{L}$ 中的任何串 σ 都不与 $\mathcal{L}$ 之外的串 α 等价，因为 σ 以 a 结尾而 α 不以 a 结尾，因此 ε 就是一个区分扩展。

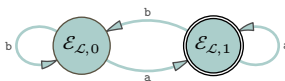
(B) 同理，该观察也意味着，对 $\sigma_0, \sigma_1 \in \mathcal{L}^c$ 这对串，不存在非空区分扩展。而且， $\tau = \varepsilon$ 也不是区分扩展。（我们无需证明 $\mathcal{L}^c$ 中没有任何串与 $\mathcal{L}$ 中的串等价，因为前一小题已证。）

(C) 初始状态是包含空串的类。在这台机器中，空串属于 $\mathcal{E}_{\mathcal{L},0}$ 。

(D) 如果一个类包含 $\mathcal{L}$ 中的串，那么它是接受状态。在这台机器中，只有 $\mathcal{E}_{\mathcal{L},1}$ 是接受状态。

(E) 我们可以选择 $\varepsilon \in \mathcal{E}_{\mathcal{L},0}$ 和长度为 1 的串 $\mathbf{a} \in \mathcal{E}_{\mathcal{L},1}$ 作为类代表。

Δ	$\mathbf{a}$	$\mathbf{b}$
$\varepsilon \in \mathcal{E}_{\mathcal{L},0}$	$\mathbf{a} \in \mathcal{E}_{\mathcal{L},1}$	$\mathbf{b} \in \mathcal{E}_{\mathcal{L},0}$
$\mathbf{+a} \in \mathcal{E}_{\mathcal{L},1}$	$\mathbf{aa} \in \mathcal{E}_{\mathcal{L},1}$	$\mathbf{ab} \in \mathcal{E}_{\mathcal{L},0}$



被接受的串是那些以 $\mathbf{a}$ 结尾的，即 $\mathbf{a}$ 、 $\mathbf{abba}$ 和 $\mathbf{bba}$ 。

IV.B.26 我们遵循例 B.17 之前的讨论：当机器是最小时， $\sim_{\mathcal{L}}$ 类就是 $\sim_{\mathcal{M}}$ 类。为了找到 $\sim_{\mathcal{M}}$ 类，下面四个蕴含关系是显然的。

$$\hat{\Delta}(\sigma) = q_0 \iff \sigma = \varepsilon$$

$$\hat{\Delta}(\sigma) = q_1 \iff \sigma = \mathbf{a}$$

$$\hat{\Delta}(\sigma) = q_2 \iff \sigma \text{ 匹配正则表达式 } \mathbf{aab*}$$

$$\hat{\Delta}(\sigma) = q_3 \iff \sigma \text{ 是其他情况 (具体来说, } \sigma \text{ 匹配 } \mathbf{b|ab|(aab*a(a|b)*)})$$

因此，这四个就是 $\sim_{\mathcal{L}}$ 类。

关联状态	类
q_0	$\mathcal{E}_{\mathcal{L},0} = \{\varepsilon\}$
q_1	$\mathcal{E}_{\mathcal{L},1} = \{\mathbf{a}\}$
q_2	$\mathcal{E}_{\mathcal{L},2} = \mathcal{L}$
q_3	$\mathcal{E}_{\mathcal{L},3} = \{\sigma \in \Sigma^* \mid \sigma \notin (\mathcal{E}_{\mathcal{L},0} \cup \mathcal{E}_{\mathcal{L},1} \cup \mathcal{E}_{\mathcal{L},2})\}$

类 $\mathcal{E}_{\mathcal{L},2}$ 包含 $\mathcal{L}$ 中的串，因此是接受状态。类 $\mathcal{E}_{\mathcal{L},0}$ 包含空串，因此是初始状态。

IV.B.27 我们遵循例 B.17 之前的讨论，即当机器是最小时， $\sim_{\mathcal{L}}$ 类就是 $\sim_{\mathcal{M}}$ 类。下面是识别该语言的一台自然的有限状态机。



它显然是最小的，因此我们找出 $\sim_{\mathcal{M}}$ 类。这些蕴含关系是直接的。

$$\hat{\Delta}(\sigma) = q_0 \iff \sigma \text{ 匹配正则表达式 } \mathbf{a*}$$

$$\hat{\Delta}(\sigma) = q_1 \iff \sigma \text{ 匹配正则表达式 } \mathbf{a*b}$$

$$\hat{\Delta}(\sigma) = q_2 \iff \sigma \text{ 为其他情况 (具体来说, 它匹配 } \mathbf{a*b(a|b)(a|b)*})$$

这些就是 $\sim_{\mathcal{L}}$ 类。

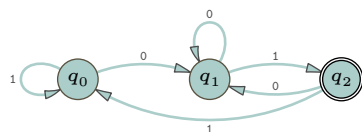
关联状态	等价类
q_0	$\mathcal{E}_{\mathcal{L},0} = \{\sigma \in \Sigma^* \mid \sigma = \mathbf{a}^n, \text{ 其中 } n \in \mathbb{N}\}$
q_1	$\mathcal{E}_{\mathcal{L},1} = \mathcal{L} = \{\sigma \in \Sigma^* \mid \sigma = \mathbf{a}^n \mathbf{b}, \text{ 其中 } n \in \mathbb{N}\}$
q_2	$\mathcal{E}_{\mathcal{L},2} = \{\sigma \in \Sigma^* \mid \sigma = \mathbf{a}^n \mathbf{b} \wedge \beta, \text{ 其中 } n \in \mathbb{N} \text{ 且 } \beta \neq \varepsilon\}$

类 $\mathcal{E}_{\mathcal{L},1}$ 包含 $\mathcal{L}$ 中的串，因此它是接受状态。类 $\mathcal{E}_{\mathcal{L},0}$ 包含空串，因此它是初始状态。

注：虽然给定的有限状态机确实看起来是最小的，但严格来说，我们尚未证明它是最小的。我们可以使用附加题 C 来证明这一点。

IV.B.28

(A) 这是一个正则语言，识别它的一台最自然的有限状态机如下图所示。



我们将这台机器视为最小的。(我们可以使用附加题 C 来验证这一点。) 我们遵循例 B.17 之前的讨论: 当机器最小时, $\sim_{\mathcal{L}}$ 类就是 $\sim_{\mathcal{M}}$ 类。

这些蕴含关系是直接的。

$$\hat{\Delta}(\sigma) = q_2 \iff \sigma \in \mathcal{L} \text{ (即匹配正则表达式 } (0|1)^*01\text{)}$$

$$\hat{\Delta}(\sigma) = q_1 \iff \sigma \text{ 以 } 0 \text{ 结尾 (匹配 } (0|1)^*0\text{)}$$

$$\hat{\Delta}(\sigma) = q_0 \iff \sigma \text{ 为其他情况}$$

它们就是 $\sim_{\mathcal{L}}$ 等价类。

关联状态	等价类
q_2	$\mathcal{E}_{\mathcal{L},2} = \mathcal{L}$
q_1	$\mathcal{E}_{\mathcal{L},1} = \{\sigma \in \Sigma^* \mid \sigma \text{ ends in } 0\}$
q_0	$\mathcal{E}_{\mathcal{L},0} = \{\sigma \in \Sigma^* \mid \sigma \in \Sigma^* - (\mathcal{E}_{\mathcal{L},2} \cup \mathcal{E}_{\mathcal{L},1})\}$

等价类 $\mathcal{E}_{\mathcal{L},2}$ 是唯一包含 $\mathcal{L}$ 中串的类, 因此它是唯一的接受状态。等价类 $\mathcal{E}_{\mathcal{L},0}$ 包含空串, 因此它是初始状态。

- (B) 我们考虑将三个集合作为等价类的候选。我们认为, 其中一个 (这里命名为 $\mathcal{E}_{\mathcal{L},2}$) 就是 $\mathcal{L}$ 本身; 另一个 (命名为 $\mathcal{E}_{\mathcal{L},1}$) 包含那些以 0 结尾的串; 而最后一个集合必然包含所有其余的串, $\mathcal{E}_{\mathcal{L},0} = \Sigma^* - (\mathcal{E}_{\mathcal{L},1} \cup \mathcal{E}_{\mathcal{L},2})$ 。

首先论证 $\mathcal{E}_{\mathcal{L},2} = \mathcal{L}$ 是一个 $\sim_{\mathcal{L}}$ 等价类。假设 $\sigma_0, \sigma_1 \in \mathcal{L}$, 目标是证明 $\sigma_0 \sim_{\mathcal{L}} \sigma_1$ 。这两个串都以 01 结尾。考虑扩展串 $\tau \in \mathbb{B}^*$ 。如果 $\tau = \varepsilon$, 那么 $\sigma_0 \frown \tau \in \mathcal{L}$ 且 $\sigma_1 \frown \tau \in \mathcal{L}$ 。如果 $\tau \neq \varepsilon$, 那么 $\sigma_0 \frown \tau$ 和 $\sigma_1 \frown \tau$ 都属于 $\mathcal{L}$ 当且仅当 τ 以 01 结尾。在这两种情况下, 这对串都没有区分扩展。

接下来, 我们通过证明不存在任何串 $\sigma \in \mathcal{E}_{\mathcal{L},2} = \mathcal{L}$ 与 $\mathcal{L}^c$ 中的串 $\alpha \sim_{\mathcal{L}}$ 等价, 来完成该集合是等价类的证明。这是因为空串 ε 能区分 σ 和 α 。

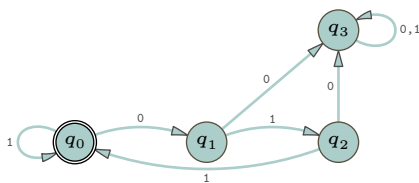
接下来论证以 0 结尾的串组成的集合是一个等价类。考虑任意两个这样的串 $\sigma_0, \sigma_1 \in \mathcal{E}_{\mathcal{L},1}$ 以及一个扩展串 τ 。如果 $\tau = \varepsilon$, 那么 $\sigma_0 \frown \tau$ 和 $\sigma_1 \frown \tau$ 都不是 $\mathcal{L}$ 的元素。如果 τ 的长度为一, 那么 $\sigma_0 \frown \tau$ 和 $\sigma_1 \frown \tau$ 都是 $\mathcal{L}$ 的元素当且仅当 $\tau = 1$ 。如果 τ 的长度大于一, 那么 $\sigma_0 \frown \tau$ 和 $\sigma_1 \frown \tau$ 都是 $\mathcal{L}$ 的元素当且仅当 τ 以 01 结尾。在这三种情况下, τ 都不能区分 σ_0 和 σ_1 。

为了完成“这是一个等价类”的证明, 我们论证: 没有任何 $\sigma \in \mathcal{E}_{\mathcal{L},1}$ 与某个 $\alpha \notin \mathcal{E}_{\mathcal{L},1} \sim_{\mathcal{L}}$ 不可区分。理由是: 任何以 0 结尾的串都不与不以 0 结尾的串等价, 因为 1 就是一个区分扩展。

最后, 考虑剩下的串, 即那些不以 01 结尾且也不以 0 结尾的串。对于任意这样的 $\sigma \in \mathcal{E}_{\mathcal{L},0}$, 扩展 $\sigma \frown \tau$ 属于该语言当且仅当 τ 以 01 结尾。因此, 不存在任何扩展能区分这样的一对串, 它们彼此等价。

我们无需证明 $\mathcal{E}_{\mathcal{L},0}$ 中的任何 σ 不能与 $\mathcal{E}_{\mathcal{L},0}$ 之外的 α 等价, 因为对于前两个类, 我们已经完成了此类证明。

IV.B.29 我们遵循例 B.17 之前的讨论, 即当机器最小时, $\sim_{\mathcal{L}}$ 类就是 $\sim_{\mathcal{M}}$ 类。下面是识别该语言的一台自然有限状态机。



它显然是最小的，这意味着 $\sim_{\mathcal{L}}$ 类等于 $\sim_{\mathcal{M}}$ 类。这些蕴含关系是直接的。

$$\hat{\Delta}(\sigma) = q_0 \iff \sigma \text{ 匹配正则表达式 } (1|011)^*$$

$$\hat{\Delta}(\sigma) = q_1 \iff \sigma \text{ 匹配正则表达式 } (1|011)^*0$$

$$\hat{\Delta}(\sigma) = q_2 \iff \sigma \text{ 匹配正则表达式 } (1|011)^*01$$

$$\hat{\Delta}(\sigma) = q_3 \iff \sigma \text{ 属于其他情况；它不匹配 } (1|011)^*(\varepsilon|0|01)$$

这些就是 $\sim_{\mathcal{L}}$ 类。

对应状态	等价类
q_0	$\mathcal{E}_{\mathcal{L},0} = \mathcal{L}$
q_1	$\mathcal{E}_{\mathcal{L},1} = \{\sigma \in \Sigma^* \mid \sigma \text{ 匹配 } (1 011)^*0\}$
q_2	$\mathcal{E}_{\mathcal{L},2} = \{\sigma \in \Sigma^* \mid \sigma \text{ 匹配 } (1 011)^*01\}$
q_3	$\mathcal{E}_{\mathcal{L},3} = \{\sigma \in \Sigma^* \mid \sigma \text{ does not match } (1 011)^*(\varepsilon 0 01)\}$

类 $\mathcal{E}_{\mathcal{L},0}$ 是唯一的接受类，因为它是唯一包含 $\mathcal{L}$ 中字符串的类。类 $\mathcal{E}_{\mathcal{L},0}$ 也包含空串，所以它是初始类。

IV.B.30 当然有很多可能的答案。一种合理的方法是考虑这样一个正则语言：在为其构造有限状态机时，需要两个或更多自然的接受状态。

考虑 $\mathcal{L} \subseteq \{a, b\}^*$ 包含所有以 **a** 结尾的字符串，以及所有长度为二的字符串。那么字符串 **a** 和 **bb** 都是 $\mathcal{L}$ 的成员，但它们有一个区分扩展 $\tau = b$ ，所以集合 $\mathcal{L}$ 至少分裂成两个等价类。

IV.B.31 我们将通过证明存在无穷多个不等价的字符串，来证明存在无穷多个 $\sim_{\mathcal{L}}$ 等价类。

我们断言没有两个不相等的形如 a^n 的字符串是等价的。举例来说，考虑 $\sigma_2 = aa$ 和 $\sigma_3 = aaa$ 。

取 $\tau = baa$ 。因为 $\sigma_2 \wedge \tau \in \mathcal{L}$ 且 $\sigma_3 \wedge \tau \notin \mathcal{L}$ ，所以两者不等价。显然这对任何两个这样的字符串都成立。

IV.B.32 这些类是：**a** 和 **b** 数量相同的字符串集合，**a** 比 **b** 多一个的，**b** 比 **a** 多一个的，**a** 比 **b** 多两个的，等等。我们不必证明这一点，只需要证明存在无穷多个可区分的字符串。

因此，令 $n(\sigma)$ 为 **a** 的数量减去 **b** 的数量。它可能是正数、负数或零。假设 $n(\sigma_0) \neq n(\sigma_1)$ 对于 $\sigma_0, \sigma_1 \in \{a, b\}^*$ 。那么区分串是任何 $\tau \in \{a, b\}^*$ ，其中 $n(\tau) = -1 \cdot n(\sigma_0)$ 。

IV.B.33 我们将证明存在无穷多个不等价的字符串。

$$\varepsilon \quad 0 \quad 00 \quad 000 \quad \dots$$

如果 $i \neq j$ ，则 $0^i \not\sim_{\mathcal{L}} 0^j$ ，因为 $\tau = 1^i$ 是一个区分扩展。这是因为 $0^i \wedge 1^i$ 是 $\mathcal{L}$ 的成员，但 $0^j \wedge 1^i$ 不是。

IV.B.34 我们将证明，该语言中的每个串都位于其自身的 $\sim_{\mathcal{L}}$ 等价类中，因此这样的类有无穷多个。

考虑 $\sigma_0 = 0^{2^n}$ 和 $\sigma_1 = 0^{2^m}$ ，其中 $n \neq m$ 。其中一个必然更小；假设 $n < m$ 。那么，扩展串 $\tau = 0^{2^n}$ 使得 $\sigma_0 \wedge \tau = 0^{2^n} \wedge 0^{2^n} = 0^{2 \cdot 2^n} = 0^{2^{n+1}}$ 是该语言的一个元素。但由于 m 大于 n ，串 $\sigma_1 \wedge \tau$ 不是该语言的元素（串中 0 的个数是 $2^m + 2^n$ ，由于 n 严格小于 m ，该数严格小于 $2^{2(m+1)}$ ）。因此，它们存在一个区分扩展。

IV.B.35 对于两个证明，都将语言记为 $\mathcal{L}$ ，字母表记为 Σ 。

(A) 根据定义， $\sigma_0 \sim_{\mathcal{L}} \sigma_1$ 当且仅当不存在区分扩展 τ 。这成立当且仅当 $\sigma_0 \wedge \tau$ 和 $\sigma_1 \wedge \tau$ 要么都是 $\mathcal{L}$ 的元素，要么都不是。

关系 $\sim_{\mathcal{L}}$ 是自反的，因为对任何串 $\sigma \in \Sigma^*$ ， $\sigma \wedge \tau$ 和 $\sigma \wedge \tau$ 要么都在该语言中，要么都不在。

为验证关系是对称的，假设 $\sigma_0 \sim \sigma_1$ 。那么对任何 $\tau \in \Sigma^*$ ， $\sigma_0 \wedge \tau$ 和 $\sigma_1 \wedge \tau$ 要么都在该语言中，要么都不在。由“与”一词的对称性， $\sigma_1 \wedge \tau$ 和 $\sigma_0 \wedge \tau$ 也要么都在该语言中，要么都不在。这等价于 $\sigma_1 \sim \sigma_0$ 。

最后，对于传递性，假设 $\sigma_0 \sim \sigma_1$ 且 $\sigma_1 \sim \sigma_2$ 。令 τ 为一个串。 $\sigma_1 \wedge \tau$ 要么属于 $\mathcal{L}$ ，要么不属于。如果 $\sigma_1 \wedge \tau \in \mathcal{L}$ ，则 $\sigma_0 \sim \sigma_1$ 蕴含 $\sigma_0 \wedge \tau \in \mathcal{L}$ ，且 $\sigma_1 \sim \sigma_2$ 蕴含 $\sigma_2 \wedge \tau \in \mathcal{L}$ ，因此 $\sigma_0 \wedge \tau$ 和 $\sigma_2 \wedge \tau$ 都是该语言的元素。类似地，如果 $\sigma_1 \wedge \tau \notin \mathcal{L}$ ，则 $\sigma_0 \wedge \tau$ 和 $\sigma_2 \wedge \tau$ 都不是该语言的元素。因此， $\sigma_0 \sim_{\mathcal{L}} \sigma_2$ 。

(B) 根据定义, $\sigma_0 \sim_{\mathcal{M}} \sigma_1$ 当且仅当 $\hat{\Delta}(\sigma_0) = \hat{\Delta}(\sigma_1)$ 。关系 $\sim_{\mathcal{M}}$ 是自反的, 因为对任意串 $\sigma \in \Sigma^*$, $\hat{\Delta}(\sigma) = \hat{\Delta}(\sigma)$ 。

为验证关系是对称的, 假设 $\sigma_0 \sim_{\mathcal{M}} \sigma_1$ 。那么 $\hat{\Delta}(\sigma_0) = \hat{\Delta}(\sigma_1)$, 进而由等式的对称性, $\hat{\Delta}(\sigma_1) = \hat{\Delta}(\sigma_0)$ 。因此, $\sigma_1 \sim_{\mathcal{M}} \sigma_0$ 。

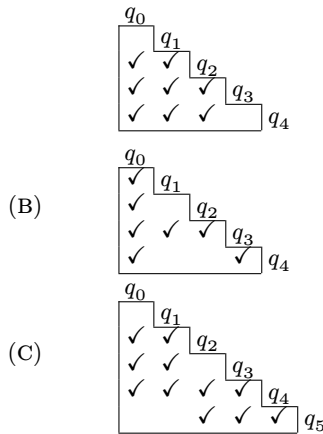
对于传递性, 假设 $\sigma_0 \sim_{\mathcal{M}} \sigma_1$ 且 $\sigma_1 \sim_{\mathcal{M}} \sigma_2$ 。第一个假设给出 $\hat{\Delta}(\sigma_0) = \hat{\Delta}(\sigma_1)$, 而第二个给出 $\hat{\Delta}(\sigma_1) = \hat{\Delta}(\sigma_2)$ 。因此, $\hat{\Delta}(\sigma_0) = \hat{\Delta}(\sigma_2)$, 并且 $\sigma_0 \sim_{\mathcal{M}} \sigma_2$ 。

IV.B.36 两个串 $\sigma_0, \sigma_1 \in \Sigma^*$ 是 $\mathcal{L}$ -不可区分的, 当且仅当对每个扩展 τ , $\sigma_0 \hat{\sim} \tau$ 与 $\sigma_1 \hat{\sim} \tau$ 要么都是 $\mathcal{L}$ 的元素, 要么都是 $\mathcal{L}^c$ 的元素。这等价于它们是 $\mathcal{L}^c$ -不可区分的。

IV.C.8 在 q_0, q_1 、 q_0, q_2 和 q_1, q_2 条目处有一簇空白单元格。在 q_3, q_5 处也有一处空白。没有与 q_4 相关的空白, 所以它单独构成一个类。等价类是 $\mathcal{E}_{i,0} = \{q_0, q_1, q_2\}$ 、 $\mathcal{E}_{i,1} = \{q_3, q_5\}$ 和 $\mathcal{E}_{i,2} = \{q_4\}$ 。

IV.C.9

(A) 给定的类说明我们可以 i -区分 q_0 与 q_2, q_3 以及 q_4 。所以, 顺着以 q_0 为表头的列向下看, 我们在标为 q_2, q_3 和 q_4 的行里打钩。其他列同理。



IV.C.10

(A) 对于 q_0, q_1 、 q_0, q_2 、 q_1, q_5 和 q_2, q_5 这几对, 它们的类不同 (差异均出现在 b 列上)。

(B) 相应的 $\sim_1$ 类是 $\mathcal{E}_{1,0} = \{q_0, q_5\}$ 、 $\mathcal{E}_{1,1} = \{q_1, q_2\}$ 、以及 $\mathcal{E}_{1,2} = \{q_3, q_4\}$ 。

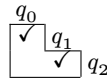
IV.C.11 答案是“是的。”最小机器的初始状态是包含 q_0 的那个状态。在极小化后的机器中, 一个状态是接受状态当且仅当它包含原机器中的某个接受状态。所以, 如果 q_0 是接受状态, 那么最小机器的初始状态也是接受状态。

IV.C.12 经观察, 所有状态都是可达的。

第 0 步是将接受状态与非接受状态分到不同的类中。

$$\mathcal{E}_{0,0} = \{q_0, q_2\} \quad \mathcal{E}_{0,1} = \{q_1\}$$

我们同时构造初始的三角表, 在 q_i, q_j 条目处打钩, 条件是 q_i 与 q_j 中一个是接受状态而另一个不是。



打钩表示状态对是 0-可区分的, 而空白表示状态对是 0-不可区分的。

第 1 步检查是否有 $\sim_0$ 类发生分裂。计算如下。

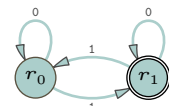
	0	1
q_0, q_2	$q_0 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$	$q_1 \in \mathcal{E}_{0,1}, q_1 \in \mathcal{E}_{0,1}$

两个类都没有分裂。算法的第一阶段停止, 得到两个 $\sim$ 等价类。

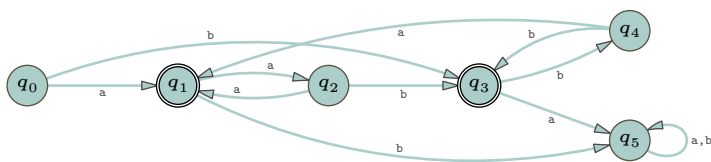
$$\mathcal{E}_0 = \{q_0, q_2\} \quad \mathcal{E}_1 = \{q_1\}$$

结合原始机器的信息, 得到这台极小化后的机器。

	a	b		a	b
q_0	$q_0 \in \mathcal{E}_0$	$q_1 \in \mathcal{E}_1$	r_0	r_0	r_1
q_2	$q_2 \in \mathcal{E}_0$	$q_1 \in \mathcal{E}_1$	r_1	r_1	r_0
$+ q_1$	$q_1 \in \mathcal{E}_1$	$q_2 \in \mathcal{E}_0$			



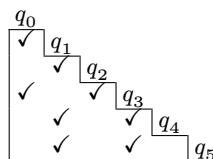
IV.C.13 作为参考，机器如下。



通过观察可知，每个状态都是可达的。

步骤 0 将状态分为两个 $\sim_0$ 等价类，一类是非接受状态，另一类是接受状态。同时给出了相应的三角表，标记了 0-可区分状态，即 q_i 和 q_j 中一个是接受状态而另一个不是。

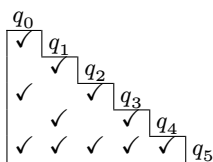
$\mathcal{E}_{0,0} = \{q_0, q_2, q_4, q_5\}$	$\mathcal{E}_{0,1} = \{q_1, q_3\}$
--	------------------------------------



步骤 1 检查这两个 $\sim_0$ 等价类是否会分裂。计算如下。

	a	b
q_0, q_2	$q_1 \in \mathcal{E}_{0,1}, q_1 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,1}, q_3 \in \mathcal{E}_{0,1}$
q_0, q_4	$q_1 \in \mathcal{E}_{0,1}, q_1 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,1}, q_3 \in \mathcal{E}_{0,1}$
q_0, q_5	$q_1 \in \mathcal{E}_{0,1}, q_5 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,1}, q_5 \in \mathcal{E}_{0,0}$
q_2, q_4	$q_1 \in \mathcal{E}_{0,1}, q_1 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,1}, q_3 \in \mathcal{E}_{0,1}$
q_2, q_5	$q_1 \in \mathcal{E}_{0,1}, q_5 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,1}, q_5 \in \mathcal{E}_{0,0}$
q_4, q_5	$q_1 \in \mathcal{E}_{0,1}, q_5 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,1}, q_5 \in \mathcal{E}_{0,0}$
q_1, q_3	$q_2 \in \mathcal{E}_{0,0}, q_5 \in \mathcal{E}_{0,0}$	$q_5 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,0}$

由此得到三角表和 $\sim_1$ 等价类，表明 $\mathcal{E}_{0,0}$ 分裂成两个。

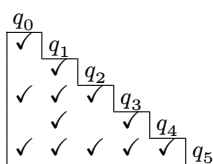


$$\mathcal{E}_{1,0} = \{q_0, q_2, q_4\} \quad \mathcal{E}_{1,1} = \{q_5\} \quad \mathcal{E}_{1,2} = \{q_1, q_3\}$$

接下来是步骤 2。

	a	b
q_0, q_2	$q_1 \in \mathcal{E}_{1,2}, q_1 \in \mathcal{E}_{1,2}$	$q_3 \in \mathcal{E}_{1,2}, q_3 \in \mathcal{E}_{1,2}$
q_0, q_4	$q_1 \in \mathcal{E}_{1,2}, q_1 \in \mathcal{E}_{1,2}$	$q_3 \in \mathcal{E}_{1,2}, q_3 \in \mathcal{E}_{1,2}$
q_2, q_4	$q_1 \in \mathcal{E}_{1,2}, q_1 \in \mathcal{E}_{1,2}$	$q_3 \in \mathcal{E}_{1,2}, q_3 \in \mathcal{E}_{1,2}$
q_1, q_3	$q_2 \in \mathcal{E}_{1,0}, q_5 \in \mathcal{E}_{1,1}$	$q_5 \in \mathcal{E}_{1,1}, q_4 \in \mathcal{E}_{1,0}$

这个三角表概括了上述计算，表明 $\mathcal{E}_{1,2}$ 发生分裂。



$$\mathcal{E}_{2,0} = \{q_0, q_2, q_4\} \quad \mathcal{E}_{2,1} = \{q_5\} \quad \mathcal{E}_{2,2} = \{q_1\} \quad \mathcal{E}_{2,3} = \{q_3\}$$

步骤 3 的计算表明没有进一步的分裂。

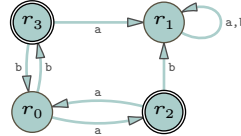
	a	b
q_0, q_2	$q_1 \in \mathcal{E}_{2,2}, q_1 \in \mathcal{E}_{2,2}$	$q_3 \in \mathcal{E}_{2,3}, q_3 \in \mathcal{E}_{2,3}$
q_0, q_4	$q_1 \in \mathcal{E}_{2,2}, q_1 \in \mathcal{E}_{2,2}$	$q_3 \in \mathcal{E}_{2,3}, q_3 \in \mathcal{E}_{2,3}$
q_2, q_4	$q_1 \in \mathcal{E}_{2,2}, q_1 \in \mathcal{E}_{2,2}$	$q_3 \in \mathcal{E}_{2,3}, q_3 \in \mathcal{E}_{2,3}$

这些就是 $\sim$ 等价类。

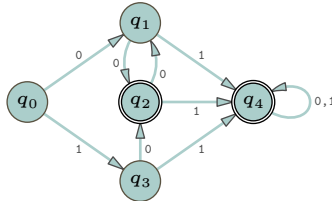
$$\mathcal{E}_0 = \{q_0, q_2, q_4\} \quad \mathcal{E}_1 = \{q_5\} \quad \mathcal{E}_2 = \{q_1\} \quad \mathcal{E}_3 = \{q_3\}$$

Moore 算法的第二阶段将这些等价类作为状态，产生如下最小的机器。

	a	b		a	b
q_0	$q_1 \in \mathcal{E}_2$	$q_3 \in \mathcal{E}_3$	r_0	r_2	r_3
q_2	$q_1 \in \mathcal{E}_2$	$q_3 \in \mathcal{E}_3$	r_1	r_1	r_1
q_4	$q_1 \in \mathcal{E}_2$	$q_3 \in \mathcal{E}_3$	$+ r_2$	r_0	r_1
q_5	$q_5 \in \mathcal{E}_1$	$q_5 \in \mathcal{E}_1$	$+ r_3$	r_1	r_0
$+ q_1$	$q_4 \in \mathcal{E}_0$	$q_5 \in \mathcal{E}_1$			
$+ q_3$	$q_5 \in \mathcal{E}_1$	$q_4 \in \mathcal{E}_0$			



IV.C.14 作为参考，初始机器如下。



通过观察可知，所有状态都是可达的。

步骤 0 将状态分成两类：接受状态和其他状态。我们还给出了初始的三角表，该表在 q_i, q_j 表项上打勾，其中 q_i 和 q_j 一个是接受状态而另一个不是。

$\mathcal{E}_{0,0} = \{q_0, q_1, q_3\}$	q_0
$\mathcal{E}_{0,1} = \{q_2, q_4\}$	q_1
	q_2
	q_3
	q_4

接着，步骤 1 检查 $\sim_0$ 等价类中是否有类发生分裂。首先是计算部分。

	0	1
q_0, q_1	$q_1 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,1}$
q_0, q_3	$q_1 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,1}$
q_1, q_3	$q_2 \in \mathcal{E}_{0,1}, q_2 \in \mathcal{E}_{0,1}$	$q_4 \in \mathcal{E}_{0,1}, q_4 \in \mathcal{E}_{0,1}$
q_2, q_4	$q_1 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,1}$	$q_4 \in \mathcal{E}_{0,1}, q_4 \in \mathcal{E}_{0,1}$

分裂情况如下：那些 0-不可区分但 1-可区分状态对是 q_0, q_1 、 q_0, q_3 和 q_2, q_4 。三角表反映了这些变化，空白单元格表明现在有四个 $\sim_1$ 等价类。

q_0	$\mathcal{E}_{1,0} = \{q_0\}$
q_1	$\mathcal{E}_{1,1} = \{q_1, q_3\}$
q_2	$\mathcal{E}_{1,2} = \{q_2\}$
q_3	$\mathcal{E}_{1,3} = \{q_4\}$
q_4	

步骤 2 判断 $\sim_1$ 等价类中是否有类分裂。

	0	1
q_1, q_3	$q_2 \in \mathcal{E}_{1,2}, q_2 \in \mathcal{E}_{1,2}$	$q_4 \in \mathcal{E}_{1,3}, q_4 \in \mathcal{E}_{1,3}$

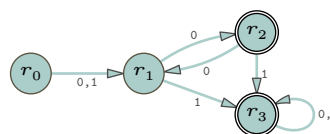
没有发生进一步分裂。这些就是 $\sim$ 等价类。

$$\mathcal{E}_0 = \{q_0\} \quad \mathcal{E}_1 = \{q_1, q_3\} \quad \mathcal{E}_2 = \{q_2\} \quad \mathcal{E}_3 = \{q_4\}$$

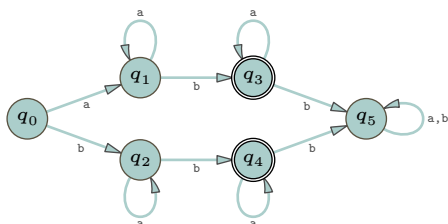
第二阶段确定转移函数。

	0	1
q_0	$q_1 \in \mathcal{E}_1$	$q_3 \in \mathcal{E}_1$
q_1	$q_2 \in \mathcal{E}_2$	$q_4 \in \mathcal{E}_3$
q_3	$q_2 \in \mathcal{E}_2$	$q_4 \in \mathcal{E}_3$
$+ q_2$	$q_1 \in \mathcal{E}_1$	$q_4 \in \mathcal{E}_3$
$+ q_4$	$q_4 \in \mathcal{E}_3$	$q_4 \in \mathcal{E}_3$

	0	1
r_0	r_1	r_1
r_1	r_2	r_3
$+ r_2$	r_1	r_3
$+ r_3$	r_3	r_3



IV.C.15 肉眼观察，所有状态都是可达的。这是初始机器，供参考。



步骤 0 将所有状态的集合分裂成接受状态类和非接受状态类。

$$\mathcal{E}_{0,0} = \{q_0, q_1, q_2, q_5\} \quad \mathcal{E}_{0,1} = \{q_3, q_4\}$$

我们还给出初始三角表，对 q_i 和 q_j 中一个是接受状态而另一个不是的 q_i, q_j 项打上勾号。

q_0						
	q_1					
		q_2				
			q_3			
				q_4		
					q_5	

步骤 1 计算是否有状态对是 1-可区分的，但不是 0-可区分的。

	a	b
q_0, q_1	$q_1 \in \mathcal{E}_{0,0}, q_1 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}, q_3 \in \mathcal{E}_{0,1}$
q_0, q_2	$q_1 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,1}$
q_0, q_5	$q_1 \in \mathcal{E}_{0,0}, q_5 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}, q_5 \in \mathcal{E}_{0,0}$
q_1, q_2	$q_1 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,1}, q_4 \in \mathcal{E}_{0,1}$
q_1, q_5	$q_1 \in \mathcal{E}_{0,0}, q_5 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,1}, q_5 \in \mathcal{E}_{0,0}$
q_2, q_5	$q_2 \in \mathcal{E}_{0,0}, q_5 \in \mathcal{E}_{0,0}$	$q_4 \in \mathcal{E}_{0,1}, q_5 \in \mathcal{E}_{0,0}$
q_3, q_4	$q_3 \in \mathcal{E}_{0,1}, q_4 \in \mathcal{E}_{0,1}$	$q_5 \in \mathcal{E}_{0,0}, q_5 \in \mathcal{E}_{0,0}$

该计算区分了 q_0, q_1 、 q_0, q_2 、 q_1, q_5 和 q_2, q_5 。以下是更新后的三角表，以及由空白单元格给出的 $\sim_1$ 等价类。

q_0						
	q_1					
		q_2				
			q_3			
				q_4		
					q_5	

$$\begin{aligned} \mathcal{E}_{1,0} &= \{q_0, q_5\} \\ \mathcal{E}_{1,1} &= \{q_1, q_2\} \\ \mathcal{E}_{1,2} &= \{q_3, q_4\} \end{aligned}$$

等价类 $\mathcal{E}_{0,0}$ 分裂成了两个。

迭代。步骤 2 计算这些 $\sim_1$ 等价类中是否有发生分裂的。

	a	b
q_0, q_5	$q_1 \in \mathcal{E}_{1,1}, q_5 \in \mathcal{E}_{1,0}$	$q_2 \in \mathcal{E}_{1,1}, q_5 \in \mathcal{E}_{1,0}$
q_1, q_2	$q_1 \in \mathcal{E}_{1,1}, q_2 \in \mathcal{E}_{1,1}$	$q_3 \in \mathcal{E}_{1,2}, q_4 \in \mathcal{E}_{1,2}$
q_3, q_4	$q_3 \in \mathcal{E}_{1,2}, q_4 \in \mathcal{E}_{1,2}$	$q_5 \in \mathcal{E}_{1,0}, q_5 \in \mathcal{E}_{1,0}$

唯一一个 2-可区分但不是 1-可区分的状态对是 q_0, q_5 。三角表反映了这个新增的勾号。

q_0	$\checkmark$	q_1								
q_1	$\checkmark$		q_2							
q_2	$\checkmark$	$\checkmark$		q_3						
q_3	$\checkmark$	$\checkmark$	$\checkmark$		q_4					
q_4	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$		q_5				
q_5	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$					

$$\begin{aligned}\mathcal{E}_{2,0} &= \{q_0\} \\ \mathcal{E}_{2,1} &= \{q_1, q_2\} \\ \mathcal{E}_{2,2} &= \{q_3, q_4\} \\ \mathcal{E}_{2,3} &= \{q_5\}\end{aligned}$$

再次。步骤 3 的计算如下。

	a	b
q_1, q_2	$q_1 \in \mathcal{E}_{2,1}, q_2 \in \mathcal{E}_{2,1}$	$q_3 \in \mathcal{E}_{2,2}, q_4 \in \mathcal{E}_{2,2}$
q_3, q_4	$q_3 \in \mathcal{E}_{2,2}, q_4 \in \mathcal{E}_{2,2}$	$q_5 \in \mathcal{E}_{2,3}, q_5 \in \mathcal{E}_{2,3}$

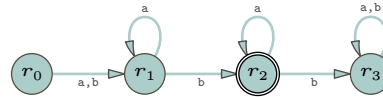
没有发生分裂。Moore 算法的第一阶段终止。这些就是 $\sim$ 等价类。

$$r_0 = \mathcal{E}_0 = \{q_0\} \quad r_1 = \mathcal{E}_1 = \{q_1, q_2\} \quad r_2 = \mathcal{E}_2 = \{q_3, q_4\} \quad r_3 = \mathcal{E}_3 = \{q_5\}$$

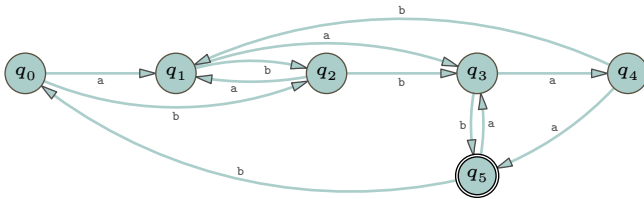
至于第二阶段，以下是最小化后的机器。

	a	b
q_0	$q_1 \in \mathcal{E}_1$	$q_2 \in \mathcal{E}_1$
q_1	$q_1 \in \mathcal{E}_1$	$q_3 \in \mathcal{E}_2$
q_2	$q_2 \in \mathcal{E}_1$	$q_4 \in \mathcal{E}_2$
+ q_3	$q_3 \in \mathcal{E}_2$	$q_5 \in \mathcal{E}_3$
+ q_4	$q_4 \in \mathcal{E}_2$	$q_5 \in \mathcal{E}_3$
q_5	$q_5 \in \mathcal{E}_3$	$q_5 \in \mathcal{E}_3$

	a	b
r_0	r_1	r_1
r_1	r_1	r_2
+ r_2	r_2	r_3
r_3	r_3	r_3



IV.C.16 应用该算法后，我们会发现所有状态都是可区分的。下面给出这台机器作为参考。



显然，所有状态都是可达的。

步骤 0 将接受状态与非接受状态分开。

$\mathcal{E}_{0,0} = \{q_0, q_1, q_2, q_3, q_4\}$	$\mathcal{E}_{0,1} = \{q_5\}$
---	-------------------------------

q_0	q_1	q_2	q_3	q_4	q_5
$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$

步骤 1 的表格太大，已单独列为 Figure 96 on page 167。该计算发现，虽然这些状态对是 0-不可区分的，但它们却是 1-可区分的： q_0, q_3 、 q_0, q_4 、 q_1, q_3 、 q_1, q_4 、 q_2, q_3 、 q_2, q_4 以及 q_3, q_4 。这次计算得到了如下三角表与等价类集合。

q_0	q_1	q_2	q_3	q_4	q_5
$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$

$$\mathcal{E}_{1,0} = \{q_0, q_1, q_2\} \quad \mathcal{E}_{1,1} = \{q_3\} \quad \mathcal{E}_{1,2} = \{q_4\} \quad \mathcal{E}_{1,3} = \{q_5\}$$

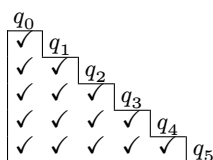
	a	b
q_0, q_1	$q_1 \in \mathcal{E}_{0,0}, q_3 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$
q_0, q_2	$q_1 \in \mathcal{E}_{0,0}, q_1 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}, q_3 \in \mathcal{E}_{0,0}$
q_0, q_3	$q_1 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}, q_5 \in \mathcal{E}_{0,1}$
q_0, q_4	$q_1 \in \mathcal{E}_{0,0}, q_5 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,0}, q_1 \in \mathcal{E}_{0,0}$
q_1, q_2	$q_3 \in \mathcal{E}_{0,0}, q_1 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}, q_3 \in \mathcal{E}_{0,0}$
q_1, q_3	$q_3 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}, q_5 \in \mathcal{E}_{0,1}$
q_1, q_4	$q_3 \in \mathcal{E}_{0,0}, q_5 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,0}, q_1 \in \mathcal{E}_{0,0}$
q_2, q_3	$q_1 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,0}, q_5 \in \mathcal{E}_{0,1}$
q_2, q_4	$q_1 \in \mathcal{E}_{0,0}, q_5 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,0}, q_1 \in \mathcal{E}_{0,0}$
q_3, q_4	$q_4 \in \mathcal{E}_{0,0}, q_5 \in \mathcal{E}_{0,1}$	$q_5 \in \mathcal{E}_{0,1}, q_1 \in \mathcal{E}_{0,0}$

图 g6, FOR QUESTION IV.C.16: 习题 C.16 的第 (1) 步。

步骤 2 的计算要小得多。

	a	b
q_0, q_1	$q_1 \in \mathcal{E}_{1,0}, q_3 \in \mathcal{E}_{1,1}$	$q_2 \in \mathcal{E}_{1,0}, q_2 \in \mathcal{E}_{1,0}$
q_0, q_2	$q_1 \in \mathcal{E}_{1,0}, q_1 \in \mathcal{E}_{1,0}$	$q_2 \in \mathcal{E}_{1,0}, q_3 \in \mathcal{E}_{1,1}$
q_1, q_2	$q_3 \in \mathcal{E}_{1,1}, q_1 \in \mathcal{E}_{1,0}$	$q_2 \in \mathcal{E}_{1,0}, q_3 \in \mathcal{E}_{1,1}$

这次计算 2-区分了所有三对状态： q_0, q_1 、 q_0, q_2 和 q_1, q_2 。结果是三角表被完全填满，且每个状态都单独构成一个等价类。



$$\mathcal{E}_{2,0} = \{q_0\} \quad \mathcal{E}_{2,1} = \{q_1\} \quad \mathcal{E}_{2,2} = \{q_2\} \quad \mathcal{E}_{2,3} = \{q_3\} \quad \mathcal{E}_{2,4} = \{q_4\} \quad \mathcal{E}_{2,5} = \{q_5\}$$

由于没有不可区分的状态可以合并，原始机器就是最小的。

IV.C.17 我们将逐步执行算法，但在开始之前，请注意，答案必然是：一台没有接受状态的机器不接受任何串。这样的机器的最小化版本只有一个状态，即初始状态，且所有箭头都是自循环。因此我们在开始之前就已经知道答案应该是什么，现在只是验证算法确实能给出正确的结果。

经检查，所有状态都是可达的。步骤 0 将状态集合划分为接受状态和其他状态。这里我们还包含了初始的三角表。

$$\mathcal{E}_{0,0} = \{q_0, q_1, q_2, q_3\}$$

关键在于，这里只有一个 $\sim_0$ -等价类，而不是我们在其他最小化例子中看到的两个类。

步骤 1 检查这些类是否会分裂。

	0	1
q_0, q_1	$q_1 \in \mathcal{E}_{0,0}, q_1 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}, q_3 \in \mathcal{E}_{0,0}$
q_0, q_2	$q_1 \in \mathcal{E}_{0,0}, q_0 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$
q_0, q_3	$q_1 \in \mathcal{E}_{0,0}, q_0 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$
q_1, q_2	$q_1 \in \mathcal{E}_{0,0}, q_0 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$
q_1, q_3	$q_1 \in \mathcal{E}_{0,0}, q_0 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$
q_2, q_3	$q_0 \in \mathcal{E}_{0,0}, q_0 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$

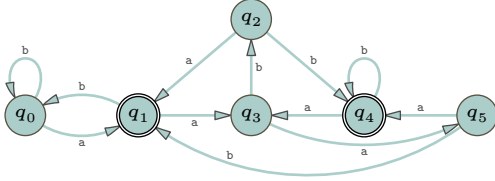
该计算没有 1-区分任何状态对（怎么可能区分呢？只有一个类）。因此 Moore 算法的第一阶段就此停止。

在第二阶段中只有一个类，所以得到的最小化机器如下，正如所预测的。



如果我们从一个所有状态都是接受状态的机器开始，那么类似的情况也会发生。这样的机器识别语言 Σ^* ，其最小化版本只有一个状态，该状态既是初始状态又是接受状态。同上面一样，算法永远不会区分状态对，因为只有一个类，即接受状态类。

IV.C.18 下面是起始机器的参考图示。



经检查，它的所有状态都是可达的。

步骤 0 将状态分成两个类：接受状态和非接受状态。

$$\mathcal{E}_{0,0} = \{q_0, q_2, q_3, q_5\} \quad \mathcal{E}_{0,1} = \{q_1, q_4\}$$

下面是初始的三角表，当 q_i 和 q_j 中一个是接受状态而另一个不是时，条目 q_i, q_j 会打勾。

q_0	q_1	q_2	q_3	q_4	q_5
✓					
	✓				
		✓			
			✓		
				✓	
					✓

步骤 1 计算这些类是否分裂。

	a	b
q_0, q_2	$q_1 \in \mathcal{E}_{0,1}, q_1 \in \mathcal{E}_{0,1}$	$q_0 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,1}$
q_0, q_3	$q_1 \in \mathcal{E}_{0,1}, q_5 \in \mathcal{E}_{0,0}$	$q_0 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$
q_0, q_5	$q_1 \in \mathcal{E}_{0,1}, q_4 \in \mathcal{E}_{0,1}$	$q_0 \in \mathcal{E}_{0,0}, q_1 \in \mathcal{E}_{0,1}$
q_2, q_3	$q_1 \in \mathcal{E}_{0,1}, q_5 \in \mathcal{E}_{0,0}$	$q_4 \in \mathcal{E}_{0,1}, q_2 \in \mathcal{E}_{0,0}$
q_2, q_5	$q_1 \in \mathcal{E}_{0,1}, q_4 \in \mathcal{E}_{0,1}$	$q_4 \in \mathcal{E}_{0,1}, q_1 \in \mathcal{E}_{0,1}$
q_3, q_5	$q_5 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,0}, q_1 \in \mathcal{E}_{0,1}$
q_1, q_4	$q_3 \in \mathcal{E}_{0,0}, q_3 \in \mathcal{E}_{0,0}$	$q_0 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,1}$

在这个计算中，我们发现在对应于状态对 q_0, q_2 、 q_0, q_3 、 q_0, q_5 、 q_2, q_3 、 q_3, q_5 以及 q_1, q_4 的行条目里，后继状态进入了不同的类。因此，除了 q_2, q_5 之外，所有的状态对都是可区分的。更新三角表并分裂这些类。

q_0	q_1	q_2	q_3	q_4	q_5
✓					
✓	✓				
✓	✓	✓			
✓	✓	✓	✓		
✓	✓	✓	✓	✓	
✓	✓	✓	✓	✓	✓

$\mathcal{E}_{1,0} = \{q_0\}$
 $\mathcal{E}_{1,1} = \{q_1\}$
 $\mathcal{E}_{1,2} = \{q_2, q_5\}$
 $\mathcal{E}_{1,3} = \{q_3\}$
 $\mathcal{E}_{1,4} = \{q_4\}$

最后，步骤 2 计算剩下的那个多状态类是否分裂。

	a	b
q_2, q_5	$q_1 \in \mathcal{E}_{1,1}, q_4 \in \mathcal{E}_{1,4}$	$q_4 \in \mathcal{E}_{1,4}, q_1 \in \mathcal{E}_{1,1}$

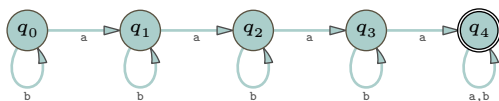
发生了分裂，因此所有状态都是 2-可区分的。每个 $\sim$ 等价类都是单点集。

$$\mathcal{E}_0 = \{q_0\} \quad \mathcal{E}_1 = \{q_1\} \quad \mathcal{E}_2 = \{q_2\} \quad \mathcal{E}_3 = \{q_3\} \quad \mathcal{E}_4 = \{q_4\} \quad \mathcal{E}_5 = \{q_5\}$$

原机器本身就是最小的。

IV.C.19 我们会得到一台识别相同语言的机器。这台机器的可达状态将构成一个最小集。但是仍然会存在不可达状态（不过数量可能会减少，因为其中一些可能会合并在一起）。

IV.C.20 这是初始机器。



通过观察，所有状态都是可达的。

第 0 步将状态集合划分为两类：接受状态和非接受状态。初始的三角表会在那些 q_i, q_j 单元格打勾，其中 q_i 和 q_j 中一个是接受状态而另一个不是。

$$\begin{aligned}\mathcal{E}_{0,0} &= \{q_0, q_1, q_2, q_3\} \\ \mathcal{E}_{0,1} &= \{q_4\}\end{aligned}$$

第 1 步计算 $\sim_0$ 类中是否有任何类分裂。

	a	b
q_0, q_1	$q_1 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$	$q_0 \in \mathcal{E}_{0,0}, q_1 \in \mathcal{E}_{0,0}$
q_0, q_2	$q_1 \in \mathcal{E}_{0,0}, q_3 \in \mathcal{E}_{0,0}$	$q_0 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$
q_0, q_3	$q_1 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,1}$	$q_0 \in \mathcal{E}_{0,0}, q_3 \in \mathcal{E}_{0,0}$
q_1, q_2	$q_2 \in \mathcal{E}_{0,0}, q_3 \in \mathcal{E}_{0,0}$	$q_1 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$
q_1, q_3	$q_2 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,1}$	$q_1 \in \mathcal{E}_{0,0}, q_3 \in \mathcal{E}_{0,0}$
q_2, q_3	$q_3 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,0}, q_3 \in \mathcal{E}_{0,0}$

我们必须在这几个三角表单元格中打勾： q_0, q_3 、 q_1, q_3 和 q_2, q_3 。简言之， q_3 与所有其他状态都是 1-可区分的。

$$\begin{aligned}\mathcal{E}_{1,0} &= \{q_0, q_1, q_2\} \\ \mathcal{E}_{1,1} &= \{q_3\} \\ \mathcal{E}_{1,2} &= \{q_4\}\end{aligned}$$

接下来是第 2 步。

	a	b
q_0, q_1	$q_1 \in \mathcal{E}_{1,0}, q_2 \in \mathcal{E}_{1,0}$	$q_0 \in \mathcal{E}_{1,0}, q_1 \in \mathcal{E}_{1,0}$
q_0, q_2	$q_1 \in \mathcal{E}_{1,0}, q_3 \in \mathcal{E}_{1,1}$	$q_0 \in \mathcal{E}_{1,0}, q_2 \in \mathcal{E}_{1,0}$
q_1, q_2	$q_2 \in \mathcal{E}_{1,0}, q_3 \in \mathcal{E}_{1,1}$	$q_1 \in \mathcal{E}_{1,0}, q_2 \in \mathcal{E}_{1,0}$

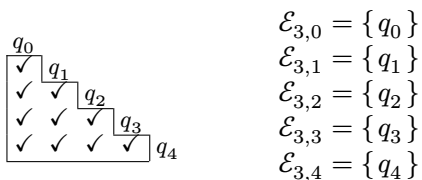
我们需要在单元格 q_0, q_2 和 q_1, q_2 打勾。因此，这次计算区分出了单个状态 q_2 。

$$\begin{aligned}\mathcal{E}_{2,0} &= \{q_0, q_1\} \\ \mathcal{E}_{2,1} &= \{q_2\} \\ \mathcal{E}_{2,2} &= \{q_3\} \\ \mathcal{E}_{2,3} &= \{q_4\}\end{aligned}$$

只剩下一个可能分裂的类，所以无论是否发生分裂，第 3 步都将是最后一步。

	a	b
q_0, q_1	$q_1 \in \mathcal{E}_{2,0}, q_2 \in \mathcal{E}_{2,1}$	$q_0 \in \mathcal{E}_{2,0}, q_1 \in \mathcal{E}_{2,0}$

它们确实分裂了； q_1 与 q_0 是 3-可区分的。这些就是 $\sim_3$ 等价类。



$$\begin{aligned}\mathcal{E}_{3,0} &= \{q_0\} \\ \mathcal{E}_{3,1} &= \{q_1\} \\ \mathcal{E}_{3,2} &= \{q_2\} \\ \mathcal{E}_{3,3} &= \{q_3\} \\ \mathcal{E}_{3,4} &= \{q_4\}\end{aligned}$$

Moore 算法的第一阶段结束。最小化后的机器有五个状态；它就是我们最初的那台机器。

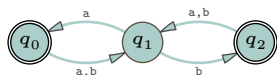
如果我们从一台多了一个状态的类似机器开始，那么算法就会多走一步。步骤数比状态数少 2，也就是说，耗时随状态数呈线性增长。

IV.C.21

(A) 各步集合如下。

Step n	R_n
0	$\{q_0\}$
1	$\{q_0, q_1\}$
2	$\{q_0, q_1, q_2\}$
3	$\{q_0, q_1, q_2\}$

因此可达状态集为 $R = \{q_0, q_1, q_2\}$ 。不可达状态集为 $Q - R = \{q_3\}$ 。以下是只包含可达状态的机器。



(B) 各步集合如下。

Step n	R_n
0	$\{q_0\}$
1	$\{q_0, q_1, q_3\}$
2	$\{q_0, q_1, q_3, q_4\}$
3	$\{q_0, q_1, q_3, q_4\}$

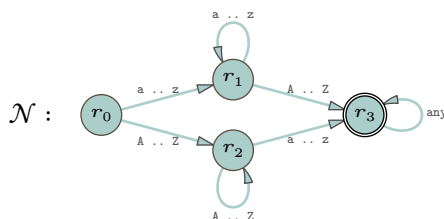
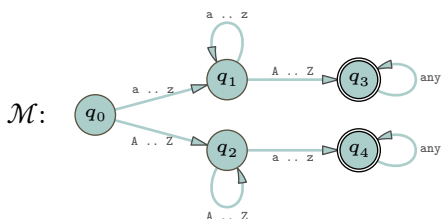
可达状态集为 $R = R_2 = \{q_0, q_1, q_3, q_4\}$ 。不可达状态集为 $Q - R = \{q_2, q_5\}$ 。

IV.C.22 要证明一个二元关系是等价关系，必须证明它是自反、对称且传递的。

(A) 固定某个 $n \in \mathbb{N}$ 。对于自反性，显然对于任何 n ，任何状态都与自身 n -不可区分。对称性也是显然的。现在假设对于某台机器，有 $q_0 \sim_n q_1$ 和 $q_1 \sim_n q_2$ 。设 σ 是一个长度小于或等于 n 的串。分两种情况讨论。第一种情况是 σ 使机器从 q_0 到达一个接受状态，那么因为 $q_0 \sim_n q_1$ ，它也使得机器从 q_1 到达一个接受状态；由此再根据 $q_1 \sim_n q_2$ ，可知它也使得机器从 q_2 到达一个接受状态。另一种情况，即 σ 使机器从 q_0 到达一个非接受状态，证明是类似的。

(B) 有限状态机只有有限多个状态。因此，只有有限多个状态是可区分的。一旦 k 大于最后一个能使状态 n -可区分的索引 n ，那么 $\sim$ 就是 $\sim_k$ 。前一项的结论适用于此。

IV.C.23 作为参考，下面给出两台机器。



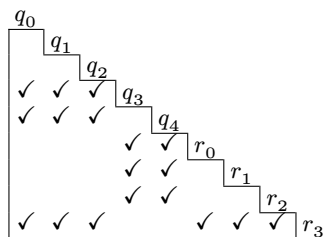
(A) 等价类如下。

$$\mathcal{E}_{0,0} = \{q_3, q_4, r_3\} \quad \mathcal{E}_{0,1} = \{q_0, q_1, q_2, r_0, r_1, r_2\}$$

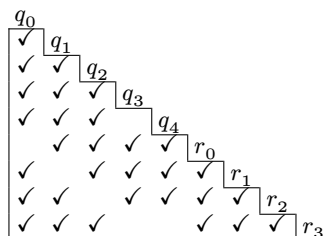
下面是初始三角表。

	a	A
q_3, q_4	$q_3 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,0}, q_4 \in \mathcal{E}_{0,0}$
q_3, r_3	$q_3 \in \mathcal{E}_{0,0}, r_3 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,0}, r_3 \in \mathcal{E}_{0,0}$
q_4, r_3	$q_4 \in \mathcal{E}_{0,0}, r_3 \in \mathcal{E}_{0,0}$	$q_4 \in \mathcal{E}_{0,0}, r_3 \in \mathcal{E}_{0,0}$
q_0, q_1	$q_1 \in \mathcal{E}_{0,1}, q_1 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,1}, q_3 \in \mathcal{E}_{0,0}$
q_0, q_2	$q_1 \in \mathcal{E}_{0,1}, q_4 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,1}, q_2 \in \mathcal{E}_{0,1}$
q_0, r_0	$q_1 \in \mathcal{E}_{0,1}, r_1 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,1}, r_2 \in \mathcal{E}_{0,1}$
q_0, r_1	$q_1 \in \mathcal{E}_{0,1}, r_1 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,1}, r_3 \in \mathcal{E}_{0,0}$
q_0, r_2	$q_1 \in \mathcal{E}_{0,1}, r_3 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,1}, r_2 \in \mathcal{E}_{0,1}$
q_1, q_2	$q_1 \in \mathcal{E}_{0,1}, q_4 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,1}$
q_1, r_0	$q_1 \in \mathcal{E}_{0,1}, r_1 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,0}, r_2 \in \mathcal{E}_{0,1}$
q_1, r_1	$q_1 \in \mathcal{E}_{0,1}, r_1 \in \mathcal{E}_{0,1}$	$q_3 \in \mathcal{E}_{0,0}, r_3 \in \mathcal{E}_{0,0}$
q_1, r_2	$q_1 \in \mathcal{E}_{0,1}, r_3 \in \mathcal{E}_{0,0}$	$q_3 \in \mathcal{E}_{0,0}, r_2 \in \mathcal{E}_{0,1}$
q_2, r_0	$q_4 \in \mathcal{E}_{0,0}, r_1 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,1}, r_2 \in \mathcal{E}_{0,1}$
q_2, r_1	$q_4 \in \mathcal{E}_{0,0}, r_1 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,1}, r_3 \in \mathcal{E}_{0,0}$
q_2, r_2	$q_4 \in \mathcal{E}_{0,0}, r_3 \in \mathcal{E}_{0,0}$	$q_2 \in \mathcal{E}_{0,1}, r_2 \in \mathcal{E}_{0,1}$
r_0, r_1	$r_1 \in \mathcal{E}_{0,1}, r_1 \in \mathcal{E}_{0,1}$	$r_2 \in \mathcal{E}_{0,1}, r_3 \in \mathcal{E}_{0,0}$
r_0, r_2	$r_1 \in \mathcal{E}_{0,1}, r_3 \in \mathcal{E}_{0,0}$	$r_2 \in \mathcal{E}_{0,1}, r_2 \in \mathcal{E}_{0,1}$
r_1, r_2	$r_1 \in \mathcal{E}_{0,1}, r_3 \in \mathcal{E}_{0,0}$	$r_3 \in \mathcal{E}_{0,0}, r_2 \in \mathcal{E}_{0,1}$

图 97, FOR QUESTION IV.C.23: 习题 C.23 的第 (1) 步。



- (B) 第一步考察来自两个 $\sim_0$ 类的状态对, 计算字符转移是否会分裂某个 $\sim_0$ 类。该表过大, 以 Figure 97 on page 171 给出。存在许多分裂, 例如 q_0 和 q_1 是 1-可区分的。



有六个空单元格, 分别位于 q_0, r_0 、 q_1, r_1 、 q_2, r_2 、 q_3, q_4 、 q_3, r_3 以及 q_4, r_3 。我们得出结论: $\mathcal{E}_{0,0}$ 未分裂, 而 $\mathcal{E}_{0,1}$ 分裂为三个。

$$\mathcal{E}_{1,0} = \{q_3, q_4, r_3\} \quad \mathcal{E}_{1,1} = \{q_0, r_0\} \quad \mathcal{E}_{1,2} = \{q_1, r_1\} \quad \mathcal{E}_{1,3} = \{q_2, r_2\}$$

- (C) 第二步的计算未产生更多区分。
 (D) Moore 算法将 q_0 与 r_0 归为一组, 将 q_1 与 r_1 归为一组, 将 q_2 与 r_2 归为一组, 同时将 q_3 和 q_4 与 r_3 归为一组。由于每个类中恰有一个 r_i , 该机器与 q_j 机器识别相同的语言, 并且是最小的。

IV.C.24

- (A) 算法在没有类分裂的步骤停机。由于这些机器只有有限多个状态, 这样的步骤最终必然发生。
 (B) 如果存在一个 $\sim_n$ 类, 其中某个字符将该类中的两个状态送到不同的 $\sim_n$ 类, 那么算法不会在该步终止。相反, 它会分裂该类并进入下一步。因此算法不可能以未良定义的转移函数结束。
 (C) 在 Moore 算法中, 第 0 步将状态划分为两个类, 一个包含 $\mathcal{M}$ 的所有接受状态, 另一个包含 $\mathcal{M}$ 的所有非接受状态。最终, $\sim$ 类是这些类的子类, 因此完全由接受状态或完全由非接受状态组成。

- (D) 考虑一个串 $\sigma \in \Sigma^*$ 。我们将证明它被机器 $\mathcal{M}$ 接受当且仅当它被另一台机器 $\mathcal{N}$ 接受。论证采用归纳法，但我们将非正式地叙述。

输入机器从 q_0 开始，而输出机器从一个包含 q_0 的状态 $r_0 = \varepsilon_0$ 开始。 σ 的第一个字符将 $\mathcal{M}$ 从 q_0 移到某个状态 q_{i_1} ，并将 $\mathcal{N}$ 从一个包含 q_0 的状态移到一个包含 q_{i_1} 的状态。 σ 的第二个字符将 $\mathcal{M}$ 从 q_{i_1} 移到某个 q_{i_2} ，并将 $\mathcal{N}$ 从一个包含 q_{i_1} 的状态移到一个包含 q_{i_2} 的状态。两台机器以这种方式继续，直到串耗尽。最后， $\mathcal{M}$ 处于接受状态当且仅当 $\mathcal{N}$ 处于一个包含该接受状态的状态，而该状态本身是 $\mathcal{N}$ 的一个接受状态。

- (E) 为明确记号：起始机器是 $\mathcal{M}$ ，将该机器运行 Moore 算法的结果是 $\mathcal{N}$ ，而 $\hat{\mathcal{M}}$ 是一台识别与 $\mathcal{M}$ 相同语言的有限状态机。作为预备，通过重命名使 $\hat{\mathcal{M}}$ 和 $\mathcal{N}$ 不共享任何状态，使得 $\hat{\mathcal{M}}$ 的状态为 $q_0, q_1, \dots$ ， $\mathcal{N}$ 的状态为 $r_0, r_1, \dots$

在开始之前，我们对最小机器做两点观察。首先，在最小机器 $\mathcal{N}$ 中，所有状态都必须是可达的，否则我们可以消除一个不可达状态，得到一个更小的识别相同语言的机器。其次，将最小机器运行 Moore 算法，最终每个 $\sim$ 类有且仅有一个成员。

我们必须证明 $\hat{\mathcal{M}}$ 的状态数至少和 $\mathcal{N}$ 一样多。首先在两个状态集的并集上执行 Moore 算法（如习题 C.23 所示）。

两台机器 $\hat{\mathcal{M}}$ 和 $\mathcal{N}$ 识别相同的语言，所以将两者都置于初始状态并在带上放置一个串 $\tau \in \Sigma^*$ ，结果总是相同的：要么两台机器都以接受状态结束，要么都以拒绝状态结束。因此没有串能区分 q_0 和 r_0 ，也就是说，当我们在两个状态集的并集上运行 Moore 算法时，它们的初始状态是不可区分的， $q_0 \sim r_0$ 。

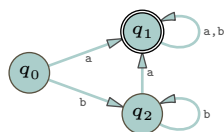
我们用一个归纳论证来完成。假设状态 q 和 r 是不可区分的。我们断言，对于所有 $x \in \Sigma$ ，状态 $\Delta_{\mathcal{N}}(q, x)$ 与状态 $\Delta_{\mathcal{M}}(r, x)$ 是不可区分的。这是因为，如果 τ 是一个能区分 $\Delta_{\mathcal{N}}(q, x)$ 和 $\Delta_{\mathcal{M}}(r, x)$ 的串，那么 $x \wedge \tau$ 就能区分 q 和 r 。

上面的第一个观察表明，存在某个串 $\sigma \in \Sigma^*$ 使得 $\hat{\Delta}_{\mathcal{N}}(\sigma) = r$ 。这个 σ 将机器 $\hat{\mathcal{M}}$ 带到某个状态，即 $\hat{\Delta}_{\hat{\mathcal{M}}}(\sigma)$ 是 $\hat{\mathcal{M}}$ 中的一个状态，并且根据前一段，它与 r 是不可区分的。

根据上面的观察，每个 $\sim$ 类包含且仅包含 $\mathcal{N}$ 的一个状态 r 。前一段说明， $\mathcal{N}$ 的每个状态 r 与 $\hat{\mathcal{M}}$ 的至少一个状态 q 在同一个 $\sim$ 类中。因此， $\mathcal{M}$ 中的状态数至少和 $\mathcal{N}$ 中一样多。

IV.C.25

- (A) 这是原机器，以供参考。



通过观察，所有状态都是可达的。

第 0 步将状态分为两类，一类是接受状态，一类是非接受状态。算法首先在 q_i, q_j 表项中打勾。

$$\mathcal{E}_{0,0} = \{q_0, q_2\} \quad \mathcal{E}_{0,1} = \{q_1\}$$

下面是初始的三角表，如果 q_i 和 q_j 中一个是接受状态而另一个不是，则对应的表项已打勾。

q_0	q_1	q_2
☒	☒	☐

接下来，第 1 步计算这些 $\sim_0$ 等价类是否有分裂。

	a	b
q_0, q_2	$q_1 \in \mathcal{E}_{0,1}, q_1 \in \mathcal{E}_{0,1}$	$q_2 \in \mathcal{E}_{0,0}, q_2 \in \mathcal{E}_{0,0}$

q_0	q_1	q_2
☒	☒	☒

没有发生分裂。

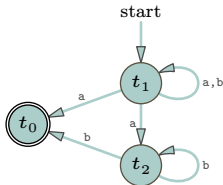
这些就是 $\sim$ 等价类。

$$r_0 = \mathcal{E}_0 = \{q_0, q_2\} \quad r_1 = \mathcal{E}_1 = \{q_1\}$$

下面是极小化后的机器。



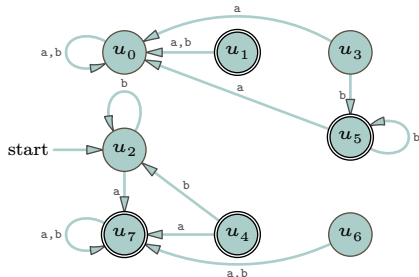
(B) 下面是将箭头反转、节点重命名、初始状态改为接受状态、接受状态改为初始状态后得到的机器。



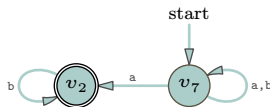
它是非确定性的；例如，从节点 t_1 出发有两条标记为 a 的边。
(c) 这是将那个非确定性有限状态机转换为确定性机器的表格。

节点名称	节点集合	a	b
u_0	$\emptyset$	u_0	u_0
+ u_1	$\{t_0\}$	u_0	u_0
u_2	$\{t_1\}$	u_7	u_2
u_3	$\{t_2\}$	u_0	u_5
+ u_4	$\{t_0, t_1\}$	u_7	u_2
+ u_5	$\{t_0, t_2\}$	u_0	u_5
u_6	$\{t_1, t_2\}$	u_7	u_7
+ u_7	$\{t_0, t_1, t_2\}$	u_7	u_7

箭头图如下。



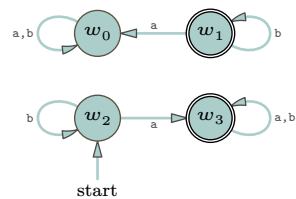
唯一可达的状态是 u_2 和 u_7 。
(D) 示意图如下。



(E) 将那个非确定性有限状态机转换为确定性机器。

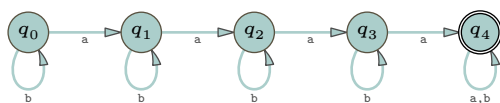
节点名称	节点集合	a	b
w_0	$\emptyset$	w_0	w_0
+ w_1	$\{v_2\}$	w_0	w_1
w_2	$\{v_7\}$	w_3	w_2
+ w_3	$\{v_2, v_7\}$	w_3	w_3

箭头图如下。



省略不可达状态后，这台机器（除了状态重命名外）与第一项中的机器相同。

IV.C.26 习题 C.20 中的机器



有五个状态，并且对于由 $b^*ab^*ab^*ab^*a(a|b)^*$ 描述的语言来说是最小的。显然，这一模式可以推广。

Chapter V: 计算复杂性

V.1.31 正确。非正式地说，大 $\mathcal{O}$ 的含义类似于“增长速度慢于或相当于”。增长速度慢于或相当于 x^2 ，也就意味着增长速度慢于或相当于 x^3 。

更形式化地说，假设 f 是 $\mathcal{O}(x^2)$ 。那么存在常数 $N, C \in \mathbb{R}$ ，使得如果 $x \geq N$ 则 $f(x) \leq C \cdot x^2$ 。由于对所有 $x \geq 1$ 都有 $x^3 \geq x^2$ ，因此可得：如果 $\hat{x} \geq \max(N, 1)$ 则 $f(\hat{x}) \leq C \cdot \hat{x}^3$ 。故 f 是 $\mathcal{O}(x^3)$ 。

V.1.32 首先，“该算法的运行时间是 $\mathcal{O}(n^2)$ ” 这个断言并不意味着运行时间函数是二次的，因为如果 $f(n) = n$ ，那么 f 也是 $\mathcal{O}(n^2)$ 。

其次，即使运行时间是二次的，它也可能包含常数项和低阶项。因此， $g_0(n) = 1000 \cdot n^2$ 是 $\mathcal{O}(n^2)$ ， $g_1(n) = n^2 + 3n + 42$ 也是。但 $g_0(5)$ 和 $g_1(5)$ 都不等于 25。

(还有另一个问题：即使运行时间是二次的，该运行时间特性也仅对足够大的输入成立。因此，除非我们知道 5 大于定义中的那个 N ，否则我们完全没有任何理由去考虑 25。)

V.1.33 或许他们的意思是，对于他们正在处理的问题，他们目前使用的算法是 $\mathcal{O}(n^3)$ 的。(如果他们想把运行时间与问题本身而非特定算法关联起来，那么也许他们的意思是目前已知的最佳算法是 $\mathcal{O}(n^3)$ 的?)

V.1.34

- (A) 因为 5 的二进制是 101，所以需要三比特。
- (B) 数字 50 的二进制是 11 0010，所以需要六比特。
- (C) 数字 500 的二进制是 1 1111 0100，所以需要九比特。
- (D) 5 000 的二进制是 1 0011 1000 1000，所以需要十三比特。

下表使用公式 $\text{bits}(n) = 1 + \lfloor \lg(n) \rfloor$ 验证了这些结果。

十进制整数 n	5	50	500	5 000
二进制表示	101	11 0010	1 1111 0100	1 0011 1000 1000
$\lg(n)$	2.322	5.644	8.966	12.288
$1 + \lfloor \lg(n) \rfloor$	3	6	9	13

V.1.35 第二个蕴含式为真，而第一个为假。

回顾直观理解：“ f 是 $\mathcal{O}(g)$ ” 大致意思是“ f 的增长率小于或等价于 g 的”，而“ f 是 $\Theta(g)$ ” 大致意思是“ f 的增长率等价于 g 的”。因此第二个陈述讲得通，因为如果 f 与 g 以等价的速率增长，那就意味着 f 的速率小于或等价于 g 的。严格论证，第二个为真，因为根据定义，若 f 是 $\Theta(g)$ ，则 f 是 $\mathcal{O}(g)$ 且 g 是 $\mathcal{O}(f)$ 。

至于第一个陈述，直观理解是：假设 f 的增长小于或等价于 g 的，并不能迫使两者等价。也就是说，存在函数 f, g 使得 f 是 $\mathcal{O}(g)$ 但 f 不是 $\Theta(g)$ 。一个这样的对是 $f(n) = n$ 和 $g(n) = n^2$ 。

V.1.36 正确答案是“以上都不是”。我们不能将大 $\mathcal{O}$ 应用于一个特定的输入规模。大 $\mathcal{O}$ 是关于函数如何随输入规模增长而增长的，并不适用于单个输入。首先，如果 61 669 不大于定义中的 N ，那么关于二次性质的陈述就根本不适用。

评论。有时在实践中，人们会这样提及一个 $\mathcal{O}(n^2)$ 算法（或实现它的代码）的运行时间：“当 $n = 60\,000$ 时，运行时间大约应该是 $n = 30\,000$ 时的四倍。”他们要么是有点不严谨，要么是指他们熟悉该算法或代码，并且在输入规模为 n 时其运行时间行为已经大致是二次的了。但仅凭本题所给的信息，我们不能那样说。

V.1.37 我们使用第 262 页给出的简化大 $\mathcal{O}$ 表达式的原则。

- (A) $\mathcal{O}(n^2)$
- (B) $\mathcal{O}(2^n)$

- (C) $\mathcal{O}(n^4)$
 (D) $\mathcal{O}(\lg n)$

V.1.38

- (A) 对于足够大的输入 n ，此函数为 $f(n) = 0$ 。所以它是 $\mathcal{O}(1)$ 。
 (B) 对于足够大的输入 n ，此函数为 $f(n) = n^2$ 。所以它是 $\mathcal{O}(n^2)$ 。
 (C) 对于足够大的 n ，此函数为 $f(n) = \lg(n)$ 。所以它是 $\mathcal{O}(\lg(n))$ 。

V.1.39 因为我们将应用洛必达法则，所以将 n 改写为 x ，以表明必须将它们视为实函数。

- (A) 为应用定理，考虑比值的极限。

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{3x^3 + 2x + 4}{\ln(x) + 6}$$

这是一个 “ ∞/∞ ” 型极限。应用洛必达法则。

$$\lim_{x \rightarrow \infty} \frac{3x^3 + 2x + 4}{\ln(x) + 6} = \lim_{x \rightarrow \infty} \frac{9x^2 + 2}{1/(\ln(2)x)} = \ln(2) \cdot \lim_{x \rightarrow \infty} \frac{x \cdot (9x^2 + 2)}{1} = \infty$$

所以 g 是 $\mathcal{O}(f)$ 但 f 不是 $\mathcal{O}(g)$ 。简言之， f 的增长率严格大于 g 的。

- (B) 这是 “ ∞/∞ ” 型。多次使用洛必达法则。

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{3x^3 + 2x + 4}{x + 5x^3} = \lim_{x \rightarrow \infty} \frac{9x^2 + 2}{1 + 15x^2} = \lim_{x \rightarrow \infty} \frac{18x}{30x} = \lim_{x \rightarrow \infty} \frac{18}{30}$$

得出结论 f 是 $\Theta(g)$ (我们也可以说 g 是 $\Theta(f)$)。简言之， f 和 g 的增长率等价。

- (C) 这是比值的极限。

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{(1/2)x^3 + 12x^2}{x^2 \ln(x)}$$

我们可以先化简分式，从分子和分母中约去 x^2 。但我们选择直接对这个 “ ∞/∞ ” 型极限应用洛必达法则。注意分母中使用了乘积法则。

$$\begin{aligned} \lim_{x \rightarrow \infty} \frac{(1/2)x^3 + 12x^2}{x^2 \ln(x)} &= \lim_{x \rightarrow \infty} \frac{(3/2)x^2 + 24x}{2x \ln(x) + x} = \lim_{x \rightarrow \infty} \frac{(3/2)x^2 + 24x}{x(2 \ln(x) + 1)} = \lim_{x \rightarrow \infty} \frac{(3/2)x + 24}{2 \ln(x) + 1} \\ &= \lim_{x \rightarrow \infty} \frac{3/2}{2/x} = \lim_{x \rightarrow \infty} \frac{3x}{4} = \infty \end{aligned}$$

所以 g 是 $\mathcal{O}(f)$ 但 f 不是 $\mathcal{O}(g)$ 。

- (D) 极限是 “ ∞/∞ ” 型。洛必达法则给出此结果。

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{\lg(x)}{\ln(x)} = \lim_{x \rightarrow \infty} \frac{(1/x) \cdot (1/\ln(2))}{(1/x)} = \lim_{x \rightarrow \infty} \frac{1}{\ln(2)} \cdot \frac{(1/x)}{(1/x)} = \frac{1}{\ln(2)} \cdot \lim_{x \rightarrow \infty} 1$$

所以它们以等价的速率增长： f 是 $\Theta(g)$ 。

- (E) 注意到这是一个 “ ∞/∞ ” 型极限。

$$\begin{aligned} \lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} &= \lim_{x \rightarrow \infty} \frac{x^2 + \lg(x)}{x^4 - x^3} = \lim_{x \rightarrow \infty} \frac{2x + 1/(\ln(2) \cdot x)}{4x^3 - 3x^2} = \frac{1}{\ln(2)} \cdot \lim_{x \rightarrow \infty} \frac{2 \ln(2) \cdot x^2 + 1}{4x^4 - 3x^3} \\ &= \frac{1}{\ln(2)} \cdot \lim_{x \rightarrow \infty} \frac{4 \ln(2) \cdot x}{16x^3 - 9x^2} = \frac{1}{\ln(2)} \cdot \lim_{x \rightarrow \infty} \frac{4 \ln(2)}{16x^2 - 9x} = 0 \end{aligned}$$

所以 f 是 $\mathcal{O}(g)$ 但 g 不是 $\mathcal{O}(f)$ 。

(F) 我们考虑函数的比值。

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{55}{x^2 + x}$$

这不是一个“ ∞/∞ ”型极限（而是“ $55/\infty$ ”型），所以洛必达法则不适用。但这不要紧，因为极限很容易计算。极限为零。因此 f 是 $\mathcal{O}(g)$ 但 g 不是 $\mathcal{O}(f)$ 。

V.1.40 注意 $\cos(t)$ 以 1 为界。并且， $\lg(5^n) = n \cdot \lg(5)$ 。因此，增长率小于或等价于 n^2 的是第一、第二、第四和第五个。第三个的增长率是 $\Theta(n^3)$ ，大于 n^2 的。

V.1.41 我们可以用引理 1.25 和表 1.27，以及定理 1.15 来做。这里我们前两个用两种方法做，以作说明。

(A) 真。由引理 1.25，这是一个 $\mathcal{O}(n^2)$ 函数与一个 $\mathcal{O}(n)$ 函数的和。由同一引理和表 1.27，和是 $\mathcal{O}(n^2)$ ，因此是 $\mathcal{O}(n^3)$ 。

这是另一个使用定理 1.15 的论证。

$$\lim_{x \rightarrow \infty} \frac{5x^2 + 2x}{x^3} = \lim_{x \rightarrow \infty} \frac{10x + 2}{3x^2} = \lim_{x \rightarrow \infty} \frac{10}{6x} = 0$$

(B) 假，由定理 1.15。

$$\lim_{x \rightarrow \infty} \frac{2 + 4x^3}{\lg x} = \lim_{x \rightarrow \infty} \frac{12x^2}{(1/x)} = \lim_{x \rightarrow \infty} 12x^3 = \infty$$

或者，我们可以应用引理 1.25 得到 $2 + 4n^3$ 是 $\Theta(n^3)$ ，它在增长阶层级表 1.27 中严格高于 $\lg n$ 。

(C) 真。如本节所述，所有对数函数以相同速率增长。所以 $\ln n$ 是 $\Theta(\lg n)$ ，因此 $\ln n$ 是 $\mathcal{O}(\lg n)$ 。

(D) 真，由定理 1.15。洛必达法则给出

$$\lim_{x \rightarrow \infty} \frac{x^3 + x^2 + x}{x^3} = \lim_{x \rightarrow \infty} \frac{3x^2 + 2x + 1}{3x^2} = \lim_{x \rightarrow \infty} \frac{6x + 2}{6x} = \lim_{x \rightarrow \infty} \frac{6}{6} = 1$$

（这是一系列“ ∞/∞ ”型极限）。

(E) 真。对任何多项式函数 p ，引理 1.23 给出 p 是 $\mathcal{O}(2^n)$ 。

V.1.42 解答这些问题时，我们有两种选择：要么使用引理 1.25 中的代数性质进行化简，然后参考表 1.27；要么直接使用定理 1.15。

前一种方法更能让人体会到表达式是如何构造的。例如，在下面的第一小题中，严格来说我们并不需要第一段，因为第二段足以证明 $k = 4$ 是正确的。但如果只使用第二段，可能会让那些在此题上遇到困难的学生难以体会如何自行得出 $k = 4$ 。

(A) 最小的幂是 $k = 4$ 。根据引理的项 A，函数 $(1/10\,000\,000)n^4$ 是 $\mathcal{O}(n^4)$ ，而函数 n^3 显然是 $\mathcal{O}(n^3)$ 。根据表 1.27， n^3 是 $\mathcal{O}(n^4)$ 。因此，由该引理的项 B 可知，函数 $n^3 + (1/10\,000\,000)n^4$ 是 $\mathcal{O}(n^4)$ 。

要说明任何小于 4 的幂 k 都不满足要求，可以考虑极限 $\lim_{x \rightarrow \infty} x^3/(x^3 + (x^4/10\,000\,000))$ 。

$$\begin{aligned} \lim_{x \rightarrow \infty} \frac{x^4}{x^3 + (x^4/10\,000\,000)} &= \lim_{x \rightarrow \infty} \frac{x}{1 + (x/10\,000\,000)} = \lim_{x \rightarrow \infty} \frac{x}{(10\,000\,000 + x)/10\,000\,000} \\ &= \lim_{x \rightarrow \infty} \frac{10\,000\,000x}{10\,000\,000 + x} = \lim_{x \rightarrow \infty} \frac{10\,000\,000}{1} = 10\,000\,000 \end{aligned}$$

（倒数第二个等号使用了洛必达法则）。所以 $n^3 + (n^4/10\,000\,000)$ 是 $\Theta(n^4)$ 。因此 $k = 4$ 是最小值。

(B) 最小的幂是 $k = 4$ 。乘开可得 $n^4 + 5n^3 + 6n^2 - n^2 \lg n - 5n \lg n - 6 \lg n$ 。引理 1.25 的项 B 指出，整个表达式是 $\mathcal{O}(g)$ ，其中 g 是增长阶最大的项。在这个表达式中，表 1.27 显示该项是 n^4 。

要验证没有更小的幂能满足要求，可以通过求极限来比较这些函数。

$$\lim_{x \rightarrow \infty} \frac{x^4 + 5x^3 + 6x^2 - x^2 \lg x - 5x \lg x - 6 \lg x}{x^4} = \lim_{x \rightarrow \infty} \frac{x^4}{x^4} + \frac{5x^3}{x^4} + \frac{6x^2}{x^4} - \frac{x^2 \lg x}{x^4} - \frac{5x \lg x}{x^4} - \frac{6 \lg x}{x^4}$$

这些极限除了第一个为 1 外，其余均为零。因此 $x^4 + 5x^3 + 6x^2 - x^2 \lg x - 5x \lg x - 6 \lg x$ 是 $\Theta(x^4)$ 。

- (C) 答案是 $k = 3$ 。余弦函数有界，不超过 1。所以引理 1.25 给出整个表达式是 $\mathcal{O}(n^3)$ 。我们可以用定理 1.15，通过求以下极限来论证 $n = 3$ 是正确的。

$$\lim_{x \rightarrow \infty} \frac{5x^3 + 25 + \lceil \cos(x) \rceil}{x^3} \lim_{x \rightarrow \infty} \frac{5x^3}{x^3} + \frac{25 + \lceil \cos(x) \rceil}{x^3} = 5$$

该极限为一个介于零和无穷大之间的常数，所以 $5x^3$ 是 $\Theta(x^3)$ 。

- (D) 答案是 $k = 12$ 。展开表达式后，主导项是 $9n^{12}$ 。然后使用引理 1.25 或定理 1.15 都能得出答案。
(E) 该函数是 $\mathcal{O}(n^{7/2})$ ，但由于题目指定了 $k \in \mathbb{N}$ ，所以答案是 $k = 4$ 。

V.1.43

- (A) 答案是“两者都是”。这两个函数都是 $\Theta(n^2)$ 。所以 f 是 $\Theta(g)$ 。
(B) 答案是 g 为 $\mathcal{O}(f)$ 但反之不成立。这里， g 是 $\Theta(\lg n)$ ，而 f 是 $\Theta(n^3)$ 。基准表给出 g 为 $\mathcal{O}(f)$ ，但 f 不为 $\mathcal{O}(g)$ 。
(C) 函数 f 是二次函数，而 g 是平方根函数。因此 g 为 $\mathcal{O}(f)$ ，但 f 不为 $\mathcal{O}(g)$ 。
(D) 函数 f 是 $\mathcal{O}(n^{1.2})$ ，而函数 g 是 $\mathcal{O}(n^{\sqrt{2}})$ ，近似为 $\mathcal{O}(n^{1.414})$ 。我们有 f 为 $\mathcal{O}(g)$ ，但 g 不为 $\mathcal{O}(f)$ 。

V.1.44 我们使用引理 1.25 来化简大 $\mathcal{O}$ 表达式。

- (A) 引理 1.23 指出指数函数 $2^{(1/6)^n}$ 的增长速度比多项式函数 n^6 快。所以 f 是 $\mathcal{O}(g)$ ，但 g 不是 $\mathcal{O}(f)$ 。
(B) 表 1.27 表明函数 2^n 是 $\mathcal{O}(3^n)$ ，而 3^n 不是 $\mathcal{O}(2^n)$ 。因此， g 是 $\mathcal{O}(f)$ 但 f 不是 $\mathcal{O}(g)$ 。
(C) 对数的一条规则是 $\lg(3n) = \lg(3) + \lg(n)$ 。因为 $\lg(3)$ 是 $\Theta(1)$ ，由引理 1.25 可知 $\lg(3n)$ 是 $\Theta(\lg n)$ 。由此可得两者具有等价的增长率，所以 f 是 $\Theta(g)$ 。

V.1.45 我们将应用大 $\mathcal{O}$ 的定义（定义 1.6）来证明，对于任意复杂度函数 g ， Z 都是 $\mathcal{O}(g)$ 。

因为 g 是复杂度函数，所以存在 $N \in \mathbb{N}$ 使得当 $x \geq N$ 时 $g(x)$ 有定义且非负。取 $C = 1$ 。那么对于 $x_0 > N$ ，有 $|Z(x_0)| = 0 \leq C \cdot |g(x_0)| = C \cdot g(x_0)$ 。

V.1.46

	$n = 1$	$n = 10$	$n = 100$	$n = 1000$
$\lg n$	—	1.05×10^{-17}	2.10×10^{-17}	3.15×10^{-17}
n	3.16×10^{-18}	3.16×10^{-17}	3.16×10^{-16}	3.16×10^{-15}
$n \lg n$	—	1.05×10^{-16}	2.10×10^{-15}	3.15×10^{-14}
n^2	3.16×10^{-18}	3.16×10^{-16}	3.16×10^{-14}	3.16×10^{-12}
n^3	3.16×10^{-18}	3.16×10^{-15}	3.16×10^{-12}	3.16×10^{-9}
2^n	6.33×10^{-18}	3.24×10^{-15}	4.01×10^{12}	3.39×10^{283}

V.1.47 一年有 $60 \cdot 60 \cdot 24 \cdot 365.25 = 31\,557\,600$ 秒。以每秒 $10,000 \times 10^6$ 个 ticks 计算，一年大约有 3.16×10^{17} 个 ticks（如表 1.29 所述）。

- (A) 解方程 $\lg(n) = 3.16 \times 10^{17}$ 得到 $n = 2^{3.16 \times 10^{17}} = 4.60 \times 10^{94\,997\,841\,911\,656\,529}$ 。
(B) $(3.16 \times 10^{17})^2 = 9.96 \times 10^{34}$
(C) 3.16×10^{17}
(D) $(3.16 \times 10^{17})^{1/2} = 5.62 \times 10^8$
(E) 因为 $(3.16 \times 10^{11})^{1/3} \approx 680823.684681371$ ，所以答案为 680 823。
(F) 我们有 $\lg(3.16 \times 10^{17}) \approx 58.131$ ，所以答案为 58。

V.1.48 解 $100\,000 \cdot n^2 = n^3$ 得到 $n = 100\,000$ 。因此，第一个使得 $f(n) = 100\,000 \cdot n^2$ 小于 $g(n) = n^3$ 的数 n 是 $n = 100\,001$ 。

V.1.49 线性的。有限状态机每一步消耗一个字符。因此它们的运行时间等于输入的长度。用定义 1.30 中的记号表示，就是 $t_M(\sigma) = |\sigma|$ 。

V.1.50 令 g 为函数 $g(x) = 1$ 。

- (A) 我们可以直接应用大 $\mathcal{O}$ 的定义，定义 1.6。取 $N = 1$ 和 $C = 8$ 。如果 $x > N = 1$ ，那么 $C \cdot |g(x)| = 8 \cdot 1$ 而 $|f(x)| = 7$ ，所以 $|f(x)| \leq C|g(x)|$ 。
(B) 正弦函数对所有 x 满足 $-1 \leq \sin(x) \leq 1$ 。取 $N = 1$ 和 $C = 9$ 。如果 $x > N$ ，那么 $C \cdot |g(x)| = 9$ 而 $|f(x)| = 7 + \sin(x) \leq 8$ 。

- (c) 固定 $N = 1$ 和 $C = 8$ 。如果 $x \geq N = 1$, 那么 $C \cdot |g(x)| = 8$ 大于或等于 $|f(x)| = 7 + (1/x)$ 。
 (d) 我们先证明如果有界, 则它属于 $\mathcal{O}(1)$ 。假设 f 被 $K \in \mathbb{R}$ 界定。如果它被一个更小的数界定, 那么它也被更大的数界定, 所以我们可以取 $K \in \mathbb{R}^+$ 。固定 $N = 1$ 和 $C = K$ 。如果 $x \geq N = 1$, 那么 $C \cdot |g(x)| = K \geq |f(x)|$, 因此 f 是 $\mathcal{O}(g)$ 。

反之, 假设 f 是 $\mathcal{O}(g)$ 。那么存在常数 $N, C \in \mathbb{R}^+$ 使得 $x \geq N$ 蕴含 $C \cdot |g(x)| = C \geq |f(x)|$ 。选择 $L = \max(C, f(0), \dots, f(N-1))$ 。那么对于任意 $x \in \mathbb{N}$, 我们有 $f(x) \leq L$ 。

V.1.51 根据前一题的最后一小题, 函数 $f: \mathbb{R} \rightarrow \mathbb{R}$ 是 $\mathcal{O}(1)$ 当且仅当它被常数界定。所以 $g(x) \leq x^{\mathcal{O}(1)}$ 说明函数 g 至多具有多项式增长阶。

V.1.52 令 $N = 4$, $C = 1$ 。注意到 $4! = 24 > 2^4 = 16$ 。由此, 如果 $n \geq N = 4$, 那么 $C \cdot n! = n! = 4! \cdot 5 \cdot 6 \cdots (n-1) \cdot n \geq 2^4 \cdot 2^{n-4} = 2^n$ 。

V.1.53

(A) 根据例 1.20 中的计算, 这个极限等于 0

$$\lim_{x \rightarrow \infty} \frac{(\log_b(x))^2}{x^d} = \lim_{x \rightarrow \infty} \frac{2 \log_b(x) \cdot \frac{1}{x \ln(b)}}{dx^{d-1}} = \frac{2}{d \ln(b)} \cdot \lim_{x \rightarrow \infty} \frac{\log_b(x)}{x^d}$$

(B) 根据前一项中的计算, 这个极限等于 0

$$\lim_{x \rightarrow \infty} \frac{(\log_b(x))^3}{x^d} = \lim_{x \rightarrow \infty} \frac{3(\log_b(x))^2 \cdot \frac{1}{x \ln(b)}}{dx^{d-1}} = \frac{3}{d \ln(b)} \cdot \lim_{x \rightarrow \infty} \frac{(\log_b(x))^2}{x^d}$$

(c) 前一项提示可以用归纳法论证。所以, 假设结论对幂次 $n = 1, n = 2, \dots, n = k$ 成立, 考虑 $n = k+1$ 的情况。

$$\lim_{x \rightarrow \infty} \frac{(\log_b(x))^{k+1}}{x^d} = \lim_{x \rightarrow \infty} \frac{(k+1)(\log_b(x))^k \cdot \frac{1}{x \ln(b)}}{dx^{d-1}} = \frac{k+1}{d \ln(b)} \cdot \lim_{x \rightarrow \infty} \frac{(\log_b(x))^k}{x^d}$$

最后应用 $n = k$ 的归纳假设。

V.1.54 该函数 (不可计算)

$$K(x) = \begin{cases} 0 & \text{若 } \phi_x(x) \uparrow \\ 1 & \text{若 } \phi_x(x) \downarrow \end{cases}$$

是有界的, 因此是 $\mathcal{O}(1)$ 。

V.1.55 使用定理 1.15。

$$\lim_{x \rightarrow \infty} \frac{2^x}{3^x} = \lim_{x \rightarrow \infty} \left(\frac{2}{3} \right)^x = 0$$

对于第二部分, $3^x = 2^{\lg(3) \cdot x}$, 其中 $\lg(3)$ 是 3 的以 2 为底的对数。

V.1.56 考虑比值。

$$\frac{n!}{n^n} = \frac{1}{n} \cdot \frac{2}{n} \cdots \frac{n}{n} \leq \frac{1}{n} \cdot 1 \cdots 1$$

显然比值 $n!/n^n$ 的极限是 0。

V.1.57 答案是 (D)。

为了确定 $\mathcal{O}(f_2) < \mathcal{O}(f_3)$, 我们有

$$\lim_{x \rightarrow \infty} \frac{x^{\ln x}}{x^{\sqrt{x}}} = \lim_{x \rightarrow \infty} x^{\ln x - \sqrt{x}} = 0$$

, 因为 $\lim_{x \rightarrow \infty} (\ln x - \sqrt{x}) = -\infty$ 。(证明此极限的一种方法是使用例 1.20。另一种方法是将 $\ln x - \sqrt{x}$ 乘以分数 $(\ln x + \sqrt{x})/(\ln x + \sqrt{x})$ 。这得到:

$$\begin{aligned}\lim_{x \rightarrow \infty} \frac{(\ln x)^2 - x}{\sqrt{x} + \ln(x)} &= \lim_{x \rightarrow \infty} \frac{2 \ln x \cdot (1/x) - 1}{1/(2x^{1/2}) + 1/x} = \lim_{x \rightarrow \infty} \frac{2x}{2x} \cdot \frac{2 \ln x \cdot (1/x) - 1}{1/(2x^{1/2}) + 1/x} = 2 \cdot \lim_{x \rightarrow \infty} \frac{2 \ln x - x}{\sqrt{x} + 2} \\ &= 2 \cdot \lim_{x \rightarrow \infty} \frac{(2/x) - 1}{1/(2x^{1/2})} = 4 \cdot \lim_{x \rightarrow \infty} x^{1/2} \cdot \left(\frac{2}{x} - 1\right) = 4 \cdot \lim_{x \rightarrow \infty} 2x^{-1/2} - x^{1/2} = 4 \cdot \lim_{x \rightarrow \infty} \frac{2 - x}{\sqrt{x}} \\ &= 4 \cdot \lim_{x \rightarrow \infty} \frac{-1}{1/(2x^{1/2})} = 4 \cdot \lim_{x \rightarrow \infty} -2\sqrt{x}\end{aligned}$$

结果是 $-\infty$ 。)

对于 $\mathcal{O}(f_3) < \mathcal{O}(f_1)$, 取分子和分母的自然对数的极限。由于自然对数是严格递增函数, 这足以得出结论。

$$\lim_{x \rightarrow \infty} \frac{\ln(f_3(x))}{\ln(f_1(x))} = \lim_{x \rightarrow \infty} \frac{\ln(x^{\sqrt{x}})}{\ln(10^x)} = \lim_{x \rightarrow \infty} \frac{\sqrt{x} \ln(x)}{x \ln(10)} = \frac{1}{\ln(10)} \cdot \lim_{x \rightarrow \infty} \frac{\ln(x)}{\sqrt{x}}$$

这里例 1.20 也适用, 或者注意到这是一个 “ ∞/∞ ” 型极限并应用洛必达法则。

$$\lim_{x \rightarrow \infty} \frac{1/x}{1/(2x^{1/2})} = 2 \cdot \lim_{x \rightarrow \infty} \frac{\sqrt{x}}{x} = 2 \cdot \lim_{x \rightarrow \infty} \frac{1}{\sqrt{x}} = 0$$

V.1.58

(A) 先验证它们都是 “ ∞/∞ ” 型极限, 然后应用洛必达法则两次。

$$\lim_{x \rightarrow \infty} \frac{x^2 + 5x + 1}{x^2} = \lim_{x \rightarrow \infty} \frac{2x + 5}{2x} = \lim_{x \rightarrow \infty} \frac{2}{2} = 1$$

(B) 此处也应用洛必达法则, 记得对分子使用链式法则。

$$\lim_{x \rightarrow \infty} \frac{\lg(x+1)}{\lg(x)} = \lim_{x \rightarrow \infty} \frac{(1/x \ln(2)) \cdot 1}{1/x \ln(2)} = 1$$

V.1.59 不存在。如果 $\mathcal{O}(f)$ 是所有多项式的集合, 那么由于它是该集合的成员, f 必须是一个多项式。然而, 这样 f 就有某个次数, 而没有更高次数的多项式是 $\mathcal{O}(f)$ 的成员。

V.1.60 我们有 $x^{\lg x} = (2^{\lg x})^{\lg x} = 2^{(\lg x)^2} \in \mathcal{O}(2^{\text{poly}(\lg x)})$ (第一个等式来自恒等式 $x = 2^{\lg x}$)。

V.1.61 一条对数法则是 $\log_b(x^a) = a \cdot \log_b x$ 且 $b^{\log_b(x)} = x$ 。

(A) 取定一个常数 $k \in \mathbb{R}$ 。考虑比值。

$$\lim_{x \rightarrow \infty} \frac{k \lg x}{(\lg x)^2} = k \cdot \lim_{x \rightarrow \infty} \frac{1}{\lg x} = 0$$

由定理 1.15 即得所需结论。

(B) 根据定理 1.15, 只需证明该极限为 0。

$$\lim_{x \rightarrow \infty} \frac{x^k}{x^{\lg(x)}} = \lim_{x \rightarrow \infty} x^{k - \lg(x)}$$

利用恒等式 $a = 2^{\lg(a)}$ 重写。

$$= \lim_{x \rightarrow \infty} 2^{\lg(x^{k - \lg(x)})} = \lim_{x \rightarrow \infty} 2^{(k - \lg(x)) \cdot \lg(x)} = \lim_{x \rightarrow \infty} 2^{k \lg(x) - (\lg(x))^2}$$

上一项已给出指数的极限为 $-\infty$, 因此整个表达式的极限是 0。

(C) 应用洛必达法则可得。

$$\lim_{x \rightarrow \infty} \frac{(\lg x)^2}{x} = \lim_{x \rightarrow \infty} \frac{2 \lg x \cdot (1/x)}{1} = 2 \lim_{x \rightarrow \infty} \frac{\lg x}{x} = 2 \lim_{x \rightarrow \infty} \frac{(1/x)}{1} = 2 \lim_{x \rightarrow \infty} \frac{1}{x} = 0$$

(D) 应用定理 1.15 及洛必达法则

$$\lim_{x \rightarrow \infty} \frac{x^{\lg x}}{2^x} = \lim_{x \rightarrow \infty} \frac{(2^{\lg x})^{\lg x}}{2^x} = \lim_{x \rightarrow \infty} \frac{2^{(\lg x)^2}}{2^x} = \lim_{x \rightarrow \infty} 2^{(\lg x)^2 - x}$$

(第一个等号利用了恒等式 $a = 2^{\lg(a)}$)。上一项给出指数的极限是 $\lim_{x \rightarrow \infty} (\lg x)^2 - x = -\infty$ ，因而整个表达式的极限是 0。

V.1.62 我们将使用引理陈述中的记号。

(A) 假设 $a \in \mathbb{R}$ 非负。由于 f 是复杂度函数，存在 $N \in \mathbb{R}$ 使得当 $x \geq N$ 时， f 有定义且非负。取 $C = a$ 。当 $x \geq N$ 时， $|af(x)| = af(x) \leq a \cdot f(x) = a|f(x)|$ ，符合要求。

(B) 假设 f_1 是 $\mathcal{O}(f_0)$ 。根据定义，存在常数 $N, C \in \mathbb{R}$ ，使得当 $x \geq N$ 时有 $|f_1(x)| \leq C \cdot |f_0(x)|$ 。因为这两个函数都是复杂度函数，我们可以假设 N 足够大，使得对所有输入 x ，两者均为非负。于是有 $f_0(x) + f_1(x) \leq f_0(x) + Cf_0(x) = (1+C) \cdot f_0(x)$ ，因此存在常数 $1+C$ ，表明 $f_0 + f_1$ 属于 $\mathcal{O}(f_0)$ 。
 $f_0 - f_1$ 的论证是类似的，只不过常数变为 $1 - C$ 。

(C) 取 $f_0(x) = x^2$ 和 $f_1(x) = x$ 。则函数 $f_0 f_1$ 由 $f_0 f_1(x) = x^3$ 给出，所以 $f_0 f_1$ 不在 $\mathcal{O}(f_0)$ 中。

V.1.63

(A) 我们必须证明 f 属于 $\mathcal{O}(f)$ 。因为 f 是复杂度函数，所以存在某个数，超过该数后函数有定义且为正。取 N 为该数，并取 $C = 1$ 。

(B) 假设 f_0 是 $\mathcal{O}(f_1)$ 且 f_1 是 $\mathcal{O}(f_2)$ 。则存在 $N_0, N_1, C_0, C_1 \in \mathbb{R}$ 使得若 $x \geq N_0$ 则 $C_0 \cdot f_1(x) \geq f_0(x)$ ，且若 $x \geq N_1$ 则 $C_1 \cdot f_2(x) \geq f_1(x)$ 。取 $\hat{N} = \max(N_0, N_1)$ 及 $\hat{C} = C_0 \cdot C_1$ 。则当 $x \geq \hat{N}$ 时，有 $\hat{C} \cdot f_2(x) = C_0 C_1 \cdot f_2(x) \geq C_0 \cdot f_1(x) \geq f_0(x)$ 。

V.1.64 根据定义， f 是 $\mathcal{O}(g)$ 当且仅当存在数 $N, C \in \mathbb{R}$ 使得如果 $x \geq N$ 则 $|f(x)| \leq C \cdot |g(x)|$ 。

(A) 如果 $\lim_{x \rightarrow \infty} f(x)/g(x)$ 存在且等于 0，那么对于任意 $\varepsilon > 0$ ，存在 $M \in \mathbb{R}$ 使得 $x \geq M$ 蕴含 $|f(x)/g(x)| < \varepsilon$ ，因此 $|f(x)| < \varepsilon \cdot |g(x)|$ 。取 N 为 M ， C 为 ε ，这正是得出结论 f 是 $\mathcal{O}(g)$ 所需的。

(B) 假设相反，即 g 是 $\mathcal{O}(f)$ 。那么根据定义，存在常数 $K, P \in \mathbb{R}$ 使得如果 $x \geq K$ 则 $|g(x)| \leq P \cdot |f(x)|$ ，这给出 $1/P \leq |f(x)/g(x)|$ 。

问题的假设是 $\lim_{x \rightarrow \infty} f(x)/g(x)$ 存在且等于 0。取 $\varepsilon = 1/(P+1)$ 。那么存在常数 $M \in \mathbb{R}$ 使得如果 $x \geq M$ 则 $|f(x)/g(x)| \leq 1/(P+1)$ 。

当 x 大于 K 和 M 的最大值时，这两段矛盾。

V.1.65 首先使用洛必达法则得到，如果 $g(x) = a_m x^m + \dots + a_0$ 则 g 是 $\Theta(x^m)$ 。然后例 1.20 表明任何对数函数是 $\mathcal{O}(g)$ 。

对于另一半，固定底数 $b > 1$ 并考虑指数函数 $h(x) = b^x$ 。洛必达法则给出

$$\lim_{x \rightarrow \infty} \frac{x^m}{b^x} = \frac{m}{\ln(b)} \cdot \lim_{x \rightarrow \infty} \frac{x^{m-1}}{b^x} = \frac{m(m-1)}{(\ln(b))^2} \cdot \lim_{x \rightarrow \infty} \frac{x^{m-2}}{b^x} = \dots = \frac{m!}{(\ln(b))^m} \cdot \lim_{x \rightarrow \infty} \frac{1}{b^x} = 0$$

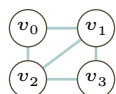
而定理 1.15 给出所需结论。

V.2.41 对于正八面体，路径 $\langle v_0, v_1, v_5, v_4, v_3, v_2, v_0 \rangle$ 可行。对于正二十面体，一条路径是 $\langle v_0, v_{10}, v_{11}, v_8, v_9, v_6, v_7, v_2, v_3, v_0 \rangle$ 。

V.2.42 与顶点数相同。遍历 Hamilton 路径意味着每个顶点恰好有一条边离开。因此，如果一个图有 n 个顶点，那么一条 Hamilton 路径就有相同数量的边，即 n 条。

V.2.43

(A) 这个图有一条 Hamilton 回路，即外围的回路。



为了包含顶点 v_0 和 v_3 ，任何回路都必须包含那四条外围的边。在每条 Hamilton 回路中，边的数量等于顶点的数量，因此任何 Hamilton 回路都不可能包含边 v_1v_2 ，因为那样边的数量就会变成五条。

- (B) 假设所有边的权重都是正的（如果给定的图不满足此条件，则对所有权重加上一个足够大的数，使它们全为正）。为边 e 赋予如下权重。

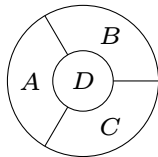
$$-(\text{顶点数量}) \cdot (\text{最大权重}) - 1$$

每条 Hamilton 路径都有相同数量的边，具体来说，边的数量等于顶点的数量。因此，一条经过 e 的路径总权重为负，这显然是获得负总权重的唯一方法，所以解决 Traveling Salesman 问题的 Hamilton 路径必须包含 e 。

V.2.44 我们可以目测得出。

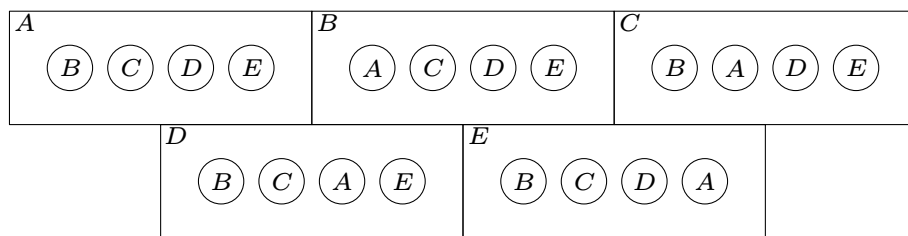
- (A) 最短的是 q_2 到 q_1 到 q_7 ，长度为 14。
 (B) 最短的是 q_0 到 q_3 到 q_4 到 q_8 ，长度为 23。
 (C) 这两个顶点之间没有路径。

V.2.45 这是一个很自然的例子。每个区域都与另外三个相邻，所以三种颜色不够。



V.2.46

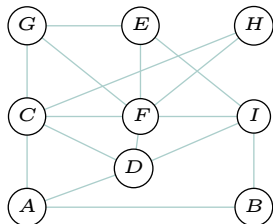
- (A) 这张地图展示了五个国家。它们不是连续的；例如，国家 A 有五个独立的部分（第一个部分包含 B、C、D 和 E）。



由于左上角的部分，国家 A 必须使用与其他所有国家都不同的颜色。类似地，其他部分也使得所有其他国家彼此颜色不同。总共有五个国家，每个国家的颜色都与其他所有国家不同。

- (B) 做一个分成五个扇形的圆。

V.2.47 为方便参考，这里给出该图。

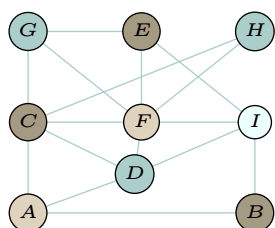


- (A) 假设三着色即可满足要求。顶点 A、C 和 D 构成一个三角形，所以它们必须是三种不同的颜色。由于 C、D、F 也构成一个三角形，我们得出结论：A 和 F 是同一种颜色。至此，我们有： $\mathcal{C}_0 = \{A, F\}$ ， $\mathcal{C}_1 = \{C\}$ ， $\mathcal{C}_2 = \{D\}$ 。

接着，考虑三角形 D、F、I。它的存在推出 C 和 I 是同一种颜色。所以我们有： $\mathcal{C}_0 = \{A, F\}$ ， $\mathcal{C}_1 = \{C, I\}$ ， $\mathcal{C}_2 = \{D\}$ 。然后三角形 E、F、I 推出： $\mathcal{C}_0 = \{A, F\}$ ， $\mathcal{C}_1 = \{C, I\}$ ， $\mathcal{C}_2 = \{D, E\}$ 。

最后，三角形 C、F、G 给出 D 和 G 是同一种颜色，从而得到： $\mathcal{C}_0 = \{A, F\}$ ， $\mathcal{C}_1 = \{C, I\}$ ， $\mathcal{C}_2 = \{D, E, G\}$ 。但这是不可能的，因为 G 与 E 有边相连。

(B) 这是一个四着色方案。



$$\begin{aligned} C_1 &= \{A, F\} \\ C_2 &= \{B, C, E\} \\ C_3 &= \{D, G, H\} \\ C_4 &= \{I\} \end{aligned}$$

V.2.48 如果一个公式的真值表最后一列中存在 T ，则该公式是可满足的。

(A) 公式 $(P \wedge Q) \vee (\neg Q \wedge R)$ 是可满足的。

P	Q	R	$P \wedge Q$	$\neg Q$	$\neg Q \wedge R$	$(P \wedge Q) \vee (\neg Q \wedge R)$
F	F	F	F	T	F	F
F	F	T	F	T	T	T
F	T	F	F	F	F	F
F	T	T	F	F	F	F
T	F	F	F	T	F	F
T	F	T	F	T	T	T
T	T	F	T	F	F	T
T	T	T	T	F	F	T

(B) 公式 $(P \rightarrow Q) \wedge \neg((P \wedge Q) \vee \neg P)$ 是不可满足的。

P	Q	$P \rightarrow Q$	$P \wedge Q$	$\neg P$	$(P \wedge Q) \vee \neg P$	$\neg((P \wedge Q) \vee \neg P)$	$(P \rightarrow Q) \wedge \neg((P \wedge Q) \vee \neg P)$
F	F	T	F	T	T	F	F
F	T	T	F	T	T	F	F
T	F	F	F	F	F	T	F
T	T	T	T	F	T	F	F

V.2.49

(A) 有四行输入使表达式取值为 F 。

行	子句
$F-T-F$	$P \vee \neg Q \vee R$
$F-T-T$	$P \vee \neg Q \vee \neg R$
$T-F-T$	$\neg P \vee Q \vee \neg R$
$T-T-T$	$\neg P \vee \neg Q \vee \neg R$

(B) 任何由 $\vee$ 连接元素构成的子句，除非所有元素都为 F ，否则其值为 T 。这些子句的构造方式使得在目标行上每个元素都为 F 。在任何其他行上，至少有一个元素为 T 。

(C) 合取范式为：

$$(P \vee \neg Q \vee R) \wedge (P \vee \neg Q \vee \neg R) \wedge (\neg P \vee Q \vee \neg R) \wedge (\neg P \vee \neg Q \vee \neg R)$$

这个语句取值为 F 当且仅当其至少一个组成子句取值为 F 。根据前一项，这当且仅当输入是某个目标行时才发生。

V.2.50 并非如此。举个例子，设布尔表达式为 $P \vee Q$ 。

P	Q	$P \vee Q$	$\neg(P \vee Q)$
F	F	F	T
F	T	T	F
T	F	T	F
T	T	T	F

取两个变量均为 T 时，它是可满足的。而其否定式在取两个变量均为 F 时也是可满足的。

V.2.51 一个有限命题逻辑语句集合 $\{S_0, \dots, S_{k-1}\}$ 是可满足的，当且仅当单个语句 $S_0 \wedge S_1 \dots \wedge S_{k-1}$ 是可满足的。

五个顶点的组合					缺失的一条边	五个顶点的组合					缺失的一条边
v_0	v_1	v_2	v_3	v_4	$v_1 v_3$	v_0	v_2	v_3	v_4	v_6	$v_0 v_4$
v_0	v_1	v_2	v_3	v_5	$v_2 v_5$	v_0	v_2	v_3	v_5	v_6	$v_0 v_6$
v_0	v_1	v_2	v_3	v_6	$v_2 v_6$	v_0	v_2	v_4	v_5	v_6	$v_0 v_4$
v_0	v_1	v_2	v_4	v_5	$v_0 v_4$	v_0	v_3	v_4	v_5	v_6	$v_0 v_4$
v_0	v_1	v_2	v_4	v_6	$v_0 v_6$	v_1	v_2	v_3	v_4	v_5	$v_1 v_3$
v_0	v_1	v_2	v_5	v_6	$v_0 v_6$	v_1	v_2	v_3	v_4	v_6	$v_1 v_3$
v_0	v_1	v_3	v_4	v_5	$v_0 v_4$	v_1	v_2	v_3	v_5	v_6	$v_1 v_3$
v_0	v_1	v_3	v_4	v_6	$v_0 v_6$	v_1	v_2	v_4	v_5	v_6	$v_1 v_5$
v_0	v_1	v_3	v_5	v_6	$v_0 v_6$	v_1	v_3	v_4	v_5	v_6	$v_1 v_3$
v_0	v_1	v_4	v_5	v_6	$v_0 v_6$	v_2	v_3	v_4	v_5	v_6	$v_2 v_5$
v_0	v_2	v_3	v_4	v_5	$v_0 v_4$						

图 98, FOR QUESTION V.2.58: 所有可能的 5-团的检验。

V.2.52

- (A) AT&T 与 Citigroup 相连, 后者与 Ford Motor 相连。
 (B) Haliburton 与 Citigroup 相连, 后者与 Ford Motor 相连。
 (C) Caterpillar 与 Ford Motor 之间没有连接。
 (D) 同样没有连接。

V.2.53 (A) 2 (B) 2 (C) 0 (D) 4 (我只在删减片段中出现过。不过这仅供娱乐。)

V.2.54

- (A) 一个包含 $B = 2$ 个元素的顶点覆盖是 $S = \{v_1, v_3\}$ 。一个包含 $\hat{B} = 4$ 个元素的独立集是 $\hat{S} = \{q_0, q_2, q_4, q_5\}$ 。
 (B) 一个包含 $B = 3$ 个元素的顶点覆盖是 $S = \{v_1, v_2, v_4\}$ 。一个包含 $\hat{B} = 3$ 个元素的独立集是 $\hat{S} = \{v_0, v_3, v_5\}$ 。
 (C) 一个包含 $B = 4$ 个元素的顶点覆盖是 $S = \{v_2, v_5, v_8, v_9\}$ 。一个包含 $\hat{B} = 6$ 个元素的独立集是 $\hat{S} = \{v_0, v_1, v_3, v_4, v_6, v_7\}$ 。
 (D) 对于任意一条边, 如果它至少有一个端点属于 S , 那么它在补集 $\hat{S} = \mathcal{N} - S$ 中至多有一个端点。反之, 如果一条边在 $\hat{S}$ 中至多有一个端点, 那么它在补集 $S = \mathcal{N} - \hat{S}$ 中至少有一个端点。

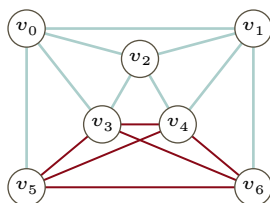
V.2.55 一个 3-团是一个三角形。一个 2-团是一条边。

V.2.56 一个 k -团有 $k(k-1)/2$ 条边, 即每对顶点之间都有一条边。

V.2.57 我们可以列出所有顶点三元组, 然后逐一检查它们是否在图中构成团。但我们这里选择目测, 结果如下。

$$\{v_0, v_1, v_2\}, \{v_0, v_1, v_6\}, \{v_0, v_2, v_6\}, \{v_2, v_3, v_4\}, \{v_2, v_3, v_5\}, \{v_2, v_4, v_5\}, \{v_3, v_4, v_5\}$$

V.2.58 它有一个 4-团, 因此必然有 3-团。



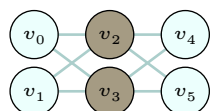
但它没有 5-团, 这通过 Figure 98 on page 184 中的穷举法得以验证。

V.2.59 右图的划分并没有将顶点分割成两个不相交的集合, 而是分成了五个集合: $\{v_0\}$ 、 $\{v_1\}$ 、 $\{v_2, v_3\}$ 、 $\{v_4\}$ 和 $\{v_5\}$ 。

V.2.60 图割 $\{\{v_0, v_1, v_4, v_5\}, \{v_2, v_3\}\}$

21	33	49	42	19	总和	21	33	49	42	19	总和
N	N	N	N	N	0	Y	N	N	N	N	21
N	N	N	N	Y	19	Y	N	N	N	Y	40
N	N	N	Y	N	42	Y	N	N	Y	N	63
N	N	N	Y	Y	61	Y	N	N	Y	Y	82
N	N	Y	N	N	49	Y	N	Y	N	N	70
N	N	Y	N	Y	68	Y	N	Y	N	Y	89
N	N	Y	Y	N	91	Y	N	Y	Y	N	112
N	N	Y	Y	Y	110	Y	N	Y	Y	Y	131
N	Y	N	N	N	33	Y	Y	N	N	N	54
N	Y	N	N	Y	52	Y	Y	N	N	Y	73
N	Y	N	Y	N	75	Y	Y	N	Y	N	96
N	Y	N	Y	Y	94	Y	Y	N	Y	Y	115
N	Y	Y	N	N	82	Y	Y	Y	N	N	103
N	Y	Y	N	Y	101	Y	Y	Y	N	Y	122
N	Y	Y	Y	N	124	Y	Y	Y	Y	N	145
N	Y	Y	Y	Y	143	Y	Y	Y	Y	Y	164

图 99, FOR QUESTION V.2.63: 子集和实例中的所有可能情况。



给出了一个包含八个元素的割集, $\{v_0v_2, v_0v_3, v_1v_2, v_1v_3, v_2v_5, v_3v_4, v_3v_5\}$ 。这个图割是极大的, 因为它的割集包含了图中所有的边。

V.2.61 三维匹配问题的定义要求三个集合, 本例中即教师、课程和时间段, 且它们大小相同, 此处均为五。输入必须是一个集合 $M \subseteq X \times Y \times Z$, 而此处的 M 由图表显示的所有可能三元组构成, 即 $M = \{(A, 1, \beta), (A, 1, \varepsilon), (A, 2, \gamma), \dots\}$ 。我们需要判定是否存在一个包含 n 个元素 (此处为五个元素) 的集合 $\hat{M} \subseteq M$, 使得 $\hat{M}$ 中任意两个三元组的第一坐标互不相同、第二坐标互不相同、第三坐标也互不相同。这恰好符合本问题的要求, 因为我们不能让同一位教师教两个班, 不能让两位教师教同一个班, 也不能让两个班在同一时间段使用教室。

一个合适的匹配是 $\hat{M} = \{(A, 2, \varepsilon), (B, 0, \alpha), (C, 3, \gamma), (D, 1, \beta), (E, 4, \delta)\}$ 。

V.2.62

(A) 集合 $M = X \times Y \times Z$ 包含以下二十七个三元组 $\langle x, y, z \rangle$ 。

$z = a$:

$\langle a, b, a \rangle$ $\langle a, c, a \rangle$ $\langle a, d, a \rangle$
 $\langle b, b, a \rangle$ $\langle b, c, a \rangle$ $\langle b, d, a \rangle$
 $\langle c, b, a \rangle$ $\langle c, c, a \rangle$ $\langle c, d, a \rangle$

$z = d$:

$\langle a, b, d \rangle$ $\langle a, c, d \rangle$ $\langle a, d, d \rangle$
 $\langle b, b, d \rangle$ $\langle b, c, d \rangle$ $\langle b, d, d \rangle$
 $\langle c, b, d \rangle$ $\langle c, c, d \rangle$ $\langle c, d, d \rangle$

$z = e$:

$\langle a, b, e \rangle$ $\langle a, c, e \rangle$ $\langle a, d, e \rangle$
 $\langle b, b, e \rangle$ $\langle b, c, e \rangle$ $\langle b, d, e \rangle$
 $\langle c, b, e \rangle$ $\langle c, c, e \rangle$ $\langle c, d, e \rangle$

(B) 一个正确答案是 $\hat{M} = \{\langle a, b, a \rangle, \langle b, c, d \rangle, \langle c, d, e \rangle\}$ 。

V.2.63 图 Figure 99 on page 185 展示了对所有可能情况的检查。

V.2.64 Figure 100 on page 186 展示了一个小脚本的运行结果。

V.2.65 选举人票总数为 538 张。很容易构造出一个 269-269 的平局。例如, CA、TX、FL、NY、IL、PA、OH、GA、MI、NC、VA 和 VT 这些州的选举人票加起来正好是 269。

V.2.66 共有二十五个: 2、3、5、7、11、13、17、19、23、29、31、37、41、43、47、53、59、61、67、71、73、79、83、89 和 97。

V.2.67

(A) 是, 它是素数。

(B) 否, 它不是素数。除了平凡因数, 6165 的因数还有 3, 5, 9, 15, 45, 137, 411, 685, 1233 和 2055。

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	重量	价值	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	重量	价值
N	N	N	N	N	0	0	Y	N	N	N	N	21	50
N	N	N	N	Y	19	40	Y	N	N	N	Y	40	90
N	N	N	Y	N	42	44	Y	N	N	Y	N	63	94
N	N	N	Y	Y	61	84	Y	N	N	Y	Y	82	134
N	N	Y	N	N	49	34	Y	N	Y	N	N	70	84
N	N	Y	N	Y	68	74	Y	N	Y	N	Y	89	124
N	N	Y	Y	N	91	78	Y	N	Y	Y	N	112	128
N	N	Y	Y	Y	110	118	Y	N	Y	Y	Y	131	168
N	Y	N	N	N	33	48	Y	Y	N	N	N	54	98
N	Y	N	N	Y	52	88	Y	Y	N	N	Y	73	138
N	Y	N	Y	N	75	92	Y	Y	N	Y	N	96	142
N	Y	N	Y	Y	94	132	Y	Y	N	Y	Y	115	182
N	Y	Y	N	N	82	82	Y	Y	Y	N	N	103	132
N	Y	Y	N	Y	101	122	Y	Y	Y	N	Y	122	172
N	Y	Y	Y	N	124	126	Y	Y	Y	Y	N	145	176
N	Y	Y	Y	Y	143	166	Y	Y	Y	Y	Y	164	216

图 100, FOR QUESTION V.2.64: 一个穷举脚本的输出, 用以说明该 背包问题 实例无解。

- (C) 是。
(D) 否。除了它自身和 1, 这个数还能被 7 和 601 整除。
(E) 否。除了它自身和 1 这两个平凡因数, 这个数还有因数 3, 11, 33, 233, 699 和 2563。

V.2.68

- (A) 一个是 3。非平凡因数的完整列表是 3, 9, 3469 和 10407。
(B) 一个是 2。非平凡因数的完整列表是 2, 4, 8, 6553, 13106 和 26212。
(C) 一个是 2。非平凡因数的完整列表是 2, 3, 4, 5, 6, 8, 10, 12, 15, 16, 20, 24, 25, 30, 32, 40, 48, 50, 60, 64, 75, 80, 96, 100, 120, 128, 150, 160, 192, 200, 240, 300, 320, 384, 400, 480, 600, 640, 800, 960, 1200, 1600, 1920, 2400, 3200 和 4800。
(D) 一个是 61。非平凡因数的完整列表是 61, 71。
(E) 它的因数只有 1 和 877。它是素数。

V.2.69 解决素性判定的 AKS 算法只回答“是”或“否”, 而不会找出具体的因数。

V.3.7 注: 此处给出的含义为常见用法。但由于这些术语没有正式定义, 不同作者的用法可能略有不同。

- (A) 启发式是一种捷径, 是处理问题的一种方式: 它比简单猜测更可靠, 比完整解法更快 (或者也许根本不存在完整解法)。但“启发式”一词的内涵在于, 它不是理论上正确的完整解法。

一个例子是用贪心算法来处理旅行商问题, 即总是前往最近的尚未访问的城市。另一个例子是反病毒程序使用启发式方法来判断一个可执行文件是否带有恶意, 例如将文件开头的几个字节与已知恶意字节序列的列表进行比对。

第三个例子基于 Figure 101 on page 187 中的数据。在 1900 年以来的美国总统竞选中, 较高的候选人赢得了 31 次中的 21 次, 样本比例为 0.68 (不计 1992 年)。纯属巧合出现这一结果的概率不到百分之四。你可以决定在下次选举中押注较高的候选人。这虽不合逻辑, 但确实是一种方法, 而且在实践中有人使用。

因此, 启发式是一种可能在实践中使用、但不保证最优、完美甚至合理的方法。相比之下, 针对某个问题的算法保证能解决该问题。

- (B) 伪代码是对算法的描述。它给出一个草图或大纲, 通常层次相当高。它的排版常常看起来像是可编译的代码, 但通常不遵循任何特定语言的语法; 例如, 它一般不包含变量声明之类的细节。
以下是二叉树递归深度优先遍历的伪代码。

年份	获胜者	对手	较高者？	年份	获胜者	对手	较高者？
1900	W McKinley	W J Bryan	N	1964	L B Johnson	B Goldwater	Y
1904	T Roosevelt	A B Parker	Y	1968	R Nixon	H Humphrey	Y
1908	W H Taft	W J Bryan	Y	1972	R Nixon	G McGovern	N
1912	W Wilson	– 其他两位 –	N	1976	J Carter	G Ford	N
1916	W Wilson	C E Hughes	Y	1980	R Reagan	J Carter	Y
1920	W G Harding	J M Cox	Y	1984	R Reagan	W Mondale	Y
1924	C Coolidge	J W Davis	N	1990	G H W Bush	M Dukakis	Y
1928	H Hoover	A Smith	Y	1992	W Clinton	G H W Bush	相同
1932	F D Roosevelt	H Hoover	Y	1996	W Clinton	R Dole	Y
1936	F D Roosevelt	A Landon	Y	2000	G W Bush	A Gore	N
1940	F D Roosevelt	W Willkie	N	2004	G W Bush	J Kerry	N
1944	F D Roosevelt	T Dewey	Y	2008	B Obama	J McCain	Y
1948	H S Truman	T Dewey	Y	2012	B Obama	M Romney	N
1952	D Eisenhower	A Stevenson II	Y	2016	D Trump	H Clinton	Y
1956	D Eisenhower	A Stevenson II	Y	2020	J Biden	D Trump	N
1960	J F Kennedy	R Nixon	Y	2024	D Trump	K Harris	Y

图 101, FOR QUESTION V.3.7: 1900 年以来美国总统获胜者与主要对手身高比较。

```

process_subtree(v)
  process_subtree(v's left child)
  process_vertex(v)
  process_subtree(v's right child)

```

另一个例子描述了检查一个数是否为素数的最简单算法，即用所有比它小但大于一的自然数来除它。

```

is_prime(n)
  for 2 <= i < n
    if i divides n
      return False
  return True

```

本书中还出现了许多其他例子，通常以流程图形式给出。

因此，伪代码接近于算法的规格说明。但在通常用法中，算法是完全确定的，使得程序员在实现时无需再确定任何重要细节，而伪代码可能遗留一些细节未加指定。（打个比方：一位有经验的厨师可能会这样描述土豆的做法：“将切薄片的土豆放在白酱汁里烤，上面撒上面包屑。”这段描述并不完整，因为它依赖于实施者对许多细节的理解，但它正朝着一份食谱的方向前进。）

- (C) 当然，在第一章第一节中，我们见过许多 Turing 机，例如一台计算前驱函数的，另一台计算两个输入之和的，还有一台永不停机的。

Turing 机更像程序而非算法，因为它是算法的实现。例如，我们可以想到冒泡排序算法，然后用一台 Turing 机来实现它。

- (D) 流程图是以图形形式呈现的伪代码。它描述算法，通常在较高层次上（有时甚至比通常用于伪代码的层次更高）。我们在本书中见过许多流程图，包括本章中出现的。流程图与算法的区别，和伪代码与算法的区别相同。
- (E) 源代码与算法的不同之处在于，它是算法的实现（有时是将多个算法捆绑在一起构成一份源代码）。源代码面向特定的编译器或解释器，可能还面向特定的硬件和软件平台。

例如，有一段用 Racket 编写的源代码，实现了如下算法：通过尝试用所有小于 n 的数来除 n ，以判定输入自然数 n 是否为素数。

- (F) 可执行文件是可在操作系统上运行的二进制文件。它通常是编译源代码的产物，因此也是算法的实现。三个例子是：编译（和链接）C 语言“Hello World!”程序所得的可执行文件、你的浏览器的可执行文件，以及排版输入 L^AT_EX 代码的可执行文件。

(g) 进程是已加载到机器内存中等待执行的程序实例。因此，它是算法（也可能是多个算法）的实现。三个例子是：调用“Hello World!”程序的可执行文件所产生的进程、调用浏览器可执行文件所产生的进程，以及调用 $\text{\LaTeX}$ 可执行文件所产生的进程。

有些进程（例如操作系统 shell 或 web 服务器）被设计为永远运行，而在计算复杂性的讨论中，“算法”一词通常指执行某项任务后即停机的事物。

V.3.8 解决方案就是一个作为该字符串集合特征函数的算法。

V.3.9 判定问题是指任何需要回答“是”或“否”的问题。语言判定问题是其特例，特指针对给定语言中的成员资格回答“是”或“否”。（然而，我们通常把语言当作我们正在处理的问题的精确描述，所以这两个术语有时会不经意地混用。）

V.3.10

(A) 乘法很简单： $(1/0.01)^2 \cdot 21 \cdot 30 = 6.30 \times 10^6$ 。

(B) $1 + \lfloor \lg(1\,000\,000) \rfloor = 20$

(C) 相乘得 $(6.30 \times 10^6) \cdot 20 = 1.26 \times 10^8$ 。一兆字节 (megabyte) 有八百万比特。所以总量不到 16 兆字节，大约相当于 Project Gutenberg 的八本书，或者一段十分钟的电话视频 (H.264 编码, 1080p 分辨率)。

V.3.11 区别在于，搜索问题的输出不由输入唯一决定。例如，如果输入查询要求给出一个包含名为 Springfield 的城市的美国州，那么一个正确的输出可以是阿拉斯加，另一个可以是佛蒙特。但函数必须是良定义的，所以输出必须由输入唯一确定。

V.3.12 正确。算法被明确定义的要求，意味着两种实现必须具有相同的输入/输出行为。

其逆命题不成立；计算相同函数的两个程序不一定是实现了相同的算法。例如，两个程序可能都对输入字符串进行排序，但一个使用冒泡排序 (Bubble Sort) 而另一个使用快速排序 (Quicksort)。

V.3.13

(A) 识别一个已正确排序的字符串，与找到一种对未排序字符串进行排序的好方法（甚至是最好的方法），是不同的任务。

(B) 类似地，识别一个正确的排列，与在给定未排序字符串时生成该排列相比，感觉不同——而且容易得多。

V.3.14 这些算法通过返回 1 来表示输入是该语言的成员，通过返回 0 来表示它不是。

(A) 由于数字 4 是一个比素数大 1 的平方数，三个成员是 $\langle 0, 4 \rangle$ 、 $\langle 1, 3 \rangle$ 和 $\langle 2, 2 \rangle$ 。计算特征函数的算法是：输入序列，解包得到 $n, m \in \mathbb{N}$ ，然后对这两个数求和。若其和既是平方数又比某个素数大 1，则返回 1；否则返回 0。

(B) 语言 $\mathcal{L}$ 的三个串成员是 $\sigma_0 = 200$ 、 $\sigma_1 = 100$ 和 $\sigma_2 = 300$ 。算法是：若输入是单个 0 或以两个 0 结尾，则返回 1；否则返回 0。

(C) 语言 $\mathcal{L}_2$ 包含 11、110 和 1。一个自然的算法是：扫描输入串，同时动态统计 0 和 1 的个数。扫描结束后，若 1 的个数更多则返回 1；否则返回 0。

(D) 回顾一下，等于其自身反转的串称为回文。在 $\mathbb{B}^*$ 上的三个回文是 101、11 和 1。计算特征函数的方法是：读取并保存输入串，将其反转并与原串比较。若匹配则返回 1；否则返回 0。

V.3.15 这些算法在接受输入为语言成员时返回 1，否则返回 0。

(A) 空语言是 $\mathcal{L} = \emptyset = \{\}$ 。其特征函数为 $\mathbb{1}_{\mathcal{L}}(n) = 0$ 。解决此语言判定问题的算法是：对所有输入，返回 0。

(B) 此语言包含两个串 $\mathbb{B} = \{0, 1\}$ （严格来说这些是字符，而非长度为 1 的串，但本书在方便时不计较这种差异）。其特征函数是：

$$\mathbb{1}_{\mathbb{B}}(\sigma) = \begin{cases} 1 & \text{若 } \sigma = 0 \text{ 或 } \sigma = 1 \\ 0 & \text{其他情形} \end{cases}$$

计算该函数的算法显而易见。

(C) 语言 $\mathbb{B}^*$ 包含所有的比特串。解决该语言判定问题的算法将在给定任意比特串时返回 1。

V.3.16 这些算法在将输入作为语言成员接受时返回 1，在拒绝时返回 0。

(A) 对于算法，固定一个识别语言 $\mathbf{a}^*\mathbf{b}\mathbf{a}^*$ 的有限状态机 $\mathcal{M}$ 。读取输入，并模拟运行 $\mathcal{M}$ 处理该输入。

这将在有限步内终止。如果 $\mathcal{M}$ 结束于一个接受状态，则返回 1；否则返回 0。

更简单地说，算法在输入仅由 $\mathbf{a}$ 和一个 $\mathbf{b}$ 组成时输出 1，否则输出 0。

- (B) 该文法定义的语言是 $\mathcal{L} = \{\mathbf{a}^n \mathbf{b}^m \mid n, m \in \mathbb{N}\}$ 。算法是：输入一个串，如果所有的 $\mathbf{b}$ 都出现在所有的 $\mathbf{a}$ 之后，则返回 1；否则返回 0。

V.3.17 我们将仅给出算法的思路。

首先我们观察到，存在一个算法，给定机器 $\mathcal{M}$ 和其中的一个状态 q ，能够找到从 q 可达的状态集合。该算法按步骤进行。步骤 1 初始化，令 $R_0 = \{q\}$ 。步骤 $i+1$ 从集合 R_i 开始， R_i 是从 q 经过 i 次或更少转移可达的状态。在这一步中，构造 R_{i+1} 时先以 R_i 为基础。然后加入从 R_i 中某个状态经过一次转移可达的任何状态。如果没有这样的新状态，即 $R_{i+1} = R_i$ ，那么算法停止。这个算法最终一定会终止，因为机器中的状态数是有限的。

- (A) 语言为空当且仅当从初始状态无法到达任何接受状态。
 (B) 给定 $\mathcal{M}$ ，构造一个新机器 $\hat{\mathcal{M}}$ ，它识别 $\mathcal{L}(\mathcal{M})$ 的补集，方法是复制 $\mathcal{M}$ ，但将 $\mathcal{M}$ 中的每个接受状态在 $\hat{\mathcal{M}}$ 中设为非接受状态，并将 $\mathcal{M}$ 中的每个非接受状态在 $\hat{\mathcal{M}}$ 中设为接受状态。那么 $\mathcal{L}(\mathcal{M}) = \Sigma^*$ 当且仅当 $\mathcal{L}(\hat{\mathcal{M}})$ 为空，从而我们归约到了本练习的第一项。
 (C) 语言是无限的当且仅当存在一个状态 $\hat{q}$ ，它可以从初始状态到达，并且从 $\hat{q}$ 可以到达某个接受状态，而且 $\hat{q}$ 可以通过一个非空串到达自身（即可以进行 pumping）。

V.3.18 这里我们仅给出每个算法的思路。

- (A) 检查输入串是否以 1 比特开头。
 (B) 十进制数 1000 转换为二进制是 1111101000，共十位。因此，如果输入是十一位，那么它在语言中。如果输入少于十位，那么它不在语言中。如果输入恰好是十位，那么有固定数量的情况需要处理，例如检查第二位是否为 0，但这并不改变算法是 $\mathcal{O}(1)$ 的事实。

V.3.19 这是一个搜索问题，因为我们只需要找出一座桥。

V.3.20 判定问题和语言判定问题有时不易区分。区别在于，后者是用语言来表述的。

- (A) 这是一个是或否的问题，因此是判定问题。
 (B) 函数问题。
 (C) 判定问题。
 (D) 搜索问题。
 (E) 搜索问题。
 (F) 判定问题。
 (G) 语言判定问题。

V.3.21 判定问题和语言判定问题有时不易区分。区别在于，后者是用语言来表述的。

- (A) 如问题 2.10 所述，它是一个判定问题。
 (B) 搜索问题。
 (C) 函数问题。
 (D) 搜索问题。
 (E) 判定问题。

V.3.22

- (A) 搜索问题。
 (B) 判定问题。
 (C) 搜索问题。
 (D) 优化问题。（问题类型之间的区别可能是模糊的。有人可能认为存在多个这样的子矩阵，因此称其为搜索问题。）

V.3.23 对每个任务，我们需要给出一个字符串集合。

- (A) 适用于一元输入的机器的语言是 $\mathcal{L}_0 = \{1^n \mid n \in \mathbb{N} \text{ 是完全平方数}\}$ 。适用于更常见输入的语言是 $\mathcal{L}_1 = \{d \in \{0, \dots, 9\}^+ \mid d \text{ 以十进制表示一个完全平方数}\}$ 。我们用 $\{0, \dots, 9\}^+$ 表示至少一位十进制数字的字符串。
 (B) 一个语言是如下字符串三元组集合：

$$\{(x, y, z) \in (\{0, \dots, 9\}^+)^3 \mid \text{解释为十进制表示, } x^2 + y^2 = z^2\}$$

我们用 $\{0, \dots, 9\}^+$ 表示至少一位十进制数字的字符串。

- (C) 一个语言是 $\{\sigma \in \mathbb{B}^* \mid \sigma \text{ 表示图 } \mathcal{G} = \langle \mathcal{N}, \mathcal{E} \rangle, \text{ 其中 } \mathcal{E} \text{ 的边数为偶数}\}$ 。
 (D) 该语言是 $\{\sigma \in \mathbb{B}^* \mid \sigma \text{ 表示序对 } \langle \mathcal{G}, p \rangle, \text{ 其中 } p \text{ 是含重复顶点的路}\}$ 。

V.3.24 对每个任务，我们需要给出一个字符串集合。

- (A) Graph Coloring 问题已表述为判定问题，所以我们只需要给出一个合适的语言。

$$\mathcal{L} = \{\sigma \in \mathbb{B}^* \mid \sigma \text{ 表示 } \langle \mathcal{G}, k \rangle, \text{ 其中 } \mathcal{G} \text{ 是 } k\text{-可着色的}\}$$

通常该语言会写成如下形式：

$$\mathcal{L} = \{\langle \mathcal{G}, k \rangle \mid \mathcal{G} \text{ 是 } k\text{-可着色的}\}$$

，尽管这种描述没有明确提到字符串表示。

- (B) 正文中给出的版本是搜索问题。这是语言判定问题版本对应的语言。

$$\{\sigma \in \mathbb{B}^* \mid \sigma \text{ 表示一个图，该图有一条恰好遍历每条边一次的回路}\}$$

- (C) Shortest Path 问题在正文中是作为函数问题给出的。这是其语言判定问题对应的语言。

$$\{\sigma \in \mathbb{B}^* \mid \sigma \text{ 表示 } \langle \mathcal{G}, v_0, v_1, p \rangle, \text{ 其中 } p \text{ 是两顶点间的最短路}\}$$

V.3.25 语言是字符串的集合。因此对于每个问题，集合的成员严格来说都是比特串。所以在第一个答案中，我们可以写成表示十进制数的比特串构成的集合，但通常无需强调这一细节。

- (A) $\{n \in \mathbb{N} \mid n \text{ 的因数之和大于 } 2n\}$ 。
 (B) $\{\langle \mathcal{P}, i \rangle \mid \text{机器 } \mathcal{P} \text{ 在输入 } i \text{ 上不到十步便停机}\}$
 (C) $\{E \mid \text{逻辑表达式 } E \text{ 可由三种不同指派满足}\}$
 (D) $\{\langle \mathcal{G}, B \rangle \mid \text{对所有 } v_0, v_1 \in \mathcal{G}, \text{ 均存在成本小于 } B \text{ 的路}\}$

V.3.26 它是关于判定语言 $\{\langle \mathcal{P}_e, e \rangle \mid \mathcal{P}_e \text{ 在输入 } e \text{ 上停机}\}$ 的成员资格。(为了强调语言是字符串集合，我们可以写 $\mathcal{L} = \{\sigma \in \mathbb{B}^* \mid \sigma \text{ 表示序对 } \langle \mathcal{P}_e, e \rangle, \text{ 且 } \mathcal{P}_e \text{ 在 } e \text{ 上停机}\}$ 。)

V.3.27 注意，本题并未明确要求语言判定问题，而是要求判定问题。

- (A) 给定三元组 $\langle n_0, n_1, p \rangle \in \mathbb{N}^3$ ，判断是否 $p = n_0 \cdot n_1$ 。
 (B) 给定三元组 $\langle \mathcal{G}, v_0, v_1 \rangle$ ，其中 $\mathcal{G}$ 是一个加权图，且 $v_0 \neq v_1$ 是该图中的顶点，判断 v_1 是否是离 v_0 最近的顶点。

V.3.28 这个族

$$\mathcal{L}_B = \{\langle \mathcal{G}, v_0, v_1 \rangle \mid \text{在 } \mathcal{G} \text{ 中存在一条从 } v_0 \text{ 到 } v_1 \text{ 的路，其总权小于等于 } B\}$$

由界 B 参数化。

V.3.29

- (A) 我们用 $\mathcal{B}$ 表示一个游戏棋盘，一个 4×4 数组，包含数字 1 到 14 以及一个空白格。将 $B \in \mathbb{N}$ 作为参数。

$$\mathcal{L}_B = \{\mathcal{B} \mid \mathcal{B} \text{ 可以在少于 } B \text{ 步内解开}\}$$

- (B) 将 $B \in \mathbb{N}$ 作为界。用 $\mathcal{R}$ 表示一个魔方配置，即一个由六个 3×3 数组组成的序列，这六个数组分别由集合 C_0 的九个成员、...、集合 C_5 的九个成员填充。

$$\mathcal{L}_B = \{\mathcal{R} \mid \mathcal{R} \text{ 可以在少于 } B \text{ 步内解开}\}$$

- (C) 将 $B \in \mathbb{R}^+ \cup \{0\}$ 作为参数。我们可以用一个序列来表示一种在满足约束条件下完成汽车的方式，其元素是任务 $C = \langle j_0, \dots, j_k \rangle$ 。对于每个这样的序列，都有一个总时间 $t(C)$ 。

$$\mathcal{L}_B = \{C \mid t(C) \leq B\}$$

V.3.30 函数版本输入一个图，输出所有 Hamilton 回路的集合。搜索版本输入一个图，如果存在 Hamilton 回路则输出一个，否则输出某个名义值，例如 0。优化版本根据某个准则输出一个最优的回路。例如，在加权图中，它返回权重最小的回路；这就是 Traveling Salesman 问题（旅行商）。

V.3.31 注意空顶点集和单顶点集都是独立的。所以每个图都有其顶点的某个子集是独立的。

- (A) 基于以上观察，我们不能合理地说，“给定一个图，判定它是否有独立集。”我们可以改用参数 B ：给定一个图，判定它是否有一个大小为 B 的独立集。
- (B) 对于 $B \in \mathbb{N}^+$ ，使用语言 $\mathcal{L}_B = \{\mathcal{G} \mid \mathcal{G} \text{ 有 } B \text{ 个互不相邻的顶点}\}$ 。
- (C) 给定一个图 G ，找出一个大小为 B 的独立集。（可能不止一个，也可能没有，但如果我们找到一个就算完成。）
- (D) 给定一个图，返回所有大小为 B 的独立集，或者字符串 **None**。
- (E) 给定一个图，返回一个极大的独立集，即对于该图尽可能大的独立集。

V.3.32 一个例子是在非空列表中寻找最大整数的任务。自然的判定问题“判断列表是否有最大值”是平凡的，因为每个列表都有最大值（可能有多个元素并列），因此属于 $\mathcal{O}(1)$ 。但其自然的搜索变体“找出最大值”需要遍历列表，因此属于 $\mathcal{O}(n)$ 。

V.4.13 正确。此类问题的求解算法基本上是一些固定数量的 `if ... then ...` 分支，因此任何输入所能触发的最大步骤数是一个常数。所以，该求解算法是 $\mathcal{O}(1)$ 的。

V.4.14 集合 P 包含的不是算法，而是问题。

有时人们会混淆“在多项式时间内运行”和“属于 P 类”。这是一个类型错误，因为“在多项式时间内运行”的东西的类型与“属于 P 类”的东西的类型是不同的。你的同事应该说：“我有一个问题，它的求解算法需要多项式时间”或者“我有一个属于 P 类的问题。”

V.4.15 增长阶是函数的集合。复杂度类是语言判定问题的集合。

V.4.16 你的朋友观察到，表示一个电路的字符串长度可能相对于电路深度是指数级的，因为一个深度为 r 的电路可能有大约 2^{r+1} 个门。但多项式时间的定义是指，所花费的步骤数是输入表示大小的多项式函数，而不是电路元素数量的函数。一个深度为 4 的电路只需 4 步。

V.4.17 这位学生的机器接受了所有输入。换句话说，学生提出的机器判定的是 Σ^* 。而定义所要求的是一台能判定给定语言 $\mathcal{L}$ 的机器，它要能确定输入串是否属于这个给定的语言。

V.4.18 正确；在表 1.27 中，对数函数在多项式函数之前。

V.4.19 正确，尽管不一定是同一台机器。设 $\mathcal{L}$ 由机器 $\mathcal{P}$ 判定，从而该机器具有这样的输入-输出行为。

$$\phi_{\mathcal{P}}(\sigma) = \mathbb{1}_{\mathcal{L}}(\sigma) = \begin{cases} 1 & \text{若 } \sigma \in \mathcal{L} \\ 0 & \text{其他情形} \end{cases}$$

通过交换输出的 0 和 1 来修改这台机器，得到一台具有互补行为的新机器 $\hat{\mathcal{P}}$ 。

$$\phi_{\hat{\mathcal{P}}}(\sigma) = \mathbb{1}_{\mathcal{L}^c}(\sigma) = \begin{cases} 0 & \text{若 } \sigma \in \mathcal{L} \\ 1 & \text{其他情形} \end{cases}$$

V.4.20 一个复杂度类就是一组语言的集合。所以两个这样的集合的并仍然是一个复杂度类。交集也是如此。对于补集，这里是指在全集 $\mathcal{P}(\mathbb{B}^*)$ 内取补集，其结果也是一个复杂度类。

V.4.21 一个可计算枚举集是某个可计算函数 ϕ_e 的值域。这与串集合相匹配， $\mathcal{L}_e = \{\text{str}(k) \mid k \in \mathbb{N}, \text{ 且存在输入 } i \in \mathbb{N} \text{ 这个汇集 } RE = \{\mathcal{L}_e \mid e \in \mathbb{N}\} \text{ 是一个复杂度类，因为它是语言的一个汇集。}$

V.4.22 我们必须证明，存在一个判定该集合成员资格的算法能在多项式时间内运行。任何直接的算法都可以。下面是一个例子：给定输入 σ ，首先将其反转。然后逐项比较 σ 和 σ^R 这两个串。该算法的运行时间为 $\mathcal{O}(|\sigma|)$ 。

V.4.23 这对应语言 $\{\langle m, n \rangle \mid \text{二者互素}\}$ 的判定问题。显然的解法是使用 Euclid 算法判断它们的最大公约数是否为 1。该算法在多项式时间内运行（事实上，在对数时间内）。

V.4.24 对每种情况，我们给出一个判定语言成员资格且在输入长度上运行时间为多项式的算法。

- (A) 给定 σ , 判定其长度是否能被三整除 (求长度需要约 $|\sigma|$ 步)。如果不能, 返回 0。若长度可被三整除, 则设 τ 为 σ 的前三分之一子串 (找到 τ 需要约 $|\sigma|$ 步)。然后, 仅通过遍历该串一次, 检查是否满足 $\sigma = \tau^3$ (这同样需要 $|\sigma|$ 步)。若 τ 符合条件则返回 1, 否则返回 0。

- (B) 表述为语言判定问题, 我们有 $\{i \in \mathbb{N} \mid \mathcal{P}_i \text{ 在空串上于 } 10 \text{ 步内停机}\}$ 。(我们也可以显式地使用带有 $\{\text{str}(i) \mid i \in \mathbb{N} \text{ 且 } \mathcal{P}_i \text{ 在空串上于 } 10 \text{ 步内停机}\}$ 的串。)

从输入索引 i 转换到 Turing 机 $\mathcal{P}_i$, 用通用 Turing 机模拟它等过程会产生一些开销。但暂且将这些视为常数开销, 那么无论输入 i 是什么, 我们只需运行模拟十步。因此, 确定表达式 $\mathcal{O}(x^n)$ 中确切指数的细节虽繁琐, 但该问题是 P 的一个元素。

V.4.25

- (A) 在团问题中, 团的大小是一个参数。给定一对 $\langle \mathcal{G}, B \rangle$, 并询问该图是否有一个大小为 B 的团。而此处团的大小固定为 $B = 3$ 。
- (B) 给定 $\mathcal{G}$, 枚举所有三元组 $\langle v_0, v_1, v_2 \rangle \in \mathcal{N}^3$ 并检查 v_0v_1 、 v_0v_2 和 v_1v_2 是否都是边, 即 $\mathcal{E}$ 的成员。这需要多项式时间。具体来说, 其中 $|\mathcal{N}| = n$, 有 $\binom{n}{2}$ 对 v_iv_j ; 大约是 n^2 个。我们检查这些对的三元组, 所以是 $\binom{n^2}{3}$, 即 $\mathcal{O}(|\mathcal{N}|^6)$, 这是多项式的。查找操作可能增加一个 $|\mathcal{E}|$ 的因子, 但无论如何, 整个过程都在多项式时间内完成。

V.4.26

- (A) 要判定该串是否按字母顺序排列, 仅单次遍历即可。对于除第一个外的每个项, 我们将其与前一项比较。由于字母表给定 $\Sigma = \{a, \dots, z\}$, 比较操作可在多项式时间内完成 (例如, 一个 26×26 的查找表就足够了), 因此, 这个单次遍历算法总体上需要多项式时间。
- (B) 如素性判定问题的讨论中所提及的, 问题 2.40, 判断一个数是合数还是素数已知可以在多项式时间内完成。
- (C) 我们可以给出一个在图描述的的长度上运行时间为多项式的算法。固定集合 $S_0 = \{v_0\}$ 。接着, 跟随从 v_0 出发的每条边, 得到在至多一步内连接到 v_0 的顶点集合, $S_1 = S_0 \cup \{v \mid \{v_0, v\} \in \mathcal{E}\}$ 。之后, 对于 S_1 中的所有顶点, 跟随所有边, 得到在至多两步内连接到 v_0 的所有顶点的集合 $S_2 = S_1 \cup \{v \mid \{v, v\} \in \mathcal{E}, \text{ 其中 } v \in S_1\}$ 。这个过程必然在某个点停止, 达到 $S_{i+1} = S_i$, 因为图是有限的。然后, 根据 $v_1 \in S_i$ 与否, 即可判断是否存在从 v_0 到 v_1 的路径。该算法的步数 i 以图中顶点数为界, 因此该算法相对于图表示的大小是多项式的。

V.4.27

- (A) 根据 Dijkstra 算法, Shortest Path 问题可以在多项式时间内解决。所以该问题属于 P 。
- (B) Knapsack 被认为相当困难 (在后续章节中我们将看到它是 NP -完全的)。目前没有人知道它的多项式时间算法。
- (C) Euler Path 很简单; 我们只需要检查每个节点的度数是否为偶数。存在线性时间的算法来解决它。
- (D) 目前没有人知道 Hamiltonian Circuit 的多项式时间算法。(在后续章节中我们将看到它是 NP -完全的。)

V.4.28 是。接收单个输入并返回 0 的算法计算的是空语言的特征函数, 并且显然在多项式时间内运行。

V.4.29

- (A) 我们关于复杂度类的定义是完全通用的 — 它就是语言的集合。根据这一定义, 正则语言的集合构成一个复杂度类。
- (B) 是的。一个语言是正则的, 当且仅当它能被一个有限状态机接受。对于每个正则语言, 我们可以构造一个模拟该有限状态机的 Turing 机, 并返回 0 或 1 来表示拒绝或接受输入串。每个输入串 σ 被其有限状态机处理的时间是 $\mathcal{O}(|\sigma|)$, 因为机器仅仅是读取字符并转移到指定的状态。所以, 正则语言的集合在 P 中。

V.4.30 当然, P 是那些拥有在多项式时间内运行的判定器的语言的集合。所以, 语言的数量受判定器数量的限制。有可数多个 Turing 机可用作判定器, 因此 P 是可数的。

V.4.31 不一定。取 $\mathcal{L}_0 = \emptyset$ 和 $\mathcal{L}_1 = \mathcal{P}(\mathbb{B}^*)$ 。因为 P 是可数的 (由于只有可数多个 Turing 机可用作判定器), 并且因为所有语言的集合 $\mathcal{P}(\mathbb{B}^*)$ 是不可数的, 所以存在不可数多个语言满足 $\mathcal{L}_0 \subseteq \mathcal{L} \subseteq \mathcal{L}_1$ 。因此, 至少有一个这样的语言不在 P 中。

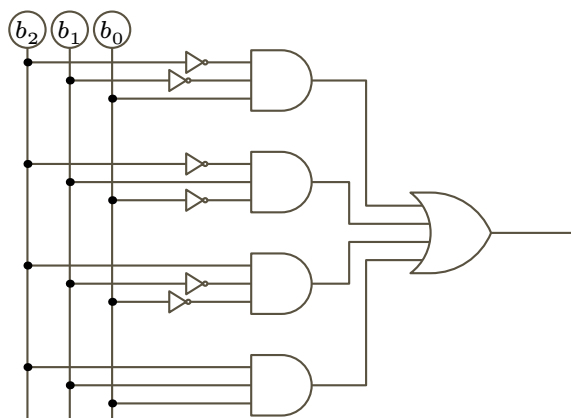


图 102, FOR QUESTION V.4.33: 电路求值问题的初步电路。

V.4.32 不在。我们当然可以将其表述为一个语言判定问题，使用语言 $\mathcal{L} = \{n \in \mathbb{N} \mid \phi_n(n) \downarrow\}$ (为了显式地使用字符串， $\mathcal{L} = \{\text{str}(n) \mid n \in \mathbb{N} \text{ 且 } \phi_n(n) \downarrow\}$)。但要成为 P 的成员，必须存在一个 Turing 机来求解该问题 (并且该机在多项式时间内运行)。没有 Turing 机能判定 **停机** 问题，因此它不在 P 中。

V.4.33 该电路期望的输入输出行为如下。

b_2	b_1	b_0	$b_0 + b_1 + b_2$	$b_0 + b_1 + b_2 \pmod{2}$
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	2	0
1	0	0	1	1
1	0	1	2	0
1	1	0	2	0
1	1	1	3	1

下面的命题逻辑公式给出了该表格中的行为 (这是析取范式)。

$$(\neg b_2 \wedge \neg b_1 \wedge b_0) \vee (\neg b_2 \wedge b_1 \wedge \neg b_0) \vee (b_2 \wedge \neg b_1 \wedge \neg b_0) \vee (b_2 \wedge b_1 \wedge b_0)$$

Figure 102 on page 193 展示了该电路，其中使用了标准记号： $\square$ 表示“与”门， $\bigcup$ 表示“或”门， $\neg$ 表示“非”门。

这个电路并不完全符合问题要求，因为有些门不具有两个输入和一个输出。我们可以给命题逻辑公式加上括号，使得每个运算符都是二元的，这利用了 $x_0 \wedge x_1 \wedge x_2 = (x_0 \wedge x_1) \wedge x_2$ 这一事实，并且类似关系对“ $\vee$ ”也成立。

$$((\neg b_2 \wedge \neg b_1) \wedge b_0) \vee ((\neg b_2 \wedge b_1) \wedge \neg b_0) \vee ((b_2 \wedge \neg b_1) \wedge \neg b_0) \vee ((b_2 \wedge b_1) \wedge b_0)$$

Figure 103 on page 194 修改了前面的图示以反映这一点。

要得到一个有向无环图，我们必须处理好所有四个子句。对于第一个子句 $(\neg b_2 \wedge \neg b_1) \wedge b_0$ ，内部的括号需要一个实现 $\neg P \wedge \neg Q$ 的门，我们称此 Boolean 函数为 f_0 。然后将 $f_0(b_2, b_1)$ 与 b_0 用一个 $\wedge$ 门结合。对于第二个子句 $(\neg b_2 \wedge b_1) \wedge \neg b_0$ ，内部的括号需要一个实现 $\neg P \wedge Q$ 的门，我们称此 Boolean 函数为 f_1 。然后将 $f_1(b_2, b_1)$ 与 b_0 用一个实现 Boolean 函数 $P \wedge \neg Q$ 的门结合，我们称此函数为 f_2 。对于第三个子句 $(b_2 \wedge \neg b_1) \wedge \neg b_0$ ，内部的括号需要 f_2 ，然后将 $f_2(b_2, b_1)$ 与 b_0 再次用 f_2 函数结合。第四个子句只使用了 $\wedge$ 。下面是电路中用到的 Boolean 函数。

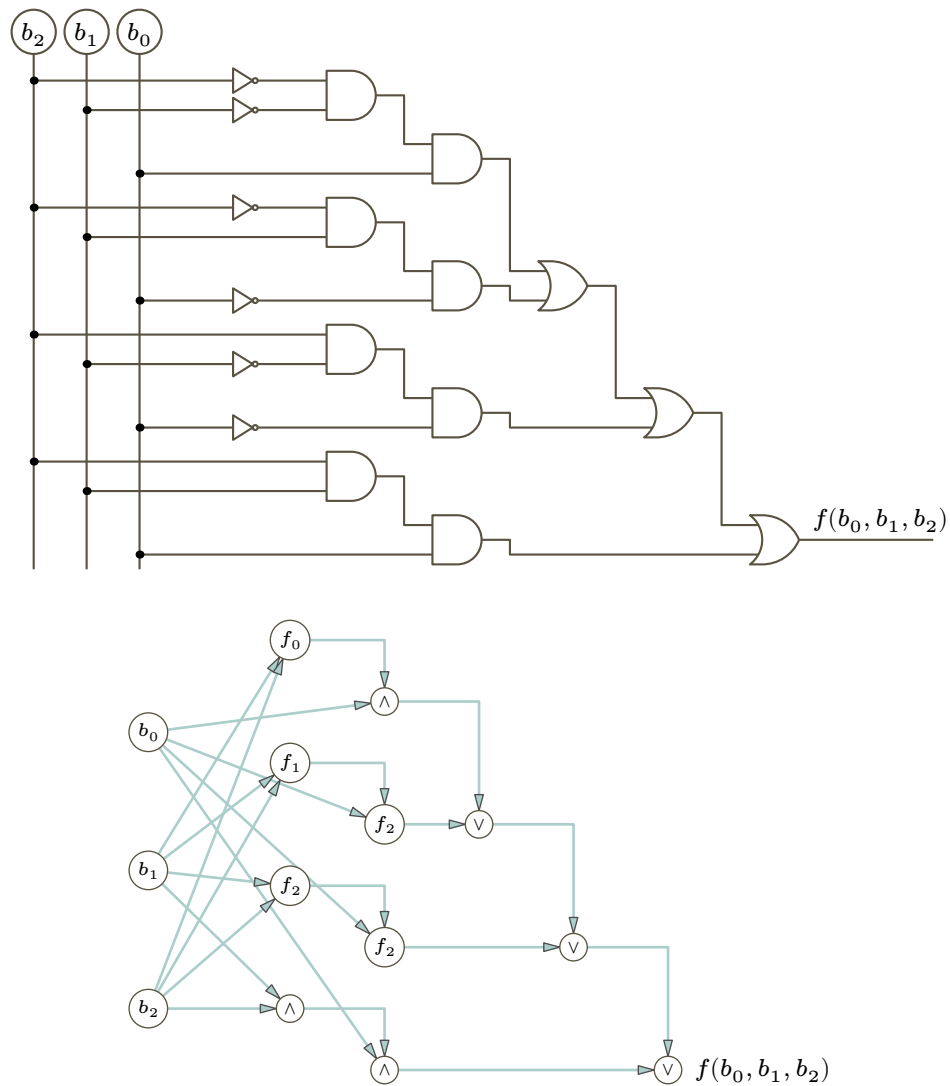


图 103, FOR QUESTION V.4.33: 电路求值问题的修正电路及其对应的 DAG。

P	Q	$\wedge$	P	Q	$\vee$	P	Q	f_0	P	Q	f_1	P	Q	f_2
0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0	1	0	0	1	1	0	1	0	0	1	1	0	1	0
1	0	0	1	0	1	1	0	0	1	0	0	1	0	1
1	1	1	1	1	1	1	1	0	1	1	0	1	1	0

V.4.34

- (A) 假设 $\mathcal{L}_0 \in P$ 的特征函数 ϕ_{e_0} 在时间 $\mathcal{O}(x^{k_0})$ 内运行, 并且假设 $\mathcal{L}_1$ 的特征函数 ϕ_{e_1} 在时间 $\mathcal{O}(x^{k_1})$ 内运行。交运算的标准算法是: 首先测试输入是否属于 $\mathcal{L}_0$, 如果通过, 则继续测试其是否属于 $\mathcal{L}_1$ 。该算法的时间复杂度为 $\mathcal{O}(x^k)$, 其中 $k = \max(\{k_0, k_1\})$ 。因此, 该算法是多项式时间的。
- (B) 我们必须证明, 如果一个语言 $\mathcal{L}$ 属于 P , 那么它的补集 $\mathcal{L}^c$ 也属于 P 。如果 $\mathcal{L} \in P$, 那么存在一台确定性 Turing 机 $\mathcal{P}$, 它在多项式时间内运行, 并且当且仅当 $\sigma \in \mathcal{L}$ 时接受输入串 $\sigma \in \mathbb{B}^*$ 。(回顾一下, 我们所说的“接受”是指机器停机, 并且停机时带上只有 1; 而“拒绝”是指带上只有 0。)换句话说, 存在一个可计算函数 $f: \mathbb{B}^* \rightarrow \mathbb{B}^*$, 使得

$$f(\sigma) = \begin{cases} 0 & \text{若 } \sigma \notin \mathcal{L} \\ 1 & \text{若 } \sigma \in \mathcal{L} \end{cases}$$

, 并且对于某个多项式 p , 计算 f 的时间是 $\mathcal{O}(p)$ 。(更准确地说, $\mathcal{O}(p)$ 指的是函数 $t(n)$, 其定义为对所有满足 $n = |\tau|$ 的 $\tau \in \mathbb{B}^*$, $f(\tau)$ 的最大运行时间。)

那么显然, 下面的函数也是可计算的, 并且也是多项式时间的: $\bar{f}(\sigma) = 1 - f(\sigma)$ 。因此, $\mathcal{L}^c \in P$ 。

V.4.35 取定一个语言 $\mathcal{L} \in P$ 。那么存在一台 Turing 机, 它能计算 $\mathcal{L}$ 的特征函数 $f: \mathbb{B}^* \rightarrow \mathbb{B}^*$, 并且对于某个指数 $n \in \mathbb{N}$, 其运行时间为 $\mathcal{O}(x^n)$ 。为了计算反转语言的特征函数 $\mathcal{L}^R = \{\tau^R \mid \tau \in \mathcal{L}\}$, 算法接收到一个输入串 τ 。它构造出反转串 τ^R , 并判定 $\sigma^R \in \mathcal{L}$ 是否成立。算法可以通过一次遍历输入来构造反转串, 而判定该反转串是否属于 $\mathcal{L}$ 需要时间 $\mathcal{O}(x^n)$, 所以总的运行时间是 $\mathcal{O}(x^k)$, 其中 $k = \max(\{1, n\})$ 。

V.4.36 取定两个语言 $\mathcal{L}_0, \mathcal{L}_1 \in \mathcal{P}(\mathbb{B}^*)$, 它们都是 P 的成员。那么存在指数 $n_0, n_1 \in \mathbb{N}$, 使得 $\mathcal{L}_0$ 能被一台运行时间为 $\mathcal{O}(x^{n_0})$ 的 Turing 机接受, 并且 $\mathcal{L}_1$ 能被一台运行时间为 $\mathcal{O}(x^{n_1})$ 的 Turing 机接受。

下面是一个自然的算法, 用于判定一个给定的输入串 $\sigma \in \mathbb{B}^*$ 是否是 $\mathcal{L}_0 \cap \mathcal{L}_1$ 的成员。我们将遍历该串, 首先将其分解为 $\sigma = \varepsilon \cap \sigma$, 然后分解为 $\sigma = \sigma[0] \cap \sigma[1:]$, 等等。在遍历的每个阶段, 我们都有 $\sigma = \alpha \cap \beta$, 并检查 $\alpha \in \mathcal{L}_0$ 且 $\beta \in \mathcal{L}_1$ 是否成立。

检查 $\alpha \in \mathcal{L}_0$ 需要时间 $\mathcal{O}(|\alpha|^{n_0})$, 而检查 $\beta \in \mathcal{L}_1$ 需要时间 $\mathcal{O}(|\beta|^{n_1})$ 。(为简单起见, 我们采用高估的方式, 使用 $|\sigma|$ 而不必考虑子串的长度。)因此, 遍历的每一步都可以在 $\mathcal{O}(|\sigma|^k)$ 时间内完成, 其中 $k = \max(\{n_0, n_1\})$ 。遍历本身需要 $|\sigma| + 1$ 步, 所以整体上算法在多项式时间内运行, 即 $\mathcal{O}(|\sigma|^m)$, 其中 $m = 1 + \max(\{n_0, n_1\})$ 。

V.4.37 这需要一个归纳论证。

归纳基础是: 如果两个语言 $\mathcal{L}_0, \mathcal{L}_1 \in \mathcal{P}(\mathbb{B}^*)$ 是 P 中的元素, 那么它们的并集也是 P 中的元素。我们在引理 4.12 的证明中已经建立了这一点。

归纳步骤的假设是: 对于 i 个语言 ($2 < i < n$), 它们的并集 $\mathcal{L}_0 \cup \dots \mathcal{L}_i$ 在 P 中。现在考虑 $n+1$ 个语言。其并集是 $\mathcal{L}_0 \cup \dots \mathcal{L}_{i+1} = (\mathcal{L}_0 \cup \dots \mathcal{L}_i) \cup \mathcal{L}_{i+1}$ 。这是 P 中两个成员的并集。根据第一段的结论, 它因此是 P 的成员。

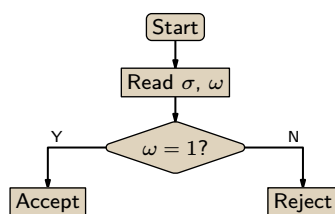
至于无限多个语言的并集, 注意到对于 $n \in \mathbb{N}$, 单元素语言 $\mathcal{L} = \{n\}$ 在 P 中, 因为判定器基本上就是一个 if .. then .. 语句, 所以运行时间是 $\mathcal{O}(1)$ 。而任何语言都是无限多个单元素语言的并集。这包括任何不在 P 中的语言。因此, P 在无限多个语言的并集下不是封闭的。

V.4.38 引理 4.12 指出 P 在补运算下封闭。也就是说, 如果 $\mathcal{L} \in P$, 那么 $\mathcal{L}^c \in P$ 。由此立即得到 $P \subseteq co-P$ 。

包含关系 $co-P \subseteq P$ 也由同一个结果得出, 因为如果 $\mathcal{L}^c \in P$, 那么由于 $\mathcal{L}^{cc} = \mathcal{L}$, 我们有 $\mathcal{L} \in P$ 。

V.5.16 那条引理要求验证器具有以下性质: 如果 $\sigma \in \mathcal{L}$, 则存在一个见证能导致接受; 而如果 $\sigma \notin \mathcal{L}$, 则不存在任何能导致验证器接受的见证。提议的这种验证器并不进行验证, 它只是盲目依赖

ω 的值。



因为它跳过了检查步骤，所以只要比特指示接受，它就会接受输入 σ ，即使 σ 根本不属于该语言。

举一个具体例子，考虑问题 $\mathcal{L} = \{n \in \mathbb{N} \mid n \text{ 为偶数}\}$ 。提议的验证器输入一个数对 $\langle \sigma, \omega \rangle$ ，其中 σ 是一个数， ω 是一个比特，然后当 $\omega = 1$ 时就接受这个数对。没错，给定数对 $\langle 5, 0 \rangle$ 时，验证器不会接受，这正是我们期望的，因为 $5 \notin \mathcal{L}$ 。然而，给定 $\langle 5, 1 \rangle$ 时，验证器却会接受。所以，对于 $\sigma = 5$ ，存在一个能导致接受的 ω ，这就违反了验证器的定义。

V.5.17 是“每个分支都拒绝”。

V.5.18 判断可满足性最简便的做法就是列真值表。

(A) 表达式 $(P \wedge Q) \vee (\neg Q \wedge R)$ 是可满足的。

P	Q	R	$P \wedge Q$	$\neg Q$	$\neg Q \wedge R$	$(P \wedge Q) \vee (\neg Q \wedge R)$
F	F	F	F	T	F	F
F	F	T	F	T	T	T
F	T	F	F	F	F	F
F	T	T	F	F	F	F
T	F	F	F	T	F	F
T	F	T	F	T	T	T
T	T	F	T	F	F	T
T	T	T	T	F	F	T

(B) 表达式 $(P \rightarrow Q) \wedge \neg((P \wedge Q) \vee \neg P)$ 是不可满足的。

P	Q	$P \rightarrow Q$	$P \wedge Q$	$\neg P$	$(P \wedge Q) \vee \neg P$	$\neg((P \wedge Q) \vee \neg P)$	$(P \rightarrow Q) \wedge \neg((P \wedge Q) \vee \neg P)$
F	F	T	F	T	T	F	F
F	T	T	F	T	T	F	F
T	F	F	F	F	F	T	F
T	T	T	T	F	T	F	F

V.5.19 这台机器不判定任何语言。要成为一台判定器，一台非确定性 Turing 机的计算树必须满足：对于所有输入，所有极大路径都是有限的。

V.5.20 不对。定义是：如果存在一条结束于接受格局的极大路径，则机器接受输入。说得稍微随意一点（或者说更有画面感），我们说如果机器存在一种方式能猜出一条引发接受的路径，它就接受其输入。

这个定义意味着两种模型在结论上是一致的。无界并行模型关心所有路径，而机器猜测模型关心是否存在一条路径，但我们需要它们在机器何时接受输入、何时拒绝上保持一致。

V.5.21

(A) 回溯法确实能解决这个问题。但回溯法是确定性的算法。（当然，确定性算法是非确定性算法的特例，但当你提出这一点时，你的教授会说：“这道练习的重点是展示对非确定性的理解，不是在钻牛角尖。”）

(B) 你的算法确实做了选择，但它是随机选择的。如果存在穿过迷宫的路径，但随机选择恰好没有导向这样的路径，那么你的算法就检测不到它们。一个非确定性算法要求：如果存在一条结束于接受格局的极大路径，它就接受输入的迷宫；如果不存在，它就不接受。

(C) 下面我们给出两种答案，一种使用无界并行的思维模型，一种使用选择模型。

对于无界并行模型，每当算法发现自己在迷宫中可以进行多种选择时，它就分支创建子进程来处理每一种可能性。所以它可能有大量分支，其中许多可能走进死胡同，但只要存在一条穿过迷宫

的路，这个算法就能找到它。就算只有一条通路，它也会接受输入的迷宫；而当没有通路时，它绝不接受。

基于选择的算法会进行选择。也就是说，机器猜出一条路径。（我们也可以说，它从一个恶魔那里收到了路径 ω 。）根据该思维模型的定义，只要存在一种方式能让机器猜对，机器就接受输入。（再次注意，这个算法不是随机猜测，它总能猜对。）

V.5.22 对。由引理 5.8，只需证明该语言判定问题属于 P 。而这很明显：给定三元组 $\langle a, b, c \rangle$ ，检查前两个数之和是否等于第三个数显然是一个多项式时间任务。

V.5.23 在两种模型中，输入都是一个数组 $\langle a_0, a_1, \dots, a_{n-1} \rangle$ 。

在无界并行模型中，机器为每个索引 $0 \leq i < n$ 派生子进程。子进程 i 检查 a_i 是否等于 k ，如果是，则向机器发出接受信号。

在猜测模型中，机器猜测一个索引 i ，如果发现 a_i 等于 k ，则发出接受信号。在此模型下，机器会接受与另一种模型相同的数组，因为根据定义，机器接受一个输入，当且仅当存在一条结束于接受格局的极大路径，也就是说，至少存在一种猜测是正确的。如果数组中有一个等于 k 的条目，那么就有办法猜中它，非确定性机器就会接受这个数组。如果数组中没有这样的条目，那么就没有任何猜测索引能指向 k ，非确定性机器就会拒绝这个输入数组。

在确定性机器上，运行时间是线性的，即 $\mathcal{O}(n)$ 。在两种非确定性模型中，运行时间都是常数的，即 $\mathcal{O}(1)$ 。例如，在无界并行模型中，机器查看每个条目，但不是依次查看，而是同时查看所有条目。

V.5.24 为了重新表述，我们可以将每个语言判定问题表示为一个元组

$$\langle d_0, \dots, d_n, a_{0,0}, \dots, a_{0,n}, b_0, \dots, a_{m,0}, \dots, a_{m,n}, b_m \rangle \quad (\text{每个数字都是整数})$$

它表示 $F(x_0, \dots, x_n) = d_0 x_0 + \dots + d_n x_n \geq B$ 且对于 $0 \leq i \leq m$ 有 $a_{i,0} x_0 + \dots + a_{i,n} x_n \leq b_i$ 。（注意，我们可以通过将数字取负来表示“ $\geq$ ”约束。）

一个无界并行算法会尝试所有对变量 x_j 的可行整数赋值。（如果赋值满足所有约束，则称其为“可行”的。）如果存在一个可行赋值，那么此算法将找到它并接受输入元组。如果不存在可行赋值，则输入不可能被接受。

一个猜测算法会猜测一个赋值 $\langle x_0, \dots, x_n \rangle$ 。如果存在一个可行赋值，那么就存在一种猜对它的方式，因此，根据此模型下机器接受的定义，机器接受该输入元组作为语言 $\mathcal{L}_B$ 的成员。

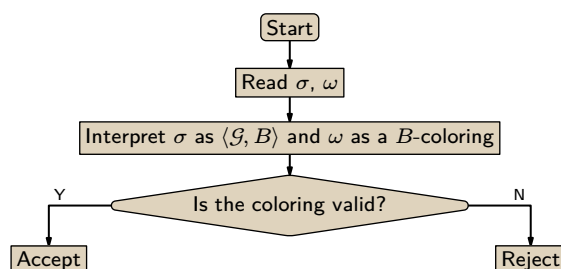
V.5.25 其对应的语言是 $\mathcal{L} = \{ \langle n \in \mathbb{N} \mid n \text{ 的素因数分解为 } n = p_0^{e_0} p_1^{e_1}, \text{ 其中 } e_0, e_1 > 0 \text{ 且 } p_0 \neq p_1 \text{ 为素数} \} \}.$

一种无界并行算法会同时尝试所有可能的素因数分解。（这里的“可能”指的是不包含任何大于输入数的素数）。例如，给定输入 8，它可以检查乘积 $2^0 3^0 5^0 7^0$ 、 $2^1 3^0 5^0 7^0$ 、... $2^7 3^7 5^7 7^7$ ，看是否有任何一个等于 8，并且恰好有两个素数的指数非零。乘法运算很快，检查一个数是合数还是素数也可以在多项式时间内完成。因此，每一次这样的检查都在多项式时间内完成。由于机器同时运行所有这些检查，总体上仍在多项式时间内运行。

猜测式算法是：机器以数字 n 为输入，并猜测一个由两个素数构成的因数分解，或者由恶魔给出一个。例如，根据此模型的定义，机器接受输入 45，因为它可以猜测 $3^2 5^1$ 。如果输入不是半素数，那么就无法猜出合适的因数分解。显然，这在多项式时间内运行。

V.5.26

- (A) 该语言是问题的直接翻译， $\mathcal{L} = \{ \langle \mathcal{G}, B \rangle \mid \mathcal{G} \text{ 是平面图且是 } B \text{ 可着色的} \}.$
 (B) Take the witness to be a coloring of the given graph with B 种颜色。
 (C)



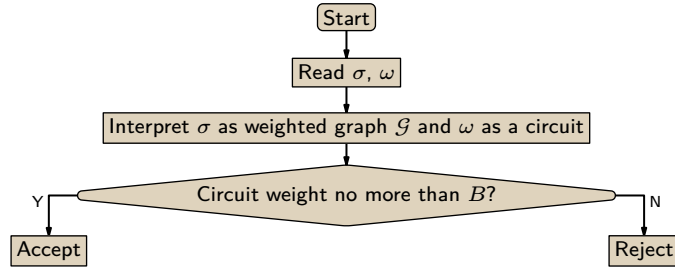
- (D) 引理 5.11 需要一个确定性验证器。其第一个输入 σ 是一个图和一个非零自然数的对 $\langle \mathcal{G}, B \rangle$ 。其第二个输入 ω 是 $\mathcal{G}$ 的 B -着色；我们可以将其视为机器的猜测，或恶魔的提示。

如果该图存在一个 B -着色，那么就存在一个合适的见证，并且验证器 $\mathcal{V}$ 可以在 $|\sigma|$ 的多项式时间内完成检查。如果该图没有 B -着色，那么任何 ω 都无法使得验证器接受其输入。

- (E) 设 $\mathcal{N}$ 为 $\mathcal{G}$ 的顶点集，最多需要检查 $|\mathcal{N}|$ choose 2 条边。这是 $\mathcal{O}(|\mathcal{N}|^2)$ ，即 $\mathcal{O}(|\mathcal{G}|^2)$ 。

V.5.27

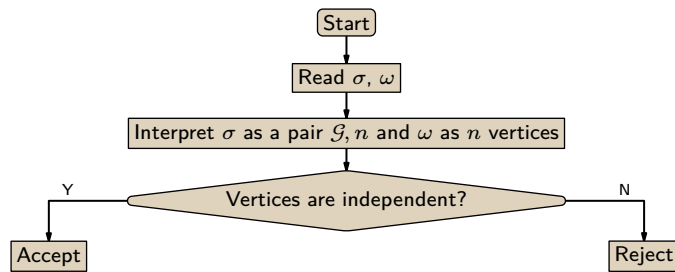
- (A) $\mathcal{L}_B = \{ \mathcal{G} \mid \mathcal{G} \text{ 是一个包含长度小于 } B \text{ 的回路} \}$
 (B) The witness is a circuit of weight no more than the bound B .
 (C)



- (D) 引理 5.11 需要一个确定性 Turing 机验证器 $\mathcal{V}$ 。它必须接收二元组 $\langle \sigma, \omega \rangle$ 作为输入。其中， σ 是一个作为语言成员候选的加权图 $\mathcal{G}$ ，而 ω 是一个总成本不超过 B 的见证回路。如果 $\sigma \in \mathcal{L}_B$ ，则存在一个见证回路 ω ，使得 $\mathcal{V}$ 接受这个输入二元组；而如果 $\sigma \notin \mathcal{L}_B$ ，则不存在可以验证的 ω ，因此对于该 σ ， $\mathcal{V}$ 不接受任何输入二元组。
- (E) 如果图有 n 个顶点，那么一条回路有 $n - 1$ 条边。检查回路中的顶点各不相同需要 n 步，而将边权相加所需的时间与边的数量以及权值的位数呈线性关系。因此，验证时间是 $|\sigma|$ 的多项式。

V.5.28

- (A) $\mathcal{L} = \{ \langle \mathcal{G}, n \rangle \mid \text{图 } \mathcal{G} \text{ 至少有 } n \text{ 个独立顶点} \}$
 (B) 给定一个图和一个自然数，见证是该图顶点的一个独立集。
 (C)



- (D) 引理 5.11 要求我们构造一个确定性 Turing 机验证器 $\mathcal{V}$ 。它的第一个输入 σ 是一个图和自然数的二元组 $\langle \mathcal{G}, n \rangle$ 。它的第二个输入 ω 是 $\mathcal{G}$ 的一个包含 n 个元素的独立顶点集。

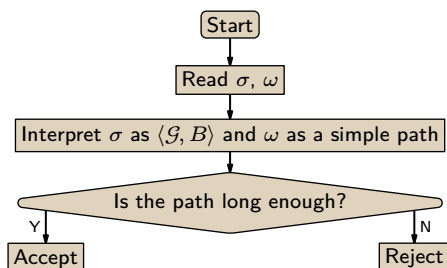
验证器检查这些顶点是否在图中，并确认任意两个顶点之间都没有边相连。这些检查所需的时间是 $|\sigma|$ 的多项式。

如果二元组 $\langle \mathcal{G}, n \rangle$ 确实是 $\mathcal{L}$ 中的一个元素，那么存在这样一个见证 ω ，因此存在一个验证器会接受的二元组 $\langle \sigma, \omega \rangle$ 。如果 $\langle \mathcal{G}, n \rangle$ 不是 $\mathcal{L}$ 的元素，那么就不存在这样的 ω ，因此也不存在验证器会接受的二元组。

- (E) 验证器必须对 ω 中的顶点两两检查，看它们在图中是否有边相连。图中边的数量是顶点数的多项式，因此验证时间也是 $|\sigma|$ 的多项式。

V.5.29

- (A) $\mathcal{L} = \{ \langle \mathcal{G}, B \rangle \mid \mathcal{G} \text{ 有长度至少为 } B \text{ 的简单路} \}$
 (B) 对于给定的图与界，见证是一条长度不小于该界的路径。
 (C)



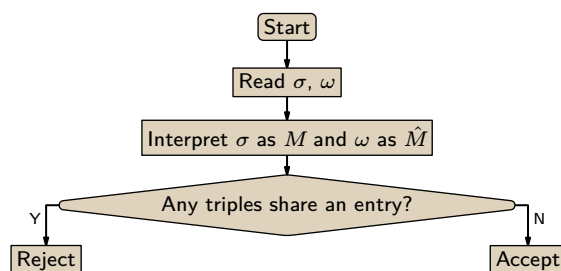
- (D) 引理 5.11 要求一个确定性 Turing 机验证器 $\mathcal{V}$ 。它接收对 $\langle \sigma, \omega \rangle$ 作为输入，其中 $\sigma = \langle \mathcal{G}, B \rangle$ 且 $\omega = \langle v_0, \dots, v_{n-1} \rangle$ 是一条边数不少于 B 的简单路径。

验证器检查该路径是否在图 $\mathcal{G}$ 中、顶点是否互不相同，以及长度是否至少为 B 。这些检查均可在 $|\sigma|$ 的多项式时间内完成。若 σ 确实属于 $\mathcal{L}$ ，则存在这样的路径 ω ，因而存在验证器会接受的对 $\langle \sigma, \omega \rangle$ 。若 σ 不属于 $\mathcal{L}$ ，则不存在这样的 ω ，因而也不存在验证器会接受的对。

- (E) 当路径 ω 的长度为 $n-1$ 时，通过遍历路径来检查顶点是否互不相同，运行时间为 $\mathcal{O}(n)$ ，这显然是 $|\sigma|$ 的多项式。

V.5.30

- (A) 该语言是如下三元组集合 M 的全体： $M \subseteq X \times Y \times Z$ ，其中 $|X| = |Y| = |Z| = n$ (对某个 $n \in \mathbb{N}^+$)，且 M 包含一个大小为 n 的子集 $\hat{M}$ ，使得其中没有两个三元组在任何一个坐标上相同。
- (B) 给定 $\sigma = M$ ，取 $\omega = \hat{M}$ (若存在)。
- (C)



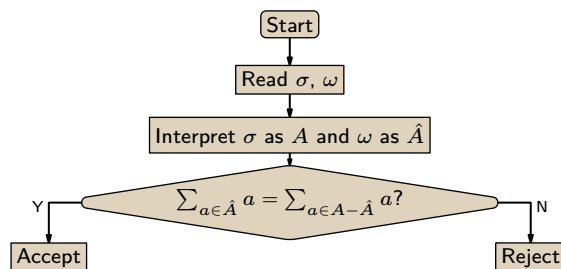
- (D) 验证器的输入 σ 是一个三元组集合 M ，其第一坐标取自某个大小为 n 的集合 X ，第二坐标取自某个大小为 n 的集合 Y ，第三坐标取自某个大小为 n 的集合 Z 。它还接收见证 ω 作为输入，即满足条件的 $\hat{M} \subseteq M$ (我们可以将其理解为非确定性机器的猜测或恶魔的提示)。

若 $\sigma = M$ 有这样的子多重集 $\hat{M}$ ，则验证 ω 是否有效可在 $|\sigma|$ 的多项式时间内完成。因此在这种情况下，存在以 σ 开头的输入对能使验证器接受。若对于输入 $\sigma = M$ 不存在这样的 $\hat{M}$ ，则对于任何 ω 验证器都无法验证通过，因此它不会接受任何以 σ 开头的输入对。

- (E) 检查没有两个三元组具有相同的第一坐标，需要进行 $|\hat{M}|$ 中取 2 的组合数次比较，即 $\mathcal{O}(|\hat{M}|^2)$ 。检查所有三个坐标需要三倍于此的步骤数，同样为 $\mathcal{O}(|\hat{M}|^2)$ 。由于 $\hat{M} \subseteq M$ ，故总时间为 $\mathcal{O}(|\sigma|^2)$ 。

V.5.31

- (A) $\mathcal{L} = \{A \mid A \text{ 是自然数多重集, 且有子多重集 } \hat{A} \subseteq A \text{ 满足 } \sum_{a \in \hat{A}} a = \sum_{a \in A - \hat{A}} a\}$
- (B) 见证 ω 就是子多重集 $\hat{A}$ 。
- (C)



该图位于?? on page ??的右侧。

(D) 引理 5.11 需要一个确定性验证器。它以问题实例 σ (即 M) 和见证 ω (即 $\hat{A}$) 作为输入 (我们可以将其视为非确定性机器的猜测或恶魔的提示)。若 $\sigma \in \mathcal{L}$, 则根据定义这样的 $\hat{A}$ 存在, 因此验证器至少会接受一个以 σ 开头的输入对 (验证在 $|\sigma|$ 的多项式时间内完成)。若 $\sigma \notin \mathcal{L}$, 则没有任何 ω 能通过验证, 故 $\mathcal{V}$ 不接受任何以 σ 开头的输入对。

(E) 将这两个多重集中的数相加所需时间是它们位数的多项式, 也是 $|\sigma|$ 的多项式。

V.5.32

(A) 这是语言 $\mathcal{L}$ 。

$\{\sigma \mid \sigma \text{ 表示一个命题逻辑陈述, 其真值表至少有两行以 } T \text{ 结束}\}$

合适的填空如下: (1) 一个命题逻辑语句 E ; (2) 一个对, 指向真值表中最终项为 T 的两行; (3) 对固定的命题逻辑语句求值所需时间是该语句长度的多项式, 因而也是 $|\sigma|$ 的多项式。

(B) 该问题表述为语言的判定问题如下:

$$\mathcal{L} = \{\langle S, T \rangle \mid \text{存在子多重集 } A \subseteq S \text{ 满足 } \sum_{a \in A} a = T\}$$

填空如下: (1) 一个由多重集和数构成的数对 $\langle S, T \rangle$; (2) S 的一个子多重集 $\hat{S}$, 其元素之和为 T ; (3) 加法耗时是输入数量的多项式, 又因 $\hat{S} \subseteq S$, 故也是 $|\sigma|$ 的多项式。

V.5.33 引理 5.11 要求我们构造一个确定性 Turing 机验证器 $\mathcal{V}$ 。它输入序对 $\langle \sigma, \omega \rangle$, 其中 σ 是一个问题实例 $\langle s_0, \dots, s_5, T \rangle \in S^6 \times \{100, \dots, 999\}!$ 。见证 ω 是一个算术表达式, 它计算得到目标数 T , 并且使用了数字 $s_0, \dots, s_5$ 的一个子多重集。

验证器检查该表达式是否确实计算得到目标值, 以及是否确实最多使用了每个数字一次。这些检查可以在 $|\sigma|$ 的多项式时间内完成。(简要说明: 两个 n 位数相加是 $\mathcal{O}(n)$, 因此六个 n 位数相加也是线性的。) 如果 $\sigma \in \mathcal{L}$, 那么根据定义存在这样的算术表达式, 因此 $\mathcal{V}$ 可以接受一个输入序对。如果 $\sigma \notin \mathcal{L}$, 那么不存在这样的算术表达式, 因此没有任何见证 ω 会使 $\mathcal{V}$ 接受。

V.5.34 背包问题是这个语言 $\mathcal{L}$ 的判定问题。

$$\{\langle (w_0, v_0), \dots, (w_{n-1}, v_{n-1}), B, T \rangle \mid \text{存在 } I = \{i_0, \dots, i_k\} \subseteq \{0, \dots, n-1\}, \text{ 使 } \sum_{i \in I} w_i \leq B \text{ 且 } \sum_{i \in I} v_i \geq T\}$$

总结来说, 我们取见证为一个索引集合 $\{i_0, \dots, i_k\} \subset \{0, \dots, n-1\}$ 来指明那些共同满足约束的物品。

更详细地说: 引理 5.11 要求构造一个确定性 Turing 机验证器 $\mathcal{V}$ 。它的第一个输入 σ 是一个问题实例 $\langle (w_0, v_0), \dots, (w_{n-1}, v_{n-1}), B, T \rangle$ 。如前段所述, 它的第二个输入指明了满足约束 B 和 T 的物品。

如果 $\sigma \in \mathcal{L}$, 那么存在一个见证 ω , 并且 $\mathcal{V}$ 可以在 $|\sigma|$ 的多项式时间内验证它。(论证如下: 两个 m 位整数相加的时间关于位数是线性的, 即 $\mathcal{O}(m)$ 。当有更多潜在物品要加入背包时, 表示 σ 会变大, 但验证时间仍然是 $|\sigma|$ 的线性函数。) 因此, 在这种情况下验证器接受其输入。如果 $\sigma \notin \mathcal{L}$, 那么不存在这样的 ω , 验证器将不会接受任何以 σ 开头的序对。

V.5.35 对于两者, 我们都使用引理 5.11。

(A) 以下是相应的语言。

$$\mathcal{L} = \{\langle a, b \rangle \in \mathbb{N}^2 \mid \text{存在 } x \in \mathbb{N} \text{ 使得 } ax + 1 = b\}$$

论证的简短版本是: 我们可以取一个合适的 x 作为见证。

更详细地说, 引理 5.11 要求我们构造一个确定性 Turing 机验证器 $\mathcal{V}$ 。它的第一个输入 σ 是数对 $\langle a, b \rangle$ 。它的第二个输入 ω 是一个数 x , 使得 $ax + 1 = b$ (如果存在的话)。

如果对于 a 和 b 确实存在这样的 x , 那么验证器可以检查 ω 是否有效, 即检查 $ax + 1 = b$ 。因为算术运算可以在多项式时间内完成, 所以验证器可以在 $|\sigma|$ 的多项式时间内完成检查。另一方面, 如果对于给定的 a 和 b 没有这样的数, 那么就不存在以 σ 开头且能让验证器接受其输入的序对。

(B) 这是对应的语言。

$$\mathcal{L} = \{M \mid \text{存在一个数字集合 } P \subset \mathbb{Z}, \text{ 使得 } M \text{ 是两两距离构成的多重集}\}$$

证明概要: 给定多重集问题实例 $\sigma = M$, 取见证 $\omega = P$ 。

更详细地说，我们需要构造一个确定性 Turing 机验证器 $\mathcal{V}$ ，它输入序对 $\langle \sigma, \omega \rangle$ ，其中 σ 是一个多重集 M 。我们取一组位置 $\omega = P$ 作为见证。

如果 $\sigma \in \mathcal{L}$ ，那么存在这样的一组位置 P ，因此存在一个序对 $\langle \sigma, \omega \rangle$ 使得验证器接受。（检查 P 中元素的两两距离是否等于 M 可以在 $|\sigma|$ 的多项式时间内完成，因为算术运算可以在多项式时间内完成。）另一方面，如果 $\sigma \notin \mathcal{L}$ ，那么就不存在这样的 ω ，因此不存在任何以 σ 开头的输入序对能让验证器接受。

V.5.36 我们使用引理 5.11。这是一个合适的语言 $\mathcal{L}_B$ ，其中道路地图是加权图 $\mathcal{G} = \langle \mathcal{N}, \mathcal{E} \rangle$ 。

$\{V \subseteq \mathcal{N} \mid \text{存在两个圈，每个圈的权都不超过 } B，\text{ 并且它们的顶点合起来构成 } V\}$

证明概要：给定 V ，一个合适的见证是两个圈构成的序对 $\langle C_0, C_1 \rangle$ 。

为了使用引理 5.11，我们必须构造一个验证器。它输入序对 $\langle \sigma, \omega \rangle$ ，其中 $\sigma = V$ 。如果 $\sigma \in \mathcal{L}$ ，那么存在一对合适的圈，因此存在一个见证 ω 使得验证器接受该输入序对。（显然，验证条件可以在 $|\sigma|$ 的多项式时间内完成，因为算术运算可以在多项式时间内完成。）如果 $\sigma \notin \mathcal{L}$ ，那么就不存在两个这样的圈，因此没有合适的 ω ，所以验证器不会接受任何以 σ 开头的输入序对。

V.5.37 证明与定理 5.4 的证明类似。这里同样有一个方向是容易的，因为确定性 Turing 机是非确定性 Turing 机的一个特例，所以如果一台确定性机器能识别一个语言，那么一台非确定性机器也能识别它。

对于另一个方向，假设一台非确定性 Turing 机 $\mathcal{P}$ 识别语言 $\mathcal{L}$ 。我们将描述一台实现相同功能的确定性机器 $\mathcal{Q}$ 。

固定一个输入 σ 。让确定性机器 $\mathcal{Q}$ 对 $\mathcal{P}$ 的计算树进行广度优先搜索。如果 $\mathcal{P}$ 接受 σ ，则存在一条终端格局为接受状态的极大路径，而 $\mathcal{Q}$ 最终会找到这个格局，因为搜索是广度优先的。如果计算树中没有接受的极大路径，那么 $\mathcal{Q}$ 永远也找不到这样的路径，因此不会接受 σ 。

V.5.38

(A) 最自然的语言是 $\mathcal{L} = \{ \langle \mathcal{G}_0, \mathcal{G}_1 \rangle \mid \text{它们同构} \}$ 。

(B) 我们将利用引理 5.11 来证明该问题属于 NP 。概要如下：给定问题实例 $\sigma = \langle \mathcal{G}_0, \mathcal{G}_1 \rangle$ ，选择同构函数 f 作为见证 ω 。

更详细地说，我们必须构造一个确定性的 Turing 机验证器。它的输入是 $\langle \sigma, \omega \rangle$ ，其中 σ 是一个图对， $\langle \mathcal{G}_0, \mathcal{G}_1 \rangle$ 。如果 $\sigma \in \mathcal{L}$ ，则存在一个同构映射，因此存在一个见证 ω ，使得验证器能够接受这个输入对。（检查该函数是否单射、满射，以及 $\{v, \hat{v}\}$ 是 $\mathcal{G}_0$ 的一条边当且仅当 $\{f(v), f(\hat{v})\}$ 是 $\mathcal{G}_1$ 的一条边，显然可以在关于 $|\sigma|$ 的多项式时间内完成。）如果 $\sigma \notin \mathcal{L}$ ，则没有这样的函数，因此没有任何见证 ω 能够通过检查，验证器绝不会接受一个以 σ 开头的输入对。

V.5.39 可数的。验证器是一台确定性 Turing 机，而这样的机器只有可数多台。

V.5.40

(A) NP 类是由非确定性机器在多项式时间内判定的语言集合。任何能被非确定性 Turing 机判定的问题也能被确定性 Turing 机判定。停机问题不能被确定性 Turing 机判定，因此它不在 NP 中。

(B) 步骤数没有多项式上界，因此见证 ω 的长度也没有多项式上界。

V.5.41 一台非确定性 Turing 机 $\mathcal{P}$ 的格局是一个四元组， $\langle q, s, \tau_L, \tau_R \rangle \in Q \times \Sigma \times \Sigma^* \times \Sigma^*$ 。这些项分别表示当前状态 q 、读写头下的字符 s ，以及读写头左侧和右侧的纸带内容 τ_L 和 τ_R 。若机器的输入是串 σ ，且该串的第一个字符是 s ，即 $\sigma = \langle s \rangle \wedge \alpha$ ，则计算的初始格局是 $\mathcal{C}_0 = \langle q_0, s, \varepsilon, \alpha \rangle$ 。（如果输入为空，则我们取 $s = \mathbf{B}$ 。）

接下来我们描述在什么情况下格局 $\hat{\mathcal{C}}$ 是 $\mathcal{C}$ 的后继，记作 $\mathcal{C} \vdash \hat{\mathcal{C}}$ 。假设 $\mathcal{C} = \langle q, s, \tau_L, \tau_R \rangle$ 。计算集合 $\Delta(q, s)$ 。如果该集合为空，则没有下一个格局 $\hat{\mathcal{C}}$ 。

否则该集合非空。对于每一条指令 $\langle q_p T_p T_n q_n \rangle \in \Delta(q, s)$ ，有三种可能。这三种情况中恰好有一种成立，并且它定义了一个后继格局 $\hat{\mathcal{C}}$ 。(1) 如果动作符号 T_n 是纸带字母表中的一个成员，即 $T_n \in \Sigma$ ，则将其写入纸带， $\hat{\mathcal{C}} = \langle q_n, T_n, \tau_L, \tau_R \rangle$ 。(2) 如果 T_n 是字符 $\mathbf{L}$ ，则将读写头向左移动。即， $\hat{\mathcal{C}} = \langle q_n, \hat{s}, \hat{\tau}_L, \hat{\tau}_R \rangle$ 其中 $\hat{s}$ 是串 τ_L 的最右字符（如果 $\tau_L = \varepsilon$ ，则 $\hat{s}$ 是空白符 $\mathbf{B}$ ），其中 $\hat{\tau}_L$ 是 τ_L 去掉其最右字符后剩下的部分（如果 $\tau_L = \varepsilon$ ，则 $\hat{\tau}_L = \varepsilon$ 同样成立），并且其中 $\hat{\tau}_R$ 是通过将纸带字符 s 推到 τ_R 的前端形成的串，得到 $\hat{\tau}_R = \langle s \rangle \wedge \tau_R$ 。(3) 如果 $T_n = \mathbf{R}$ ，则将读写头向右移动。这与上一项

非常相似，故略去细节。

一条计算路径是一个格局序列 $\mathcal{C}_0 \vdash \mathcal{C}_1 \vdash \dots$ ，从初始格局开始。所有这些路径的并形成有一个有向图，称为计算树，或简称为计算。如果对于给定的输入 σ ，该树中至少有一条极大路径终止于一个状态为接受状态的格局，那么我们称机器接受 σ 。如果机器不接受 σ ，则它拒绝 σ 。

V.6.10 第一种方式是正确的：当 $\mathcal{L}_1 \leq_p \mathcal{L}_0$ 时， $\mathcal{L}_1$ reduces to $\mathcal{L}_0$ 。

V.6.11 不正确。这个直观解释适合 Cook 归约。Karp 归约最多调用一次黑盒 A 。这就是定义 6.1 末尾的“ $f(\sigma) \in \mathcal{L}_0$ ”。

V.6.12 是第一项。一个 $\mathcal{L}_0$ 的快速算法可以给出一个 $\mathcal{L}_1$ 的快速算法。

V.6.13 关于 P 的断言直接来自引理 6.9 的项 (4)。对于 NP ，根据项 (5)，它也成立。

V.6.14 如果 $\mathcal{L}_1$ 通过归约函数 f 归约到 $\mathcal{L}_0$ ，那么可能存在另一个语言 $\mathcal{L}$ ，使得 $\mathcal{L}_1$ 通过同一个归约函数 f 也归约到 $\mathcal{L}$ 。这样的其他语言可以有不可数多个。

稍微展开：在引理 6.9 的项 (3) 的证明中，对于输入串 α ，归约函数如下，使用一个成员 $\sigma \in \mathcal{L}_0$ 和一个非成员 $\tau \notin \mathcal{L}_0$ 。

$$f(\alpha) = \begin{cases} \sigma & \text{若 } \alpha \in \mathcal{L}_1 \\ \tau & \text{其他情形} \end{cases}$$

这个定义并未涉及不等于 σ 或 τ 的所有串是否属于 $\mathcal{L}$ 的问题。所以这一个归约函数将 $\mathcal{L}_1$ 与不可数多个其他集合建立了归约关系。

V.6.15 确实存在一个和为 T 的子集，即 $A = \{23, 31, 72\}$ 。

V.6.16 引理 6.9 指出 $\leq_p$ 是自反关系，所以下表的对角线均为 Y。由同一引理可知， P 中的任何问题都可以在多项式时间内归约到任意其他问题。因此，第一行、第二行和第四行全部是 Y。

	$\mathcal{L}_0$	$\mathcal{L}_1$	$\mathcal{L}_2$	$\mathcal{L}_3$
$\mathcal{L}_0$	Y	Y	Y	Y
$\mathcal{L}_1$	Y	Y	Y	Y
$\mathcal{L}_2$	N	N	Y	N
$\mathcal{L}_3$	Y	Y	Y	Y

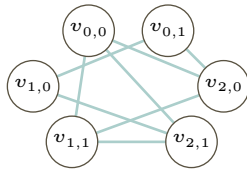
同样根据该引理，如果一个问题可以在多项式时间内归约到某个属于 P 的问题，那么它本身也属于 P 。但题目给定 $\mathcal{L}_2$ 不属于 P ，这就解释了第三行中的 N。

V.6.17 作为参考，该表达式为： $(x_0 \vee x_1) \wedge (\neg x_0 \vee \neg x_1) \wedge (x_0 \vee \neg x_1)$ 。

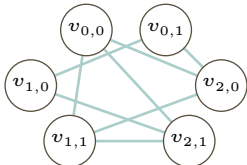
例 6.5 指出，我们需要构建一个图 $\mathcal{G}$ ，其顶点是形如 $\langle c, \ell \rangle$ 的有序对，其中 c 是子句的编号， ℓ 是该子句中的一个文字。两个顶点 $v_0 = \langle c_0, \ell_0 \rangle$ 和 $v_1 = \langle c_1, \ell_1 \rangle$ 之间有边相连，当且仅当它们来自不同的子句 ($c_0 \neq c_1$)，且这两个文字不互为否定 ($\ell_0 \neq \neg \ell_1$)。

给定的命题逻辑表达式有三个子句： $x_0 \vee x_1$ 、 $\neg x_0 \vee \neg x_1$ 和 $x_0 \vee \neg x_1$ ，每个子句有两个文字。得到的图如下。

$$\begin{aligned} v_{0,0} &= \langle 0, x_0 \rangle \\ v_{0,1} &= \langle 0, x_1 \rangle \\ v_{1,0} &= \langle 1, \neg x_0 \rangle \\ v_{1,1} &= \langle 1, \neg x_1 \rangle \\ v_{2,0} &= \langle 2, x_0 \rangle \\ v_{2,1} &= \langle 2, \neg x_1 \rangle \end{aligned}$$



V.6.18 为方便参考，此处给出该图。



每个顶点对应一个子句。

$$a_{0,0} \vee b_{0,0} \vee c_{0,0} \quad a_{0,1} \vee b_{0,1} \vee c_{0,1} \quad a_{1,0} \vee b_{1,0} \vee c_{1,0} \quad a_{1,1} \vee b_{1,1} \vee c_{1,1} \quad a_{2,0} \vee b_{2,0} \vee c_{2,0} \quad a_{2,1} \vee b_{2,1} \vee c_{2,1}$$

每条边对应三个子句。例如，对于连接 $v_{0,0}$ 和 $v_{1,1}$ 的边，其三个子句如下。

$$(\neg a_{0,0} \vee \neg a_{1,1}) \quad (\neg b_{0,0} \vee \neg b_{1,1}) \quad (\neg c_{0,0} \vee \neg c_{1,1})$$

通过取各子句的合取，构造出整个表达式

$$\begin{aligned} E = & (a_{0,0} \vee b_{0,0} \vee c_{0,0}) \wedge (a_{0,1} \vee b_{0,1} \vee c_{0,1}) \wedge (a_{1,0} \vee b_{1,0} \vee c_{1,0}) \\ & \wedge (a_{1,1} \vee b_{1,1} \vee c_{1,1}) \wedge (a_{2,0} \vee b_{2,0} \vee c_{2,0}) \wedge (a_{2,1} \vee b_{2,1} \vee c_{2,1}) \\ & \wedge (\neg a_{0,0} \vee \neg a_{1,1}) \wedge (\neg b_{0,0} \vee \neg b_{1,1}) \wedge (\neg c_{0,0} \vee \neg c_{1,1}) \\ & \wedge (\neg a_{0,0} \vee \neg a_{2,0}) \wedge (\neg b_{0,0} \vee \neg b_{2,0}) \wedge (\neg c_{0,0} \vee \neg c_{2,0}) \\ & \wedge (\neg a_{0,0} \vee \neg a_{2,1}) \wedge (\neg b_{0,0} \vee \neg b_{2,1}) \wedge (\neg c_{0,0} \vee \neg c_{2,1}) \\ & \wedge (\neg a_{0,1} \vee \neg a_{1,0}) \wedge (\neg b_{0,1} \vee \neg b_{1,0}) \wedge (\neg c_{0,1} \vee \neg c_{1,0}) \\ & \wedge (\neg a_{0,1} \vee \neg a_{2,0}) \wedge (\neg b_{0,1} \vee \neg b_{2,0}) \wedge (\neg c_{0,1} \vee \neg c_{2,0}) \\ & \wedge (\neg a_{1,0} \vee \neg a_{2,1}) \wedge (\neg b_{1,0} \vee \neg b_{2,1}) \wedge (\neg c_{1,0} \vee \neg c_{2,1}) \\ & \wedge (\neg a_{1,1} \vee \neg a_{2,0}) \wedge (\neg b_{1,1} \vee \neg b_{2,0}) \wedge (\neg c_{1,1} \vee \neg c_{2,0}) \\ & \wedge (\neg a_{1,1} \vee \neg a_{2,1}) \wedge (\neg b_{1,1} \vee \neg b_{2,1}) \wedge (\neg c_{1,1} \vee \neg c_{2,1}) \end{aligned}$$

。

V.6.19

- (A) 其中三个是： $\beta_0 = \alpha = 0110010$ 、 $\beta_1 = 1100100$ 和 $\beta_2 = 1001001$ 。
 (B) 它是从索引 $k = 6$ 开始的循环移位。
 (C) 它是以下语言的判定问题：

$$\mathcal{L}_0 = \{ \langle \sigma, \tau \rangle \in \Sigma^2 \mid \tau \text{ 是 } \sigma \text{ 的子串} \}$$

- (D) 它是以下语言的判定问题：

$$\mathcal{L}_1 = \{ \langle \alpha, \beta \rangle \in \Sigma^2 \mid \beta \text{ 是 } \alpha \text{ 的循环移位} \}$$

- (E) 根据提示，归约函数 f 输入 $\langle \alpha, \beta \rangle$ ，如果两者长度相同，则输出 $\langle \alpha \frown \alpha, \beta \rangle$ 。（如果长度不同，则输出一个必然失败的实例，例如 $\langle \varepsilon, 0 \rangle$ 。）显然， $\langle \alpha, \beta \rangle \in \mathcal{L}_1$ 当且仅当 $f(\langle \alpha, \beta \rangle) \in \mathcal{L}_0$ 。同样显然， f 是多项式时间可计算的。因此， $\mathcal{L}_1 \leq_p \mathcal{L}_0$ 。

V.6.20

- (A) **Hamiltonian Circuit** (Hamilton 回路) 问题是以下语言的判定问题：

$$\mathcal{L}_0 = \{ \mathcal{H} \mid \text{这个无权图包含一个经过每个顶点恰好一次的回路} \}$$

Traveling Salesman (旅行商) 问题是以下判定问题：

$$\mathcal{L}_1 = \{ \langle \mathcal{G}, B \rangle \mid \text{加权图 } \mathcal{G} \text{ 的某个经过每个顶点的回路总代价不超过 } B \}$$

- (B) 归约函数输入一个 **Hamiltonian Circuit** 实例，即一个边不带权的图 $\mathcal{H} = \langle \mathcal{N}, \mathcal{E} \rangle$ 。它返回一个 **Traveling Salesman** 实例，即一个有序对 $\langle \mathcal{G}, B \rangle$ 。其中的第一个分量 $\mathcal{G}$ 是一个加权图，它使用 $\mathcal{H}$ 的顶点集 $\mathcal{N}$ 作为城市，城市间的距离定义为：若 $v_i v_j$ 是 $\mathcal{H}$ 的边，则为 $d(v_i, v_j) = 1$ ；若 $v_i v_j \notin \mathcal{E}$ ，则为 $d(v_i, v_j) = 2$ 。该有序对的第二个分量是界限 $B = |\mathcal{N}|$ 。

由于这个界限，存在 **Traveling Salesman** 解当且仅当存在 **Hamiltonian Circuit** 解，也就是说，此时 **Traveling Salesman** 的解使用了 Hamilton 回路的边。也就是说， $\mathcal{N} \in \mathcal{L}_0$ 当且仅当 $f(\mathcal{H}) = \langle \mathcal{G}, B \rangle \in \mathcal{L}_1$ 。

- (c) 边数少于顶点数的两倍, 因此, 关于输入图大小的多项式时间等价于关于顶点数的多项式时间。为了输出 **Traveling Salesman** 实例, 算法需要检查所有顶点对, 这是一个嵌套循环, 因此时间是顶点数的平方。多项式的平方仍是多项式。所以 f 在多项式时间内运行。

V.6.21

- (A) 这是 3-SAT 问题的语言。

$\{E \mid E \text{ 是一个可满足的 CNF 布尔表达式, 其中每个子句最多包含三个文字}\}$

这是 Strict 3-Satisfiability 问题的语言。

$\{E \mid E \text{ 是一个可满足的 CNF 布尔表达式, 其中每个子句恰好包含三个文字}\}$

- (B) Strict 3-Satisfiability 问题的任何实例同时也是 3-SAT 问题的实例。因此, 归约函数就是恒等函数。
 (c) 题目要求证明, 包含两个文字的命题逻辑表达式 $E_0 = P \vee Q$ 等价于表达式 $E_1 = (P \vee Q \vee R) \wedge (P \vee Q \vee \neg R)$, 其中每个子句都有三个文字。

最直接的做法是使用真值表来比较这两个表达式。

P	Q	R	E_0	$P \vee Q \vee R$	$P \vee Q \vee \neg R$	E_1
F	F	F	F	F	T	F
F	F	T	F	T	F	F
F	T	F	T	T	T	T
F	T	T	T	T	T	T
T	F	F	T	T	T	T
T	F	T	T	T	T	T
T	T	F	T	T	T	T
T	T	T	T	T	T	T

同样的技巧适用于只包含一个文字的表达式 $E_2 = P$ 。它等价于表达式 $E_3 = (P \vee Q \vee R) \wedge (P \vee \neg Q \vee R) \wedge (P \vee Q \vee \neg R) \wedge (P \vee \neg Q \vee \neg R)$, 其中每个子句都有三个文字。

P	Q	R	E_2	$(P \vee Q \vee R)$	$(P \vee \neg Q \vee R)$	$(P \vee Q \vee \neg R)$	$(P \vee \neg Q \vee \neg R)$	E_3
F	F	F	F	F	T	T	T	F
F	F	T	F	T	T	F	T	F
F	T	F	F	T	F	T	T	F
F	T	T	F	T	T	T	F	F
T	F	F	T	T	T	T	T	T
T	F	T	T	T	T	T	T	T
T	T	F	T	T	T	T	T	T
T	T	T	T	T	T	T	T	T

- (D) 归约函数输入的表达式是合取范式形式 $E = \sigma_0 \wedge \sigma_1 \wedge \dots \wedge \sigma_k$, 其中子句 σ_i 是三个或更少文字的析取。它必须输出等价的命题逻辑表达式, 这些表达式也为合取范式, 但其子句是恰好三个文字的析取。前一项已展示了该函数如何进行这种转换。显然, 该函数可以在多项式时间内运行。

V.6.22 以下是 3-SAT 问题对应的语言。

$\{E \mid E \text{ 是一个可满足的、每个子句至多含有三个文字的合取范式布尔表达式}\}$

以下是 SAT 问题对应的语言。

$\{E \mid E \text{ 是一个可满足的合取范式布尔表达式}\}$

- (A) 归约函数 f 必须以一个 3-SAT 的实例为输入, 即一个子句至多含有三个文字的合取范式命题逻辑表达式。它必须输出一个 SAT 的实例, 也是一个合取范式命题逻辑表达式。因此令 f 为恒等函数 $f(\sigma) = \sigma$ 。显然, 输入可满足当且仅当输出可满足。同样显然, 它在多项式时间内运行。

- (B) 首先考虑为变量 $P_0, \dots, P_4$ 分配真值 T 或 F 使得 $C = p_0 \vee p_1 \vee p_2 \vee p_3 \vee p_4$ 求值为 T 的情况。那么文字 $p_0, \dots, p_4$ 中至少有一个求值为 T 。如果 p_0 或 p_1 是 T ，则将 A 设为 F 可使 $E = (A \vee p_0 \vee p_1) \wedge (\neg A \vee p_2 \vee p_3 \vee p_4)$ 求值为 T 。否则，如果 C 是 T 但 p_0 和 p_1 都不是 T ，那么将 A 设为 T 可使 E 求值为 T 。

现在假设不存在对变量 $P_0, \dots, P_4$ 的真值指派能使 C 求值为 T 。那么无论我们为 A 分配什么真值， E 的两个子句 $(A \vee p_0 \vee p_1)$ 和 $(\neg A \vee p_2 \vee p_3 \vee p_4)$ 中总有一个是 F 。因此，对于 $P_0, \dots, P_4$ 和 A 的每一种真值指派，表达式 E 求值结果都是 F 。

- (C) 归约函数以合取范式表达式 S 为输入。它必须输出子句含有三个或更少文字的合取范式表达式。

上一问提示了归约函数如何做到这一点。它将每个包含多于三个文字的子句 C 替换为若干个含有三个或更少文字的子句。例如，从输入 $p_0 \vee p_1 \vee p_2 \vee p_3 \vee p_4$ 开始，归约函数可以先转换为 $(A \vee p_0 \vee p_1) \wedge (\neg A \vee p_2 \vee p_3 \vee p_4)$ ，然后得到输出 $(A \vee p_0 \vee p_1) \wedge (B \vee \neg A \vee p_2) \wedge (\neg B \vee p_3 \vee p_4)$ 。显然，这一转换可以在多项式时间内完成。

V.6.23

- (A) **独立集**问题是以下语言的判定问题 ($\mathcal{N}_G$ 是 G 的顶点集):

$$\mathcal{L}_0 = \{ \langle G, B \rangle \mid \text{存在 } I \subseteq \mathcal{N}_G, \text{ 使 } |I| \geq B \text{ 且没有边包含 } I \text{ 的两个成员} \}$$

顶点覆盖是以下语言的判定问题:

$$\mathcal{L}_1 = \{ \langle \mathcal{H}, D \rangle \mid \text{存在 } C \subseteq \mathcal{N}_\mathcal{H}, \text{ 使 } |C| \leq D \text{ 且每条边至少包含 } C \text{ 的一个成员} \}$$

- (B) 一个包含 $k = 4$ 个元素的顶点覆盖是 $S = \{q_2, q_5, q_8, q_9\}$ 。
 (C) 一个包含 $\hat{k} = 6$ 个元素的独立集是 $\hat{S} = \{q_0, q_1, q_3, q_4, q_6, q_7\}$ 。
 (D) 根据定义，顶点子集 I 是独立的，当且仅当每条边至多包含 I 中的一个元素。同样根据定义，顶点子集 C 是一个顶点覆盖，当且仅当每条边至少包含 C 中的一个元素。

设 I 是一个独立集。对于所有边 e ， e 的端点中至多有一个在 I 中。这意味着 e 的端点中至少有一个在 $\mathcal{N} - I$ 中。因此，如果 I 是独立集，那么 $\mathcal{N} - I$ 就是顶点覆盖。

类似地，设 I 满足 $\mathcal{N} - I$ 是顶点覆盖。那么对于所有边 $\hat{e}$ ， $\hat{e}$ 的端点中至少有一个是 $\mathcal{N} - I$ 的成员。因此， $\hat{e}$ 的端点中至多有一个是 I 的成员。所以，如果 $\mathcal{N} - I$ 是顶点覆盖，那么 I 就是独立集。

因此，一个顶点子集是独立集，当且仅当其（关于顶点全集）的补集是顶点覆盖。

- (E) 我们将证明 $\text{Independent Set} \leq_p \text{Vertex Cover}$ ；另一个归约是类似的。根据前一项，独立集和顶点覆盖之间存在一种关系。具体来说，如果存在一个大小为 $\hat{k}$ 的独立集，那么就存在一个大小为 $|\mathcal{N}| - \hat{k}$ 的顶点覆盖。

归约的定义要求我们构造一个多项式时间可计算的函数 f 。给定一个**独立集**问题的实例 $\sigma = \langle G, \hat{k} \rangle$ ，该函数通过调整界限值将其转换为一个**顶点覆盖**问题的实例，即 $f(\sigma) = \langle G, |\mathcal{N}| - \hat{k} \rangle$ 。根据前一项， $\sigma \in \mathcal{L}_0$ 当且仅当 $f(\sigma) \in \mathcal{L}_1$ 。同时，显然 f 可以在多项式时间内运行。

V.6.24

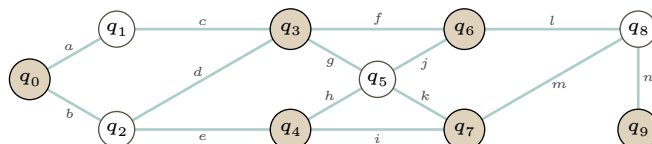
- (A) **顶点覆盖**是以下语言的判定问题。

$$\mathcal{L}_0 = \{ \langle G, B \rangle \mid \text{存在 } C \subseteq \mathcal{N}(G) \text{ 满足 } |C| \leq B, \text{ 使得每条边都至少包含 } C \text{ 中的一个顶点} \}$$

集合覆盖是以下语言的判定问题。

$$\mathcal{L}_1 = \{ \langle S, S_0, \dots, S_{k-1}, D \rangle \mid \text{对每个 } j \text{ 有 } S_j \subseteq S, \text{ 并且存在 } I \subset \{0, \dots, k-1\} \text{ 满足 } |I| \leq D, \text{ 且 } \cup_{i \in I} S_i = S \}$$

- (B) 一个包含六个元素的顶点覆盖是 $C = \{q_0, q_3, q_4, q_6, q_7, q_9\}$ 。



(c) 边集是 $S = \{a, b, \dots, n\}$ 。子集如下。

$$S_0 = \{a, b\}, S_1 = \{a, c\}, S_2 = \{b, d, e\}, S_3 = \{c, d, f, g\}, S_4 = \{e, h, i\}, \\ S_5 = \{g, h, j, k\}, S_6 = \{f, j, l\}, S_7 = \{i, k, m\}, S_8 = \{l, m, n\}, S_9 = \{n\}$$

为了找到一个集合覆盖，我们可以取与该顶点覆盖中的顶点相关联的子集。

$$S_{q_0} \cup S_{q_3} \cup S_{q_4} \cup S_{q_6} \cup S_{q_7} \cup S_{q_9} = \{a, b\} \cup \{c, d, f, g\} \cup \{e, h, i\} \cup \{f, j, l\} \cup \{i, k, m\} \cup \{n\} = S$$

(d) 归约函数 f 以顶点覆盖问题的一个实例 $\langle \mathcal{G}, B \rangle$ 为输入。其计算过程为：令 $S = \mathcal{E}$ ，即所有边的集合。对于每个顶点 v ，构造 $S_v \subseteq S$ ，包含所有与 v 关联的边。然后，该函数的输出是集合覆盖问题的一个实例： $f(\langle \mathcal{G}, B \rangle) = \langle S, S_{v_0}, \dots, B \rangle$ 。显然，我们可以在多项式时间内计算这个函数。并且，如前面两项所示， $\sigma = \langle \mathcal{G}, B \rangle \in \mathcal{L}_0$ 当且仅当 $f(\sigma) = \langle S, S_{v_0}, \dots, B \rangle \in \mathcal{L}_1$ ，因为每个顶点覆盖都对应于一个集合覆盖的索引集。

V.6.25 在例 6.3 中，给出了最短路径问题的语言定义：

$$\mathcal{L}_0 = \{ \langle \mathcal{G}, v_0, v_1, B \rangle \mid \text{存在一条从 } v_0 \text{ 到 } v_1 \text{ 总权至多为 } B \text{ 的路径} \}$$

(A) **无权最短路径**问题是一个优化问题，要求找出最短路径（如果存在）。像许多此类问题一样，要将其表述为语言判定问题，我们需要引入一个参数界 B_1 （图 $\mathcal{G}$ 是无权的）。其语言为：

$$\mathcal{L}_1 = \{ \langle \mathcal{G}, v_0, v_1, B_1 \rangle \mid \text{存在一条从 } v_0 \text{ 到 } v_1 \text{ 边数至多 } B_1 \text{ 的路径} \}$$

（与此不同，**点到点路径**是判定问题：它只要求回答“是，存在路径”或“否，不存在路径”。）

归约函数的输入为 $\langle \mathcal{G}_1, v_0, v_1, B_1 \rangle$ ，即针对**无权最短路径**问题的输入，其中 $\mathcal{G}_1$ 是无权图。它利用 $\mathcal{G}_1$ 的顶点和边构造一个加权图 $\mathcal{H}_1$ ，具体方法是为每条边赋权值 1。归约函数的输出为 $\langle \mathcal{H}_1, v_0, v_1, B_1 \rangle$ 。如果在 $\mathcal{H}_1$ 中存在从顶点 v_0 到 v_1 总权至多 B_1 的路径，那么在原图中就存在一条边数至多 B_1 的无权路径。显然，这个归约函数在多项式时间内运行。

(B) **最长路径**问题是一个优化问题，因此其规范语言定义为：

$$\mathcal{L}_2 = \{ \langle \mathcal{G}, v_0, v_1, B_2 \rangle \mid \text{存在一条从 } v_0 \text{ 到 } v_1 \text{ 总权至少为 } B_2 \text{ 的路径} \}$$

为了将其归约到**最短路径**问题，我们构造一个归约函数。它输入 $\langle \mathcal{G}, v_0, v_1, B_2 \rangle$ ，其中 $\mathcal{G}$ 是一个加权图。它将 $\mathcal{G}$ 转换为一个新图 $\mathcal{H}$ ，后者具有相同的顶点和边，但每条边的权值取 $\mathcal{G}$ 中对应边权值的相反数。然后，归约函数的输出是 $\langle \mathcal{H}, v_0, v_1, -B_2 \rangle$ 。此函数显然在多项式时间内运行。同样显然的是，在权值取反后最短的路径，对应原来的权值就是最长的路径。

V.6.26

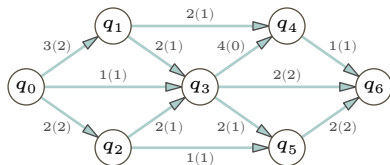
(A) **最大流**对应的语言如下：

$$\mathcal{L}_0 = \{ \langle \mathcal{G}, B \rangle \mid \text{存在一个满足 } \mathcal{G} \text{ 约束的流，使得流入汇点的流量至少为 } B \}$$

这是**线性规划**优化问题对应的语言：

$$\mathcal{L}_1 = \{ \langle C, Z, D \rangle \mid \text{存在一个点 } \vec{x} \text{ 满足约束 } C \text{ 且满足 } Z(\vec{x}) \geq D \}$$

(B) 有算法可以解决这些问题，但对于规模较小的实例，我们可以直接凭目测得出答案。汇点的总流量限制为五，我们可以从右向左浏览此图。下图显示了每条边的容量，并用括号标出了所达到的流量。



(C) 变量是 $x_{0,1}, \dots, x_{5,6}$ 。这些是容量约束：

$$\begin{aligned} 0 \leq x_{0,1} \leq 3, \quad 0 \leq x_{0,2} \leq 2, \quad 0 \leq x_{0,3} \leq 1, \quad 0 \leq x_{1,3} \leq 2, \quad 0 \leq x_{1,4} \leq 2, \\ 0 \leq x_{2,3} \leq 2, \quad 0 \leq x_{2,5} \leq 1, \quad 0 \leq x_{3,4} \leq 4, \quad 0 \leq x_{3,5} \leq 2, \quad 0 \leq x_{3,6} \leq 2, \\ 0 \leq x_{4,6} \leq 1, \quad \text{与} \quad 0 \leq x_{5,6} \leq 2 \end{aligned}$$

(这些符合问题的定义，因为每个都是两个线性约束的组合。例如， $0 \leq x_{0,1} \leq 3$ 是 $0 \leq x_{0,1}$ 和 $x_{0,1} \leq 3$ 的组合。) 这些是流量约束：

$$\begin{aligned} x_{0,1} = x_{1,3} + x_{1,4}, \quad x_{0,2} = x_{2,3} + x_{2,5}, \quad x_{0,3} + x_{1,3} + x_{2,3} = x_{3,4} + x_{3,5} + x_{3,6}, \\ x_{1,4} + x_{3,4} = x_{4,6}, \quad \text{与} \quad x_{2,5} + x_{3,5} = x_{5,6} \end{aligned}$$

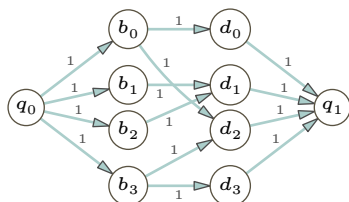
(这些也符合问题的定义，因为每个都是两个线性约束的组合。例如， $x_{0,1} = x_{1,3} + x_{1,4}$ 是 $0 \leq x_{0,1} - x_{1,3} - x_{1,4}$ 与 $x_{0,1} - x_{1,3} - x_{1,4} \leq 0$ 的组合。) 我们想要最大化 $Z = x_{3,6} + x_{4,6} + x_{5,6}$ 。

(D) 我们必须构造一个归约函数 f 。输入是一个序对 $\langle \mathcal{G}, B \rangle$ 。利用该信息构造变量、容量约束、流量约束 C ，以及一个优化表达式 Z 。于是，跨越网络的流对应于线性规划实例中的可行点 $\vec{x}$ 。因此， $\sigma = \langle \mathcal{G}, F, B \rangle \in \mathcal{L}_0$ 当且仅当 $f(\sigma) = \langle C, Z, \vec{x}, B \rangle \in \mathcal{L}_1$ 。显然，归约函数可以在多项式时间内完成此构造。

V.6.27

(A) 最大匹配数是三组，例如 (b_0, d_0) 、 (b_1, d_1) 和 (b_3, d_2) 。

(B) 这是一个网络图。每条边的权为 1。



(C) 最大流问题的语言如下，其中 W 是一个界限值。

$$\mathcal{L}_0 = \{ \langle \mathcal{G}, W \rangle \mid \text{存在一个流满足 } \mathcal{G} \text{ 的约束, 且流入汇点的流量至少为 } W \}$$

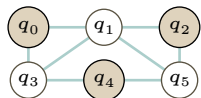
匹配问题的语言如下，其中 $B = \{b_0, \dots, b_{k-1}\}$ 。

$$\mathcal{L}_1 = \{ \langle B, S_0, \dots, S_{k-1}, X \rangle \mid \text{对于 } i \in I \text{ 中的每一个 } b_i, \text{ 存在一个匹配使得 } I \subseteq \{0, \dots, k-1\} \text{ 有 } |I| \geq X \}$$

(D) 归约函数 f 输入一个序列 $\sigma = \langle B, S_0, \dots, S_{k-1}, X \rangle$ 。如上文第二项所述，它构建一个从左到右依次包含源点、一排所有乐队节点、一排所有鼓手节点和汇点的网络。对于每个 b_j 和 S_j 中的每个成员，都有一条从左到右、权为 1 的边。归约函数的输出是二元组 $f(\sigma) = \langle \mathcal{G}, X \rangle$ 。那么根据构造， $\sigma \in \mathcal{L}_1$ 当且仅当 $f(\sigma) \in \mathcal{L}_0$ 。显然我们可以在多项式时间内计算这个 f 。

V.6.28

(A) 一个自然的独立集如下。



(B) 这是该问题作为语言判定问题所对应的语言。

$$\mathcal{L}_0 = \{ \langle \mathcal{G}, B \rangle \mid \mathcal{G} \text{ 包含至少 } B \text{ 个顶点, 且这些顶点之间没有边相连} \}$$

(C) 这是 3-SAT 问题对应的语言。

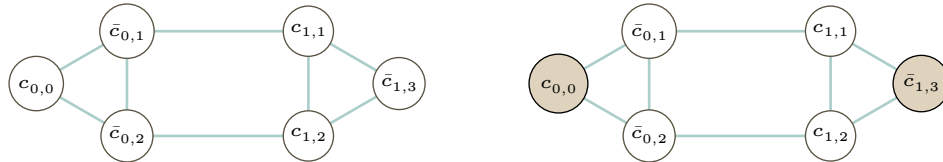
$$\mathcal{L}_1 = \{ E \mid E \text{ 是一个可满足的合取范式 (CNF) 布尔表达式, 且每个子句至多有三个文字} \}$$

(D) 以下是真值表。

P_0	P_1	P_2	P_3	$P_0 \vee \neg P_1 \vee \neg P_2$	$P_1 \vee P_2 \vee \neg P_3$	E
F	F	F	F	T	T	T
F	F	F	T	T	F	F
F	F	T	F	T	T	T
F	F	T	T	T	T	T
F	T	F	F	T	T	T
F	T	F	T	T	T	T
F	T	T	F	F	T	F
F	T	T	T	F	T	F
T	F	F	F	T	T	T
T	F	F	T	T	F	F
T	F	T	F	T	T	T
T	F	T	T	T	T	T
T	T	F	F	T	T	T
T	T	F	T	T	T	T
T	T	T	F	T	T	T
T	T	T	T	T	T	T

任何以 T 结尾的行都表明该表达式是可满足的。

(E) 左图如下。



右边是同一个图，其中标出了一个独立集（还存在其他最大独立集，例如 $\{\bar{c}_{0,1}, c_{1,2}\}$ ）。

取 $P_0 = T$ 且 $\neg P_3 = T$ （即 $P_3 = F$ ）将迫使表达式 E 求值为 T 。给 P_1 和 P_2 赋什么值都无关紧要。

(F) 归约函数 f 输入一个表达式 $\sigma = E$ ，它是 3-SAT 的一个实例。例 6.5 中的论述指出，一个 CNF 公式可满足当且仅当每个子句中至少有一个文字求值为 T 。来自不同子句的两个文字相容的条件是：它们不基于同一个布尔变量 P_i ，或者它们基于同一个 P_i 且同为 P_i 或同为 $\neg P_i$ 。

因此， f 按照前一项所述构造图。与子句 i 相关联的顶点为：如果子句 i 含有文字 P_j ，则顶点为 $c_{i,j}$ ；如果子句 i 含有 $\neg P_j$ ，则顶点为 $\bar{c}_{i,j}$ 。至于边，对于任意子句 C_i ，将该子句中所有顶点 $c_{i,j}$ 相互连接，构成一个三角形（或退化三角形）。此外，当 $i \neq \hat{i}$ 时，连接所有形如 $c_{i,j}$ 和 $\bar{c}_{\hat{i},j}$ 的顶点对，因为它们出自不相容的文字（ P_j 和 $\neg P_j$ ）。

那么 $f(\sigma)$ 就是该图所对应的问题实例。显然，该函数可以在多项式时间内构造此实例。

至于证明 $\sigma \in \mathcal{L}_1$ 当且仅当 $f(\sigma) \in \mathcal{L}_0$ ，回顾独立集的定义： $I \subseteq \mathcal{N}$ 是独立集，是指对每条边而言，至多有一个顶点属于 I 。由于与每个子句相关联的顶点都相互连接，因此一个集合是独立集就意味着在该组顶点中至多有一个顶点属于该集合。冲突顶点之间都有边相连，这意味着至多只能选中其中一个。因此，该图有一个所含顶点数与子句数相等的独立集，当且仅当原表达式可满足。

V.6.29

(A) 这是一个合适的语言。

$$\mathcal{L} = \{\langle I_0, \dots, I_{k-1} \rangle \mid \text{存在整数序列 } \vec{s}, \text{ 满足不等式 } I_0, \dots\}$$

(B) 以下约束在此整数规划背景下确保了每个变量要么是 0，要么是 1。

$$0 \leq x_0, \quad x_0 \leq 1, \quad 0 \leq x_1, \quad x_1 \leq 1, \quad 0 \leq x_2, \quad x_2 \leq 1,$$

一个合适的线性不等式是 $x_0 + (1 - x_1) + (1 - x_2) \geq 1$ 。

- (c) 我们必须构造一个归约函数 f ，它输入一个命题逻辑表达式，并输出 $\mathcal{L}$ 的一个成员。对于该表达式中的每个变量 P_i ，创建一个整数变量 x_i 。如上一项所示，为强制每个 x_i 取值为 0 或 1，需包含线性约束 $0 \leq x_i$ 和 $x_i \leq 1$ 。同样按照上一项的方法，对命题逻辑表达式中的每个子句，向问题实例添加一个相应的约束。

$$P_i \vee \neg P_j \vee \neg P_k \iff 1 \leq x_i + (1 - x_j) + (1 - x_k)$$

(它是各项之和：如果文字 p_m 是 P_m ，则对应项为 x_m ；如果文字是 $p_m = \neg P_m$ ，则对应项为 $(1 - x_m)$ 。) 显然， $\sigma \in 3\text{-SAT}$ 当且仅当 $f(\sigma) \in \mathcal{L}$ 。同样显然的是，我们可以编写 f 使其在多项式时间内运行。

V.6.30

- (A) 它是下述语言的判定问题。

$$D = \{p \mid p \text{ 是多元多项式, 且存在 } \vec{c} = \langle c_0, \dots, c_{n-1} \rangle \in \mathbb{Z}^n \text{ 使 } p(\vec{c}) = 0\}$$

- (B) 作为参考， $S_{E_0} = \{x_0(1 - x_0), x_1(1 - x_1), x_0(1 - x_1)\}$ 。

假设这三个多项式的值都为 0。第一个多项式 $x_0(1 - x_0)$ 取值 0 当且仅当 x_0 是 0 或 1。第二个多项式对 x_1 给出同样的结论。第三个多项式取值 0 当且仅当要么 $0 = x_0$ ，要么 $1 = x_1$ 。

反之，如果所有变量都等于 0 或 1，并且要么 $0 = x_0$ ，要么 $1 = x_1$ ，那么所有三个多项式都取值 0，这是显然的。

- (c) 对于 $E_1 = (P_0 \vee \neg P_1 \vee \neg P_2) \wedge (P_1 \vee P_2 \vee \neg P_3)$ ，类似的集合如下。

$$S_{E_1} = \{x_0(1 - x_0), x_1(1 - x_1), x_2(1 - x_2), x_3(1 - x_3), x_0(1 - x_1)(1 - x_2), x_1x_2(1 - x_3)\}$$

前四个多项式确保每个变量取值 0 或 1。最后两个多项式对应 E_1 中的两个子句。

- (D) 平方和取值 0 当且仅当每一项取值 0。

$$\begin{aligned} p(x_0, x_1, x_2, x_3) &= [x_0(1 - x_0)]^2 + [x_1(1 - x_1)]^2 + [x_2(1 - x_2)]^2 + [x_3(1 - x_3)]^2 \\ &\quad + [x_0(1 - x_1)(1 - x_2)]^2 + [x_1x_2(1 - x_3)]^2 \end{aligned}$$

- (E) 我们必须构造归约函数 f ，它以 3-SAT 的一个实例（即一个所有子句至多三个文字的 Boolean 表达式 E ）作为输入，并输出 D 的一个多项式实例 p 。我们将验证： E 是可满足的当且仅当对某个整数 n 元组 $\vec{s}$ 有 $p(\vec{s}) = 0$ 。并且， f 必须在多项式时间内运行。

显然，我们遵循前面各项中的模式。对于输入表达式 E 中用到的每个变量 P_i ，将 $x_i(1 - x_i)$ 放入集合 S_E 。对于每个三文字子句 $L_{i_0} \vee L_{i_1} \vee L_{i_2}$ ，也向 S_E 添加一个多项式：如果 $L_i = P_i$ ，则该多项式包含因子 x_i ；如果该文字是否定 $L_i = \neg P_i$ ，则多项式包含因子 $(1 - x_i)$ 。最后，归约函数 f 输出的多项式 p ，是集合 S_E 中所有多项式的平方和。

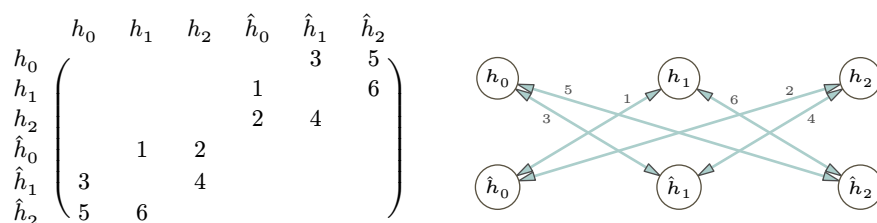
先验证一个方向。假设 E 是可满足的。对所有被赋值为 T 的原子 P_i ，令 $x_i = 0$ ；如果 P_i 被赋值为 F ，则令 $x_i = 1$ 。结果， S_E 中的每个多项式取值 0。因此，平方和多项式 p 取值 0，从而由这些 x_i 构成的元组是该多项式的一个零点。

现在验证另一个方向。假设我们有一个整数元组 $\vec{c}$ 使得 $p(\vec{c}) = 0$ 。因为 p 是平方和，所以每一项的取值必须为 0。形如 $x_i(1 - x_i)$ 的项意味着要么 $x_i = 0$ ，要么 $x_i = 1$ 。那些匹配子句形式的项意味着存在对 Boolean 变量 P_i 的一种赋值使得该子句为真（即 $x_i = 0$ 对应 $P_i = T$ ， $x_i = 1$ 对应 $P_i = F$ ）。

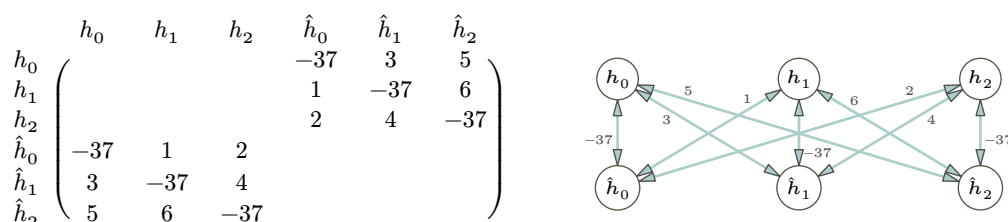
V.6.31

- (A) 归约函数输入一个对称问题的实例，并返回一个非对称问题的实例。对称实例是非对称实例的一个特例。因此归约函数可以是恒等函数。
- (B) 这是构建 $\mathcal{H}$ 第一阶段对应的图矩阵。该矩阵沿着左上到右下的对角线是对称的。下图也展示了第一阶段 $\mathcal{H}$ 对应的图。（它通过在两端显示箭头来强调边的对称性，尽管我们通常在无向图中完全

不显示任何箭头。)



(c) 调整后的矩阵, 即 $\mathcal{H}$ 的图矩阵, 同样是对称的。



(D) 归约函数输入 Asymmetric Traveling Salesman 问题的实例 σ , 并输出 Symmetric Traveling Salesman 问题的实例 $f(\sigma)$ 。

我们首先对 $\sigma = \mathcal{G}$ 的情况进行论证, 其中 $\mathcal{G}$ 是题干中图示的图, 而 $f(\sigma)$ 是前几项推导出的对称图 $\mathcal{H}$ 。要在 $\mathcal{G}$ 中形成一个回路, 只有两种可能: 逆时针走和顺时针走。逆时针方向, $g_0 \rightarrow g_1 \rightarrow g_2 \rightarrow g_0$, 成本为 10。顺时针方向成本为 11。所以成本最小的回路需要 10。

注意到 $\mathcal{G}$ 顶点数的两倍等于 $\mathcal{H}$ 的顶点数, 因此 w 等于 $\mathcal{H}$ 的边数乘以 $\mathcal{H}$ 中的最大边权。我们断言, 在 $\mathcal{H}$ 中包含权重为 $-w-1$ 的大负权边, 确保了这些边必须出现在任何最小回路中。为了证明这一点, 首先注意, 要访问每个顶点, 一个回路必须具有的边数等于顶点数, 这里是六条边 (存在边与该边末端顶点之间的对应关系)。一个不使用任何权重为 -37 的边的回路, 其总权重至少为 $6 \cdot 1$ 。用一条权重为 -37 的边替换一条正权边后, 任何回路的总权重介于 $-37 \cdot 1 + 5 \cdot 6 = -7$ 和 $-37 \cdot 1 + 5 \cdot 1 = -32$ 之间, 从而降低了总权重。由此, 再替换掉另一条正权边, 回路的总权重将介于 $(-37) \cdot 2 + 4 \cdot 6 = -50$ 和 $(-37) \cdot 2 + 4 \cdot 1 = -70$ 之间, 这是一个改进。最后, 包含全部三条 -37 权边的回路达到某个总权重, 而用正权边替换其中一条 -37 边意味着回路的总权重至多为 $(-37) \cdot 3 + 6 \cdot 3 = -93$ 。所以三个 -37 是无法被超越的, 并且通过观察可知存在这样的回路。

因此 $\mathcal{H}$ 的最优解是 $h_0 \rightarrow \hat{h}_0 \rightarrow h_1 \rightarrow \hat{h}_1 \rightarrow h_2 \rightarrow \hat{h}_2 \rightarrow h_0$ 。它在形式为 $h_i \hat{h}_i$ 的边和形式为 $\hat{h}_i h_j$ 的边之间交替, 其中具有匹配下标 $g_i g_j$ 的边位于 $\mathcal{G}$ 的最优回路中。

这个论证可以推广到 $\mathcal{G}$ 中有任何多条边的情况。令 v 为 $\mathcal{G}$ 的顶点数, m 为其最大边权, 则 $w = vm$ 。替换一条正权边将使回路可能的权重改变 $-w-1 = -vm-1$, 而任何使用这条替换边的回路中, 其余边的总权重小于 vm , 因此存在净减少。并且存在一个使用 v 条权重为 $-w-1$ 的边的回路。

显然, 归约函数可以在多项式时间内完成从 $\mathcal{G}$ 到 $\mathcal{H}$ 的变换。

V.6.32

(A) 花一点时间摆弄表格中的数字

$C(t_i, w_j)$	w_0	w_1	w_2	w_3
t_0	13	4	7	6
t_1	1	11	5	4
t_2	6	7	2	8
t_3	1	3	5	9

可以看出最优解是配对 t_0, w_1 , 以及 t_1, w_3 、 t_2, w_2 , 最后是 t_3, w_0 。我们可以用一段程序来验证。

```
(define cost-table
```

```

#(#[13 4 7 6]
   #[1 11 5 4]
   #[6 7 2 8]
   #[1 3 5 9]))

(define (get-cost i j cost-table)
  (vector-ref (vector-ref cost-table i) j))

;; Find minimum cost of matching workers with tasks
;; For every permutation of task indices, make a list of costs,
;; then add, and then find the min.
(define (minimize-assignment cost-table)
  (apply min
    (for*/list ([task-perm (in-permutations '(0 1 2 3))])
      (apply + (map (lambda (i j) (get-cost i j cost-table))
                    '(0 1 2 3)
                    task-perm))))))

```

总成本为 11。

```

> (minimize-assignment cost-table)
11

```

(B) 指派问题的语言如下。

$$\mathcal{L}_0 = \{ \langle W, T, C, B \rangle \mid W \text{ 和 } T \text{ 是大小相同的集合, 且 } C: W \times T \rightarrow \mathbb{R}, \\ \text{并且存在一种指派 } a: W \rightarrow T, \text{ 其成本不超过 } B \}$$

非对称旅行商问题的语言如下。

$$\mathcal{L}_1 = \{ \langle \mathcal{G}, D \rangle \mid \mathcal{G} \text{ 是一个有向加权图, 其中存在一条总权重不超过 } D \text{ 的回路} \}$$

归约函数输入指派问题实例 $\langle W, T, C, B \rangle$, 输出非对称旅行商问题实例 $\langle \mathcal{G}, D \rangle$ 。如提示所述, 令输出图 $\mathcal{G}$ 有两组顶点: 一组标记为每个 w_i , 另一组标记为每个 t_j 。没有 w 之间的边或 t 之间的边。对于每对 w_i, t_j , 每个方向各有一条有向边。将从 w_i 到 t_j 的边的权重设为 $C(w_i, t_j)$ 。将从 t_j 到 w_i 的边的权重设为 0。

如果输入 $\sigma = \langle W, T, C, B \rangle$ 是 $\mathcal{L}_0$ 的元素, 那么某种指派 a 的总成本不超过 B 。设该指派为 $w_0 \mapsto t_{i_0}$, $w_1 \mapsto t_{i_1}$, 等等。那么在 $\mathcal{G}$ 中, 从标记为 w_0 的顶点出发, 前往顶点 t_{i_0} , 然后到 w_1 , 再到 t_{i_1} , 等等, 直到返回 w_0 , 该回路的总权重将小于 $D = B$ 。也就是说, $f(\sigma) = \langle \mathcal{G}, B \rangle$ 是 $\mathcal{L}_1$ 的元素。

显然, 我们可以写出关联此输入和输出的归约函数, 使其在多项式时间内运行。

V.6.33

- (A) 这与“多项式时间”完全无关。如果它们可归约, 那么就会存在一个函数 f , 使得 $n \in \mathbb{N}$ 当且仅当 $f(n) \in \emptyset$ 。但前者恒真, 而后者恒假。
- (B) 假设存在一个可计算函数 f , 使得 $x \in A$ 当且仅当 $f(x) \in B$, 并假设 B 是可计算枚举的。那么, 我们可以通过将 B 的枚举与 f 值域的枚举 (即 $f(0)$ 、 $f(1)$ 等等) 进行交错并行, 来可计算地枚举集合 A 。当一个元素同时出现在两个列表中, 即 $f(i) = b_j$ 时, 就将 i 枚举进 A 。

关于 $K^c \not\leq_p K$, 集合 K 是可计算枚举的, 但其补集 K^c 不是。

- (C) 假设 $\mathcal{L} \leq_p \mathcal{L}^c$ 。那么就存在一个多项式时间函数 f , 使得 $\sigma \in \mathcal{L}$ 当且仅当 $f(\sigma) \in \mathcal{L}^c$ 。这等价于 $\sigma \notin \mathcal{L}$ 当且仅当 $f(\sigma) \notin \mathcal{L}^c$, 而后者成立的条件正是 $\sigma \in \mathcal{L}$ 当且仅当 $f(\sigma) \in \mathcal{L}$ 。

V.6.34

- (A) 自反性很容易证明; 我们需要证明对任意 $\mathcal{L} \in \mathcal{P}(\Sigma^*)$, 都有 $\mathcal{L} \leq_p \mathcal{L}$ 。取归约函数为恒等函数, $f(\sigma) = \sigma$ 。它显然可以在多项式时间内计算, 并且同样显然有 $\sigma \in \mathcal{L}$ 当且仅当 $f(\sigma) \in \mathcal{L}$ 。

对于传递性, 设 $\mathcal{L}_0 \leq_p \mathcal{L}_1$ 由函数 f 给出, $\mathcal{L}_1 \leq_p \mathcal{L}_2$ 由函数 g 给出, 二者均为多项式时间可计算的。那么对于所有 $\sigma \in \Sigma^*$, 我们有 $\sigma \in \mathcal{L}_0$ 当且仅当 $f(\sigma) \in \mathcal{L}_1$, 并且 $f(\sigma) \in \mathcal{L}_1$ 当且仅当 $g \circ f(\sigma) \in \mathcal{L}_2$ 。因为两个函数都是多项式时间的, 所以它们的复合也是多项式时间的。

- (B) 我们首先刻画空集可以归约到哪些问题, 即满足 $\emptyset \leq_p \mathcal{L}$ 的 $\mathcal{L}$ 。如果 $\emptyset \leq_p \mathcal{L}$ 成立, 那么存在一个归约函数 f , 满足

$$\sigma \in \emptyset \quad \text{当且仅当} \quad f(\sigma) \in \mathcal{L} \quad (*)$$

$\sigma \in \emptyset$ 永远不为真, 所以 $\mathcal{L} \neq \Sigma^*$, 因为若 $\mathcal{L} = \Sigma^*$, 则 $f(\sigma) \in \mathcal{L}$ 将总是为真。令 $\mathcal{L} \neq \Sigma^*$, 取 α 为一个不属于 $\mathcal{L}$ 的串, 则常值函数 $f(\sigma) = \alpha$ 就足以作为归约函数。

接下来刻画哪些问题可以归约到空集, 即满足 $\mathcal{L} \leq_p \emptyset$ 的 $\mathcal{L}$ 。

$$\sigma \in \mathcal{L} \quad \text{当且仅当} \quad f(\sigma) \in \emptyset \quad (**)$$

$f(\sigma) \in \emptyset$ 永远不为真, 所以 $\sigma \in \mathcal{L}$ 也永远不为真。换句话说, 唯一能归约到空集的就是空集自身。

对 Σ^* 的推理是类似的。为刻画哪些问题满足 $\Sigma^* \leq_p \mathcal{L}$, 考虑双向蕴含式: $\sigma \in \Sigma^*$ 当且仅当 $f(\sigma) \in \mathcal{L}$ 。条件 $\sigma \in \Sigma^*$ 总是成立, 所以 $\mathcal{L}$ 不能是空集。但任何其他集合 $\mathcal{L} \neq \emptyset$ 都行, 因为若 $\alpha \in \mathcal{L}$, 则常值函数 $f(\sigma) = \alpha$ 就可以作为归约函数。

为刻画哪些集合满足 $\mathcal{L} \leq_p \Sigma^*$, 考虑 $\sigma \in \mathcal{L}$ 当且仅当 $f(\sigma) \in \Sigma^*$ 。条件 $f(\sigma) \in \Sigma^*$ 总是成立, 所以 $\sigma \in \mathcal{L}$ 也必须总是成立, 因此唯一这样的问题就是 $\mathcal{L} = \Sigma^*$ 。

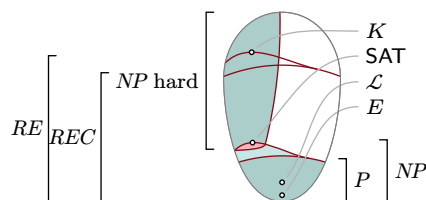
- (C) 假设 $\mathcal{L}_0 \leq_p \mathcal{L}_1$ 成立, 即存在一个多项式时间函数 $f: \Sigma^* \rightarrow \Sigma^*$ 使得 $\sigma \in \mathcal{L}_0$ 当且仅当 $f(\sigma) \in \mathcal{L}_1$ 。我们将利用这个归约函数来证明, 若 $\mathcal{L}_1 \in NP$ 则 $\mathcal{L}_0 \in NP$ 。

假设 $\mathcal{L}_1 \in NP$ 。那么存在一台非确定性 Turing 机 $\mathcal{P}_1$ 判定 $\mathcal{L}_1$ (特别地, 存在一个与这台机器关联的多项式 p , 使得在输入 τ 上每条极大路径都在 $p(\tau)$ 步内终止)。为了构造一台判定 $\mathcal{L}_0$ 的机器 $\mathcal{P}_0$, 从一个候选输入串 $\tau \in \Sigma^*$ 开始。计算 $f(\tau)$ 。以 $f(\tau)$ 为输入运行 $\mathcal{P}_1$ 。如果 $\mathcal{P}_1$ 接受, 则 $\mathcal{P}_0$ 接受; 如果 $\mathcal{P}_1$ 拒绝, 则 $\mathcal{P}_0$ 拒绝。(并且, 可以观察到存在一个多项式 q , 使得这个非确定性算法的每条极大路径都在 $q(|\tau|)$ 步内终止)。

V.6.35 我们针对每种情况给出具体例子, 但还有许多其他正确答案。例如, 对于第一种情况, 我们可以使用 P 中任意两个语言, 其中一个为另一个的真子集。

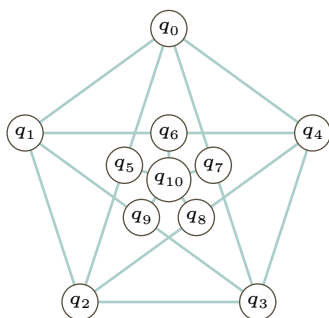
- (A) 取 $\mathcal{L}_0$ 为长度为 4 的倍数的串的语言, $\mathcal{L}_1$ 为长度为偶数的串的语言。显然 $\mathcal{L}_0 \subset \mathcal{L}_1$, 且两个语言都在 P 中, 因此 $\mathcal{L}_0 \leq_p \mathcal{L}_1$ 。
- (B) 取 $\mathcal{L}_0$ 为长度为偶数的串的语言, $\mathcal{L}_1$ 为长度为奇数的串的语言。显然两个语言都在 P 中, 故 $\mathcal{L}_0 \leq_p \mathcal{L}_1$ 。
- (C) 直接从定义易知, 唯一能多项式时间归约到 $\mathbb{B}^*$ 的集合是 $\mathbb{B}^*$ 自身。取 $\mathcal{L}_0$ 为长度为偶数的串的语言, $\mathcal{L}_1 = \mathbb{B}^*$ 。
- (D) 类似地, 唯一能多项式时间归约到 $\emptyset$ 的集合是 $\emptyset$ 自身。取 $\mathcal{L}_0$ 为长度为偶数的串的语言, 取 $\mathcal{L}_1 = \emptyset$ 。

V.7.13 这是包含了这些集合的图。



- (A) 每一个可计算枚举集都能归约到 K 。所以语言 K 位于 RE 的顶端。
- (B) 任何集合都能计算偶数集 E , 所用时间就是读入输入的时间 (其实只需读入输入最低位的时间)。所以它有一个 $O(n)$ 算法, 位置非常靠近底部。
- (C) 最短路径问题有各种各样的算法, 但都很快 (二次或更快), 所以 $\mathcal{L}$ 位于 P 的中部。
- (D) SAT 问题是 NP -完全的, 所以它位于 NP 问题的顶端。

V.7.14 检查一个图是否含有 4-团，所需时间确实是图大小的多项式。但是，除了含有 4-团之外，一个图不能 3-着色还可能其他原因。例如，下面这个图没有三角形，没有 3-团，因此也不会有 4-团。但它不是 3-可着色的。



V.7.15 第一个也是最主要的错误是：“不能在多项式时间内计算”并不是属于 NP 的准则。相反，一个问题属于 NP 的准则是，它有一个多项式时间验证器。

另一个错误是，这个说法似乎混淆了 NP 和差集 $NP - P$ 。也就是说，你的学习伙伴想表达的可能是：“专家们怀疑 Satisfiability 是一个属于 NP 但不属于 P 的问题。”（除此之外，他们很可能指的是 NP -完全的，而不仅仅是 NP 中的一个元素。）

还有一个错误是，这个说法似乎因为“尚未知道有算法”就认为该问题属于那个复杂度类。问题是否属于某个类，并不取决于我们是否知道这样的算法，而是取决于这样的算法是否存在。

V.7.16 这个特定的算法确实不是多项式时间的。但是，如何证明不存在另一个算法，而那个算法是多项式时间的呢？没有人知道。

V.7.17 评论者说这个问题有多项式时间算法，就是在说该问题在 P 中。但是 $P \subseteq NP$ ，所以这个问题其实在 NP 中。

V.7.18 像 NP 这样的计算复杂度类包含的是语言，而不是算法。（我们虽不严格区分语言和问题，因此可以说一个问题属于某个复杂度类，但我们确实严格区分语言和判定该语言的算法。）

V.7.19

- (A) “ NP 是 NP -完全集的子集”这一陈述是错的。事实正好相反： NP -完全集是 NP 的子集。另一方面，“ NP -完全集是 NP -难集的子集”这一论断是正确的。 NP -完全集是 NP -难集与 NP 的交集。
- (B) 非确定性机器并非确定性机器的特化。恰恰相反，非确定性机器是确定性机器的推广，也就是说，任何确定性机器也是一台非确定性机器。（至于 NP 是否是 P 的子集，这就更微妙了。正如本节所讨论的，我们不知道 $P = NP$ 是否成立，尽管大多数专家猜想它不成立，所以我们也不知道 $NP \subseteq P$ 是否成立。）

V.7.20

- (A) 对。因为 素性判定 $\in NP$ ，它能在非确定性 Turing 机上求解。我们可以用确定性机器模拟这样的机器，因此我们至少在原则上可以用现有的物理计算机运行该算法。
- (B) 错；我们不能从给定事实推断出这一点。可能存在高效算法，但我们不知道。（从给出的陈述我们不知道，而且目前的研究者们也不知道。）
- (C) 错。旅行商问题是 NP -完全的。因此，由 素因数分解 $\in NP$ ，我们可以推断出 Prime Factorization $\leq_p$ Traveling Salesman，而且，如果我们拥有旅行商的高效算法，我们也将得到素因数分解的高效算法。但这与题目所问的方向相反。题目说素因数分解未知是否为 NP -完全，所以我们不知道另一个方向是否成立。

V.7.21

- (A) 错；例如，暴力检查所有回路的算法就能解决所有实例。
- (B) 对，这里我们依据 Cobham 论题，将“快速”理解为多项式时间，并牢记本题假设 $P \neq NP$ 。
- (C) 错。如果 旅行商 $\in P$ ，那么每个 NP 问题都属于 P ，因为每个 NP 问题都归约到旅行商。这与 $P \neq NP$ 的假设矛盾。
- (D) 错。至少存在一种算法（暴力算法），所以如果该陈述为真，将意味着旅行商属于 P ，而上一项已指出这不可能。

V.7.22

- (A) 错；根据 $\mathcal{L}_1 \leq_p \mathcal{L}_0$ 和 $\mathcal{L}_0$ 是 NP -完全的，我们不能推出 $\mathcal{L}_1$ 是 NP -完全的。例如，可以设 $\mathcal{L}_0$ 是 **可满足性问题**，而 $\mathcal{L}_1$ 是判定输入串中是否包含偶数个 1 的问题。
- (B) 错；有可能 $\mathcal{L}_0$ 不是 NP 的成员。例如，设 $\mathcal{L}_1 = \text{SAT}$ ，而 $\mathcal{L}_0$ 是停机问题集 K 。
- (C) 错。这重复了第一项的内容：根据 $\mathcal{L}_1 \leq_p \mathcal{L}_0$ 和 $\mathcal{L}_0$ 是 NP -完全的，即使知道 $\mathcal{L}_1 \in NP$ ，也不能推出 $\mathcal{L}_1$ 是 NP -完全的。例如，设 $\mathcal{L}_0$ 是 **可满足性问题**，而 $\mathcal{L}_1$ 是判定输入数字是否为偶数的问题。
- (D) 对；这正是引理 7.4。如果 $\mathcal{L}_1$ 是 NP -完全的，那么任何 $\mathcal{L} \in NP$ 都可归约到 $\mathcal{L}_1$ ，即 $\mathcal{L} \leq_p \mathcal{L}_1$ ，再由 $\leq_p$ 的传递性得到 $\mathcal{L} \leq_p \mathcal{L}_0$ 。结合 $\mathcal{L}_0 \in NP$ 这一陈述，就表明 $\mathcal{L}_0$ 是 NP -完全的。
- (E) 错。一方面，每个问题都可以归约到自身，因此如果 $\mathcal{L}_0 = \mathcal{L}_1$ ，那么显然若其中一个是 NP -完全的，则两者都是。不那么平凡地，一个问题如果属于 NP 且 NP 中的每个问题 $\mathcal{L}$ 都能归约到它，那么它就是 NP -完全的。因此，两个 NP -完全问题可以互相归约。例如，基础列表中的任何问题都可以归约到任何其他问题，或者归约到 **可满足性**。
- (F) 错。例如，设 $\mathcal{L}_1$ 是判定数字是否为偶数的问题，而 $\mathcal{L}_0$ 是一个其最快求解算法是指数时间的问题。
- (G) 对。这是引理 6.9，即 P 对向下归约封闭。

V.7.23 我们只为每一项提供一个答案，但其他答案可能也正确。（如果 $P = NP$ ，则所有非平凡集合都是 NP -难的，但这里给出的答案在 $P \neq NP$ 时也成立。）

- (A) SAT
- (B) $E = \{\sigma \in \mathbb{B}^* \mid \sigma \text{ 以二进制表示一个偶数}\}$
- (C) SAT
- (D) K （更确切地说，是 K 中数字的比特串表示所组成的集合。）
- (E) 上一项集合的补集。
- (F) $\emptyset$

V.7.24 假设 $P = NP$ 。固定一个 P 中的非平凡语言 $\mathcal{L}$ ，即 $\mathcal{L} \neq \emptyset$ 且 $\mathcal{L} \neq \mathbb{N}$ 。因为 $P \subseteq NP$ ，所以 $\mathcal{L} \in NP$ 。

剩下的部分是证明 $\mathcal{L}$ 是 NP -难的，即对于每个语言 $\hat{\mathcal{L}} \in NP$ ，都有 $\hat{\mathcal{L}} \leq_p \mathcal{L}$ 。但在 $P = NP$ 的条件下，有 $\hat{\mathcal{L}} \in P$ ，而引理 6.9 指出 $\hat{\mathcal{L}} \leq_p \mathcal{L}$ 。

V.7.25

- (A) 该语言是如下字符串的集合 $\mathcal{L} = \{\sigma \in \mathbb{B}^* \mid \sigma \text{ 以二进制表示一个偶数}\}$ 。该集合属于 P ，因为 σ 表示一个偶数当且仅当其最后一个比特是 0。因此它属于 NP 。
如果 $\mathcal{L}$ 是 NP -完全的，那么每个 $\hat{\mathcal{L}} \in NP$ 都会满足 $\hat{\mathcal{L}} \leq_p \mathcal{L}$ ，而引理 6.9 将推出 $P = NP$ ，这与练习的假设矛盾。
- (B) 此项与前一项类似，区别在于这个问题并不像前一项那样平凡地属于 P 。为了说明它确实属于 P ，假设给定一个图 $\mathcal{G}$ 。我们可以通过检查所有由顶点组成的 4-元组，来核查是否存在大小为四的顶点覆盖（即包含每条边的至少一个端点的顶点集合）。这需要一个包含 $\binom{k}{4} \in \mathcal{O}(k^4)$ 次迭代的循环，其中 k 是 $\mathcal{G}$ 的顶点数。

V.7.26 将读写头左移，写一个 1，然后停机。这是一个常数时间的算法。

V.7.27 自然的尝试是证明 $3\text{-Satisfiability} \leq_p 4\text{-Satisfiability}$ 。

归约函数是恒等函数。它接收一个最多包含三个文字的子句作为输入，并输出相同的子句（显然该子句最多包含四个文字）。显然，输入是可满足的当且仅当输出是可满足的。同样显然的是，该归约函数在多项式时间内运行。

V.7.28

a	b	c	n_0	n_1	n_2	n_3	n_4	n_5
F	F	T	F	G	T	T	G	F
F	T	F	T	F	T	G	F	G
F	T	T	F	G	T	T	G	F
T	F	F	T	F	T	F	G	G
T	F	T	F	G	T	G	T	F
T	T	F	T	F	T	F	G	G
T	T	T	T	F	T	F	G	G

V.7.29

- (A) 子集 $\hat{S} = \{4, 6, 7, 13\}$ 的元素之和为 $T = 30$ 。
 (B) $\hat{A} = \{6, 7, 19\}$ 中元素的总和等于子集 $A - \hat{A} = \{3, 4, 12, 13\}$ 中元素的总和。
 (C) 假设我们可以将一个集合 $A \subset \mathbb{N}$ 划分为 $\hat{A} \subset A$ 和 $A - \hat{A}$ 两部分, 且两部分的和相等。那么, A 中所有元素的总和是 $\hat{A}$ 中元素总和的两倍, 因此是偶数。
 (D) **子集和**问题是以下语言的判定问题。

$$\mathcal{L}_0 = \{ \langle S, T \rangle \mid \text{存在子集 } \hat{S} \subseteq S \text{ 满足 } \sum_{s \in \hat{S}} s = T \}$$

划分问题是以下语言的判定问题。

$$\mathcal{L}_1 = \{ \langle A \rangle \mid \text{存在 } \hat{A} \subset A \text{ 满足 } \sum_{a \in \hat{A}} a = \sum_{a \in A - \hat{A}} a \}$$

- (E) 我们必须构造一个多项式时间可计算的函数, 它接收输入 $\sigma = \langle A \rangle$ 并输出 $f(\sigma) = \langle S, T \rangle$, 使得 $\sigma \in \mathcal{L}_0$ 当且仅当 $f(\sigma) \in \mathcal{L}_1$ 。
 给定 $\sigma = \langle A \rangle$, 考虑其元素之和 $x = \sum_{a \in A} a$ 。在

$$f(\sigma) = \begin{cases} \langle S, x/2 \rangle & \text{— 若 } x \text{ 为偶数} \\ \langle S, x+1 \rangle & \text{— 其他情形} \end{cases}$$

的情况下, 有 $\sigma \in \mathcal{L}_1$ 当且仅当 $f(\sigma) \in \mathcal{L}_0$, 符合要求, 且显然 f 可在多项式时间内计算。

- (F) **划分**问题是定理 7.5 中列出的 NP -完全集合之一。所以**子集和**问题是 NP -难的。但**子集和**问题属于 NP , 因为我们可以用子集本身作为见证 ω 。因此它是 NP -完全的。

V.7.30 背包问题是以下语言的判定问题。

$$\{ \langle (w_0, v_0), \dots, (w_{n-1}, v_{n-1}), B, T \rangle \mid \text{存在 } I = \{i_0, \dots, i_k\} \subseteq \{0, \dots, n-1\}, \text{ 使 } \sum_{i \in I} w_i \leq B \text{ 且 } \sum_{i \in I} v_i \geq T \}$$

简而言之(如习题 5.34), **背包**问题属于 NP , 因为我们可以将见证取为一个索引集合 $\{i_0, \dots, i_k\} \subset \{0, \dots, n-1\}$, 这些索引对应的物品共同满足约束条件。

最后回顾例 6.4, $\text{Subset Sum} \leq_p \text{Knapsack}$ 。由于前一个练习表明**子集和**问题是 NP -难的, 因此**背包**问题也是 NP -难的。

V.7.31 我们将证明 $\text{Vertex Cover} \leq_p \text{Set Cover}$ 。

归约函数 f 的输入是**顶点覆盖**问题的一个实例, $\sigma = \langle \mathcal{G}, B \rangle$ 。其输出为 $f(\sigma) = \langle S, \{S_0, \dots, S_n\}, B \rangle$, 其中 S 是图 $\mathcal{G}$ 的边集, 并且对于 $\mathcal{G}$ 中的每个顶点, 构造一个由所有包含该顶点的边组成的子集。

观察到, σ 存在顶点覆盖, 当且仅当相应的子集族能覆盖 S 。显然, 该归约函数可以设计为运行时间仅为 $|\mathcal{G}|$ 的多项式。

V.7.32

- (A) 顶点 q_2 是一个汇点, 因此它必须是该最长路径的终点: $\langle q_3, q_6, q_4, q_7, q_8, q_5, q_0, q_1, q_2 \rangle$ 。
 (B) 取 $B \in \mathbb{N}$ 作为参数, 并考虑以下语言族。

$$\mathcal{L} = \{ \langle \mathcal{G}, B \rangle \mid \mathcal{G} \text{ 有一条长度至少为 } B \text{ 的简单路径} \}$$

- (C) 归约函数输入一个 **Hamilton 路径**问题的实例 $\langle \mathcal{G}, v_0, v_1 \rangle$ 。它构造一个图 $\hat{\mathcal{G}}$, 从 $\mathcal{G}$ 出发, 添加两个顶点及两条有向边。这两个顶点是 w_0 和 w_1 。其中一条有向边从 w_0 连向 v_0 , 另一条有向边从 v_1 连向 w_1 。然后, 归约函数输出有序对 $\langle \hat{\mathcal{G}}, |\mathcal{G}| + 1 \rangle$, 其中 $|\mathcal{G}|$ 是图 $\mathcal{G}$ 的顶点数。在 $\hat{\mathcal{G}}$ 中, 一条包含 $|\mathcal{G}| + 1$ 条边的路径必然是一条从 w_0 到 w_1 的 Hamilton 路径。 $\hat{\mathcal{G}}$ 中存在这样一条路径, 当且仅当在 $\mathcal{G}$ 中存在一条从 v_0 到 v_1 的 Hamilton 路径。显然, 该归约函数可在 $|\mathcal{G}|$ 的多项式时间内运行。
 (D) 前一个练习证明了 **Hamilton 路径**问题是 NP -完全的, 结合本练习的其他各项, 即可得出**最长路径**是 NP -完全的。

V.7.33

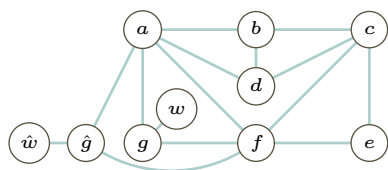
(A) 简而言之, 见证 ω 就是那条路径。

详细来说, 为了应用引理 5.11, 我们将构造一个确定性 Turing 机验证器 $\mathcal{V}$ 。它以二元组 $\langle \mathcal{G}, \omega \rangle$ 作为输入, 并且必须满足: 如果 $\mathcal{G}$ 有一条 Hamilton 路径, 则存在一个见证 ω 使得 $\mathcal{V}$ 接受该输入对; 而如果 $\mathcal{G}$ 没有 Hamilton 路径, 则没有任何见证能使该输入对被接受。验证器必须在多项式时间内运行。

这里的验证器将见证 ω 解释为图中的一条 Hamilton 路径, 因此它会检查该路径确实是给定图中的一条路径, 并且每一个顶点都出现在该路径中。如果存在这样一条路径, 那么就存在一个该验证程序能够验证的见证。如果不存在这样一条路径, 则没有任何串 ω 能满足条件。验证器显然在多项式时间内运行 (图的大小是关于顶点数的多项式, 因为边数 $\mathcal{O}(|\mathcal{N}|^2)$)。

(B) 一条 Hamilton 路径是 $\langle v_0, v_7, v_1, v_2, v_3, v_6, v_5, v_4, v_8 \rangle$ 。顶点 v_0 和 v_8 必须是这条路径的起点和终点, 因为它们的度是 1。

(C) 在原图中, 一条 Hamilton 回路是 $\langle g, a, d, b, c, e, f, g \rangle$ 。下面是增强后的图。



一条 Hamilton 路径是 $\langle w, g, a, d, b, c, e, f, \hat{g}, \hat{w} \rangle$ 。

(D) 我们必须构造一个归约函数 f 。它输入一个图 $\mathcal{G}$, 并输出一个图 $f(\mathcal{G}) = \mathcal{H}$, 且满足性质: $\mathcal{G}$ 有一条 Hamilton 回路当且仅当 $\mathcal{H}$ 有一条 Hamilton 路径。

如同上一项, 给定 $\mathcal{G}$, 我们通过添加三个顶点来构造 $\mathcal{H}$ 。首先, 固定 $\mathcal{G}$ 中的一个顶点 v 。在 $\mathcal{H}$ 中添加一个新顶点 $\hat{v}$, 使其拥有与 v 相同的连接。也就是说, 对于每一条边 $v_i v$, 添加边 $\hat{v} v_i$; 对于每一条形如 $v v_j$ 的边, 添加边 $\hat{v} v_j$ 。同时添加两个度为 1 的顶点: w 仅连接到 v , 而 $\hat{w}$ 仅连接到 $\hat{v}$ 。

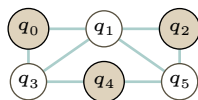
我们必须证明: $\mathcal{H}$ 中有一条 Hamilton 路径当且仅当 $\mathcal{G}$ 中有一条 Hamilton 回路。如果 $\mathcal{G}$ 中有一条 Hamilton 回路, 那么 $\mathcal{H}$ 中的 Hamilton 路径可以通过在 v 处将回路“断开”得到, 并将其连接为 $\mathcal{H}$ 中的一条路径, 该路径从 w 开始, 走到 v , 然后遍历回路, 再到 $\hat{v}$, 最后到达 $\hat{w}$ 。

对于另一个方向, 假设 $\mathcal{H}$ 中有一条 Hamilton 路径。那么这条路径的起点和终点必须是 w 和 $\hat{w}$, 因为这些顶点的度是 1。接着它必须走到 v 和 $\hat{v}$ 。因为 $\hat{v}$ 是 v 的一个副本, 所以 $\mathcal{H}$ 中一条从这些顶点出发的 Hamilton 路径, 对应于 $\mathcal{G}$ 中的一条 Hamilton 回路。

(E) Hamilton 回路问题在基本 NP-完全问题列表中, 因此它可归约到 Hamilton 路径这一事实意味着 Hamilton 路径是 NP-难的。因为本练习第一项已经证明 Hamilton 路径是 NP 的成员, 所以它是 NP-完全的。

V.7.34

(A) 一个自然的独立集如下。



(B) 这是独立集问题对应的语言。

$$\mathcal{L}_0 = \{ \langle \mathcal{G}, B \rangle \mid \mathcal{G} \text{ 至少有 } B \text{ 个顶点, 且它们之间不共享边} \}$$

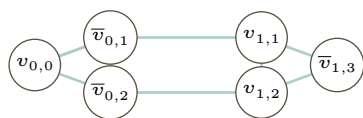
(C) 这是 3-SAT 问题对应的语言。

$$\mathcal{L}_1 = \{ \langle E \rangle \mid E \text{ 是一个可满足的 CNF 表达式, 且每个子句最多包含三个文字} \}$$

(D) E 的真值表见 Figure 104 on page 217。使 E 为真的一种赋值方法是取 $P_0 = T$ 和 $P_1 = P_2 = P_3 = F$ 。

(E) 与该表达式关联的图如下。

P_0	P_1	P_2	P_3	$P_0 \vee \neg P_1 \vee \neg P_2$	$P_1 \vee P_2 \vee \neg P_3$	E
F	F	F	F	T	T	T
F	F	F	T	T	F	F
F	F	T	F	T	T	T
F	F	T	T	T	T	T
F	T	F	F	T	T	T
F	T	F	T	T	T	T
F	T	T	F	F	T	F
F	T	T	T	F	T	F
T	F	F	F	T	T	T
T	F	F	T	T	F	F
T	F	T	F	T	T	T
T	F	T	T	T	T	T
T	T	F	F	T	T	T
T	T	F	T	T	T	T
T	T	T	F	T	T	T
T	T	T	T	T	T	T

图 104, FOR QUESTION V.7.34: $E = (P_0 \vee \neg P_1 \vee \neg P_2) \wedge (P_1 \vee P_2 \vee \neg P_3)$ 的真值表。

(F) 我们必须构造一个归约函数 f 。它输入一个 3-可满足性问题的实例 E , 并构造一个图 $\mathcal{G}$, 方法如前一项所述。即, 对于第 j 个子句, 它添加三个顶点, 如果文字是 P_i 则标记为 $v_{j,i}$, 如果文字是 $\neg P_i$ 则标记为 $\bar{v}_{j,i}$ 。将这三个顶点两两相连, 构成一个三角形。对于任意两个标记有相同第二个下标 i 且一个有上划线而另一个没有的顶点, 归约函数也会在它们之间连接一条边, 从而可能形成三角形之间的边。显然, $\mathcal{G}$ 的这种构造可以在多项式时间内完成。

我们断言, 一个具有 B 个子句的表达式 E (每个子句至多三个文字) 是可满足的, 当且仅当 $\langle \mathcal{G}, B \rangle \in \mathcal{L}_0$ 其中 $\mathcal{G} = f(E)$ 。

首先假设 E 是可满足的。那么在 E 的每一个子句中, 至少有一个文字为 T 。从每个子句中选取一个这样的文字, 记第 i 个子句中的文字为 $\ell(i)$, 并考虑相关联的顶点集合。

$$S = \{v_{i,\ell(i)} \text{ 或 } \bar{v}_{i,\ell(i)} \mid \text{子句 } i \text{ 中相关联的文字为 } T\}$$

我们必须验证 S 有 B 个不同的元素并且是一个独立集。它有 B 个元素, 因为每个成员关联于 E 的一个不同子句, 因此属于不同的三角形。它是一个独立集, 因为没有两个成员之间有边相连: 在构造中, 边有两种, 三角形内部的边和三角形之间的边。 S 中没有两个成员在同一个三角形中, 所以第一种情况不成立。至于三角形之间的边, 它们连接的是与一个文字及其否定相关联的顶点。但是一个文字及其否定不可能同时为 T , 因此相关联的顶点不会同时是 S 的成员。

最后, 假设 $\langle \mathcal{G}, B \rangle \in \mathcal{L}_0$, 从而 $\mathcal{G}$ 包含一个大小为 B 的顶点独立集 $\hat{S}$ 。我们断言, 将相关联的文字集合赋值为 T 将满足表达式 E 。注意, 因为 $\mathcal{G}$ 的构造使得与一个文字及其否定相关联的顶点之间有一条边, 所以我们可以将所有相关联的文字都设为 T 。还要注意, 因为一个三角形内部的所有顶点都是相连的, 所以 $\hat{S}$ 的 B 个成员来自不同的三角形, 因此关联于 B 个不同子句中的文字。所以每个子句都有一个赋值为 T 的文字, 从而 E 被满足。

V.7.35 没有。如果我们知道任何一个属于 $NP - P$ 的问题, 无论它是否为 NP -完全的, 那么我们就知道 $P \neq NP$ 。而我们尚不知道这一点。

正文中提到, 在假设 $P \neq NP$ 的前提下, Ladner 定理告诉我们的这类问题是存在的, 但尚未证明任何自然出现的问题属于此类。

V.7.36 对于 $\mathcal{L}_1$ ，取所有以 0 开头的比特串构成的集合，即 $\{0\sigma \mid \sigma \in \mathbb{B}^*\}$ 。判定成员资格只需检查第一个字符是否为 0，因此 $\mathcal{L}_1 \in P$ 。

对于另外两个语言，任取一个由比特串构成的 NP -完全语言 $\mathcal{L}$ 。令 $\mathcal{L}_2 = \{0\sigma \mid \sigma \in \mathcal{L}\}$ 以及 $\mathcal{L}_0 = \mathcal{L}_1 \cup \{1\sigma \mid \sigma \in \mathcal{L}\}$ 。

V.7.37 假设 $\mathcal{L}_0$ 是 NP -完全的。再假设 $\mathcal{L}_0 \leq_p \mathcal{L}_1$ 且 $\mathcal{L}_1 \in NP$ 。为了验证 $\mathcal{L}_1$ 是 NP -完全的，我们必须证明它是 NP -难的，即对每个 $\mathcal{L} \in NP$ 都有 $\mathcal{L} \leq_p \mathcal{L}_1$ 。

由 $\mathcal{L}_0$ 是 NP -完全的假设，可得 $\mathcal{L} \leq_p \mathcal{L}_0$ 。于是 $\mathcal{L} \leq_p \mathcal{L}_0 \leq_p \mathcal{L}_1$ ，再根据引理 6.9 中多项式时间归约的传递性，即得 $\mathcal{L} \leq_p \mathcal{L}_1$ 。

V.7.38

- (A) 固定 $\mathcal{L}, \hat{\mathcal{L}} \in NP$ 。存在关联的多项式时间验证器 $\mathcal{V}$ 和 $\hat{\mathcal{V}}$ 。 $\mathcal{L} \cup \hat{\mathcal{L}}$ 的一个验证器只需对输入运行这两个验证器，任一个接受就接受。
- (B) 固定 $\mathcal{L}, \hat{\mathcal{L}} \in NP$ 。存在关联的多项式时间验证器 $\mathcal{V}$ 和 $\hat{\mathcal{V}}$ 。 $\mathcal{L}\hat{\mathcal{L}}$ 的一个验证器输入一个串 σ ，然后循环尝试所有将其拆分为子串 $\sigma = \sigma[0:i] \wedge \sigma[i:]$ 的方式，其中拆分需满足 $i \in [0..|\sigma|]$ ，并测试两个验证器是否都接受。每个验证器都是多项式时间的，该循环的执行不会使整体算法的运行时间超出多项式时间。
- (C) 类 P 对补运算封闭。因此，如果 NP 对补运算不封闭，则 $P \neq NP$ 。（然而，就目前的认知而言， NP 也有可能对补运算封闭，同时仍与 P 不同。）

V.7.39 可数的。它是 NP 的一个子集，而 NP 是可数的，因为验证器就是一台确定性 Turing 机，而确定性 Turing 机只有可数无穷多台。

V.A.10 它们是 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 和 97。

V.A.11

- (A) 我们有 $n = pq = 11 \cdot 13 = 143$ 和 $(p-1) \cdot (q-1) = 10 \cdot 12 = 120$ 。
- (B) 有多种选择。这里我们使用 $e = 7$ 。
- (C) Euclid 算法两行就找到最大公约数为 1。

$$120 = 7 \cdot 10 + 1$$

$$7 = 1 \cdot 7 + 0$$

重写使得最大公约数表示为两者的线性组合。

$$1 = 120 + 7(-17)$$

两边取模 120 得到 $7 \cdot (-17) \equiv 1 \pmod{120}$ 。因此，模 120 下，7 的乘法逆是 $120 - 17 = 103$ 。

- (D) 加密是 $m^e \pmod{n} = 9^7 \pmod{143} = 48$ 。解密是 $48^{103} \pmod{143} = 9$ 。

V.A.12

- (A) 若 n 有两个因子 $n = k_0 \cdot k_1$ ，如果其中一个大于 $\sqrt{n}$ ，则另一个必然小于 $\sqrt{n}$ 。
- (B) 如果 n 是完全平方数 $n = k \cdot k$ ，那么它的最小非平凡因子就是 $k = \sqrt{n}$ 。
- (C) 10^{12} 的平方根是 $10^6 = 1\,000\,000$ ，即一百万。
- (D) 输入 n 的规模约为 $\lg(n)$ 比特。因此 $\sqrt{n} \approx (2^n \text{ 的数字位数})^{1/2} \approx 2^{(1/2) \cdot n \text{ 的数字位数}}$ 。

V.B.1 本节的相关要点是，对于这个问题，我们有非常强大的求解器，即使在普通机器上，它们也能相当快地返回中等规模问题的答案。一个问题是 NP 难的，并不意味着它的大多数不太大的实例都求解缓慢。

V.B.2 共有 $n!$ 个回路。

V.B.3 参见 Figure 105 on page 219。基本上，你可以走外围得到 14，或者穿过中间得到 15。

V.C.1 我们可以调整初始棋盘以匹配新的棋盘。这是前几行代码。

```
; Exercise
(define INITIAL-CLAUSES
  (list (list '(1 2 3)) ; there is a 3 in position (1,2)
```

顶点序列	第一条边	第二条	第三条	返回边	总长
$v_0-v_1-v_2-v_3$	3	1	4	7	15
$v_0-v_1-v_3-v_2$	3	5	4	2	14
$v_0-v_2-v_1-v_3$	2	1	5	7	15
$v_0-v_2-v_3-v_1$	2	4	5	3	14
$v_0-v_3-v_1-v_2$	7	5	1	2	15
$v_0-v_3-v_2-v_1$	7	4	1	3	15
$v_1-v_0-v_2-v_3$	3	2	4	5	14
$v_1-v_0-v_3-v_2$	3	7	4	1	15
$v_1-v_2-v_0-v_3$	1	2	7	5	15
$v_1-v_2-v_3-v_0$	1	4	7	3	15
$v_1-v_3-v_0-v_2$	5	7	2	1	15
$v_1-v_3-v_2-v_0$	5	4	2	3	14
$v_2-v_0-v_1-v_3$	2	3	5	4	14
$v_2-v_0-v_3-v_1$	2	7	5	1	15
$v_2-v_1-v_0-v_3$	1	3	7	4	15
$v_2-v_1-v_3-v_0$	1	5	7	2	15
$v_2-v_3-v_0-v_1$	4	7	3	1	15
$v_2-v_3-v_1-v_0$	4	5	3	2	14
$v_3-v_0-v_1-v_2$	7	3	1	4	15
$v_3-v_0-v_2-v_1$	7	2	1	5	15
$v_3-v_1-v_0-v_2$	5	3	2	4	14
$v_3-v_1-v_2-v_0$	5	1	2	7	15
$v_3-v_2-v_0-v_1$	4	2	3	5	14
$v_3-v_2-v_1-v_0$	4	1	3	7	15

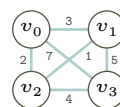


图 105, FOR QUESTION V.B.3: Traveling Salesman 问题的穷举搜索。

```
(list '(1 3 4))
(list '(1 4 5))
(list '(1 6 6))
(list '(1 7 9))
```

然后在 Dr Racket 中点击运行按钮，会输出所有子句。这个命令将它们全部输出到文件 `sudoku.cfg`。

```
> (dump-to-file)
```

有了外部文件中的数据，我们就可以运行命令来实现 SAT 求解器。

```
jim@millstone:~/Documents/computing/src/scheme/complexity$ ./run-sat-solver.sh
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:          729
| Number of clauses:           2467
| Parse time:                  0.00 s
| Eliminated clauses:          0.00 Mb
| Simplification time:         0.00 s
|
===== [ Search Statistics ] =====
| Conflicts |          ORIGINAL          |          LEARNT          | Progress |
|           | Vars  Clauses Literals |   Limit  Clauses Lit/C1 |          |
=====
|    100 |    414    1928    6686 |    706    100    12 | 30.864 % |
|    250 |    414    1928    6686 |    777    250    12 | 30.864 % |
|    475 |    414    1928    6686 |    855    475    13 | 30.864 % |
|    812 |    414    1928    6686 |    940    812    13 | 30.864 % |
|   1318 |    414    1928    6686 |   1035    717    13 | 30.865 % |
|   2077 |    413    1921    6659 |   1138   1254    14 | 31.002 % |
|   3216 |    411    1914    6635 |   1252    980    13 | 31.277 % |
|   4924 |    400    1854    6458 |   1377    775    12 | 32.785 % |
|   7486 |    378    1780    6256 |   1515   1620    11 | 35.803 % |
|  11330 |    368    1664    5938 |   1666   1784    12 | 37.174 % |
=====
restarts                : 60
conflicts                : 13476          (255867 /sec)
decisions                : 21367          (0.00 % random) (405692 /sec)
propagations            : 240090          (4558555 /sec)
conflict literals       : 166762          (18.93 % deleted)
Memory used              : 11.00 MB
CPU time                 : 0.052668 s

SATISFIABLE
```

最后，在文件 `sat-solver.rkt` 中还有更多例程，包括下面这个。

```
> (show-solved-board)
'#(#(7 3 4 5 1 6 9 2 8)
  #(8 1 5 4 9 2 7 6 3)
  #(6 9 2 3 8 7 4 1 5)
  #(9 4 6 8 2 3 1 5 7)
  #(1 2 8 7 5 4 3 9 6)
  #(5 7 3 1 6 9 2 8 4)
  #(2 8 7 6 3 1 5 4 9)
  #(3 5 1 9 4 8 6 7 2)
  #(4 6 9 2 7 5 8 3 1))
```

V.E.1 分为两部分。这是第一部分。(我们用 n 表示 $|\sigma|$ 。)

$$E_{1,0} = \bigwedge_{k \in [0..p(n)]} H_{-p(n),k} \vee \cdots \vee H_{p(n),k} = \bigwedge_{k \in [0..p(n)]} \bigvee_{i \in [-p(n)..p(n)]} H_{i,k}$$

第二部分涉及成对的谓词。

$$\begin{aligned} E_{1,1} &= \bigwedge_{k \in [0..p(n)]} (\neg S_{-p(n),k} \vee \neg S_{-p(n)+1,k}) \wedge \cdots \wedge (\neg S_{p(n)-1,k} \vee \neg S_{p(n),k}) \\ &= \bigwedge_{k \in [0..p(n)]} \bigwedge_{\text{对 } a, b \in [0..|Q|-1] \text{ 有 } a < b} \neg S_{a,k} \vee \neg S_{b,k} \end{aligned}$$

然后 $E_1 = E_{1,0} \wedge E_{1,1}$ 。

V.E.2 我们用 n 表示 $|\sigma|$ 。

(A) 这里的总体表达式也分为两部分。

这部分描述在第 $k = 0$ 步时, 从编号 $i = -5$ 到 5 的每个单元格至少包含三个字符中的一个。

$$(C_{-5,0,0} \vee C_{-5,1,0} \vee C_{-5,2,0}) \wedge (C_{-4,0,0} \vee C_{-4,1,0} \vee C_{-4,2,0}) \wedge \cdots \wedge (C_{5,0,0} \vee C_{5,1,0} \vee C_{5,2,0})$$

要得到完整的表达式, 我们必须为每一个 $k \in [0..p(n)]$ 将这样的语句连接起来。

对于第二部分, 可以通过这段描述获得感性认识: 在第 $k = 0$ 步, 编号为 $i = -p(n)$ 的单元格不包含两个符号。

$$(\neg C_{-p(n),0,0} \vee \neg C_{-p(n),1,0}) \wedge (\neg C_{-p(n),0,0} \vee \neg C_{-p(n),2,0}) \wedge (\neg C_{-p(n),1,0} \vee \neg C_{-p(n),2,0})$$

完整的表达式需要为每个单元格编号 $i \in [-p(n)..p(n)]$ 和每一步 $k \in [0..p(n)]$ 准备三个这样的子句。

(B) 我们将其写成两部分, 即 $E_2 = E_{2,0} \wedge E_{2,1}$ 。第一部分确保每个单元格至少包含一个字符。

$$E_{2,0} = \bigwedge_{\substack{i \in [-p(n)..p(n)] \\ k \in [0..p(n)]}} C_{i,0,k} \vee \cdots \vee C_{i,|\Sigma|-1,k} = \bigwedge_{\substack{i \in [-p(n)..p(n)] \\ k \in [0..p(n)]}} \bigvee_{j \in [0..|\Sigma|-1]} C_{i,j,k}$$

第二部分确保没有单元格包含多于一个字符。

$$\begin{aligned} E_{2,1} &= \bigwedge_{\substack{i \in [-p(n)..p(n)] \\ k \in [0..p(n)]}} (\neg C_{i,0,k} \vee \neg C_{i,1,k}) \wedge \cdots \wedge (\neg C_{i,|\Sigma|-2,k} \vee \neg C_{i,|\Sigma|-1,k}) \\ &= \bigwedge_{\substack{i \in [-p(n)..p(n)] \\ k \in [0..p(n)]}} \bigwedge_{\text{对 } a, b \in [0..|\Sigma|-1] \text{ 有 } a < b} \neg C_{i,a,k} \vee \neg C_{i,b,k} \end{aligned}$$

V.E.3 表达式首先包含下面这个表达式, 用以强制规定机器的初始状态是 q_0 。

$$S_{0,0} \wedge \neg S_{1,0} \wedge \cdots \wedge \neg S_{|Q|-1,0}$$

类似地, 它包含表达式

$$H_{-p(|\sigma|),0} \wedge \cdots \wedge \neg H_{-1,0} \wedge H_{0,0} \wedge \neg H_{1,0} \wedge \cdots \wedge H_{p(|\sigma|),0}$$

以强制规定在第 $k = 0$ 步时读写头指向编号为 $i = 0$ 的单元格。

对于其他子句, 设输入串为 $\sigma = \langle s_{j_0}, \dots, s_{j_t} \rangle$ 其中 $s_{j_0}, \dots, s_{j_t} \in \Sigma$ 。我们观察到, 对于以 σ 为输入的转移函数, 只有编号从 $-p(|\sigma|)$ 到 $p(|\sigma|)$ 的单元格参与计算; 所有其他单元格在计算中不起作用。(注: 对于此答案, 取满足对所有 $x \in \mathbb{N}$ 都有 $p(x) > x$ 的多项式 p 会比较方便。这样初始字符串肯定能放入从 $-p(|\sigma|)$ 到 $p(|\sigma|)$ 的区域。)

如同节正文讨论“符合”时的等式 (*), 对于具有负索引的单元格, 我们包含了确保该单元格包含且仅包含空白符 (我们取空白符为符号 0) 的子句。这是编号为 $i = -p(|\sigma|)$ 的单元格的子句。

$$C_{-p|\sigma|,0,0} \wedge \neg C_{-p|\sigma|,1,0} \wedge \cdots C_{-p(|\sigma|),|\Sigma|-1,0}$$

对于具有非负索引的单元格, 有两种情况。如果索引大于 σ 的长度, 那么我们也有一个类似的子句, 同样确保它包含且仅包含空白符。这是编号为 $p(n)$ 的单元格的子句。

$$C_{p|\sigma|,0,0} \wedge \neg C_{p|\sigma|,1,0} \wedge \cdots C_{p(|\sigma|),|\Sigma|-1,0}$$

另一组子句覆盖那些存放着 σ 中符号的单元格。对于 $i \in [0 .. |\sigma| - 1]$ 我们有这个子句。

$$\neg C_{i,0,0} \wedge \cdots \neg C_{i,j_i-1,0} \wedge C_{i,j_i,0} \wedge \neg C_{i,j_i+1,0} \cdots C_{i,|\Sigma|-1,0}$$

附录

A.1

- (A) $\sigma \wedge \tau = 10110 \wedge 110111 = 10110110111$ 。
- (B) $\sigma \wedge \tau \wedge \sigma = 10110 \wedge 110111 \wedge 10110 = 1011011011110110$
- (C) $\sigma^R = 10110^R = 01101$
- (D) $\sigma^3 = 10110 \wedge 10110 \wedge 10110 = 101101011010110$
- (E) $0^3 \wedge \sigma = 000 \wedge 10110 = 00010110$

A.2

- (A) abbca
- (B) ababbcabca
- (C) baacb
- (D) ababab

A.3 长度为 4 的比特串有 $2^4 = 16$ 个, 其中一半以 0 开头, 因此该语言中有 8 个串。

A.4

- (A) 是。
- (B) 是。
- (C) 否。
- (D) 是。

A.5 拼接串的长度等于两串长度之和: $|\sigma \wedge \tau| = |\langle s_0, \dots, s_{i-1}, t_0, \dots, t_{j-1} \rangle| = i + j = |\sigma| + |\tau|$ 。

A.6

- (A) 真。第一式 $\alpha \wedge \varepsilon$ 的拼接结果就是 $\alpha \wedge \varepsilon = \langle a_0, \dots, a_{i-1} \rangle \wedge \langle \rangle = \langle a_0, \dots, a_{i-1} \rangle = \alpha$ 。第二式的证明方法类似。
- (B) 假。反例为两个比特串 $\alpha = 0$ 和 $\beta = 1$ 。
- (C) 假。与前一项情况类似, 反例仍为两个比特串 $\alpha = 0$ 和 $\beta = 1$ 。
- (D) 真。当 $\alpha = \langle a_0, \dots, a_{i-1} \rangle$ 时, 我们有 $\alpha^R = \langle a_{i-1}, \dots, a_0 \rangle$, 因此 $\alpha^{RR} = \langle a_0, \dots, a_{i-1} \rangle$ 。
- (E) 假。反例为比特串 $\alpha = 01$, 取幂次 $i = 1$ 。

B.1

- (A) 证明单射: 假设对 $x_0, x_1 \in \mathbb{R}$ 有 $f(x_0) = f(x_1)$ 。则有 $3x_0 + 1 = 3x_1 + 1$ 。两边减去 1 再除以 3, 可得 $x_0 = x_1$ 。
证明满射: 任取陪域中的一个元素 $c \in \mathbb{R}$ 。注意到 $d = (c - 1)/3$ 是定义域中的一个元素, 且 $f(d) = 3 \cdot ((c - 1)/3) + 1 = (c - 1) + 1 = c$ 。因此 f 是满射。
- (B) 函数 g 不是单射, 因为 $g(2) = g(-2)$ 。它不是满射, 因为定义域 $\mathbb{R}$ 中没有元素被 g 映射到陪域中的元素 0。

B.2

- (A) g 是 f 的一个左逆, 因为对所有 x 和 $y, g \circ f$ 的作用为 $(x, y) \mapsto (x, y, 0) \mapsto (x, y)$ 。它不是 f 的右逆, 因为 $f \circ g$ 的作用为 $(x, y, z) \mapsto (x, y) \mapsto (x, y, 0)$, 这不是恒等函数; 例如, $(1, 2, 3) \mapsto (1, 2) \mapsto (1, 2, 0)$ 。
- (B) 注意到 $f(2) = f(-2) = 4$ 。任何左逆都必须将 4 映到 2, 同时也必须将 4 映到 -2。这就不是良定义的了, 因此不存在这样的左逆函数。
- (C) 一个右逆是 $g_0: C \rightarrow D$, 定义为 $10 \mapsto 0, 11 \mapsto 1$ 。它是右逆, 因为 $f \circ g_0(10) = f(g_0(10)) = f(0) = 10$ 且 $f \circ g_0(11) = f(g_0(11)) = f(1) = 11$ 。第二个右逆是 $g_1: C \rightarrow D$, 定义为 $10 \mapsto 2, 11 \mapsto 3$ 。

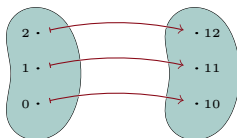
B.3

- (A) 我们需要证明它是双侧逆。首先, 每个函数的定义域都等于另一个函数的陪域, 因此复合 $f \circ g$ 和

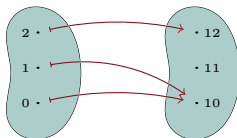
$g \circ f$ 都有定义。接下来, g 是 f 的左逆, 因为 $g \circ f(a) = g(f(a)) = g(a+3) = (a+3)-3 = a$ 是恒等映射。类似地, 它也是右逆, 因为 $f \circ g(a) = (a-3)+3$ 也是恒等映射。

- (B) h 的逆函数是它自身。其定义域和陪域相等, 符合要求。若 n 为奇数, $h \circ h(n) = h(n-1) = (n-1)+1 = n$ (第二个等式成立是因为此时 $n-1$ 是偶数)。类似地, 若 n 为偶数, 则 $h \circ h(n) = (n+1)-1 = n$ 。
- (C) s 的逆函数是映射 $r: \mathbb{R}^+ \rightarrow \mathbb{R}^+$, 定义为 $r(x) = \sqrt{x}$ (即取正根)。每个函数的定义域都是另一个函数的陪域, 且 $s \circ r$ 和 $r \circ s$ 都是恒等函数。

B.4 这是函数 f 的豆子图,



而这是 g 的图。

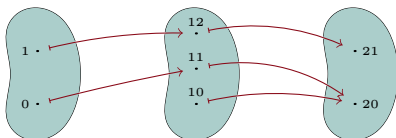


- (A) 通过观察, f 既是单射的又是满射的。
- (B) 反转 f 建立的对对应关系, 使得 $f^{-1}(10) = 0$, $f^{-1}(11) = 1$, 以及 $f^{-1}(12) = 2$ 。
- (C) 映射 g 不是单射的, 因为 $g(0) = g(1)$ 。(另一个正确答案是 g 不是满射的, 因为没有输入被映到 $11 \in C$ 。)
- (D) 如果 h 是 g 的逆函数, 那么我们将同时有 $h(10) = 0$ 和 $h(10) = 1$, 这将违反函数定义中的“良定义”条件。

B.5

- (A) 设 $f: D \rightarrow C$ 和 $g: C \rightarrow B$ 是单射的。为了验证复合函数是单射的, 假设对于某些元素 $x_0, x_1 \in D$ 有 $g \circ f(x_0) = g \circ f(x_1)$ 。那么 $g(f(x_0)) = g(f(x_1))$ 。函数 g 是单射的, 因此, 既然两个值 $g(f(x_0))$ 和 $g(f(x_1))$ 相等, 它们的自变量 $f(x_0)$ 和 $f(x_1)$ 也相等。函数 f 是单射的, 因此由 $f(x_0)$ 和 $f(x_1)$ 相等可得 $x_0 = x_1$ 。因为由 $g \circ f(x_0) = g \circ f(x_1)$ 可推出 $x_0 = x_1$, 所以复合函数是单射的。
- (B) 设 $f: D \rightarrow C$ 和 $g: C \rightarrow B$ 是满射的。如果 B 是空集, 那么根据函数的定义, C 也是空集, 再次根据函数的定义, $D = \emptyset$, 因此复合函数平凡地是满射的。否则, 固定复合函数陪域中的一个元素 $b \in B$ 。函数 g 是满射的, 所以存在 $c \in C$ 使得 $g(c) = b$ 。函数 f 是满射的, 所以存在 $d \in D$ 使得 $f(d) = c$ 。那么 $g \circ f(d) = b$, 因此复合函数是满射的。
- (C) 设 $f: D \rightarrow C$ 和 $g: C \rightarrow B$ 满足复合函数 $g \circ f$ 是单射的。假设 f 不是单射的。那么存在 $d_0, d_1 \in D$, 满足 $d_0 \neq d_1$, 使得 $f(d_0) = f(d_1)$ 。但这意味着 $g(f(d_0)) = g(f(d_1))$, 与复合函数是单射的矛盾。
- (D) 设 $f: D \rightarrow C$ 和 $g: C \rightarrow B$ 满足复合函数 $g \circ f$ 是满射的。反设 g 不是满射的。那么对于某个 $b \in B$, 不存在 $c \in C$ 使得 $g(c) = b$ 。但这意味着不存在 $d \in D$ 使得 $b = g(f(d))$, 与复合函数是满射的矛盾。所以 g 必须是满射的。
- (E) 两个问题的答案都是“不。”

设 $D = \{0, 1\}$, $C = \{10, 11, 12\}$, $B = \{20, 21\}$ 。设 $f: D \rightarrow C$ 和 $g: C \rightarrow B$ 如下图所示。则 $g \circ f$ 是满射的, 因为 $g \circ f(0) = 20$ 和 $g \circ f(1) = 21$ 。但 f 不是满射的, 因为其定义域 D 中没有元素被映到其陪域 C 中的元素 10。



同样的设置也提供了一个例子：复合函数 $g \circ f$ 是单射的，但 g 不是单射的。具体来说， $g \circ f$ 是单射的，因为其定义域 D 中只有一个元素被映到其陪域中的元素 20。类似地，其定义域中也只有一个元素被映到 21。

B.6

- (A) 首先证明：若函数 $f: D \rightarrow C$ 有逆 $f^{-1}: C \rightarrow D$ ，则 f 必须是一个双射。为验证 f 是单射的，假设 $d_0, d_1 \in D$ 满足 $f(d_0) = f(d_1)$ 。令 $c \in C$ 为 $c = f(d_0) = f(d_1)$ ，并考虑 $f^{-1}(c)$ 。因为 $f^{-1} \circ f$ 是 D 上的恒等映射，我们既有 $f^{-1}(c) = f^{-1}(f(d_0)) = d_0$ 也有 $f^{-1}(c) = f^{-1}(f(d_1)) = d_1$ 。故 $d_0 = d_1$ ，因此 f 是单射的。

接下来验证 f 是满射的。假设相反，即存在 $c \in C$ 不与任何 $d \in D$ 关联。但这会导致矛盾，因为映射 $f \circ f^{-1}$ 无法满足 $c = f \circ f^{-1}(c) = f(f^{-1}(c))$ ，原因在于 $c \notin \text{ran}(f)$ 。因此 f 必须是满射的。

最后，我们证明：若映射 $f: D \rightarrow C$ 是一个双射，则它有逆。我们将考虑一个逆关联 g ，它把 C 的元素与 D 的元素联系起来，并证明这一关联是一个函数。

对每个 $c \in C$ ，令 g 将 c 与满足 $f(d) = c$ 的元素 $d \in D$ 相关联。由于 f 是满射的，这样的元素 d 存在；又因 f 是单射的，该元素唯一。因此，关联 g 是良定义的，也就是说，它是一个函数。显然，复合 $g \circ f$ 与 $f \circ g$ 都是恒等映射。

- (B) 假设 $f: D \rightarrow C$ 有两个逆 $g_0, g_1: C \rightarrow D$ 。我们将证明这两个函数在定义域与陪域上关联相同的元素，即它们有相同的图像，因而是同一个函数。

令 c 是 g_0 和 g_1 的定义域 C 中的任意元素。本练习的前一项已表明 f 是满射的，故存在 $d \in D$ 使得 $f(d) = c$ 。前一项还表明 f 是单射的，因此满足此性质的 d 唯一：于是 $d = g_0(c)$ 且 $d = g_1(c)$ 。所以这两个函数都将输入 c 映射到相同的输出 $g_0(c) = g_1(c)$ 。

- (C) 注意 f^{-1} 是一个有逆的函数，其逆正是 f 。这仅仅是因为在方程 $g \circ f^{-1} = \text{id}$ 与 $f^{-1} \circ g = \text{id}$ 中，我们可以取 g 为 f 。于是根据本练习第一项， f^{-1} 是一个双射。
- (D) 以下验证按某一顺序复合会得到恒等函数：

$$(f^{-1} \circ g^{-1}) \circ (g \circ f)(x) = (f^{-1} \circ g^{-1})(g(f(x))) = f^{-1}(g^{-1}(g(f(x)))) = f^{-1}(f(x)) = x$$

另一顺序的验证类似。

B.7

- (A) 我们将对值域中元素的个数进行归纳。在证明之前，我们先给出一个示意图。取定 $D = \{0, 1, 2, 3\}$ 和 $C = \{10, 11, 12\}$ 。考虑这里给出的函数 $f: D \rightarrow C$ ，其值域为 C 。

$$0 \mapsto 10 \quad 1 \mapsto 10 \quad 2 \mapsto 11 \quad 3 \mapsto 12$$

为了进行归纳，我们选取值域中的一个元素并将其去掉；假设我们去掉了 10。然后我们考虑满射到 $C - \{10\}$ 的限制映射。为使该映射是良定义的，我们必须从其定义域中去掉所有映到 10 的元素，即集合 $f^{-1}(10) = \{0, 1\}$ 。于是我们得到函数 $\hat{f}$ ，其定义域为 $\hat{D} = \{2, 3\}$ ，陪域为 $\hat{C} = \{11, 12\}$ ，如下所示。

$$2 \mapsto 11 \quad 3 \mapsto 12$$

这个限制映射 $\hat{f}$ 的值域是 $\hat{C}$ 。因此，由归纳假设可知， $\hat{f}$ 的定义域 $\hat{D}$ 的元素个数至少与其值域 $\hat{C}$ 的元素个数一样多。为了完成论证，我们将把去掉的元素加回去以还原 f ：向 f 的定义域 D 中加回两个元素，向其值域 C 中加回一个元素，从而得出结论： f 的定义域包含的元素至少与其值域一样多。

有了这个例子作为铺垫，我们可以进行归纳证明了。基例是考虑值域为空的函数 f 。根据函数的定义，定义域的每个元素都必须映到某个地方，因此定义域也必须为空。于是 $|\text{ran}(f)| \leq |D|$ ，因为两者都为零。

对于归纳步骤，假设该命题对所有值域包含 0 个、1 个、……或 k 个元素的有限集之间的函数成立。考虑 $f: D \rightarrow C$ ，其中 $C = \text{ran}(f)$ 且 $|C| = k + 1$ 。

取定某个 $c_0 \in C$ （因为 $k + 1 > 0$ ，值域非空）。令 $\hat{C} = C - \{c_0\}$ 。因为 c_0 在 f 的值域中，集合 $f^{-1}(c_0) = \{d \in D \mid f(d) = c_0\}$ 至少有一个元素。换言之，令 $\hat{D} = D - f^{-1}(c_0)$ ，则有 $|\hat{D}| < |D|$

(严格小于)。现在的关键点是：映射 $\hat{f}: \hat{D} \rightarrow \hat{C}$ 是满射的，因为任意 $c \in \text{ran}(f)$ 都是某个 $d \in D$ 的像，从而任意满足 $c \neq c_0$ 的元素必定是某个 $d \in \hat{D}$ 的像。因此归纳假设适用，可得 $|\hat{C}| \leq |\hat{D}|$ 。最后两端加 1，即得 $|\text{ran}(f)| = |\hat{C}| + 1 \leq |\hat{D}| + 1 \leq |D|$ 。

另证。我们可以改为对定义域 D 中元素的个数进行归纳。如前所述，我们先用例子说明论证思路。取定 $D = \{0, 1, 2, 3\}$ 和陪域 $C = \{10, 11, 12, 13\}$ 。为了进行归纳，我们将去掉定义域中的一个元素，在这个例子中去掉 0，得到 $\hat{D} = D - \{0\}$ 。分两种情况。第一种情况：被去掉的元素 0 与至少一个其他未被去掉的定义域元素映到陪域中的同一个成员。下图所示的函数 f 即属此情况。

$$0 \mapsto 10 \quad 1 \mapsto 10 \quad 2 \mapsto 11 \quad 3 \mapsto 12$$

考虑限制映射 $\hat{f}: \hat{D} \rightarrow C$ ，由 $\hat{f}(d) = f(d)$ 给出。归纳假设适用，可得 $|\text{ran}(\hat{f})| \leq |\hat{D}|$ 。在第一种情况下，0 不是 D 中唯一映到 10 的元素，因此虽然 $|\hat{D}| < |D|$ ，但有 $|\text{ran}(f)| = |\text{ran}(\hat{f})|$ ，从而立得所需结论 $|\text{ran}(f)| = |\text{ran}(\hat{f})| \leq |\hat{D}| < |D|$ 。

另一种情况是：被去掉的定义域元素 0 不与任何其他定义域元素映到陪域中的同一成员。下图所示的函数 f 即属此情况。

$$0 \mapsto 10 \quad 1 \mapsto 11 \quad 2 \mapsto 12 \quad 3 \mapsto 13$$

为了进行归纳，我们从定义域中去掉 0，得到 $\hat{D} = D - \{0\}$ 。考虑限制映射 $\hat{f}: \hat{D} \rightarrow C$ ，定义为 $\hat{f}(d) = f(d)$ 。归纳假设适用，所以 $|\text{ran}(\hat{f})| \leq |\hat{D}|$ 。然后两端加 1 可得 $|\text{ran}(f)| = |\text{ran}(\hat{f})| + 1 \leq |\hat{D}| + 1 = |D|$ 。

现在我们开始正式证明。基例是考虑定义域 D 为空的函数 f 。唯一具有空定义域的函数是空函数，其值域为空，因此 $0 = |\text{ran}(f)| \leq |D| = 0$ 。

对于归纳步骤，假设该命题对所有定义域包含 0 个、1 个、……或 k 个元素的有限集之间的函数成立。考虑 $f: D \rightarrow C$ ，其中 $|D| = k + 1$ 。

取定某个 $d_0 \in D$ (因为 $k + 1 > 0$ ，定义域非空，所以这样的元素存在)。令 $\hat{D} = D - \{d_0\}$ ，并令 $\hat{f}: \hat{D} \rightarrow C$ 为 f 到 $\hat{D}$ 的限制，定义为 $\hat{f}(d) = f(d)$ 。由归纳假设可得 $|\text{ran}(\hat{f})| \leq |\hat{D}|$ 。

考虑陪域元素 $c_0 = f(d_0)$ 。有两种情况：要么 (1) c_0 也是另一个定义域元素的像，即存在 $d \in \hat{D}$ 满足 $d \neq d_0$ 且 $f(d) = c_0$ ；要么 (2) c_0 不是任何其他定义域元素的像。在第一种情况下， $\text{ran}(f) = \text{ran}(\hat{f})$ ，且我们有 $|\text{ran}(f)| = |\text{ran}(\hat{f})| \leq |\hat{D}| < |D|$ 。在第二种情况下， $\text{ran}(f) = \text{ran}(\hat{f}) + 1$ ，且我们有 $|\text{ran}(f)| = |\text{ran}(\hat{f})| + 1 \leq |\hat{D}| + 1 = |D|$ 。

(B) 我们将对值域中元素的个数进行归纳。基例是值域为空。唯一具有空值域的函数是空函数，其定义域为空，因此 $0 = |\text{ran}(f)| = |D| = 0$ 。

对于归纳步骤，假设该命题对任何值域包含 0 个、1 个、……或 k 个元素的有限集之间的单射函数成立。考虑一个单射的 $f: D \rightarrow C$ ，满足 $|\text{ran}(f)| = k + 1$ 。

取定 $c_0 \in \text{ran}(f)$ (因为 $k + 1 > 0$ ，这样的元素存在)。令 $\hat{C} = C - \{c_0\}$ 。因为 c_0 是 f 值域的一个元素，所以存在一个定义域元素映到它。由于 f 是单射的，这样的元素唯一。记之为 d_0 ，并令 $\hat{D} = D - \{d_0\}$ 。

限制函数 $\hat{f}: \hat{D} \rightarrow \hat{C}$ 定义为 $\hat{f}(d) = f(d)$ 。该映射也是单射的：若将 f 视为有序对的集合，则其单射性意味着没有两个有序对具有相同的第二分量；而限制 $\hat{f}$ 是 f 的子集，因此它也没有两个有序对具有相同的第二分量。根据归纳假设， $|\text{ran}(\hat{f})| = |\hat{D}|$ 。两端加 1 可得 $|\text{ran}(f)| = |\text{ran}(\hat{f})| + 1 = |\hat{D}| + 1 = |D|$ 。

另证。我们可以改为对定义域 D 中元素的个数进行归纳。基例是定义域为空，即 $|D| = 0$ 。唯一具有空定义域的函数是空函数，我们有 $0 = |\text{ran}(f)| = |D| = 0$ 。

对于归纳步骤，假设该命题对任何定义域包含 0 个、……或 k 个元素的有限集之间的单射函数成立。考虑一个单射的函数 $f: D \rightarrow C$ ，其定义域满足 $|D| = k + 1$ 。

取定某个 $d_0 \in D$ (因为 $k + 1 > 0$ ，集合 D 非空)，并令 $\hat{D} = D - \{d_0\}$ 。考虑 $c_0 = f(d_0)$ 。因为 f 是单射的， c_0 不是除 d_0 外任何定义域元素的像。所以限制映射 $\hat{f}: \hat{D} \rightarrow C$ 的值域少了一个元素， $\text{ran}(f) = \text{ran}(\hat{f}) + 1$ 。

关键点在于：因为 f 是单射的， $\hat{f}$ 也是单射的：若将 f 视为有序对的集合，则其单射性意味着

没有两个有序对具有相同的第二分量；而 $\hat{f}$ 是 f 的子集，因此也具有相同的性质。所以归纳假设适用，可得 $|\text{ran}(f)| = |\hat{D}|$ 。两端加 1 即得所需结论，即 $|\text{ran}(f)| = |\text{ran}(\hat{f})| + 1 = |\hat{D}| + 1 = |D|$ 。

C.1 显然，“清晰”一词见仁见智，这里给出一种尝试：两件事必须同时为真。第一件是，要么 7 是奇数，要么 9 是完全平方数。第二件是，以下情形不能成立：7 是奇数，或者 11 是素数且 9 是完全平方数。无论如何，该想法用语言表述比用符号表述难理解得多。

C.2

(A) $\neg(P \vee Q)$ 的真值表如下。

P	Q	$P \vee Q$	$\neg(P \vee Q)$
F	F	F	T
F	T	T	F
T	F	T	F
T	T	T	F

(B) $\neg P \wedge \neg Q$ 的真值表如下。

P	Q	$\neg P$	$\neg Q$	$\neg P \wedge \neg Q$
F	F	T	T	T
F	T	T	F	F
T	F	F	T	F
T	T	F	F	F

它的最后一列与上一小题相同。

(C) $\neg(P \wedge Q)$ 的真值表如下。

P	Q	$P \wedge Q$	$\neg(P \wedge Q)$
F	F	F	T
F	T	F	T
T	F	F	T
T	T	T	F

(D) $\neg P \vee \neg Q$ 的真值表如下。

P	Q	$\neg P$	$\neg Q$	$\neg P \vee \neg Q$
F	F	T	T	T
F	T	T	F	T
T	F	F	T	T
T	T	F	F	F

最后一列与上一小题的最后一列相同。

C.3

(A) 这是 $(P \wedge Q) \wedge R$ 的真值表。

P	Q	R	$P \wedge Q$	$(P \wedge Q) \wedge R$
F	F	F	F	F
F	F	T	F	F
F	T	F	F	F
F	T	T	F	F
T	F	F	F	F
T	F	T	F	F
T	T	F	T	F
T	T	T	T	T

(B) 这是 $P \wedge (Q \wedge R)$ 的真值表。它的最后一列与上一项相同。

P	Q	R	$Q \wedge R$	$P \wedge (Q \wedge R)$
F	F	F	F	F
F	F	T	F	F
F	T	F	F	F
F	T	T	T	F
T	F	F	F	F
T	F	T	F	F
T	T	F	F	F
T	T	T	T	T

(c) 这是 $P \wedge (Q \vee R)$ 的真值表。

P	Q	R	$Q \vee R$	$P \wedge (Q \vee R)$
F	F	F	F	F
F	F	T	T	F
F	T	F	T	F
F	T	T	T	F
T	F	F	F	F
T	F	T	T	T
T	T	F	T	T
T	T	T	T	T

(d) 这是 $(P \wedge Q) \vee (P \wedge R)$ 的真值表。它的最后一列与上一项相同。

P	Q	R	$P \wedge Q$	$P \wedge R$	$(P \wedge Q) \vee (P \wedge R)$
F	F	F	F	F	F
F	F	T	F	F	F
F	T	F	F	F	F
F	T	T	F	F	F
T	F	F	F	F	F
T	F	T	F	T	T
T	T	F	T	F	T
T	T	T	T	T	T

C.4 十六个真值表如下。

P	Q	O_0	P	Q	O_1	P	Q	O_2	P	Q	O_3
F	F	F	F	F	F	F	F	F	F	F	F
F	T	F	F	T	F	F	T	F	F	T	F
T	F	F	T	F	F	T	F	T	T	F	T
T	T	F	T	T	T	T	T	F	T	T	T
P	Q	O_4	P	Q	O_5	P	Q	O_6	P	Q	O_7
F	F	F	F	F	F	F	F	F	F	F	F
F	T	T	F	T	T	F	T	T	F	T	T
T	F	F	T	F	F	T	F	T	T	F	T
T	T	F	T	T	T	T	T	F	T	T	T
P	Q	O_8	P	Q	O_9	P	Q	O_{10}	P	Q	O_{11}
F	F	T	F	F	T	F	F	T	F	F	T
F	T	F	F	T	F	F	T	F	F	T	F
T	F	F	T	F	F	T	F	T	T	F	T
T	T	F	T	T	T	T	T	F	T	T	T
P	Q	O_{12}	P	Q	O_{13}	P	Q	O_{14}	P	Q	O_{15}
F	F	T	F	F	T	F	F	T	F	F	T
F	T	T	F	T	T	F	T	T	F	T	T
T	F	F	T	F	F	T	F	T	T	F	T
T	T	F	T	T	T	T	T	F	T	T	T

这些运算符如下。

运算符	表达式	运算符	表达式
O_0	F (称为 falsum, 有时写作 $\perp$)	O_8	$\neg(P \vee Q)$
O_1	$P \wedge Q$	O_9	$P \leftrightarrow Q$
O_2	$\neg(P \rightarrow Q)$	O_{10}	$\neg Q$
O_3	P	O_{11}	$Q \rightarrow P$
O_4	$\neg(Q \rightarrow P)$	O_{12}	$\neg P$
O_5	Q	O_{13}	$P \rightarrow Q$
O_6	$\neg(P \leftrightarrow Q)$	O_{14}	$\neg(P \wedge Q)$
O_7	$P \vee Q$	O_{15}	T (有时写作 $\top$)

C.5 在析取范式中, 子句之间以析取连接。

- (A) 第二、第三和第六行结果为 T 。对应的子句是 $\neg P \wedge \neg Q \wedge R$ 、 $\neg P \wedge Q \wedge \neg R$ 和 $P \wedge \neg Q \wedge R$ 。得到的 CNF 表达式如下。

$$(\neg P \wedge \neg Q \wedge R) \vee (\neg P \wedge Q \wedge \neg R) \vee (P \wedge \neg Q \wedge R)$$

- (B) 结果为 T 的各行给出四个子句。

$$(\neg P \wedge \neg Q \wedge \neg R) \vee (\neg P \wedge Q \wedge \neg R) \vee (P \wedge Q \wedge \neg R) \vee (P \wedge Q \wedge R)$$

C.6 在合取范式中, 子句之间以合取连接。

- (A) 结果为 F 的各行是第一、第四、第五、第七和第八行。

$$E_0 \equiv (P \vee Q \vee R) \wedge (P \vee \neg Q \vee \neg R) \wedge (\neg P \vee Q \vee R) \wedge (\neg P \vee \neg Q \vee R) \wedge (\neg P \vee \neg Q \vee \neg R)$$

- (B) 找出结果为 F 的各行。

$$E_1 \equiv (P \vee Q \vee \neg R) \wedge (P \vee \neg Q \vee \neg R) \wedge (\neg P \vee Q \vee R) \wedge (\neg P \vee Q \vee \neg R)$$